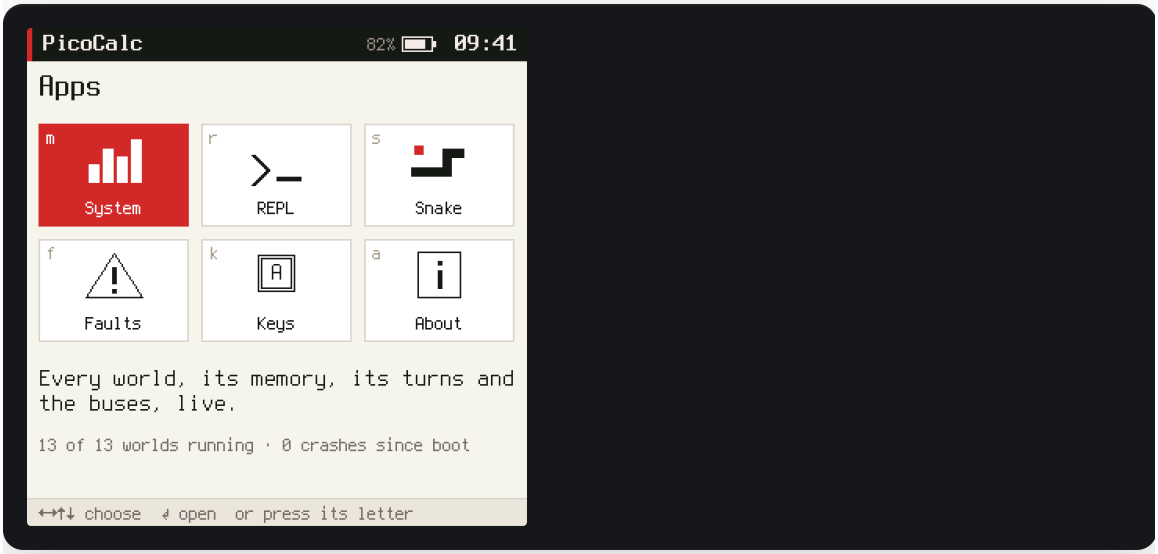
A TEXTBOOK

Small Worlds

Operating Systems, Isolation and Self-Healing Drivers, Built on a JavaScript Handheld

With PicoCalc OS as the worked example: a JavaScript operating system for an ESP32-P4 handheld, its C nucleus, drivers and apps, simulated, measured and broken on purpose.



The home screen, read from the simulated ST7365P panel's memory.

Preface

This is a textbook about operating systems: what they are for, how their mechanisms work, and how to build and measure one. It covers the classical subjects of a first course (concurrency, scheduling, memory, protection, communication and device drivers) and then goes further into three areas that a first course usually treats briefly or not at all: embedded and real-time systems, isolation enforced by a programming language rather than by hardware, and recovery from failure, in particular drivers that crash and come back. It ends with the browser, the most widely used operating-system platform that is not called one, and with the practice of building, simulating and measuring systems software.

The book is written for a reader who programs in C and JavaScript and knows the basics of computer architecture: a senior undergraduate, a graduate student, or an engineer who builds systems and wants the theory behind them. No previous course in operating systems is assumed.

One system, all the way through

Every chapter uses the same system as its worked example. PicoCalc OS is a small operating system for the PicoCalc, a pocket computer with a 320 × 320 display and a full keyboard, rebuilt around an ESP32-P4 microcontroller. Its core, the nucleus, is about 2,300 lines of C. Everything above it is JavaScript: the keyboard, power and display drivers, the compositor that owns the screen, and the apps. Each of those components is a *world*, a separate JavaScript runtime with its own heap and its own time budget, and the worlds share nothing but the messages they send each other and the hardware handles the nucleus lets them hold. A supervisor restarts any world that fails, drivers included.

The system is small enough to read in a weekend and complete enough to show every idea in the book in working code. When a chapter explains priority inversion, the non-preemptive lane of PicoCalc OS is there to analyse. When a chapter explains capabilities, its handles are there to break. When a chapter explains self-healing drivers, its display driver is killed and the trace shows, microsecond by microsecond, how the screen comes back.

PicoCalc OS runs today on a simulated board. The host build compiles the same nucleus and the same JavaScript for Linux, with register-level models of the display and keyboard controllers in place of the real chips, and runs them on a virtual clock calibrated against measurements taken on the ESP32-P4. It is deterministic, so every experiment in this book can be repeated exactly. The port to the device itself is planned and described in Part II and Chapter 20, and the book is careful to say which numbers come from the model and what the model leaves out. A measurement on the device will check them.

Three threads

Three themes run through every part of the book.

Isolation by language. PicoCalc OS has no memory-management unit protecting one component from another. Its isolation comes from the fact that a JavaScript program cannot forge a reference into another runtime's heap. The book follows that idea from the language-based systems of the 1970s and 1980s to WebAssembly and the browser, and is precise about where it holds and where it does not.

Time. A handheld must respond to a key press quickly and predictably. The book develops scheduling and real-time theory and applies it to a system whose clock is a model: every interpreter step, every byte on a bus and every pixel has a cost, and the costs add up to the latency a person feels.

Recovery. Components fail. The book treats detection, containment and restart as an operating system's fourth job, alongside multiplexing, isolation and mediation, and spends two chapters on the hardest case: restarting the code that drives a device while the rest of the system keeps using it.

How to read it

The book has six parts. Part I (Chapters 1 to 4) lays the foundations: what an operating system does, the hardware underneath, concurrency and scheduling. Part II (5 to 7) covers real-time theory, memory in small systems and embedded engineering on the ESP32-P4. Part III (8 to 11) is about protection: kernel structure, capabilities, language as the isolation boundary, and resource control. Part IV (12 to 15) covers communication, device drivers, failure and self-healing drivers. Part V (16 and 17) treats the browser as an operating system and builds a small one inside this book's own page. Part VI (18 to 20) is about engineering: deterministic simulation, measurement, and the practice of keeping a system's claims true. The appendices document the world API, the host build and the vocabulary.

A course on operating-system fundamentals can follow Parts I, III and IV. Embedded engineers may prefer Parts I, II, IV and VI. Readers coming from the web can start with Chapter 1, then Parts III and V.

Most chapters contain *Try* sections: simulators, playgrounds and live systems in the page, including, where the viewer allows WebAssembly, PicoCalc OS itself running in the browser. Each is described in the text, so the printed edition, which shows each one in its initial state, can be read without them. Code excerpts labelled with a file name are taken from the PicoCalc OS sources and checked against them when the book is built. Numbers given in milliseconds are virtual milliseconds of the host build unless the text says otherwise. Chapters end with key points, exercises and further reading.

Contents

PART I. FOUNDATIONS

1 **What an operating system does** 9

Multiplexing, isolation, mediation and recovery, and one key press followed from the keyboard controller to the glass of PicoCalc OS.

2 **The machine underneath** 32

What a RISC-V microcontroller, its memories, two serial buses and two peripheral controllers give an operating system, and how to compute what moving a byte costs.

3 **Concurrency: threads, events and actors**

How systems structure work that overlaps in time, what goes wrong when activities share memory, and why PicoCalc OS runs isolated event loops whose effects land at the end of each segment.

4 **Scheduling** 83

What a scheduler decides, how the classic policies trade response time, throughput and fairness, and what PicoCalc OS's single non-preemptive lane costs, measured.

PART II. EMBEDDED AND REAL-TIME

5 **Real-time systems** 103

The periodic task model and its analyses (rate monotonic, exact response times, EDF, non-preemptive blocking), priority inversion and the protocols that bound it, and an honest analysis of what PicoCalc OS's lane can guarantee.

6 **Memory in small systems** 125

Why a microcontroller's memory fails hard, how allocators fragment and garbage collectors pause, and how PicoCalc OS gives each world a heap with a limit it cannot escape.

7 **Embedded systems engineering** 146

How firmware for a chip like the ESP32-P4 is structured, booted, watched and debugged, and what moving PicoCalc OS from its simulated board onto the real one will involve.

PART III. ISOLATION AND PROTECTION

8 **Protection and kernel structure** 166

The principles of protection, the reference monitor and the trusted computing base, how kernel structures place their boundaries, and where PicoCalc OS's language-enforced microkernel stands among them.

9 **Capabilities** 188

The access matrix, ambient authority and the confused deputy, capability systems from Dennis and Van Horn to Zircon, and PicoCalc OS's handles verified, broken and repaired.

10 **Language as the isolation boundary** 219

How memory-safe languages replace hardware protection, from the B5000 to Rust kernels, how JavaScript can be isolated, where ambient authority leaks from a language's built-ins, and why no language boundary keeps secrets on speculative hardware.

11 **Resource control** 240

How PicoCalc OS bounds what a misbehaving world can take, from CPU budgets counted in interpreter steps and native work to heap limits, and which resources are still left without a bound.

PART IV. COMMUNICATION, DRIVERS AND RECOVERY

12 **Messages and IPC** 261

How worlds communicate without sharing memory: send, ask and publish, when messages are delivered and in what order, how authority travels with them, what they cost, and how to design protocols that survive restarts.

13 **Device drivers** 287

What a driver is responsible for, how operating systems structure drivers, and how PicoCalc OS's keyboard, power and display drivers and its display pipeline are written from a datasheet and tested against register-level models.

14 **Failure and recovery** 310

Faults, errors and failures; why restarting works; crash-only design; supervision with backoff and restart limits; and how PicoCalc OS turns a crash into a clean restart.

15 **Self-healing device drivers** 334

Why drivers fail, how systems from Nooks to MINIX 3 recover them, seven rules for writing restartable drivers, and PicoCalc OS's keyboard, display and compositor recoveries traced event by event.

PART V. THE BROWSER AS AN OPERATING-SYSTEM PLATFORM

16 **The browser as an operating system** 369

Processes, principals, an event-loop scheduler, message passing, device access and a portable executable format, found in the web browser and compared with PicoCalc OS.

17 **An operating system inside the page** 388

Building a small operating system whose worlds are Web Workers, with budgets, supervision and a driver, then running PicoCalc OS itself in the same page as WebAssembly.

PART VI. ENGINEERING THE SYSTEM

18 **Deterministic simulation** 411

How PicoCalc OS runs, unchanged, inside a deterministic simulator with virtual time, calibrated costs and register-level device models, and what such a simulator can and cannot tell you.

19 **Measuring systems** 431

How to measure a system so that the number means something, taught through the key-to-pixels study of PicoCalc OS and the mistakes made on the way to it.

20 **Systems engineering practice** 456

The habits that make systems software buildable and its claims true: narrow interfaces, a small counted core, observability and recovery designed in, checking every claim against the code, and a plan for meeting the hardware.

APPENDICES

A **The world API** 477

Every function, method and option a PicoCalc OS world can use, what each does immediately and what it does at the end of the segment, the errors it raises, the manifest's fields and the user-interface vocabulary.

B **The host build** 493

How to build PicoCalc OS for Linux and for the browser, drive it with scripts, trace it, read its fault frames, and reproduce every measured number in this book.

C **Glossary** 505

Short definitions of the terms of PicoCalc OS and of the general operating-systems terms the book uses, each with the chapter that develops it.

D **Known issues** 514

What PicoCalc OS does not yet do correctly, what the first and second passes fixed, where its timing model is approximate, and what remains planned.

· **References** 521

CHAPTER 1 PART I · FOUNDATIONS

What an operating system does

An operating system is the software that lets several programs share one machine without having to trust one another. That sentence carries the whole subject of this book, and this chapter unpacks it. By the end of it you should be able to name the four jobs an operating system does (multiplexing, isolation, mediation and recovery), say which part of a real system does each one, and follow a single key press from the moment a switch closes under a fingertip to the moment new pixels reach a panel. The system we follow is PicoCalc OS, a small operating system for a handheld computer, and it is the worked example in every chapter that comes after this one.

PicoCalc OS is small enough to read: about 2,300 lines of C in its nucleus (2,800 with the simulated board and the host runner) and about 1,200 lines of JavaScript. It is not a toy, though. Its drivers speak to a display controller and a keyboard controller register by register, its scheduler runs thirteen isolated components on one processor, and its supervisor restarts any of them, including the display driver, when they fail. Every idea in the chapters that follow, from real-time scheduling to capability systems to self-healing drivers, can be found in it, measured in it, and broken in it on purpose.

1.1 One machine, many programs

Start with a machine and no operating system. The PicoCalc is a pocket computer the size of a paperback: a 4-inch IPS panel of 320 × 320 pixels driven over SPI, a 67-key backlit keyboard scanned by an STM32 microcontroller that talks over I2C, and two 18650 cells [CNX Software 2025]. The kit ships with a Raspberry Pi Pico module. In this book the Pico is replaced by a board built around Espressif's ESP32-P4, a microcontroller with two RISC-V cores rated at up to 400 MHz (the board used here runs them at 360 MHz), 768 KB of on-chip SRAM and external PSRAM, 32 MB of it on the board used here [Espressif ESP32-P4 n.d.]. There is no disk and no memory-management unit of the kind a desktop processor has.

The simplest way to program such a machine is a *superloop*: one program that owns every device and does everything in turn.

```
// A superloop: one program, every device, no operating system (a sketch, not PicoCalc OS
code).
int main(void) {
  board_init();                    // clocks, pins, SPI, I2C
  lcd_init();                      // the panel's power-up sequence
  for (;;) {
    int key = kbd_poll();          // ask the keyboard controller for an event
    if (key) app_handle_key(key);  // update the application's state
    app_draw(framebuffer);         // draw everything
    lcd_send(framebuffer);         // send the whole frame over SPI
  }
}
```

For one fixed application this is a good design. It is predictable, easy to debug, and has no overhead. Its weaknesses appear as soon as the machine has to do more than one thing, and they are the problems an operating system exists to solve.

Sharing the processor. Suppose the PicoCalc should run a REPL where the user types JavaScript, a system monitor that redraws its charts every second, and a snake game that advances every 110 ms. In the superloop each of them must be written to give the processor back quickly, because while one runs, nothing else does: not the keyboard poll, not the other programs. A loop typed at the REPL, `for (;;) {}`, stops the machine.

Sharing the devices. The keyboard controller also reports the battery level, so a battery gauge and the keyboard code must take turns on the same I2C bus, and a transaction from one must not be split by the other. The display has one memory and one SPI bus; three programs drawing to it directly would overwrite each other's pixels.

Containing mistakes. A bug in the game that writes past the end of an array can corrupt the keyboard code's variables. A crash anywhere is a crash everywhere. On a machine with one program that is the program's problem; on a machine with many it is the system's.

Running untrusted code. The REPL runs whatever the user types. A plug-in downloaded from somewhere runs whatever its author wrote. Neither should be able to read another program's data, reprogram the display controller, or keep the processor forever.

An operating system answers these problems with a small number of mechanisms. The superloop's single owner of every device is replaced by a component that owns all of them on behalf of everyone else, and that decides who runs, what each program may touch, and what happens when one of them fails.

1.2 Four jobs: multiplex, isolate, mediate, recover

There are several classical ways to describe what an operating system is. One view says it is a *virtual* (or extended) *machine*: it hides the awkward details of the hardware behind cleaner abstractions, so that a program writes to a file rather than programming a disk controller [Arpaci-Dusseau and Arpaci-Dusseau 2018]. Another says it is a *resource manager*: it divides the processor, memory and devices among competing programs. Both are true, and both leave out the part that matters most once programs do not trust each other. This book uses four jobs, because each corresponds to a distinct set of mechanisms and each can be checked separately in a real system.

> DEFINITION: OPERATING SYSTEM
>
> An **operating system** is the software that lets several programs share one machine. It **multiplexes** the machine's resources among them, **isolates** them from one another, **mediates** every use of shared hardware, and **recovers** when one of its parts fails.

Multiplexing means giving each program the impression of a resource it does not own exclusively. The processor is multiplexed in time: a *scheduler* decides which program runs next and for how long. Memory is multiplexed in space: each program gets its own region, and an allocator hands out and reclaims parts of it. Devices are multiplexed by a driver that queues requests from many programs and serves them in some order. Chapters 4, 5 and 6 are about multiplexing.

Isolation means that what one program does cannot change another program's state except through channels both agreed to. On a desktop processor, isolation comes from hardware: each process runs in its own virtual address space, enforced by the memory-management unit, and the kernel runs in a privileged mode that user code cannot enter except through system calls. On PicoCalc OS, isolation comes from the programming language: each component is a separate JavaScript runtime with its own heap, and JavaScript gives a program no way to forge a pointer into another heap. Chapters 8 and 10 compare the two approaches and their limits.

Mediation means that every use of a shared resource passes through the operating system, which checks that the request is allowed before carrying it out. This is the idea of a *reference monitor*, and it is what makes isolation useful: a program that cannot touch the display controller directly must ask, and the operating system can say no. In PicoCalc OS, a program that has been given access to hardware holds a *handle*, a kernel object it can name but not forge, and the kernel checks the handle on every use. Chapters 9 and 11 are about mediation.

Recovery means that when a component fails, the failure is detected, contained, and repaired without stopping the rest of the system. Early operating systems treated recovery as an afterthought: a failing kernel component meant a reboot. Systems designed for reliability, from Tandem's process pairs [Gray 1985] to Erlang's supervisors [Armstrong 2003] to MINIX 3's reincarnation server [Herder et al. 2007], made it a first-class job. PicoCalc OS restarts any failed component, including its device drivers, with exponential backoff. Chapters 14 and 15 are about recovery, and the second of them is about the hardest case: restarting the code that controls a piece of hardware while other programs are still using it.

Table 1.1. The four jobs of an operating system, how a conventional system does each, and how PicoCalc OS does it.

Job	A conventional OS (Linux on a laptop)	PicoCalc OS	Chapters
Multiplex	Preemptive scheduler with timer interrupts; virtual memory; per-device queues	One *lane* of non-preemptive segments, picked by priority class; one heap per component; a nucleus that serialises the transfers on each bus	4, 5, 6
Isolate	Hardware address spaces and privilege modes	One JavaScript runtime per component; no shared objects	8, 10
Mediate	System calls checked against file permissions and process credentials	Handles checked by the nucleus on every use; a manifest of claims	9, 11
Recover	Restart the process (or reboot if the kernel failed)	A supervisor restarts any component, drivers included, with backoff	14, 15

A useful distinction runs through all four jobs: the difference between *mechanism* and *policy*. A mechanism is the means of doing something: a way to switch from one program to another, a way to check a handle. A policy is the decision about what to do: which program should run next, which program should get the display. Per Brinch Hansen made the distinction the basis of the RC 4000 system in 1969. Its *nucleus* provided

processes, messages between them (send a message, wait for a message, send an answer, wait for an answer) and a hierarchy for creating and controlling them; it deliberately had no built-in strategy for scheduling programs or allocating resources, which were left to operating systems written as ordinary processes on top [Hansen 1970]. Lampson's hints for system design give the same advice in general form: keep the basic interface simple, and leave room for the client to choose [Lampson 1983].

PicoCalc OS borrows the word. Its C core is called the *nucleus*, and its primitives are close relatives of the RC 4000's: a component can send a message, ask a question and wait for the answer, and publish to everyone listening on a channel. The policies, which component draws the screen, which app has the keyboard, how often the battery is read, live in JavaScript components above it.

1.3 The machine: a PicoCalc with an ESP32-P4

An operating system is shaped by the hardware under it, so before looking at the software it pays to see what the software has to work with. Figure 1.1 shows the parts of the PicoCalc that PicoCalc OS uses.

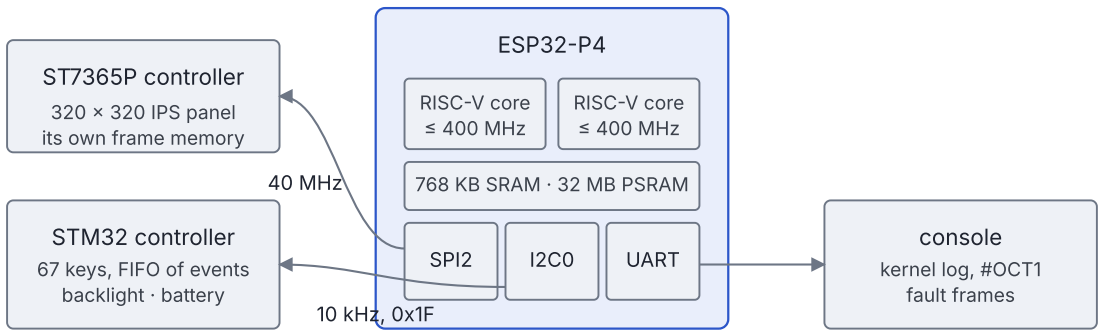


Figure 1.1. The parts of the PicoCalc that PicoCalc OS drives. The ESP32-P4 talks to the display controller over SPI and to the keyboard controller over I2C; the console UART will carry the kernel log and fault reports (on the host build they go to a file). Bus speeds are the ones the drivers configure.

Three features of this hardware matter for everything that follows.

First, the processor is fast and the buses are slow. At 360 MHz a core executes hundreds of instructions in a microsecond. The keyboard controller's I2C bus runs at 10 kHz, so a single byte takes 0.9 ms to cross it (nine clock cycles: eight data bits and an acknowledgement). The display's SPI bus runs at 40 MHz, which sounds fast until you count the bytes: a full screen is 320 × 320 pixels at two bytes each, 204,800 bytes, and takes 41 ms. Most of the time a user waits for, as we will see, is time on a wire.

Second, the peripherals are computers of their own. The ST7365P display controller holds the frame in its own memory and must be woken with a sequence of commands separated by delays it specifies. The STM32 scans the key matrix, debounces it, and keeps a first-in, first-out queue (FIFO) of key events that the main processor reads over I2C. A driver for either one is a client of another program, running on another chip, speaking a protocol defined in a datasheet or, for the STM32, in its firmware source [ClockworkPi n.d.].

Third, there is no memory-management unit that gives each program its own address space. The ESP32-P4 has hardware to restrict which regions of memory code may access, and Chapter 8 discusses using it, but it does not provide the per-process virtual memory that desktop operating systems are built on. That is one reason PicoCalc OS isolates its components with a language instead.

Chapter 2 goes through this hardware in detail: how a bus transaction is timed, how a datasheet specifies a controller, and how the cost of a byte on a wire shapes a driver.

1.4 PicoCalc OS in one page

PicoCalc OS is built in three layers, shown in Figure 1.2. At the bottom is the *board*: the code that touches hardware. Above it is the *nucleus*, written in C, which runs everything else. At the top are the *worlds*, written in JavaScript: three device drivers, the compositor that owns the screen, a sandbox for the REPL, and the apps.

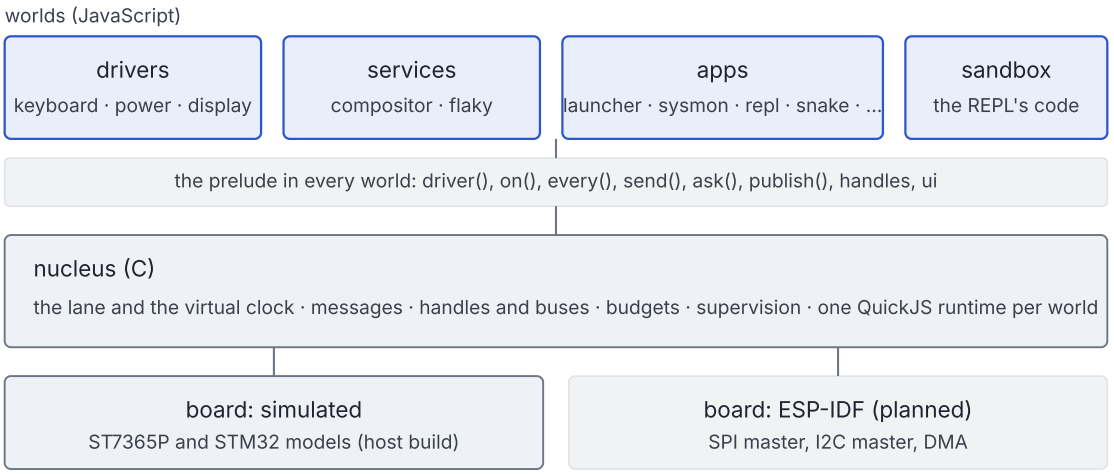


Figure 1.2. The layers of PicoCalc OS. Every driver, service and app is a world: a separate JavaScript runtime. The nucleus runs them one at a time, carries their messages, checks their use of hardware through handles, and restarts them when they fail. The board is a register-level simulation today; the ESP-IDF board is planned.

The central idea is the world.

> **DEFINITION: WORLD**
>
> A **world** is one component of PicoCalc OS: a separate QuickJS runtime with its own heap and heap limit, a time budget for each segment of its turns, a mailbox of incoming messages, and an entry in the system's manifest. A world can change nothing outside itself except by sending messages and by using hardware through handles the nucleus has given it.

Worlds come in three kinds and four priority classes: **drivers** (priority 3), **services** (2), **apps** (1), and the **sandbox** (0), a service that the manifest places below everything else. The nucleus runs one world at a time. Each world reacts to messages: a world declares handlers, and when a message arrives, the nucleus runs the matching handler to completion. That run is called a **turn**. If the handler is asynchronous and waits for something, such as a bus transfer, the turn is divided into **segments**, each of which runs without interruption; between segments other worlds run. Chapter 3 explains why the turn is the right unit and what guarantees segments give.

Which worlds exist, and what each may claim, is set by the **manifest**, a JSON file the nucleus reads at boot. Here are its first entries:

▸ history/first-pass/js/system.json

```
{
…
  "worlds": [
    { "name": "keyboard",   "file": "drivers/keyboard.js",  "claims": ["i2c0:0x1f"],
"budget_ms": 20,  "mem_kb": 320,
      "provides": ["keys"] },
    { "name": "power",      "file": "drivers/power.js",     "claims": ["i2c0:0x1f"],
"budget_ms": 20,  "mem_kb": 320,
      "provides": ["power"] },
    { "name": "display",    "file": "drivers/display.js",   "claims": ["spi2"],
"budget_ms": 30,  "mem_kb": 320 },
    { "name": "compositor", "file": "system/compositor.js", "claims": ["dma"],
"budget_ms": 80,  "mem_kb": 768,
      "needs": ["frames", "keys", "power", "worlds.status"], "sends": ["display",
"power", "apps"], "asks": ["display"],
      "handles": 6, "bufs": 2, "mbox": 128 },
    { "name": "launcher",   "file": "apps/launcher.js",     "claims": ["sys.read"],
"mem_kb": 384,
      "sends": ["compositor"] },
…
    { "name": "sandbox",    "file": "system/sandbox.js",
"budget_ms": 200, "mem_kb": 512, "priority": 0 },
```

The first line of each entry says three things about a world. Its **claims** say which hardware it may ask for: the keyboard and power drivers may open the I2C device at address 0x1F, the display driver may open SPI bus 2, the compositor may allocate DMA-capable memory for a framebuffer. A world without a claim cannot obtain a handle for that hardware at all. Its **budget** is the most virtual time one segment of its turns may take; a segment that runs longer is stopped, and the world counts as crashed. Its **memory limit** caps its heap; an allocation that would cross it fails, and the world counts as crashed. Apps get the default budget of 50 ms; every entry sets its own heap limit, 320 to 512 KB for the apps (the default, unused here, is 1 MB); the sandbox, which runs whatever the user types at the REPL, gets 200 ms and the lowest priority. The other keys (which channels a world provides and needs, whom it may send to and ask, and quotas such as `handles` and `mbox`) were added by a later revision of the system, its first pass; a separate book, *From L4 to Small Worlds*, describes them in its Chapter 5 and Appendix A. A second pass then replaced the keyboard and power drivers with one STM32 driver and a power service, replaced `sends` and `asks` with typed

endpoints, made the sandbox passive (it runs on the budget of whoever asks it) and moved restart policy into a supervisor world; `SECOND-PASS.md` in the repository records it. This book describes the system before the second pass, and its excerpts of files that pass changed quote the copy in `history/first-pass/`.

A world is written against a small API that the nucleus installs in every runtime before the world's own file runs. Here is the whole of one app, the About screen, slightly shortened:

▸ `js/apps/about.js`

```
app("about")
  .title("About")
  .state({ focused: false, st: null, worlds: [] })
  .on("focus", (s) => { s.focused = true; s.st = sys.stats(); s.worlds = sys.worlds(); })
  .on("blur", (s) => { s.focused = false; })
  .every(1000, (s) => { if (s.focused) { s.st = sys.stats(); s.worlds = sys.worlds(); }
})
  .screen((s) => {
    if (!s.focused || !s.st) return ui.text("");
    const st = s.st, mem = s.worlds.reduce((a, w) => a + w.mem_kb, 0);
    return ui.col({ gap: 6 },
      ui.text("PicoCalc OS", { font: "l" }),
…
    );
  });
```

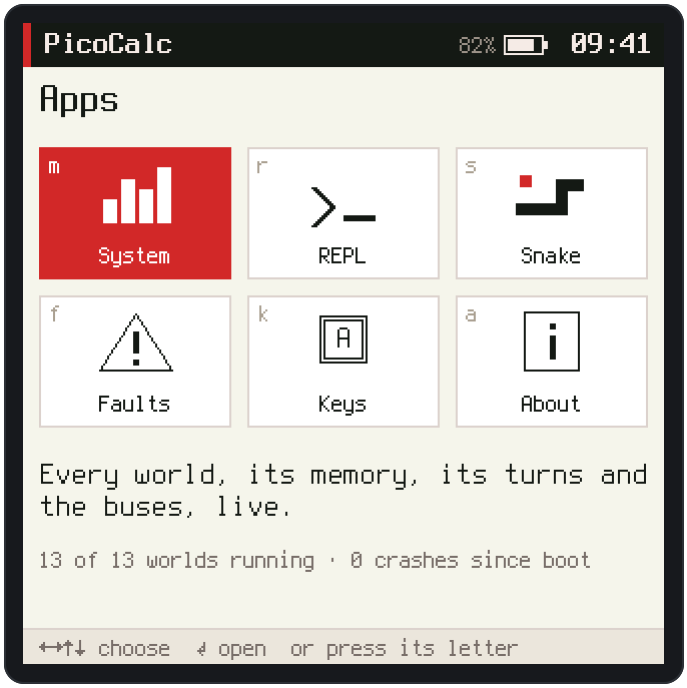
The app declares its state, two message handlers (`focus` and `blur`, which the compositor sends when the app comes to the front or leaves it), a timer that runs every 1,000 ms, and a `screen` function that describes what the app shows as a tree of plain objects. It never draws a pixel. After each segment of the app's turns, the nucleus calls `screen`, and if the tree differs from the last one, publishes it as a *frame* to the compositor, which lays it out and draws it. `sys.stats()` works only because the manifest gives About the `sys.read` claim.

The vocabulary of the rest of the book comes from this picture. A **message** is a JSON value sent from one world to another: `send` delivers it, `ask` delivers it and returns a promise for the reply, and `publish` delivers it to every world that subscribed to a *channel*. A **handle** is the nucleus's record that a world may use a piece of hardware with certain rights; the world sees it as an object with methods such as `read`, `cmd` or `fill`. The **lane** is the single sequence in which segments run. The **virtual clock** is the nucleus's notion of time, advanced by the measured cost of each segment and each bus transfer, which Section 1.6 explains.

TRY 1.1

Use PicoCalc OS in this page

The panel below is PicoCalc OS itself, compiled to WebAssembly: the same C nucleus, the same JavaScript worlds, and the same simulated display and keyboard controllers as the host build, paced to real time. Click the screen and use the arrow keys and Enter, or the buttons on phones. Open the System app to see every world with its heap and turns. Then press **Kill display**: the display driver's world is killed, the screen freezes, and about half a second later it is back, because the supervisor restarted the driver, the driver re-initialised the panel, and the compositor sent every pixel again. Chapter 15 takes that half second apart. In the printed book this panel shows the home screen as captured by the host build.



1.5 One key press, end to end

The best way to see an operating system's mechanisms is to follow one event through all of them. We follow a press of the right-arrow key on the home screen, which moves the launcher's selection from one tile to the next. The trace that follows was recorded by the host build, which writes one JSON line for every

segment, message, bus transfer and pixel change.

Figure 1.3 shows the participants: the STM32 that scanned the key, the keyboard driver, the compositor, the launcher, the display driver and the panel.

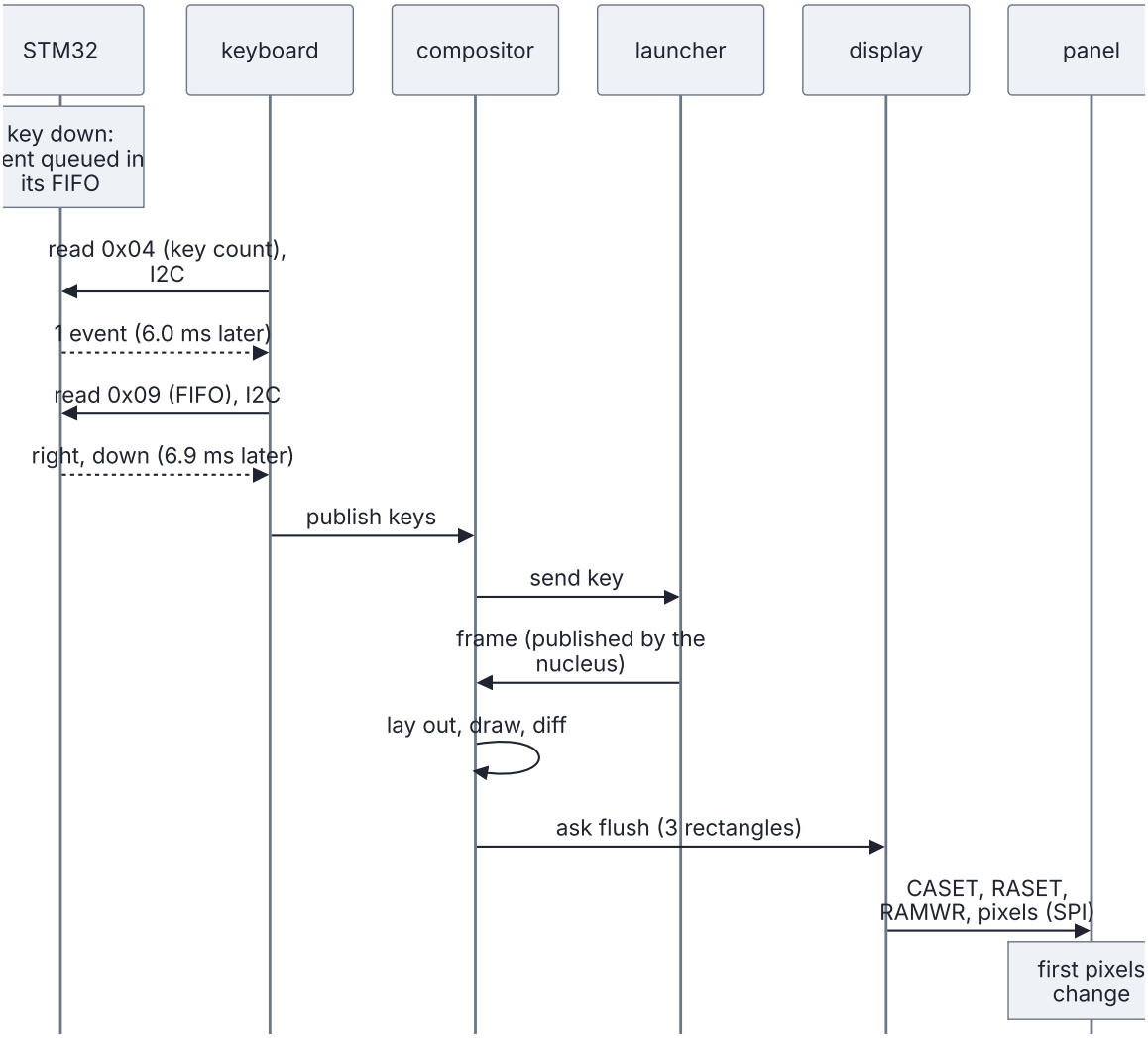


Figure 1.3. One key press. The keyboard driver finds the event on its next poll, two I2C transactions later publishes it, the compositor forwards it to the focused app, the app's new frame comes back to the compositor, and the display driver sends the changed rectangles over SPI.

The poll. The STM32 queues the key event the moment it debounces the switch, but it cannot interrupt the ESP32-P4; it waits to be asked. The keyboard driver asks every 20 ms:

▸ history/first-pass/js/drivers/keyboard.js

```
const readReg = (dev, reg, n) => dev.read(reg, n, 2);

driver("keyboard")
  .provides("keys")
…
  .every(20, async (s) => {
    if (!s.dev) return;
    s.polls++;
    try {
      const [status] = await readReg(s.dev, REG_KEY, 1);
      let count = status & 0x1f;
      while (count-- > 0) {
        const [state, code] = await readReg(s.dev, REG_FIF, 2);
        if (!state) break;
…
        publish("keys", { code, key, ch, state: STATES[state] || String(state), caps: !!
(status & 0x20) });
      }
```

Each `readReg` is one I2C transaction: write the register number, hold the bus for 2 ms while the controller prepares its answer (its firmware needs the gap), then read. The first reads register 0x04, whose low five bits count the events waiting; the second reads one event from the FIFO register 0x09. Both calls return promises. When the driver writes `await`, its segment ends; the nucleus starts the transfer on the I2C bus and lets other worlds run until the transfer finishes, then runs the rest of the handler as a new segment.

The trace. Here are the first lines of the recorded trace for one press, with times in virtual microseconds since boot:

▸ bench/traces/keypress.jsonl


```
{"k":"key","t":16987260,"w":"","text":"0xb7 down"}
{"k":"seg","w":"keyboard","t":17000078,"dt":59,"steps":26,"native":2,"kind":"msg","type":
"every 20 ms"}
{"k":"bus","bus":"i2c0","w":"keyboard","t":17000137,"t1":17006137,"bytes":4,"what":"write
0x04, wait, read 1"}
{"k":"seg","w":"keyboard","t":17006137,"dt":50,"steps":14,"native":1,"kind":"settle","typ
e":"every 20 ms"}
{"k":"bus","bus":"i2c0","w":"keyboard","t":17006187,"t1":17013087,"bytes":5,"what":"write
0x09, wait, read 2"}
{"k":"seg","w":"keyboard","t":17013087,"dt":76,"steps":50,"native":3,"kind":"settle","typ
e":"every 20 ms"}
{"k":"msg","t":17013163,"from":"keyboard","to":"compositor","type":"keys","ask":false,"by
tes":62}
```


Read it line by line. The key went down at 16,987.260 ms. The keyboard driver's timer fired 12.8 ms later, at 17,000.078 ms: that wait is pure chance, the time until the next 20 ms poll. Its first segment took 59 µs, of which 26 interpreter steps cost 17 µs, 2 µs went to native work, and 40 µs is the fixed cost of entering a world. The status read held the bus from 17,000.137 to 17,006.137, exactly 6.0 ms: 4 bytes on the wire (address and register, then address and answer) at 9 clocks each plus start and stop conditions, 40 clocks at 10 kHz, plus the 2 ms gap. The driver resumed (a `settle` segment: its promise had settled), started the second read, which took 6.9 ms because it reads two bytes, and resumed again to publish the event. The nucleus delivered the `keys` message to its subscribers, the compositor and the Keys app, at the end of that segment.

The rest of the press continues in the same file:

▸ bench/traces/keypress.jsonl


```
{"k":"seg","w":"compositor","t":17013163,"dt":80,"steps":56,"native":3,"kind":"msg","type
":"keys"}
{"k":"msg","t":17013243,"from":"compositor","to":"launcher","type":"key","ask":false,"byt
es":62}
{"k":"seg","w":"launcher","t":17013310,"dt":127,"steps":89,"native":28,"kind":"msg","type
":"key"}
{"k":"msg","t":17013437,"from":"launcher","to":"compositor","type":"frames","ask":false,"
bytes":1135}
{"k":"seg","w":"compositor","t":17013437,"dt":492,"steps":637,"native":29,"kind":"msg","t
ype":"frames"}
{"k":"seg","w":"compositor","t":17013929,"dt":1412,"steps":1204,"native":572,"kind":"msg"
,"type":"draw"}
{"k":"msg","t":17015341,"from":"compositor","to":"display","type":"flush","ask":true,"byt
es":55}
{"k":"seg","w":"display","t":17015341,"dt":240,"steps":204,"native":65,"kind":"msg","type
":"flush"}
{"k":"px","t":17015589,"t1":17018135,"changed":6336}
{"k":"bus","bus":"spi2","w":"display","t":17015589,"t1":17018135,"bytes":12672,"what":"pi
xels 96x66 at 216,62"}
```


The compositor, which knows which app is in front, forwarded the key to the launcher. The launcher's handler moved its selection and, as part of the same segment, the nucleus called its `screen` function and published the resulting frame, 1,135 bytes of JSON, on the `frames` channel. The compositor took the frame, then in a second turn drew the whole layout into its framebuffer and compared it with what the panel already shows. The comparison found three changed rectangles: the old selected tile, the new one, and the description text under the grid. The compositor asked the display driver to flush them. The driver queued, for each rectangle, three commands (set the column range, set the row range, start writing to memory) and the pixel data, and the first rectangle, 96 × 66 pixels, crossed the SPI bus in 2.5 ms. The first changed pixels reached the panel at 17,018.135 ms, 30.9 ms after the key went down.

Table 1.2. Where the time went in the press traced above. Durations are virtual milliseconds on the host build's clock, which is calibrated against the ESP32-P4 (Section 1.6).

Stage	Time	Share
Waiting for the next 20 ms poll	12.9 ms	42%
I2C: read the key count (register 0x04)	6.0 ms	19%
I2C: read the event (register 0x09)	6.9 ms	22%
Routing: keyboard → compositor → launcher, which renders	0.35 ms	1%
Compositor: take the frame, lay out, draw, diff	1.9 ms	6%
Display driver queues the transfers	0.25 ms	1%
SPI: the first changed rectangle	2.5 ms	8%
First pixels	**30.9 ms**	
SPI: the rest of the update	5.9 ms	

Table 1.2 shows the striking fact about this system: the processor's share of the latency is under 3 ms. The keyboard path, the wait for the poll and the two reads at 10 kHz, accounts for 83% of the time to first pixels. Across 100 presses the time to first pixels has a median of 30.9 ms and a maximum of 40.7 ms; the variation comes almost entirely from where the press lands in the 20 ms poll period. Chapter 19 shows how this decomposition was derived and checked against all 100 presses, and Chapter 5 asks what can be *guaranteed* about it rather than merely measured.

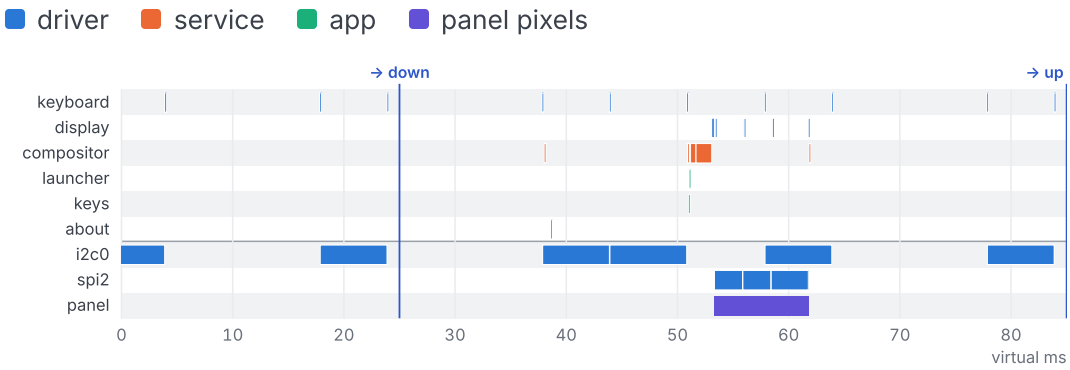
TRY 1.2

Explore the key press

The timeline shows the same press: one row per world that ran, a row for each bus, and a row for the panel, where a bar means pixels changed value. Hover over a bar for its details, drag across the chart to zoom into the cluster of short segments between 50 and 54 ms, where the event travels from the keyboard driver to the display driver, and switch to the list view to read the events in order. Notice how little of the timeline the processor occupies: the long bars are bus transfers.

One key press 85 virtual ms

The launcher, the right-arrow key pressed at 25 ms. The keyboard driver finds it on its next poll, two I2C reads later the event is published, and the compositor draws and flushes the change.



Every one of the four jobs appears in this trace. The lane *multiplexed* the processor among the keyboard driver, the compositor, the Keys app, the launcher and the display driver, and while the I2C bus was busy for 12.9 ms it ran other worlds' timers. The worlds were *isolated*: the launcher never touched the compositor's framebuffer, and the compositor never read the launcher's state; they exchanged a frame as data. The nucleus *mediated* every use of hardware: the keyboard driver's reads went through its I2C handle, and the display driver's pixel transfer read the framebuffer through a read-only handle the compositor had given it. *Recovery* did not happen in this trace, but its preparation did: each segment's messages were held back until the segment completed, so that if any segment had failed, its effects would have been discarded rather than half applied.

1.6 A simulated board and a virtual clock

The trace above was not recorded on a PicoCalc. PicoCalc OS runs today in a *host build*: the same nucleus and the same JavaScript worlds, compiled for Linux and running against software models of the two peripheral controllers, the ST7365P display controller and the STM32 keyboard controller. The models implement the controllers at the level of their registers and commands, so the drivers that run against them are the drivers that would run on the device. The port to the ESP32-P4 itself, which replaces the models with Espressif's SPI and I2C drivers, is planned work, and the book says so wherever it matters.

A simulation is only useful if its time means something. The host build keeps a **virtual clock**. Each segment advances it by a cost computed from what the segment did:

$$\text{cost} = c_0 + \mathit{steps} \cdot \tau + \mathit{native} \tag{1.1}$$

where $c_0 = 40$ µs is a fixed cost for entering a world, *steps* counts the QuickJS interpreter's control-flow events (every jump, conditional branch and call), and *native* is the estimated time of the native work the segment asked for, such as filling rectangles or blending glyphs. The cost per step, $\tau = 665$ ns, was calibrated against a measurement on the real ESP32-P4: a 100,000-iteration loop takes 133 ms there and 200,002 steps here. As a check, the same model predicts 29.1 ms for a recursive `fib(20)`, which the device ran in 31 ms. Bus transfers take the time their bits take on the wire at the configured clock. Chapter 18 explains the model, its calibration, and what it leaves out.

The virtual clock has three consequences that this book relies on. Runs are **repeatable**: the same script produces the same trace and the same pixels every time, so a timing problem that appears once appears on every run and can be read off the trace. Runs are **fast**: the host build runs about 800 times faster than real time when the system is idle and about 150 times faster with the snake game playing itself, so a benchmark of a hundred key presses takes a fraction of a second. And runs are **observable**: every segment, message and transfer is recorded, as the trace above showed.

Here is a short session with the 32-bit host build, which gives each world the heap it would have on the 32-bit ESP32-P4. It runs the system for one second, presses the right-arrow key, runs 100 ms more, and prints the nucleus's statistics:

```
$ printf 'wait 1000\nkey right\nwait 100\nstats\nquit\n' | ./build32/picocalc-host -
t = 1170.0 ms, lane busy 19.8 ms (1.7%), 97 turns, 9 published, 20 delivered, 0 crashes
  world       kind     state        turns    cpu ms   mem KB  limit  rst  last error
  keyboard    driver   idle            55      6.33    187.9    320    0
  power       driver   idle             1      0.30    185.7    320    0
  display     driver   idle             5      3.45    183.7    320    0
  compositor  service  idle            16      6.96    265.1    768    0
  launcher    app      idle             2      0.41    190.2    384    0
  sysmon      app      idle             3      0.29    201.4    512    0
  repl        app      idle             0      0.06    184.8    384    0
  sandbox     service  idle             0      0.06    181.9    512    0
  snake       app      idle            11      1.43    197.8    320    0
  faults      app      idle             0      0.07    199.0    384    0
  flaky       service  idle             0      0.06    181.9    256    0
  keys        app      idle             3      0.27    183.8    320    0
  about       app      idle             1      0.12    183.9    320    0
  bus i2c0  busy   351.60 ms, 58 transfers, 236 bytes, 0 errors
  bus spi2  busy    72.86 ms, 46 transfers, 362838 bytes, 0 errors
```

Two numbers deserve a second look. The lane, the processor, was busy for 19.8 ms of the 1,170 ms, 1.7% of the time, including the boot. The I2C bus was busy for 351.6 ms, 30% of the time, almost all of it the keyboard driver's 54 polls, each holding the bus for 6 ms; the two FIFO reads for the key press and the power driver's two start-up reads make up the rest. A system that is idle from the processor's point of view can be saturated from a bus's. Thirteen worlds take about 2.5 MB of heap together, most of it the cost of a QuickJS runtime and the prelude in each; on a board with 32 MB of PSRAM that is affordable, and Chapter 6 examines the cost.

The figure below is the home screen as the host build captured it, from the simulated panel's own memory. The PNG shows exactly what the glass would show.



Figure 1.4. The home screen of PicoCalc OS, captured from the simulated ST7365P's frame memory at twice its size. The launcher's tiles open the six apps; the status bar shows the battery level the power driver read from the STM32 and the virtual clock's time of day.

1.7 How this book is organised

The book has six parts. Each part teaches a body of operating-systems knowledge from first principles, then shows it in PicoCalc OS, measured and, where possible, broken on purpose.

Table 1.3. The parts of the book and what PicoCalc OS contributes to each.

Part	Chapters	What PicoCalc OS shows
I. Foundations	1 What an OS does · 2 The machine · 3 Concurrency · 4 Scheduling	Buses and their timing; turns and segments; the lane's priority classes
II. Embedded and real-time	5 Real-time systems · 6 Memory · 7 Embedded engineering	What a non-preemptive lane can guarantee; one heap per world; the road to the device
III. Isolation and protection	8 Protection and kernel structure · 9 Capabilities · 10 Language isolation · 11 Resource control	A language-isolated microkernel; handles and a confinement bug; the clock leak; budgets
IV. Communication, drivers and recovery	12 IPC · 13 Device drivers · 14 Failure and recovery · 15 Self-healing drivers	Messages with deferred effects; drivers in JavaScript; supervision; drivers that restart under load
V. The browser as an OS platform	16 The browser as an OS · 17 An OS inside the page	The same design at the scale of the web; PicoCalc OS running in this page
VI. Engineering the system	18 Deterministic simulation · 19 Measuring systems · 20 Practice	The virtual clock; latency anatomy; how checking claims against code found real bugs

Three threads run through all six parts.

The first is **isolation by language**. PicoCalc OS separates its components with separate JavaScript runtimes rather than hardware address spaces. This is the same choice, at small scale, that the web browser made at large scale: a browser runs code from thousands of mutually distrustful sources, and for most of its history the boundary between them was the JavaScript engine. Part III examines what that boundary guarantees and what it does not, and Part V follows the browser's own journey from language isolation to a hybrid of language and process isolation after the Spectre attacks.

The second is **time**. An embedded system is judged by when it responds, not only by whether it responds correctly. The virtual clock makes time in PicoCalc OS explicit and measurable; budgets bound how long any component can hold the processor; and real-time theory says what those bounds imply.

The third is **recovery**. Device drivers are the part of an operating system most likely to fail, and the part whose failure has historically been hardest to contain. PicoCalc OS restarts its drivers routinely, and Part IV builds up, from fault models and supervision to the specific design rules that make a driver restartable, to the question of how a driver can heal itself while programs keep using the device.

KEY POINTS

- An operating system lets several programs share one machine without trusting one another; it does four jobs: multiplexing, isolation, mediation and recovery.
- Multiplexing divides the processor in time, memory in space and devices by queuing; isolation keeps one program from changing another's state; mediation checks every use of shared hardware; recovery contains and repairs failures.
- Mechanism and policy are separate: the nucleus provides processes, messages and checks, and the decisions about who runs and who gets which device live above it, as Brinch Hansen's RC 4000 nucleus first made explicit.
- PicoCalc OS has three layers: a board that touches hardware, a C nucleus, and worlds, which are separate JavaScript runtimes for each driver, service and app.
- A world reacts to messages in turns; an asynchronous turn is divided into segments, and the messages a segment sends are delivered only when it completes.
- On the PicoCalc, most of the time a user waits for is time on a wire: of the 30.9 ms from a key press to the first changed pixels, the keyboard's 20 ms poll and two I2C reads at 10 kHz take 83%, and the processor less than 3 ms.
- The host build runs PicoCalc OS against register-level models of the display and keyboard controllers on a virtual clock calibrated to the ESP32-P4, which makes runs repeatable, fast and fully traced; the device port is planned.

The key press showed every layer of PicoCalc OS at once, but it took for granted what the layers stand on: a processor that runs instructions, memory that holds them, and buses whose timing is set by their clocks. Chapter 2 looks at that machine closely, because an operating system's designers have to know exactly what the hardware promises before they can decide what to promise to programs.

EXERCISES

1. BASIC For each of the four jobs of an operating system, name the mechanism PicoCalc OS uses and the mechanism Linux uses on a laptop. Which pair differs most, and why?

2. BASIC Classify each as mechanism or policy: the nucleus's check that a handle has the `w` right; the compositor's decision to give the keyboard to the app in front; the manifest's 200 ms budget for the sandbox; the backoff schedule of 100, 200, 400 and 800 ms.

3. ANALYSIS Using the formula for an I2C transaction in this chapter (9 clocks per byte, one byte of address per phase, start and stop conditions, plus the 2 ms gap), verify the 6.0 ms and 6.9 ms reads in the trace. How long would each take at 100 kHz? What would the median time to first pixels become, assuming everything else stays the same?

4. ANALYSIS A full screen is 320 × 320 pixels at 16 bits per pixel. Compute the SPI time at 40 MHz and at 80 MHz, ignoring command overhead. Why do you think the display driver's author chose 40 MHz? (Read the comment in `js/drivers/display.js`.)

5. ANALYSIS The first 96 × 66 rectangle took 2.546 ms on the SPI bus. Account for every microsecond: pixel bytes, clocks per byte, the bus clock, and the 12 µs DMA setup charged per 32 KiB chunk.

6. CODE Run the 32-bit host build with `printf 'wait 1000\nstats\nquit\n' | ./build32/picocalc-host -` and then the same with `wait 10000`. How do the I2C bus's busy percentage and the lane's busy percentage change, and why?

7. CODE Record your own trace of a key press: start the host build with `./build/picocalc-host -`, then enter `trace press.jsonl` (the file is written under `out/`), `wait 1000`, `key right`, `wait 100`, `quit`. Find the press in the file and compute its time to first pixels. Repeat with `wait 1010` before the key. Explain the difference.

8. CODE Remove the `sys.read` claim from the About app's manifest entry in `js/system.json`, rebuild nothing (the manifest is read at boot) and open About. What happens, and which part of the system decided it?

9. DISCUSSION The superloop in Section 1.1 has no operating-system overhead at all. Describe an application for the PicoCalc for which it is the better design, and one feature request that would make you replace it with an operating system.

10. DISCUSSION PicoCalc OS's worlds are isolated by the JavaScript language rather than by hardware. List what a malicious app could still do to other apps or to the system if the language runtime had no bugs. Then list what it could do if QuickJS had a memory-safety bug. Keep your lists; Chapter 8 and Chapter 10 return to them.

11. PROJECT Write a new app world that shows the number of key presses since boot. It will need to subscribe to the `keys` channel (look at `js/apps/keys.js`), keep a count in its state, and describe its screen with `ui.text`. Add it to the manifest and the launcher's `APPS` list, and capture its screen with the host build's `capture` command.

FURTHER READING

[Hansen 1970] The short paper that introduced the nucleus: processes, message passing and a hierarchy of operating systems built on top. Read it for the clearest early statement of mechanism without policy.

[Lampson 1983] Hints drawn from building real systems at Xerox PARC and elsewhere. Many of the decisions in PicoCalc OS can be traced to one of them.

[Arpaci-Dusseau and Arpaci-Dusseau 2018] A free, clear introduction to the classical material on virtualisation, concurrency and persistence, organised around the same questions this book asks of a much smaller machine.

CHAPTER 2 PART I · FOUNDATIONS

The machine underneath

An operating system makes promises about time, memory and devices, and it can only promise what the hardware underneath makes possible. This chapter describes the machine PicoCalc OS is built for: a RISC-V microcontroller with two privilege modes and no virtual memory, a few hundred kilobytes of fast on-chip memory and megabytes of slower external memory, and two peripherals that are not memory-mapped at all but sit at the far end of serial buses, each run by a controller with its own firmware or its own datasheet. By the end of the chapter you should be able to read the relevant parts of a datasheet, compute how long a bus transaction takes from its clock rate, and explain why a key press takes about 31 ms to reach the screen on the host build's virtual clock (which is calibrated to the ESP32-P4; the system has not yet run on the device) and why more than four fifths of that time has nothing to do with the processor.

The chapter works from the inside out. It starts with what a processor gives an operating system (instructions, privilege, interrupts), moves to memory, then to peripherals and the buses that reach them, and then reads two real devices: the display controller from its datasheet and the keyboard controller from its firmware. It ends by adding up the time of one key press, stage by stage, and checking the sum against a recorded trace.

2.1 What the hardware gives an operating system

Every operating system rests on a small set of hardware facilities, and it is worth naming them before looking at any particular chip. The processor executes instructions from memory, one stream per core. It runs at a **privilege level** that decides which instructions and which memory are allowed, and it can be diverted by an **interrupt** or an **exception** into code chosen by the most privileged software. Memory holds code and data at different speeds and sizes. Devices are reached either through **memory-mapped registers**, ad-

dresses that behave like memory but cause effects, or through buses whose transactions are themselves driven by such registers. A **DMA** engine can move data between memory and a device without the processor copying each word. Everything an operating system does is built from these: multiplexing the processor is done with interrupts and saved register state; isolating programs is done with privilege and memory protection; mediating devices is done by keeping their registers out of reach of untrusted code.

A desktop operating system can assume a lot of this hardware: a memory management unit that translates every address, several privilege rings, gigabytes of uniform memory, and devices on PCI Express that announce themselves. A microcontroller offers much less, and the differences shape the whole design of PicoCalc OS. Figure 2.1 shows the machine.

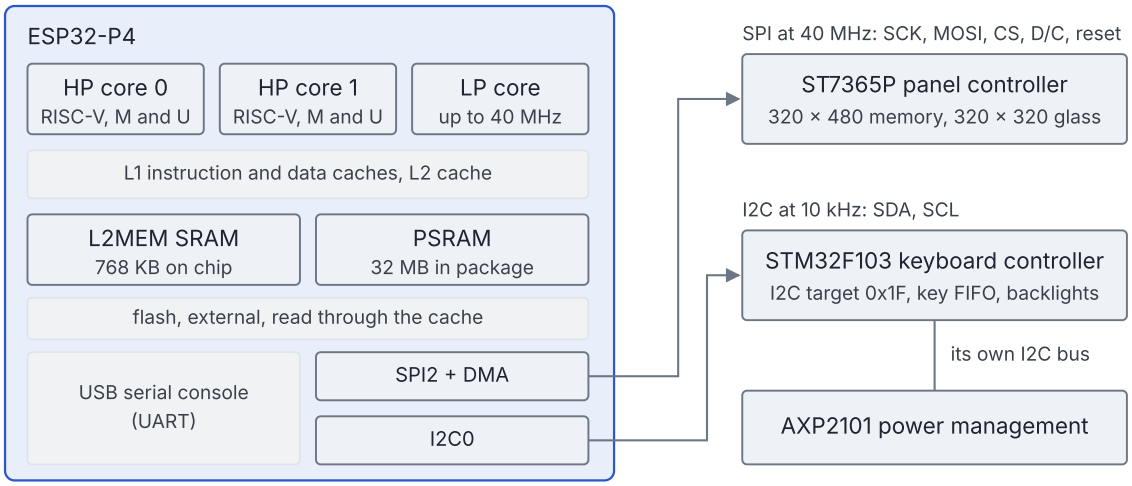


Figure 2.1. The PicoCalc with an ESP32-P4 in place of the Raspberry Pi Pico. The processor reaches the panel controller over SPI and the keyboard controller over I2C; the keyboard controller reaches the power-management chip over a second I2C bus of its own. Clock rates are the ones PicoCalc OS uses.

Table 2.1 lists the parts and where each fact in this chapter comes from. The processor's figures come from Espressif's datasheets; the panel controller's from Sitronix's datasheet; the keyboard controller's from clockworkpi's firmware and the board's schematic, both published in the PicoCalc repository [ClockworkPi n.d.].

Table 2.1. The parts of the machine and the sources this chapter takes their facts from.

Part	What it is	How the processor reaches it	Source
ESP32-P4	Two RISC-V high-performance cores, one low-power core, 768 KB SRAM, 32 MB PSRAM in the package	(the processor)	[Espressif 2026b; Espressif 2026a]
ST7365P	Display controller with its own frame memory	SPI with a data/command line	[Sitronix 2023]
STM32F103R8T6	A microcontroller running the keyboard firmware	I2C, target address 0x1F	[ClockworkPi n.d.]
AXP2101	Power-management chip: battery gauge, charger	Only through the STM32	[ClockworkPi n.d.]

2.2 Processors, privilege and protection

The ESP32-P4's high-performance (HP) subsystem has two 32-bit RISC-V cores implementing RV32IMAFC with compressed-instruction extensions and Espressif's own DSP and loop extensions, with a five-stage pipeline. Espressif rates them at up to 400 MHz on chip revision v3.x [Espressif 2026b]; on the earlier revision v1.3 the default is 360 MHz, with 400 MHz available on request [Espressif 2026a]. The board used to calibrate PicoCalc OS's virtual clock runs its cores at 360 MHz. A third core, in the low-power (LP) subsystem, runs at up to 40 MHz. PicoCalc OS uses one HP core; Chapter 4 discusses what a second lane on the other core would take.

RISC-V defines privilege levels in its privileged architecture specification [Waterman et al. 2021]. Machine mode (M) is the most privileged and is the only level every implementation must have; supervisor mode (S) exists for operating systems with virtual memory; user mode (U) is for untrusted code. The ESP32-P4's HP cores implement M and U only [Espressif 2026a]. There is no S mode, and so no page tables and no virtual memory: every address a program uses is a physical address.

> **DEFINITION: PRIVILEGE MODE**
>
> A **privilege mode** is a state of the processor that determines which instructions may execute and which memory and control registers may be accessed. Code in a less privileged mode that attempts a forbidden operation causes an exception, which transfers control to code in the most privileged mode.

Without page tables, protection on a RISC-V microcontroller comes from **physical memory protection** (PMP). A PMP entry is a pair of control registers: an address register and an 8-bit configuration register that gives read, write and execute permissions and an address-matching mode. An entry can describe a range between its address and the previous entry's (top of range, TOR), or a naturally aligned power-of-two region (NAPOT). The specification allows zero, 16 or 64 entries, and the lowest-numbered entry that matches an access decides whether it succeeds. Entries apply to U-mode accesses; an entry whose lock bit L is set also applies to M-mode, and stays locked until the core is reset [Waterman et al. 2021, §3.7]. The ESP32-P4's HP cores have up to 32 PMP regions on revision v3.x (16 on v1.3), plus physical memory attribute regions, and the chip adds an access permission manager that limits what each DMA master and each core may touch [Espressif 2026b; Espressif 2026a].

What does this give an operating system? It gives exactly enough to build a two-level protection scheme: a kernel in M mode, and untrusted code in U mode confined to the regions PMP opens for it. It does not give address translation, so two programs cannot both be linked at the same address, and moving a program means relocating it. A system call is an `ecall` instruction that traps from U to M mode, and every entry and exit costs saving and restoring registers.

PicoCalc OS does not use any of this today. Its nucleus and all thirteen worlds run in the same address space, at the same privilege, on one core; worlds are isolated from each other because each is a separate JavaScript runtime that can only act through the nucleus's host functions (Chapter 10). That is a deliberate trade, and Chapter 8 examines it: language isolation is cheap and fine-grained, but a bug in the nucleus or in the JavaScript engine is not contained by anything. The hardware would allow a middle ground, in which the nucleus and the engine run in M mode with a locked PMP entry protecting the nucleus's own data, so that at least a wild write from engine code could not rewrite the handle table. That is a plan, not something the host build can show, since the host build runs on Linux and has neither PMP nor M mode.

The other facility a processor provides is the interrupt. A peripheral raises a signal, the core finishes or abandons the current instruction, saves a little state and jumps to a handler. The time from the signal to the first instruction of the handler is the **interrupt latency**, and the time until the work the interrupt stands for has been done is usually much longer, because the handler typically records the event and leaves the work to a task (Chapter 3 treats this deferred-work pattern in detail, and Chapter 7 shows how FreeRTOS structures it). PicoCalc OS's worlds never see interrupts. On the host build, nothing interrupts at all: a bus transfer's completion is an event on the virtual clock's queue, delivered as a settled promise to the world that started it. On the device, the nucleus will take the SPI and I2C completion interrupts through ESP-IDF's drivers and turn them into the same events.

2.3 Memory: speed, size and where things live

A microcontroller's memory is not one thing. The ESP32-P4 has several memories with different sizes and speeds, and deciding what goes where is part of designing the system. Table 2.2 lists them with the figures from the datasheets.

Table 2.2. The ESP32-P4's memories, from Espressif's datasheets, and what PicoCalc OS would keep in each.

Memory	Size	Notes	In PicoCalc OS
L1 instruction cache	16 KB	4-way	
L1 data cache	64 KB	write-back or write-through	
L2 cache	128, 256 or 512 KB	configurable	caches flash and PSRAM
L2MEM (HP SRAM)	768 KB	on chip, fastest large memory	nucleus data, stacks, DMA descriptors
LP SRAM	32 KB	in the low-power subsystem	unused
PSRAM	16 or 32 MB	in the package, 16-bit bus, up to 200 MHz DDR	world heaps, framebuffers
Flash	external	mapped through the cache	code, JavaScript sources

The numbers that matter for an operating system are the ratios. A world's heap in PicoCalc OS is about 190 KB (Chapter 6 measures it); thirteen of them take about 2.5 MB. The display's framebuffer is 320 × 320 pixels at two bytes each, 204,800 bytes, and the compositor keeps two copies, a back buffer to draw into and a front buffer to send from, so about 400 KB. Neither fits beside everything else in 768 KB of SRAM. The world heaps and the framebuffers therefore have to live in PSRAM, and PSRAM is reached through the cache. A sequential walk through a framebuffer streams well through a cache; a JavaScript engine chasing pointers through a heap that is much larger than the cache does not. This is the main reason the cost model of Chapter 18 charges an interpreter step a single measured average (665 ns) rather than deriving it from the clock rate: at 360 MHz a step takes about 240 cycles, and much of that is memory stalls that no simple formula predicts.

The PSRAM itself is fast for an external memory. It sits in the same package as the processor, on a 16-bit data bus clocked at up to 200 MHz on both edges [Espressif 2026a], a theoretical peak of 800 MB/s. Peak bandwidth is a poor guide to a heap's behaviour, though, because each cache miss costs a full transaction on the PSRAM bus regardless of how few bytes were wanted. Chapter 6 returns to what this means for allocation and garbage collection.

DMA adds a constraint of its own. A DMA engine reads memory directly, without going through the core's cache, so a buffer it sends must be written back from the cache first, and on many chips must sit in memory the DMA engine can reach at all. ESP-IDF's SPI master driver documents that buffers it can use directly must be allocated in DMA-capable memory and 32-bit aligned, and that otherwise it allocates a temporary buffer and copies [ESP-IDF SPI master 2026]. Whether the 200 KB framebuffer can be sent from PSRAM directly, or must go through a smaller DMA-capable buffer in pieces, is one of the questions the port in Chapter 7 has to answer on the hardware. The host build's cost model charges each 32 KiB chunk of a transfer 12 µs of setup, which is an estimate standing in for whichever of the two it turns out to be.

2.4 Peripherals are registers

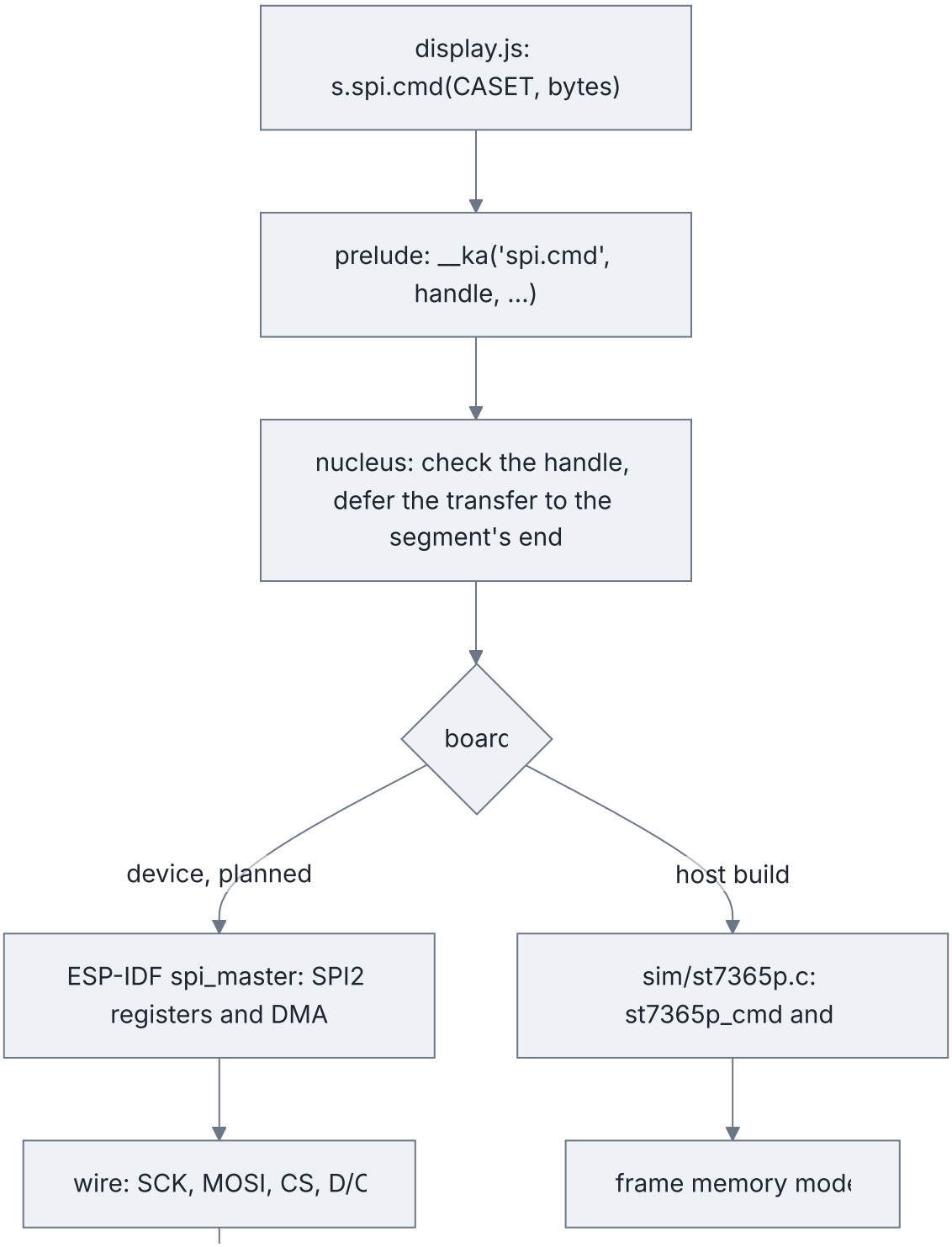
From the processor's side, a peripheral is a block of addresses. Writing to one of them may start a transfer; reading another may report whether the transfer has finished. These are **memory-mapped I/O registers**, and they differ from memory in three ways that matter to a driver writer. Reads can have side effects (reading a receive register may consume the byte it returns). Writes can take effect later than the instruction that made them. And the value read back need not be the value written, because the device changes registers on its own. Code that touches device registers therefore has to be written with exactly the order and width of accesses the device expects, and an operating system keeps such code in drivers so that nothing else can disturb that order.

A driver learns that a device has finished in one of two ways. It can **poll**: read a status register repeatedly, or at a fixed period, until the device reports that it is done. Or it can ask for an **interrupt**: configure the device to raise a signal when it is done, and handle the signal. Polling costs processor time and adds up to one period of latency; interrupts cost a handler and the complexity of concurrency, but the latency is short and the processor is free while waiting. When a device offers no interrupt, polling is the only choice.

The PicoCalc's two peripheral controllers are a level further away than this. Neither the ST7365P nor the STM32 is mapped into the ESP32-P4's address space. Each is a separate chip with its own registers, reached through a bus transaction, and the bus transaction is itself performed by an on-chip controller (SPI2 or I2C0) that is memory-mapped. So there are two layers of registers. The ESP32-P4's SPI controller has registers for clock divisors, transfer length and DMA descriptors; the ST7365P has registers (it calls them

commands) for its window, pixel format and sleep state. A driver for the panel speaks to the second layer through the first. In PicoCalc OS the first layer belongs to the nucleus (on the host build, its place is taken by direct calls into the simulated controllers), and the second layer belongs to the JavaScript driver, which composes command bytes and parameters and hands them to the nucleus as transfers.

Figure 2.2 shows the layers a single SPI command passes through, on the host build today and on the device as planned.


display.js:
s.spi.cmd(CASET, bytes)
prelude: __ka('spi.cmd',
handle, ...)
nucleus: check the handle,
defer the transfer to the
segment's end
boarc
device, planned
host build
ESP-IDF spi_master: SPI2
registers and DMA
sim/st7365p.c:
st7365p_cmd and
wire: SCK, MOSI, CS, D/C
frame memory mode

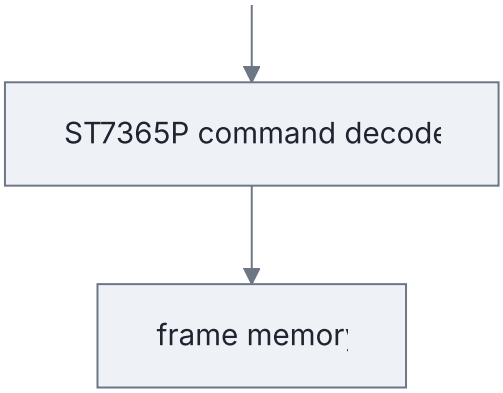


Figure 2.2. One command to the panel, from the JavaScript driver to the controller. The world never touches a register; the nucleus turns the transfer into a model call on the host, and will turn it into an ESP-IDF transaction on the device.

2.5 Serial buses: I2C and SPI

Both of the PicoCalc's peripherals sit on serial buses, which send one bit at a time on a clock line. Two protocols dominate small systems, and they make opposite trade-offs.

I2C uses two wires, a clock (SCL) and a data line (SDA), both open-drain with pull-up resistors: any device can pull a line low, and a line is high only when every device lets go. That wired-AND arrangement lets several devices share the two wires without contention. A transaction begins with a START condition (SDA falling while SCL is high) and ends with a STOP (SDA rising while SCL is high). After the START, the **controller** (the specification's current term for what used to be called the master) sends a 7-bit **target** address and a direction bit; then bytes follow, each of eight data bits and a ninth clock on which the receiver pulls SDA low to acknowledge. Every byte therefore costs nine clocks. A target that needs time may hold SCL low after a byte, stretching the clock, and the controller must wait. The specification defines Standard-mode at up to 100 kbit/s, Fast-mode at up to 400 kbit/s, Fast-mode Plus at up to 1 Mbit/s and High-speed mode at up to 3.4 Mbit/s [NXP 2021]. Nothing stops a controller from running slower than 100 kHz, and the PicoCalc's keyboard controller is run at 10 kHz.

SPI uses a clock (SCK), a line from controller to device (MOSI), a line back (MISO), and a chip-select line per device (CS). There are no addresses and no acknowledgements: the controller selects a device by pulling its CS low and clocks one bit out and one bit in per clock cycle (shifted on one edge, sampled on the other), eight clocks per byte. Displays usually add a fifth line, data/command (D/C), which tells the device

whether the byte on the wire is a command or a parameter. Because the lines are push-pull rather than open-drain, SPI runs at tens of megahertz where I2C runs at hundreds of kilohertz. The cost is wires: every device needs its own chip select, and there is no standard for how a device frames its commands.

The time a transaction takes follows from the bits it puts on the wire and the clock. For an I2C transaction that writes w bytes, optionally keeps the bus for a gap g, and then reads r bytes, each phase sends an address byte plus its data bytes at nine clocks each, plus about two clocks for the START and STOP:

$$t_{\mathrm{I2C}} = \left((1 + w) \cdot 9 + 2 + (1 + r) \cdot 9 + 2 \right) / f_{\mathrm{SCL}} + g \tag{2.1}$$

where a phase with no bytes is left out. An SPI transfer of n bytes takes $8n / f_{\mathrm{SCK}}$, plus whatever the controller spends toggling chip select and setting up DMA. The host build implements these formulas directly; they are the whole of its I2C and SPI timing model.

▸ history/first-pass/nucleus/hw.c

```
static vtime_t wire_us(double bits, int hz) { return (vtime_t)(bits * 1e6 / (hz > 0 ? hz
: 1) + 0.5); }
…
  // a write phase (address + bytes, 9 clocks each), an optional gap with the bus held,
then a read phase
  double bits = (d->nbytes ? (1 + d->nbytes) * 9 + 2 : 0) + (d->nread ? (1 + d->nread) *
9 + 2 : 0);
  vtime_t end = start + wire_us(bits, h ? h->hz : 10000) + d->delay;
…
  vtime_t end = start + wire_us((1 + d->nbytes) * 8.0, hz) + 2;   // + chip-select and
D/C toggling
```

Two consequences are worth stating before any measurement. First, the gap between the two phases is part of the transaction: while the controller holds the bus, no other transfer can use it, so the bus is busy for the whole time, not just for the bits. Second, at 10 kHz a single byte costs 0.9 ms. Reading two bytes from the keyboard controller costs more than sending a 96 × 66 rectangle of pixels at 40 MHz, which is 12,672 bytes. The two buses differ in speed by a factor of more than four thousand per byte.

TRY 2.1

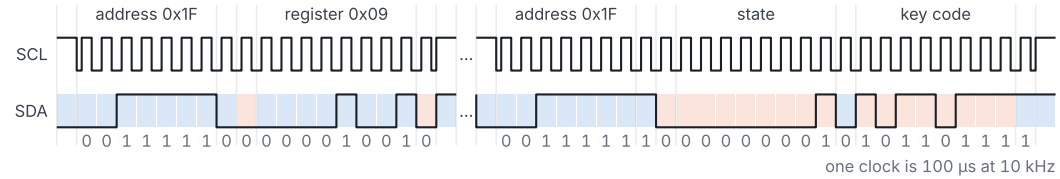
Time an I2C read and an SPI transfer

The first calculator is set to the keyboard driver's FIFO read: one byte written (the register number 0x09), two bytes read, at 10 kHz, with the bus held for 2 ms between the phases. Before reading its answer, predict the time from Equation 2.1; it should come to 6.90 ms. Then raise the clock to 100 kHz and 400 kHz and watch the bit-level diagram compress while the 2 ms gap stays. The second calculator sends 12,672 bytes at 40 MHz, the first rectangle of a launcher update, in 2.55 ms. Which bus would you speed up first to make the PicoCalc feel faster, and by how much would it help?

6.90 ms

`(1 + 1)·9 + 2 + (1 + 2)·9 + 2 = 49 clocks at 10 kHz = 4.90 ms, plus 2 ms held`

driven by the processor driven by the keyboard controller

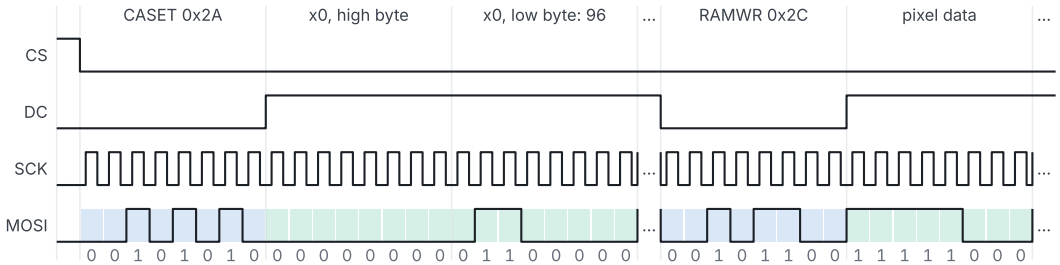


one clock is 100 µs at 10 kHz

2.55 ms

`3 commands 8.2 µs + 12,672 B × 8 / 40 MHz + 1 × 12 µs = 2.55 ms`

command byte data byte



one clock is 25.0 ns at 40 MHz

2.6 Reading a datasheet: the ST7365P

A datasheet is the contract between a chip and its drivers. It is also long (the ST7365P's runs to 287 pages), written for hardware engineers, and partly irrelevant to any given board. Reading one well means finding four things: how the chip is connected on this board, what state it is in after reset, what sequence takes it to a working state, and what timing that sequence must respect.

The ST7365P is a display controller with its own frame memory: 320 × 480 pixels of 18 bits each, 2,764,800 bits [Sitronix 2023]. It accepts 16-, 18- or 24-bit pixels over several interfaces: 8-, 9-, 16- and 18-bit parallel buses, an RGB interface, MIPI, and three- or four-wire SPI. The PicoCalc wires it for four-wire SPI, with the D/C line and a reset line, as the mainboard schematic shows [ClockworkPi n.d.]. Its glass is 320 × 320, so only the first 320 rows of the controller's memory are ever visible. The host build's model keeps the full 480 rows and shows the first 320, so a driver that writes past row 319 writes into memory nobody sees, exactly as on the device.

The controller wakes up asleep, with its display off, in a default pixel format and orientation. PicoCalc OS's display driver was transcribed into JavaScript from a C driver for the ESP32-P4 board, `picocalc_lcd.c`, which is not part of this repository or of clockworkpi's; this chapter calls it and its keyboard counterpart, `picocalc_keyboard.c`, the reference C drivers. The display driver takes the controller to a working state with six commands.

▸ js/drivers/display.js

```
  .onStart(async (s) => {
    s.spi = hw.spi("spi2", 40e6);   // 40 MHz: 80 MHz worked once but ghosted on the
removable display
    await s.spi.cmd(SWRESET);
    await sleep(150);
    await s.spi.cmd(SLPOUT);
    await sleep(150);
    await s.spi.cmd(COLMOD, [0x55]);  // 16 bits per pixel, RGB565
    await sleep(10);
    await s.spi.cmd(MADCTL, [0x48]);  // column order mirrored (MX), BGR panel
    await s.spi.cmd(INVON);           // this panel shows true colours with inversion on
    await sleep(10);
    await s.spi.cmd(DISPON);
    await sleep(50);
    s.ready = true;
  })
```

Table 2.3 gives each command's meaning and the datasheet's timing requirement next to what the driver does. The hard requirements are about power. After a software reset the controller reloads its factory defaults and must not be sent a command for 5 ms. If the reset was sent in sleep-in mode, as it is at power-up, sleep-out must wait 120 ms. After sleep-out its supply voltages and clocks need 5 ms to stabilise, and the controller must not be put back to sleep for 120 ms [Sitronix 2023]. The datasheet sets no minimum between sleep-out and display-on. The driver waits 150 ms after each of the first two commands and 370 ms in all, far more than these rules require, which is why the PicoCalc's screen stays dark for most of its first half second (Chapter 7 follows the whole boot).

Table 2.3. The commands the display driver sends, what they do, and the datasheet's timing rules. The driver's waits are longer than required, as the reference C driver's are.

Command	Code	Parameters	Effect	Required wait	Driver waits
SWRESET	0x01	none	reload factory defaults	5 ms before any command; 120 ms before sleep-out if sent while asleep	150 ms
SLPOUT	0x11	none	leave sleep, start power circuits	5 ms before any command; 120 ms before sleep-in	150 ms
COLMOD	0x3A	0x55	16 bits per pixel (RGB565)	none	10 ms
MADCTL	0x36	0x48	MX: mirror columns; BGR: colour order	none	0
INVON	0x21	none	invert colours (this panel needs it)	none	10 ms
DISPON	0x29	none	show the frame memory	none	50 ms
CASET, RASET	0x2A, 0x2B	4 bytes each	set the column and row window	none	0
RAMWR	0x2C	pixels	write pixels into the window	none	0

The last three commands are the ones used during normal operation. To update a rectangle, the driver sets the column window with CASET (start and end column, two bytes each), the row window with RASET, and then sends RAMWR followed by the pixels, which the controller writes left to right, top to bottom, wrapping at the window's right edge. The window is what makes partial updates cheap: a 96 × 66 rectangle costs 12,672 bytes of pixels plus 11 bytes of commands, not a whole frame.

MADCTL deserves a closer look because it is where boards differ. Its bits choose row order (MY), column order (MX), whether rows and columns are exchanged (MV) and whether the panel's subpixels are in RGB or BGR order. The right value depends on how the glass was mounted, and the datasheet cannot know that. The reference driver's value, 0x48, sets MX and BGR; the comment in the host build's model records what that implies about this panel's wiring, and the model applies the bits pixel by pixel:

▸ sim/st7365p.c

```
static void put_pixel(uint16_t v) {
  if (P.cy > P.r1) return;                          // past the window: the controller
ignores it
  int col = P.cx, row = P.cy;
  if (P.madctl & 0x20) { int s = col; col = row; row = s; }      // MV: exchange rows and
columns
  int x = (P.madctl & 0x40) ? col : (W - 1 - col);               // mirrored scan, MX
undoes it
  int y = (P.madctl & 0x80) ? (GRAM_H - 1 - row) : row;
…
  if (++P.cx > P.c1) { P.cx = P.c0; P.cy++; }
}
```

A model like this is useful exactly because it encodes the datasheet's rules, including the ones a driver can break. The host build's ST7365P model warns when a command arrives within 5 ms of a reset and when pixels arrive while the controller sleeps. It also warns, more strictly than the datasheet, when the display is turned on less than 120 ms after sleep-out. A capture of a broken driver shows the noise of uninitialised frame memory, as the glass would.

▸ sim/st7365p.c

```
void st7365p_cmd(uint8_t cmd, vtime_t t) {
  if (t - P.t_reset < MS(5)) warn(t, "command 0x%s arrived less than 5 ms after SWRESET",
(char[]){ "0123456789abcdef"[cmd >> 4], "0123456789abcdef"[cmd & 15], 0 });
…
    case 0x29:
      if (t - P.t_slpout < MS(120)) warn(t, "DISPON %s less than 120 ms after SLPOUT",
"sent");
```

One timing figure in the datasheet is not respected. The four-wire SPI interface's minimum serial clock cycle for writes is 66 ns [Sitronix 2023], which corresponds to about 15 MHz. The reference driver runs the bus at 40 MHz, more than twice that, and the panel works; at 80 MHz it worked once and then ghosted on the PicoCalc's removable display. This is common with display controllers, whose datasheets state limits guaranteed across temperature, voltage and process rather than what a typical part does on a short trace. It is also a warning. A system that runs a device beyond its datasheet is depending on margins nobody has promised, and the evidence that it works is the test on the hardware, not the document. Chapter 19 shows what the 40 MHz choice costs: a full frame takes 41.0 ms at 40 MHz against 20.6 ms at 80 MHz, but the interactive updates that dominate use are small, and moving the launcher's selection gets its first pixels only 2.6 ms sooner at the higher clock.

2.7 The keyboard controller: a computer on the bus

The PicoCalc's keyboard is not wired to the main processor. Its key matrix, the keyboard and screen backlights, and the battery gauge are all managed by an STM32F103R8T6 microcontroller, which runs firmware written with the Arduino framework and published in the PicoCalc repository [ClockworkPi n.d.]. To the ESP32-P4, the STM32 is an I2C target at address 0x1F that exposes a handful of registers. Table 2.4 lists the ones PicoCalc OS uses.

Table 2.4. The keyboard controller's registers that PicoCalc OS uses, from the firmware's reg.h. A write sets bit 7 of the register number.

Register	Number	Read returns	Used by
KEY	0x04	number of events in the FIFO (bits 0–4), Caps Lock (bit 5), Num Lock (bit 6)	keyboard driver, every 20 ms
FIF	0x09	the oldest event: state (1 down, 2 hold, 3 up) and key code	keyboard driver, once per event
BKL	0x05	the screen backlight level	power driver
BAT	0x0B	battery percentage, bit 7 set while charging	power driver, every 5 s

The firmware's I2C handling explains the protocol a driver must follow. When the controller writes a register number, the STM32's receive handler runs, decodes the register, and prepares a two-byte answer in a buffer; for the FIFO register, preparing the answer removes the oldest event from the FIFO. When the controller then reads, the request handler sends whatever the buffer holds. A simplified version of the firmware's two handlers, from `Code/picocalc_keyboard/picocalc_keyboard.ino` in clockworkpi's repository (not part of this repository), is:

```
void receiveEvent(int howMany) {          // the controller wrote bytes
  /* rcv_data[0] is the register number, bit 7 set for a write */
  const uint8_t reg = rcv_data[0] & ~WRITE_MASK;
  switch (reg) {
    case REG_ID_FIF: {                    // pops the FIFO now, not at the read
      const struct fifo_item item = fifo_dequeue();
      write_buffer[0] = item.state; write_buffer[1] = item.key;
    } break;
    case REG_ID_KEY: write_buffer[0] = fifo_count() /* | lock bits */; break;
    case REG_ID_BAT: write_buffer[0] = reg; write_buffer[1] = reg_get_value(REG_ID_BAT);
break;
    /* ... */
  }
}
void requestEvent() {                     // the controller reads
  Wire.write(write_buffer, write_buffer_len);
}
```

Three properties of this protocol shape the driver, and each is a general lesson about devices that are themselves computers.

The first is that a register read is two transactions with state in between. The answer to a read depends on the last register number written, by anyone. If two drivers share the chip (the keyboard driver reads the FIFO, the power driver reads the battery), and one's write is followed by the other's read, the reads get each other's answers. This happened in PicoCalc OS's first full boot in the simulator: the keyboard driver reported key `R` in state 11, and the battery read 0%. The keyboard driver had collected the battery register's answer: its first byte is the register number, 0x0B, which is 11, and its second is the battery level, 82 percent, and 82 is the ASCII code for `R`. The power driver's read then got an empty answer, because the simulator's model clears its prepared answer after each read. The firmware does not clear it, so on the device the power driver would have read the battery level a second time; the keyboard's lost event would be the same. The fix was to make the write, the wait and the read one bus transaction that holds the bus throughout, so nothing can come between them. clockworkpi's own example driver for the Raspberry Pi Pico (`Code/picocalc_helloworld/i2ckbd/i2ckbd.c`) shows the opposite choice: it ends the write with a STOP, which releases the bus, and only then waits. Chapter 13 tells the whole story; here the point is that the device's protocol, not the bus's, defined what had to be atomic.

The second is that the device needs time. The receive handler runs on the STM32 when the write completes, and the controller must not start the read until it has finished. The reference C driver waits 2 ms between the two phases, and PicoCalc OS's drivers do the same by holding the bus for 2 ms inside the transaction. clockworkpi's example driver waits 16 ms, at the same 10 kHz clock [ClockworkPi n.d.]. Neither figure is in any datasheet; both are empirical margins for an Arduino interrupt handler, and whether 2 ms is enough while the STM32 is busy scanning keys is something only the hardware can show. The margin is expensive either way: 2 ms costs more than all the bits of the status read at 100 kHz, and with clockworkpi's 16 ms the two reads of a key press would take 40.9 ms instead of 12.9.

The third is that the device has no way to ask for attention. The firmware keeps an interrupt-status register and sets bits in it when a key event arrives, but the code that would pulse an interrupt line to the host is commented out, so the host learns of a key press only by asking. PicoCalc OS's keyboard driver therefore polls, every 20 ms:

▸ history/first-pass/js/drivers/keyboard.js

```
const readReg = (dev, reg, n) => dev.read(reg, n, 2);
…
  .every(20, async (s) => {
    if (!s.dev) return;
    s.polls++;
    try {
      const [status] = await readReg(s.dev, REG_KEY, 1);
      let count = status & 0x1f;
      while (count-- > 0) {
        const [state, code] = await readReg(s.dev, REG_FIF, 2);
```

The battery register shows a related effect. The STM32 reads the battery percentage from the AXP2101 power-management chip over its own I2C bus, and updates its register every 20 seconds (the firmware's `check_pmu_int`), and otherwise only when the charger's state changes (a battery inserted or removed, charging started or finished). The power driver reads that register every 5 seconds, so unless the charger changes state, three reads in four return a value that cannot have changed. Polling faster than the source updates buys nothing, and it is a cost: every read holds the shared bus for 6.9 ms during which the keyboard cannot be polled.

The host build's model of the STM32 implements the same two-phase protocol, prepared answer and all, so that the interleaving bug could happen in the simulator as it would on the device (with the one difference about clearing the answer described above):

▸ sim/stm32kbd.c

```
static uint8_t answer[2];
int stm32_i2c(const uint8_t *wr, int nwr, uint8_t *rd, int nrd) {
  if (nwr >= 1) {
    const bool is_write = wr[0] & WRITE_MASK;
    const uint8_t reg = wr[0] & ~WRITE_MASK;
…
    answer[0] = answer[1] = 0;
    switch (reg) {
…
      case REG_FIF:
        if (S.count) { answer[0] = S.fifo[S.head].state; answer[1] = S.fifo[S.head].key;
S.head = (S.head + 1) % FIFO_SIZE; S.count--; }
        break;
      case REG_KEY: answer[0] = (uint8_t)S.count; break;
```

Figure 2.3 follows one status read through the layers, with the times the host build charges.

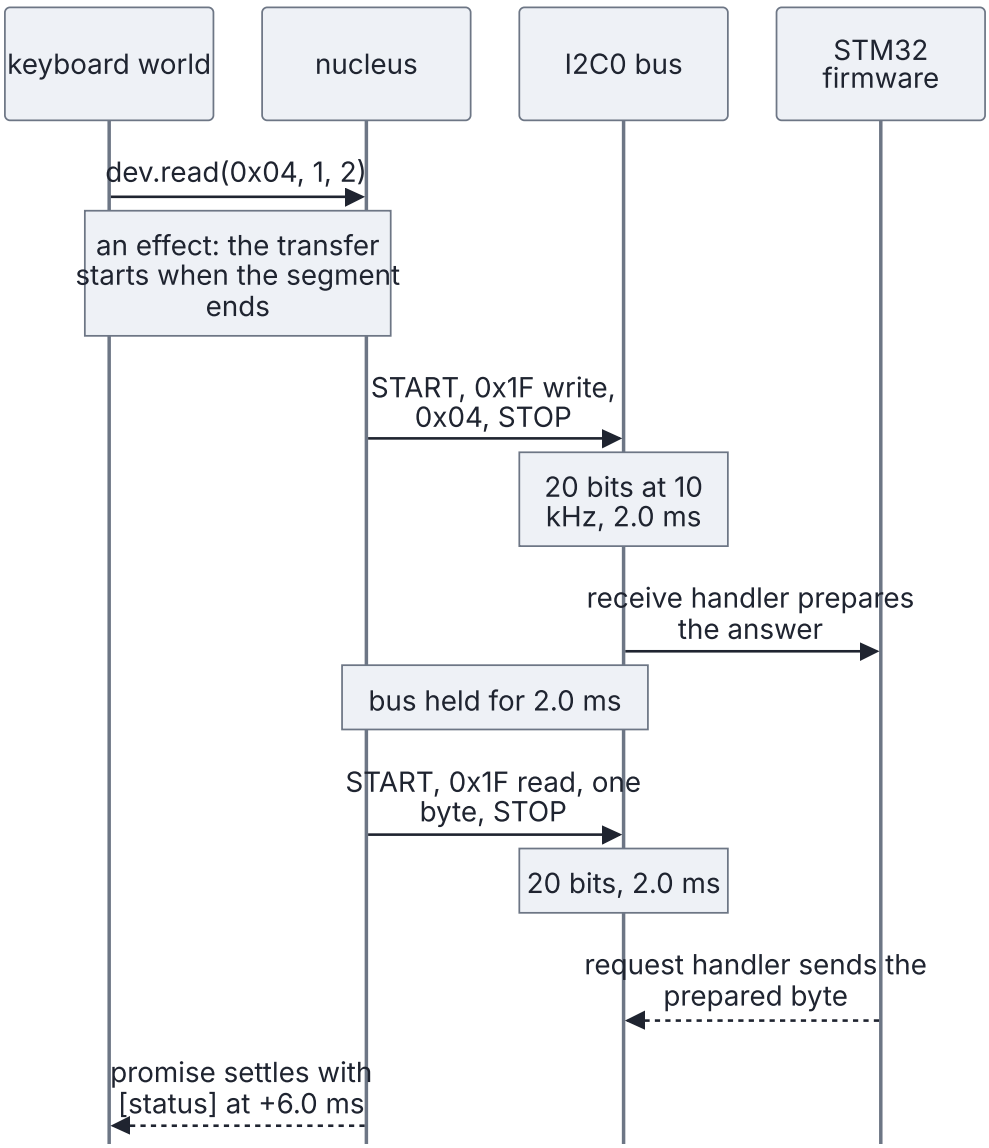


Figure 2.3. One status read of the keyboard controller at 10 kHz: a write of the register number, 2 ms with the bus held, and a one-byte read. The whole transaction holds the bus for 6.0 ms.

2.8 Time on the wire: anatomy of a key press

With the buses and both controllers understood, the time from a key press to the first changed pixels on the glass can be computed before it is measured. The path has five stages.

1. **The wait for the poll.** The STM32 puts the event in its FIFO at once, but nothing happens until the keyboard driver's next 20 ms tick. If key presses arrive at random times, the wait is uniformly distributed between 0 and 20 ms, with a mean of 10 ms.
2. **The status read.** Equation 2.1 with $w = 1$ and $r = 1$ gives 40 bits, 4.0 ms at 10 kHz, plus the 2 ms gap: 6.0 ms.
3. **The FIFO read.** With $w = 1$ and $r = 2$ it is 49 bits, 4.9 ms, plus 2 ms: 6.9 ms.
4. **The processor's work.** The keyboard driver publishes the event, the compositor routes it to the launcher, the launcher updates its state and publishes a new frame, the compositor lays the frame out, draws it, and finds the changed rectangles, and the display driver queues the transfers. On the virtual clock this costs about 2.5 ms.
5. **The first rectangle on SPI.** A command to set the window and the rectangle's pixels: for a 96 × 66 rectangle, 12,672 bytes, 2.53 ms at 40 MHz plus a 12 µs DMA chunk.

The fixed part, stages 2 to 4, is 6.0 + 6.9 + 2.5 = 15.4 ms, which the evaluation rounds to 15.5 ms once the short segments between the reads are included. Adding the first rectangle and the poll wait gives a predicted range of about 18 to 41 ms depending on where the press falls in the poll period and how large the first rectangle is, and a median near 28 to 31 ms.

The prediction can be checked against a trace. The host build records every segment, every bus transfer and every change of pixels on the simulated panel as a line of JSON. Here is the press closest to the median of 100 recorded presses, a right-arrow key in the launcher, with the lines of two other worlds' timers left out:

▸ bench/traces/keypress.jsonl


```
{"k":"key","t":16987260,"w":"","text":"0xb7 down"}
{"k":"seg","w":"keyboard","t":17000078,"dt":59,"steps":26,"native":2,"kind":"msg","type":
"every 20 ms"}
{"k":"bus","bus":"i2c0","w":"keyboard","t":17000137,"t1":17006137,"bytes":4,"what":"write
0x04, wait, read 1"}
{"k":"seg","w":"keyboard","t":17006137,"dt":50,"steps":14,"native":1,"kind":"settle","typ
e":"every 20 ms"}
{"k":"bus","bus":"i2c0","w":"keyboard","t":17006187,"t1":17013087,"bytes":5,"what":"write
0x09, wait, read 2"}
{"k":"seg","w":"keyboard","t":17013087,"dt":76,"steps":50,"native":3,"kind":"settle","typ
e":"every 20 ms"}
{"k":"seg","w":"compositor","t":17013163,"dt":80,"steps":56,"native":3,"kind":"msg","type
":"keys"}
{"k":"seg","w":"keys","t":17013243,"dt":67,"steps":38,"native":2,"kind":"msg","type":"key
s"}
{"k":"seg","w":"launcher","t":17013310,"dt":127,"steps":89,"native":28,"kind":"msg","type
":"key"}
{"k":"seg","w":"compositor","t":17013437,"dt":492,"steps":637,"native":29,"kind":"msg","t
ype":"frames"}
{"k":"seg","w":"compositor","t":17013929,"dt":1412,"steps":1204,"native":572,"kind":"msg"
,"type":"draw"}
{"k":"seg","w":"display","t":17015341,"dt":240,"steps":204,"native":65,"kind":"msg","type
":"flush"}
{"k":"bus","bus":"spi2","w":"display","t":17015581,"t1":17015584,"bytes":5,"what":"cmd
0x2a"}
{"k":"bus","bus":"spi2","w":"display","t":17015584,"t1":17015587,"bytes":5,"what":"cmd
0x2b"}
{"k":"bus","bus":"spi2","w":"display","t":17015587,"t1":17015589,"bytes":1,"what":"cmd
0x2c"}
{"k":"px","t":17015589,"t1":17018135,"changed":6336}
{"k":"bus","bus":"spi2","w":"display","t":17015589,"t1":17018135,"bytes":12672,"what":"pi
xels 96x66 at 216,62"}
```


Times are virtual microseconds since boot. Reading it against the five stages:

- The key goes down at 16,987,260 µs and the keyboard's next tick starts at 17,000,078 µs: a wait of 12.8 ms, plus 59 µs for the tick's own segment.
- The status read holds I2C0 from 17,000,137 to 17,006,137 µs, exactly 6.0 ms; a 50 µs segment handles its result and starts the FIFO read, which takes exactly 6.9 ms, to 17,013,087 µs.
- From there to the start of the pixels at 17,015,589 µs is 2.5 ms of processor work: the keyboard publishes, the compositor and the keys app (which also subscribes to `keys`) receive the event, the

compositor routes it to the launcher, which renders; the compositor lays out (492 µs) and draws and diffs (1,412 µs, of which 572 µs are native drawing); the display driver queues the transfers (240 µs). Each command on SPI takes 2 or 3 µs, almost all of it the 2 µs the model charges for chip select.

- The pixels of the first rectangle, 96 × 66 at (216, 62), take 2,546 µs, and the `px` line confirms that all 6,336 of them changed. The first pixels are on the glass at 17,018,135 µs, 30.9 ms after the press.

Table 2.5 summarises the stages. The first two rows, both on the keyboard side, are 83% of the total. The processor, which is where an engineer's instinct says to optimise, is 8%.

Table 2.5. The median key press in the launcher, stage by stage, on the host build's virtual clock.

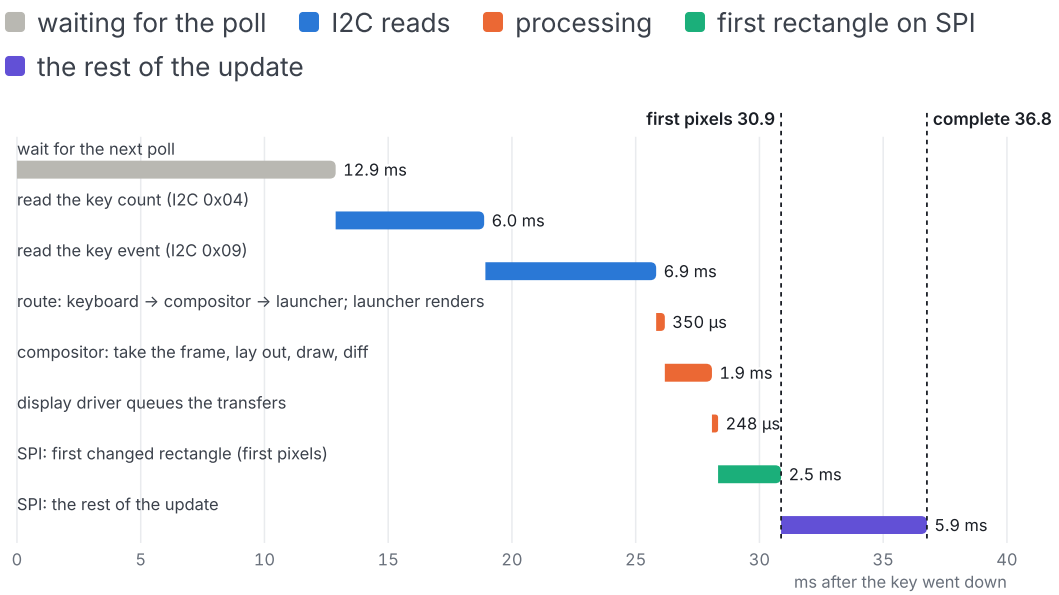
Stage	Time	Share	Set by
Wait for the next poll	12.9 ms	42%	the 20 ms poll period
Two register reads on I2C	12.9 ms	42%	10 kHz clock and the 2 ms gaps
Processor: route, render, draw, diff	2.5 ms	8%	interpreter steps and native work
First rectangle on SPI	2.5 ms	8%	40 MHz clock and the rectangle's size
First pixels	**30.9 ms**		

Across all 100 presses the same stages explain the whole distribution: the poll wait ranges from 0.14 to 19.9 ms, the fixed part stays near 15.5 ms, and the first rectangle is either 2.5 ms (96 × 66) or 5.3 ms (200 × 66, when the old and new selection share a row and one band covers both). The model predicts a range of 18.1 to 40.8 ms; the measured range is 18.8 to 40.7 ms.

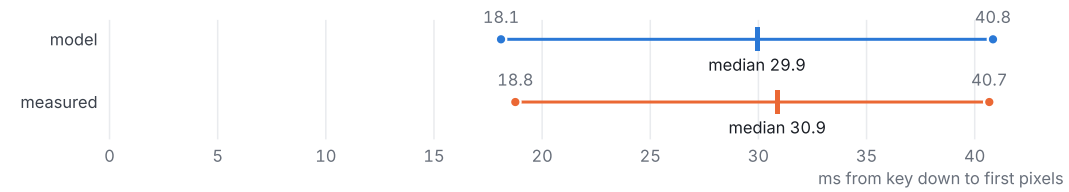
TRY 2.2

Change the hardware, predict the latency

The bars show one press's stages with the PicoCalc's settings: a 20 ms poll, a 10 kHz I2C bus and a 40 MHz SPI bus. The settings offer the values the benchmarks swept (I2C at 10, 100 and 400 kHz; polls of 5 to 80 ms; SPI at 10 to 80 MHz). Before changing one, decide which you expect to matter most. The bars are the measured press and stay fixed; the settings move the model's predicted range beside them. Raise the I2C clock to 100 kHz: the two reads shrink from 12.9 to 4.9 ms and the predicted median falls by about 8 ms. Going on to 400 kHz gains less than 1 ms more, because the 2 ms the STM32 needs to prepare its answer does not depend on the clock. Halve the poll period instead, and the worst case drops by 10 ms at the price of a busier bus. Raise the SPI clock to 80 MHz and the range moves only by the first rectangle's saving, 1.3 to 2.6 ms.



The model at other settings. Measured values appear where the benchmark ran those settings.



The model: wait 0–20 ms for the poll, 12.9 ms of I2C reads at 10 kHz, 2.7 ms of processing, then the first rectangle: 2.5 ms (33% of presses) or 5.3 ms at 40 MHz. On average the wait for the poll (half the period) and the I2C reads are about 76% of the model's median; the traced press above, which waited 12.9 ms for its poll, spent 83% of its time on that path. Measured: 18.8–40.7 ms, median 30.9 ms (n = 100).

The evaluation in Chapter 19 confirms the Try's lesson with full runs: at 100 kHz and 400 kHz, the status read still takes 2.4 and 2.1 ms, and the best measured configuration, a 5 ms poll at 400 kHz, brings the median from 30.9 ms to 13.7 ms with the bus 42% busy. The system monitor's bus page (Figure 2.4) shows the cost of the default settings on an idle system: polling alone keeps I2C0 about 30% busy, because each 6.0 ms status read repeats every 20 ms.

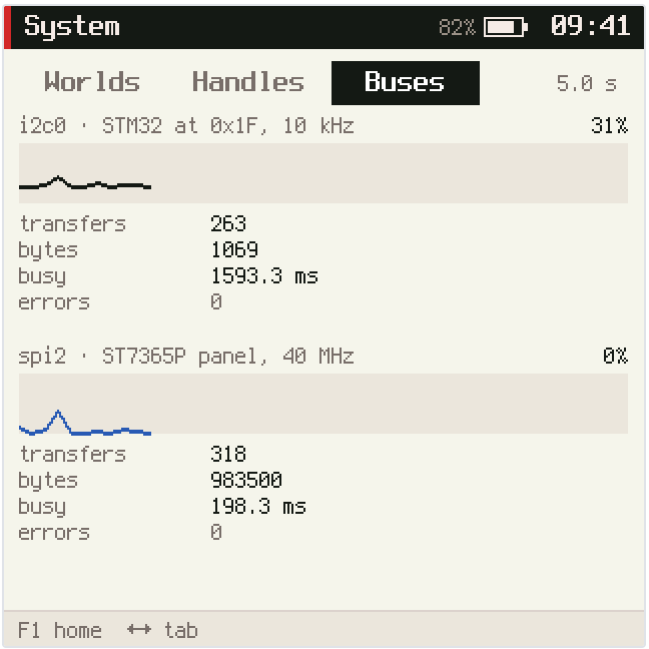


Figure 2.4. The system monitor's bus page on the host build, captured from the simulated panel at twice its size. With no key pressed, the keyboard driver's status reads keep the 10 kHz I2C bus about 30% busy; the SPI bus is idle between screen updates.

Two conclusions follow, and both recur in later chapters. The first is that an operating system's responsiveness on a small machine is often decided by its devices' protocols, not by its scheduler or its processor. No amount of optimising the compositor could recover more than 2.5 ms of the 30.9; a firmware change that raised an interrupt line on a key event would remove up to 20. The second is that a bus is a shared resource with its own schedule. While the power driver's battery read holds I2C0 for 6.9 ms, a key press waits for it, just as a world waits for the lane. Chapter 4 treats the lane as a scheduled resource; the buses are scheduled too, first come first served, and their queueing delays appear in the latency distribution as surely as the processor's.

KEY POINTS

- The ESP32-P4's HP cores have machine and user modes but no supervisor mode and no MMU, so protection between programs can come only from PMP regions or from the language; PicoCalc OS currently uses neither PMP nor user mode.
- The chip's 768 KB of on-chip SRAM cannot hold thirteen 190 KB world heaps and two 200 KB framebuffers, so on the device those must live in 32 MB of PSRAM behind the cache, and their access cost has to be measured, not derived.
- The PicoCalc's display and keyboard controllers are separate chips on serial buses: drivers speak to their command sets through bus transactions that the processor's own memory-mapped controllers perform.
- An I2C byte costs nine clocks and an SPI byte eight; at the PicoCalc's 10 kHz and 40 MHz, one keyboard register read takes longer than sending a 12,672-byte rectangle of pixels.
- The keyboard controller's register reads are two-phase, with the answer prepared on the write and returned on the read, so the write, a 2 ms wait and the read must form one transaction that no other driver can split.
- The keyboard controller has no interrupt line in its firmware, so the host must poll; the poll period sets up to 20 ms of every key press's latency.
- The datasheet's limits and a board's practice can differ: the ST7365P is specified for a 66 ns write clock cycle (about 15 MHz) and runs at 40 MHz on the PicoCalc, which only a test on the hardware can justify.
- A key press's 30.9 ms median decomposes into 12.9 ms of poll wait, 12.9 ms of I2C, 2.5 ms of processing and 2.5 ms of SPI, and the same model predicts the whole measured range.

The machine sets the terms; the operating system decides how to use them. The next chapter takes up the first decision every system makes about its processor: how to organise concurrent activity, from the interrupt handlers the hardware provides to the threads, event loops and isolated actors that systems build on top of them. Chapter 3 explains why PicoCalc OS runs each component as an event loop that handles one message at a time, and what that choice costs.

EXERCISES

1. BASIC Using Equation 2.1, compute the time of the keyboard driver's status read and FIFO read at 10 kHz, 100 kHz and 400 kHz, each with the 2 ms gap. At which clock does the gap become more than half of the total?

2. BASIC How many bytes does a full 320 × 320 frame occupy in RGB565, and how long does it take to send at 10, 20, 40 and 80 MHz, counting 12 µs for each 32 KiB DMA chunk? Compare your figures with the host build's: 163.9, 82.0, 41.0 and 20.6 ms.

3. BASIC The display driver sends MADCTL with the parameter 0x48. Which bits are set, and what does each mean according to Table 2.3 and the model's `put_pixel`? What would the screen show if the driver sent 0x08 instead?

4. ANALYSIS The poll wait is uniform between 0 and the poll period P. Write the expected value and the 95th percentile of the key-to-first-pixels latency as functions of P, assuming the other stages are constant at 15.5 + 2.5 ms. Compare your 95th percentile for P = 20 ms with the measured 39.9 ms and explain the difference.

5. ANALYSIS With the keyboard polled every 20 ms and the power driver reading the battery every 5 s, what fraction of the time is I2C0 busy on an idle system? Check your answer against Figure 2.4 and the system monitor in the host build.

6. ANALYSIS Suppose the STM32's firmware were changed to pulse an interrupt line on every key event, and the ESP32-P4 took that interrupt and started the status read at once. Estimate the new minimum, median and maximum latency for the launcher. Which stage would then dominate?

7. ANALYSIS The ST7365P's datasheet gives a minimum write clock cycle of 66 ns. Give three physical reasons why a part might work far beyond its specified limit on one board and fail on another, and describe a test you would run before shipping a product at 40 MHz.

8. CODE Run the host build with `./build/picocalc-host -` and the commands `wait 1000`, `panel` and `stats`. Explain every number in the `bus` lines from the drivers' code: how many transfers did each driver make in the first second, and why is SPI2's byte count larger than one frame?

9. CODE Change the power driver (`js/drivers/power.js`) to read the battery as two separate transfers, a write and then a read, without holding the bus. Run `scripts/tour.txt` and look for the symptom described in Section 2.7. Then restore the code and explain, using Figure 2.3, why holding the bus fixes it.

10. CODE The keyboard driver polls every 20 ms. Use the host build's `latency` command (see `bench/run.py` 's poll sweep for an example) to measure the launcher's median latency with polls of 10 and 40 ms, and compare with your answer to exercise 4.

11. DESIGN The power driver reads a register that changes at most every 20 s, every 5 s. Propose a polling schedule for both drivers that keeps the keyboard's worst-case latency under 35 ms and minimises I2C0's busy time. State your assumptions about the STM32.

12. PROJECT Write a model of a third I2C device (for example, a temperature sensor with a 16-bit register and a 10 ms conversion time) in the style of `sim/stm32kbd.c` , attach it at another address, and write a driver world that polls it. Measure the effect on keyboard latency when both share I2C0.

FURTHER READING

[NXP 2021] The I2C specification is short and readable; its sections on the bus's electrical arrangement, START and STOP conditions, acknowledgement and clock stretching are the background for every I2C driver.

[Waterman et al. 2021, §3.7] The definition of physical memory protection, including the locking rule that lets a machine-mode kernel protect itself.

[Sitronix 2023] A real display controller's datasheet: read the command list, the reset and sleep-out timing, and the serial interface characteristics, and compare them with `js/drivers/display.js` .

[ClockworkPi n.d.] The keyboard controller's firmware (`Code/picocalc_keyboard`) and the mainboard schematic; reading the firmware's I2C handlers is the quickest way to understand why the keyboard driver looks as it does.

[Espressif 2026b] The ESP32-P4 datasheet's chapters on the CPU, memory organisation and peripherals.

CHAPTER 3 PART I · FOUNDATIONS

Concurrency: threads, events and actors

A computer that does more than one thing at a time has to decide what "at the same time" means for its programs. The keyboard controller holds a key event, the display wants its next rectangle of pixels, the REPL is evaluating a loop someone typed, and a timer has just expired. The processor executes one instruction stream at a time, yet all four activities are in progress at once. **Concurrency** is the property of a system in which several activities have started and not yet finished, so that their steps interleave. It is a matter of structure, and it is present even on a single core; **parallelism**, steps that execute at the same instant on different processors, is a separate matter that this book mostly sets aside until Chapter 4.

The operating system cannot avoid concurrency, because the devices it manages run on their own schedules. What it can choose is how programs see it: where interleaving may happen, what state concurrent activities share, and how they hand work to one another. There are three classic answers. Preemptive threads share memory and may be interleaved between any two instructions. Event loops run one handler at a time to completion and interleave only between handlers. Actors keep their state private and interact only by sending messages. This chapter explains each model in its own terms, shows how each one fails, and then shows the model PicoCalc OS chose: every world is an isolated event loop, a turn handles one message, and a turn is made of segments whose effects take place only when the segment ends. By the end you should be able to read a trace of an asynchronous turn and say which of its effects a crash would have cancelled, and why a key pressed during a long computation waits where it does.

3.1 Interrupts: the first concurrency

The oldest source of concurrency in an operating system is the interrupt. An **interrupt** is a hardware signal that makes the processor stop the instruction stream it is executing, save enough state to resume it, and jump to an **interrupt service routine** (ISR). When the ISR returns, the interrupted code continues as if nothing had happened, except that memory may have changed underneath it. Nothing in the interrupted code marks where this can occur: an interrupt can arrive between any two instructions for which interrupts are enabled.

That property is enough to produce the central bug of concurrent programming. Suppose a keyboard ISR counts key events in a global variable, and the main loop consumes them. A sketch in C (not code from this repository):

```
volatile int pending;            /* written by the ISR and by the main loop */

void key_isr(void) { pending = pending + 1; }

void main_loop_step(void) {
  if (pending > 0) {
    handle_one_key();
    pending = pending - 1;       /* load, subtract, store: three instructions */
  }
}
```

`pending = pending - 1` compiles to a load, a subtraction and a store. If the ISR runs after the main loop has loaded `pending` (say 1) and before it stores the result (0), the ISR's increment to 2 is overwritten by the store of 0, and a key is lost. This is a **race condition**: the result depends on the relative timing of two activities that touch the same memory. The fix is to make the read–modify–write **atomic**, so that no other activity can observe or change the variable between its parts. On a single core the classic way is to disable interrupts around the update; the code between disabling and re-enabling is a **critical section**.

Disabling interrupts is a blunt instrument. While they are disabled, the system cannot respond to anything, so the critical section must be short, and an ISR must be short for the same reason: while it runs, everything below its priority waits. Operating systems therefore split interrupt handling in two. The ISR does the minimum (acknowledge the device, copy out a byte, note that work is pending) and defers the rest to code that runs later in an ordinary context: the bottom half or softirq in Linux, a deferred procedure call in Windows,

a task woken from the ISR in FreeRTOS (Chapter 7 describes ESP-IDF's version). Deferred work is the seed of every model in this chapter. Once the urgent part of an event has been recorded, the question becomes how the rest of the work is organised.

PicoCalc OS on the host build has no interrupts at all. The STM32 keyboard controller is polled over I2C every 20 ms, and every hardware event, a finished bus transfer, a timer, a message arriving, is an entry in the nucleus's event queue, a binary heap ordered by virtual time and then by a sequence number. The nucleus takes events from the heap only between segments of JavaScript, never in the middle of one:

▸ nucleus/kernel.c

```
void kernel_run_until(vtime_t until) {
  for (;;) {
    bool is_settle;
    world_t *R = pick(&is_settle);
    vtime_t t_lane = K.vnow > K.lane_free ? K.vnow : K.lane_free;
    vtime_t t_ev = heap_n ? heap[0]->t : VT_INF;
    if (R && t_lane <= until && t_lane < t_ev) { K.vnow = t_lane; run_segment(R,
is_settle); continue; }
    if (heap_n && t_ev <= until) { event_t *e = pop(); if (e->t > K.vnow) K.vnow = e->t;
handle_event(e); continue; }
    break;
  }
  if (K.vnow < until) K.vnow = until;
}
```

Every hardware event is therefore deferred work by construction. A bus transfer that finishes while a segment is running becomes an `E_SETTLE` event with the time at which its last bit left the wire; it is handled after the running segment ends, and it makes a world runnable. The urgent part (noting that the transfer finished and when) happens in the nucleus; the rest happens in a world, in its own turn. When the system moves to the ESP32-P4, the board layer will receive real interrupts from the SPI and I2C controllers, and they will do exactly this: record a completion in the queue and return.

The lesson of interrupts carries through the chapter. Concurrency becomes dangerous where two activities share mutable state and can interleave at a point neither controls. Each model below can be read as a different answer to the question of where interleaving is allowed and what is shared when it happens.

3.2 Threads and shared memory

A **thread** is an execution context, a program counter, a set of registers and a stack, that runs code in an address space it shares with other threads. A **preemptive** scheduler can stop a thread after any instruction and run another, typically when a timer interrupt fires. Threads are the dominant model in general-purpose operating systems and languages because they let a programmer write each activity as straight-line code ("read a request, compute, write the reply") and let the system overlap the waiting.

3.2.1 Races and atomicity

The ISR example generalises directly. Two threads that each execute `count = count + 1` a thousand times, without coordination, can finish with any value of `count` between 2 and 2,000. The 2,000 comes from no overlap at all. The value 2 needs a particular interleaving: thread A loads 0 and is preempted; thread B completes 999 increments; A stores 1, erasing them; B loads 1 for its last increment and is preempted; A completes its remaining 999 increments; B stores 2. Every value in between is reachable too. A test that runs the program a million times will usually print 2,000, which is why races survive testing: the bad interleavings are rare, not impossible.

The number of interleavings grows combinatorially. Two threads of n atomic steps each can interleave in $(2n)!/(n!)^2$ ways, which for $n = 10$ is already 184,756. A programmer reasoning about shared memory under preemption has to show that every one of them is acceptable, or must cut the number down by making groups of steps atomic.

3.2.2 Locks, monitors and condition variables

A **lock** (or mutex) makes a critical section atomic with respect to every other critical section guarded by the same lock: a thread acquires it before touching the shared state and releases it after, and a thread that tries to acquire a held lock waits. Locks scale the interrupt-disabling idea to many processors and to critical sections of any length, at the cost of a new obligation: every access to the shared state must hold the right lock, and nothing in C or JavaScript checks this.

Hoare's **monitor** packages the convention into a language construct [Hoare 1974]. A monitor is a module whose procedures run under an implicit lock, so its data can only be touched while the lock is held. Inside a monitor a thread may need to wait for a condition ("the buffer is not empty"); it does so on a **condition variable**, which atomically releases the monitor lock and suspends the thread until another thread signals the condition. Hoare specified that a signal hands the monitor directly to the waiting thread, so the condition is guaranteed true when the waiter resumes. The Mesa language at Xerox PARC adopted monitors for a

real operating system and changed that rule: a signal is only a hint, the signalled thread runs later, and it must re-check the condition in a loop [Lampson and Redell 1980]. Mesa semantics won. `pthread_cond_wait` is used inside `while (!condition)` for exactly this reason.

3.2.3 Deadlock

Locks introduce a failure that shared memory without locks does not have. A **deadlock** is a state in which a set of threads each wait for a resource held by another member of the set, so none can proceed. Coffman, Elphick and Shoshani identified four conditions that must all hold for a deadlock to occur [Coffman et al. 1971]:

- **Mutual exclusion:** a resource can be held by only one thread at a time.
- **Hold and wait:** a thread holds one resource while waiting for another.
- **No preemption:** a resource cannot be taken away from the thread holding it.
- **Circular wait:** there is a cycle of threads, each waiting for a resource held by the next.

Breaking any one condition prevents deadlock. The usual engineering rule breaks circular wait by acquiring locks in a single global order. Timeouts break no preemption, crudely: a thread that waits too long gives up its claim and reports an error. We will meet that remedy again in PicoCalc OS, where it was the only defence against a deadlock between two worlds until a later revision removed the circular wait itself (Section 3.6).

Lee's critique summarises why this model is hard to get right: threads start from a model in which everything can interleave with everything, and correctness is recovered by adding locks until enough nondeterminism has been removed; the programmer's job is to prune, and pruning is never verified to be complete [Lee 2006]. The remaining models in this chapter start from the other end. They begin with a program in which nothing interleaves and add concurrency at explicitly marked points.

3.3 Event loops and run-to-completion

An **event loop** is a single thread that repeatedly takes the next event from a queue and calls a handler for it. Each handler runs to completion before the next one starts. This **run-to-completion** rule is the whole point: two handlers in the same loop never interleave, so the state they share needs no lock. A handler is a critical section by construction.

```
loop forever:
    event = queue.take()          # waits if the queue is empty
    handler = handlers[event.type]
    handler(event)                # runs to completion; nothing else runs meanwhile
```

The price is that no handler may wait. A handler that blocks on a slow device stops every other handler in the loop, and a handler that computes for a long time delays every event behind it. This **head-of-line blocking** is the event loop's characteristic failure, and PicoCalc OS shows it in its purest form: in Section 3.8 a 133 ms loop typed at the REPL holds the entire system for 133 ms.

3.3.1 Callbacks and stack ripping

Because a handler cannot wait, any operation that takes time is split in two: the handler starts the operation and registers a **callback**, and the loop calls the callback as a new event when the operation finishes. A sequence that a thread would write as three lines ("read the status register, read the key, publish it") becomes three handlers, and the local variables that the thread kept on its stack must be moved into heap objects that survive from one handler to the next. Adya and colleagues called this **stack ripping** [Adya et al. 2002]: the logical sequence of a task is torn across functions, and its state is managed by hand. Callback-heavy code is hard to read, and error handling has to be repeated in every fragment.

3.3.2 Promises and async functions

JavaScript repairs stack ripping without giving up run-to-completion. A **promise** is an object that stands for a value not yet available; code attaches continuations to it with `then`, and when the promise settles (is resolved with a value or rejected with an error) the continuations are queued as **jobs** that the engine runs after the current handler finishes. An `async` function is a function whose body may contain `await`. At each `await` the function returns to its caller, and its remaining body becomes a continuation attached to the awaited promise. The compiler performs the ripping, so the programmer writes straight-line code again.

PicoCalc OS's keyboard driver shows the result. Every 20 ms it reads the controller's status register over I2C, then reads one FIFO entry per pending key, and publishes each event:

▸ history/first-pass/js/drivers/keyboard.js

```
  .every(20, async (s) => {
    if (!s.dev) return;
    s.polls++;
    try {
      const [status] = await readReg(s.dev, REG_KEY, 1);
      let count = status & 0x1f;
      while (count-- > 0) {
        const [state, code] = await readReg(s.dev, REG_FIF, 2);
        if (!state) break;
        …
        publish("keys", { code, key, ch, state: STATES[state] || String(state), caps: !!
(status & 0x20) });
      }
    } catch (e) {
```

Written with callbacks, this handler would be at least three functions plus an explicit loop counter kept in the world's state. Written with `await`, it reads like the C driver it was modelled on. But the reading is misleading in one respect that matters for everything that follows: the handler does not run as one piece. It runs as three pieces separated by the two I2C transfers, and between the pieces other code can run. Run-to-completion now applies to each piece, not to the function.

Where exactly the pieces break depends on what is awaited. An `await` on a promise that is already settled still yields, but its continuation is a job in the same job queue, and the engine drains that queue before returning to the loop. Only a promise that something outside the engine will settle later, a timer, a bus transfer, a reply from another component, ends the piece. This distinction is precise in PicoCalc OS, where the nucleus drains a world's job queue (`run_jobs` in `vm.c`) before it counts a segment as finished.

3.4 Threads, events and their duality

Are threads and events really different? Lauer and Needham argued in 1978 that they are two forms of one design [Lauer and Needham 1979]. They contrasted *procedure-oriented* systems, in which processes call into monitors and block on locks and condition variables, with *message-oriented* systems, in which processes own their data and communicate by sending messages and waiting for replies. They showed a mechanical mapping between the two: a monitor corresponds to a process that serves requests on a message channel; a procedure call into the monitor corresponds to sending a request and awaiting the reply; waiting

on a condition variable corresponds to deferring a request until a later message arrives. Under the mapping the two systems perform the same computations with the same amount of waiting, so neither structure is inherently faster; the choice should follow from what the hardware and the programmers handle better.

Table 3.1. Lauer and Needham's duality, with the forms each side takes in later systems.

Procedure-oriented (threads)	Message-oriented (events, actors)	In PicoCalc OS
Monitor with an implicit lock	Process that owns its data and serves one message at a time	A world
Procedure call into a monitor	Send a request, wait for the reply	`ask(to, type, payload)`
Procedure return	Reply message	The value an `on` handler returns
Wait on a condition variable	Keep the request queued until a later message makes it serviceable	A message waiting in the mailbox
Fork a thread	Send a message without waiting	`send` or `publish`
Lock contention	Queueing at the receiver	The world's mailbox and the lane

The duality did not end the argument. Ousterhout's 1996 talk "Why Threads Are A Bad Idea (for most purposes)" made the practical case for events: threads are hard to program correctly, hard to debug and hard to make fast because of locking, whereas an event loop has no races between handlers; threads are worth their cost only where true CPU parallelism is needed [Ousterhout 1996]. Von Behren, Condit and Brewer replied in 2003 with "Why Events Are A Bad Idea (for high-concurrency servers)": the problems Ousterhout attributed to threads came from poor implementations, while events force stack ripping and obscure control flow, and a thread package with compiler support can match event systems in performance [von Behren et al. 2003].

Both sides are right about different costs, and the debate becomes clearer if we ask one question of each model: at which points can another activity observe or change the state this code is using?

Table 3.2. Where interleaving can happen, by model.

Model	Interleaving points	Shared mutable state	Characteristic failures
Preemptive threads	Between any two instructions	The whole address space	Races, deadlock, priority inversion (Chapter 5)
Event loop with callbacks	Between handlers	Everything in the loop	Head-of-line blocking, stack ripping
Event loop with `async` / `await`	Between handlers and at every `await` of an outside promise	Everything in the loop	Head-of-line blocking, state changed across an `await`
Actors	Between messages	None (state is private)	Mailbox growth, deadlock by mutual waiting
PicoCalc OS worlds	Between turns (not at `await`)	None between worlds	Head-of-line blocking on the lane, a turn that never ends

The table shows the trade. Threads put interleaving everywhere and make the programmer remove it. Event loops put it at handler boundaries, which the programmer can see, and `async` functions add one more visible point, the `await`. The last two rows go further: they remove shared state altogether, which is the step the next section takes.

3.5 Actors and vats

Hewitt, Bishop and Steiger's **actor** model makes isolation the starting point [Hewitt et al. 1973]. An actor is an entity with private state that reacts to messages. In response to a message it can send messages to actors it knows, create new actors, and decide how it will respond to the next message. Actors share no memory. Each processes its messages one at a time, so its state, like an event loop's, needs no lock; unlike an event loop's, it cannot be touched by anyone else at all.

Two lines of systems turned the model into practice. Erlang gives each process its own heap and mailbox; processes are cheap enough to create by the thousand, communicate only by asynchronous messages, and are expected to crash and be restarted by a supervisor [Armstrong 2003]. That last idea is the subject of Chapter 14. The E language and its descendants organise a program into **vats**: a vat is a heap of objects plus one event loop, and a **turn** is the delivery of one message to one object in the vat, run to completion [Miller 2006]. Within a vat, objects call each other synchronously; between vats, they send messages that return

promises. When a promise resolves, the code waiting for it runs in a later turn. Between those two turns the vat goes on taking other messages, so an object's state can change between the turn that sent a request and the turn that handles the answer. Miller calls such unexpected changes *plan interference* and shows how promise-based code can be written to tolerate it.

The actor and vat models make data races impossible by construction, because there is no shared memory to race on. They do not make concurrency easy. Two actors can wait for each other's replies forever, which is deadlock in message form, and an actor whose state changes between sending a request and receiving its answer has the same reasoning problem as an `async` function whose state changes across an `await`. What the models change is the unit of reasoning: a programmer verifies one actor's handler against one message at a time, and treats every other actor as something that can only be reached by message.

3.6 Worlds, turns and segments

PicoCalc OS combines the event loop and the actor. Each component is a **world**: a separate QuickJS runtime with its own heap, its own job queue and a mailbox (Chapter 10 explains why a separate runtime and not a compartment). Worlds share no JavaScript objects. They interact only through the nucleus, by `send`, `ask` and `publish` (Chapter 12). Inside a world, code is written as handlers in the event-loop style, and they may be `async`.

Three definitions fix the vocabulary for the rest of the book.

> **DEFINITION: TURN**
>
> A **turn** is the handling of one message, one timer tick or the world's start, to completion. A synchronous handler's turn ends when the handler returns. An asynchronous handler's turn lasts until the promise it returned settles. While a turn is in progress, the world takes no other message from its mailbox.

> **DEFINITION: SEGMENT**
>
> A **segment** is one uninterrupted run of a world's code on the lane: the start of a turn, or the continuation that runs when a promise the turn is waiting for is settled by the nucleus. A segment ends when the world's job queue is empty. One segment runs at a time in the whole system.

> **DEFINITION: EFFECT**
>
> An **effect** is anything a segment asks the nucleus to do that is visible outside the world: deliver a message, start a bus transfer, set a timer, revoke a handle, answer an ask. Effects are collected while the segment runs and applied when it ends.

The prelude, the JavaScript every world runs before its own file, is where a handler's result decides whether the turn is synchronous or asynchronous:

▸ history/first-pass/js/prelude.js

```
  g.__dispatch = (kind, type, json, from, isAsk) => {
    const fn = kind === 0 ? def.on[type] : kind === 1 ? def.every[type] &&
def.every[type].fn : def.start;
    if (!fn) {
      if (isAsk) k("turn.reject", `${g.self} has no handler for "${type}"`);
      return undefined;
    }
    const r = fn(S, decode(json), { from, type, ask: isAsk });
    if (r && typeof r.then === "function") {
      k("turn.async");
      r.then((v) => { const [j, hs] = enc(v); k("turn.done", j, hs); }, (e) =>
k("turn.fail", errText(e)));
      return undefined;
    }
    if (!isAsk) return undefined;
    const [j, hs] = enc(r);
    if (hs.length) { k("turn.done", j, hs); return undefined; }
    return j;
  };
```

If the handler returns a promise, the prelude tells the nucleus that the turn is asynchronous (`turn.async`) and arranges for `turn.done` or `turn.fail` to be called when the promise settles. Until then the world is *waiting*, and the nucleus's choice of the next segment respects that. Here is the function that picks it:

▸ history/first-pass/nucleus/kernel.c

```
static world_t *pick(bool *is_settle) {
  world_t *best = NULL; vtime_t bt = 0; uint64_t bs = 0; bool bset = false; int bp = 0;
  for (int i = 0; i < K.n_worlds; i++) {
    world_t *W = K.worlds[i];
    vtime_t t; uint64_t s; bool set = false, ok = false;
    if (W->boot_ready) { t = W->boot_t; s = W->boot_seq; ok = true; }
    else if (W->status == W_IDLE) {
      if (W->settles) { t = W->settles->t; s = W->settles->seq; ok = true; set = true; }
      if (!W->in_turn && W->mbox && (!ok || W->mbox->t < t || (W->mbox->t == t && W-
>mbox->seq < s))) { t = W->mbox->t; s = W->mbox->seq; ok = true; set = false; }
    }
    int pr = W->boot_ready ? 10 : W->prio;   // starting a world comes first, in manifest
order
    if (ok && (!best || pr > bp || (pr == bp && (t < bt || (t == bt && s < bs))))) { best
= W; bt = t; bs = s; bset = set; bp = pr; }
  }
  *is_settle = bset;
  return best;
}
```

A world is runnable if it has a settle waiting (a promise of the current turn that the nucleus has resolved or rejected) or, only when it is not in a turn (`!W->in_turn`), a message in its mailbox. That one condition is the semantic decision of this section: **a world's mailbox is closed for the whole of an asynchronous turn**. Chapter 4 explains the rest of `pick` , the priorities and the tie-breaking by arrival.

Compare this with E. In a vat, the turn ends at the point where the code waits for a promise, and the vat takes other messages in the meantime. In a world, the turn spans every `await` , and other messages wait in the mailbox until it ends. The consequence for the programmer is that a world's state cannot be changed by another message between two `await` s of the same handler. Within a world, the reasoning is that of a monitor whose lock is held across the whole operation, including its waits. A handler can read `s.count` , await a bus transfer, and write `s.count + 1` without losing an update, which neither an E vat nor a browser's event loop guarantees.

The cost is symmetric. A world that awaits something slow is deaf to everything else for that long. The REPL, which awaits the sandbox's evaluation of whatever was typed, does not see keys while the evaluation runs; they wait in its mailbox (or, as the trace below shows, earlier still). A world whose handler awaits a promise that nothing will ever settle stays in its turn forever. The nucleus records when each turn started (`turn_started` in `world_t`), but it does not act on it: there is no watchdog on the length of an asynchronous turn, and Exercise 5 asks you to add one. (The first pass added it; see the note below.)

Closing the mailbox also creates a deadlock that E does not have. If world A asks world B while B, in its own turn, asks A, each request waits in a closed mailbox. Two of Coffman's conditions are present by design (each world holds its turn while waiting, and a turn cannot be taken from it); the circular wait is the programmer's mistake. The nucleus breaks the cycle in the only way it can, by preemption of the wait: every `ask` has a timeout, 2,000 ms unless the caller gives another, after which its promise is rejected. The degenerate case, a world asking itself, would wait for its own turn to end and is refused at the call:

▸ nucleus/vm.c

```
    else if (T == W) r = fail(ctx, "a world cannot ask itself (it would wait for its own
turn to end)");
```

NOTE

The first pass, a later revision described in a separate book, *From L4 to Small Worlds* (Chapter 6), changed both behaviours described here. The manifest now lists whom each world may ask, and the boot refuses a manifest whose asks form a cycle and prints the cycle (`asks form a cycle: cyc_a → cyc_b → cyc_a …`), so the deadlock between A and B can no longer be configured. And a turn still unfinished `turn_ms` after its first segment started (8,000 ms unless the manifest says otherwise) is a crash, so a world that awaits a promise nothing will settle is restarted instead of staying in its turn forever.

Inside one world there is no shared-memory concurrency at all, and between worlds there is no shared memory. What remains to define is when one world's actions become visible to another. That is the job of effects.

3.7 Effects at the end of a segment

When a segment calls `send`, `publish`, `ask`, `sleep` or a bus operation, the host function does not act immediately. It builds a deferred effect and appends it to a list that belongs to the running segment:

▸ nucleus/kernel.c

```
void defer(deferred_t *d) {
  if (!K.cur) { d->run(d, apply_at >= 0 ? apply_at : K.vnow); free(d); return; }
  d->next = NULL;
  if (K.defer_tail) K.defer_tail->next = d; else K.defer_head = d;
  K.defer_tail = d;
}
static void apply_deferred(vtime_t at) {
  apply_at = at;
  while (K.defer_head) {
    deferred_t *d = K.defer_head;
    K.defer_head = d->next;
    if (!K.defer_head) K.defer_tail = NULL;
    d->run(d, at);
    free(d);
  }
  apply_at = -1;
}
static void discard_deferred(void) {
  while (K.defer_head) {
    deferred_t *d = K.defer_head;
    K.defer_head = d->next;
    free_msg(d->msg); free(d->bytes); free(d->json); free(d);
  }
  K.defer_tail = NULL;
}
```

When the segment finishes normally, `run_segment` computes its cost on the virtual clock (Chapter 18), sets the time at which the lane becomes free, and calls `apply_deferred` with that time. Every effect of the segment takes place at the instant the segment ends: messages are delivered with that timestamp, bus transfers start no earlier, timers count from it. When the segment fails, by throwing, running past its budget or running out of memory, the crash handler throws the list away instead:

▸ nucleus/kernel.c

```
static void crash_at(world_t *W, const char *why) {
  if (K.cur == W) discard_deferred();
  …
}
```

There are two reasons for this design, one about failure and one about time.

The first is **atomicity under failure**. A segment either completes, and then all of its effects happen, or it fails, and none of them do. Its world's own state is discarded with its runtime when it crashes (Chapter 14), so no other world can ever observe the partial result of a failed segment. A handler that sends a message and then throws has sent nothing. Without this rule, every handler would have to be written so that each prefix of its effects left the system consistent, which is the reasoning burden that transactions exist to remove.

The second is **time**. A segment has a duration on the virtual clock, computed from its interpreter steps and native work. If a message sent halfway through a 133 ms computation were delivered immediately, its receiver could run, in virtual time, before the sender had finished, on a lane that by definition runs one segment at a time. Applying effects at the segment's end gives every effect a single, well-defined time, consistent with the lane: nothing a segment does is visible before the lane is free again. This also makes the system's history deterministic (Chapter 18): the order and the times of all deliveries follow from the sequence of segments.

The guarantee is per segment, not per turn. An asynchronous turn is several segments, and each one's effects are applied when it ends. If a handler sends a message, awaits a bus transfer and then throws, the message was delivered when the first segment ended, and the crash cannot recall it. Only the second segment's effects are discarded. Making whole turns atomic would require holding every effect until the turn ends, which would delay a driver's bus transfers (themselves effects) until after the transfers they were meant to start: an asynchronous turn could never make progress. The per-segment rule is the strongest one compatible with turns that wait for the outside world.

TRY 3.1

Step through an asynchronous turn

The widget shows a handler that asks another world, awaits the answer and replies, in PicoCalc OS's semantics. Step through it and watch three lists: the world's mailbox (closed while the turn is in progress), the effects collected by the running segment, and the effects already applied. Then inject a fault into the handler's second segment: the effects of the first segment have already been applied, the second segment's effects are discarded, and the world that asked receives an error instead of an answer. The "two messages" scenario shows a second message waiting in the mailbox until the asynchronous turn ends. (The widget's times are illustrative: they are computed from assumed step counts, not recorded.)

```
app("weather")
  .state({ temp: null })
  .on("refresh", async (S) => {
    send("log", "note", "refreshing");
    const t = await ask("sensor", "read");
    S.temp = t;
    publish("weather", { temp: t });
    return t;
  });
```

Segment 2 ends at 0.206 ms; its effects are applied and the turn is over. The mailbox opens.

world	segment	start ms	end ms	what
weather	1	0.000	0.073	refresh
sensor	1	0.073	0.136	read
weather	2	0.136	0.206	refresh (resumed)

applied at ms	effect
0.073	deliver note "refreshing" to log
0.073	deliver read (ask #1) to sensor
0.073	start the 2,000 ms timeout of ask #1
0.136	settle ask #1 for weather: 21.5

applied at ms	effect
0.206	deliver weather {temp: 21.5} to each subscriber
0.206	reply 21.5 to launcher's ask

3.8 Reading a turn in a trace

The host build can record every segment, message and bus transfer as one line of JSON (Appendix B). The trace `bench/traces/repl-loop.jsonl` was recorded while someone pressed Enter in the REPL on the line `let n = 0; for (let i = 0; i < 1e5; i++) n += i; n`, a loop that costs 133 ms on the ESP32-P4 and therefore on the virtual clock, and then pressed `x` while the loop was running. Times are in virtual microseconds since boot; `dt` is a segment's duration.

▸ bench/traces/repl-loop.jsonl

```
{"k":"seg","w":"keyboard","t":6280078,"dt":59,"steps":26,"native":2,"kind":"msg","type":"
every 20 ms"}
{"k":"bus","bus":"i2c0","w":"keyboard","t":6280137,"t1":6286137,"bytes":4,"what":"write
0x04, wait, read 1"}
{"k":"seg","w":"keyboard","t":6286137,"dt":50,"steps":14,"native":1,"kind":"settle","type
":"every 20 ms"}
{"k":"bus","bus":"i2c0","w":"keyboard","t":6286187,"t1":6293087,"bytes":5,"what":"write
0x09, wait, read 2"}
{"k":"seg","w":"keyboard","t":6293087,"dt":75,"steps":49,"native":3,"kind":"settle","type
":"every 20 ms"}
{"k":"msg","t":6293162,"from":"keyboard","to":"compositor","type":"keys","ask":false,"byt
es":61}
{"k":"seg","w":"compositor","t":6293162,"dt":80,"steps":56,"native":3,"kind":"msg","type"
:"keys"}
{"k":"msg","t":6293242,"from":"compositor","to":"repl","type":"key","ask":false,"bytes":6
1}
{"k":"seg","w":"repl","t":6293309,"dt":118,"steps":98,"native":13,"kind":"msg","type":"ke
y"}
{"k":"msg","t":6293427,"from":"repl","to":"sandbox","type":"eval","ask":true,"bytes":61}
{"k":"msg","t":6293427,"from":"repl","to":"compositor","type":"frames","ask":false,"bytes
":434}
…
{"k":"seg","w":"sandbox","t":6294909,"dt":133084,"steps":200064,"native":3,"kind":"msg","
type":"eval"}
…
{"k":"seg","w":"repl","t":6428752,"dt":108,"steps":83,"native":13,"kind":"settle","type":
"key"}
{"k":"msg","t":6428860,"from":"repl","to":"compositor","type":"frames","ask":false,"bytes
":485}
```

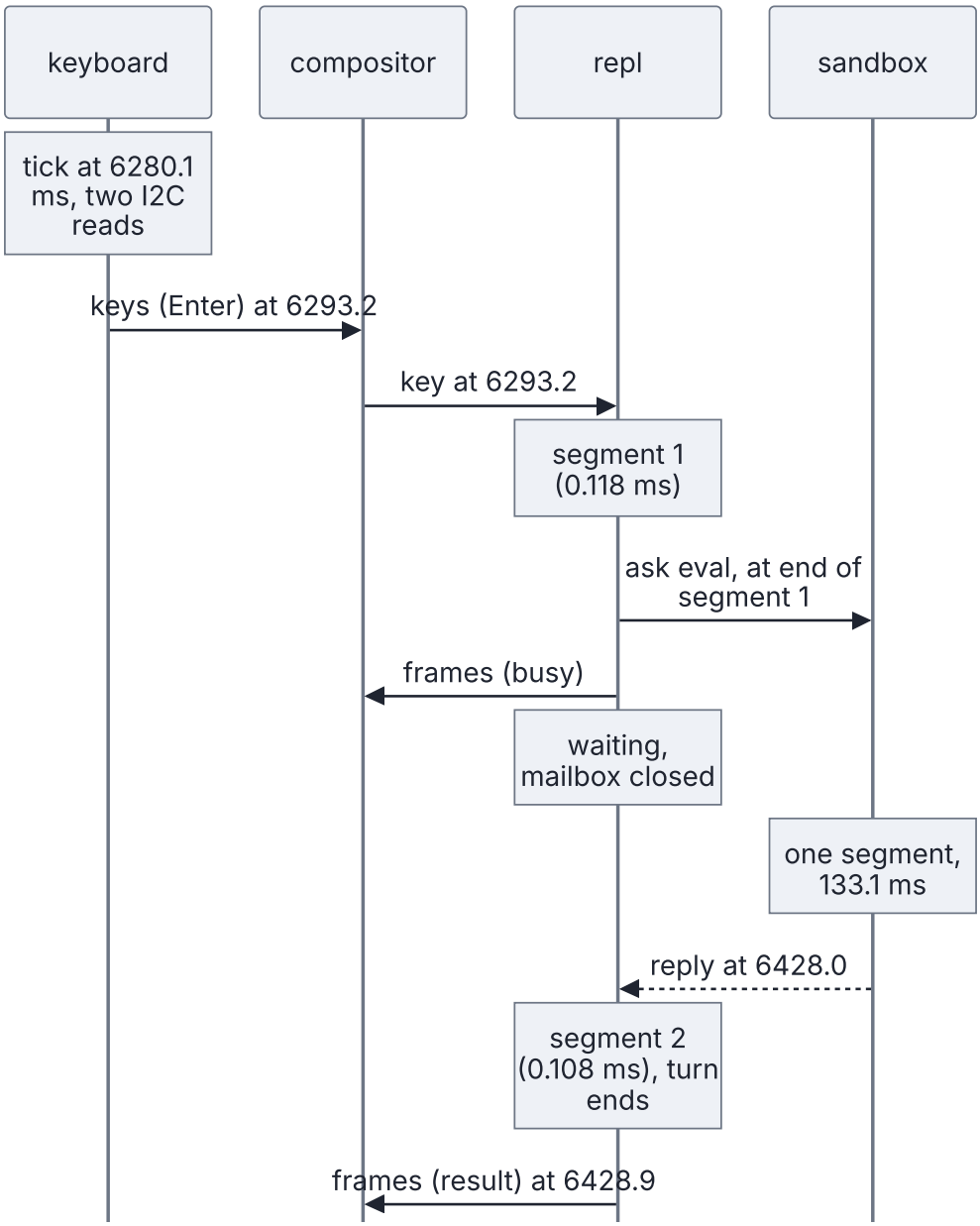
Read with the definitions of the last two sections, the excerpt contains three turns of interest.

The keyboard driver's poll is one turn of three segments. The first (59 µs, at 6,280.078 ms) starts the status read; its effect, the I2C transfer, begins when the segment ends at 6,280.137 ms and occupies the bus for 6.0 ms. The settle that ends the transfer makes the second segment runnable (50 µs), which starts the FIFO read, another 6.9 ms on the 10 kHz bus. The third segment (75 µs) publishes the key. The two messages it produces carry the time 6,293.162 ms, which is when the third segment ended, not when `publish` was called inside it.

The REPL's turn starts at 6,293.309 ms with the `key` message for Enter. Its first segment (118 µs) asks the sandbox to evaluate the line and, because the REPL's screen now says it is busy, publishes a new frame. Both effects are applied at 6,293.427 ms, together. Then the REPL waits. The sandbox runs at priority 0, below everything else, so it starts only at 6,294.909 ms, after the compositor has drawn the frame and the display driver has started sending it. Its single segment lasts 133.084 ms: 200,064 interpreter steps at 665 ns each, plus 40 µs to enter and leave the world and 3 µs of native work.

While the sandbox runs, nothing else can. Enter is released at 6,310 ms and `x` is pressed at 6,350 ms; both events sit in the STM32's FIFO, because the keyboard driver's tick due at 6,300 ms cannot run until the lane is free. When the sandbox finishes at 6,427.993 ms, its reply is an effect of its segment, and the REPL's second segment runs at 6,428.752 ms, after the display driver's queued completions, a keyboard poll and the snake game's tick, all of which became runnable earlier or have higher priority. The REPL's second segment prints the result and publishes a new frame at 6,428.860 ms. Its turn has lasted 135.6 ms of virtual time, of which its own segments used 0.226 ms.

The `x` reaches the REPL as a `key` message at 6,448.584 ms and its turn begins at 6,448.651 ms, 98.7 ms after the key was pressed. Most of that delay is head-of-line blocking: the lane was held by one segment that could not be interrupted (Chapter 4 discusses why it cannot) and whose length the sandbox's 200 ms budget bounds (Chapter 11). The rest is the keyboard driver reading the FIFO entries one at a time over the slow bus. No key was lost and no state was corrupted; the system was simply deaf for 133 ms.


keyboard
compositor
repl
sandbox
tick at 6280.1 ms, two I2C reads
keys (Enter) at 6293.2
key at 6293.2
segment 1 (0.118 ms)
ask eval, at end of segment 1
frames (busy)
waiting, mailbox closed
one segment, 133.1 ms
reply at 6428.0
segment 2 (0.108 ms), turn ends
frames (result) at 6428.9

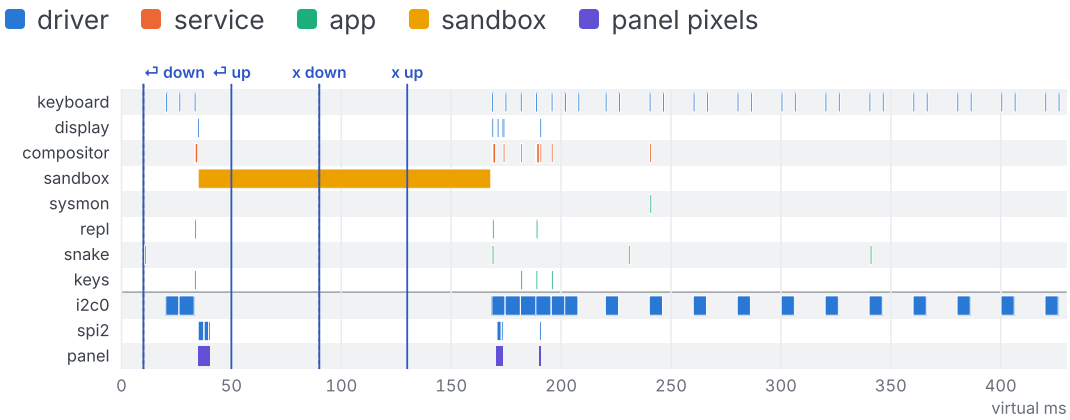
TRY 3.2

Find the segments of the REPL's turn

This is the same trace on a timeline: one row per world, then the I2C and SPI buses and the panel. Hover a bar for its segment or transfer; drag across the chart to zoom. Find the REPL's two segments and the sandbox's long one, the keyboard's tick that could not run while the sandbox held the lane, and the burst of display completions that ran as soon as the lane was free. Switch to the event list to read the same events in order.

A 133 ms loop in the sandbox 430 virtual ms

Enter is pressed in the REPL on a 100,000-iteration loop. The sandbox holds the lane for 133 ms: the keyboard's polls, and a key pressed meanwhile, wait until it ends.



3.9 Choosing a model

Each model in this chapter can be judged by the reasoning it asks of the programmer. With preemptive threads and shared memory, a programmer must consider every interleaving and remove the bad ones with locks; the system gives parallelism in return. With an event loop, the programmer considers interleaving only at handler boundaries and, with `async` functions, at awaits; the system gives up the ability to wait inside a handler. With actors, the programmer considers one actor and one message at a time, and treats everything else as a correspondent that can be slow, absent or wrong; the system gives up shared data structures.

PicoCalc OS's worlds take the most restrictive point of each axis that the hardware allows. They share nothing, which removes data races between components. They run one segment at a time, which removes races inside the nucleus. They keep the mailbox closed across awaits, which removes interference inside a world. And they apply effects at segment ends, which removes partial failures from what other worlds can see. Three costs remain, and the next chapters take them in turn. A long segment blocks everything (Chapter 4, Chapter 5). A segment must be bounded, or one world can stop the system (Chapter 11). A world that fails must be restarted, and the worlds that depended on it must cope (Chapter 14).

The restrictions are not free in expressiveness either. A world cannot overlap two independent conversations in the way a vat can. The compositor's `draw` handler paints the screen into its back buffer in its first segment and then awaits the display driver's `flush`; until the flush is done, new frames from apps and key events, which the compositor routes to the focused app, wait in its mailbox. The compositor limits the cost by coalescing redraws (a `draw` message is sent only if none is queued) and by sending only the changed rectangles, so a flush usually lasts a few milliseconds of SPI time (Chapter 13). Whether a different balance would be better is an open design question for a system of this kind, and Exercise 10 asks you to try one.

KEY POINTS

- Concurrency is the overlap of activities in time; its danger comes from shared mutable state combined with interleaving at points the programmer does not control.
- An interrupt is the first source of concurrency in an operating system, and deferring all but the urgent part of its handling is the pattern every later model builds on.
- Preemptive threads allow interleaving between any two instructions, so correctness depends on locks that the programmer must place everywhere they are needed; deadlock requires mutual exclusion, hold and wait, no preemption and circular wait.
- An event loop runs one handler at a time to completion, which removes races between handlers but makes any long handler block every other event.
- An `async` function is compiled into pieces that run as separate jobs; it restores straight-line code but adds an interleaving point at every `await` of an outside promise.
- Actors and vats share no memory and communicate only by messages, so data races are impossible, but mutual waiting can still deadlock and state can change between a request and its answer.
- In PicoCalc OS a turn handles one message to completion, a segment is one uninterrupted run on the lane, and the mailbox stays closed for the whole of an asynchronous turn, so no other message can change a world's state across an `await`.

- A segment's effects are collected while it runs and applied at the virtual time it ends, or discarded if it crashes; the guarantee is per segment, because whole-turn atomicity would stop a turn from starting the transfers it waits for.

The model gives each world a simple story: one message at a time, nothing shared, effects at the end of each segment. It leaves the system as a whole with a different question, which this chapter has only touched: when several worlds are runnable at once, which segment runs next, and what does it cost the others to wait? That is the subject of Chapter 4.

EXERCISES

1. BASIC Two threads each execute `count = count + 1` 1,000 times on a shared variable initialised to 0, with no lock. Give the largest and the smallest possible final value, and describe an interleaving that produces the smallest.

2. BASIC In the trace of Section 3.8, list every segment of the REPL's turn for Enter, with its start time and duration, and the time at which each segment's effects became visible to other worlds.

3. BASIC Classify each of these as an effect (applied at the segment's end) or as immediate in PicoCalc OS: `send`, `publish`, `ask`, `sleep`, an I2C `read`, drawing into a framebuffer with `fill`, `log`. Check your answers against `nucleus/vm.c` and `nucleus/hw.c`.

4. ANALYSIS A world's handler reads `s.n`, awaits `sleep(10)` and stores `s.n + 1`. Two messages for this handler arrive 1 ms apart. Show that PicoCalc OS ends with `s.n` increased by 2. Then show how an E-style vat, whose mailbox is open across the wait, could end with an increase of 1.

5. CODE Write a world whose `on("hang")` handler awaits `new Promise(() => {})`. Send it a message, then a second one, and use `stats` in the host build to show the world's state and its mailbox. Add a watchdog to the nucleus that fails a turn whose age (now minus `turn_started`) exceeds a limit given in the manifest, and decide whether a failed turn should crash the world.

6. ANALYSIS Worlds A and B each, in a handler for message `go`, `ask` the other for `"ping"`, and both receive `go` at the same virtual time. Describe what happens in PicoCalc OS, when each promise settles and with what, and which of Coffman's four conditions the ask timeout breaks.

7. ANALYSIS The REPL's key handler begins with `if (s.busy) return;`. Argue from the turn semantics of Section 3.6 that this test can never succeed in PicoCalc OS. Under which semantics would it be necessary?

8. BASIC Use Lauer and Needham's mapping to describe the keyboard driver as a procedure-oriented design: which thread runs, which monitor it calls, and where it blocks.

9. CODE Record a trace of typing `1+1` and Enter in the REPL with the host build's `trace FILE` command (see Appendix B). Identify the REPL's segments and the sandbox's segment, and compare the REPL's turn length with the 135.6 ms of Section 3.8.

10. PROJECT Add an option to `pick` in `nucleus/kernel.c` that keeps a world's mailbox open during an asynchronous turn, as in E. Find a world in `js/` whose behaviour changes under the option, and either fix it so that it is correct under both semantics or explain why it cannot be.

11. ANALYSIS Give two reasons why effects are applied at the virtual time a segment ends rather than when the host function is called, and describe one observable difference in a trace if they were applied immediately.

12. DISCUSSION Ousterhout argued that threads should be used only where true CPU parallelism is needed. The ESP32-P4 has two high-performance cores. What would PicoCalc OS have to give up to use the second core for worlds, and which of the guarantees in Section 3.7 would survive?

FURTHER READING

[Lauer and Needham 1979] The duality argument in its original form; the mapping table is worth checking against any system you know.

[Ousterhout 1996; von Behren et al. 2003] The two sides of the threads-versus-events debate, each short and each persuasive.

[Lee 2006] Why the thread model makes nondeterminism the default and how other models of computation start from determinism instead.

[Miller 2006] Chapters on concurrency control in E: vats, turns, eventual sends and plan interference, the closest relative of PicoCalc OS's worlds.

[Adya et al. 2002] Names stack ripping and shows how cooperative tasks and threads can be mixed in one program.

CHAPTER 4 PART I · FOUNDATIONS

Scheduling

Whenever more than one activity is ready to run and there is one processor, something must choose. A **scheduler** is the part of an operating system that decides which runnable activity uses the processor next, and for how long. Every choice serves some goals at the expense of others: finishing work quickly on average, answering the user promptly, sharing the machine fairly, or meeting deadlines. No policy is best for all of them, and the policy that a system needs follows from what it is for.

This chapter develops the classic policies on one small example that you can work by hand, then turns to the scheduler of PicoCalc OS: one lane, a handful of priority classes, arrival order within a class, and no preemption. That last property is unusual for an operating system and has a cost that the paper behind this book measured. By the end you should be able to compute response and turnaround times for the standard policies, explain why PicoCalc OS does not preempt a running segment, predict when its lane will starve a world, and read the measurements that show what a long segment does to everyone else. Chapter 5 then asks the stronger question of what can be *guaranteed*.

4.1 What a scheduler decides

The unit of work a scheduler handles is a **job**: something that becomes ready at an **arrival time**, needs the processor for some amount of **service time** (also called its burst), and leaves when that service is complete. For a thread, a job is one CPU burst between two waits; for PicoCalc OS, a job is one segment (Chapter 3). A scheduler acts at **dispatch points**: when a job arrives, when the running job finishes or waits, and, in a preemptive system, when a timer says the running job has had enough.

Four measures describe the result for each job:

- **Turnaround time** is completion time minus arrival time: how long the job was in the system.

- **Waiting time** is turnaround time minus service time: how long the job was ready but not running.
- **Response time** is the time of the job's first run minus its arrival time: how long before anything happened.
- **Throughput** is the number of jobs completed per unit of time, and **utilisation** is the fraction of time the processor is busy.

Turnaround matters for batch work, response time matters for anything a person is waiting on, and utilisation matters for anything that runs on a battery. Fairness, the idea that equal jobs should receive equal treatment and none should wait indefinitely, cuts across all four.

The running example of this chapter is three jobs on one processor. Times are in arbitrary units (think of them as milliseconds):

Table 4.1. The chapter's example: three jobs that arrive close together and differ in length.

Job	Arrival	Service
A	0	6
B	1	3
C	2	1

A long job arrives first, a medium one shortly after, and a short one last. Any reasonable policy finishes all three by time 10, because there are 10 units of work and the processor is never idle; what differs is who waits.

4.2 First come, first served and shortest job first

The simplest policy is **first come, first served** (FCFS): run jobs to completion in arrival order. A runs from 0 to 6, B from 6 to 9, C from 9 to 10. A's turnaround is 6, B's is 8 and C's is 8, a mean of 7.33. C, which needed one unit of work, waited seven. This is the **convoy effect**: short jobs queue behind a long one and inherit its length.

Shortest job first (SJF) runs the shortest available job when the processor becomes free. At time 0 only A has arrived, so A still runs to time 6; then C (1 unit) runs before B (3 units). Turnarounds become 6, 9 and 5, a mean of 6.67. The improvement comes entirely from reordering B and C, and it is the general rule: among

jobs waiting together, running the shortest first minimises mean waiting time, because each unit of a job's length is waited for by every job behind it. Moving a short job forward saves the long job less than it costs the short one to wait.

SJF still let A run for six units after B and C arrived. **Shortest remaining processing time** (SRPT) is the preemptive version: whenever a job arrives, the scheduler compares its service time with what the running job has left and runs the smaller. B arrives at 1 needing 3, while A still needs 5, so B preempts A. C arrives at 2 needing 1, while B needs 2, so C preempts B and finishes at 3. B finishes at 5, A at 10. Turnarounds are 10, 4 and 1, a mean of 5.0, and every job starts the moment it arrives. Schrage proved in 1968 that SRPT minimises the mean number of jobs in the system, and with it mean turnaround, for any arrival pattern on a single server [Schrage 1968].

SJF and SRPT have one serious problem: they need to know how long a job will take. Batch systems asked users for an estimate; interactive systems predict the next CPU burst from past ones, for example with an exponentially weighted average. A scheduler that cannot predict can still approximate SRPT by noticing which jobs have run for a long time and lowering them, which is the idea behind the feedback queues in Section 4.4.

4.3 Round robin

Round robin (RR) gives each ready job a fixed **time slice** (or quantum) q in turn. A job that uses its whole slice goes to the back of the queue; a job that finishes or waits leaves it. One rule must be fixed to make the policy precise: when a job arrives at the same instant another's slice expires, the arriving job joins the queue first. With $q = 2$: A runs from 0 to 2 (B has arrived at 1, C arrives at 2, then A rejoins behind them); B runs 2 to 4; C runs 4 to 5 and finishes; A runs 5 to 7; B runs 7 to 8 and finishes; A runs 8 to 10. Turnarounds are 10, 7 and 3, a mean of 6.67, the same as SJF. Response times are 0, 1 and 2, a mean of 1.0, far better than SJF's 3.33.

Figure 4.1 draws the four schedules, and Table 4.2 collects the numbers.

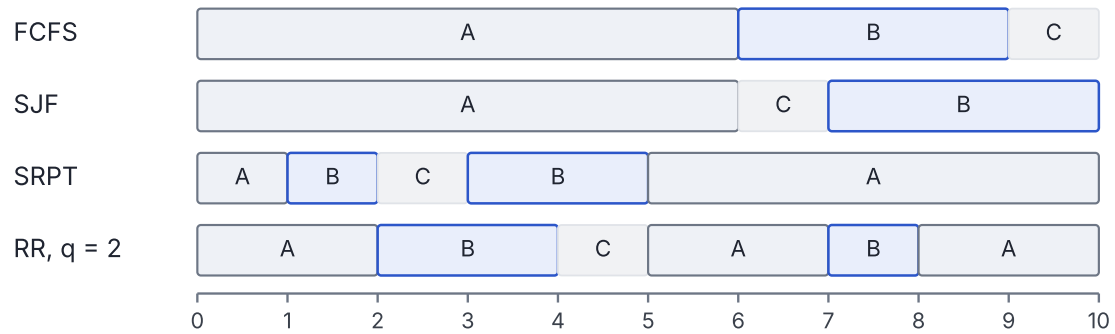


Figure 4.1. The three jobs of Table 4.1 under four policies. FCFS lets the long job delay both others; SJF reorders the two short ones; SRPT preempts on every arrival; round robin with a slice of 2 interleaves all three.

Table 4.2. Turnaround and response times for the jobs of Table 4.1.

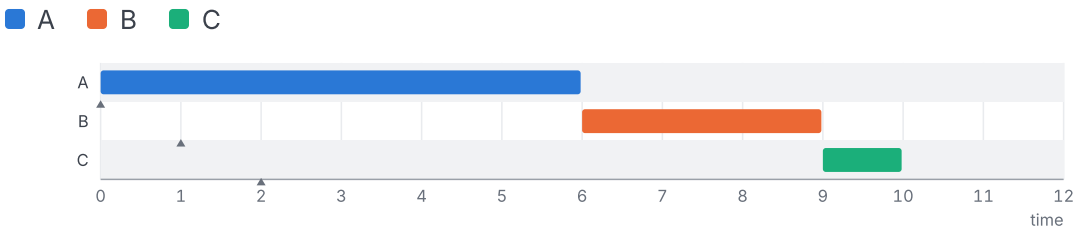
Policy	Turnaround A	B	C	Mean	Response A	B	C	Mean
FCFS	6	8	8	7.33	0	5	7	4.00
SJF	6	9	5	6.67	0	6	4	3.33
SRPT	10	4	1	5.00	0	0	0	0.00
RR, $q = 2$	10	7	3	6.67	0	1	2	1.00

Round robin's virtue is that it needs no knowledge of service times and still bounds how long any ready job waits for its next slice: with n jobs ready, at most $(n - 1)q$. Its parameter is a trade. As q grows without bound, RR becomes FCFS. As q shrinks, every job progresses at $1/n$ of the processor's speed, but each switch between jobs costs something (saving and restoring registers, cold caches), so a slice of 10 ms with a switch cost of 0.1 ms spends about 1% of the processor on switching, and a slice of 1 ms spends about 10%. General-purpose systems pick slices of a few to a few tens of milliseconds for this reason.

TRY 4.1

Schedule the three jobs yourself

The widget runs the jobs of Table 4.1 (each with a period longer than the horizon, so each runs once). It opens on FCFS. Switch to round robin and vary the slice: with a slice of 6 or more the chart is identical to FCFS, and with a slice of 1 (and the tie rule of Section 4.3) C finishes at time 4 instead of 5 and B starts the moment it arrives. Then change C's arrival to 0 and watch FCFS's mean turnaround change while round robin's barely moves.



U = 0.100. Liu–Layland bound for n = 3: 0.780. no misses in the simulation

Task	C	T	D	jobs	max R	mean R	misses
A	6	100	100	1	6	6	0
B	3	100	100	1	8	8	0
C	1	100	100	1	8	8	0

4.4 Priorities, feedback and fair shares

The policies so far treat jobs as equals. Real systems need to say that some activities matter more: an interrupt handler's deferred work more than a batch compile, the keyboard driver more than the snake game.

4.4.1 Static priorities and starvation

In **priority scheduling**, each job carries a priority and the scheduler always runs the highest-priority ready job. It can be preemptive (a newly arrived higher-priority job takes the processor at once) or non-preemptive (it waits for the running job to finish). Priorities express importance directly, but they introduce **starvation**: a low-priority job may wait forever if higher-priority work never runs out. The classic remedy is **aging**, raising a job's priority the longer it waits, so that every job eventually reaches the top.

4.4.2 The multilevel feedback queue

The **multilevel feedback queue** (MLFQ) uses priorities to approximate SRPT without knowing service times. It was first described for CTSS, the Compatible Time-Sharing System at MIT [Corbató et al. 1962], and the textbook formulation in Arpaci-Dusseau and Arpaci-Dusseau is a convenient statement of it [Arpaci-Dusseau and Arpaci-Dusseau 2018]:

1. If job X has higher priority than job Y, X runs.
2. Jobs of equal priority run in round robin.
3. A new job starts at the highest priority.
4. When a job has used up its time allotment at a level, however many times it gave up the processor along the way, its priority drops one level.
5. After some period, every job is moved back to the highest level.

A short interactive job finishes or waits before its allotment runs out and stays near the top, so it gets the quick response that SRPT would give it. A long computation sinks to the bottom and runs when nothing else needs the processor. Rule 4's accounting stops a job from gaming the scheduler by yielding just before its slice expires, and Rule 5 is aging: it prevents starvation and lets a job that has changed behaviour be reclassified.

4.4.3 Proportional share: lottery, stride and CFS

A different goal is to divide the processor in fixed proportions. **Lottery scheduling** gives each job a number of tickets and, at each slice, draws a ticket at random; the holder runs [Waldspurger and Weihl 1994]. A job with a quarter of the tickets receives a quarter of the slices on average, and the scheme composes well: tickets can be transferred to a server while it does work on a client's behalf, and groups can be given currencies. Its weakness is variance over short intervals, which the same authors' stride scheduling removes by replacing the lottery with a deterministic pass counter.

Linux took the deterministic route. From version 2.6.23 its general-purpose scheduler was the Completely Fair Scheduler (CFS), which, in the words of its documentation, "basically models an 'ideal, precise multi-tasking CPU' on real hardware" [Linux kernel documentation 2023]. Each task accumulates *virtual runtime*, the CPU time it has received, scaled by its weight; runnable tasks are kept in a red-black tree ordered by virtual runtime, and the scheduler runs the leftmost one, the task that has had least. Beginning with version 6.6 Linux has been moving to EEVDF, the earliest eligible virtual deadline first algorithm of Stoica and Abdel-Wahab [Stoica and Abdel-Wahab 1995; Linux kernel documentation 2025], which keeps CFS's accounting but gives each task a virtual deadline, so that latency-sensitive tasks with short requests can be served sooner without receiving more than their share.

These policies answer different questions, and Table 4.3 puts them side by side.

Table 4.3. The classic policies and what each one needs and gives.

Policy	Needs to know	Preempts	Optimises	Typical failure
FCFS	nothing	no	simplicity	convoys behind long jobs
SJF / SRPT	service times	SRPT only	mean turnaround	starves long jobs; estimates go wrong
Round robin	nothing	yes	response time, equal progress	switching overhead; poor turnaround
Static priority	priorities	either	importance	starvation, priority inversion (Chapter 5)
MLFQ	nothing (learns)	yes	response for short jobs	tuning; gaming without good accounting
Lottery, stride, CFS, EEVDF	weights	yes	proportional share	no notion of deadlines (CFS)

4.5 Preemption, cooperation and budgets

Most of these policies assume that the scheduler can take the processor away from a running job. That ability is **preemption**, and in a conventional kernel it rests on two mechanisms: a timer interrupt that returns control to the kernel, and a separate stack for every thread, so that the interrupted thread's state, including its call stack in the middle of whatever function it was in, can be set aside and resumed later. A **cooperative** scheduler lacks the first: a job runs until it yields, and a job that never yields keeps the processor. Classic Mac OS and Windows 3.1 worked this way, and so does an event loop: a handler is a job that always yields when it returns.

PicoCalc OS sits between the two in a way that follows from how its worlds run. A segment is a call from the nucleus into a QuickJS runtime: `JS_Call` on the world's dispatch function, or on a promise's resolve function for a settle. That call returns only when the JavaScript returns or throws, and all thirteen worlds share the nucleus's one C stack. To suspend a world in the middle of a segment and run another, the nucleus would need to keep the first world's C stack aside, which means a stack per world: a thread, or a FreeRTOS

task on the device. QuickJS lets a world use up to 192 KB of C stack (`JS_SetMaxStackSize` in `vm_create`), so a task per world would cost up to 192 KB per world for stacks that are idle almost all the time, and every host call would have to lock the nucleus's tables against other worlds' tasks.

What QuickJS does provide is an interrupt hook. The interpreter decrements a counter at every jump, branch and call, and when it reaches zero calls a handler the embedder installed. The handler can only say "continue" or "stop":

▸ third_party/quickjs/quickjs.c

```
static no_inline __exception int __js_poll_interrupts(JSContext *ctx)
{
    JSRuntime *rt = ctx->rt;
    ctx->interrupt_counter = JS_INTERRUPT_COUNTER_INIT;
    if (rt->interrupt_handler) {
        if (rt->interrupt_handler(rt, rt->interrupt_opaque)) {
            JS_ThrowInterrupted(ctx);
            return -1;
        }
    }
    return 0;
}
```

Stopping throws an uncatchable exception that unwinds the world's code back to the nucleus; there is no way to resume it. PicoCalc OS's nucleus uses the hook to enforce a **budget**: the maximum virtual time a segment may take, checked every 1,000 interpreter steps (0.665 virtual ms) and after every host call (Chapter 11 derives the bound on how far past its budget a segment can run). A segment that exceeds its budget is not suspended but killed, its effects discarded and its world restarted (Chapter 14).

The resulting discipline might be called **budget-enforced run to completion**. Every segment runs to completion like an event handler, except that completion has a deadline, and missing it is a fault. Table 4.4 compares the three disciplines.

Table 4.4. Three ways a scheduler can get the processor back.

Discipline	How control returns	Stack per activity	A job that runs forever
Preemptive	timer interrupt, at any instruction	yes	is descheduled and resumed later
Cooperative	the job yields or returns	for threads, yes; for handlers, no	holds the processor forever
Budget-enforced run to completion (PicoCalc OS)	the segment returns, or is aborted at its budget	no	is killed and its world restarted

The design has costs and benefits that the rest of this chapter makes concrete. It needs no per-world stack and no locks in the nucleus, it keeps the virtual clock exact (a segment's cost is a function of its own work, never of when it was interrupted), and it makes the system deterministic (Chapter 18). In exchange, a segment that is allowed to be long makes everything else wait.

4.6 The PicoCalc lane

PicoCalc OS has exactly one **lane**, the sequence of segments that the nucleus runs one after another. Chapter 3 showed the function that chooses the next segment, `pick` in `kernel.c`; here is what its comparison means as a policy.

4.6.1 Priority classes and arrival order

Every world has a priority. The manifest may set it; otherwise it comes from the world's kind when the world boots:

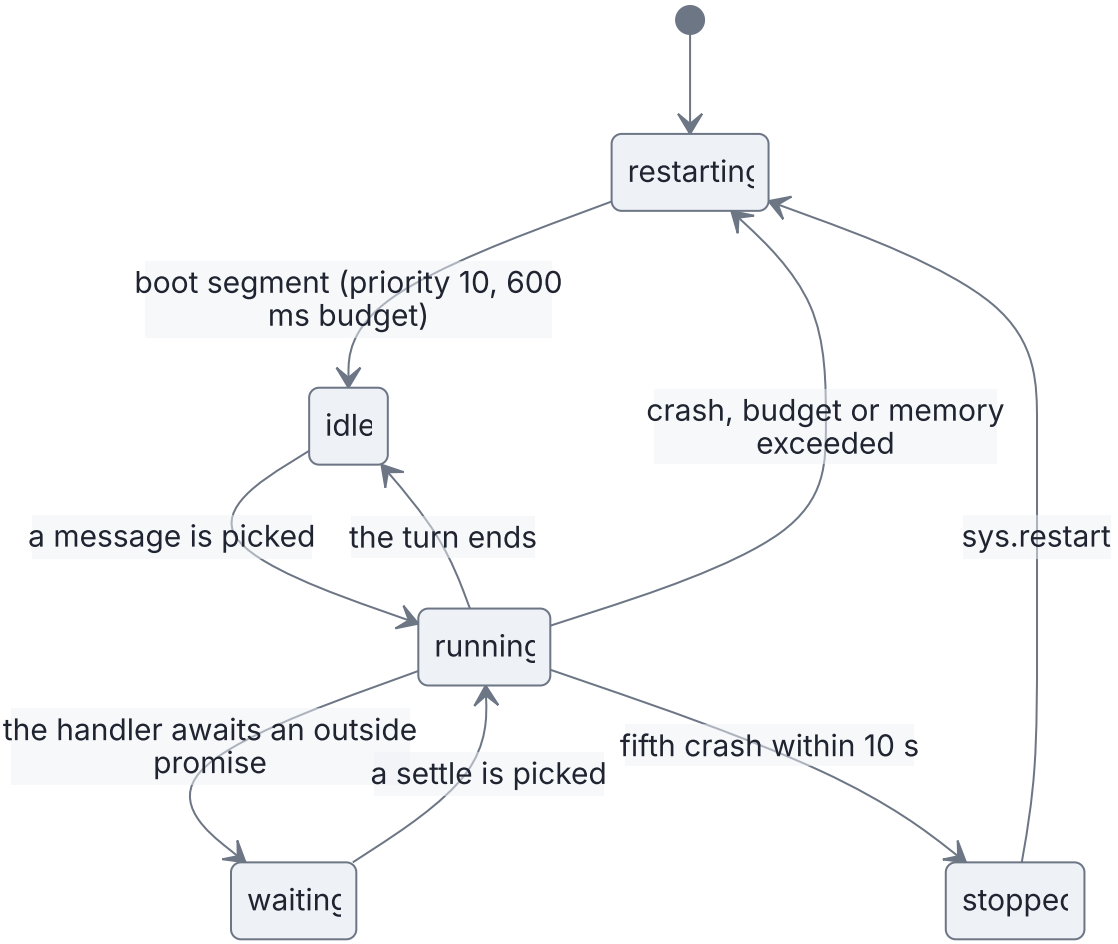
▸ history/first-pass/nucleus/kernel.c

```
  if (W->prio < 0) W->prio = !strcmp(W->kind, "driver") ? 3 : !strcmp(W->kind, "service")
? 2 : 1;
```

Drivers get 3, services 2 and apps 1. The sandbox, a service that evaluates whatever is typed at the REPL, is given 0 in the manifest so that it runs only when nothing else wants the lane. One more class sits above all of them: a world that is starting, or restarting after a crash, is treated as priority 10, so that its boot segment

(which evaluates its source file, with a fixed budget of 600 ms) runs before anything else.

Within a class, `pick` compares the virtual time at which each world's candidate work arrived: the time stamp of the message at the head of its mailbox, or of the settle it is waiting for, whichever is earlier. Ties are broken by a global sequence number. Within a priority class, therefore, the lane is first come, first served. Across classes it is non-preemptive priority scheduling. And the decision is made only when the lane is free, at the end of every segment.



The diagram shows the states a world passes through as the lane sees it; the host build's `stats` command prints them (as `idle`, `busy`, `waiting`, `restarting` and `stopped`). Only a world that is idle with mail, or waiting with a settle, is a candidate for the next segment. A waiting world with mail but no settle is not, because its mailbox is closed during its asynchronous turn.

4.6.2 Periodic timers that do not pile up

Worlds declare periodic work with `every(ms, fn)`. The keyboard driver polls every 20 ms, the power driver samples the battery every 5 s, the compositor checks its clock every 500 ms, the snake game ticks every 110 ms. Each timer is an event in the nucleus's queue, and when it fires it puts a tick message in the world's mailbox:

▸ nucleus/kernel.c

```
    case E_TICK:
      if (!W || W->epoch != e->epoch || W->status != W_IDLE) break;
      if (!W->tick_queued[e->a]) {
        msg_t *m = new_msg(M_TICK, "tick", NULL, -1);
        m->timer = e->a; m->t = e->t; m->seq = ++K.seq;
        if (W->mbox_tail) W->mbox_tail->next = m; else W->mbox = m;
        W->mbox_tail = m;
        W->mbox_len++;
        W->tick_queued[e->a] = true;
      }
      { event_t *n = new_event(E_TICK, e->t + MS(W->every_ms[e->a])); n->w = e->w; n-
>epoch = e->epoch; n->a = e->a; schedule(n); }
      break;
```

Two decisions are visible in these lines. The next tick is scheduled one period after this one was due, not one period after the handler ran, so ticks keep a fixed rate however late a handler runs. And a tick is enqueued only if the previous tick of the same timer has not yet been handled (`tick_queued`). A world that falls behind therefore loses ticks rather than accumulating a backlog that it would have to work through later. For a poll this is the right behaviour: two polls in a row would read the same FIFO. In Section 3.8 the keyboard's tick due at 6,300 ms waited in the mailbox while the sandbox held the lane, and the ticks due at 6,320, 6,340 ms and so on were dropped because it was still there. When the lane came free, one poll ran and read every key that had accumulated in the STM32's FIFO.

4.6.3 Budgets

The manifest gives each world a budget, and a world that does not declare one gets 50 ms (unless the manifest's `defaults` object sets another, which the shipped manifest does not):

▸ nucleus/kernel.c

```
  double d_budget = has_defs ? mnum(ctx, dv, "budget_ms", 50, "defaults") : 50;
…
    W->budget_us = (vtime_t)(mnum(ctx, o, "budget_ms", d_budget, where) * 1000);
```

In the manifest, the keyboard and power drivers have 20 ms, the display driver 30 ms, the compositor 80 ms and the sandbox 200 ms; every app, and the `flaky` test service, has the default 50 ms:

▸ history/first-pass/js/system.json

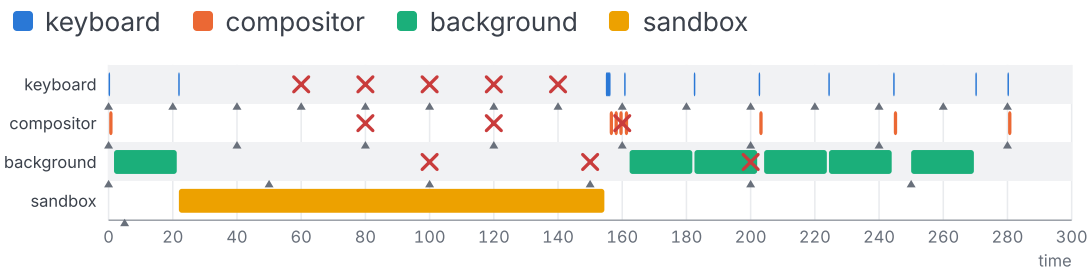
```
    { "name": "compositor", "file": "system/compositor.js", "claims": ["dma"],
"budget_ms": 80,  "mem_kb": 768,
    …
    { "name": "sandbox",    "file": "system/sandbox.js",
"budget_ms": 200, "mem_kb": 512, "priority": 0 },
```

A budget plays two roles in scheduling. It is the limit beyond which a segment is treated as a fault, which protects the system from a world that loops. It is also, because the lane does not preempt, an upper bound on how long any other world can be kept waiting by that world's segment. Chapter 5 turns the second role into an analysis.

TRY 4.2

Watch a long segment block the lane

The widget models the lane: four tasks with PicoCalc OS's priority classes, each segment running to completion once started. The keyboard driver is a short task every 20 ms; the compositor draws every 40 ms; a background app computes for 20 ms every 50 ms; and the sandbox runs one 133 ms evaluation. Look at the keyboard row. Its response time is a fraction of a millisecond until a lower-priority segment happens to be running when its tick arrives; then it waits for that segment to end: up to 20 ms behind the background app, and more than 100 ms behind the sandbox's evaluation, during which several of its periods pass. The widget marks each such period as a missed deadline (✕); the real lane skips those ticks instead of queueing them (Section 4.6.2), but the delay is the same. Change the algorithm to preemptive priority and the waits disappear.



U = 0.581. Liu–Layland bound for n = 4: 0.757. 11 deadline misses in the simulation

Task	C	T	D	jobs	max R	mean R	misses
keyboard	0.20	20	20	15	115.10	28.07	5
compositor	1.50	40	40	8	117.60	31.66	3
background	20	50	50	6	132.30	65.85	3
sandbox	133	1000	1000	1	149.90	149.90	0

4.7 What not preempting costs

The cost of a non-preemptive lane is **blocking**: a high-priority world that becomes ready while a lower-priority segment runs waits until that segment ends. Priority decides only among worlds that are *waiting together* when the lane comes free; it cannot shorten what is already running.

The paper measured this directly. A background app was added that computes for L ms every 50 ms, and the latency from a key press in the launcher to the first changed pixels on the panel was measured for 100 presses at each L (the measurement method is in Chapter 19). The experiment was run twice, with the background app at priority 1 and at priority 0.

Table 4.5. Key-to-pixel latency in the launcher as a background app takes more of the lane (host build, virtual ms, 100 presses each). The results at priority 0 were identical to these at every load.

Background work L every 50 ms	Lane busy	Median	95th percentile	Maximum
0 ms	2%	30.9	39.9	40.7
2 ms	6%	30.9	39.9	40.7
5 ms	12%	30.9	39.9	42.8
10 ms	22%	31.8	43.2	47.9
20 ms	42%	39.9	54.2	57.8
40 ms	82%	77.6	91.4	103.8

Three things in the table follow from the lane's design. First, small loads are invisible. A key's path through the system is a chain of short segments (three for the keyboard driver's poll, one for the compositor's routing, one for the app, the compositor's draw and the display driver's flush), separated by bus transfers during which the lane is free. With L = 2 ms, the chance that any of these segments arrives while the background app is running is small, and when it happens the wait is at most 2 ms.

Second, large loads add their whole length. At L = 40 ms the lane is busy 82% of the time, and nearly every key press finds a 40 ms segment in progress at some step of its path; even the fastest press took 54.6 ms, against 18.8 ms on an idle system. Several segments of the chain can each be blocked, which is how the maximum (103.8 ms) can exceed the idle maximum by more than one L.

Third, and most instructive, the background app's priority made no difference at all, to a tenth of a millisecond. Lowering it from 1 to 0 would matter only if the background app and some segment on the key's path were waiting for the lane at the same moment. But the background app is never waiting when the keyboard driver becomes ready; it is running, because its whole job is one long segment. What the keyboard driver waits for is the end of a segment that started before it arrived, and no priority can change that. Priorities order queues. They do not preempt.

The remedies follow from the diagnosis. A shorter budget for apps bounds the blocking any app can cause. A world with long work can split it into shorter segments by awaiting `sleep(0)` between pieces, which returns to the lane and lets anything of higher priority run. A second lane, on the ESP32-P4's second high-performance core, could carry the drivers and the compositor so that apps never block them (Section 4.9). Or the lane could preempt, at the price described in Section 4.5.

4.8 Starvation and the sandbox

The opposite failure belongs to the lowest class. The sandbox runs at priority 0, which means it gets the lane only when no other world is runnable. On an idle PicoCalc that is almost always: the whole lane is busy 0.72% of the time on the home screen and 12.1% with the snake game playing itself. But priority 0 is a promise of nothing, and it is easy to break.

To show it, we added to a copy of the manifest an app whose timer fires every millisecond and computes for 2 ms each time:

```
// An app that always has work: every millisecond it computes for 2 ms of virtual time.
app("hog").every(1, () => { const end = now() + 2; while (now() < end) {} });
```

Because ticks do not pile up, the app always has exactly one tick waiting when its segment ends, and as an app (priority 1) it always outranks the sandbox. We then typed `1+1` at the REPL. The keys reached the REPL, which is also priority 1 and therefore shares the lane with the hog in arrival order, and the REPL asked the sandbox to evaluate the line. The sandbox never ran. After 5 seconds the REPL's ask timed out:

```
$ ./build/picocalc-host -j hog/js -m hog/js/system.json -
wait 1200
key r
wait 300
type 1+1
key enter
wait 6000
stats
t = 7910.0 ms, lane busy 7910.4 ms (100.0%), 4397 turns, 23 published, 62 delivered, 0
crashes
  world       kind     state        turns    cpu ms    mem KB  limit  rst  last error
  …
  repl        app      idle             5      0.68     195.7    384    0
  sandbox     service  idle             0      0.06     193.1    512    0
  …
  hog         app      idle          3819   7829.02     180.0    256    0
```



Figure 4.2. The REPL after the sandbox has been starved by an app that always has work: the evaluation of 1+1 never started, and the REPL's ask failed after its 5,000 ms timeout.

The lane was 100% busy, the hog had used 7.8 seconds of it, and the sandbox had handled no turns at all. Nothing crashed and no budget was exceeded: each of the hog's segments took 2 ms, far inside its 50 ms budget. Starvation is not a fault the supervisor can see. It is a property of the policy, and PicoCalc OS has no aging that would eventually lift the sandbox's priority. In practice the system relies on apps being idle most of the time, which the measurements in Chapter 19 confirm for the apps it ships with. A system that must run untrusted apps alongside a priority-0 world would need either aging or a share for the lowest class, for example a rule that every *n*th dispatch goes to the oldest waiting world regardless of priority.

4.9 More than one core

The ESP32-P4 has two high-performance RISC-V cores, and PicoCalc OS uses one lane. A multiprocessor scheduler can be **global**, with one queue from which every core takes work, or **partitioned**, with each activity assigned to one core and each core scheduling its own set. Global scheduling balances load automatically but moves activities between caches and needs a shared, locked run queue. Partitioned scheduling keeps each core's decisions independent and each activity's cache warm, but can leave one core idle while the other is overloaded.

For PicoCalc OS the natural design is partitioned: a second lane for the drivers and the compositor, with apps and the sandbox on the first. Worlds already share no memory, so two segments of different worlds on different cores cannot race on JavaScript state. What they would share is the nucleus: the event queue, the mailboxes, the handle table and the deferred-effect lists, which today need no locks because only one segment runs at a time. A two-lane nucleus would need either locks on those structures or a message channel between the lanes. It would also have to preserve determinism. The virtual clock can model two lanes exactly if each lane keeps its own notion of when it is free and the nucleus resolves simultaneous events by a fixed rule, which is how discrete-event simulators of multiprocessors work (Chapter 18). This is planned work, not part of the system as it stands.

KEY POINTS

- A scheduler chooses the next job at dispatch points; turnaround, waiting, response time, throughput and utilisation measure the result, and fairness constrains it.
- FCFS suffers convoys; SJF minimises mean waiting among jobs waiting together; SRPT minimises mean turnaround for any arrival pattern but needs service times; round robin bounds response time without knowing them.

- Priorities express importance but can starve low-priority work; the multilevel feedback queue learns which jobs are short, and proportional-share schedulers such as lottery, stride, CFS and EEVDF divide the processor by weight.
- Preemption needs a timer and a stack per activity; QuickJS's interrupt hook can only abort a world's code, so PicoCalc OS runs every segment to completion and treats running past the budget as a fault.
- The lane is non-preemptive priority scheduling over classes (booting 10, drivers 3, services 2, apps 1, sandbox 0) with first come, first served by arrival time within a class.
- Periodic ticks keep a fixed rate and never queue more than one per timer, so a world that falls behind skips ticks instead of building a backlog.
- Blocking by a lower-priority segment that is already running is the cost of not preempting; measurements show it is invisible at small loads, adds whole segment lengths at large ones, and does not depend on the blocking world's priority.
- A world at the lowest priority can be starved indefinitely by one higher-priority world that always has work, and nothing in the supervisor detects it.

The lane's policy is simple enough to state in one sentence, and the measurements show that it behaves as the sentence predicts. What they cannot show is the worst case: the longest a driver could ever wait, and whether its work could miss a deadline. Answering that requires a model of the work as periodic tasks and a theory of response times, which is Chapter 5.

EXERCISES

1. BASIC Jobs P (arrival 0, service 4), Q (arrival 1, service 2), R (arrival 2, service 5) and S (arrival 3, service 1) share one processor. Compute each job's turnaround and response time under FCFS, SJF, SRPT and round robin with $q = 1$ (arrivals join the queue before a job whose slice expires at the same instant). Which policy gives the smallest mean turnaround?
2. ANALYSIS Prove that among jobs that are all waiting at time 0, running them in order of increasing service time minimises the mean waiting time. (Hint: consider exchanging two adjacent jobs that are out of order.)
3. ANALYSIS A round-robin scheduler has slice q and a switch cost c. Derive the fraction of processor time spent switching when n jobs are always ready, and compute it for $q = 10$ ms, $c = 0.1$ ms and for $q = 1$ ms, $c = 0.1$ ms.

4. ANALYSIS Under MLFQ without Rule 4's accounting (a job keeps its priority if it gives up the processor before its slice expires), describe a job that monopolises the processor while staying at the top level. Show how Rule 4 stops it.

5. BASIC Under lottery scheduling a job holds 1 of 4 tickets. What is the probability that it receives no slice in 10 consecutive draws? What would stride scheduling give?

6. ANALYSIS Explain from the code of `pick` why lowering the background app's priority from 1 to 0 left every number in Table 4.5 unchanged. Describe a different background workload for which the change of priority would matter.

7. CODE Set the sandbox's `budget_ms` to 50 in a copy of `js/system.json` and repeat the 100,000-iteration loop of Section 3.8 at the REPL (run the host build with `-j` and `-m` pointing at your copy). What does the REPL show, what does the UART log record, and how long until the sandbox can evaluate again?

8. CODE Reproduce the starvation experiment of Section 4.8. Then modify `pick` so that a world that has been runnable for more than 1,000 virtual ms is treated as priority 2 (aging), and show that the sandbox now answers. What does the change do to the keyboard driver's worst case?

9. ANALYSIS With the budgets in `js/system.json`, and assuming no world is restarting, what is the longest a keyboard driver segment can wait for the lane? How does the answer change while some world is restarting? (Use the overrun bound from Chapter 11: a segment runs at most 0.70 ms plus one host call past its budget.)

10. CODE Change the keyboard driver's period from 20 ms to 5 ms and measure the I2C bus utilisation and the lane utilisation with `stats` over 10 virtual seconds on the home screen. Compare with the poll-period sweep reported in Chapter 19.

11. PROJECT Implement a second lane in the nucleus for drivers and the compositor. Keep the system deterministic: define what happens when both lanes would deliver a message to the same world at the same virtual time. Measure the launcher's key latency under the $L = 40$ ms background load of Table 4.5.

12. DISCUSSION PicoCalc OS could make segments preemptible by giving each world a FreeRTOS task on the device. List what this would change in the nucleus, the virtual clock and the guarantees of Chapter 3, and argue for or against it.

FURTHER READING

[Arpaci-Dusseau and Arpaci-Dusseau 2018] The scheduling chapters of *Operating Systems: Three Easy Pieces* develop FCFS, SJF, round robin, MLFQ and proportional share with worked examples in the same style as this chapter.

[Waldspurger and Weihl 1994] Lottery scheduling: tickets, transfers and currencies, with measurements.

[Schrage 1968] A two-page proof that SRPT is optimal; worth reading for how short an optimality argument can be.

[Linux kernel documentation 2023; Linux kernel documentation 2025] The kernel's own description of CFS and EEVDF, the schedulers most computers in the world are running.

Real-time systems

A **real-time system** is one whose correctness depends on when its results are produced as well as on what they are. A motor controller that computes the right torque 5 ms too late has computed the wrong torque; a keyboard driver that reads the controller's FIFO after it has overflowed has lost a key, however carefully it parses the rest. Real time is therefore not the same thing as fast. A fast system does well on average; a real-time system must do well enough in the worst case, and the engineering problem is to know what the worst case is.

Chapter 4 described scheduling policies by their averages and their failure modes. This chapter asks the stronger question: given a set of activities with known demands and deadlines, can we prove before running the system that every deadline will be met? The classical theory answers it for periodic tasks under fixed priorities and under earliest-deadline-first scheduling, with and without preemption and shared resources. We develop the theory on small examples worked by hand, recount the priority inversion that reset a spacecraft on Mars, and then apply the analysis to PicoCalc OS. The result is a precise statement of what the lane can and cannot guarantee for the keyboard driver, and of how far the guarantee you can prove from budgets is from the behaviour you can measure.

5.1 Deadlines and what real time means

A **deadline** is the time by which a piece of work must finish. Deadlines are classified by what happens when one is missed:

- A **hard** deadline must never be missed, because a late result is a failure of the system: an airbag that fires late, a control loop that goes unstable.

- A **firm** deadline may occasionally be missed, but a late result is worthless and is discarded: a video frame decoded after its display time.
- A **soft** deadline may be missed, and a late result still has value that decreases with lateness: a key echoed to the screen after 200 ms instead of 30.

Most of PicoCalc OS's timing requirements are soft. Nobody is harmed if a key appears late. One requirement is closer to firm: the keyboard controller buffers key events in a FIFO of 31 entries (`FIFO_SIZE` in `sim/stm32kbd.c`, after the clockworkpi firmware), and an event that arrives when the FIFO is full is dropped. The driver's real deadline is therefore not "poll every 20 ms" but "drain the FIFO before 31 events accumulate", and we will return to it at the end of the chapter.

Two more terms recur. **Jitter** is the variation in when something happens: the spread of a periodic task's start times, or of a response time, around their nominal values. **Predictability** is the ability to bound such variation in advance. A system with a 1 ms average response and an unknown worst case is less useful for real-time work than one with a 5 ms average and a proven 8 ms worst case.

A note on notation. Earlier chapters used τ for the virtual time of one interpreter step (665 ns), and it keeps that meaning here. Tasks are numbered $i = 1, \ldots, n$, and their parameters carry the subscript.

5.2 The periodic task model

The theory starts from a simple model of the work. A **periodic task** i is a sequence of **jobs** released every T_i time units. Each job needs at most C_i units of processor time, its **worst-case execution time** (WCET), and must finish within D_i of its release, its **relative deadline**. If releases are separated by *at least* T_i rather than exactly, the task is **sporadic**, and every analysis below that assumes periodic release also holds for sporadic tasks with T_i as the minimum separation. A task may also suffer **release jitter** J_i, a delay between the event that should release a job and the moment it becomes ready.

The **utilisation** of task i is $U_i = C_i/T_i$, the fraction of the processor it needs in the long run, and the utilisation of the set is the sum:

$$U = \Sigma_{i=1}^{n} C_i / T_i \tag{5.1}$$

$U \le 1$ is necessary for any schedule on one processor, since otherwise the work released in a long interval exceeds the length of the interval. It is not sufficient for most policies. The pattern of releases repeats with the **hyperperiod**, the least common multiple of the periods, so a schedule that meets every deadline over

one hyperperiod, starting with all tasks released together, meets them forever (for the policies in this chapter).

Liu and Layland's 1973 paper, the foundation of the field, made the model precise with five assumptions [Liu and Layland 1973]: requests for each task are periodic with a constant interval; each job must finish before the next request of the same task (so $D_i = T_i$); tasks are independent; each task's run time is constant (in practice, bounded by C_i); and any non-periodic work is special, such as initialisation or recovery. Their analysis also takes the scheduler to be preemptive and switching to be free. Much of the rest of this chapter is about what happens when these assumptions are relaxed, because PicoCalc OS relaxes three of them: its scheduler does not preempt, its worlds are not independent (they share buses), and its work is not strictly periodic.

The running example of this chapter is three tasks with $D_i = T_i$:

Table 5.1. The chapter's example task set. Its utilisation is 0.25 + 0.333 + 0.25 = 0.833 and its hyperperiod is 12.

Task	C	$T = D$	U
1	1	4	0.250
2	2	6	0.333
3	3	12	0.250

5.3 Cyclic executives

The oldest way to meet periodic deadlines needs no scheduler at all. A **cyclic executive** runs a fixed table, computed before the system runs. Time is divided into **minor frames** of length f; a **major cycle** of frames repeats with the hyperperiod; and each frame lists the jobs to run in it, in order. A timer interrupt at the start of each frame starts the next entry of the table, and the executive checks that the previous frame finished in time. Baker and Shaw analysed the model and its use with Ada [Baker and Shaw 1989].

The frame length must satisfy three conditions. It must be at least as long as every job, $f \geq \max C_i$, so that no job has to be split across frames. It should divide the hyperperiod, so that the table repeats. And for every task, $2f - \gcd(f, T_i) \leq D_i$: this guarantees that between a job's release and its deadline there is at least one whole frame, which is where the table can place it. For the example, $f = 4$ works: it is at least 3, it divides 12, and the third condition gives $4 \leq 4$, $6 \leq 6$ and $4 \leq 12$. The major cycle is three frames:

Table 5.2. A cyclic executive for the example, with three minor frames of length 4.

Frame	Interval	Jobs (in order)	Busy
1	0–4	task 1, task 2	3
2	4–8	task 1, task 3	4
3	8–12	task 1, task 2	3

Task 2's second job is released at 6 and runs in frame 3, finishing by 11, before its deadline at 12. The table meets every deadline with no scheduler, no priorities and no preemption, and its timing is completely predictable: the same job runs at the same offset in every major cycle. Its weakness is maintenance. Adding a task, lengthening one, or changing a period can force the whole table to be recomputed, and a long job must be split by hand into pieces that fit frames. Cyclic executives survive where certification demands complete predictability; elsewhere they gave way to priority-driven scheduling.

5.4 Fixed priorities and the Liu–Layland bound

A **fixed-priority** scheduler gives each task a priority and always runs the highest-priority ready job, preempting a lower one when necessary. The question is which priorities to choose. **Rate monotonic** (RM) assignment gives higher priority to tasks with shorter periods. Liu and Layland proved that it is optimal among fixed-priority assignments when deadlines equal periods: if any fixed assignment meets every deadline, rate monotonic does [Liu and Layland 1973]. When deadlines are shorter than periods, the optimal fixed assignment is **deadline monotonic**: shorter relative deadline, higher priority [Leung and Whitehead 1982].

Their proof rests on a lemma that every later analysis uses. A **critical instant** of a task is a release time at which its job has the longest possible response time; Liu and Layland showed that it occurs when the task is released together with all tasks of higher priority. So to find the worst case of each task we need only examine one scenario: everything released at time 0. Figure 5.1 shows it for the example under rate monotonic priorities (task 1 highest).

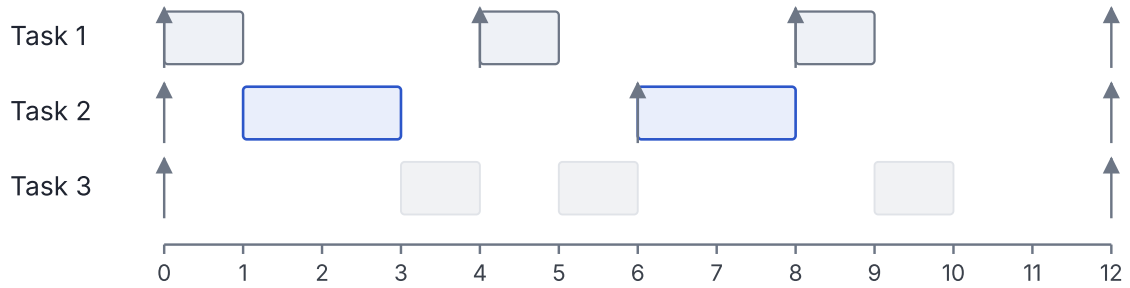


Figure 5.1. The example under rate monotonic priorities, from a critical instant. Arrows mark releases. Task 3's job is preempted twice by task 1 and once by task 2 and finishes at 10, before its deadline at 12.

The most quoted result of the paper is a sufficient test that needs only the utilisation. A set of n tasks is schedulable under rate monotonic priorities if

$$U \le n(2^{1/n} - 1) \tag{5.2}$$

The bound is 1 for one task, 0.828 for two, 0.780 for three, 0.757 for four and 0.743 for five, and it decreases towards $\ln 2 \approx 0.693$ as n grows. Any set whose utilisation is below 0.693 is schedulable by rate monotonic priorities, however many tasks it has.

The test is sufficient, not necessary. The example has $U = 0.833 > 0.780$, so the bound says nothing, and yet the schedule in Figure 5.1 meets every deadline. Sets whose periods are **harmonic**, each period dividing the next longer one, are schedulable under rate monotonic priorities right up to $U = 1$. The bound is a quick check for sets with comfortable slack; for the rest we need an exact test.

5.5 Exact response-time analysis

The **response time** of a job is the time from its release to its completion; a task's worst-case response time R_i is the maximum over its jobs. Under fixed preemptive priorities and a critical instant, task i's job completes when the processor has done C_i of its own work plus all the higher-priority work released in the meantime. A task j of higher priority released at 0, T_j, $2T_j$, … contributes $\lceil R/T_j \rceil$ jobs in an interval of length R. So the worst-case response time is the smallest positive solution of

$$R_i = C_i + \sum_{j \in \mathrm{hp}(i)} \lceil R_i / T_j \rceil \, C_j \tag{5.3}$$

where hp(i) is the set of tasks of higher priority than i. The equation appears on both sides, so it is solved by iteration: start with $R = C_i$, evaluate the right-hand side, and repeat until the value stops changing (the task is schedulable if that value is at most D_i) or exceeds D_i (it is not). The iteration only ever increases, so it terminates. Joseph and Pandya introduced this analysis in 1986 [Joseph and Pandya 1986], and Audsley and colleagues extended it to deadlines shorter than periods, release jitter and blocking [Audsley et al. 1993].

For the example, task 1 has no interference: $R_1 = 1$. Task 2 starts at 2; $2 + \lceil 2/4 \rceil \cdot 1 = 3$; $2 + \lceil 3/4 \rceil \cdot 1 = 3$, so $R_2 = 3$. Task 3 needs more steps:

Table 5.3. Response-time iteration for task 3 of the example.

Step	R in	$\lceil R/4 \rceil \cdot 1$	$\lceil R/6 \rceil \cdot 2$	3 + interference
1	3	1	2	6
2	6	2	2	7
3	7	2	4	9
4	9	3	4	10
5	10	3	4	10

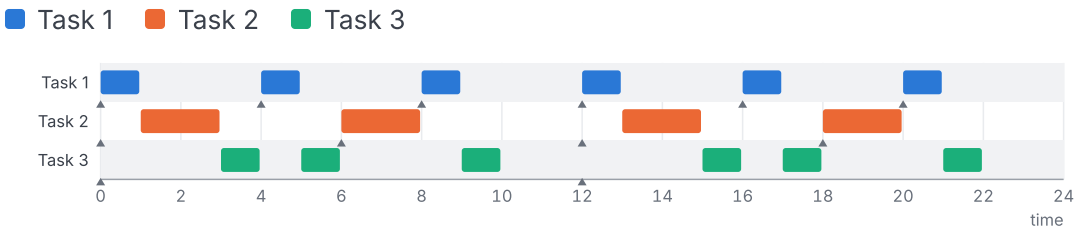
The iteration settles at $R_3 = 10 \le 12$, the completion time visible in Figure 5.1. All three tasks meet their deadlines although the utilisation bound could not say so.

Two extensions make the equation useful for real systems. **Release jitter** J_j of a higher-priority task can make two of its jobs arrive closer together than T_j, so its term becomes $\lceil (R_i + J_j)/T_j \rceil C_j$, and task i's own response time grows by J_i. **Blocking** B_i, time during which a lower-priority job prevents task i from running, adds a constant term. Section 5.7 and Section 5.8 show where blocking comes from.

TRY 5.1

Check the example by simulation and by analysis

The widget opens on the example under rate monotonic priorities, over two hyperperiods. The Gantt tab shows the schedule; the analysis tab shows the utilisation, the Liu–Layland bound for three tasks and the response-time iteration for each task, step by step. Raise task 3's C from 3 to 4 (U = 0.917): the iteration for task 3 now runs past its deadline, and the Gantt chart shows the miss at time 12. Then try C = (1, 2, 6) with periods (4, 8, 16): the periods are harmonic and the set is schedulable at U = 1.



U = 0.833. U > 0.780 (the bound for n = 3): the bound cannot decide; response-time analysis can.

no misses in the simulation

Task	C	T	D	jobs	max R	mean R	misses
Task 1	1	4	4	6	1	1	0
Task 2	2	6	6	4	3	2.50	0
Task 3	3	12	12	2	10	10	0

Exact response times under rate-monotonic priorities.

Task	C	R	D	
Task 1	1	1	4	meets D
Task 2	2	3	6	meets D
Task 3	3	10	12	meets D

5.6 Earliest deadline first

A fixed-priority scheduler cannot always use the whole processor. **Earliest deadline first** (EDF) assigns priorities dynamically: at every moment, the ready job with the earliest absolute deadline runs. Liu and Layland proved that for periodic tasks with $D_i = T_i$ on one preemptive processor, EDF meets every deadline if and only if $U \le 1$ [Liu and Layland 1973]. No algorithm can do better, since $U \le 1$ is necessary for any.

Consider two tasks with $C = (2, 4)$ and $T = (5, 7)$, so $U = 0.4 + 0.571 = 0.971$. Under rate monotonic priorities task 1 is higher. The response-time iteration for task 2 runs 4, then $4 + \lceil 4/5 \rceil \cdot 2 = 6$, then $4 + \lceil 6/5 \rceil \cdot 2 = 8 > 7$: task 2 misses its first deadline. Directly: task 1 runs 0–2, task 2 runs 2–5, task 1 preempts it at its second release and runs 5–7, and task 2 still needs one unit at its deadline. Under EDF, at time 5 task 2's deadline (7) is earlier than task 1's new one (10), so task 2 continues and finishes at 6. Following the schedule through the hyperperiod of 35 shows every deadline met, as the theorem says it must be, with the processor idle for exactly one unit.

EDF's advantages are the full use of the processor and a test that is one line. Its disadvantages are practical. Dynamic priorities need a priority queue ordered by deadline instead of a fixed array. And EDF behaves badly in overload: when $U > 1$, which jobs miss depends on the exact deadlines and the moment the overload began, and a miss can cascade as late jobs push others past their deadlines. Under fixed priorities, overload falls on the lowest-priority tasks and the highest ones are unaffected, which is often what a designer wants. Many real-time operating systems, FreeRTOS among them, offer only fixed priorities for this reason (Chapter 7).

5.7 Non-preemptive scheduling and blocking

Everything so far assumed preemption. PicoCalc OS's lane does not preempt (Chapter 4), and neither does a CAN bus, on which a frame being transmitted cannot be interrupted by a more urgent one. Without preemption, a job of high priority that is released just after a lower-priority job has started must wait for that job to finish. This is **blocking**, and the worst case for task i is waiting for the longest job of any lower-priority task:

$$B_i = \max_{k \in \mathrm{lp}(i)} C_k \tag{5.4}$$

It is tempting to add B_i to the response-time equation and stop. That is wrong in two ways, and the second one went unnoticed in the analysis of CAN for more than a decade until Davis, Burns, Bril and Lukkien published the correction [Davis et al. 2007].

The first error is minor. Once a non-preemptive job starts, nothing can interfere with it, so the interference that matters is what happens before it *starts*. The analysis computes a **queuing delay** w, the time from release to start, counting higher-priority jobs released up to and including the start time (a higher-priority job released at the same instant would start first, hence $\lfloor w/T_j \rfloor + 1$ rather than a ceiling), and then adds the job's own C_i.

The second error is serious. Without preemption, a job of task i can delay the next job of the same task: if the first job runs late, the second may find the processor occupied by higher-priority work that accumulated meanwhile. The worst case is not necessarily the first job after a critical instant. The correct test examines every job of task i in the **level-i busy period**, the longest interval starting at a critical instant during which the processor is continuously busy with work of priority i or higher plus the initial blocking. Its length t_i and the number of jobs Q_i it contains are

$$t_i = B_i + \Sigma_{j \in \text{hep}(i)} \lceil t_i / T_j \rceil C_j, \; Q_i = \lceil t_i / T_i \rceil \tag{5.5}$$

where hep(i) includes task i itself. For each job $q = 0, \ldots, Q_i - 1$ the queuing delay and response time are

$$w_i(q) = B_i + qC_i + \Sigma_{j \in \text{hp}(i)} \left(\lfloor w_i(q) / T_j \rfloor + 1\right) C_j, \; R_i(q) = w_i(q) - qT_i + C_i \tag{5.6}$$

and R_i is the largest $R_i(q)$. (Davis and colleagues write the CAN version, in which the bit time of the bus replaces the "+1"; the structure is the same.)

Their counterexample has three tasks of equal length, each taking 1 time unit, with priorities A > B > C: A has period 2.5, while B and C have period 3.5 and deadline 3.25. Consider C, the lowest. It suffers no blocking. Its busy period iterates 1, 3, 4, 6, 7, 7, so $t = 7$ and $Q = 2$: two jobs of C must be checked. For the first, w settles at 2 and $R(0) = 2 + 1 = 3 \le 3.25$, which is all the old analysis computed. For the second, $w(1)$ iterates 3, 4, 5, 6, 6 and $R(1) = 6 - 3.5 + 1 = 3.5 > 3.25$. Figure 5.2 shows what happens: C's first job occupies the processor from 2 to 3, so A's job released at 2.5 runs late, from 3 to 4; B's second job runs 4 to 5; A's next job, released at 5, starts before C; and C's second job, released at 3.5, finishes at 7, after its deadline at 6.75.

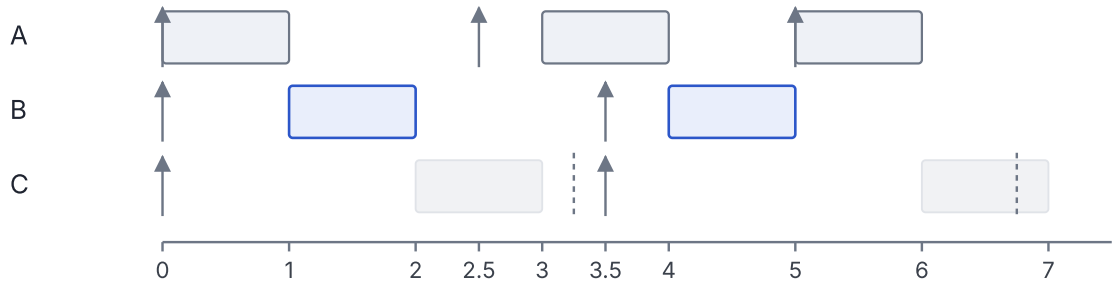


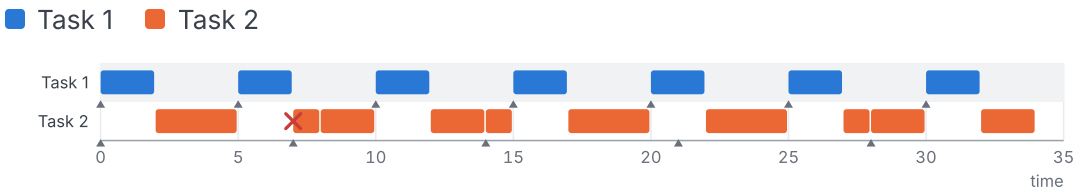
Figure 5.2. The counterexample of Davis et al. on a non-preemptive processor. Dashed lines mark C's deadlines. C's first job meets its deadline; its second, delayed by work that C's first job pushed back, misses it.

The lesson applies directly to PicoCalc OS. Any analysis of a non-preemptive lane that checks only the first job after a critical instant is optimistic, and the lane is exactly such a system.

TRY 5.2

EDF against rate monotonic, and a non-preemptive surprise

The first widget runs the two tasks of Section 5.6 (C = 2, 4; T = 5, 7) under rate monotonic priorities: task 2 misses its deadline at 7. Switch the algorithm to EDF and the miss disappears for the whole hyperperiod. The second widget runs the counterexample of Figure 5.2 with non-preemptive fixed priorities; its analysis tab lists both jobs of C in the busy period and shows that the second one is the one that misses.

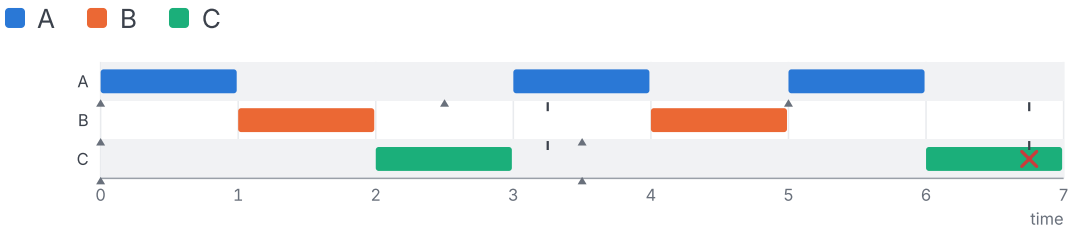


U = 0.971. U > 0.828 (the bound for n = 2): the bound cannot decide; response-time analysis can. 1 deadline miss in the simulation

Task	C	T	D	jobs	max R	mean R	misses
Task 1	2	5	5	7	2	2	0
Task 2	4	7	7	5	8	6.80	1

Exact response times under rate-monotonic priorities.

Task	C	R	D	
Task 1	2	2	5	meets D
Task 2	4	8	7	misses D



U = 0.971. Liu–Layland bound for n = 3: 0.780. 1 deadline miss in the simulation

Task	C	T	D	jobs	max R	mean R	misses
A	1	2.50	2.50	3	1.50	1.17	0
B	1	3.50	3.25	2	2	1.75	0
C	1	3.50	3.25	2	3.50	3.25	1

Exact response times under the given priorities, non-preemptive. Blocking B is the longest lower-priority C; every job in the level-i busy period is checked.

Task	B	R	D	
A	1	2	2.50	meets D
B	1	3	3.25	meets D
C	0	3.50	3.25	misses D

5.8 Sharing resources: priority inversion

Liu and Layland assumed independent tasks. Real tasks share data, and shared data needs mutual exclusion (Chapter 3). Locks interact with priorities in a way that defeats them.

Suppose a low-priority task L holds a lock, and a high-priority task H is released and tries to take it. H must wait for L to release the lock; that is unavoidable and bounded by the length of L's critical section. But now a medium-priority task M, which does not use the lock at all, is released. M has higher priority than L, so it preempts L, and as long as M runs, L cannot finish its critical section, so H cannot run either. H is effectively waiting for M, a task of lower priority, for as long as M (and any other medium-priority task) wants to run. This is **priority inversion**, and the unbounded part of it, the waiting for M, is what makes it dangerous.

The best-known instance happened on Mars. The Pathfinder lander, which reached Mars on 4 July 1997, ran the VxWorks real-time kernel. In December 1997, Mike Jones circulated an account of a keynote by David Wilner of Wind River describing what had gone wrong, and Glenn Reeves of JPL, who worked on the lander's flight software, followed with his own [Jones 1997; Reeves 1997]. The lander's "information bus", a shared memory area used to pass data between tasks, was protected by a mutex. A high-priority bus distribution task (`bc_dist`) ran every cycle of an 8 Hz schedule. A low-priority meteorological data task (ASI/MET) published to the bus and, inside the interprocess-communication mechanism it used, held a mutex that `bc_dist` also needed. Medium-priority tasks preempted the meteorological task while it held the mutex, so `bc_dist` was blocked past the end of its cycle. A scheduling task (`bc_sched`) that checked at

the start of each cycle whether `bc_dist` had completed found that it had not, declared the error, and the spacecraft reset. The resets had been seen before launch and attributed to hardware; after they appeared on Mars, the team reproduced them on a replica in the laboratory within 18 hours, with VxWorks's tracing turned on. The mutex had been created with priority inheritance disabled. The fix was to change a global configuration variable so that the semaphores used by that mechanism were created with priority inheritance enabled, and to upload the change to the spacecraft.

Two protocols bound priority inversion, both introduced by Sha, Rajkumar and Lehoczky [Sha et al. 1990].

Under the **priority inheritance protocol** (PIP), a task that holds a lock and blocks a higher-priority task temporarily runs at that task's priority. In the scenario above, L inherits H's priority when H blocks, so M can no longer preempt L; L finishes its critical section, releases the lock, drops back to its own priority, and H runs. H's blocking is now bounded by the length of critical sections rather than by the behaviour of unrelated tasks. PIP has two remaining weaknesses: a job can be blocked once for each lock it needs (**chained blocking**), and it does not prevent deadlock between tasks that take two locks in opposite orders.

Under the **priority ceiling protocol** (PCP), each lock has a **ceiling**, the highest priority of any task that uses it. A task may take a lock only if its priority is higher than the ceilings of all locks currently held by other tasks; otherwise it blocks, and the task holding the relevant lock inherits its priority. The rule looks restrictive, and it is: it can block a task before it touches a contended lock. In return, each job is blocked for at most the duration of one critical section of one lower-priority task, and deadlock is impossible. A simpler variant, the **immediate ceiling protocol**, raises a task's priority to the lock's ceiling as soon as it takes the lock rather than when it blocks someone; it gives the same bound of one critical section, and POSIX offers it as the `PTHREAD_PRIO_PROTECT` mutex protocol, alongside `PTHREAD_PRIO_INHERIT` for inheritance. Baker's **stack resource policy** (SRP) achieves the same bound by deciding at the moment a job starts whether it may run at all, so a job never blocks once started; this makes it work with EDF as well as with fixed priorities, and lets all tasks share a single run-time stack [Baker 1991].

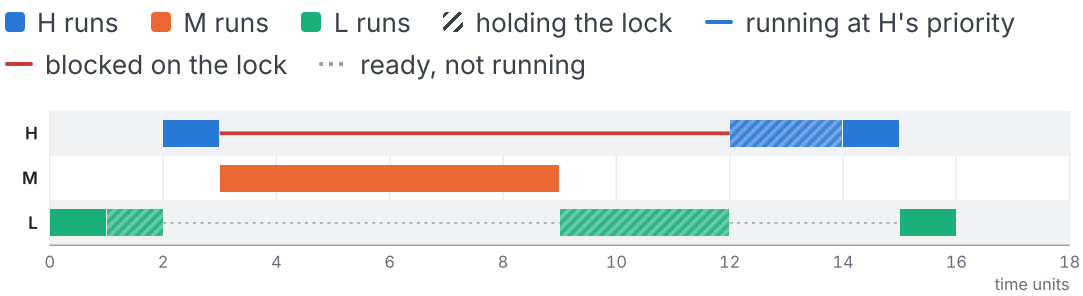
With either protocol, the blocking bound B_i enters the response-time equation as a constant:

$$R_i = C_i + B_i + \Sigma_{j \in \mathrm{hp}(i)} \lceil R_i / T_j \rceil \, C_j \tag{5.7}$$

TRY 5.3

Reproduce the inversion and remove it

The widget runs three tasks, H, M and L, of which H and L share one lock. L takes the lock at time 1; H arrives at 2 and asks for the lock at 3; M, which never touches the lock, arrives at 3. With no protocol, M preempts L inside its critical section, and H waits for M as well: its response time is 13, of which 9 is blocking (3 by L's critical section, 6 by M's unrelated work). Switch to priority inheritance: H runs its first unit at 2, blocks at 3, L inherits H's priority, M must wait, and H's response falls to 7. Then switch to the ceiling variant, which here is the immediate form: L runs at the lock's ceiling from the moment it takes the lock, so H cannot even start when it arrives at 2. It waits until L leaves the critical section, and its response is again 7: the same bound, reached by blocking earlier.



No protocol: H is released at 2 and needs 4 units of work; it finishes after 13, so it waited 9 units: 3 behind L's critical section and 6 while M ran. unbounded: grows with M

```
t=0 L is released
t=1 L takes the lock
t=2 H is released
t=2 H preempts L
t=3 M is released
t=3 H blocks on the lock held by L
t=3 M runs; L, holding the lock, does not
t=9 M finishes
t=12 L releases the lock
t=12 H takes the lock
t=12 H preempts L
t=14 H releases the lock
t=15 H finishes
```

```
t=16 L finishes
```

PicoCalc OS has no locks between worlds, since worlds share no memory, so it cannot suffer this kind of inversion. But it has two shared resources whose behaviour the same ideas describe. The first is the lane itself. Because no segment is preempted, every segment acts as a critical section on one resource that every world needs, and the lane's rule, "a world becomes runnable and waits for the segment in progress to end", is the protocol of running every critical section without preemption. Its virtue is the virtue of the ceiling protocols: a world is blocked by at most one lower-priority segment each time it becomes ready, and never by a chain of medium-priority work, because once the blocking segment ends, the higher-priority world is chosen. Its vice is that the blocking term is the length of the longest lower-priority segment of any kind, not the longest critical section on a lock the blocked world actually uses.

The second is the I2C bus. The keyboard and power drivers both talk to the STM32 on it, and the nucleus serialises their transfers in the order they are issued: a transfer that finds the bus busy starts when the previous one ends (`bus_start` in `nucleus/hw.c`). A keyboard read can therefore wait behind a battery read, 6.9 ms at the bus's 10 kHz. Bus order is first come, first served, not by priority, so this is a small, bounded inversion: one transfer of at most 6.9 ms per keyboard transfer.

5.9 Worst-case execution time

Every analysis in this chapter takes C_i as given. Finding it is a field of its own. The **worst-case execution time** of a piece of code is the longest it can take on a given processor over all inputs and initial hardware states. Measurement can show that a run took a certain time, but not that no run takes longer; static analysis can bound the time from the code and a model of the processor, but must account for caches, pipelines and branch predictors whose state depends on everything that ran before. Wilhelm and colleagues survey both approaches and the tools that implement them [Wilhelm et al. 2008].

An interpreter adds a layer. On the virtual clock, a world's cost is exact by construction: its interpreter steps times τ, plus its native work, plus 40 µs (Chapter 18). On the device, the time per step varies with the instruction mix, with whether the interpreter's data is in cache or in PSRAM, and with memory management. The calibration of τ against two programs on the ESP32-P4 matched one exactly by construction and the other within 6% (`fib(20)`, 29.1 ms predicted against 31 ms measured), which is the right order of agreement for a single-parameter model and not a WCET bound.

Garbage collection is part of the same problem. QuickJS frees most objects by reference counting, at the moment their last reference disappears, which spreads the cost of memory management evenly over the program. Cycles of objects that refer to each other are collected by a separate cycle-removal pass, which QuickJS runs automatically when a runtime's allocated memory passes a threshold: 256 KB at first, then one and a half times the memory in use after the previous collection (`js_trigger_gc` in `quickjs.c`). The pass runs inside whatever segment made the allocation, and on the virtual clock it is charged nothing, which is one of the costs the model leaves out (Chapter 18). Two properties of PicoCalc OS keep it bounded: each world has its own runtime, so a collection traverses only that world's heap, and each heap has a hard limit (between 256 and 768 KB in the manifest). Real-time garbage collectors go further and schedule collection work in small quanta so that the program is guaranteed a minimum share of every time window, as IBM's Metronome collector did for Java [Bacon et al. 2003].

PicoCalc OS's budgets are best understood as a substitute for WCET analysis. Instead of proving that a segment cannot exceed C, the nucleus enforces it: a segment that exceeds its world's budget is stopped within 0.70 ms plus one host call, which the nucleus bounds at 1.64 ms, of the budget (Chapter 11; an earlier figure of 0.26 ms was wrong, and that chapter tells how the review found it). For scheduling analysis, a world's budget plus that overrun is therefore a safe upper bound on any of its segments, whatever its code does, at the price of turning an overrun into a crash.

5.10 Time-triggered designs and logical execution time

The analyses so far are event-triggered: work is released by events and scheduled by priority. A **time-triggered** architecture releases every activity at predetermined instants from a global clock, communicates at predetermined instants too, and treats any event that arrives at another time as an error; Kopetz's book develops the approach for distributed embedded systems [Kopetz 2011]. The cyclic executive of Section 5.3 is its single-processor form.

A refinement makes time-triggered designs composable. In the **logical execution time** (LET) model of the Giotto language, a task reads its inputs at the instant it is released and its outputs become visible at the end of its period, regardless of when within the period it actually ran [Henzinger et al. 2003]. The observable timing of the system, which values are read and written at which instants, is then fixed by the specification and independent of execution times, priorities and processor speed, provided each task finishes within its period.

PicoCalc OS's deferred effects (Chapter 3) are a relative of LET. A segment's outputs become visible at a single instant, the end of the segment, not scattered through its execution. But the instant is the segment's actual end, which depends on its own execution time and on whatever ran before it. The design makes the

system deterministic, since the same inputs produce the same sequence of segments, but not time-invariant: a change in one world's execution time moves the visible times of other worlds' effects. Exercise 11 asks what a LET version of the lane would look like.

5.11 What the lane can guarantee

We can now analyse PicoCalc OS as a real-time system. The question is the one Section 5.1 raised: how long can the path from a key press to the keyboard driver's `publish` take, and can the STM32's 31-entry FIFO overflow?

The model. The keyboard driver is a periodic task with T = 20 ms. Each poll is one turn of up to three kinds of segment, separated by I2C transfers during which the lane is free (Section 3.8): a segment that starts the status read, a segment per pending key that starts a FIFO read, and a segment that publishes. In the recorded traces these segments take 50 to 80 µs each. The status read occupies the bus for 6.0 ms and each FIFO read for 6.9 ms. In the language of real-time theory, the driver is a **self-suspending** task: it gives up the processor in the middle of a job to wait for a device. Each segment is analysed as its own job, released when the transfer before it completes, and each can be blocked afresh.

Blocking per segment. When a keyboard segment becomes ready, it can wait for four things. The segment in progress, whatever its priority, must end. Other drivers' segments that became ready earlier run first, because the display and power drivers share the keyboard driver's priority and the lane is first come, first served within a class. A world that is starting or restarting runs its boot segment before anything else. And the keyboard segment's own C follows. Table 5.4 gives each term two ways: the bound that the budgets in `js/system.json` guarantee whatever the worlds' code does, and the largest value observed in the recorded traces and benchmarks.

Table 5.4. What a keyboard-driver segment can wait for when it becomes ready. Budget bounds add the overrun bound of 0.70 ms plus one host call (computed here with 0.26 ms; the corrected bound is 1.64 ms, which adds at most 1.4 ms per blocking segment). Observed values are the longest segments in the six recorded traces and the benchmarks, in virtual ms.

Source of delay	World	Bound from budgets	Longest observed
A lower-priority segment in progress	sandbox (user code)	200.96	200.18
	compositor	80.96	1.86
	an app	50.96	11.5 (snake on autopilot)
Equal-priority segments that arrived first	display driver	30.96	0.50
	power driver	20.96	0.07
A world restarting (boot segment)	any	600.96	0.07
The segment itself	keyboard driver	20.96	0.08

The sandbox's observed maximum is its budget: in the fault experiments it was given an endless loop and was stopped 0.18 ms after its 200 ms budget. Every other observed value is far below its budget, because the budgets are fault thresholds, not estimates.

The bound. Leave restarts aside for the moment. A key pressed just after a poll read the status register waits for the next poll, which is due at most 20 ms later if the previous poll has finished. That poll then needs three segments, one for the status read, one for the FIFO read and one to publish, and each can wait for one lower-priority segment in progress (at most 200.96 ms), a display and a power segment that arrived first (30.96 and 20.96 ms), and itself (0.08 ms): 252.96 ms per segment. The two transfers take 12.9 ms of bus time, and each can wait behind one power-driver transfer, up to 6.9 ms. The total is

$$20 + 3 \times 252.96 + 12.9 + 2 \times 6.9 \approx 806 \text{ ms}$$

from key press to `publish`. With a world restarting, a boot segment of up to 600.96 ms can be added in front of any of the three segments.

The same sum with the observed segment lengths, which is the right model for the shipped worlds when nobody runs a long computation at the REPL, gives at most 11.5 + 0.50 + 0.07 + 0.08 ≈ 12.2 ms per segment and about 83 ms in total. The measurements in Chapter 19 and Chapter 4 sit below both numbers. The longest key-to-pixel latency in the launcher on an idle system was 40.7 ms over 100 presses, and that figure includes drawing and sending pixels. Under a background load of 40 ms every 50 ms it was 103.8 ms. A key pressed during the REPL's 133 ms loop reached the REPL 98.7 ms after the press (Section 3.8).

What is guaranteed. Two statements follow, and a third does not.

- The 20 ms poll period is not a deadline the system can meet. One sandbox segment exceeds it tenfold, and the lane cannot interrupt it. When a poll is late its tick waits in the mailbox and the ticks after it are skipped (Section 4.6.2), so polls are delayed, never duplicated.
- Every key event is published within about 806 ms if the FIFO held nothing else when it arrived (about 1.4 s while a world is restarting). A full FIFO drains more slowly, because each of its 31 entries needs its own read and each read can be blocked again by a long lower-priority segment: one test typing ten keys a second during a REPL computation made of 190 ms segments lost 30 of 80 key events. Typing is safe only while lower-priority segments stay short.
- No useful bound on input latency follows from the budgets alone. The 806 ms worst case is a correct bound but too weak to be a specification. The measured behaviour is an order of magnitude better because the budgets are an order of magnitude larger than the segments they bound.

What would make the guarantee tighter. The analysis points at the terms to shrink. Smaller budgets for low-priority worlds reduce the dominant term directly. Cutting only the sandbox's budget to 20 ms makes the compositor's 80 ms the largest term, and the bound per segment falls from 253 to about 133 ms; cutting the compositor's and every app's budget to 20 ms as well brings it to about 73 ms. Splitting long work into short segments (awaiting `sleep(0)` between pieces) achieves the same thing voluntarily. A second lane for the drivers (Section 4.9) removes lower-priority blocking from the drivers altogether, leaving only equal-priority segments and the bus. And the bus itself, at 10 kHz with a 2 ms wait inside each transaction, is the largest term once blocking is gone: even with no blocking at all, a poll that reads one key spends 12.9 ms on the bus, and a battery read in the way adds 6.9 ms more. A hard 20 ms guarantee for the keyboard on this board is impossible without changing the bus, whatever the scheduler does.

KEY POINTS

- A real-time system is correct only if its results are on time; hard, firm and soft deadlines differ in what a miss costs, and predictability of the worst case matters more than speed on average.

- The periodic task model describes work by worst-case execution time, period and deadline; utilisation at most 1 is necessary, and the hyperperiod bounds how far a schedule must be checked.
- Rate monotonic priorities are optimal among fixed priorities for deadlines equal to periods, and any set with utilisation below n(2^(1/n) − 1) is schedulable; the test is sufficient, not necessary.
- Exact response-time analysis solves R = C + Σ⌈R/T⌉C by iteration from a critical instant and decides schedulability under fixed preemptive priorities; jitter and blocking enter as extra terms.
- EDF meets every deadline whenever utilisation is at most 1 (with deadlines equal to periods), but its behaviour in overload is hard to predict.
- Without preemption, a task is blocked by the longest lower-priority job, and the worst case may be a later job in the busy period, not the first; analyses that check only the first job are optimistic.
- Priority inversion lets medium-priority work delay a high-priority task indefinitely through a lock; priority inheritance bounds it, the ceiling protocol and SRP bound it to one critical section and prevent deadlock.
- PicoCalc OS's lane bounds every segment's blocking by one lower-priority segment, whose length the budgets bound; the guarantee that follows (about 806 ms from key to publish, no FIFO overflow for human typing) is correct but far weaker than measured behaviour, and the 10 kHz bus rules out a hard 20 ms poll on this board.

The analysis of this chapter rests on two properties that it took for granted: that a segment really is stopped at its budget, and that a world that misbehaves cannot affect others except by occupying the lane. The first is a question of resource control, the subject of Chapter 11. The second is a question of isolation, which Part III takes up next, starting from the hardware the isolation is built on in Chapter 6.

EXERCISES

1. BASIC Compute the Liu–Layland bound for $n = 1$ to 6 to three decimal places, and the utilisation of a task set with $C = (1, 1, 2)$ and $T = (3, 5, 10)$. Does the bound decide its schedulability under rate monotonic priorities?
2. BASIC For the set of Exercise 1, find each task's worst-case response time by iteration under rate monotonic priorities, showing every step.

3. ANALYSIS Find a set of two tasks with utilisation below 1 that rate monotonic priorities cannot schedule and EDF can, different from the one in Section 5.6. Show the miss under rate monotonic priorities with a Gantt chart.

4. ANALYSIS For the task set of Table 5.1, list every minor-frame length f that satisfies the three cyclic-executive conditions, and build the table for one of them other than $f = 4$.

5. ANALYSIS Repeat the non-preemptive analysis of Section 5.7 for task B of the counterexample. Is B schedulable? How many jobs does its busy period contain?

6. ANALYSIS In the priority inversion scenario with tasks H, M and L, suppose L's critical section lasts 2 ms and M computes for 50 ms. Give H's worst-case blocking with no protocol, with priority inheritance, and with the priority ceiling protocol. Now let H also use a second lock shared with M; how does chained blocking change the inheritance figure?

7. BASIC Explain why the Mars Pathfinder resets stopped when priority inheritance was enabled, naming the three tasks and the resource involved.

8. ANALYSIS Recompute the per-segment bound and the end-to-end bound of Section 5.11 with every app's budget set to 15 ms and the sandbox's to 20 ms. Which term now dominates?

9. CODE Using the host build, give the sandbox a 1 s loop to evaluate (it will be stopped at its 200 ms budget) and press a key during it. Record a trace and measure the time from the key press to the keyboard driver's `publish`. Compare it with the per-segment bound.

10. CODE Find the worst-case number of FIFO entries that accumulate in the STM32 model during a REPL loop of 200 ms when keys are pressed every 50 ms, by scripting the host build and reading the driver's `stats` (events per poll). Does any event overflow?

11. DESIGN Sketch a LET version of the lane in which a driver's effects are applied at the next multiple of its period instead of at the end of its segment. What does it guarantee that the current lane does not, and what does it cost in latency for the keyboard path?

12. DISCUSSION The budgets in `js/system.json` are fault thresholds chosen generously. Argue for and against deriving them instead from measured segment lengths plus a margin, considering what happens when a world's code changes after the budgets were set.

13. PROJECT Implement non-preemptive response-time analysis, including the level-i busy period, as a script that reads `js/system.json`, takes observed segment lengths from the traces in `bench/traces/`, and prints a bound per driver. Validate it against the counterexample of Figure 5.2.

FURTHER READING

[Liu and Layland 1973] The founding paper: the task model, the critical instant, rate monotonic optimality, the utilisation bound and EDF. Short, precise and still the best introduction to its own results.

[Audsley et al. 1993] Response-time analysis with deadlines, jitter and blocking, written for engineers applying it.

[Davis et al. 2007] How a widely used analysis of non-preemptive scheduling was wrong, why, and how to fix it; essential for anyone analysing a system like the lane.

[Sha et al. 1990] Priority inheritance and the priority ceiling protocol, with their proofs.

[Reeves 1997] The Pathfinder account from the engineer who led the investigation, including what tracing on the ground replica showed.

[Wilhelm et al. 2008] What it takes to know a worst-case execution time, and why measurement alone does not.

CHAPTER 6 PART II · EMBEDDED AND REAL-TIME

Memory in small systems

On a desktop, running low on memory is a slowdown: the kernel pages to disk, the allocator asks for more address space, and the user notices the fan before anything fails. On a microcontroller there is no disk to page to and no address space to grow into, so running out of memory is a failure, and it happens at a definite moment, inside a definite allocation. This chapter explains how small systems manage memory so that the moment can be predicted, bounded and contained. By the end you should be able to explain what address translation would give a microcontroller and what it does without, how heap allocators work and why they fragment, what garbage collection costs and when it runs, and exactly how PicoCalc OS gives each world a heap with a hard limit and measures where its 2.5 MB of world heaps go.

The chapter follows one question through four layers. A world in PicoCalc OS creates a JavaScript object. That object is a block in QuickJS's own small-object allocator, which takes 4 KB arenas from the nucleus's accounting allocator, which takes memory from the C library's allocator, which on the device takes it from PSRAM through ESP-IDF's heap. Each layer adds a reason for memory to be used but not useful, and each layer is where a limit could be enforced. The answer to "how much memory does this world use?" depends on which layer you ask.

6.1 Address spaces, and what a microcontroller lacks

A process on a desktop operating system sees an **address space**: a range of virtual addresses that the memory management unit translates, page by page, to physical memory. Translation buys five things at once. It isolates processes, because a process cannot name another's pages. It relocates them, because every program can be linked at the same virtual address. It makes memory appear contiguous, because a large array

can occupy pages scattered across physical memory. It lets the kernel allocate lazily, mapping a page only when it is first touched. And it lets the kernel page memory out to disk and back, so that the sum of all processes' memory can exceed the machine's.

The ESP32-P4 has none of this for ordinary data. Its RISC-V cores have no supervisor mode and no page tables (Chapter 2), so every address is physical, and the only protection available is a small number of PMP regions. A cache does map external flash and PSRAM into the address space, but that mapping is set up once and shared by everything running on the chip. The consequences reach every layer of a small system's memory management:

- There is one heap, or a few heaps by kind of memory, shared by everything. Two components' allocations interleave in the same address range.
- Fragmentation of physical memory is visible to programs. A 200 KB allocation needs 200 KB of contiguous free memory; there is no translation to paper over holes.
- Allocation is eager. Reserving memory is using it.
- A limit cannot be a region. Without translation, a component's memory is not a separate range that can be capped by its size; it is a set of blocks scattered through the shared heap, and a cap on it has to be a counter that the allocator checks.

Figure 6.1 shows the last point for PicoCalc OS. Three worlds' heaps are made of blocks in the same PSRAM, interleaved with each other and with the nucleus's own allocations. What bounds each world is not where its blocks are but how many bytes the nucleus has counted against it.

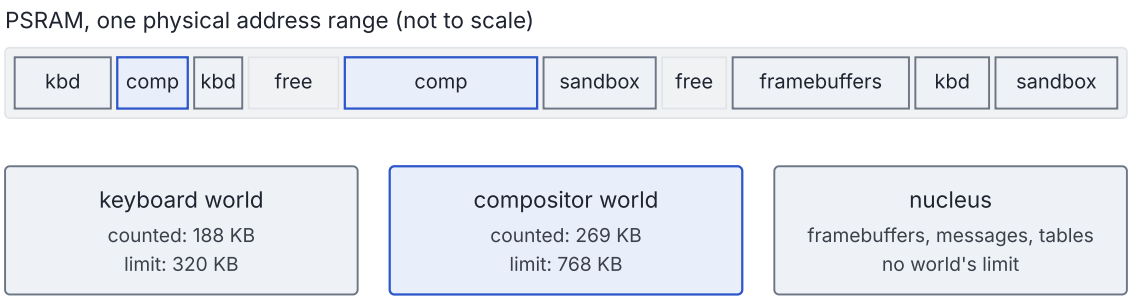


Figure 6.1. Without address translation, worlds share one physical heap. Each world's blocks are scattered through PSRAM among other worlds' and the nucleus's; a world's limit is a count of bytes kept by the allocator, not a region of memory.

The figure also shows what PicoCalc OS's limits do not cover, which Section 6.5 returns to: the framebuffers and the messages waiting in mailboxes belong to the nucleus, not to any world.

6.2 Static allocation, and why embedded systems prefer it

The first answer embedded systems give to the risk of running out is not to allocate at run time at all. If every buffer, every task stack and every queue is sized when the program is built, the linker reports whether it all fits, and nothing can fail later. FreeRTOS's documentation states the trade plainly: dynamic allocation reduces planning and RAM footprint, while static allocation is more deterministic, removes the need to handle allocation failures, and removes the risk of heap fragmentation [Barry 2016, ch. 3].

When an embedded system does allocate dynamically, it usually chooses its allocator with the same priorities. FreeRTOS ships five heap implementations, and they are a compact survey of the design space [Barry 2016, ch. 3]:

Table 6.1. FreeRTOS's heap implementations, from the FreeRTOS kernel book, and what each trades away.

Scheme	How it allocates	Frees?	Deterministic?	Fragmentation
heap_1	carves blocks off one static array	no	yes	none, since nothing is freed
heap_2	best fit over a free list	yes, but never merges neighbours	no	high when block sizes vary
heap_3	wraps the C library's `malloc` and `free`	yes	no	whatever the library does
heap_4	first fit, merges adjacent free blocks	yes	no	reduced by coalescing
heap_5	heap_4 across several separate memory regions	yes	no	as heap_4

heap_1 is the instructive extreme. It never frees, so it can never fragment and its time is constant; it is suitable for systems that create everything at start-up and never destroy anything, which describes many safety-critical systems.

PicoCalc OS's nucleus follows the static style for its own tables. Worlds, handles, DMA buffers and outstanding asks live in fixed arrays whose sizes are compile-time constants:

▸ nucleus/nucleus.h

```
#define MAX_WORLDS 32
…
#define MAX_HANDLES 128
…
#define MAX_BUFS 16
…
#define MAX_ASKS 1024
```

A world that asks for a 129th handle gets an error ("out of handles") rather than a growing table, and the nucleus's memory for these tables is known when it is built. The worlds themselves cannot be static: a JavaScript program allocates as it runs, and its garbage is reclaimed only by a collector. The design question for PicoCalc OS was therefore not whether to allocate dynamically but how to keep each world's dynamic allocation from affecting the others.

6.3 Allocators and fragmentation

A heap allocator manages a range of memory as a set of blocks, each either in use or free. To allocate *n* bytes it finds a free block of at least *n* bytes, splits off the remainder if it is large enough to be useful, and returns the rest. To free, it marks the block free and, ideally, merges it with free neighbours. Most allocators keep a small header before each block recording its size and whether it is free, and a footer or a flag that lets the block's successor find it. These **boundary tags**, which go back to Knuth's treatment of dynamic storage, make merging neighbours a constant-time operation [Wilson et al. 1995].

The policy question is which free block to use. **First fit** takes the first block large enough, scanning from the start of a list. **Best fit** takes the smallest block large enough. **Segregated fits** keep separate free lists for ranges of sizes and look only in the list that can satisfy the request. Wilson, Johnstone, Neely and Boles's survey remains the standard account of these policies and of the evidence about them; its central argument is that real programs' allocation patterns are far from random, so studies that measured policies on random traces misjudged them, and the evidence it reviews suggests that simple policies such as best fit fragment little on real programs [Wilson et al. 1995].

Fragmentation is memory that is free but cannot be used. *Internal* fragmentation is space inside allocated blocks that the program did not ask for: rounding a 20-byte request up to a 24-byte size class wastes 4 bytes. *External* fragmentation is free memory split into pieces too small for the requests that arrive: a heap can have 40 KB free in total and fail a 4 KB allocation because no single hole is 4 KB long. Without address translation, external fragmentation cannot be repaired by moving pages; only a compacting garbage collector, which moves objects and updates every reference to them, can close holes, and QuickJS does not compact.

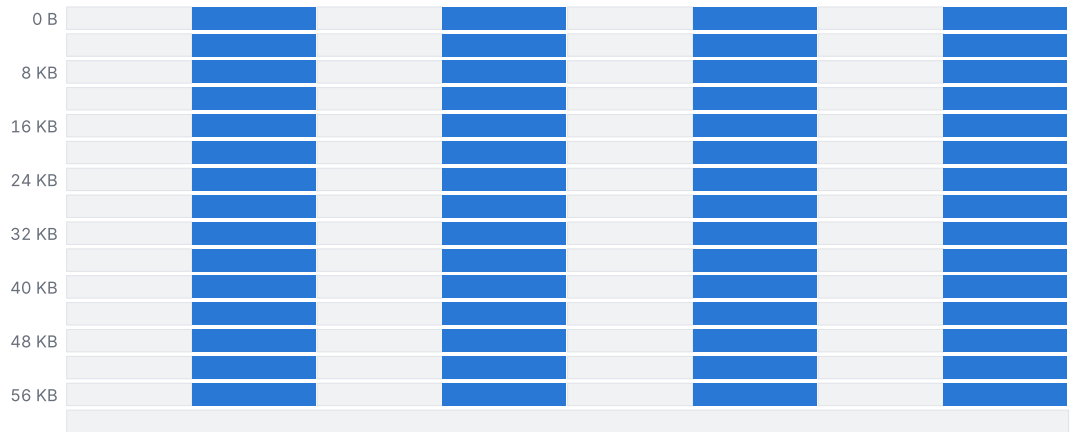
TRY 6.1

Watch a heap fragment

The arena below is 64 KB. The "fragmenting" workload allocates blocks of mixed sizes and frees every other one, which leaves many small holes. The widget opens at the first allocation that fails under first fit; in its run, 34 KB is free in all but the largest free block is 4 KB, so a request larger than 4 KB cannot be met. It compares the same workload under best fit and the segregated (TLSF-style) policy. Step through the run to see where the holes come from, then switch to the "mixed" and "growing" workloads. Which policy lasts longest on each? The difference is the subject of Wilson's survey: a policy's quality depends on the workload, not on the policy alone.

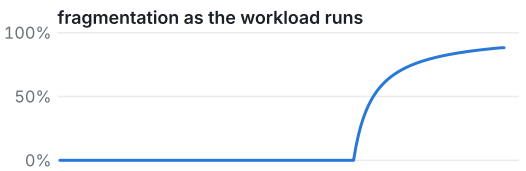
181 / 187: allocate 8 KB (one large block)

failed: 34 KB free in all, but the largest block is 4 KB



64 KB arena, 4 KB per row; allocated blocks in blue, the one just touched in orange

allocated	30 KB
free	34 KB
free blocks	61
largest free block	4 KB
fragmentation, 1 − largest/free	88%
failed allocations	1



The same workload, run to the end under each policy:

Policy	failed	largest free	free blocks	fragmentation
First fit	1	3.4 KB	60	89%

Policy	failed	largest free	free blocks	fragmentation
Best fit	1	3.4 KB	60	89%
Segregated fit (TLSF)	1	3.4 KB	60	89%

Real-time systems add a requirement that general-purpose allocators do not meet: a bound on the time of each operation. First fit's scan is as long as the free list. Masmano, Ripoll, Crespo and Real's **TLSF** (Two-Level Segregated Fit) allocator achieves constant-time allocation and freeing with low fragmentation [Masmano et al. 2004]. It keeps an array of free lists indexed by two levels: the first level by the position of the request size's highest set bit (a power-of-two range), the second by the next few bits (a linear subdivision of that range). Two bitmaps record which lists are non-empty, so finding a suitable list is a pair of find-first-set instructions rather than a search. In outline (a sketch of the algorithm, not ESP-IDF's code):

```
# TLSF's mapping from a request size to a free-list index (SL_BITS = 5: 32 lists per
power of two)
def mapping(size):
    fl = highest_set_bit(size)                  # first level: which power of two
    sl = (size >> (fl - SL_BITS)) & (2**SL_BITS - 1)   # second level: which 1/32 of it
    return fl, sl

def malloc(size):
    size = round_up_to_next_list(size)          # so any block in the list found is large
enough
    fl, sl = mapping(size)
    sl_map = second_level_bitmap[fl] & (~0 << sl)
    if sl_map == 0:                             # nothing in this power of two: go up a
level
        fl = find_first_set(first_level_bitmap & (~0 << (fl + 1)))
        sl_map = second_level_bitmap[fl]
    sl = find_first_set(sl_map)
    block = remove_head(free_list[fl][sl])
    split(block, size)                          # the remainder goes back on its own list
    return block
```

ESP-IDF, the framework PicoCalc OS will run on, builds its heaps on TLSF: its `multi_heap` layer, which turns each contiguous region of memory into a heap, calls `tlsf_malloc`, `tlsf_free` and `tlsf_memalign_offs` from a TLSF implementation kept as a submodule of the heap component [ESP-IDF multi-heap 2026]. On top of that, ESP-IDF tags each region with **capabilities** (internal memory, DMA-

capable memory, PSRAM, 32-bit-only memory), and `heap_caps_malloc` chooses a heap by the capabilities the caller asks for [ESP-IDF heap 2026]. A DMA buffer, for example, must be allocated with the DMA capability; the documentation notes that on the chips it describes this excludes external PSRAM, which is why Chapter 2 left open whether the framebuffer can be sent from PSRAM directly.

On the host build, the C library's allocator plays this role; in the WebAssembly build of Chapter 17, WASI's. That is one of the reasons memory figures differ slightly between builds: the nucleus counts what the underlying allocator says a block's usable size is, and allocators round differently.

6.3.1 The allocator inside the engine

QuickJS adds a layer of its own. The version PicoCalc OS uses (2026-06-04) does not pass every small allocation to the host allocator. It keeps 31 size classes from 16 to 512 bytes, carves blocks of each class out of 4 KB arenas obtained from the host allocator, and returns an arena only when its last block is freed; anything larger than 512 bytes goes to the host allocator directly.

▸ third_party/quickjs/quickjs.c

```
#define JS_MALLOC_ALIGN 8
#define JS_MALLOC_ARENA_SIZE 4096
#define JS_MALLOC_BLOCK_SIZE_COUNT 31
#define JS_MALLOC_MIN_SMALL_SIZE 16
#define JS_MALLOC_MAX_SMALL_SIZE 512
#if defined(__SANITIZE_ADDRESS__)
/* use the host malloc() for all allocations */
#define JS_MALLOC_LARGE_BLOCKS_ONLY 1
```

This is a segregated-fit allocator with fixed-size classes, a design that makes small allocations fast and keeps objects of one size together. Its cost is internal fragmentation at two levels: within a block, a request is rounded up to its class (the source's comment bounds this at 12.5% for sizes of 64 bytes and more), and within an arena, free blocks of one class cannot be used for another. A runtime that has touched all 31 classes holds at least 31 arenas, each partly empty.

How large is that cost? QuickJS can report its own memory use, and a short program in the repository, `tools/memdump.c`, measures it. It evaluates a file in a fresh runtime (with stand-ins for the nucleus's two host functions, so that the prelude can run), collects garbage, and prints QuickJS's own counts:

▸ tools/memdump.c

```c
// memdump.c: what QuickJS holds after evaluating a file, by its own accounting
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "quickjs.h"

int main(int argc, char **argv) {
  JSRuntime *rt = JS_NewRuntime();
  JSContext *ctx = JS_NewContext(rt);
  // stand-ins for the nucleus's two host functions, enough for the prelude to evaluate
  const char *stub = "globalThis.__k = (op) => op === 'seed' ? 1 : op === 'now' ? 0 :"
                     " op === 'self' ? 'probe' : undefined; globalThis.__ka = () =>
Promise.resolve();";
  JS_FreeValue(ctx, JS_Eval(ctx, stub, strlen(stub), "stub.js", JS_EVAL_TYPE_GLOBAL));
  FILE *f = fopen(argv[1], "rb");
  static char src[1 << 20];
  size_t n = fread(src, 1, sizeof src - 1, f);
  JS_FreeValue(ctx, JS_Eval(ctx, src, n, argv[1], JS_EVAL_TYPE_GLOBAL));
  JS_RunGC(rt);
  JSMemoryUsage u;
  JS_ComputeMemoryUsage(rt, &u);
  printf("allocated %lld bytes in %lld blocks; used %lld bytes in %lld blocks\n",
         (long long)u.malloc_size, (long long)u.malloc_count,
         (long long)u.memory_used_size, (long long)u.memory_used_count);
  printf("atoms %lld (%lld bytes), objects %lld, shapes %lld (%lld bytes)\n",
         (long long)u.atom_count, (long long)u.atom_size, (long long)u.obj_count,
         (long long)u.shape_count, (long long)u.shape_size);
  return 0;
}
```

Linked against the 32-bit build's QuickJS objects, and a second time against a `quickjs.c` compiled with `-D__SANITIZE_ADDRESS__`, which turns the arena layer off (see the excerpt above), it gives for the prelude:

```
$ cc -m32 -O2 -Ithird_party/quickjs tools/memdump.c build32/qjs/*.o -lm -o memdump-arenas
$ ./memdump-arenas js/prelude.js
allocated 186236 bytes in 59 blocks; used 90054 bytes in 1886 blocks
atoms 784 (22657 bytes), objects 270, shapes 135 (19984 bytes)
$ ./memdump-no-arenas js/prelude.js
allocated 135896 bytes in 1894 blocks; used 90054 bytes in 1886 blocks
atoms 784 (22657 bytes), objects 270, shapes 135 (19984 bytes)
```

The runtime's own data is the same in both: 90,054 bytes in 1,886 blocks, of which the atom table (the runtime's interned strings: property names, built-in identifiers) is 22.7 KB and object shapes 20 KB. Passed straight to the C library, those blocks occupy 135,896 bytes, the difference being the library's per-block overhead and rounding. Through QuickJS's arenas they occupy 186,236 bytes in 59 large blocks: 50 KB more, about 27% of the total. The first figure, 186,236 bytes, is exactly what the nucleus's own probe reports for a world with the prelude (`memprobe` in the host build), so the arenas are part of every world's fixed cost. For an empty runtime the effect is larger: the nucleus's probe reports 146,884 bytes, while the program above, run on an empty file without arenas, allocates about 78 KB. Nearly half of an empty runtime's footprint is free space inside arenas.

This is a textbook case of a trade that is right in one setting and costly in another. Arenas suit a single large runtime, where 31 partly empty arenas are noise; they cost more when a system runs thirteen small runtimes, each paying the same fixed slack. It also explains a detail from the paper's evaluation: the AddressSanitizer build, which defines `__SANITIZE_ADDRESS__`, turns the arena layer off, which is one reason its heap figures differ from the normal build's.

6.4 Garbage collection

A JavaScript program never frees memory. The engine decides when an object can no longer be reached and reclaims it, and how it decides determines both how much memory is wasted and when the program pauses.

There are two families of collectors. A **tracing** collector periodically starts from the roots (globals, the stack, registers) and marks everything reachable; whatever is unmarked is garbage. Tracing handles any shape of object graph, but each collection touches every live object, and unless the collector is incremental or concurrent, the program stops while it runs. A **reference-counting** collector keeps a count in each object of the references to it, increments and decrements the count as references are created and destroyed, and frees an object the moment its count reaches zero. Reclamation is immediate and spread across the program's run, but every pointer store costs a count update, and objects that refer to each other in a cycle keep each other's counts above zero forever.

QuickJS uses both. Its documentation states it directly: reference counting frees objects automatically and deterministically, and a separate cycle removal pass runs when the allocated memory becomes too large [Bellard 2026]. The cycle pass uses **trial deletion**, the technique Bacon and Rajan's synchronous cycle collector also uses [Bacon and Rajan 2001]: for every object that can participate in a cycle, subtract the references that come from other such objects; whatever is left with a count of zero is referenced only from within the set, and is garbage unless something with a positive count reaches it.

Figure 6.2 shows the three passes as QuickJS's source names them.

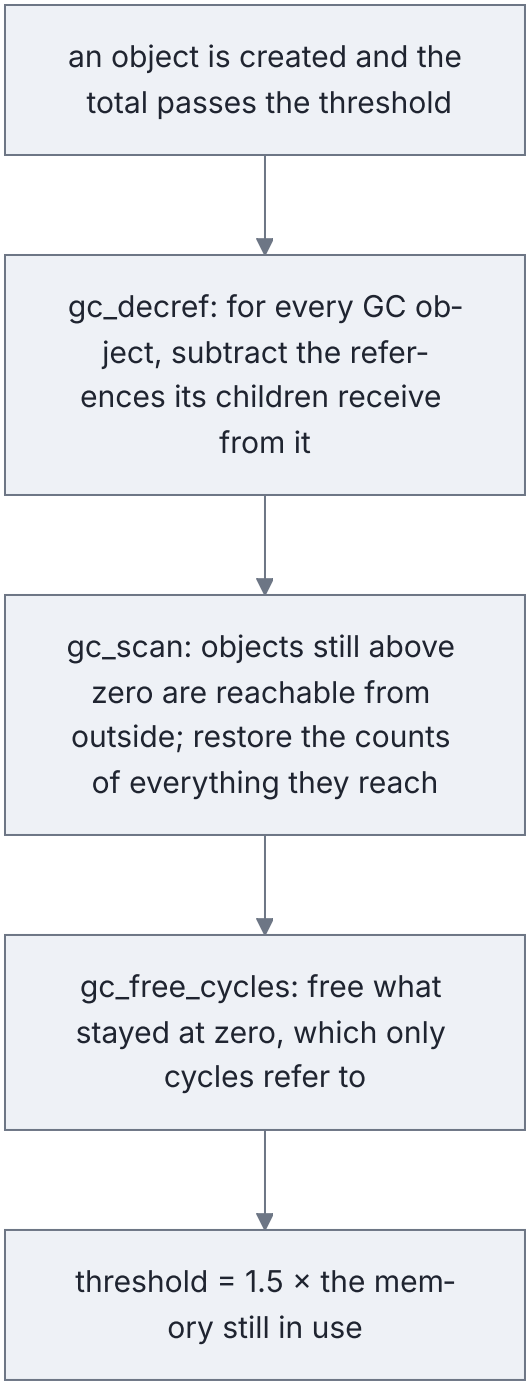

an object is created and the total passes the threshold
gc_decref: for every GC object, subtract the references its children receive from it
gc_scan: objects still above zero are reachable from outside; restore the counts of everything they reach
gc_free_cycles: free what stayed at zero, which only cycles refer to
threshold = 1.5 × the memory still in use

Figure 6.2. QuickJS's cycle removal, run when the runtime's allocated memory passes a threshold. Ordinary garbage never gets here: reference counting has already freed it.

The threshold logic is short enough to quote. A runtime starts with a threshold of 256 KB, and after each collection the threshold is set to one and a half times the memory still allocated:

▸ third_party/quickjs/quickjs.c

```
static void js_trigger_gc(JSRuntime *rt, size_t size)
{
    BOOL force_gc;
#ifdef FORCE_GC_AT_MALLOC
    force_gc = TRUE;
#else
    force_gc = ((rt->malloc_ctx.malloc_state.malloc_size + size) >
                rt->malloc_gc_threshold);
#endif
    if (force_gc) {
…
        JS_RunGC(rt);
        rt->malloc_gc_threshold = rt->malloc_ctx.malloc_state.malloc_size +
            (rt->malloc_ctx.malloc_state.malloc_size >> 1);
    }
}
```

The check is made only when a new object is created. An allocation of a string or an array's storage does not trigger it, and an allocation that fails is not retried after a collection. For PicoCalc OS's worlds this matters: a driver world sits at about 188 KB with a 320 KB limit, so its first cycle collection happens only when its allocated memory passes 256 KB, and if 188 KB is still in use after that collection, the next threshold, 1.5 × 188 KB ≈ 282 KB, still leaves room below the limit. A world whose limit were close to 256 KB and whose garbage were mostly cyclic could reach its limit before the collector ran (exercise 6 asks you to construct one).

6.4.1 Collection and time

A collector's pauses are a real-time problem: if collection can stop the program for an unpredictable time, no deadline can be guaranteed. Bacon, Cheng and Rajan's Metronome collector addressed this for Java by scheduling collection work in small, time-based quanta, so that over any window of a chosen length the

program was guaranteed a minimum share of the processor (its minimum mutator utilisation) [Bacon et al. 2003]. That is one answer: make the collector incremental and schedule it like any other real-time task.

PicoCalc OS's structure gives a different answer, by size rather than by scheduling. Each world is its own runtime with its own heap, so a cycle collection in the compositor scans the compositor's objects and nothing else: at most a few hundred kilobytes, never the 2.5 MB of all worlds together. The pause happens inside the world's own segment, on the lane, where the lane's non-preemptive scheduling already accounts for segments as indivisible (Chapter 5). Reference counting means that most garbage is freed at once and never reaches the cycle pass.

There is a gap in the model here, and it is better to state it than to hide it. The virtual clock charges a segment for interpreter steps and for the native work of specific host calls (Chapter 18), but QuickJS's collection is neither: it is native code inside the engine, and the host build charges it nothing. The same is true of the allocator. In the paper's out-of-memory experiments, the segment in which a world fills a 256 KB limit lasts 0.09 virtual ms, which is short only because allocation and collection are free on the virtual clock. On the device, both will cost real time, and the budget check, which reads the virtual cost, will not see them until the port charges them. Measuring collection pauses for each world on the ESP32-P4 is part of the port plan in Chapter 7.

6.5 Bounding a world's memory

QuickJS lets an embedder supply its own allocator when it creates a runtime, through `JS_NewRuntime2` [Bellard 2026]. The nucleus uses that to give every world an allocator that counts and refuses:

▸ nucleus/vm.c

```
static size_t host_limit(world_t *W) { return (size_t)((double)W->mem_limit *
PROFILE.mem_scale); }

static void *w_malloc(JSMallocState *s, size_t size) {
  world_t *W = s->opaque;
  if (W->mem_used + size > host_limit(W)) { W->oom = true; return NULL; }
  void *p = malloc(size);
  if (!p) return NULL;
  size_t u = malloc_usable_size(p) + 8;
  W->mem_used += u; s->malloc_size += u; s->malloc_count++;
  return p;
}
```

Each world's runtime is created with this allocator and the world's record as its opaque pointer, so every byte QuickJS takes from the host (including its 4 KB arenas) passes through `w_malloc` and is counted against that world alone. The count is the block's usable size plus 8 bytes, an estimate of the host allocator's header. The limit comes from the manifest's `mem_kb`, scaled on 64-bit hosts by `mem_scale` (1.17, measured), because 64-bit pointers make the same program's heap about 17% larger than on the 32-bit device.

When an allocation would cross the limit, `w_malloc` sets the world's `oom` flag and returns `NULL`. QuickJS turns the `NULL` into an out-of-memory exception, which JavaScript code could in principle catch. The nucleus does not let that matter. The interrupt handler stops the world at its next check when the flag is set, and when the segment ends, the nucleus classifies it as a crash whatever the JavaScript did:

▸ nucleus/kernel.c

```
  if (W->oom) snprintf(why, sizeof why, "ran out of memory (its limit is %zu KB)", W-
>mem_limit / 1024);
```

Running out of memory is thus fail-stop: a world cannot catch the failure and continue with half-built data structures. Its runtime is disposed of, which frees everything the runtime held, since once its context is gone nothing in it is reachable, and the supervisor restarts it with a fresh one (Chapter 14). This is one of the strongest arguments for a runtime per component. Freeing a component's memory after a failure requires knowing which memory was the component's, and with separate runtimes the answer is exact.

Here is the Faults app's allocation experiment on the 32-bit host build. It asks the sandbox to evaluate `const a = []; for (;;) a.push('x'.repeat(8192) + a.length)`, and the nucleus writes this to its console log:

```
[  2536.082] sandbox crashed handling eval: ran out of memory (its limit is 512 KB)
#OCT1
{"ev":"fault","t":2536.082,"world":"sandbox","file":"system/sandbox.js","handling":"eval"
,"why":"ran out of memory (its limit is 512 KB)","crashes":2,"mem_kb":181,"limit_kb":512}
```

The `#OCT1` line is the machine-readable fault frame Chapter 7 describes. Its `mem_kb` field is worth a second look: the world hit 512 KB, but the frame reports 181 KB in use. By the time the nucleus handles the crash, the evaluation that built the array has unwound, the array is unreachable, and reference counting has already freed it. The limit field records what was hit; the memory field records what was left.

6.5.1 What the limits do not cover

A per-world count bounds what passes through `w_malloc`. Two kinds of memory a world causes to be allocated do not, and both belong to the nucleus.

The first is framebuffers. A world with the `dma` claim (in the standard manifest, only the compositor) can allocate a framebuffer through `dma.alloc`; the nucleus allocates its back and front buffers with plain `calloc`, outside any runtime:

▸ nucleus/hw.c

```
int dmabuf_alloc(int w, int h, world_t *owner) {
  for (int i = 0; i < MAX_BUFS; i++) {
…
    b->bytes = (size_t)w * h * 2;
    b->back = calloc((size_t)w * h, 2);
    b->front = calloc((size_t)w * h, 2);
```

The compositor's 320 × 320 framebuffer therefore costs 409,600 bytes that appear in no world's memory column. What bounds them is the claim (a world without `dma` cannot allocate any), the 640 × 640 maximum size, and the table of 16 buffers. In the worst case, a world with the claim could allocate 16 buffers of 640 × 640, 26 MB, which would exhaust the PSRAM. The claim is the real limit, and it rests on the manifest giving `dma` only to a trusted world; Chapter 11 discusses how to close this gap by charging buffers to their owner. The first pass, a later revision described in a separate book (*From L4 to Small Worlds*, Chapter 7), narrowed the worst case: a world may hold only as many framebuffers as its manifest entry's `bufs` quota (1 unless the entry says otherwise; the compositor's is 2), and the boot fails unless the quotas of the worlds with the claim add up to at most 16. The pixels are still not charged to any world's heap limit.

The second is mailboxes. A message sent to a world is copied into a nucleus allocation and queued in the recipient's mailbox until the recipient handles it. The mailbox has no maximum length. A world that sends faster than its recipient handles makes the nucleus allocate without bound, and neither world's limit notices. The nucleus records each world's mailbox peak (it appears in `sys.worlds()`) for this reason; Chapter 12 discusses backpressure, the usual cure. The first pass bounded the mailbox: each world's entry has an `mbox` quota of messages and an `mbox_kb` quota of bytes (64 and 128 KB by default), counted from the moment a message is sent; a send or ask to a full mailbox fails at the sender, and a channel post to a full subscriber is dropped and counted (*From L4 to Small Worlds*, Chapter 7).

6.6 Measuring memory: where 2.5 MB goes

With the accounting understood, the measurements can be read. The host build's `stats` command prints each world's counted memory (divided by `mem_scale` on 64-bit builds, so the figures are comparable with the 32-bit device). On the 32-bit build, after one second of running:

```
$ printf 'wait 1000\nstats\nmemprobe\nquit\n' | ./build32/picocalc-host -
t = 1000.0 ms, lane busy 15.2 ms (1.5%), 73 turns, 4 published, 10 delivered, 0 crashes
  world       kind     state        turns    cpu ms   mem KB  limit  rst  last error
  keyboard    driver   idle            46      5.18    187.9    320    0
  power       driver   idle             1      0.30    185.7    320    0
  display     driver   idle             4      2.90    183.7    320    0
  compositor  service  idle             9      4.61    257.3    768    0
  launcher    app      idle             1      0.28    190.2    384    0
  sysmon      app      idle             2      0.21    201.4    512    0
  repl        app      idle             0      0.06    184.8    384    0
  sandbox     service  idle             0      0.06    181.9    512    0
  snake       app      idle            10      1.31    197.8    320    0
  faults      app      idle             0      0.07    199.0    384    0
  flaky       service  idle             0      0.06    181.9    256    0
  keys        app      idle             0      0.07    183.8    320    0
  about       app      idle             0      0.07    183.9    320    0
  bus i2c0  busy   283.80 ms, 47 transfers, 190 bytes, 0 errors
  bus spi2  busy    64.02 ms, 38 transfers, 318842 bytes, 0 errors
RESULT {"bench":"mem","pointer_bits":32,"empty_bytes":146884,"prelude_bytes":186236}
```

The `memprobe` line gives the two fixed costs: an empty runtime with one context, 146,884 bytes (143.4 KB), and a runtime with the prelude evaluated, 186,236 bytes (181.9 KB). Every world pays these before running a line of its own code. The sandbox and the flaky service, which do almost nothing, sit exactly at the prelude's 181.9 KB.

The paper's memory experiment measures the worlds after every app has been opened once, which is a better estimate of steady state. Table 6.2 summarises it.

Table 6.2. Heap per world on the 32-bit host build after every app has been opened once (from the paper's evaluation).

Quantity	Value
Empty runtime and context	143 KB
With the prelude	182 KB (+38 KB)
Smallest world	182 KB
Median world	190 KB
Largest world (the compositor)	269 KB
Worlds whose own code and state take under 20 KB	10 of 13
All 13 worlds together	2.5 MB
Sum of their limits	5.0 MB
PSRAM on the board	32 MB

Three conclusions follow. The first is that the fixed cost dominates. For ten of the thirteen worlds, the world's own code and state are under 20 KB; the other 180 KB are the engine's built-in objects, atom table, shapes and arena slack, repeated thirteen times. The compositor, at 87 KB of its own, is the exception, because it keeps the latest frame of every app it has shown.

The second is that the budget is comfortable. Thirteen worlds take 2.5 MB of 32 MB of PSRAM. At about 190 KB each, a hundred worlds would fit, and the limits, which sum to 5.0 MB, can be generous without risk. On a chip with a few megabytes of PSRAM, such as the ESP32-S3, the arithmetic is different, and an engine built for tiny heaps would be the realistic choice for small worlds: Bellard's MicroQuickJS runs a subset of JavaScript close to ES5 in as little as 10 kB of RAM, with a tracing collector instead of reference counting [Bellard 2025].

The third is where to look for savings, in order of size. Sharing the prelude's compiled bytecode and the built-in objects between runtimes would remove most of the fixed cost, but QuickJS keeps all of these per runtime, so it needs a change to the engine. Shrinking or disabling the arena layer for small runtimes would save roughly 50 KB per world, about 650 KB across thirteen, at some cost in allocation speed that would have to be measured on the device. The 64-bit host's figures, scaled by 1.17, agree with the 32-bit build's to

within about 6% for every world, though the empty runtime alone differs by 11% (128 against 143 KB): the factor is a fair average, not a law, because pointer-sized fields are a different share of each world's heap and the two hosts' allocators round differently.

KEY POINTS

- Without address translation, all components share one physical heap: a component's memory is a set of scattered blocks, and a limit on it has to be a counter in the allocator, not a region.
- Embedded systems prefer static allocation because it cannot fail or fragment at run time; PicoCalc OS's nucleus keeps its worlds, handles, buffers and asks in fixed tables for the same reason.
- Allocators trade speed, determinism and fragmentation: first and best fit search free lists, segregated fits index them by size, and TLSF does both levels of indexing with bitmaps to allocate in constant time, which is why ESP-IDF's heap uses it.
- QuickJS 2026-06-04 adds a size-class arena allocator inside the engine; it costs about 50 KB of slack per world with the prelude, roughly a quarter of each world's fixed footprint.
- QuickJS frees most garbage immediately by reference counting and removes cycles by trial deletion when allocated memory passes a threshold that starts at 256 KB and is then 1.5 times the live memory.
- A runtime per world bounds every collection to one world's heap and lets the nucleus free all of a failed world's memory at once by disposing of its runtime.
- Running out of memory is fail-stop: once a world's allocator refuses, the segment is a crash whatever the JavaScript caught, and the supervisor restarts the world.
- Framebuffers and queued messages are nucleus memory charged to no world; the `dma` claim and the table sizes bound the former, and nothing yet bounds the latter.
- The virtual clock charges nothing for allocation or collection, so the device port must measure both before budgets can account for them.

Memory and time meet in the firmware that ties a system to its chip: the boot sequence that sets up the heaps, the watchdogs that notice when something stops making progress, and the tools for finding out what happened when it does. Chapter 7 turns to that firmware: how an ESP-IDF application is structured, built

and booted, how it is kept alive, and what moving PicoCalc OS from the host build onto the ESP32-P4 will involve.

EXERCISES

1. BASIC List the five things address translation gives a desktop operating system, and for each say what PicoCalc OS uses instead, or that it goes without.
2. BASIC A heap has free blocks of 5, 200 and 100 bytes, in that address order. Show which block first fit and best fit each choose for a request of 90 bytes, then for a following request of 150 bytes. Which policy satisfies both?
3. BASIC Using the size classes in `quickjs.c`, what block size does QuickJS use for a 100-byte request (including its header) and for a 300-byte one? What is the internal fragmentation of each?
4. ANALYSIS For TLSF with 32 second-level lists per power of two, compute the first- and second-level indices for requests of 100, 1,000 and 100,000 bytes. What is the largest internal fragmentation the rounding in `malloc` can cause, as a fraction of the request?
5. ANALYSIS With the prelude, QuickJS's data takes 90,054 bytes in 1,886 blocks, and the arenas hold them in 59 host blocks totalling 186,236 bytes. Estimate how many of those host blocks are 4 KB arenas and their average occupancy. Why is the arena overhead a smaller fraction of the total with the prelude than for an empty runtime?
6. ANALYSIS Construct a world, with a memory limit and a JavaScript loop, that reaches its limit even though all of its garbage is collectable, because QuickJS's cycle collection is not triggered in time. Use the threshold rule in `js_trigger_gc` and the fact that only object creation checks it. Then explain how the nucleus could prevent it.
7. ANALYSIS A world's heap is 190 KB, of which 150 KB is live. Estimate the work of one cycle collection in that world compared with a single shared runtime holding all thirteen worlds' 2.5 MB. Which of QuickJS's three passes is proportional to what?
8. CODE Build the two memory-dump programs described in Section 6.3.1 and reproduce the figures. Then extend the program to evaluate the compositor's file after the prelude (it needs more stand-in host functions) and report how much of its 269 KB is arena slack.
9. CODE Lower the sandbox's `mem_kb` in `js/system.json` to 200 KB and boot the host build. What happens, and what does the `#OCT1` frame say? Explain the result from Table 6.2.

10. CODE Change `w_malloc` so that it also counts the bytes of each world's queued messages against the recipient's limit. What should happen to a sender when the recipient's limit would be exceeded? Implement your choice and test it with a world that sends 10,000 messages in one turn.

11. DESIGN Design a change to the nucleus that charges framebuffers to the world that allocated them, including what happens when a narrowed handle to the buffer is held by another world (see Chapter 9). Who should be charged when the allocating world crashes but another world still holds a handle?

12. PROJECT Add a host call that reports QuickJS's own memory usage (`JS_ComputeMemoryUsage`) for a world, show it in the system monitor next to the counted figure, and use it to find the largest objects in the compositor's heap.

FURTHER READING

[Wilson et al. 1995] The survey of allocation policies and fragmentation; its argument that real allocation patterns have structure is still the right starting point.

[Masmano et al. 2004] The TLSF paper: a constant-time allocator with low fragmentation, and the reasoning that made it the default in several real-time systems and in ESP-IDF.

[Barry 2016, ch. 3] How an RTOS thinks about memory: static allocation, the five heap schemes and their trade-offs, in a few pages.

[Bacon and Rajan 2001] Cycle collection by trial deletion in reference-counted systems, the technique behind QuickJS's cycle removal.

[Bacon et al. 2003] Metronome: how to make a garbage collector compatible with real-time guarantees by scheduling its work.

[Bellard 2026] QuickJS's own documentation, short and precise on runtimes, contexts, memory limits and the interrupt handler.

CHAPTER 7 PART II · EMBEDDED AND REAL-TIME

Embedded systems engineering

The previous chapters described the machine and how its memory is managed. This one describes the software that every program on such a machine sits on: a real-time operating system, a vendor framework of drivers and build tools, a boot sequence that turns a reset into a running application, watchdogs that turn a hang into a restart, and the logging that lets an engineer find out afterwards what happened. By the end of the chapter you should be able to read an ESP-IDF application's structure and boot log, explain what each of the chip's watchdogs protects against, and say precisely what PicoCalc OS would have to change to run on the ESP32-P4 rather than on its simulated board.

That last point needs stating at the start. PicoCalc OS runs today on Linux and in a browser against models of its two peripheral controllers; it has not yet run on the device. This chapter's account of ESP-IDF and FreeRTOS describes the platform the port will use, from Espressif's and FreeRTOS's own documentation, and its account of the port is a plan. Where the plan depends on something nobody has measured, the chapter says so.

7.1 What a real-time operating system provides

Most firmware on a chip like the ESP32-P4 runs on a small real-time operating system, and the most widely used is FreeRTOS. It provides a narrow set of mechanisms, each of which this book discusses in general elsewhere [Barry 2016]:

- **Tasks.** A task is a function with its own stack and a priority. The scheduler runs the highest-priority task that is ready, preempting a lower-priority one the moment a higher-priority one becomes ready (Chapter 4). Tasks of equal priority share the processor in time slices.

- **The tick.** A periodic timer interrupt advances the kernel's notion of time; delays and timeouts are counted in ticks.
- **Queues and semaphores.** A queue passes fixed-size items between tasks, copying them; a task can block on an empty queue until something arrives. Semaphores and mutexes are built on the same mechanism.
- **Interrupt-safe calls.** An interrupt handler may not block, so FreeRTOS gives handlers separate functions (named with `FromISR`) that post to a queue or give a semaphore and report whether doing so woke a task of higher priority than the one interrupted; if it did, the handler requests a context switch on its way out.

The last item defines the standard structure of embedded drivers, which FreeRTOS's book calls deferred interrupt processing. The handler does as little as possible: it acknowledges the device, records the event, and wakes a task. The task does the rest at a priority the designer chose, where it can block, allocate and take as long as it needs. The pattern looks like this (a sketch, not code from this repository):

```
static QueueHandle_t done;                      // carries completed transfers to the
task

static void IRAM_ATTR spi_done_isr(void *arg) {   // runs in interrupt context: short,
never blocks
  BaseType_t woken = pdFALSE;
  struct xfer *x = arg;
  xQueueSendFromISR(done, &x, &woken);          // hand the work to a task
  portYIELD_FROM_ISR(woken);                    // switch now if that task outranks the
interrupted one
}

static void display_task(void *arg) {           // runs as a task: may block, may take
its time
  for (;;) {
    struct xfer *x;
    xQueueReceive(done, &x, portMAX_DELAY);     // sleep until the interrupt hands
something over
    finish_transfer(x);
  }
}
```

Espressif's variant, which ESP-IDF calls IDF FreeRTOS, is based on FreeRTOS 10.5.1 with changes for symmetric multiprocessing on its dual-core chips, the ESP32-P4 among them [ESP-IDF FreeRTOS 2026]. Each core schedules independently; a task can be pinned to one core with `xTaskCreatePinnedToCore` or

allowed on both; and critical sections take a spinlock, because disabling interrupts on one core does not stop the other.

Table 7.1 sets FreeRTOS's mechanisms beside the ones PicoCalc OS gives its worlds. The two systems answer the same questions differently, and the port will stack one on the other: the nucleus will be a FreeRTOS task, and the worlds will run inside it.

Table 7.1. FreeRTOS's mechanisms and PicoCalc OS's answers to the same questions. On the device, the nucleus will be one FreeRTOS task and the worlds will run inside it.

Question	FreeRTOS	PicoCalc OS
Unit of concurrency	task: a stack, a priority, preemptible	world: a separate JavaScript runtime, a priority, a mailbox
When does it give up the processor?	at any instruction (preemption) or when it blocks	only at the end of a segment (run to completion)
How is work bounded?	not bounded; a watchdog notices a task that never yields	per-segment budget, checked by the interpreter
How do units communicate?	queues of fixed-size copied items; shared memory	messages copied between runtimes; no shared memory
Who may touch a device?	any task that has the driver's handle in its address space	only a world holding a handle the nucleus checks
Memory	one heap shared by all tasks, or static	one heap per world with a limit
What happens on a fault?	the panic handler resets the chip	the world is restarted; others run on

The rightmost column trades away FreeRTOS's preemption for isolation and recovery. A FreeRTOS task can be interrupted in the middle of anything, which is what gives an RTOS its response times, and also why its tasks need locks. A world cannot be interrupted, so it needs no locks, but a long segment delays everything behind it; Chapter 5 quantifies that cost, and Chapter 4 discusses running a second lane on the ESP32-P4's other core.

7.2 ESP-IDF: the platform around the RTOS

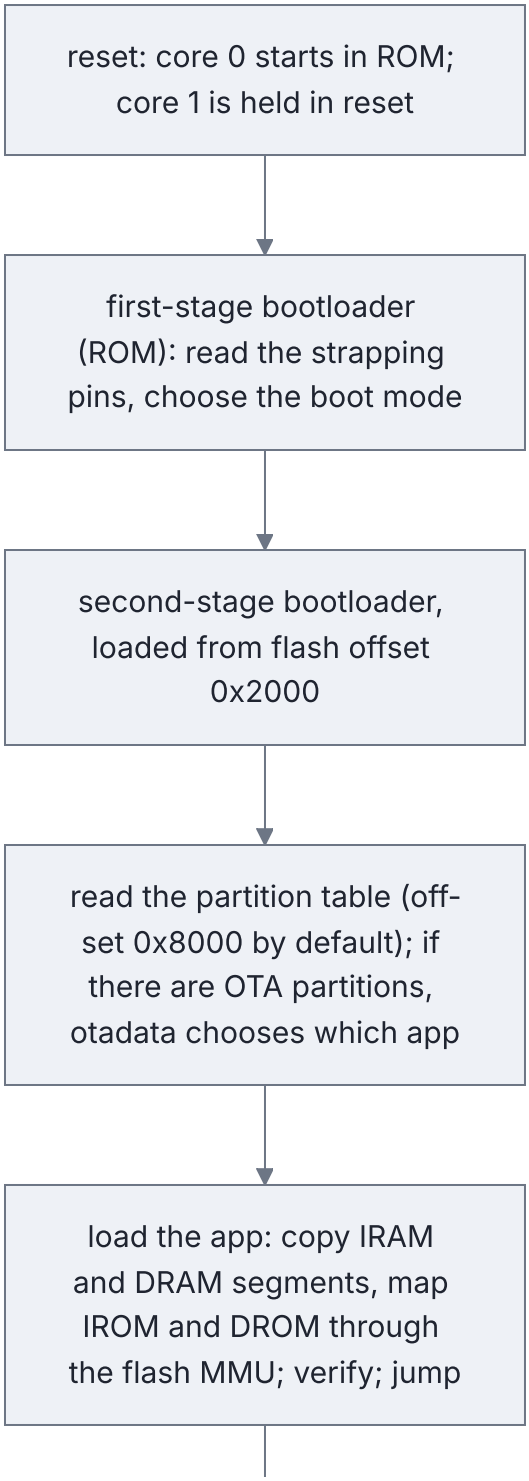
An RTOS kernel schedules tasks; it does not know how to set up a chip's clocks, map its flash, talk to its SPI controller or allocate memory a DMA engine can read. Espressif's IoT Development Framework (ESP-IDF) supplies all of that around FreeRTOS. An ESP-IDF application is a set of **components**, each a directory with sources and a build description, compiled together by a CMake-based build system and configured through a menu of options (Kconfig) that become preprocessor constants. The framework's own components include the heap allocator of Chapter 6, the drivers for each peripheral, the bootloader, and the panic handler and watchdogs described below.

Two drivers matter to PicoCalc OS. ESP-IDF's SPI master driver initialises a bus on one of the chip's general-purpose SPI controllers (SPI2 or SPI3; the driver does not support SPI0 and SPI1), attaches devices to it, and runs **transactions**, either queued (`spi_device_queue_trans` , with the result collected later by `spi_device_get_trans_result`) or polled (`spi_device_polling_transmit`). With DMA, a transaction's buffers must be in DMA-capable memory and 32-bit aligned, or the driver copies them into temporary buffers [ESP-IDF SPI master 2026]. The same documentation gives typical durations for a one-byte transaction: 44 µs for an interrupt-driven transaction using DMA, 26 µs without DMA, 27 µs for a polled transaction using DMA and 12 µs without. Those are overheads per transaction, paid before the first bit moves, and they will matter for the port: the host build charges 2 µs per command.

ESP-IDF's I2C master driver creates a bus with `i2c_new_master_bus` , attaches a device with `i2c_master_bus_add_device` (its address and SCL frequency, at most 400 kHz in controller mode), and transfers with `i2c_master_transmit` , `i2c_master_receive` or `i2c_master_transmit_receive` , each with a timeout. The ESP32-P4 has two I2C controllers [ESP-IDF I2C 2026]. The keyboard controller's protocol (Chapter 2) wants a write, a 2 ms pause with the bus held, and a read; whether a write followed directly by a read, as `i2c_master_transmit_receive` performs it, gives the STM32's firmware enough time to prepare its answer is a question for the hardware. If it does not, the nucleus can issue the write, wait, and issue the read, since it already serialises every transfer on the bus and nothing else will come between them.

7.3 From reset to `app_main`

A microcontroller's boot is a chain of programs, each loading the next. ESP-IDF documents the ESP32-P4's chain in three stages [ESP-IDF startup 2026], and Figure 7.1 shows it.


reset: core 0 starts in ROM; core 1 is held in reset
first-stage bootloader (ROM): read the strapping pins, choose the boot mode
second-stage bootloader, loaded from flash offset 0x2000
read the partition table (offset 0x8000 by default); if there are OTA partitions, otadata chooses which app
load the app: copy IRAM and DRAM segments, map IROM and DROM through the flash MMU; verify; jump

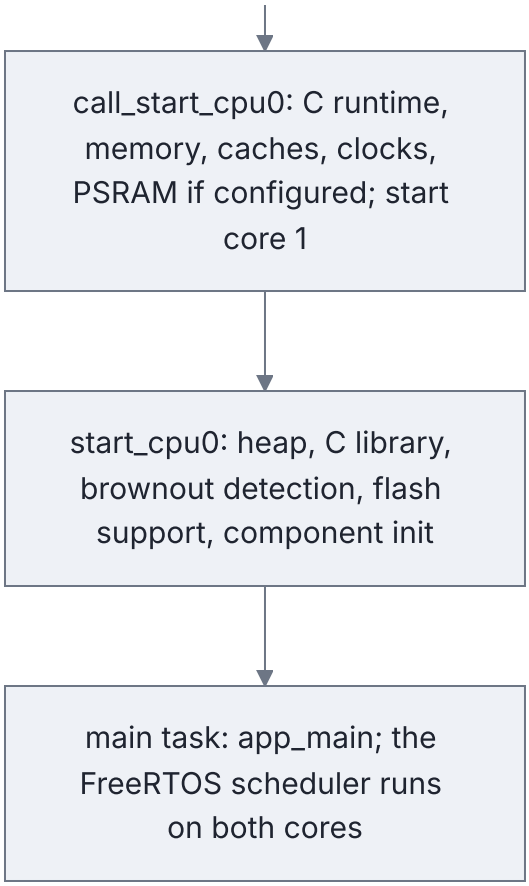


Figure 7.1. The ESP32-P4's boot as ESP-IDF documents it: the ROM loads the second-stage bootloader from flash, which loads the application, whose start-up code initialises the system before calling app_main. PicoCalc OS's nucleus would start in app_main.

The first stage is in ROM and cannot be changed. After reset, core 0 starts running it while core 1 is held in reset; it reads the **strapping pins**, whose levels at reset choose between booting from flash and waiting for a download over the serial port. The second stage is ESP-IDF's bootloader, read from the start of flash (at offset 0x2000 on the ESP32-P4). It reads the **partition table**, by default at offset 0x8000, which divides the flash into named regions: the application, perhaps two application slots for over-the-air updates with a small `otadata` partition recording which is current, and data partitions. It loads the chosen application by copying the segments that must run from internal RAM and mapping the rest through the flash MMU, verifies it, and jumps to it.

The application's own start-up code then does the work that would be the kernel's boot on a desktop: it sets up the C runtime, finishes the cache configuration, sets the CPU clock, enables PSRAM if configured, starts core 1, initialises the heap and the C library, enables brownout detection, and runs each component's initialisation in a defined order. Finally it creates the main task, starts the scheduler, and calls `app_main`, the application's entry point, which unlike an ordinary FreeRTOS task is allowed to return [ESP-IDF startup 2026].

Two properties of this chain matter for an operating system built on it. First, the application is a single image: everything, including PicoCalc OS's nucleus, QuickJS and the worlds' JavaScript, is linked into one binary or placed in a data partition beside it. The host build reads the worlds' sources from files; the WebAssembly build of Chapter 17 already showed the alternative, a table of files in memory that the page fills before booting. Second, an over-the-air update replaces the whole image, so updating one world's JavaScript means either rebuilding the image or keeping the JavaScript in a data partition the nucleus reads at boot.

7.3.1 PicoCalc OS's own boot

Once `app_main` runs, PicoCalc OS's boot begins, and the host build shows what it consists of. The nucleus reads the manifest, creates a runtime for each of the thirteen worlds and evaluates the prelude and the world's file in it; on the virtual clock this takes 13 segments of about 66 µs each, and all worlds have started 0.86 ms after power-on. Then the display driver's start-up sequence runs, and it dominates. The driver sends a software reset, waits 150 ms, sends sleep-out, waits 150 ms, and continues through the eight commands of Chapter 2, 370 ms of waiting in all. The panel is switched on at 322 ms, the first full frame is sent from 372 to 413 ms, and a second update of seven rectangles follows from 416 to 439 ms as the battery level and the launcher's count of running worlds arrive. In all that time, the lane is busy for 5.5 ms.

Boot 470 virtual ms

Thirteen worlds start in under a millisecond of lane time; the first frame waits on the panel's own power-up delays.

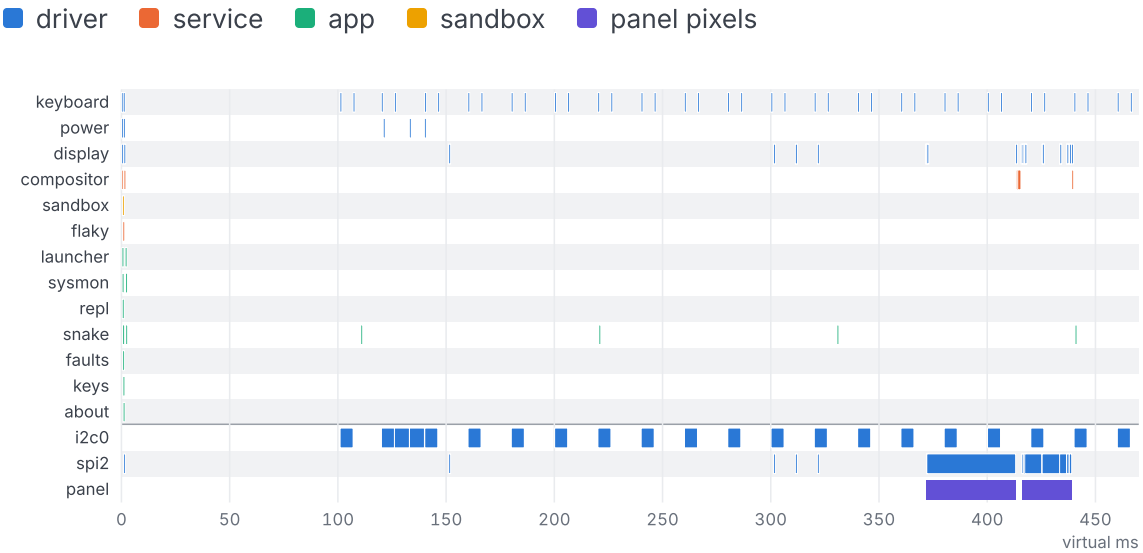


Figure 7.2. PicoCalc OS's boot on the host build's virtual clock. The worlds start within the first millisecond; the rest is the display driver waiting for the panel, then one full-frame transfer and a small second update.

The trace's SPI lines show the display driver's sequence exactly as it reaches the panel; each command is a few bytes, separated by the driver's waits:

▸ bench/traces/boot.jsonl


```
{"k":"bus","bus":"spi2","w":"display","t":1048,"t1":1050,"bytes":1,"what":"cmd 0x01"}
{"k":"bus","bus":"spi2","w":"display","t":151230,"t1":151232,"bytes":1,"what":"cmd 0x11"}
{"k":"bus","bus":"spi2","w":"display","t":301321,"t1":301323,"bytes":2,"what":"cmd 0x3a"}
{"k":"bus","bus":"spi2","w":"display","t":311412,"t1":311414,"bytes":2,"what":"cmd 0x36"}
{"k":"bus","bus":"spi2","w":"display","t":311459,"t1":311461,"bytes":1,"what":"cmd 0x21"}
{"k":"bus","bus":"spi2","w":"display","t":321550,"t1":321552,"bytes":1,"what":"cmd 0x29"}
…
{"k":"bus","bus":"spi2","w":"display","t":372079,"t1":413125,"bytes":204800,"what":"pixel
s 320x320 at 0,0"}
```

The 0.86 ms for starting thirteen worlds should not be believed for the device. The virtual clock charges a boot segment for the interpreter steps it executes and its fixed entry cost, but not for creating a runtime or for parsing and compiling JavaScript, and evaluating the prelude is mostly definitions, which take few steps. The measurement firmware that calibrated the clock found that initialising one QuickJS runtime on the ESP32-P4 took about 6 ms. Thirteen runtimes, plus compiling the prelude thirteen times and each world's own file once, will therefore take tens of milliseconds on the device, still well inside the display's 370 ms of waiting, but a hundred times what the model says. A system whose model leaves out a cost has to say which cost, and this is one.

7.4 Keeping it alive: watchdogs, brownout and panics

A desktop operating system that stops responding is noticed by a person. An embedded device may have no person nearby, so it needs hardware that notices for it. A **watchdog timer** counts down continuously and resets the chip, or raises an interrupt, when it reaches zero; software that is working **feeds** it (resets the count) regularly. If the software hangs, it stops feeding, and the watchdog acts. The art is in choosing what feeds it: a watchdog fed by an interrupt handler proves only that interrupts run, and one fed by a low-priority task proves that the higher-priority ones leave it time.

The ESP32-P4 has two main watchdog timers in its timer groups, a watchdog in the low-power subsystem, and an analog super watchdog [Espressif 2026b]. ESP-IDF builds three software mechanisms on them [ESP-IDF watchdogs 2026]:

- The **interrupt watchdog** (IWDT) uses the main watchdog of timer group 1 and is fed by the FreeRTOS tick interrupt on each core. If interrupts are disabled or a handler runs too long, the tick stops, the watchdog expires, and the panic handler runs; if even that fails, a second stage resets the chip.
- The **task watchdog** (TWDT) uses the main watchdog of timer group 0. Tasks subscribe with `esp_task_wdt_add` and feed it with `esp_task_wdt_reset`; by default it watches each core's idle task, so it fires when some task keeps a core busy without ever letting the idle task run. Its default action is to print a warning and a backtrace and continue; a configuration option makes it panic instead.
- The **RTC watchdog** in the low-power subsystem covers the boot itself, from reset until shortly before `app_main`, when the start-up code disables it unless configured otherwise.

When something goes badly wrong (an illegal instruction, a memory access PMP refuses, a watchdog in panic mode, a failed assertion, a heap corruption check) ESP-IDF's **panic handler** runs. By default it prints the registers at the point of the error and a backtrace, and restarts the chip; it can instead halt, restart silently, or start a GDB stub that a debugger can talk to over the console UART. If the core dump feature is enabled,

it also saves the tasks' stacks and registers to flash or prints them to the UART for later analysis. A separate **brownout detector**, enabled by default, resets the chip when the supply voltage falls below a safe level and prints "Brownout detector was triggered" first, if there is time [ESP-IDF fatal errors 2026].

Each of these mechanisms answers a failure by resetting the chip, or at best the core, and that is too coarse for a system made of independent components. PicoCalc OS's answer for its own components is a layer above: a world that loops is stopped by its budget, a world that runs out of memory is stopped by its allocator, a world that throws is caught by the nucleus, and in each case only that world is restarted (Chapter 14). The chip's watchdogs remain as the layer below, for failures the nucleus cannot see: a bug in the nucleus or in QuickJS, a hang in a native driver, a corrupted heap. Figure 7.3 shows the layers as the port would arrange them.

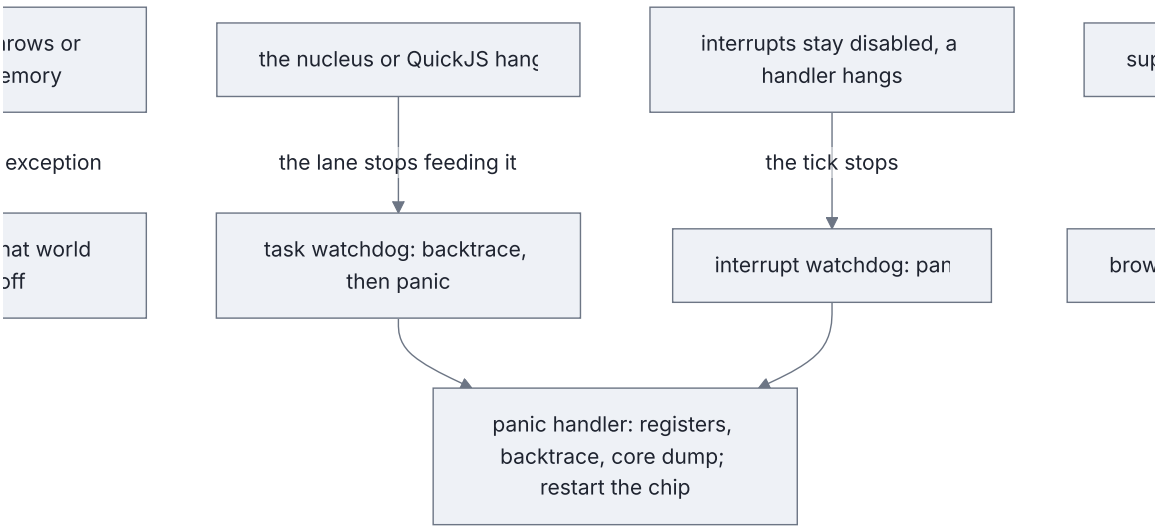


Figure 7.3. Layers of failure detection on the device, as planned. Each layer handles what the one above it cannot see; only the top layer restarts less than the whole chip.

For the task watchdog to mean something, the nucleus must feed it from the lane: once per segment, or once per pass through its event loop. A world can then never starve it, since every segment ends within its budget plus a bounded overrun (Chapter 11); the longest budget in the standard manifest is the sandbox's 200 ms, and the longest any segment may take is the 600 ms a world is given to evaluate its file when it starts, so a watchdog timeout of a few seconds leaves a wide margin. If the lane itself stops (a native call

that never returns, an infinite loop in the nucleus, a deadlock in a driver underneath) the feeding stops and the chip restarts. That is the right division of labour: the nucleus contains the failures of what it runs, and the hardware contains the failures of the nucleus.

7.5 Seeing inside: logs, the console and fault frames

When an embedded system fails, the engineer usually has three sources of information: what the system printed on its serial console, what a debugger attached through JTAG can show, and what a core dump recorded. The console is by far the most used, because it works everywhere, costs one UART and needs no special tools. ESP-IDF's logging and panic output go to it; so do the bootloader's messages.

PicoCalc OS's nucleus writes two kinds of line for its console. One is the kernel log, a timestamped human-readable line for each significant event: a world booting, crashing, restarting or being stopped. The other is the **fault frame**: whenever a world crashes, one line beginning `#OCT1` followed by a JSON object that describes the crash.

▸ nucleus/kernel.c

```
// A fault frame on the UART: one line, "#OCT1 " and a JSON object. A host-side handler
reads these
// (and could answer on the RX line) without the device needing a network.
static void uart_fault(world_t *W, const char *why, const char *detail) {
  if (!K.uart) return;
  sbuf_t b = { 0 };
  sb_printf(&b, "#OCT1 {\"ev\":\"fault\",\"t\":%.3f,\"world\":", kernel_now() / 1000.0);
  sb_json_str(&b, W->name);
  sb_add(&b, ",\"file\":"); sb_json_str(&b, W->file);
  sb_add(&b, ",\"handling\":"); sb_json_str(&b, W->in_turn ? W->turn_type : "");
  sb_add(&b, ",\"why\":"); sb_json_str(&b, why);
  sb_printf(&b, ",\"crashes\":%d,\"mem_kb\":%zu,\"limit_kb\":%zu", W->n_crash_times,
            (size_t)(W->mem_used / PROFILE.mem_scale / 1024), W->mem_limit / 1024);
  if (detail) { sb_add(&b, ",\"detail\":"); sb_json_str(&b, detail); }
```

Here is what the console receives when the Faults app asks the sandbox to loop forever, on the host build, where the console is a file (`out/uart.log`):

```
[  1895.230] sandbox crashed handling eval: ran past its turn budget of 200 virtual ms
#OCT1
{"ev":"fault","t":1895.230,"world":"sandbox","file":"system/sandbox.js","handling":"eval"
,"why":"ran past its turn budget of 200 virtual
ms","crashes":1,"mem_kb":181,"limit_kb":512,"detail":"    at <eval> (<input>:1:1)\n    at
eval (native)\n    at <anonymous> (system/sandbox.js:15:20)\n    at <anonymous>
(prelude.js:139:17)\n"}
```

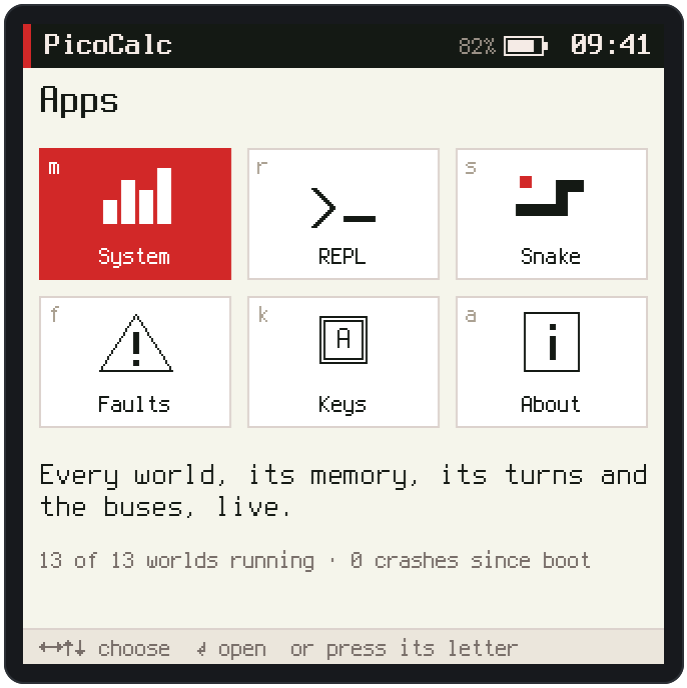
The frame's design follows from who reads it. A person reads the kernel log line above it. The frame is for a program on the other end of the cable: a script on the developer's laptop, or an agent that watches the console, collects the frames, opens the named file at the line in the stack, and proposes a fix. Three choices make that easy. The prefix is fixed and unlikely to occur in other output, so the frames can be picked out of a mixed stream without a protocol. Each frame is one line, so a reader never has to reassemble it. And the content is what a diagnosis starts from: which world, which file, which message it was handling, why it failed, how often it has failed recently, its memory against its limit, and the JavaScript stack. The comment in the code notes the other direction: the same serial line has a receive wire, and a host could answer, for example with a request to restart a world or with a replacement file, without the device needing a network. That return channel is not implemented; the frames are.

On the host build the frames go to a file; in the WebAssembly build they go to the page, which shows them in the log panel of the live system. On the device they will go to the console UART, which on the board used here is a USB serial port.

TRY 7.1

Crash a driver and read its fault frame

This is PicoCalc OS itself, compiled to WebAssembly and running in the page (Chapter 17 explains how). Wait for the home screen, then crash the display driver. The screen goes dark, the log shows the kernel's line and the `#OCT1` frame, and about half a second later the screen returns: most of that time is the driver's start-up sequence from Section 7.3.1, run again. Crash the keyboard driver and press an arrow key at once; the key is not lost, because it waits in the STM32's FIFO until the restarted driver polls. If the live system cannot start in your viewer, the figure shows a captured screen instead.



7.6 Power

A handheld runs from a battery, and power is a resource the operating system manages like time and memory. On the PicoCalc, power management is split across three chips. The AXP2101 power-management chip charges the battery and measures it. The STM32 keyboard controller reads the AXP2101 over its own

I2C bus, copies the battery percentage into its register 0x0B every 20 seconds (with bit 7 set while charging), and drives the screen and keyboard backlights from its registers 0x05 and 0x0A [ClockworkPi n.d.]. The ESP32-P4 sees only what the STM32 exposes.

PicoCalc OS's power driver reads the battery register every 5 seconds and publishes changes on the `power` channel; it also turns the screen off and on by setting the backlight to zero and restoring the previous level:

▸ history/first-pass/js/drivers/power.js

```
  .on("screen", async (s, { on }) => {
    s.screen = on === undefined ? !s.screen : !!on;
    if (!s.screen) s.saved = s.backlight || s.saved;
    await setBacklight(s, s.screen ? s.saved || 32 : 0);
    return { screen: s.screen };
  });
```

The backlight is usually the largest single consumer in a device like this, so turning it off is the most effective step the driver can take. The processor is the next opportunity. On the host build, an idle home screen keeps the lane busy 0.72% of the time, almost all of it the keyboard driver's polls. A processor that is idle 99% of the time could sleep for most of it, waking for the next event on the virtual clock's queue, which the nucleus always knows. Doing that on the ESP32-P4 means choosing among its clock and sleep modes and measuring their wake-up latency against the 20 ms keyboard poll; it is part of the port, and nothing about it has been measured yet.

7.7 The port: from the simulated board to the device

PicoCalc OS was written so that the port would replace one layer and leave the rest alone. Everything above the nucleus is JavaScript that knows nothing about the platform. The nucleus itself talks to hardware in exactly one file, `nucleus/hw.c`, and there only through a handful of calls into the board:

▸ nucleus/hw.c

```
static void run_spi_cmd(deferred_t *d, vtime_t at) {
  handle_t *h = find(d->handle);
  int hz = h ? h->hz : 1000000;
  vtime_t start = bus_start(BUS_SPI2, at);
  st7365p_cmd((uint8_t)d->op, start);
  if (d->nbytes) st7365p_data(d->bytes, d->nbytes, start + wire_us(8, hz));
  vtime_t end = start + wire_us((1 + d->nbytes) * 8.0, hz) + 2;   // + chip-select and
D/C toggling
```

On the host build, `st7365p_cmd`, `st7365p_data` and `stm32_i2c` are the models in `sim/`; on the device they will become ESP-IDF transactions. That much is mechanical. The interesting part of the port is everything the host build gets for free from its virtual clock and the device does not. Table 7.2 lists each assumption the host build makes and what the port has to establish instead.

Table 7.2. What the host build assumes and what the port to the ESP32-P4 must establish. Nothing in the right-hand column has been done yet.

On the host build	On the device	How to establish it
Time is virtual; a segment's cost is computed: 40 µs + steps × 665 ns + native work	Time is real; a segment takes what it takes	Measure each segment with the cycle counter; compare with the model per world
A bus transfer takes the time its bits take, plus 2 µs per SPI command	Each ESP-IDF SPI transaction costs 12–44 µs of overhead before its first bit	Time CASET, RASET, RAMWR and pixel transactions; batch commands if needed
Pixels go in 32 KiB chunks at 12 µs each	DMA may need internal memory; PSRAM buffers may be copied	Test DMA from the PSRAM framebuffer; measure a full frame
The STM32 answers after a 2 ms gap with the bus held	Unknown whether a direct write-read works	Try `i2c_master_transmit_receive`; fall back to write, wait, read
Allocation and garbage collection cost nothing	Both cost real time inside a segment	Measure each world's collections; charge them to budgets
Creating a runtime and compiling a world cost about 66 µs	About 6 ms per runtime, measured	Measure boot per world; consider compiling once and caching bytecode
Interrupts never happen	Bus completions arrive as interrupts	Turn completions into events on the nucleus's queue (deferred processing)
Nothing else runs	FreeRTOS, ESP-IDF and the console share the cores	Pin the nucleus to one core; measure its share

The nucleus's main loop changes most. On the host build, `kernel_run_until` pops events from a priority queue ordered by virtual time and runs them; time jumps from one event to the next. On the device, time cannot jump, so the loop waits for the next event's time or for a bus completion, whichever comes first. A sketch of the nucleus task as the port might write it follows. It is not code in the repository, and `kernel_next_event`, `board_init` and `wait_until_or_completion` are functions that do not exist yet:

```
static void nucleus_task(void *arg) {
  esp_task_wdt_add(NULL);                          // the task watchdog now watches the
lane
  kernel_init("/js", NULL);
  kernel_boot("/js/system.json");
  for (;;) {
    int64_t next = kernel_next_event();            // earliest timer, settle or delivery,
in µs
    wait_until_or_completion(next);                // block on a queue fed by SPI and I2C
completions
    kernel_run_until(esp_timer_get_time());        // run every segment that is due by
now
    esp_task_wdt_reset();                          // the lane is making progress
  }
}

void app_main(void) {
  board_init();                                    // spi_bus_initialize,
i2c_new_master_bus, devices
  xTaskCreatePinnedToCore(nucleus_task, "nucleus", 64 * 1024, NULL, 5, NULL, 1);
}
```

Budgets change meaning too. On the host build, the interrupt handler compares a segment's computed cost against its budget, so the same program always stops at the same instruction. On the device, the handler would compare elapsed real time, read from a cycle counter, and the stop would vary from run to run by whatever the caches, PSRAM and interrupts did. Determinism, which the host build gets from its virtual clock (Chapter 18), would then be a property of the host build only, and the device's job would be to stay within the bounds the host build's measurements predict.

The port is also the experiment that tests the model. The virtual clock rests on one measured constant, $\tau = 665$ ns per interpreter step, taken from a single loop, and on estimates for everything else. The first measurements on the device should be the ones the model depends on: τ for each world's real workload rather than one loop, the fill, text and diff costs of the compositor's native drawing, the SPI and I2C transaction overheads, garbage-collection pauses, and the end-to-end latency of a key press, measured from the key switch to the panel's SPI lines with a logic analyser. Where they disagree with the host build, the profile's constants change, and the host build's predictions become more trustworthy for everything that was not measured.

KEY POINTS

- FreeRTOS gives firmware preemptive tasks, a tick, queues and interrupt-safe calls; its deferred-interrupt pattern keeps handlers short and does the work in a task.
- ESP-IDF supplies the rest of the platform around FreeRTOS: components, configuration, a TLSF heap with capabilities, and drivers whose SPI transactions cost tens of microseconds each before any data moves.
- The ESP32-P4 boots through a ROM loader, a second-stage bootloader at flash offset 0x2000 and a partition table, then the application's start-up code, which ends by calling `app_main`; PicoCalc OS's nucleus would start there as one FreeRTOS task.
- On the host build, PicoCalc OS boots thirteen worlds in 0.86 ms of virtual time and spends the next 370 ms waiting for the panel; on the device, creating runtimes alone will take tens of milliseconds because the model does not charge for it.
- The chip's watchdogs, panic handler and brownout detector recover from failures by restarting the chip; PicoCalc OS adds a finer layer that restarts only the failed world, and the nucleus must feed the task watchdog so that the chip still catches the nucleus's own hangs.
- The `#OCT1` fault frame is one fixed-prefix line of JSON per crash, designed to be read by a program on the other end of the serial console.
- The port replaces the board calls in `nucleus/hw.c` and the virtual clock's main loop; everything else the host build assumes (transaction overheads, DMA from PSRAM, allocation, collection and boot costs) has to be measured on the device.

This chapter ends Part II's account of the machine and its firmware. Everything so far has treated the components of a system as cooperating; Part III asks what happens when they do not. Chapter 8 begins with the classical principles of protection and the structures operating systems use to enforce them, and places PicoCalc OS among them, with its single address space and its language boundary, precisely.

EXERCISES

1. BASIC In the deferred-interrupt sketch of Section 7.1, what would go wrong if `spi_done_isr` called `finish_transfer` directly? Name two things an interrupt handler on FreeRTOS must not do.
2. BASIC Using Figure 7.1, list in order what has run by the time `app_main` is called, and say which stage reads the partition table and which chooses between two OTA slots.
3. BASIC For each of the interrupt watchdog, the task watchdog and the brownout detector, give one failure it detects that the other two do not.

4. ANALYSIS The task watchdog by default watches the idle tasks. If the nucleus task ran at a priority above the idle task on core 1 and a world entered an infinite loop, would the idle-task watchdog fire before the world's budget stopped it? Use the budgets in `js/system.json` and assume a watchdog timeout of 5 s.

5. ANALYSIS ESP-IDF's documented transaction overhead is 12 to 44 µs. The display driver sends a launcher update as three rectangles, each a CASET, a RASET, a RAMWR and a pixel transfer. Estimate the extra time per update compared with the host build's 2 µs per command, for queued DMA transactions, and propose a way to reduce it.

6. ANALYSIS On the device, thirteen runtimes take about 6 ms each to create. If the nucleus booted all worlds before the display driver could start, how much later would the first frame appear? Propose a boot order that hides the cost, using the priorities in Chapter 4.

7. CODE Run `scripts/faults.txt` on the host build with `-u` pointing at a file of your choice. Write a short program (in any language) that reads the file, extracts every `#OCT1` frame, and prints a summary per world: crashes, reasons, and the source line of the deepest JavaScript stack frame in `detail`.

8. CODE Add a `"build"` field to the fault frame carrying a hash of the world's source text, so that a reader can tell which version of a file crashed. Where in `kernel.c` would you compute it, and when does it need to change?

9. CODE The host build's power driver reads the battery every 5 s, while the STM32 updates it every 20 s. Change the period to 20 s and measure the effect on I2C0's busy time with the system monitor. Is there any reason to keep reading more often than the source changes?

10. DESIGN Design the return channel the fault frame's comment mentions: messages a host can send on the console's receive line to the nucleus. Specify their format, which operations they allow (restart a world, replace a file, change a budget), and how the nucleus authenticates them, given that anyone with a USB cable can type on that line.

11. DESIGN Write a measurement plan for the first day on the device: which numbers to measure first, with which instrument (cycle counter, logic analyser on the SPI and I2C lines, current meter), in which order, and which constant in the host build's profile each one replaces.

12. PROJECT Port the nucleus's main loop to run against real time on Linux: replace the virtual clock with `clock_gettime`, run the board models in a second thread that completes transfers after their wire time, and measure how much the screens and latencies differ from the virtual-clock build.

FURTHER READING

[Barry 2016] The FreeRTOS kernel book: tasks, queues, interrupt management and the deferred-processing pattern, explained from first principles with working examples.

[ESP-IDF startup 2026] ESP-IDF's account of the boot chain on the ESP32-P4, from ROM to `app_main`.

[ESP-IDF watchdogs 2026] How ESP-IDF uses the chip's watchdogs, and what each one can and cannot detect.

[ESP-IDF fatal errors 2026] What happens when something goes wrong: the panic handler's options, core dumps, and the brownout detector.

[ESP-IDF SPI master 2026] The SPI master driver, including its section on transfer speed, which every display driver on the platform depends on.

CHAPTER 8 PART III · ISOLATION AND PROTECTION

Protection and kernel structure

An operating system that runs more than one program decides, on every access to a shared resource, whether that access is allowed. This chapter is about how that decision is organised: which entities the system tells apart, which principles a good protection design follows, where the code that makes the decision lives, and how the large-scale structure of a kernel follows from where that code is put. By the end you should be able to state Saltzer and Schroeder's design principles and Anderson's requirements for a reference monitor, compare monolithic kernels, microkernels, exokernels and unikernels by where they draw their boundaries, and say precisely what PicoCalc OS protects, what enforces each of its boundaries, and what it does not protect at all.

PicoCalc OS is an unusual data point among operating systems. Its structure is a microkernel's: the drivers, the compositor and the apps are separate components outside a small nucleus, and they reach hardware only through it. But the boundaries between those components are not drawn by the processor. Every world runs in the same address space, at the same privilege level, as the nucleus that supervises it. What keeps a world inside its boundary is the JavaScript engine: a world can only follow references it holds, and the only references that lead out of it are two host functions. What that design buys, and what it costs, is the subject of this chapter and of the two that follow it.

8.1 Principals, objects and rights

Protection needs a vocabulary, and the classical one has three nouns. A **principal** is an entity to which the system attributes actions and authority: a user in UNIX, a process in a microkernel, a world in PicoCalc OS. An **object** is anything whose use the system controls: a file, a device, a region of memory, a message queue, an amount of processor time. A **right** is permission for a principal to perform a particular operation on a particular object: to read a file, to write to a device, to pass a handle to someone else. Protection is the set of

mechanisms that decide, for each attempted operation, whether the principal attempting it holds the right. Security is the larger problem that includes protection but also asks who the adversary is, what they want, and whether the policy being enforced is the right one.

The distinction between policy and mechanism runs through the whole subject. A **policy** says who may do what; a **mechanism** enforces whatever policy it is given. Brinch Hansen built the RC 4000's nucleus around that separation: its job was to provide processes and message passing, while "processes should have complete freedom to choose their own strategy of program scheduling" [Hansen 1970]. In PicoCalc OS the policy for hardware is the manifest, `js/system.json`, which lists for each world the devices it may claim:

▸ history/first-pass/js/system.json

```
    { "name": "keyboard",   "file": "drivers/keyboard.js",  "claims": ["i2c0:0x1f"],
"budget_ms": 20,  "mem_kb": 320,
    …
    { "name": "power",      "file": "drivers/power.js",     "claims": ["i2c0:0x1f"],
"budget_ms": 20,  "mem_kb": 320,
    …
    { "name": "display",    "file": "drivers/display.js",   "claims": ["spi2"],
"budget_ms": 30,  "mem_kb": 320 },
    { "name": "compositor", "file": "system/compositor.js", "claims": ["dma"],
"budget_ms": 80,  "mem_kb": 768,
…
    { "name": "sandbox",    "file": "system/sandbox.js",
"budget_ms": 200, "mem_kb": 512, "priority": 0 },
```

The mechanism is a handful of checks in the nucleus's C code, which we will read in Section 8.3. The keyboard driver may claim the I2C device at address `0x1F` (the STM32 keyboard controller), the display driver may claim the SPI bus the panel sits on, the compositor may allocate DMA framebuffers, and the sandbox, which evaluates whatever is typed at the REPL, may claim nothing.

Listing the objects a system controls, and how each one is guarded, is the first step in understanding any protection design. Table 8.1 does that for PicoCalc OS. Read it row by row: it is the chapter's map.

Table 8.1. The objects PicoCalc OS controls, how a world comes to use each, and what checks the use. The last four rows are not guarded by rights at all; Chapters 9 and 11 return to them.

Object	Operations	How a world gets it	Checked by
I2C device (`i2c0:0x1f`)	write, read (one transaction)	a claim in the manifest gives a handle	`handle_use`: holder, not revoked, type, right `w`
SPI bus (`spi2`)	command, pixel transfer	claim	`handle_use`
Framebuffer (DMA buffer)	draw (`w`), send to the panel (`r`), narrow, pass (`g`)	claim `dma`, or a handle received in a message	`handle_use`, `can_pass`
System state (worlds, handles, statistics)	read	claim `sys.read`	`world_claims`
Another world's life	kill, restart	claim `sys.admin`	`world_claims`
A world's mailbox	`send`, `ask`	its name	only that the name exists
A channel	`publish`, receive	the world's own `provides` / `needs` declaration	only that the declaration exists
Processor time	run segments	being scheduled	the segment's budget
Memory	allocate	being a runtime	the world's heap limit

The first five rows are guarded by **rights**: a world can perform the operation only if the nucleus has a record that it may. The next two are guarded by **naming**: a world that knows another world's name can send it a message, and a world that declares that it provides a channel can publish on it. The last two are guarded by **quantity**: every world may compute and allocate, but only so much. These are three different kinds of protection, and they fail in different ways. A rights check fails safe: a missing claim means no access. A naming check is only as strong as the secrecy of the name, and in PicoCalc OS world names are not secret at all. A quantity check bounds damage but does not prevent it. Chapter 9 examines the naming rows and shows a world exploiting them; Chapter 11 examines the quantity rows.

NOTE

The first pass, a later revision of PicoCalc OS described in a separate book, *From L4 to Small Worlds* (Chapter 5), replaced the two naming rows with rights from the manifest. A world may now send to and ask only the worlds its manifest entry lists under `sends` and `asks`, and publish and hear only the channels its entry lists under `provides` and `needs`; a message whose type is a channel the receiver hears is refused. This chapter describes the system before that pass.

8.2 Saltzer and Schroeder's principles

In 1975 Jerome Saltzer and Michael Schroeder published a long tutorial on protecting information in computer systems whose middle section, eight design principles, has outlived nearly everything else written about protection in that decade [Saltzer and Schroeder 1975]. They are not rules that a system satisfies or violates; they are pressures, and a real design trades them against each other and against cost. They also listed two principles from physical security, work factor and compromise recording, which they judged to apply only imperfectly to computers; we leave those aside. Table 8.2 states each principle in a sentence and holds PicoCalc OS against it.

Table 8.2. Saltzer and Schroeder's eight principles, and how PicoCalc OS meets or misses each.

Principle	What it asks	PicoCalc OS
Economy of mechanism	Keep the protection mechanism small and simple enough to inspect.	The checks live in a 2,374-line nucleus; but the engine they rely on is 80,864 lines (Section 8.3).
Fail-safe defaults	Base access on permission, not exclusion; the default is no access.	A world has no claims unless the manifest lists them; the sandbox has none.
Complete mediation	Check every access to every object, not just the first.	Every use of a handle is checked, including whether it has since been revoked. Messages are not mediated.
Open design	Security must not depend on the design being secret.	The design is published; handles are small integers whose secrecy protects nothing, so they are checked against a holder set instead.
Separation of privilege	Require two independent conditions where one could be subverted.	Absent: one manifest decides everything.
Least privilege	Give each component only the rights its job needs.	The display driver receives a read-only view of the framebuffer. But the keyboard and power drivers share one I2C device with full rights.
Least common mechanism	Minimise mechanism shared by several users.	Worlds share no heap objects, but do share the lane, the buses and one table of 128 handle slots.
Psychological acceptability	Make protection easy to use correctly.	Claims are readable strings; a missing claim produces an error naming the world and the claim.

Three of these principles deserve a closer look, because PicoCalc OS illustrates both their value and their cost.

Complete mediation. UNIX checks a file's permissions when a process opens it and then hands back a file descriptor; later reads and writes on the descriptor are not checked against the file's permissions again. That is cheap, and it is why revoking a UNIX user's access to a file does not affect a process that already has it open. PicoCalc OS checks every use. Each I2C transaction, each SPI command and each fill of a framebuffer calls `handle_use`, which looks the handle up, confirms that the calling world holds it, checks that it has not been revoked, and compares rights. The price is a table scan per operation (the table has 128 slots,

so this is a few hundred nanoseconds of native work at most), and the payoff is that revocation takes effect at the next use rather than never. Chapter 9 shows how an earlier version of the revocation check could be escaped, and the fix.

Least privilege. The manifest's claims are per device, not per operation. The keyboard and power drivers both claim `i2c0:0x1f` because the STM32 controller answers both keyboard and battery queries at that address, and both receive handles with every right. Nothing stops the power driver from reading the keyboard's FIFO register and swallowing key presses. A finer design would claim registers rather than devices, or would give the I2C device to one world and have it serve the other; either moves the boundary closer to what each driver actually needs. The display driver shows the principle working: the compositor gives it a read-only view of the framebuffer, so a bug in the display driver can send stale pixels but cannot draw.

Least common mechanism. Separate runtimes share nothing in their heaps, which is the strongest form this principle can take inside one address space. But some mechanism is unavoidably common. Every handle in the system lives in one fixed table of 128 slots, and a world that has received a single handle can fill that table by narrowing it repeatedly, since each narrowing creates a new handle and the nucleus keeps no per-world quota. In a test written for this chapter, a world with no claims was given a read-only view of a framebuffer and narrowed it in a loop:

```
[    11.612] owner: hog: {"narrowed":126,"stopped":"out of handles"}
[    11.621] owner: second alloc: out of handles
[    11.631] owner: narrow: out of handles
```

After 126 narrowings the table was full, and the next world to ask for a framebuffer was refused. Nothing was read or written that should not have been; the harm is to availability. This is what Saltzer and Schroeder meant: every mechanism shared between principals is a channel through which one can affect another, and the only complete defence is not to share. The practical defence, a quota per world, is an exercise at the end of the chapter. The first pass added such a quota: every handle a world makes counts against its manifest's `handles` quota (4 by default), and the boot fails unless the quotas add up to at most the table's 128 entries (*From L4 to Small Worlds*, Chapter 7).

8.3 The reference monitor and the trusted computing base

In 1972 James Anderson's planning study for the US Air Force named the piece of a system that makes access decisions the **reference validation mechanism**, and set three requirements for it [Anderson 1972]:

- "The reference validation mechanism must be tamper proof."
- "The reference validation mechanism must always be invoked."
- "The reference validation mechanism must be small enough to be subject to analysis and tests, the completeness of which can be assured."

The abstract idea, a monitor that sits between every subject and every object, is the **reference monitor**; the code that implements it, plus everything that code depends on to behave correctly, is the **trusted computing base** (TCB). A bug outside the TCB can make a component misbehave within its rights. A bug inside the TCB can give a component rights it was never granted. The three requirements are a test any protection design can be put to, and they are the right lens for PicoCalc OS.

> **DEFINITION: TRUSTED COMPUTING BASE**
>
> The **trusted computing base** of a system is the set of hardware and software whose correct operation is necessary for the system's protection policy to hold. Everything outside it may be faulty or hostile without breaking the policy.

What the monitor is. In PicoCalc OS the access decisions are made by a few C functions in the nucleus. Hardware use goes through `handle_use`:

▸ history/first-pass/nucleus/hw.c

```
handle_t *handle_use(world_t *W, int id, int type, int right, char *err, size_t n) {
  handle_t *h = find(id);
  if (!h) { snprintf(err, n, "there is no handle %d", id); return NULL; }
  if (!(h->holders & bit(W))) { snprintf(err, n, "%s does not hold handle %d", W->name,
id); return NULL; }
  if (revoked(h)) { snprintf(err, n, "handle %d was revoked", id); return NULL; }
  if (type >= 0 && h->type != type) { snprintf(err, n, "handle %d is a %s handle, not
%s", id, type_name(h->type), type_name(type)); return NULL; }
  if ((h->rights & right) != right) {
    snprintf(err, n, "handle %d is %s", id, (right & R_WRITE) ? "read-only" : (right &
R_GRANT) ? "not grantable" : "write-only");
    return NULL;
  }
  return h;
}
```

The order of the checks is deliberate. Existence first, then holding (a world cannot use a handle number it guessed), then revocation, then the handle's type (an I2C handle cannot be used as a framebuffer), then the specific right. Every failure produces a message naming the world and the handle, which the world receives as a thrown error. Creating a handle in the first place goes through a claim check, here for an I2C device:

▸ nucleus/vm.c

```
static JSValue op_hw_i2c(JSContext *ctx, world_t *W, int argc, JSValueConst *argv) {
  char *claim = sarg(ctx, argv, argc, 0);
…
  else if (!world_claims(W, claim)) r = fail(ctx, "%s has no claim on %s", W->name,
claim);
  else {
    handle_t *h = handle_new(H_I2C, W, claim);
…
```

Passing a handle in a message goes through `check_grants`, and the two system operations go through `world_claims` against `sys.read` and `sys.admin`. That is the whole monitor: a few dozen lines.

Always invoked. A reference monitor that can be bypassed is not a monitor. In a hardware-protected system, the processor guarantees mediation: a user-mode process that touches a device register faults, and the only way into the kernel is a trap. In PicoCalc OS there is no trap. Mediation is guaranteed instead by what a world's JavaScript can reach. QuickJS creates each runtime's global object with the ECMAScript built-ins and nothing else; it has no file system, no network and no way to name a memory address. The nucleus then adds exactly two functions before running anything:

▸ nucleus/vm.c

```
  JSValue g = JS_GetGlobalObject(ctx);
  JS_SetPropertyStr(ctx, g, "__k", JS_NewCFunction(ctx, host_k, "__k", 1));
  JS_SetPropertyStr(ctx, g, "__ka", JS_NewCFunction(ctx, host_ka, "__ka", 1));
…
  JSValue r = JS_Eval(ctx, K.prelude, K.prelude_len, "prelude.js", JS_EVAL_TYPE_GLOBAL);
```

`__k` runs operations that finish immediately and `__ka` operations that finish later and return a promise. Each looks its first argument up in a fixed table of operations and calls the matching C function with the calling world, which is how every check above learns who is asking:

▸ nucleus/vm.c

```
static JSValue host_k(JSContext *ctx, JSValueConst this_val, int argc, JSValueConst
*argv) {
…
  for (size_t i = 0; i < sizeof SYNC_OPS / sizeof SYNC_OPS[0]; i++)
    if (!strcmp(op, SYNC_OPS[i].name)) {
      JS_FreeCString(ctx, op);
      K.seg_cost += 1;   // a host call costs about a microsecond
      return after_host_call(ctx, W, SYNC_OPS[i].fn(ctx, W, argc - 1, argv + 1));
    }
  JSValue r = fail(ctx, "no such kernel operation: %s", op);
```

The prelude, evaluated next, captures both functions in a closure and deletes them from the global object:

▸ js/prelude.js

```
"use strict";
(function (g) {
  const k = g.__k, ka = g.__ka;
  delete g.__k;
  delete g.__ka;
```

The deletion is tidiness, not protection. The prelude runs inside the world's own runtime, so a world could in principle tamper with it, and nothing would change: the prelude makes no access decisions. Its hardware API is a thin wrapper, `hw.i2c` being `(claim, hz) => wrap(k("hw.i2c", claim, hz))`, and every check happens in the C function that `k` reaches. A world that somehow called `__k` directly would meet the same checks. That is the right place for a monitor: below everything the monitored code can modify.

You can see what a world can reach by asking one. The REPL sends each line to the sandbox world, so typing `log(Object.getOwnPropertyNames(globalThis).join(" "))` there lists the sandbox's globals in the kernel log. This is the host build's actual output:

```
[  5834.822] sandbox: Error EvalError RangeError ReferenceError SyntaxError TypeError
URIError InternalError AggregateError SuppressedError Array Object Function Iterator
parseInt parseFloat isNaN isFinite decodeURI decodeURIComponent encodeURI
encodeURIComponent escape unescape Infinity NaN undefined eval Number Boolean String Math
Reflect Symbol DisposableStack globalThis BigInt Date RegExp JSON Proxy Map Set WeakMap
WeakSet ArrayBuffer SharedArrayBuffer Uint8ClampedArray Int8Array Uint8Array Int16Array
Uint16Array Int32Array Uint32Array BigInt64Array BigUint64Array Float16Array Float32Array
Float64Array DataView Atomics Promise AsyncDisposableStack WeakRef FinalizationRegistry
now self log print hw dma gfx sys send publish ask sleep driver service app __init
errText __dispatch __render ui show
```

Everything up to `FinalizationRegistry` is standard ECMAScript; everything from `now` on is the prelude's API. There is no `__k`. Asking for the I2C device from the sandbox produces the claim check's message, `sandbox has no claim on i2c0:0x1f`, and asking to kill the keyboard driver produces `sandbox has no claim on sys.admin`. Chapter 10 audits this list for authority that should not be there, and finds one leak that was there.

TRY 8.1

List what a world can reach

The sandbox below runs JavaScript in a Web Worker with a prelude modelled on PicoCalc OS's: `Date` reads a virtual clock and the browser's network and storage functions are removed. It differs from a PicoCalc OS world in two ways: its time budget is measured in real milliseconds, because the browser has no virtual clock to charge, and a worker has no memory limit. Run the code to list the worker's globals, and read the audit beneath the output, which lists what remains reachable. Compare both with the list above. The browser's list is longer, because a worker's global scope carries web APIs that QuickJS does not have; each extra name is authority that the sandbox's prelude has to remove or replace, and one of them, dynamic `import()`, cannot be removed at all because it is syntax rather than a global.

```
console.log(Object.getOwnPropertyNames(globalThis).sort().join("
"))
```

run 1 virtual clock at +1 s world epoch 1 ok

```
AbortController AbortSignal AggregateError Array ArrayBuffer
AsyncDisposableStack AudioData AudioDecoder AudioEncoder
BackgroundFetchManager BackgroundFetchRecord
BackgroundFetchRegistration BigInt BigInt64Array BigUint64Array
Blob Boolean ByteLengthQueuingStrategy CSSSkewX CSSSkewY Cache
CacheStorage CanvasGradient CanvasPattern CloseEvent
CompressionStream CountQueuingStrategy CreateMonitor CropTarget
Crypto CryptoKey CustomEvent DOMException DOMMatrix
DOMMatrixReadOnly DOMPoint DOMPointReadOnly DOMQuad DOMRect
DOMRectReadOnly DOMStringList DataView Date DecompressionStream
DedicatedWorkerGlobalScope DisposableStack EncodedAudioChunk
EncodedVideoChunk Error ErrorEvent EvalError Event EventTarget
File FileList FileReader FileReaderSync FileSystemDirectoryHandle
FileSystemFileHandle FileSystemHandle FileSystemObserver
FileSystemSyncAccessHandle FileSystemWritableFileStream
FinalizationRegistry Float16Array Float32Array Float64Array
```

REMOVED BEFORE YOUR CODE RUNS

STILL REACHABLE

- network: `import() (syntax)`

- network: `fetch`, `XMLHttpRequest`, `WebSocket`, `EventSource`, `WebTransport`, `importScripts`, `fonts`, `FontFace`
- storage: `indexedDB`, `caches`, `webkitRequestFileSystem`, `webkitRequestFileSystemSync`, `webkitResolveLocalFileSystemURL`, `webkitResolveLocalFileSystemSyncURL`
- time: `performance`, `setTimeout`, `setInterval`, `clearTimeout`, `clearInterval`, `requestAnimationFrame`, `cancelAnimationFrame`, `scheduler`
- concurrency: `Worker`, `BroadcastChannel`, `MessageChannel`, `Atomics`
- environment: `navigator`, `location`, `crypto`, `close`, `postMessage`, `addEventListener`, `removeEventListener`, `dispatchEvent`, `when`, `onmessage`, `onerror`, `reportError`, `OffscreenCanvas`, `createImageBitmap`, `Notification`

Other non-standard globals: none. Everything else is plain ECMAScript: objects, functions, eval, WebAssembly, the seeded Math.random and the virtual Date.

Tamper-proof. Here PicoCalc OS differs most from a hardware-protected kernel. The nucleus's handle table, its world records and its event queue are ordinary C data in the same address space as every world's heap. A world cannot write them because JavaScript gives it no way to form a pointer, not because the processor forbids it. Tamper-proofness of the monitor therefore depends on QuickJS being memory-safe: on its bounds checks, its type checks, its garbage collector and its parser all being correct. A single exploitable bug in the engine, of the kind regularly found in production JavaScript engines, would let a world write anywhere, including into the handle table.

Small enough to analyse. This is what the TCB costs. Table 8.3 lists what PicoCalc OS's protection depends on, with the line counts measured for this chapter (`wc -l nucleus/*.c nucleus/nucleus.h`, leaving out `nucleus/font_data.h`, which is a generated table of glyph bitmaps; and `wc -l` over

QuickJS's sources in `third_party/quickjs`).

Table 8.3. The trusted computing base of PicoCalc OS's isolation on the host build. The nucleus is small; the engine it relies on is not.

Component	Size	Why it is trusted
The nucleus: `kernel.c`, `vm.c`, `hw.c`, `gfx.c`, `util.c`, `trace.c`, `png.c`, `nucleus.h`	2,374 lines	makes every access decision
QuickJS	80,864 lines	keeps each world inside its own heap
The C library and the compiler	(not counted)	everything above runs on them
The board: simulated controllers on the host; ESP-IDF and FreeRTOS on the device (planned)	215 lines on the host	carries out the transfers the nucleus approves

In object code the imbalance is similar: on the 32-bit build, QuickJS's text section is about 1,063 KB and the nucleus's 147 KB, of which 64 KB is the font. The nucleus itself is in the same size class as seL4's 8,700 lines of C, a kernel small enough that its functional correctness was proved by machine-checked proof [Klein et al. 2009]. The engine is not. This is the central trade of language-based isolation, and Chapter 10 takes it up: you exchange a hardware boundary, enforced by a few thousand gates that are tested by every program that runs, for a software boundary enforced by a large, complex program.

8.4 Hardware protection

To see what PicoCalc OS gives up, it helps to be precise about what hardware offers. Three mechanisms matter.

Privilege modes. A processor that distinguishes privilege levels runs privileged code (the kernel) and unprivileged code (applications) with different permissions. RISC-V defines machine mode (M), supervisor mode (S) and user mode (U); M-mode can do anything, U-mode cannot touch control registers or, where protection is configured, memory it has not been given. The only way from U-mode to a more privileged mode is a trap: an `ecall` instruction, an interrupt, or a fault. Traps are what make mediation unavoidable: a user program that wants something from the kernel must ask through a fixed entry point.

Memory management units. On a desktop or server processor, an MMU translates every virtual address through per-process page tables, and each page-table entry carries permission bits. Translation gives every process its own address space, so one process cannot even name another's memory, and the permission bits enforce read, write and execute rights page by page. Switching processes means switching page tables, which has a cost in cache and TLB misses that microkernel designers spent the 1990s minimising.

Region-based protection. Microcontrollers usually have no translation. Instead they have a memory protection unit, which checks accesses against a small number of address regions, each with permissions. RISC-V's version is **physical memory protection** (PMP). The privileged architecture specification allows up to 64 PMP entries per hardware thread, each a configuration byte and an address register; accesses from S-mode and U-mode are checked against the entries, and an entry whose lock bit is set constrains M-mode as well and cannot be changed until the next reset [Waterman et al. 2021]. The ESP32-P4's high-performance cores are RISC-V cores with PMP; ESP-IDF programs and locks the PMP configuration during early start-up to protect the system's own code and data [ESP-IDF memory protection n.d.]. Tock uses a microcontroller's MPU in exactly this way to isolate untrusted processes on devices with 64 KB of memory, while isolating its own kernel extensions by Rust's type system [Levy et al. 2017].

Region-based protection is coarse. A handful of regions cannot describe thirteen worlds, each with a heap made of many allocations scattered through PSRAM, without either restricting each world to a contiguous arena (and switching region registers on every segment) or giving up. And ESP-IDF does not give each task its own page tables in the way a desktop operating system gives each process an address space. For a system whose components are small, numerous and frequently communicating, the practical choice on such a chip is between coarse hardware regions and a language runtime. PicoCalc OS chose the runtime and, as of this writing, uses none of the PMP. Section 8.6 describes one way the port could use it.

8.5 Kernel structures

Where a system puts its reference monitor, and how much else it puts beside it, determines its structure. Figure 8.1 draws four answers side by side, and Table 8.4 compares them.

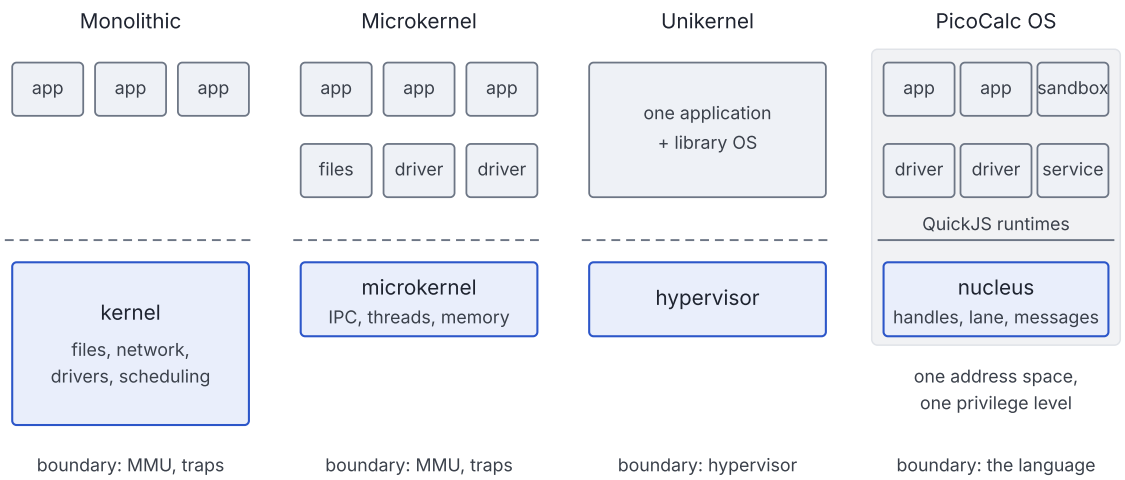


Figure 8.1. Four structures. Shaded boxes run privileged; the dashed line is the boundary the hardware or the hypervisor enforces. In PicoCalc OS there is no hardware boundary between worlds and nucleus; the language runtime draws the line.

Monolithic kernels. UNIX put the file system, the terminal handlers, the device drivers and process management into one privileged program, and protected that program from users with the hardware [Ritchie and Thompson 1974]. Its descendants, including Linux, follow the same structure. Inside the kernel, components are protected from users but not from each other: a driver runs with every privilege the kernel has, and a stray write in a driver can corrupt the file system's data structures. The design is fast, because a request from the file system to a driver is a function call, and it is the structure in which most of the world's code runs. Chapter 15 returns to what it means for reliability that drivers live inside it.

Microkernels. A microkernel keeps only what must be privileged inside the kernel and moves everything else, including file systems and drivers, into separate user-mode processes that communicate by messages. Brinch Hansen's RC 4000 nucleus, which provided processes and four message primitives (send message, wait message, send answer, wait answer) and left scheduling policy to processes, is the ancestor of the idea [Hansen 1970]. Mach made it the basis of a UNIX-compatible system, with tasks, threads, ports for messages and external pagers for memory management [Accetta et al. 1986]; its message passing was slow enough that later systems moved services back into the kernel. Liedtke's L4 answered with a principle and an implementation: a concept belongs inside the microkernel only if moving it outside, so that competing implementations could exist, would prevent the system from providing its required functionality; and the slowness of first-generation microkernels came from how they were built, not from the idea [Liedtke 1995]. QNX built a commercial real-time system on a message-passing microkernel [Hildebrand 1992]. MINIX 3 runs every driver as an isolated user process and restarts drivers that fail, which is the subject of Chapter 15

[Herder et al. 2006]. seL4, a member of the L4 family, has a machine-checked proof that its C implementation follows its specification [Klein et al. 2009]. The common thread is that the boundary between components is the hardware's, so a driver bug cannot write into another component's memory, at the cost of a message and a context switch wherever a function call used to be.

Exokernels. An exokernel goes further: the kernel only multiplexes the hardware securely (which process may use which disk blocks, which physical pages, which network packets), and the abstractions applications see are implemented in libraries linked into each application [Engler et al. 1995]. Protection stays in the kernel; abstraction moves out.

Unikernels. A unikernel compiles one application together with a library operating system into a single image that runs directly on a hypervisor, in one address space, with no separate kernel at all [Madhavapeddy et al. 2013]. Isolation between applications comes from the hypervisor, one virtual machine per application. Inside the image there is no protection boundary, though MirageOS, the system that paper describes, is written in OCaml, so inside the image the language's type safety rules out large classes of memory errors.

Table 8.4. Kernel structures compared by where protection lives. PicoCalc OS combines a microkernel's decomposition with a unikernel's single address space.

Structure	Privileged code	Isolation between components	Communication between components	Examples
Monolithic	the whole kernel, drivers included	user processes from the kernel, by the MMU; none inside the kernel	function calls inside the kernel; system calls from processes	UNIX, Linux
Microkernel	IPC, threads, address spaces	every server and driver in its own address space	messages through the kernel	Mach, L4, seL4, QNX, MINIX 3
Exokernel	secure multiplexing of hardware	applications by the MMU; abstractions in libraries	system calls, shared memory	Aegis/ExOS
Unikernel	the hypervisor	one application per virtual machine	the virtual network	MirageOS
Language-based	the runtime and a small kernel, all in one address space	by the language: components cannot forge references	messages or typed calls	Singularity, JX, PicoCalc OS

8.6 Where PicoCalc OS stands

PicoCalc OS sits in the last row of that table, and it is worth stating its position precisely, one property at a time.

Its **decomposition** is a microkernel's. The drivers for the keyboard, the power controller and the display are separate worlds; so is the compositor; so is every app. The nucleus provides what Brinch Hansen's did and a little more: running components, messages between them, handles to hardware, and supervision. Policy lives in worlds: the compositor decides layout and focus, the launcher decides what to start. If you moved the compositor into the nucleus, the system would be faster and its failures would be fatal, which is the trade Mach's successors made in the other direction.

Its **address space and privilege** are a unikernel's. There is one of each. Every world, the nucleus, the board and the C library share them.

Its **enforcement** is language-based. The boundary around each world is the edge of its QuickJS runtime: a world's code can reach the objects in its own heap, the ECMAScript built-ins and the prelude, and through the prelude, the two host functions. Figure 8.2 shows the path from a world's code to the hardware and names what enforces each step.

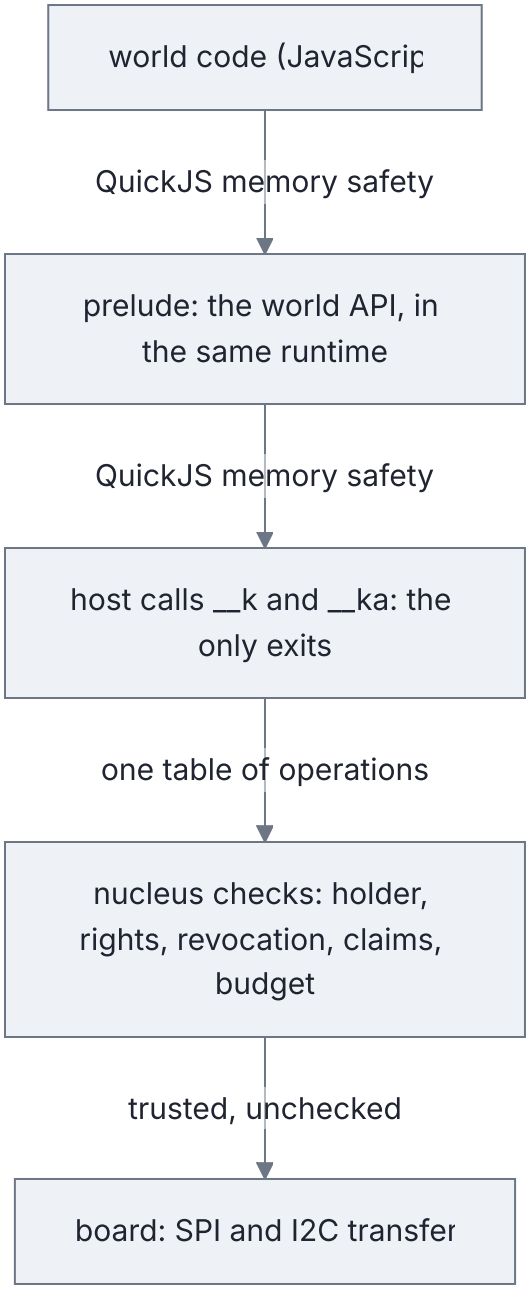


Figure 8.2. The path from a world's code to the hardware, and what enforces each step. Only the last two steps are checked by the nucleus; the first is enforced by QuickJS's memory safety, and nothing checks below the nucleus.

Put together, the system protects some things well and others not at all, and a reader who uses or extends it should know which is which.

What is protected. A world's heap is protected from every other world: there are no shared objects between runtimes, only copies made by serialising messages (Chapter 12). Hardware is protected from worlds that hold no handle for it, and every use of a handle is mediated. Processor time is protected from a runaway world by its segment budget, and memory by its heap limit (Chapter 11). The rest of the system is protected from a world's crash by supervision, which discards the crashed segment's effects, frees its runtime and restarts it (Chapter 14).

What is not protected. The nucleus is not protected from a memory-safety bug in QuickJS, because they share an address space; the TCB includes the engine. Worlds are not protected from each other's messages: any world may send to any world it can name, and channels are guarded by declarations each world makes about itself. (The first pass closed both gaps and bounded each world's share of the handle table; see the note in Section 8.1.) Chapter 9 shows a world with no claims forging a key press and another app's screen. Shared mechanisms, including the handle table, the lane and the buses, are channels through which one world can degrade service to others, as the handle-exhaustion test above showed. Confidentiality against side channels is not addressed at all; Chapter 10 explains why language-based isolation cannot address it on speculative hardware. And nothing below the nucleus is checked: the board carries out whatever the nucleus asks.

What hardware could add. The ESP32-P4's PMP could narrow the first of those gaps without giving up the language-based structure. One possible design, not implemented, runs the QuickJS interpreter for each segment in U-mode with PMP entries that allow it to read and write only the PSRAM arena holding that world's heap, turns `__k` and `__ka` into `ecall` traps into an M-mode nucleus, and locks the nucleus's code and data against U-mode access. A memory-safety bug in the engine would then corrupt at most the world that triggered it. The cost would be a trap per host call and a reprogramming of PMP entries per segment, and the engine's allocator would have to keep each world's heap in its own region. Whether that cost is worth paying on a device whose components are written by one author is a design question this book leaves open; Chapter 20 lists it among the port's open problems.

PicoCalc OS's protection goal is therefore narrower than a general-purpose system's, and it is worth saying what it is. Its components are not mutually hostile programs from different authors; they are drivers and apps written to cooperate, any of which may be buggy. The design aims for **fault isolation** (a bug in one world cannot corrupt another's state, and a crash is contained and repaired) and for **authority confinement** over hardware (a world can drive a device only if the manifest, or a world allowed to delegate, gave it a handle). It does not aim to contain a determined attacker who controls a world's code. Being precise about that goal is what lets the next two chapters judge the mechanisms fairly.

KEY POINTS

- Protection decides, for every operation, whether a principal holds the right to perform it on an object; policy says who may, mechanism enforces it, and PicoCalc OS keeps its policy in the manifest and its mechanism in the nucleus.
- PicoCalc OS guards hardware and system operations with rights, mailboxes and channels with names, and processor time and memory with quantities; only the first kind fails safe.
- Saltzer and Schroeder's principles are pressures rather than rules: PicoCalc OS mediates every handle use completely, but shared a 128-slot handle table that one world could exhaust (until the first pass gave each world a quota), and gives two drivers full rights on one I2C device.
- A reference monitor must be tamper-proof, always invoked and small enough to analyse; PicoCalc OS's checks are a few dozen lines, always invoked because `__k` and `__ka` are the only exits from a runtime, and tamper-proof only as long as QuickJS is memory-safe.
- The trusted computing base for isolation is the 2,374-line nucleus plus the 80,864-line engine; language-based isolation trades a hardware boundary for a large software one.
- Hardware offers privilege modes, MMUs with per-process translation, and region-based protection such as RISC-V PMP; the ESP32-P4 has PMP, which PicoCalc OS does not yet use.
- Monolithic kernels, microkernels, exokernels and unikernels differ in what runs privileged and what enforces the boundaries between components; PicoCalc OS has a microkernel's decomposition, a unikernel's single address space, and a language runtime's enforcement.
- PicoCalc OS aims at fault isolation and authority confinement among cooperating components, not at containing a hostile component or protecting secrets from side channels.

This chapter has located PicoCalc OS's boundaries and named what enforces them. The next chapter looks at the objects inside the most important of those boundaries, the handles, and asks the question capability systems have asked since 1966: if authority is a reference, how do you share it, narrow it, take it back, and prove that it cannot leak?

EXERCISES

1. BASIC For each of Saltzer and Schroeder's eight principles, find one place in the PicoCalc OS sources that follows it and one that does not. Name the file and function.

2. BASIC Anderson's three requirements are tamper-proofness, being always invoked, and being small enough to analyse. For PicoCalc OS's nucleus, which of the three holds by construction of the C code, and which depends on QuickJS?

3. ANALYSIS The keyboard and power drivers both claim `i2c0:0x1f` with all rights. Describe two ways a buggy power driver could interfere with keyboard input. Then design a claim syntax that grants access to individual registers of an I2C device, and say which function in `nucleus/vm.c` or `nucleus/hw.c` would have to check it.

4. CODE Reproduce the handle-exhaustion test from Section 8.2: copy `js/` to a scratch directory, add two worlds to a manifest there (one with the `dma` claim that allocates a framebuffer and passes a read-only view, one that narrows the view in a loop), and run `./build/picocalc-host -v -j DIR -m DIR/system.json -`. Then add a per-world quota to `handle_new` (count the handles a world has minted) and show that the second world now fails while the first can still allocate.

5. ANALYSIS UNIX checks permissions at `open()`; PicoCalc OS checks at every use. Estimate the cost of `handle_use` per call on the host build from the code (how many slots does `find` scan in the worst case?), and the cost of one revocation, which marks every descendant in repeated passes over the table (Chapter 9).. What does checking at every use make possible that checking once does not?

6. CODE List every operation reachable through `__k` and `__ka` (read `SYNC_OPS` and `host_ka` in `nucleus/vm.c`). For each, write down which check, if any, guards it: a claim, a handle right, a name, or nothing.

7. DISCUSSION Is the prelude part of PicoCalc OS's trusted computing base? Answer separately for authority over hardware and for determinism of the virtual clock (Chapter 10 discusses the second).

8. ANALYSIS Apply Liedtke's minimality principle to the nucleus. `gfx_fill`, `gfx_text` and the damage diff in `gfx_commit` are in C rather than in the compositor. Would moving them into a world prevent the system from providing its required functionality, or only make it slower? Estimate the slowdown using the virtual clock's cost of a step (665 ns) and the number of pixels in a full-screen fill.

9. PROJECT Sketch the PMP design from Section 8.6 for the ESP32-P4 port in detail: which code runs in U-mode, how many PMP entries are needed and what each covers, what happens on a host call, and what has to change in QuickJS's allocator. Estimate the added cost per segment if each trap costs 1 µs and each segment makes, on average, the number of host calls you count in the keyboard driver's poll.

10. DISCUSSION A unikernel and PicoCalc OS both run in a single address space. Compare what protects one component from another in each, and what an attacker who controls one component can do in each.

11. BASIC The sandbox world evaluates arbitrary code typed at the REPL. List the rows of Table 8.1 that bound what that code can do, and the rows that do not.

12. ANALYSIS Suppose QuickJS has a bug that lets a script write one 32-bit word at an address of its choosing. Using `nucleus/nucleus.h`, describe the smallest write that would let a world drive the SPI bus without a claim.

FURTHER READING

[Saltzer and Schroeder 1975] The first part contains the design principles and the vocabulary used in this chapter; the rest of the paper surveys 1970s protection hardware and repays reading for its precision.

[Anderson 1972] Volume I is short; the reference monitor appears in Section III.

[Liedtke 1995] The minimality principle and the argument that microkernels were slow because of how they were built, not because of what they are.

[Klein et al. 2009] What it takes to prove a kernel correct, and what the proof assumes (compiler, assembly, hardware).

[Engler et al. 1995] and [Madhavapeddy et al. 2013] Two arguments for putting less in the kernel and more in libraries, thirty years apart.

[Herder et al. 2006] A microkernel built around the expectation that drivers fail; the starting point for Chapter 15.

Capabilities

A capability is a reference that both names an object and carries the right to use it. That one idea, introduced in 1966, organises a whole family of operating systems, and it organises PicoCalc OS's handles. This chapter explains where it comes from and why it matters: the access matrix that every protection system implements in some form, the two ways of storing it, the failure that the more common way invites, and the capability systems built to avoid it. Then it turns to PicoCalc OS's handles and treats them as a capability system to be verified. By the end you should be able to design a handle mechanism with narrowing, delegation and revocation, state precisely what it confines, prove that it does, and recognise where it does not apply. Along the way the chapter reproduces one defect that the PicoCalc OS paper's fact-check found and fixed, and four more that were found, and fixed, while this book was being written.

9.1 The access matrix

Butler Lampson's 1971 paper "Protection" gave the field its basic model [Lampson 1971]. A system has a set of **domains** (Lampson's word for the principals of Chapter 8: the things on whose behalf code runs) and a set of **objects**. The **access matrix** has a row for each domain and a column for each object; the entry at row *d*, column *o* is the set of rights domain *d* holds on object *o*. Every access is a lookup: may *d* perform operation *op* on *o*? It is if *op*'s right is in the entry.

For PicoCalc OS, the domains are worlds and the objects of interest are the hardware devices and the system operations. Table 9.1 is the matrix a few hundred virtual milliseconds after boot, once the compositor has allocated its framebuffer and handed the display driver a view of it.

Table 9.1. Part of PicoCalc OS's access matrix shortly after boot. r, w and g are the read, write and grant rights of handles; sys.read and sys.admin are claims checked by name. Empty cells are the common case.

World	I2C `0x1F`	SPI `spi2`	framebuffer	`sys.read`	`sys.admin`
keyboard	rwg				
power	rwg				
display		rwg	r		
compositor			rwg, and r (the view it made)		
launcher				yes	
sysmon				yes	
faults				yes	yes
about				yes	
sandbox, repl, snake, keys, flaky					

The matrix is sparse, as access matrices always are, so no system stores it as a matrix. There are two natural ways to store a sparse matrix, and they correspond to the two great families of protection systems.

Stored **by column**, each object carries a list of the domains that may use it and with which rights. This is an **access control list** (ACL). UNIX file permissions are a compressed ACL (owner, group, other); Windows and most file systems have full ones. To decide an access, the system takes the object being accessed, looks up the requesting domain in its list, and checks the right.

Stored **by row**, each domain carries a list of the objects it may use and with which rights. Each entry in such a list is a **capability**, and the list is a **capability list** or C-list. To decide an access, the system takes the capability the domain presents, which both identifies the object and states the rights, and checks that it is genuine.

The two layouts hold the same information at any one moment, which led to a long-standing belief that the choice between them is a matter of implementation. Miller, Yee and Shapiro called this the Equivalence Myth and showed what it misses [Miller et al. 2003]: the matrix is not static. Authority moves. A program passes a file to a helper, a server passes a result back to a client, a compositor hands a framebuffer to a display driver, and the systems differ enormously in how such changes are expressed and controlled. The rest of this chapter is about those dynamics.

9.2 Ambient authority and the confused deputy

In an ACL system, a process designates an object by name, and the system decides whether the process may use it by looking up the process's identity in the object's list. The process does not present its authority; the system finds it. Authority that a program exercises without designating it, because the system grants it on the basis of who the program is, is **ambient authority**.

Norm Hardy's short 1988 paper shows what goes wrong [Hardy 1988]. On Tymshare's time-sharing system, whose protection was much like UNIX's, a Fortran compiler lived in a system directory, `SYSX`. Users could name a file to receive the compiler's debugging output. The compiler also wrote usage statistics to `(SYSX)STAT`, so it was given a "home files license" that let it write files in its own directory. The billing file, `(SYSX)BILL`, was in the same directory. A user who learned its name passed `(SYSX)BILL` as the debugging output file. The compiler opened it, the system checked the compiler's license and allowed the write, and the billing data was overwritten.

No line of the compiler was wrong. The compiler held authority from two sources, the user who invoked it and its own license, and when it opened a file it had no way to say which authority it meant to use. Hardy called it a **confused deputy**: a program acting on behalf of a client with authority the client does not have, tricked into exercising that authority for the client. His conclusion was that the remedy is not more checks but a different primitive: if a client passes the deputy a capability for the output file, rather than a name, the deputy can only write where the client could, and the question of which authority to use never arises. Designation and authority travel together.

PicoCalc OS has a small deputy of its own, and it is instructive because it is mild. The display driver holds two pieces of authority: the SPI bus, from its claim, and a read-only view of the framebuffer, which the compositor gives it. Its `flush` handler sends rectangles of that framebuffer to the panel:

▸ js/drivers/display.js

```
  .on("flush", async (s, { rects }) => {
    // after a restart, flushes queued for the old instance can arrive before the
compositor re-attaches
    if (!s.fb || !s.ready) return { px: 0, skipped: true };
…
    for (const [x, y, w, h] of rects) {
      // queued together: the transfers run back to back on the bus once this turn ends
      jobs.push(s.spi.cmd(CASET, span(x, x + w - 1)), s.spi.cmd(RASET, span(y, y + h -
1)), s.spi.cmd(RAMWR), s.spi.pixels(s.fb, x, y, w, h));
```

The rectangles come from the requester; the framebuffer and the bus are the driver's own. The handler does not ask who sent the request, and the nucleus lets any world send `flush` to the display. (Since the first pass the manifest lets only the compositor send to or ask the display driver; *From L4 to Small Worlds*, Chapter 5.) A world with no claims can therefore make the display driver transfer the whole screen, again and again, keeping the SPI bus busy with work nobody needed. It cannot show anything the compositor did not draw, because the pixels come from the compositor's framebuffer, so the harm is to availability rather than integrity. The capability-style repair is Hardy's: the compositor passes the framebuffer view in each `flush` request, and the driver sends only from the handle it was given with the request. A world without a view cannot then ask for a flush that does anything. The ACL-style repair is to check the sender's name. Both work here; the difference shows when the system grows a second compositor.

9.3 Capability systems

Jack Dennis and Earl Van Horn introduced the word in 1966 [Dennis and Van Horn 1966]. In their design each computation runs within a **sphere of protection** defined by a list of capabilities kept by the supervisor; a capability locates an object and states the access the holder has to it; a program refers to objects by their index in its own list, so it can only ever name what it holds; and capabilities can be passed between computations. Every capability system since has kept those elements and varied the rest: where the capabilities are stored, how they are protected from forgery, and how they move.

There are three ways to make a capability unforgeable. In **hardware capability machines**, a tag bit marks memory words that hold capabilities, and only privileged instructions can create or modify tagged words; Levy's book surveys the machines built this way, from the Plessey System 250 to the Intel iAPX 432 [Levy 1984]. In **kernel-held** (or partitioned) systems, the capabilities live in kernel memory and a program holds only an index, as in Dennis and Van Horn's design; forging an index gets you nothing, because the index is

interpreted in your own list. In **language-based** systems, an object reference in a memory-safe language is itself the capability, because the language provides no way to manufacture one. PicoCalc OS uses the second kind for hardware and, as Section 9.4 explains, gets the third kind for free inside each world.

A few systems show the range.

KeyKOS called capabilities keys, "tokens of authority", and built an entire commercial operating system on them [Hardy 1985]. Everything was reached by key: pages, segments, the domains that execute programs, and **meters**, which "account for certain system resources including CPU time". A domain could compute only while it held a key to a meter with time left on it. That is the idea behind PicoCalc OS's budgets (Chapter 11), expressed as a capability. **EROS** rebuilt KeyKOS's design and showed that a capability system could be fast [Shapiro et al. 1999].

seL4 keeps capabilities in kernel objects called CNodes, lets a holder copy a capability with fewer rights, and records which capabilities were derived from which, so that revoking one removes everything derived from it; the capability operations are part of the kernel whose functional correctness was proved [Klein et al. 2009].

Capsicum added capabilities to FreeBSD without replacing UNIX [Watson et al. 2010]. A process that calls `cap_enter` enters **capability mode**: it loses access to global namespaces such as file paths and process IDs and can act only through file descriptors it already holds, each of which carries a mask of permitted operations. Existing programs were sandboxed by opening what they need first and entering capability mode after.

Zircon, the kernel of Google's Fuchsia, gives processes **handles** to kernel objects, each with a set of rights. A handle can be duplicated only if it carries `ZX_RIGHT_DUPLICATE` and sent through a channel only if it carries `ZX_RIGHT_TRANSFER`, and rights "can be reduced via `zx_handle_replace()` or `zx_handle_duplicate()`" [Zircon rights n.d.]. PicoCalc OS's three rights correspond closely: `r` and `w` govern use, and `g` plays the role of the transfer right.

Table 9.2. How several capability systems answer four questions. Where a system has no revocation operation of its own, the general technique, a forwarder that can be switched off (Section 9.4), still applies.

System	What the program holds	Reducing rights	Passing on	Taking back what you gave
KeyKOS, EROS	keys in kernel-protected nodes	weaker key variants	in calls between domains	interpose a forwarder
seL4	slots in CNodes	copy with fewer rights	through endpoints	revoke all derived capabilities
Capsicum	file descriptors with rights masks	a narrower mask	over UNIX-domain sockets	interpose a forwarder
Zircon	handles with rights	`zx_handle_replace`, `zx_handle_duplicate`	needs `ZX_RIGHT_TRANSFER`	interpose a forwarder
PicoCalc OS	handle ids, checked against holder sets	`narrow`	needs `g`, or a narrowing of a passable handle	`revoke` what you narrowed

9.4 Object capabilities

In a memory-safe language, a reference to an object is already a capability: it designates the object, it lets the holder invoke the object's methods, and it cannot be forged, because the language has no operation that turns a number into a reference. Mark Miller's dissertation develops this observation into a discipline, the **object-capability model**, and a programming language, E, built on it [Miller 2006]. Its central rule is that "only connectivity begets connectivity": an object can come to hold a reference to another in exactly four ways. It can be created holding it (initial conditions), it can create the other object (parenthood), it can be given the reference when it is created (endowment), or a third object that holds references to both can pass one to the other in a message (introduction).

The rule turns security reasoning into reasoning about a graph. To know what a piece of code can affect, follow the references it holds. To limit what it can affect, limit what you give it. The **principle of least authority** (POLA) is least privilege applied to references: give each component the references it needs and no others.

Three patterns recur in object-capability programming, and PicoCalc OS's handles provide each of them as a kernel operation.

Attenuation gives a holder a weaker version of a capability. In a language, attenuation is a wrapper object that forwards some methods and not others. In PicoCalc OS, it is `narrow`, which makes a child handle whose rights are the intersection of the parent's and the requested ones.

Delegation passes a capability to another party. In a language, it is passing a reference as an argument. In PicoCalc OS, it is placing a handle in a message, which the nucleus checks and then grants to the receiver.

Revocation takes authority back. Capabilities have a reputation for being irrevocable, which Miller, Yee and Shapiro list as the third myth [Miller et al. 2003]. The standard answer is Redell's **caretaker** [Redell 1974]: instead of passing a capability directly, pass a forwarder that holds it and can be told to stop forwarding. The holder of the forwarder never held the original and so cannot keep it. A **membrane** extends the caretaker transitively, wrapping every reference that passes through it, so that revoking the membrane cuts off everything handed across it. In PicoCalc OS the nucleus builds the caretaker in: a world that narrows a handle can later revoke the narrowed copy, and every handle derived from it stops working.

The same model applies inside each world, without the nucleus's help. A world's JavaScript objects reach only what they hold references to; the prelude's closures hold `k` and `ka`, and nothing else in the world can reach them except by calling the prelude's functions. Chapter 10 shows that this inner layer has its own leaks, through the properties of built-in objects, and what it took to close one.

9.5 Handles in PicoCalc OS

A handle is an entry in a fixed table inside the nucleus. Its definition, from `nucleus/nucleus.h`, holds everything this chapter needs:

▸ history/first-pass/nucleus/nucleus.h

```
enum { H_I2C, H_SPI, H_DMA };
enum { R_READ = 1, R_WRITE = 2, R_GRANT = 4 };
typedef struct {
  int id, type, rights, parent, minter;
  uint64_t holders;         // bit per world id
  bool live, revoked;
  char claim[NAME_LEN];
  int bus, addr, hz;        // H_I2C / H_SPI
  int buf;                  // H_DMA: index into the buffer table (shared by a handle and
its children)
} handle_t;
```

Two fields make it a capability system rather than a lookup table. `holders` is a set of worlds, one bit per world, recording who may use the handle; a world refers to a handle by its `id`, a small integer that means nothing unless the world's bit is set. In the vocabulary of Section 9.1, each world's capability list is the set of handles whose `holders` include it, so the list is held by the kernel, as in Dennis and Van Horn's design, and the integer a world holds is an index that cannot be forged into authority. `parent` and `minter` record where the handle came from: the handle it was narrowed from, and the world that narrowed it. A handle made from a claim has no parent, and its minter is the world that claimed it.

A handle's life has five operations.

Creation by claim. `hw.i2c("i2c0:0x1f")`, `hw.spi("spi2")` and `dma.alloc(w, h)` check the manifest's claims, as Chapter 8 showed, and create a handle with all three rights, held by the claimer.

Narrowing. `h.narrow("r")` calls `handle_child`, which makes a new handle of the same type, pointing at the same device or buffer, with the intersection of rights:

▸ history/first-pass/nucleus/hw.c

```
handle_t *handle_child(world_t *W, handle_t *p, int rights) {
  handle_t *h = handle_new(p->type, W, p->claim);
  if (!h) return NULL;
  h->rights = p->rights & rights; h->parent = p->id;
  h->bus = p->bus; h->addr = p->addr; h->hz = p->hz; h->buf = p->buf;
  if (h->buf >= 0) K.bufs[h->buf].refs++;
  return h;
}
```

`p->rights & rights` is the whole of attenuation: a child can never have a right its parent lacks. The child's minter is the narrowing world (`handle_new` records it), and a framebuffer's reference count goes up, because the child shares the parent's memory.

Passing. A handle travels in a message as an ordinary value. The prelude's `Handle` class serialises itself as `{"$h": id}`, and the prelude collects those ids into a list it gives the nucleus alongside the JSON:

▸ js/prelude.js

```
  function enc(v) {
    if (v === undefined) return [undefined, []];
    const hs = [];
    const json = JSON.stringify(v, (key, x) => {
      if (x && typeof x === "object" && typeof x.$h === "number" && Object.keys(x).length
=== 1) hs.push(x.$h);
      return x;
    });
    return [json, hs];
  }
```

The nucleus checks every listed handle before it accepts the message: the sender must hold it, and must be allowed to pass it on.

▸ history/first-pass/nucleus/vm.c

```
// handles named in a message must be held by the sender, and passable by it
static bool check_grants(JSContext *ctx, world_t *W, const int *hs, int nh) {
  char err[160];
  for (int i = 0; i < nh; i++) {
    handle_t *h = handle_use(W, hs[i], -1, 0, err, sizeof err);
    if (!h) { fail(ctx, "%s", err); return false; }
    if (!can_pass(W, h)) { fail(ctx, "handle %d cannot be passed on by %s", hs[i], W-
>name); return false; }
  }
  return true;
}
```

When the message is delivered, at the end of the sender's segment like every other effect, the nucleus adds the receiver to each handle's holders:

▸ history/first-pass/nucleus/kernel.c

```
      for (int i = 0; i < m->nh; i++)
        for (int k = 0; k < MAX_HANDLES; k++) if (K.handles[k].live && K.handles[k].id ==
m->hs[i]) handle_grant(&K.handles[k], W);
```

Passing is therefore **sharing**, not moving: the sender keeps the handle. UNIX's descriptor passing over sockets behaves the same way, giving the receiver a new descriptor for the same open file while the sender keeps its own. A system that wants a hand-off to be exclusive has to release after sending; PicoCalc OS's prelude does not do this for you.

Revocation. A world can revoke a handle it narrowed. The check is that the caller is the handle's minter and the handle has a parent (so it is a narrowing, not an original); the effect is deferred to the end of the segment, so a segment that crashes revokes nothing:

▸ history/first-pass/nucleus/vm.c

```
static void run_revoke(deferred_t *d, vtime_t at) {
  (void)at;
  handle_revoke_tree(d->handle);   // the handle and every copy narrowed from it
}
static JSValue op_h_revoke(JSContext *ctx, world_t *W, int argc, JSValueConst *argv) {
  int id = iarg(ctx, argv, argc, 0, 0);
  for (int i = 0; i < MAX_HANDLES; i++) {
    handle_t *h = &K.handles[i];
    if (!h->live || h->id != id) continue;
    if (h->minter != W->id || !h->parent) return fail(ctx, "only the world that narrowed
handle %d can revoke it", id);
    deferred_t *d = calloc(1, sizeof(*d));   // like every other effect, at the end of
the segment
    d->run = run_revoke; d->handle = id;
    defer(d);
    return JS_TRUE;
  }
  return fail(ctx, "there is no handle %d", id);
}
```

When the revocation takes effect, the nucleus marks the handle and every live handle narrowed from it, directly or through other narrowings, repeating its pass over the table until a pass marks nothing new:

▸ nucleus/hw.c

```
void handle_revoke_tree(int id) {
  handle_t *root = find(id);
  if (!root) return;
  root->revoked = true;
  for (bool more = true; more; ) {
    more = false;
    for (int i = 0; i < MAX_HANDLES; i++) {
      handle_t *k = &K.handles[i];
      if (!k->live || k->revoked || !k->parent) continue;
      handle_t *p = find(k->parent);
      if (p && p->revoked) { k->revoked = true; more = true; }
    }
  }
}
```

A revoked handle cannot be narrowed, because narrowing goes through `handle_use`, which refuses it; so no unmarked descendant can appear after the marking, and checking a handle on use needs only its own flag: `static bool revoked(handle_t *h) { return h->revoked; }`. Revocation is thus **eager**: its cost is paid once, when it happens, rather than on every later use. This is the caretaker pattern built into the kernel. To give another world revocable access, narrow first and pass the narrowing; the receiver never holds the parent, so revoking the narrowing cuts it off. Section 9.7 tells how an earlier, lazy version of this code failed.

Release. A world can give up a handle with `release`, and a crashed world stops holding all of its handles at once. A handle nobody holds is freed, unless copies narrowed from it are still alive: then it stays in the table, held by nobody, so that the derivation tree stays connected and a later revocation of it or of an ancestor still reaches every copy. When the last such copy goes, any ancestor that was kept only for it is freed too. A framebuffer's memory is freed when its reference count, shared by a handle and all its narrowings, reaches zero:

▸ nucleus/hw.c

```
void buf_unref(int i) {
  if (i < 0) return;
  dmabuf_t *b = &K.bufs[i];
  if (--b->refs > 0) return;
  free(b->back); free(b->front);
  memset(b, 0, sizeof(*b));
}

static bool has_children(int id) {
  for (int i = 0; i < MAX_HANDLES; i++) if (K.handles[i].live && K.handles[i].parent ==
id) return true;
  return false;
}

// Free a handle nobody holds, unless copies narrowed from it are still alive; then free
any ancestor
// that was kept only for this one.
void handle_release(handle_t *h, world_t *W) {
  h->holders &= ~bit(W);
  while (h && !h->holders && !has_children(h->id)) {
    int parent = h->parent;
    buf_unref(h->buf);
    h->live = false;
    h = parent ? find(parent) : NULL;
  }
}
```

The system's one real delegation shows all five operations at once. The compositor allocates a 320 × 320 framebuffer, narrows it to a read-only view, and sends the view to the display driver, letting go of the view it made for any previous instance of the driver:

▸ js/system/compositor.js

```
function attach(s) {
  // the view handed to a previous display instance died with it; let go of our hold on
it, or every
  // restart of the driver would keep one more handle alive
  if (s.view) s.view.release();
  s.view = s.fb.narrow("r");
  send("display", "attach", { fb: s.view });
  s.fb.invalidate();
  dirty(s);
}
```

Figure 9.1 traces what the nucleus does with that line, and Figure 9.2 shows the result in the system monitor's handle table: handle 5, a read-only DMA handle, held by both the display driver and the compositor that made it.

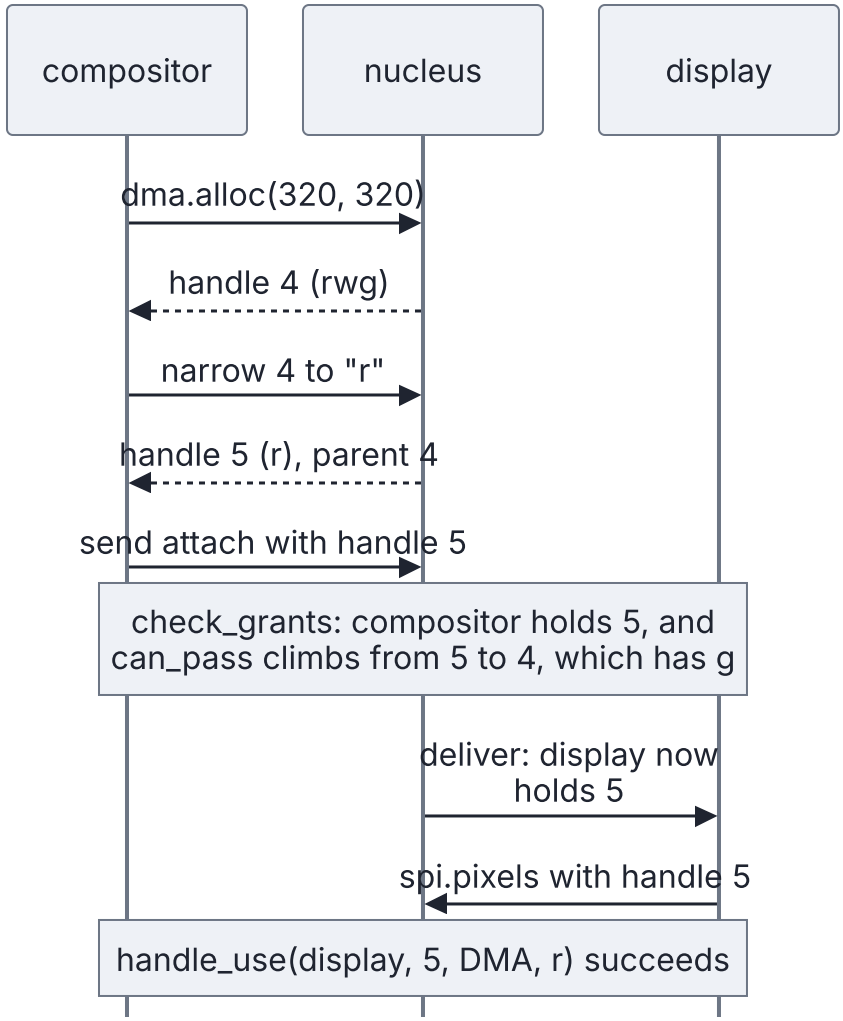


Figure 9.1. The compositor hands the display driver a read-only view of its framebuffer. The view is passable because it was narrowed by the compositor from a handle that carries the grant right.



Figure 9.2. The system monitor's Handles tab on the host build. Handle 5 is the read-only view; the compositor that narrowed it and the display driver it was passed to both hold it.

TRY 9.1

Pass, narrow and use a handle

The playground below implements PicoCalc OS's handle rules for three worlds. Start from the display scenario: the compositor holds a framebuffer. Narrow it to `r`, pass the view to the display, and use it there for a read; then try a write from the display and read the nucleus's answer. Go on to revoke the view, use it after the revocation, and release handles until the framebuffer's memory is freed. Each action shows the check the nucleus makes and its result.

```
can_pass(W, h): walk up from h
  if h has g: yes
  if W did not narrow h (or h has no parent): no
  h = h's parent, and repeat
```

compositor `claims dma`

~~**#2** `r` from #1, by compositor · revoked~~

display `claims spi2`

holds no handles

snake `no claims`

holds no handles

- ✓ **1. compositor:** `dma.alloc(320, 320)` → handle 1: a new framebuffer, rights rwg
  compositor claims dma in the manifest; framebuffer 0 has 1 handle
- ✓ **2. compositor:** `h1.narrow("r")` → handle 2: rights r, a child of handle 1
  the child gets the parent's rights (rwg) intersected with "r"; compositor is recorded as the world that narrowed it
- ✓ **3. compositor:** `send("display", "fb", {fb: h2})` → display holds handle 2 once the message is delivered (at the end of compositor's segment)
  handle 2 has no g; compositor narrowed it from handle 1, so look there
  handle 1 has g: yes
- ✓ **4. display:** `spi.pixels(…, h2, …)` → display may send handle 2's pixels to the panel
- ✓ **5. compositor:** `h2.revoke()` → handle 2 revoked at the end of compositor's segment
  revoke is an effect: if the segment crashed, the handle would stay valid

every later use, by any holder, fails

✗ **6. display:** `spi.pixels(…, h2, …)` → handle 2 was revoked

✓ **7. compositor:** `h1.release()` → compositor no longer holds handle 1
nobody holds handle 1 any more, so it is gone
framebuffer 0 still has 1 handle, so its memory stays

✓ **8. display:** `h2.release()` → display no longer holds handle 2
compositor still holds it

9.6 Confinement, a bug, and a proof

The PicoCalc OS paper states the property handles are meant to provide:

Property 4 (confinement). A world can perform an operation on a device only if it holds a handle with the right for it, and it can come to hold one only by a claim in the manifest, by narrowing a handle it holds, or by receiving one from a world allowed to pass it on.

The word "confinement" here means confinement of authority. Lampson used the word in 1973 for a stronger problem: preventing a program from leaking information to anyone but its caller, including through channels such as timing and resource usage that no access check sees [Lampson 1973]. Handles address only the first. Chapter 10 returns to the second.

The important clause is the last one: *a world allowed to pass it on*. The display driver receives a read-only, non-grantable view. It should be able to use the view and unable to give it to anyone else, directly or in any disguised form. An earlier version of the rule did not achieve that. It allowed a handle to be passed if it carried the grant right or if the passing world was its minter:

```
// the earlier rule (a sketch; this is not the current code)
static bool can_pass_old(world_t *W, handle_t *h) {
  return (h->rights & R_GRANT) || h->minter == W->id;
}
```

The second clause was meant to let the compositor pass the view it had just narrowed. But it does not say *narrowed from what*. Any world that holds a handle can narrow it, and the narrowed copy's minter is the world that narrowed it. So a world holding a non-grantable view could narrow the view, to the same rights,

and pass the copy, which the old rule accepted because the world was its minter. One extra call laundered a handle into a passable one.

The fact-check of the paper against the code found this, and the benchmark suite now tests it. Two worlds play the parts: `owner` allocates a framebuffer and gives a read-only view to `relay`; `relay` tries to pass the view on directly, then narrowed again, and then to draw with it.

▸ bench/worlds/owner.js

```
service(self).onStart(async () => {
  await sleep(10);
  const fb = dma.alloc(32, 32);
  const view = fb.narrow("r");
  const r = await ask("relay", "try", { view });
  report({ bench: "confine", ...r });
});
```

▸ bench/worlds/relay.js

```
service(self).on("try", (s, { view }) => {
  const out = {};
  try { send("ponger", "count", { h: view }); out.direct = "passed"; } catch (e) {
out.direct = e.message; }
  try { send("ponger", "count", { h: view.narrow("r") }); out.renarrowed = "passed"; }
catch (e) { out.renarrowed = e.message; }
  try { view.fill(0, 0, 1, 1, "#fff"); out.write = "wrote"; } catch (e) { out.write =
e.message; }
  return out;
});
```

Under the old rule the second attempt reported `passed`. With the current rule, `bench/results.json` records all three refused:

```
{"bench": "confine", "direct": "handle 2 cannot be passed on by relay",
 "renarrowed": "handle 3 cannot be passed on by relay", "write": "handle 2 is read-only"}
```

The current rule walks up the derivation tree:

▸ nucleus/vm.c

```
// A world can pass a handle on if the handle carries the grant right, or if the world
narrowed it from
// a handle it could pass on itself. (Narrowing a handle you may not pass does not make
it passable.)
static bool can_pass(world_t *W, handle_t *h) {
  for (int depth = 0; h && depth < 32; depth++) {
    if (h->rights & R_GRANT) return true;
    if (h->minter != W->id || !h->parent) return false;
    handle_t *p = NULL;
    for (int k = 0; k < MAX_HANDLES; k++) if (K.handles[k].live && K.handles[k].id == h-
>parent) { p = &K.handles[k]; break; }
    h = p;
  }
  return false;
}
```

Figure 9.3 shows the three handles of the test and how each rule treats handle 3.

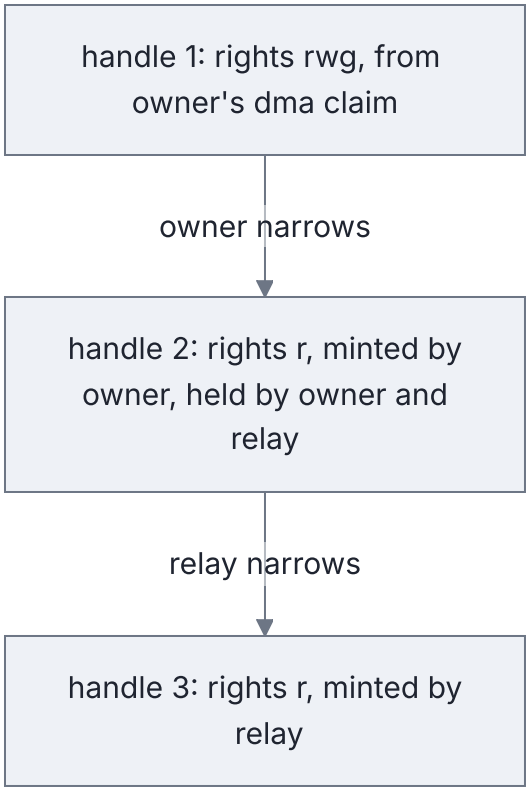


Figure 9.3. The derivation tree in the confinement test. The old rule looked only at handle 3 and saw that relay minted it. The current rule climbs from 3 to 2, finds that relay did not mint 2 and 2 has no grant right, and refuses.

A test shows that one attack fails. A proof shows that every attack of a kind fails, and this one is short enough to give in full. Write $R(h)$ for a handle's rights, $m(h)$ for its minter and $p(h)$ for its parent.

Lemma 1 (rights only shrink). For every narrowed handle c, $R(c) \subseteq R(p(c))$. This is the line `h->rights = p->rights & rights` in `handle_child`, and no other code changes a handle's rights.

Lemma 2 (what can_pass decides). Ignoring the depth limit, `can_pass(W, h)` returns true exactly when some ancestor a of h (counting h itself) has the grant right, and every handle on the path below a down to h was minted by W. *Proof*, by induction on the number of iterations of the loop. If h has `g`, the loop returns true at once, with $a = h$ and an empty path. Otherwise it returns false unless $m(h) = W$ and h has a parent,

and in that case it continues from $p(h)$, so it returns true exactly when the property holds for $p(h)$, which together with $m(h) = W$ is the property for h. (Every parent on the chain is live, because a handle with live descendants is never freed; see Section 9.5.)

Theorem (a non-grantable view cannot be passed on, in any form). Let world X hold a handle v with `g` $\notin R(v)$ and $m(v) \neq X$. Then `can_pass` refuses v, and refuses every handle that X derives from v by narrowing. *Proof.* Let d be v or any descendant of v. By Lemma 1, no handle on the path from v down to d has `g`, since v does not. So by Lemma 2 the ancestor a with `g` would have to lie strictly above v, and then v itself would be on the path below a and would have to be minted by X. It is not. ∎

The proof also locates the old bug exactly. The old rule's second clause is the induction step without the induction: it checks that W minted h and never asks whether W could pass h's parent. Lemma 2 is the precise statement of "narrowed it from a handle it could pass on itself".

The theorem says nothing about other paths to authority, and the property needs none: X has no `dma` claim, and any other handle to the same buffer would have to be given to it by a world that can pass one, which is the intended way to share.

9.7 Four defects found while writing this book

Checking the handle code against the proof above turned up four more defects. They were found while this book was being written, measured on the host build, and then fixed, and this section tells each the way the paper's fact-check is told: what was wrong, how it was found, the fix, and the evidence that the fix works. None of them violated the theorem, which is about passing; each concerned a different property that a capability system needs.

Revocation that failed open. The first two defects shared a cause. Revocation used to be **lazy**: `run_revoke` set a flag on the one handle being revoked, and every later use of any handle looked for the flag by walking up the chain of parents:

```
// before the fix (no longer in the repository)
static bool revoked(handle_t *h) {
  for (int depth = 0; h && depth < 32; depth++) {
    if (h->revoked) return true;
    h = h->parent ? find(h->parent) : NULL;
  }
  return false;
}
```

That makes revoking cheap, but a descendant is revoked only while the walk can reach the revoked ancestor, and the proof's two simplifying assumptions, ignore the depth limit and assume every parent is live, are exactly the two ways the walk could fail to reach it.

The first was found by asking what each loop returns when it hits its limit. `can_pass` gives up after 32 levels and answers *not passable*, which is safe. The old `revoked` gave up after 32 levels and answered *not revoked*, which is not. A world given a revocable view could narrow it 32 times and keep the last copy, and when the giver revoked the view the copy's walk stopped short of it. A pair of test worlds written for this chapter checks this: `owner` gives `holder` a read-write view without the grant right, `holder` narrows it *n* times, `owner` revokes the view, and `holder` tries to draw with both the view and its deepest copy. Table 9.3 gives the results before and after the fix.

Table 9.3. Drawing with a revoked view and with its deepest narrowed copy, after the owner revokes the view, before and after revocation was made eager. Each row is one run of the test on the host build.

Narrowings by the holder	The view	The deepest copy, before the fix	The deepest copy, after the fix
5	`handle 2 was revoked`	`handle 7 was revoked`	`handle 7 was revoked`
31	`handle 2 was revoked`	`handle 33 was revoked`	`handle 33 was revoked`
32	`handle 2 was revoked`	`wrote`	`handle 34 was revoked`
40	`handle 2 was revoked`	`wrote`	`handle 42 was revoked`

The second was found by asking when a parent stops being live. A handle used to be freed as soon as its last holder released it, and `find` does not return freed handles, so a walk that reached a freed parent stopped there and answered *not revoked*. Revoking and then releasing is exactly what a careful world does with a view it no longer wants to share, and it undid the revocation for every copy someone else had narrowed from the view. In this test the holder narrows its own copy of the view and lets the view itself go; the owner then revokes the view, and later releases it. Before the fix:

```
[    10.394] owner: given: "copy wrote"
[    15.567] owner: after revoke: "handle 3 was revoked"
[    20.741] owner: after release: "copy wrote"
```

and after it:

```
[    10.394] owner: given: "copy wrote"
[    15.567] owner: after revoke: "handle 3 was revoked"
[    20.741] owner: after release: "handle 3 was revoked"
```

The fix has the two halves shown in Section 9.5. Revocation is now eager: `handle_revoke_tree` marks every live descendant when the revocation takes effect, so the check on use is the handle's own flag and has no depth to run out of. Eager marking needs the tree to be connected at the moment of revocation, because it finds descendants through their parents, so `handle_release` now keeps a handle with live descendants in the table after its last holder lets go. Without that second half, a copy whose parent had been freed would be cut off from the tree, and revoking an ancestor further up would no longer reach it.

A handle slot lost per display restart. This one was found while drawing Figure 9.1. Each time the display driver restarted, the compositor's `attach` narrowed a fresh view and sent it. The display driver released a previous view when it received a new one, but a restarted driver had no previous view, because its old instance's handles were dropped when it crashed. The compositor, as the minter of every view, still held every one of them. A world with the `sys.read` claim, added for this measurement, logged the number of live handles every three seconds while the display driver was killed six times. Before the fix:

```
[  3001.014] probe: handles: 6 …
[  6001.152] probe: handles: 7 …
[  9001.024] probe: handles: 8 …
[ 12001.161] probe: handles: 9 …
[ 15001.032] probe: handles: 10 …
[ 18001.170] probe: handles: 11 …
```

Each line also listed the newest handles, and each new one was a read-only view held only by the compositor. The supervisor stops a world only at its fifth crash within ten seconds, so a display driver that crashed every three seconds would have been restarted indefinitely and would have filled the 128-slot table in about six minutes. The fix is the `release` at the top of `attach`, quoted in Section 9.5. After it:

```
[  3001.010] probe: handles: 5 …
[  6001.143] probe: handles: 5 …
[ 18001.143] probe: handles: 5 …
```

The count stays at five through all six restarts. The compositor releases rather than revokes because nothing else holds the old view: the crashed instance's hold went with it. The general lesson is that a capability's holder set includes the world that made it, and releasing is as much part of the protocol as granting.

A framebuffer leaked by a failed allocation. This one was found by reading `op_dma_alloc`, which allocated the buffer before the handle and, if the handle table was full, returned "out of handles" without freeing the buffer. The fix frees it:

▸ nucleus/vm.c

```
  int b = dmabuf_alloc(w, h, W);
  if (b < 0) return fail(ctx, "out of DMA buffers");
  handle_t *hd = handle_new(H_DMA, W, "dma");
  if (!hd) { buf_unref(b); return fail(ctx, "out of handles"); }
  hd->buf = b;
```

To measure the difference, a test fills the handle table (a second world narrows a view until "out of handles"), tries twenty allocations, then kills the second world, which frees its handles, and allocates once more. A copy of the system built with the old line gives:

```
[    11.720] owner: 20 allocations with the table full: {"out of handles":15,"out of DMA
buffers":5}
[    16.776] owner: after the hog crashed: out of DMA buffers
```

The fifteen failed allocations each leaked a buffer, the sixteen-slot buffer table filled, and the leak was permanent: even with the handle table free again, no framebuffer could ever be allocated. The current code gives:

```
[    11.720] owner: 20 allocations with the table full: {"out of handles":20}
[    16.780] owner: after the hog crashed: allocated
```

Still open. One problem from the same investigation remains, deliberately, because its fix is a policy decision rather than a correction. The handle table is shared by all worlds and has no per-world quota, so a world that has been given a single handle can fill it by narrowing; Section 8.2 showed a claimless world stop after 126 narrowings and the next world's allocation fail. The release rule above adds one more way to occupy slots, since holderless handles kept for their descendants still count. A quota per world, charged to

the minter, is the obvious repair, and the exercises ask what it would have to count. The first pass made that repair: every handle a world makes, narrowed copies and holderless handles kept for their descendants included, counts against the minter's `handles` quota until it is freed, and the boot fails unless the quotas fit the table (*From L4 to Small Worlds*, a separate book, Chapter 7).

9.8 What handles do not cover

Handles govern hardware. Two other kinds of authority in PicoCalc OS are not capabilities at all, and they matter more than the defects above.

Sending is by name. Any world can `send` or `ask` any other world it can name, and world names are listed in the manifest and shown by the system monitor. The nucleus checks only that the name exists. **Publishing is by self-declaration.** A world may publish on a channel if its own file declares, with `.provides(...)`, that it provides that channel; the declaration is read from the world's code at start-up, not from the manifest. The compositor subscribes to `keys`, which the keyboard driver declares that it provides, and handles whatever arrives on it. It also handles a `frames` message type, normally published by the nucleus on an app's behalf, and takes the app's name from the payload.

A world with no claims at all can exploit both. This test world, added to a copy of the manifest for this chapter, declares that it provides `keys`, publishes a key nobody pressed, and then sends the compositor a frame for the launcher:

```
// A world with no claims that tries to act for others: forge a key, forge another app's
frame.
app("rogue")
  .provides("keys")
  .onStart(async () => {
    await sleep(1000);
    publish("keys", { key: "down", state: "down" });        // a key nobody pressed
    log("published a forged key");
    await sleep(500);
    send("compositor", "frames", { app: "launcher", title: "Home", frame: { tree: { t:
"text", s: "forged by rogue", kids: [] }, hints: "" } });
    log("sent a forged launcher frame");
  });
```

Figure 9.4 shows the result. The launcher's selection moves down, and then the home screen is replaced by text the launcher never drew.

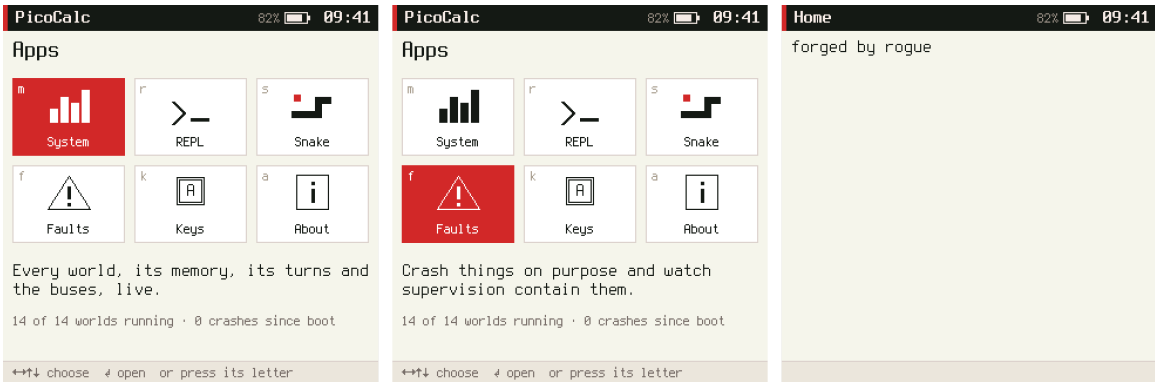


Figure 9.4. Captures from the host build with the rogue world running: before (left), after its forged key (centre), and after its forged frame (right). The world holds no handle and no claim.

Nothing in Chapter 8's table of rights was violated; the rogue world touched no device. It used two naming checks as if they were rights checks. A capability-pure design would close both gaps the way seL4, Zircon and E do, by making communication endpoints into capabilities. At boot the nucleus would create, from the manifest, an endpoint for each channel and each world's mailbox, and endow each world with send capabilities only for the endpoints its manifest entry lists. The keyboard driver would hold the only send capability for `keys`, so no other world could publish a key, however it declared itself. Frames would arrive at the compositor on an endpoint only the nucleus holds. And a world could hand another the right to talk to it by passing an endpoint capability in a message, under the same rules as handles. The cost is a longer manifest and one more thing to get right in every protocol. The ACL-style repair, checking `ctx.from` in each handler, is cheaper and closes the frame forgery, but it puts the check in every receiver, which is the opposite of complete mediation.

> NOTE
>
> The first pass, a later revision described in a separate book, *From L4 to Small Worlds* (Chapter 5), closed both gaps from the manifest, with lists of names rather than endpoint capabilities. A world's entry now lists the channels it provides and needs and the worlds it may send to and ask. A world whose file declares a channel its entry does not give is stopped at start, so the rogue world above never publishes; and a message whose type is a channel the receiver hears is refused, so its forged `frames` message is refused even from a world allowed to send to the compositor. This section describes the system before that pass.

TRY 9.2

Launder a handle under the old rule

Run the confinement scenario: `owner` holds a framebuffer and gives `relay` a read-only view. As `relay`, try to pass the view on, then narrow it and pass the copy. Both are refused, with the nucleus's own error messages. Then switch on the old pass rule and repeat: the second pass now succeeds, and the playground flags it as the leak. The printed version of this figure plays the scenario through under the current rule.

```
can_pass(W, h): walk up from h
  if h has g: yes
  if W did not narrow h (or h has no parent): no
  h = h's parent, and repeat
```

compositor `claims dma`

#1 `rwg` by compositor

#2 `r` from #1, by compositor

display `claims spi2`

#2 `r` from #1, by compositor

#3 `r` from #2, by display

snake `no claims`

holds no handles

✓ **1. compositor:** `dma.alloc(320, 320)` → handle 1: a new framebuffer, rights rwg
compositor claims dma in the manifest; framebuffer 0 has 1 handle

✓ **2. compositor:** `h1.narrow("r")` → handle 2: rights r, a child of handle 1
the child gets the parent's rights (rwg) intersected with "r"; compositor is recorded as the world that narrowed it

✓ **3. compositor:** `send("display", "fb", {fb: h2})` → display holds handle 2 once the message is delivered (at the end of compositor's segment)
handle 2 has no g; compositor narrowed it from handle 1, so look there
handle 1 has g: yes

✓ **4. display:** `spi.pixels(…, h2, …)` → display may send handle 2's pixels to the panel

× **5. display:** `h2.fill(0, 0, 8, 8, "#fff")` → handle 2 is read-only

× **6. display:** `send("snake", "fb", {fb: h2})` → handle 2 cannot be passed on by display
handle 2 has no g and display did not narrow it: no

✓ **7. display:** `h2.narrow("r")` → handle 3: rights r, a child of handle 2
the child gets the parent's rights (r) intersected with "r"; display is recorded as the world that narrowed it

× **8. display:** `send("snake", "fb", {fb: h3})` → handle 3 cannot be passed on by display
handle 3 has no g; display narrowed it from handle 2, so look there
handle 2 has no g and display did not narrow it: no

KEY POINTS

- The access matrix holds every domain's rights on every object; stored by column it is a set of access control lists, stored by row a set of capability lists, and the two differ in how authority moves rather than in what they record.
- Ambient authority, granted for who a program is rather than what it presents, produces confused deputies; capabilities avoid them by making designation and authority one value.
- Capabilities are made unforgeable by hardware tags, by keeping them in the kernel and giving programs indices, or by a memory-safe language; PicoCalc OS uses the kernel for hardware handles and the language inside each world.
- PicoCalc OS's handles provide attenuation (`narrow`), delegation (handles in messages, checked by `check_grants` and granted on delivery, with the sender keeping its copy) and revocation (a caretaker built into the kernel: revoke what you narrowed).
- The current pass rule allows a world to pass a handle that carries `g` , or a narrowing it made from a handle it could pass; a short induction proves that a non-grantable view cannot then be passed on in any form, and shows exactly where the old rule failed.
- Revocation is eager: revoking a handle marks it and every live descendant, and a handle with live descendants stays in the table after its holders let go. An earlier lazy version failed open at depth 32 and when an ancestor had been freed; it was found and fixed while this book was written, together with a handle leaked per display restart and a buffer leaked per failed allocation. A world can still fill the shared handle table.

- Sending to a world and publishing on a channel were guarded by names and self-declarations, not capabilities, and a world with no claims could forge key presses and another app's screen; endpoint capabilities endowed from the manifest would close both, and the first pass closed both with lists in the manifest.

Handles make authority over hardware explicit, checkable and, with care, provably confined. But they sit on top of a layer this chapter has taken for granted: that a world cannot reach anything except through the references it holds. The next chapter examines that layer, the language itself, as an isolation boundary: what it takes for a JavaScript runtime to hold a world in, how the history of language-based systems arrived there, where ambient authority hides in a language's built-ins, and why on modern processors no language boundary can keep a secret.

EXERCISES

1. BASIC Write out the full access matrix of PicoCalc OS for all thirteen worlds, using `js/system.json` and the handle table in Figure 9.2. Mark which entries come from claims and which from delegation.

2. BASIC For each operation in `js/drivers/display.js`, name the handle it uses and the right `handle_use` requires. Could the display driver work with a write-only SPI handle?

3. ANALYSIS Complete the proof of Lemma 2 with the depth limit included: state exactly what `can_pass` returns for a chain longer than 32 and show that the theorem still holds.

4. ANALYSIS Using the old rule `can_pass_old`, construct the shortest sequence of operations by which a world holding a non-grantable view gets a handle to the same buffer into a third world. Which line of the theorem's proof fails for that rule?

5. ANALYSIS `handle_revoke_tree` repeats its pass over the table until a pass marks nothing new. Bound the number of passes in terms of the depth of the derivation tree and the order of the handles in the table, and the total cost of one revocation with 128 slots. Compare it with the cost per use of the old lazy walk. In what workload would lazy revocation be the better design, and what would it need to be correct?

6. CODE Turn the two revocation tests of Section 9.7 (a copy narrowed 40 deep, and a copy whose revoked ancestor has been released) into benchmark worlds in the style of `bench/worlds/owner.js` and `relay.js`, add them to `bench/system-bench.json`, and check that both copies are refused. Then build a scratch copy of the repository with the old `revoked` and show that your benchmarks fail on it.

7. DESIGN Design endpoint capabilities for PicoCalc OS: the manifest syntax, the kernel data structure, the checks in `op_send`, `op_publish` and `kernel_ask`, and how a world passes another the right to reply to it. How would the compositor's `frames` input be protected?

8. ANALYSIS Is the display driver's `flush` handler a confused deputy in Hardy's sense? Identify its two sources of authority, describe what a rogue world can make it do, and compare the capability-style and ACL-style repairs for a system with two compositors.

9. DISCUSSION In PicoCalc OS, passing a handle shares it; in some systems passing moves it. Give one protocol in this system that would be safer with move semantics and one that would be harder to write. How would you implement a move with the existing operations?

10. BASIC For each of the three myths in [Miller et al. 2003], name the feature of PicoCalc OS's handles that refutes or illustrates it.

11. ANALYSIS `handle_release` keeps a holderless handle while copies narrowed from it are alive. Describe a sequence of operations by which a world given one handle keeps many holderless handles in the table, and say whether this is a new way to exhaust the table or the same one as in Section 8.2. Design a per-world quota and say exactly which handles it charges to which world.

12. PROJECT Implement a membrane for handles: an operation that narrows several handles at once under one revocation, so that a world can lend another a set of devices and take all of them back with one call. Prove the analogue of the theorem for it.

13. DISCUSSION KeyKOS made CPU time a capability (a meter). Sketch how PicoCalc OS's per-world budgets (Chapter 11) could be expressed as handles that a world could narrow and pass. What would it mean to pass half of your budget to the sandbox?

FURTHER READING

[Dennis and Van Horn 1966] The origin of capabilities, spheres of protection and C-lists, in eleven pages.

[Hardy 1988] Three pages that explain why designation and authority belong together.

[Miller et al. 2003] The clearest statement of what distinguishes capability systems from ACL systems, organised around three common misconceptions.

[Miller 2006] Part II develops the object-capability model and its patterns; the sections on caretakers and membranes apply directly to handle design.

[Watson et al. 2010] How to retrofit capabilities onto UNIX without rewriting it, with measured costs.

[Levy 1984] A book-length survey of capability hardware and software up to the early 1980s, freely available.

Language as the isolation boundary

A program in a memory-safe language cannot forge a reference. It can follow the references it was given, create new objects and pass references along, but it has no operation that turns a number into a pointer, reads past the end of an array, or calls a function it was never handed. If a language runtime guarantees that, the runtime can do a job usually given to the memory management unit: it can keep one component from touching another's state. This chapter is about using the language as the isolation boundary. It explains what a runtime must guarantee for that to work, traces the idea through sixty years of systems, compares the ways JavaScript code can be isolated (and which one PicoCalc OS chose and why), audits a PicoCalc OS world for authority that leaks in through the language's own built-in objects, including the clock leak the paper's fact-check found, and ends with the limit that no language-based design escapes: on processors that speculate, a language boundary keeps references in but cannot keep secrets in.

Chapter 8 located PicoCalc OS's boundaries and found that the first of them, between a world's code and the rest of the system, is enforced by QuickJS. Chapter 9 treated the handles that cross that boundary. This chapter examines the boundary itself.

10.1 Safety as protection

Hardware protection works by checking addresses. A user-mode load or store goes through the MMU or the PMP, and an address outside the regions the process may use causes a trap. Language-based protection works one level up, by making the bad addresses unexpressible. Three properties of a language and its runtime together make that possible.

Memory safety means that every access a program makes is to memory the program legitimately owns: no reading or writing outside an object's bounds, no use of an object after it has been freed, no reading of uninitialised memory. JavaScript gets bounds safety from checked array and typed-array accesses, and temporal safety from garbage collection: an object is freed only when nothing can reach it any more.

Type safety means that a value is only ever used as the type it is. A program cannot treat a number as an object reference or a string as a function. In JavaScript, every value carries its type at run time, and every operation checks it, so `(42).foo()` throws rather than jumping somewhere.

No ambient escape means the program's only way to affect anything outside its own heap is through objects it was given. A JavaScript engine embedded in a host exposes host functions as objects in the global scope; code that cannot reach such an object cannot use it. This is the property that turns the first two into protection. Memory and type safety stop a program from corrupting its own runtime; the absence of unexpected host objects stops it from reaching anything else.

TRY 10.1

A world's view of the world

The sandbox below runs code in a Web Worker behind a prelude modelled on PicoCalc OS's. The preset prints a computed value, the current date and a random number. Run it twice: the date is a virtual one, and the random number is the same on every run, because both come from the prelude rather than from the browser. The audit lists the globals the code can still reach. Two differences from a PicoCalc OS world remain: the worker's time budget is real milliseconds, because a browser has no virtual clock to charge, and a worker has no memory limit, because a browser gives no per-worker heap limit to set.

```
print("hello from a world");
print("the time here:", new Date().toISOString());
print("Math.random() is seeded:", Math.random().toFixed(6));
6 * 7
```

run 1 virtual clock at +1 s world epoch 1 ok

```
hello from a world
the time here: 2026-10-22T09:41:01.000Z
Math.random() is seeded: 0.316594
→ 42
```

REMOVED BEFORE YOUR CODE RUNS

- network: `fetch`, `XMLHttpRequest`, `WebSocket`, `EventSource`, `WebTransport`, `importScripts`, `fonts`, `FontFace`
- storage: `indexedDB`, `caches`, `webkitRequestFileSystem`, `webkitRequestFileSystemSync`, `webkitResolveLocalFileSystemURL`, `webkitResolveLocalFileSystemSyncURL`
- time: `performance`, `setTimeout`, `setInterval`, `clearTimeout`, `clearInterval`,

STILL REACHABLE

- network: `import() (syntax)`

Other non-standard globals: none. Everything else is plain ECMAScript: objects, functions, eval, WebAssembly, the seeded Math.random and the virtual Date.

`requestAnimationFrame`, `cancelAnimationFrame`, `scheduler`

- concurrency: `Worker`, `BroadcastChannel`, `MessageChannel`, `Atomics`
- environment: `navigator`, `location`, `crypto`, `close`, `postMessage`, `addEventListener`, `removeEventListener`, `dispatchEvent`, `when`, `onmessage`, `onerror`, `reportError`, `OffscreenCanvas`, `createImageBitmap`, `Notification`

Together these make an object reference a capability in the sense of Chapter 9, and they make the set of objects reachable from a world's global scope the exact statement of what that world can do. Chapter 8 listed that set for a PicoCalc OS world: the ECMAScript built-ins and the prelude's API. Every route from that set to the outside world passes through two functions, `__k` and `__ka`, which the nucleus implements in C and which check every request.

The guarantee is only as good as its implementation. Memory safety is a property of the engine, not of the language specification; the engine's parser, interpreter, garbage collector and built-in functions are C code with all of C's ways to fail, and a bug in any of them can give script code the ability to read and write arbitrary memory. Every major JavaScript engine has had such bugs. That is why Chapter 8 counted QuickJS's 80,864 lines into PicoCalc OS's trusted computing base, and why the engine is the part of the system on which its isolation most depends.

10.2 A short history of language-based systems

The idea is as old as protected operating systems. Table 10.1 lists systems that used a language, rather than or alongside hardware, to separate components, and the paragraphs after it draw out what changed over sixty years.

Table 10.1. Systems that used a language to separate components. The unit of isolation grows over time from nothing, to a module, to a component with its own heap that can be destroyed on its own.

System	Year	Language	Unit of isolation	What enforces it	What units share
Burroughs B5000	1961	ALGOL, COBOL	program	descriptors and trusted compilers	the machine
Smalltalk-80	1980	Smalltalk	none	(nothing)	every object in one image
Pilot	1980	Mesa	none against malice	Mesa's type checking catches errors	one address space
Oberon	1992	Oberon	module	type safety	one address space, one process
SPIN	1995	Modula-3	kernel extension	type safety; a trusted compiler	the kernel's address space
Inferno	1997	Limbo on the Dis virtual machine	process	the virtual machine	per-process name spaces
JX	2002	Java	domain, with its own heap and collector	type safety	a small kernel
House	2005	Haskell	program	type safety, typed hardware access	the machine
Singularity	2007	Sing#	software-isolated process	verified code, no shared memory	an exchange heap for messages
Tock	2017	Rust; any for processes	capsule; process	Rust's types; the MPU	the kernel
Theseus	2020	Rust	cell	Rust's types and ownership	one address space, one privilege level
RedLeaf	2020	Rust	domain, with a private heap	Rust's types and ownership	a shared heap of exchangeable objects

System	Year	Language	Unit of isolation	What enforces it	What units share
PicoCalc OS	2026	JavaScript	world, with its own runtime	QuickJS and the nucleus's checks	nothing but messages and handles

The **Burroughs B5000** is the first and the most radical. Every memory reference went through descriptors in a program reference table; tag bits distinguished descriptors from data. But the table "was not read-only"; the system "relied heavily on the compilers for proper system operation", and "assembly language was not available to the user" [Levy 1984]. Protection was a property of the code the compilers would generate, which is the founding bargain of every system in the table: trust the language implementation, and you need less from the hardware.

Smalltalk-80 and **Oberon** show the other pole. Both were complete single-user systems written in one safe language [Goldberg and Robson 1983; Wirth and Gutknecht 1992]. Type safety kept programs from corrupting memory by accident, but neither tried to separate one program from another: in Smalltalk, every object in the image is reachable from the system dictionary and can be modified by any code. Safety and isolation are different properties. A safe language makes isolation *possible*, by making references the only route to an object; it does not make isolation *happen*, which requires that components not be given references to each other's objects.

Pilot, Xerox's operating system for its personal workstations, sits between them. It ran everything in one address space and relied on the Mesa language's type checking to protect the system from errors in programs, not from hostile programs [Redell et al. 1980]. That is a deliberate and honest scope, and it is close to PicoCalc OS's (Chapter 8 ends by stating PicoCalc OS's goal as fault isolation among cooperating components).

SPIN took the idea into a conventional kernel: applications could load extensions, written in Modula-3, into the kernel's address space to specialise its behaviour, and type safety, enforced by a trusted compiler, kept extensions from corrupting the kernel [Bershad et al. 1995]. **Inferno** ran programs written in Limbo on a virtual machine, Dis, portable across processors and hosts [Dorward et al. 1997].

JX is the closest ancestor of PicoCalc OS's worlds [Golm et al. 2002]. JX was an operating system written in Java, structured as **domains**, each with its own heap and its own garbage collector, communicating through the kernel. Separate heaps mean that one domain's garbage collection does not stop another, one domain cannot exhaust another's memory, and a domain can be terminated by discarding its heap. Those are the same reasons PicoCalc OS gives each world its own runtime.

Singularity made the design a research programme at Microsoft [Hunt et al. 2007]. Its **software-isolated processes** were *sealed*: no shared memory, no code loaded after start, communication only by messages over channels whose protocols were declared in contracts and checked by the compiler. Large messages travelled through an **exchange heap** whose objects could be owned by only one process at a time, so passing a buffer moved it rather than sharing it. Singularity's processes ran without hardware protection between them by default, and its authors measured the cost that hardware isolation would have added.

The Rust systems carry the idea into a language without a garbage collector. **Tock**, for microcontrollers, isolates its kernel's own extensions ("capsules") with Rust's type system and untrusted applications with the MPU, using each mechanism where it fits [Levy et al. 2017]. **Theseus** runs everything in one address space and one privilege level and uses the compiler's knowledge of ownership to manage state and recovery [Boos et al. 2020]. **RedLeaf** isolates **domains** with private heaps, lets them exchange only objects of special types through a shared heap, and can restart a domain that faults, device drivers included [Narayanan et al. 2020]; Chapter 15 returns to it.

Two trends run through the table. First, the unit of isolation grows from nothing, to a module, to a component with its own heap that can be destroyed on its own. A component that owns its heap can be given a memory limit, collected independently and thrown away when it fails, which is what fault containment needs. Second, the systems that share least between units, Singularity, RedLeaf and PicoCalc OS, pay for it by copying or moving data at the boundary rather than sharing it.

10.3 Software fault isolation and WebAssembly

Language-based isolation requires code written in the safe language. Software that arrives as machine code needs a different technique, and the answer, **software fault isolation** (SFI), makes the machine code itself safe. Wahbe, Lucco, Anderson and Graham showed in 1993 that untrusted object code could be confined to its own fault domain within a shared address space by rewriting it so that every store and indirect jump has its target address forced into the domain's segment before use; the rewriting can be checked, so the code does not need to be trusted [Wahbe et al. 1993]. Google's Native Client applied the idea to x86 code downloaded by a browser: a validator checks that the code obeys alignment and instruction rules that make the checks unavoidable, and x86 segmentation confines its data accesses [Yee et al. 2009].

WebAssembly is the descendant that won [Haas et al. 2017]. It is a bytecode for a stack machine with structured control flow, validated before it runs, whose programs see memory only as a **linear memory**, an array of bytes whose every access is bounds-checked, and the outside world only through the functions the

host passes in as imports. A WebAssembly module therefore has exactly the authority its imports give it, which is the object-capability property again, achieved for compiled C rather than for a safe source language.

PicoCalc OS has been compiled to WebAssembly (Chapter 17), and running it in a browser stacks three of these boundaries, each enforced by a different mechanism. Figure 10.1 draws them.

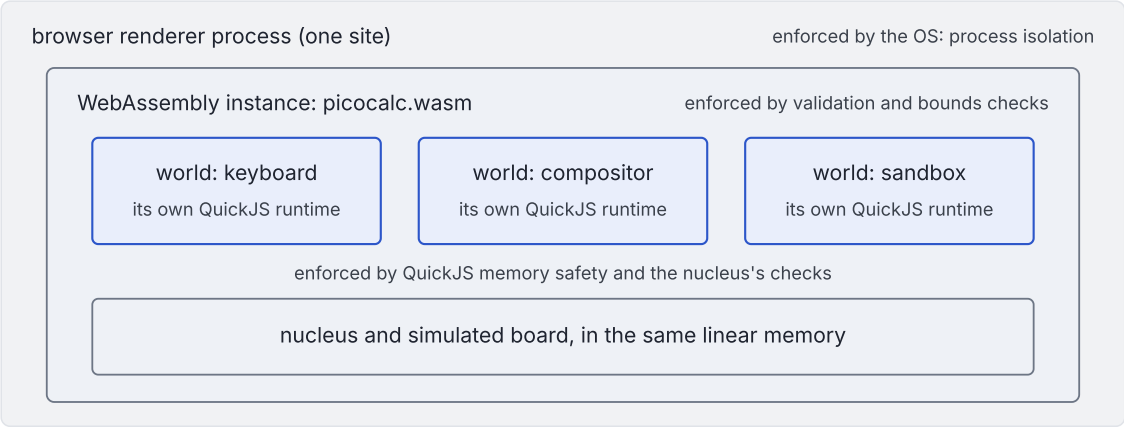


Figure 10.1. PicoCalc OS's WebAssembly build running in a browser tab: three nested boundaries, each enforced by a different mechanism. A memory-safety bug in QuickJS escapes the innermost boundary but not the WebAssembly sandbox around it.

The picture makes a point about defence in depth. On the host build, a QuickJS bug that let a world write outside its heap would let it write anywhere in the PicoCalc OS process, including the nucleus. In the browser, the same bug is still a total break of PicoCalc OS's isolation, because every world and the nucleus share one linear memory; but it cannot reach anything outside that linear memory, because the WebAssembly sandbox bounds every access the compiled engine makes.

10.4 Isolation inside JavaScript

There are three ways to isolate JavaScript code from other JavaScript code, and they differ in what the isolated pieces share. Figure 10.2 draws them.

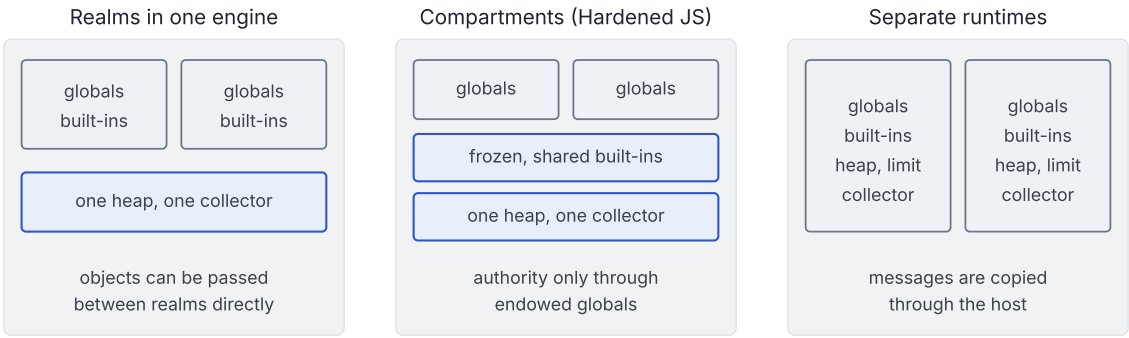


Figure 10.2. Three ways to isolate JavaScript components. Realms share a heap and a collector but have separate built-ins; compartments share frozen built-ins and a heap; separate runtimes share nothing but the host.

Realms. A realm is a global object together with its own set of built-ins: its own `Object`, `Array.prototype`, `Date` and so on. A browser page and each of its iframes are separate realms; Node's `vm` module creates realms on demand. Separate built-ins prevent one realm from poisoning another's prototypes, but realms in one engine share a heap and a collector, and objects pass freely between them. Node's documentation is blunt about what that means: "The `node:vm` module is not a security mechanism. Do not use it to run untrusted code" [Node.js n.d.]. An object from another realm carries references back into that realm's built-ins, and from them to its global object.

Compartments. Hardened JavaScript, developed by Agoric and others as the SES library, makes isolation within one realm practical [Endo n.d.]. `lockdown()` "tamper-proofs all of the JavaScript intrinsics", freezing the shared built-ins so that no code can modify them; `harden(object)` freezes an object and everything reachable from it; and `new Compartment()` creates a separate evaluation environment with its own `globalThis`, which shares the frozen built-ins and starts with no host APIs at all. Authority enters a compartment only through the globals its creator endows it with. Moddable's XS engine, written for microcontrollers, implements compartments natively: each "compartments have their own `globalThis` object, their own global lexical scope and their own module map", and built-ins are shared except `Compartment`, `Function` and `eval` [Moddable n.d.]. Compartments are cheap, because the built-ins are shared, and calls between them are ordinary function calls.

Separate runtimes. The third way shares nothing. Each component gets its own engine instance, with its own heap, built-ins and collector, and components communicate only through the host. Browsers do this between Web Workers and, since Site Isolation, between sites. PicoCalc OS does it between worlds: each

world is a separate `JSRuntime`, created by the nucleus with a custom allocator that charges every allocation to the world and refuses any that would cross the world's limit:

▸ nucleus/vm.c

```
static void *w_malloc(JSMallocState *s, size_t size) {
  world_t *W = s->opaque;
  if (W->mem_used + size > host_limit(W)) { W->oom = true; return NULL; }
  void *p = malloc(size);
  if (!p) return NULL;
  size_t u = malloc_usable_size(p) + 8;
  W->mem_used += u; s->malloc_size += u; s->malloc_count++;
  return p;
}
```

Why pay for separate runtimes when compartments are cheaper? Because compartments isolate *authority* but not *resources* or *failure*, and PicoCalc OS needs all three. In one shared heap there is no per-compartment memory limit: a compartment that allocates without bound exhausts memory for all. There is one collector, so a compartment that builds a large object graph slows every collection. There is one interrupt counter per runtime in QuickJS, so per-component time budgets (Chapter 11) need per-component runtimes. And when a component fails, a separate runtime can simply be freed, all of it, along with every object it allocated; a compartment's objects may be referenced from other compartments, and there is no way to reclaim them as a unit. Crash, free, restart is the whole of PicoCalc OS's recovery story (Chapter 14), and it depends on the runtime being the unit.

The price is memory and copying. On the 32-bit build an empty QuickJS runtime costs 143 KB of heap and the prelude adds 38 KB, so each world starts at about 182 KB before its own code; the median world uses 190 KB and all thirteen together 2.5 MB. Every message is serialised to JSON and parsed again on the other side, and an `ask` with an empty payload takes 119 µs on the virtual clock for the round trip (Chapter 12). On a device with 32 MB of PSRAM that is affordable; on one with 64 KB it would not be, which is why Tock, at that size, isolates with the MPU and the type system instead.

10.5 Ambient authority in a world

A world's authority is what its global object reaches. Most of it comes from the prelude, which the world was meant to have. But the ECMAScript built-ins also come with authority of their own, and some of it is the kind Chapter 9 called ambient: available to any code without being designated. For a deterministic sys-

tem the most important ambient authorities are **sources of information about the outside world**: clocks, randomness and anything whose value depends on the host. Table 10.2 audits the globals of the sandbox world, listed in Section 8.3, for those.

Table 10.2. An audit of a world's globals for ambient authority. Each row was checked on the host build by typing an expression at the REPL, which evaluates it in the sandbox world.

Global	From	What a world gets	Status
`Date`, `Date.now`	the engine, replaced by the prelude	virtual time since a fixed boot instant	the engine's `Date` is unreachable
`Math.random`	the engine, replaced by the prelude	a xorshift generator seeded from the world's name	deterministic
`now()`	the prelude, from the nucleus	virtual milliseconds since boot	deterministic
`WeakRef`, `FinalizationRegistry`	the engine	observation of garbage collection	deterministic for one build
`Atomics.wait`	the engine	would block the thread	throws "cannot block in this thread"
`SharedArrayBuffer`	the engine	shared memory	no second thread exists to share it with
`eval`, `Function`	the engine	new code	runs with the world's own authority
`sys.worlds`, `sys.stats`	the prelude, from the nucleus	other worlds' status and heap sizes	requires the `sys.read` claim; heap sizes vary by build

Two rows matter more than the rest. **Randomness** is replaced at the top of the prelude by a generator seeded from a value the nucleus derives from the world's name, so a world's random numbers are the same on every run. **Time** is the subject of the rest of this section, because it is where the paper's fact-check found a leak.

The determinism property of PicoCalc OS says that, given the manifest, the sources, the device profile and a script of inputs, every segment, transfer and frame is fixed (Chapter 18). For that to hold, no world may read the host's clock, because the host's clock differs from run to run. The prelude therefore replaces `Date` with a version that reads the virtual clock. The first version replaced the global binding and left the engine's prototype object as it was. That prototype object is shared by every date the engine creates, and its `constructor` property still pointed at the engine's own `Date`. So `new Date().constructor` was the real constructor, and `new (new Date().constructor)()` read the host's wall clock: one expression undid the replacement.

The current prelude closes every route to the engine's constructor:

▸ js/prelude.js

```
  const BOOT = Date.UTC(2026, 9, 22, 9, 41, 0);   // the wall clock reads this at boot
  // Date reads the virtual clock. The engine's own constructor is made unreachable:
VDate is a plain
  // function (so Object.getPrototypeOf(Date) is Function.prototype), and dates'
constructor property
  // points back at VDate.
  const RealDate = Date;
  const vnowMs = () => BOOT + k("now");
  function VDate(...a) {
    if (!new.target) return new RealDate(vnowMs()).toString();
    return a.length ? new RealDate(...a) : new RealDate(vnowMs());
  }
  VDate.prototype = RealDate.prototype;
  VDate.now = vnowMs;
  VDate.UTC = RealDate.UTC;
  VDate.parse = RealDate.parse;
  Object.defineProperty(RealDate.prototype, "constructor", { value: VDate, writable:
true, configurable: true });
  g.Date = VDate;
```

Read it as a list of the references that lead to the engine's `Date`, each of which is cut. The global name `Date` now names `VDate`. Every date's prototype chain reaches `RealDate.prototype`, whose `constructor` now names `VDate`. `VDate` is a plain function rather than a class that extends the real one, because a subclass's own prototype chain would lead to its superclass: `Object.getPrototypeOf(Date)` would have been the engine's `Date`. Since `VDate` is a plain function, that expression gives `Function.prototype`. The only remaining reference to the engine's constructor is the prelude's local

variable `RealDate`, inside a closure that no world code can enter. Dates themselves still work, because `VDate` constructs them with `RealDate`, and only the no-argument case, the one that reads the clock, is redirected. Figure 10.3 draws the object graph after the fix.

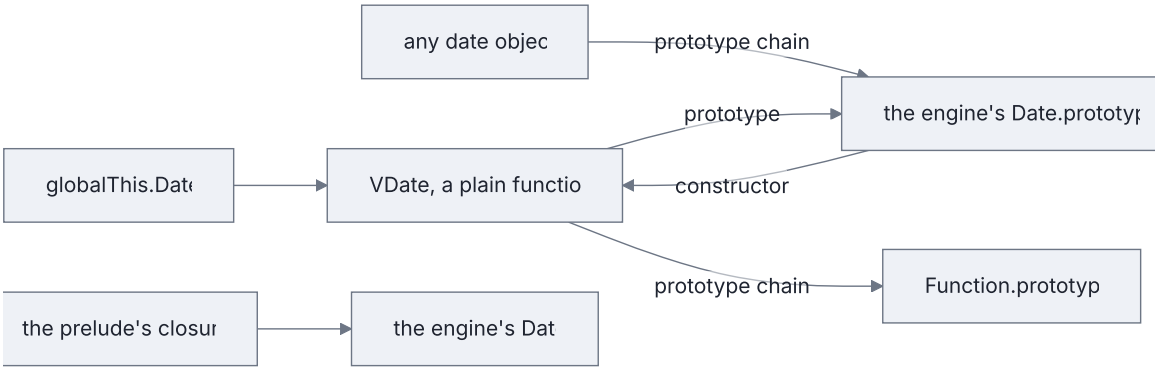


Figure 10.3. The objects a world can reach from `Date` after the fix. The engine's constructor is reachable only from inside the prelude's closure. Before the fix, the constructor edge from the shared prototype pointed at the engine's Date.

The host build confirms it. Typed at the REPL in two runs, `log(new Date().constructor === Date, Date.now(), typeof __k, typeof performance)` and `log(Object.getPrototypeOf(Date) === Function.prototype, Date.name, new Date(0).toISOString())` produce:

```
[ 13495.082] sandbox: true 1792662073495.058 undefined undefined
[ 36116.059] sandbox: true VDate 1970-01-01T00:00:00.000Z
```

`new Date().constructor` is the replacement, `Date.now()` is the fixed boot instant plus 13.5 virtual seconds, neither the raw host call nor a performance timer exists, and dates with explicit arguments work as before. The replacement's name, `VDate`, does show; it reveals that `Date` was replaced, which is harmless.

Hardened JavaScript makes the same decision for the same reason. A new compartment's global scope gets the shared built-ins with `Math.random` and `Date.now`, and the `Date` behaviours that reveal the current time, disabled, because they could serve as covert channels, and a compartment's creator can grant them explicitly [Endo n.d.]. Time is authority.

There is a subtler channel that remains. A world's only clock is the virtual one, but the virtual clock is shared: it advances with every world's segments. A world that reads `now()` at the start of two consecutive turns learns how much lane time passed in between, which depends on what other worlds did. Two worlds can therefore communicate without messages, one by computing more or less and the other by watching the clock. Lampson's confinement problem, which asks how to stop a program from leaking information to anyone but its caller, identified exactly this kind of channel in 1973, one that uses a resource's behaviour rather than its contents [Lampson 1973]. The channel here is deterministic and coarse, and PicoCalc OS's goal does not include closing it; closing it would mean giving each world a clock that advances only with its own work, as Aviram and colleagues proposed for cloud tenants, whose timing channels they removed by making all of a tenant's observable time deterministic [Aviram et al. 2010].

TRY 10.2

Reach for the clock

This sandbox runs code in a Web Worker behind a prelude modelled on PicoCalc OS's. The preset tries the routes to real time described above: the global `Date`, the constructor of a date object, the constructor's prototype chain, and the browser's own `performance` timer, and then times a loop. Every route gives the virtual date, `performance` is gone, and the timing loop measures 0 ms, because virtual time does not advance while the code runs. The audit under the output lists what the worker can still reach. One route the browser adds, `performance`, is one QuickJS never had, so the browser prelude has more to close than PicoCalc OS's does.

```
const tries = {
  "Date.now()": () => new Date(Date.now()).toISOString(),
  "new Date().constructor === Date": () => new Date().constructor
=== Date,
  "the prototype's constructor": () => new
(Object.getPrototypeOf(new Date()).constructor)().toISOString(),
  "Function(\"return Date\")() === Date": () => Function("return
Date")() === Date,
  "performance.now()": () => performance.now(),
  "setTimeout": () => typeof setTimeout,
  "a timing loop": () => { const a = Date.now(); for (let i = 0; i
< 3e6; i++); return (Date.now() - a) + " ms"; },
};
for (const [k, f] of Object.entries(tries)) {
  try { print(k, "→", f()); } catch (e) { print(k, "→", e.name +
": " + e.message); }
```

run 1 virtual clock at +1 s world epoch 1 ok

```
Date.now() → 2026-10-22T09:41:01.000Z
new Date().constructor === Date → true
the prototype's constructor → 2026-10-22T09:41:01.000Z
Function("return Date")() === Date → true
performance.now() → ReferenceError: performance is not defined
setTimeout → undefined
a timing loop → 0 ms
```

```
→ undefined
```

REMOVED BEFORE YOUR CODE RUNS

- network: `fetch`, `XMLHttpRequest`, `WebSocket`, `EventSource`, `WebTransport`, `importScripts`, `fonts`, `FontFace`
- storage: `indexedDB`, `caches`, `webkitRequestFileSystem`, `webkitRequestFileSystemSync`, `webkitResolveLocalFileSystemURL`, `webkitResolveLocalFileSystemSyncURL`
- time: `performance`, `setTimeout`, `setInterval`, `clearTimeout`, `clearInterval`, `requestAnimationFrame`, `cancelAnimationFrame`, `scheduler`
- concurrency: `Worker`, `BroadcastChannel`, `MessageChannel`, `Atomics`
- environment: `navigator`, `location`, `crypto`, `close`, `postMessage`, `addEventListener`, `removeEventListener`, `dispatchEvent`, `when`, `onmessage`, `onerror`, `reportError`, `OffscreenCanvas`, `createImageBitmap`, `Notification`

STILL REACHABLE

- network: `import() (syntax)`

Other non-standard globals: none. Everything else is plain ECMAScript: objects, functions, eval, WebAssembly, the seeded Math.random and the virtual Date.

10.6 Side channels and Spectre

Everything so far is about references: which objects code can name. A language boundary is very good at that. It is no good at all at stopping code from *inferring* things it cannot name, and since 2018 we have known that, on modern processors, what code can infer includes the contents of any memory in its address space.

Spectre attacks exploit speculative execution [Kocher et al. 2019]. A processor that predicts a branch runs ahead along the predicted path before it knows whether the prediction was right; if it was wrong, the architectural results of the speculated instructions are discarded, but their effects on the cache are not. An attacker trains a branch predictor so that a bounds check in the victim's code is predicted to pass, supplies an out-of-bounds index, and the processor speculatively loads a secret byte from beyond the array and uses it to touch a cache line chosen by the byte's value. Timing which cache line is now fast reveals the byte. The bounds check was correct; the language's guarantee held for every instruction that architecturally executed; and the secret leaked anyway.

For language-based isolation this is fatal, and the people with the most at stake said so. After building mitigations for V8, Google's JavaScript engine, its engineers concluded that "speculative vulnerabilities on today's hardware defeat all language-enforced confidentiality", that there are "no known comprehensive software mitigations", and that "untrusted code can construct a universal read gadget to read all memory in the same address space through side-channels" [Mcilroy et al. 2019]. Chrome changed its security model in response: **Site Isolation** puts each site's pages in a separate renderer process and keeps cross-site data out of it, so that the only memory a site's scripts can read speculatively is memory that was theirs anyway [Reis et al. 2019]. The browser, the largest deployment of language-based isolation there is, now draws its confidentiality boundary with the hardware.

What does this mean for PicoCalc OS? First, its goal, as Chapter 8 stated it, is fault isolation and authority confinement among cooperating components, not confidentiality against a hostile one. Spectre does not let one world call another's functions, forge a handle or corrupt another's heap; it lets it read. For a handheld whose worlds hold no secrets from each other, that is a smaller problem than for a browser.

Second, the attack needs a timer fine enough to tell a cache hit from a miss. On the host build, a world has none. Its only clock is the virtual one, which advances by counted interpreter steps and charged native work and is not affected by cache behaviour at all; and the usual substitute, a second thread incrementing a counter in shared memory, is unavailable because a world has no second thread. That removes the measuring instrument, which is a real obstacle, though not a proof of safety: a channel only needs some observable that correlates with timing.

Third, the device is a different question, and this book does not answer it. Whether the ESP32-P4's cores speculate in ways that can be exploited has not been analysed here, and the port has not yet decided whether a world's `now()` will read real time or keep the virtual clock (Chapter 7). If worlds get a real clock, the question of side channels returns with it.

The general conclusion stands without the details. Language-based isolation protects integrity: what a component can call, write and destroy. On speculative hardware it does not protect confidentiality within an address space, and a design that needs both must put mutually distrusting secrets in separate address spaces, as browsers now do.

10.7 Choosing a boundary

Table 10.3 collects the mechanisms of this chapter and Chapter 8 and compares them by what they defend against and what they cost. No row dominates another; each is a point in a trade between the size of what you trust, the cost per component, and the cost of crossing the boundary.

Table 10.3. Isolation mechanisms compared. PicoCalc OS chose a runtime per component: it defends against faults and authority leaks at the cost of a large trusted engine, about 182 KB per component and serialised messages.

Mechanism	Defends against	What you trust	Cost per component	Cost of crossing	Examples
Processes with an MMU	faults, hostile code, cross-process speculation	the kernel and the hardware	an address space	a system call and a context switch	Site Isolation, seL4
MPU or PMP regions	faults and hostile code touching memory	the kernel and the region configuration	a few regions; reprogrammed per switch	a trap	Tock's processes
SFI and WebAssembly	faults and hostile code touching memory	the validator and the runtime	bounds checks on every access	a call through an import	Native Client, browsers
A language runtime per component	faults, authority leaks, resource exhaustion	the engine and the host's checks	a runtime (about 182 KB here)	a serialised message (119 µs here)	JX, PicoCalc OS
Compartments in one runtime	authority leaks	the engine and the lockdown	small: built-ins are shared	a function call	Hardened JavaScript, XS

PicoCalc OS's choice follows from its goals. It needs a per-component heap limit, a per-component time budget and the ability to destroy a component wholesale, so it needs separate runtimes. It runs on a microcontroller with region-based protection that cannot describe thirteen scattered heaps, so it cannot get those from the MMU. It does not need to keep secrets between components, so Spectre's verdict on language-based confidentiality does not decide the question. And it accepts the largest cost in the table, an 80,864-line engine in the trusted computing base, in exchange for writing every driver, service and app in a memory-safe language, which removes the most common way drivers fail in the first place (Chapter 15).

KEY POINTS

- A memory-safe, type-safe language with no ambient escape makes an object reference a capability, so the objects reachable from a component's global scope are exactly what it can do; but the guarantee is the engine's, and the engine joins the trusted computing base.
- Language-based systems from the B5000 to RedLeaf grew their unit of isolation from nothing to a component with its own heap that can be limited, collected and destroyed independently; JX's domains are the closest ancestor of PicoCalc OS's worlds.
- Safety is not isolation: Smalltalk and Oberon were safe and shared everything, and isolation happens only when components are not given references to each other's objects.
- Software fault isolation and WebAssembly make machine code safe by rewriting or validation and bounds checks; PicoCalc OS's WebAssembly build stacks that boundary around its own.
- Realms share a heap, compartments share frozen built-ins, separate runtimes share nothing; PicoCalc OS uses separate runtimes because it needs per-world memory limits, time budgets and wholesale destruction on failure, at about 182 KB per world and 119 µs per empty round trip.
- Ambient authority enters through a language's built-ins; the prelude makes randomness and time deterministic, and the clock leak showed that replacing a global binding is not enough when shared prototypes keep a `constructor` reference to the original.
- The shared virtual clock still lets worlds signal each other by how much lane time they use, a covert channel of the kind Lampson described in 1973.
- On speculative hardware, language boundaries cannot keep secrets within an address space; browsers moved their confidentiality boundary to processes, and a design that needs confidentiality must do the same.

This chapter and the two before it have been about who may do what. They have left a gap that Table 8.1 already marked: the rows guarded by quantity, not rights. A world that holds no handle can still loop forever, allocate until memory runs out, or fill a shared table. The next chapter is about resource control: how PicoCalc OS bounds the time and memory a world can take, how to prove that the bound holds, and how a bound can quietly fail to cover the work it was meant to.

EXERCISES

1. BASIC Explain the difference between memory safety, type safety and isolation, using one system from Table 10.1 that has the first two without the third.
2. BASIC List every reference path from a world's global scope to the engine's `Date` constructor that existed before the prelude's fix, and for each say which line of the current prelude closes it.
3. CODE Type expressions at the host build's REPL (open it with `key r` in a script) to look for other routes to real time: try `Atomics.wait` with a timeout, `new Error().stack`, and the `toString` of a function the prelude defines. Report what each reveals.
4. ANALYSIS The xorshift generator in the prelude is seeded from the world's name. What can one world learn about another world's random numbers? Is that a problem for any world in `js/system.json`?
5. ANALYSIS Design a covert channel between two worlds using only `now()` and loops: one world sends a bit per turn by computing for 1 ms or 10 ms, the other reads the clock. Estimate its bandwidth on the host build from the lane's scheduling rules (Chapter 4), and propose a change to the virtual clock that closes it. What does the change cost?
6. DISCUSSION Compartments would let PicoCalc OS's worlds share the built-ins and save most of the 143 KB empty-runtime cost per world. List what the nucleus would have to provide instead of separate runtimes to keep per-world memory limits, time budgets and wholesale destruction, and say which of them QuickJS could support.
7. ANALYSIS Singularity's exchange heap moves a buffer between processes; PicoCalc OS copies messages as JSON and shares framebuffers through handles. For a 320 × 320 framebuffer at 2 bytes per pixel, estimate the cost of sending it as JSON at the virtual clock's payload charge (read `charge_json` and the profile in `nucleus/nucleus.h`), and compare with passing a handle.
8. BASIC For each row of Table 10.3, name the PicoCalc OS defect from Chapter 9 or this chapter that the mechanism would or would not have prevented.

9. DISCUSSION Spectre lets code read but not write across a language boundary. For each of PicoCalc OS's thirteen worlds, say whether anything it holds would be worth reading by another world, and whether that changes if the device gains a network connection.

10. PROJECT Build a minimal compartment-based variant of the sandbox world: evaluate REPL input in a fresh global object inside the sandbox's runtime, with only `print` endowed, and measure the memory saved against creating a new runtime per evaluation. What does the variant lose?

11. ANALYSIS In the WebAssembly build (Figure 10.1), which of PicoCalc OS's protections still hold if a QuickJS bug gives one world arbitrary reads and writes within its linear memory? Which hold if the browser's renderer is compromised?

FURTHER READING

[Levy 1984] Chapter 2 on the Burroughs B5000 is the best short account of protection by trusted compilers.

[Golm et al. 2002] JX's domains, each with its own heap and collector, anticipate most of PicoCalc OS's runtime design.

[Hunt et al. 2007] Singularity's sealed processes and exchange heap, and a measurement of what hardware isolation would add.

[Haas et al. 2017] WebAssembly's design, including how validation and linear memory make compiled code safe to run.

[Endo n.d.] The SES documentation explains `lockdown`, `harden` and compartments, and why compartments start without the clock.

[Mcilroy et al. 2019] Why language-enforced confidentiality fails on speculative hardware, from the team that tried hardest to save it.

[Reis et al. 2019] How a browser moved its isolation boundary from the language to processes, and what that cost.

Resource control

Isolation keeps one world from reading or corrupting another's state. It does not keep one world from taking everything. A world that loops forever touches nothing that belongs to anyone else, and it still stops the keyboard driver from polling, the compositor from drawing and every other world from running, because on a single lane whoever holds the processor holds the system. This chapter is about the second half of protection: deciding how much of each shared resource a component may consume, measuring what it actually consumes, and doing something when it goes over. By the end you should be able to explain how PicoCalc OS bounds a world's processor time and memory, derive the bound on how far a segment can overrun its budget, recognise the accounting leak that the book's fact-check found in an earlier version of that bound, and list the resources the system still does not control, each with a measured or reproducible consequence.

11.1 Why isolation needs resource control

An operating system shares a small number of resources among many principals. On a desktop the resources are processor time, physical memory, disk and network bandwidth, and kernel objects such as file descriptors. On the PicoCalc they are the **lane** (the one sequence of segments described in Chapter 3 and Chapter 4), the heap memory of each world's runtime, the memory the nucleus allocates for messages and bookkeeping, the two buses (I2C to the keyboard controller, SPI to the panel), and a handful of fixed-size kernel tables. A **resource control** mechanism has three parts, and it helps to keep them apart:

- **Accounting** measures what each principal uses. Accounting answers "how much has this world consumed?" and is only as good as its coverage: work that is done but not attributed to anyone is invisible to everything that follows.

- **Limits** state how much each principal may use. A limit can be fixed (a heap of 320 KB), per interval (so much processor time per period) or per operation (so much time per segment).
- **Enforcement** decides what happens at the limit: refuse the request, delay the principal until the next period, or stop it.

Each choice has consequences for the rest of the design. Delaying a principal requires that it can be suspended and resumed, which a preemptive kernel with per-thread stacks can do and a run-to-completion event loop cannot. Refusing a request requires that the principal can cope with the refusal, which a memory allocator returning `NULL` can signal but an infinite loop cannot be asked about. Stopping a principal requires that stopping it is safe, which in turn depends on what state it shares and what it was in the middle of.

PicoCalc OS makes the same choice almost everywhere: it **stops** the world. A world that runs past its time budget or its memory limit crashes, its segment's effects are discarded, and the supervisor of Chapter 14 restarts it after a backoff. This is coherent with the rest of the design. Segments are atomic (Chapter 3), so stopping a segment halfway leaves nothing half-done outside the world; the world's own state is thrown away with its runtime; and the world's mailbox survives, so the messages it had not yet handled are handled by the new instance. Stopping is the one enforcement action that the segment model makes cheap and safe.

Table 11.1 lists the resources and how each is controlled today. The rest of the chapter works through the rows.

Table 11.1. Shared resources in PicoCalc OS, how each is accounted for and limited, and what happens at the limit.

Resource	Accounted per world?	Limit	At the limit
Lane time (interpreter steps)	yes, exactly	per-segment budget from the manifest (default 50 ms)	the world crashes
Lane time (native work in host calls)	yes, by a cost model	the same budget	the world crashes
Heap of the world's runtime	yes, every allocation	`mem_kb` from the manifest (default 1 MB)	the allocation fails, then the world crashes
Interpreter stack	no	192 KB per runtime	a catchable "stack overflow" error
Nucleus memory for messages in mailboxes	no	none	grows until the host runs out
Outstanding asks	no	1,024 in a ring shared by all worlds	the oldest is silently overwritten
Bus time (I2C, SPI)	per bus, not per segment	none, beyond who holds a handle	transfers queue in order
Handles, framebuffers, worlds	no	128, 16 and 32 system-wide	the call that needs one fails

The first pass, a later revision, gave the mailbox, ask and table rows per-world limits; see the note in Section 11.8.

11.2 Two ways to take the processor back

Any system that runs code it does not fully trust needs a way to get the processor back from that code. There are two families of mechanism, and they differ in what they assume about the code being stopped.

The first is **preemption by a timer interrupt**. A hardware timer fires at a fixed rate (the FreeRTOS tick that Chapter 7 describes); the interrupt handler saves the running thread's registers, and the scheduler may choose to run a different thread. The interrupted thread is not harmed. Its stack and registers are intact, and

it resumes later exactly where it stopped. Preemption is the right tool for time-sharing: it divides the processor among threads that all deserve to finish. Its cost is that a thread can lose the processor between any two instructions, so any data it shares with other threads must be protected by locks or written with atomic operations.

The second is **counted execution**. The code runs inside an interpreter or a compiled sandbox that counts the work it does and calls the host when a count runs out. The host can then stop the code. Stopping, in this family, usually means abandoning the computation rather than suspending it, because an interpreter that unwinds its stack to return control to the host has nothing left to resume. QuickJS provides exactly this hook. An embedder registers an interrupt handler with `JS_SetInterruptHandler`; the interpreter decrements a counter at every jump, conditional branch and function call, and when the counter reaches zero it resets the counter and calls the handler. If the handler returns a non-zero value, the interpreter throws an uncatchable exception that unwinds the whole call stack back to the host [Bellard 2026].

PicoCalc OS uses counted execution, with two small changes to QuickJS made for the purpose: the counter's reset value is lowered from 10,000 to 1,000 steps, so the handler runs about every 0.665 virtual ms of interpretation (1,000 steps at τ = 665 ns), and the counter is readable, so the nucleus knows exactly how many steps a segment has taken. The handler is short:

▸ nucleus/vm.c

```
// The segment's cost so far: entry, interpreter steps, and the native work its host
calls asked for.
static bool over_budget(world_t *W) {
  vtime_t used = PROFILE.turn_base_us + (vtime_t)(vm_steps(W) * PROFILE.ns_per_step /
1000.0) + K.seg_cost;
  return used > W->seg_budget_us;
}

static int on_interrupt(JSRuntime *rt, void *opaque) {
  (void)rt;
  world_t *W = opaque;
  W->interrupts++;
  if (W->oom) return 1;
  if (over_budget(W)) { W->overrun = true; return 1; }
  return 0;
}
```

The same pattern appears in other systems that run untrusted code. The Wasmtime WebAssembly runtime offers two interruption mechanisms: **fuel**, a count of executed instructions that traps when it runs out, and **epochs**, a counter that another thread advances on a timer. Its documentation states the trade-off directly: fuel-based interruption is completely deterministic but slows execution more, while epoch-based interruption costs about 10% but depends on wall-clock time, so the same program can stop at different points on different runs [Wasmtime docs 2026]. The QuickJS counter is fuel in this sense. PicoCalc OS needs that determinism, because its virtual clock is defined by counted work (Chapter 18): a segment's duration is a function of what it executed, and the same script must produce the same trace on every run.

Why does PicoCalc stop a world that runs out of budget instead of suspending it and running it again later? QuickJS's handler can only say "continue" or "stop", and suspending a world in the middle of a segment would need a separate C stack for every world (Section 4.5 explains why). But even if suspension were cheap, it would be the wrong response. The reason is what the budget means. A world's budget is not a time slice for sharing the processor fairly; it is a statement about how long that world's handlers are supposed to take. The keyboard driver's handler takes about 0.08 ms and has a 20 ms budget. If it ever runs for 20 ms, something is wrong with it, and the useful response is the response to any other fault: discard what it was doing, record what happened, and start a fresh instance. Suspending it would hide the bug and would still delay every other world, since the lane is shared either way. The budget is a **fault detector**.

11.3 What a segment is charged

The budget limits a segment's cost on the virtual clock, so its meaning depends on what is counted in that cost. A segment is charged three things:

$$cost = c_0 + s \cdot \tau + n \tag{11.1}$$

where c_0 = 40 µs is the fixed cost of entering and leaving a world, s is the number of interpreter steps, τ = 665 ns is the calibrated cost of one step, and n is the **native work**: time the nucleus charges for work it does in C on the world's behalf. The step count comes from QuickJS's counter, as the number of whole periods elapsed times 1,000 plus the change within the current period:

▸ nucleus/vm.c

```
int64_t vm_steps(world_t *W) {
  if (!W->ctx) return 0;
  return (W->interrupts - W->seg_interrupts0) * JS_GetInterruptCounterInit() + W-
>seg_counter0 - JS_GetInterruptCounter(W->ctx);
}
```

Native work accumulates in a single counter, `K.seg_cost`, that the nucleus resets at the start of each segment. Every host call adds one microsecond to it, and the operations that do real work add a cost proportional to the work. The constants come from the device profile:

▸ nucleus/kernel.c

```
profile_t PROFILE = {
  .name = "esp32p4-picocalc",
  .ns_per_step = 1330,          // replaced by vm_calibrate()
  .turn_base_us = 40,
  .fill_ns_per_px = 2.5,        // 16-bit stores into PSRAM
  .text_ns_per_px = 6.0,        // read, blend, write
  .diff_ns_per_px = 1.2,        // 32-bit compares, cached
  .json_ns_per_byte = 25,
  .spi_chunk_us = 12,
```

A rectangle fill, for example, charges 2.5 ns per pixel after it has written the pixels:

▸ nucleus/gfx.c

```
static inline void charge(double ns) { K.seg_cost += (vtime_t)(ns / 1000.0); }
…
void gfx_fill(dmabuf_t *b, int x, int y, int w, int h, uint16_t c) {
  if (!clip(b, &x, &y, &w, &h)) return;
  for (int j = 0; j < h; j++) {
    uint16_t *p = b->back + (size_t)(y + j) * b->w + x;
    for (int i = 0; i < w; i++) p[i] = c;
  }
  charge((double)w * h * PROFILE.fill_ns_per_px);
}
```

Table 11.2 collects every charge. Only τ is calibrated against a measurement on the ESP32-P4; the native constants are estimates for a RISC-V core writing to PSRAM, as Chapter 18 discusses. For the purpose of resource control, the accuracy of the constants matters less than their coverage: every kind of work a world can cause should appear somewhere in the table, or a world can do unbounded work that its budget never sees.

Table 11.2. What a segment is charged on the virtual clock. Each charge is added to the cost of the segment that caused the work.

Charge	Amount	Where it is added
entering and leaving a world	40 µs per segment	`run_segment` (`turn_base_us`)
interpreter step	665 ns	`vm_steps` × `ns_per_step`
host call (`__k` or `__ka`)	1 µs	`host_k`, `host_ka`
fill, rectangle, line	2.5 ns per pixel	`gfx_fill` and friends
text	6.0 ns per pixel of each glyph cell	`glyph`
damage diff (`commit`)	1.2 ns per pixel of the framebuffer	`gfx_commit`
JSON in a message, reply or settle	25 ns per byte, each way	`charge_json`
queueing a pixel transfer	2.5 ns per pixel	`hw_spi_pixels`

Two figures from the evaluation show the proportions. The compositor's redraw after a key press in the launcher costs 1.47 ms at the median, of which 0.86 ms is JavaScript layout and 0.57 ms native drawing and diffing; filling the whole 320 × 320 screen once costs 320 × 320 × 2.5 ns = 0.26 ms. For a world that draws, native work is nearly 40% of its cost. A budget that counted only steps would see the other 60%.

11.4 Checking the budget, and the bound it gives

A budget limits nothing unless it is checked, and how often it is checked determines how far a segment can go past it. PicoCalc checks at two kinds of point. The first is the interrupt handler above, which runs every 1,000 steps. The second is the end of every host call, synchronous or asynchronous:

▸ nucleus/vm.c

```
// After a host call: a world whose native work has taken it past its budget stops here,
uncatchably.
static JSValue after_host_call(JSContext *ctx, world_t *W, JSValue r) {
  if (!W->overrun && !over_budget(W)) return r;
  W->overrun = true;
  JS_FreeValue(ctx, r);
  JS_ThrowInternalError(ctx, "ran past its budget");
  JS_SetUncatchableException(ctx, 1);
  return JS_EXCEPTION;
}
```

▸ nucleus/vm.c

```
  for (size_t i = 0; i < sizeof SYNC_OPS / sizeof SYNC_OPS[0]; i++)
    if (!strcmp(op, SYNC_OPS[i].name)) {
      JS_FreeCString(ctx, op);
      K.seg_cost += 1;   // a host call costs about a microsecond
      return after_host_call(ctx, W, SYNC_OPS[i].fn(ctx, W, argc - 1, argv + 1));
    }
```

The exception is marked uncatchable, so a world cannot wrap its loop in `try … catch` and carry on. When either check fails, the nucleus records why, and at the end of the segment `run_segment` turns the reason into a crash:

▸ nucleus/kernel.c

```
  if (W->oom) snprintf(why, sizeof why, "ran out of memory (its limit is %zu KB)", W-
>mem_limit / 1024);
  else if (W->overrun) snprintf(why, sizeof why, "ran past its %s budget of %lld virtual
ms", budget_kind, (long long)(budget / 1000));
```

Figure 11.1 shows the checks on a segment's timeline. Interrupt checks fall every N = 1,000 steps; host-call checks fall wherever the world calls the nucleus. The segment is stopped at the first check at which its cost exceeds the budget B.

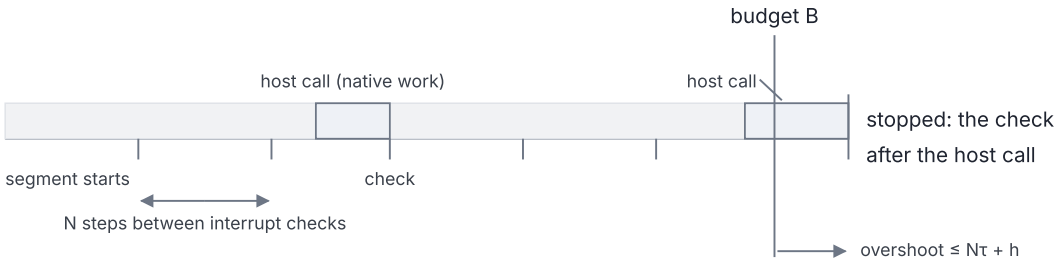


Figure 11.1. Budget checks along one segment. Interrupt checks (ticks) come every 1,000 steps; a host call (shaded) adds native work and is followed by its own check. The segment is stopped at the first check after its cost passes the budget B, so the overshoot is at most one period of steps plus one host call.

The bound follows from where the checks fall. Consider the last check at which the segment was still within budget; the segment's start counts as such a check. Between that check and the next one, the world can do at most two things: execute at most N steps, because the interrupt counter never goes more than N steps without calling the handler, and complete at most one host call, because every host call is followed by a check. The step counter is not reset by host-call checks, so the two allowances do not add up across checks. Writing h for the native cost of the most expensive single host call, the cost at the check that stops the segment is therefore at most

$$B + N\tau + h \tag{11.2}$$

The paper states the property with an additional c_0 as margin, $B + N\tau + c_0 + h$, which is 0.70 ms plus h for this profile; the derivation above does not need it, and the measurements fall inside both. For a loop with no host calls, $h = 0$ and the overshoot is at most $N\tau = 0.665$ ms. How large can h be? The first version of this chapter, like the paper, said a full-screen fill, 0.26 ms. The independent review of this book found that false in five ways: a message payload was charged 25 ns a byte with no limit (a large send overran its budget by 9 ms); a pixel transfer was charged for its rectangle before clipping (a forged flush request held the lane for 25 virtual seconds); a line with far-away ends and a very long string were charged for every invisible point and glyph; and the cost of encoding a world's frame was added after the last check (a segment ended at 29 ms on a 20 ms budget). The nucleus now caps a message, reply or frame at 64 KiB of JSON, charges fills, lines, text and pixel transfers only for what falls inside the framebuffer, and checks the budget once more when a segment ends. With those fixes the largest h is a maximal message, 64 KiB at 25 ns a byte, 1.64 ms; a fill of the largest allowed buffer (640 × 640) is 1.02 ms, and of the screen 0.26 ms.

The fault benchmark checks this with two kinds of runaway world. The spinning world's handler is `for (;;) {}`, run with budgets of 20, 50, 100 and 200 ms; the painter fills a framebuffer forever, so its time goes into native work:

▸ bench/worlds/painter.js

```
service(self).on("paint", () => { const fb = dma.alloc(320, 320); for (;;) fb.fill(0, 0,
320, 320, "#000"); });
```

Table 11.3. How long runaway segments ran before the nucleus stopped them (virtual ms, from the segment records in the trace). The overshoot is the distance past the budget.

World	Budget	Segment ran	Overshoot	Bound
`spin20` (`for (;;) {}`)	20	20.64	0.64	0.665
`spin50`	50	50.56	0.56	0.665
`spin100`	100	100.44	0.44	0.665
`spin200`	200	200.18	0.18	0.665
`painter` (fill loop)	20	20.25	0.25	0.665 + 0.26

The spinning world's overshoot varies between budgets because it depends on where in its 1,000-step period the counter stood when the budget ran out: the counter is not reset at the start of a segment, so the first check can come anywhere from 1 to 1,000 steps in. The painter's overshoot is part of one fill. Its loop does about three steps per iteration (the loop's jump, the call to `fill` and the prelude's call to `__k`) and 0.256 ms of native work; the trace of its stopped segment records 254 steps and 20.05 ms of native work, about 78 fills. It reaches its budget in the middle of a fill and is stopped by the check after that fill returns.

A world's file is evaluated once when the world starts, in a segment that has its own fixed budget of 600 ms (`vm_begin_segment(W, MS(600))` in `vm_create`), because evaluating a large file and running its top-level code legitimately takes longer than handling a message. The same checks apply, so a world whose top level loops forever crashes during its start and goes through the supervisor's backoff like any other crash.

TRY 11.1

Run code under a budget

The widget below runs JavaScript in a Web Worker with a sealed prelude, much as PicoCalc runs a world. Its program loops forever. Run it and watch the page stop the worker when the 200 ms budget expires, then run it again: the second run gets a fresh worker, which is the restart. Change the budget, or change the loop into one that finishes, and compare. The other presets try to reach the real clock and to allocate without limit.

Two differences from PicoCalc OS matter. In the page, the budget is real time measured by the main thread, and the only way to stop a worker is `terminate()`, which is preemption from outside rather than a counted check, so a run stops at a slightly different moment each time; the virtual clock never does that. And a worker has no heap limit of its own, so the allocation preset is stopped by its time budget, not by a memory limit.

```
let n = 0;
for (;;) n++;   // nothing in the loop ever yields
```

run 1 virtual clock at +1 s world epoch 2 budget

```
× stopped at its budget of 200 ms (real time on this browser's
clock). The kernel terminated the worker, discarded everything in
it, and started a fresh world (epoch 2).
```

The trace in Figure 11.2 shows the same thing inside PicoCalc OS. The Faults app asks the sandbox to run an endless loop; the sandbox holds the lane for its whole 200 ms budget, is stopped, has its effects dropped, and is restarted 100 ms later. Every world that became ready during those 200 ms waited, which is the cost of a non-preemptive lane that Chapter 5 turns into a blocking term.

A loop that never ends 570 virtual ms

The Faults app asks the sandbox to loop forever. The kernel stops it at its 200 ms budget, drops its effects, tells the asker, and restarts it 100 ms later.

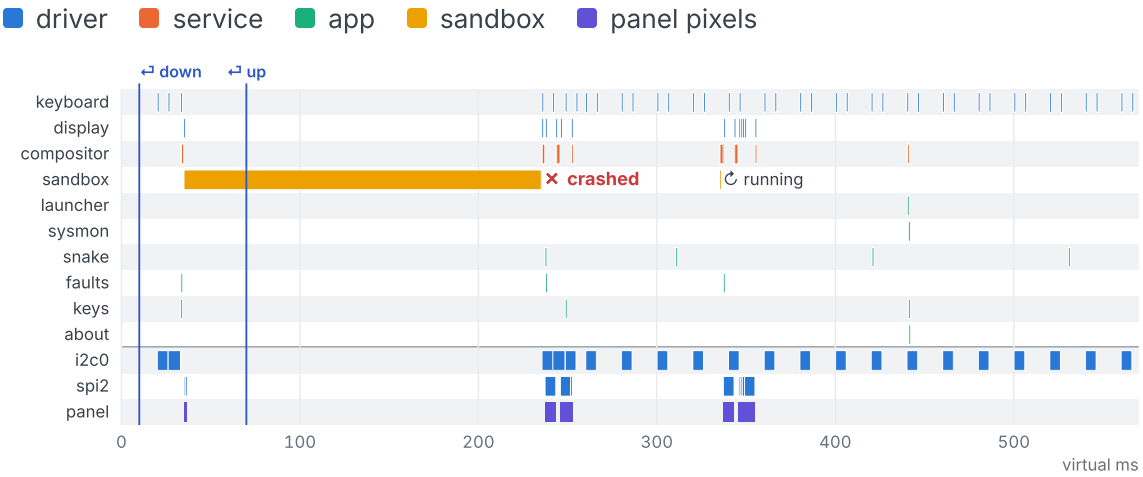


Figure 11.2. A loop that never ends, recorded by the host build: the sandbox's segment runs to its 200 ms budget and is stopped; the keyboard driver's polls wait behind it.

11.5 An accounting leak: native work outside the budget

The bound of Equation 11.2 depends on the cost in `over_budget` including native work. An earlier version of the nucleus compared only the interpreter's step count with the budget. The paper describing PicoCalc OS claimed, correctly for that version's intent and incorrectly for its code, that a world's budget bounds every one of its segments. The claim was checked line by line against the source while the paper was being prepared, and the check found that a segment spending its time in host calls could run far past its budget.

The painter makes the size of the leak easy to estimate. Its loop takes about three and a quarter steps per iteration (254 steps for 78 fills in the trace), and a 20 ms budget is about 30,000 steps at 665 ns each. A step-only check would therefore have let it run about 9,200 iterations, each charging 0.257 ms of fill and host call, for roughly 2.4 virtual seconds on the lane before the check fired. That figure is a calculation from the step counts, not a measurement: the old code is not kept in the repository. It is still the right order of magnitude, more than a hundred times the budget, and during those seconds the keyboard driver would not have polled at all.

The fix has two parts, and both are visible in the code above. `over_budget` adds `K.seg_cost` to the step-based cost, so the interrupt handler sees native work; and `after_host_call` checks after every host call, so a world that makes few steps between expensive calls is caught at the call rather than up to 1,000 steps later. The painter now runs as a regression test in the fault benchmark, where it is stopped at 20.25 ms.

The general lesson is older than PicoCalc. Banga, Druschel and Mogul observed that in a conventional operating system "the operating system's notions of protection domain and *resource principal* coincide in the process abstraction" [Banga et al. 1999], and that this coincidence breaks when a server process does work on behalf of many clients, or when the kernel does work (protocol processing, for instance) that is charged to no process at all. Their resource containers separate the two notions, so that work can be charged to the principal it is done for. PicoCalc's charge model does the same thing on a small scale: work the nucleus does in C because a world asked for it is charged to that world's segment, not to the nucleus and not to nobody.

Three kinds of work are still not charged to the segment that causes them:

- **Bus time.** A bus transfer starts after the segment that asked for it has ended and takes the time its bits take on the wire. The segment pays only for queueing it. Bus time is a separate resource with its own queue (Section 11.8).
- **The nucleus's own bookkeeping** between segments: delivering messages, firing timers, settling promises. These costs are small, and they are not part of any world's cost on the virtual clock.
- **Work a world causes in another world.** A message's handler is charged to the world that handles it. This is usually right, since the receiver chose to provide the service, and occasionally exploitable, as the next section shows.

11.6 Cross-world costs: who pays for the compositor's work?

Apps do not draw. Each app publishes its screen as a frame, a tree of plain objects (Chapter 13 describes the pipeline), and the compositor lays the frame out and draws it. The kernel publishes an app's frame on the `frames` channel after every segment in which it changed, whether the app is in front or not, and the compositor's handler decodes and stores the tree of every app that publishes one. Rendering the frame is charged to the app; decoding it, storing it and later laying it out are charged to the compositor.

That division means an app can make the compositor spend memory and time. An experiment shows how much. Two test worlds were added to a copy of the system (they are not part of the repository): an app that is never brought to the front and whose frame is one long wrapped text, and a helper that tells it how long to make the text.

```
// A sketch of the test app: never focused, it publishes a large frame every 300 ms.
app("bigframe")
  .state({ n: 1, flip: false })
  .on("grow", (s, { n }) => { s.n = n; })
  .every(300, (s) => { s.flip = !s.flip; })
  .screen((s) => ui.col(ui.text((s.flip ? "x" : "y").repeat(s.n), { wrap: true })));
```

The alternating character makes the frame differ every 300 ms, so the kernel publishes it every 300 ms. With the app given a heap limit of 1 MB (the default for a manifest entry without `mem_kb`) and frames of 300,000 characters, the compositor crashed while decoding the frame message, inside the prelude's `JSON.parse`, because decoding needs room for both the message's text and the parsed tree, and the two did not fit in its 768 KB heap next to the roughly 270 KB it uses itself:

```
[  1017.559] compositor crashed handling frames: ran out of memory (its limit is 768 KB)
[  1216.086] compositor crashed handling frames: ran out of memory (its limit is 768 KB)
[  1516.087] compositor crashed handling frames: ran out of memory (its limit is 768 KB)
[  1924.170] compositor crashed handling frames: ran out of memory (its limit is 768 KB)
[  2775.270] compositor crashed handling frames: ran out of memory (its limit is 768 KB)
[  2775.270] compositor crashed 5 times in ten seconds; it stays stopped
```

Each new frame killed the restarted compositor again, the fifth crash within ten seconds stopped it for good, and from then on the panel showed its last picture and nothing else. With the app's limit at 512 KB, the largest any app has in the shipped manifest, a 150,000-character frame did the same more slowly: the fifth crash came about seven seconds after the first large frame. With 384 KB, the launcher's limit, the app ran out of its own memory before it could build a frame large enough to hurt the compositor.

Nothing in the experiment breaks isolation. The app reached no other world's memory and used no authority it had not been given; it published frames, which every app does. The problem is accounting. The memory and time needed to receive a frame are charged to the receiver, and the receiver is a single, shared, important service whose limits were set for well-behaved frames. There are several ways to close the gap, each of which moves the cost back to the principal that causes it:

- The kernel could refuse to publish a frame above a size limit, and treat an oversized frame as a fault of the app that rendered it. The kernel already holds the frame's text when it decides whether to publish (`run_segment` compares it with the previous one), so the check costs nothing.
- The compositor could ignore frames from apps that are not in front, and ask for a fresh frame when an app comes to the front. This bounds the damage to the app the user is looking at.

- The channel could carry a per-message size limit declared by its subscriber, so that every subscriber can protect itself, not only the compositor.

None of these is implemented. The point of the experiment is that a bound on each world's own consumption is not the same as a bound on what each world can cause. Exercise 7 asks you to implement the first remedy and rerun the experiment.

11.7 Memory limits

Memory is accounted more simply than time, because every allocation a world's runtime makes goes through the allocator hooks the nucleus installs when it creates the runtime:

▸ nucleus/vm.c

```
static void *w_malloc(JSMallocState *s, size_t size) {
  world_t *W = s->opaque;
  if (W->mem_used + size > host_limit(W)) { W->oom = true; return NULL; }
  void *p = malloc(size);
  if (!p) return NULL;
  size_t u = malloc_usable_size(p) + 8;
  W->mem_used += u; s->malloc_size += u; s->malloc_count++;
  return p;
}
```

The accounting counts the allocator's usable size of each block plus 8 bytes of overhead, as QuickJS's own allocator does, and the limit is the world's `mem_kb` from the manifest, scaled by 1.17 on a 64-bit host so that the host build behaves like the 32-bit device (Chapter 6 explains the factor). A refused allocation does two things. QuickJS sees `NULL` and throws an out-of-memory exception, which the world's code could catch; and the nucleus sets `W->oom`, which the interrupt handler checks before anything else. A world that catches the exception and keeps going is therefore stopped at its next interrupt check, with the reason "ran out of memory (its limit is N KB)". The fault benchmark's allocation loop, which pushes 256-element arrays onto an array forever, is stopped after 0.09, 0.25 and 0.55 virtual ms with limits of 256, 512 and 1,024 KB. Those times are short only because the cost model charges nothing for allocation, which Chapter 18 lists among the model's omissions.

The interpreter's stack is limited separately, to 192 KB per runtime (`JS_SetMaxStackSize`). Deep recursion throws an `InternalError` with the message "stack overflow", which is an ordinary exception: a world may catch it and continue, and nothing outside the world is affected.

The limits in the manifest add up to 5.0 MB for thirteen worlds, against 2.5 MB actually used after boot and 32 MB of PSRAM on the board. The nucleus does not check that the limits fit in physical memory; a manifest whose limits exceeded it would admit worlds whose allocations could all succeed individually and fail together. On a system this small, choosing the limits is part of writing the manifest, and Chapter 6 measures what each world needs.

11.8 Resources without a budget

The remaining rows of Table 11.1 have no per-world accounting, and each has a consequence that a careful designer should know.

> NOTE
>
> The first pass, a later revision of PicoCalc OS described in a separate book, *From L4 to Small Worlds* (Chapter 7), turned three of these shared resources into per-world quotas from the manifest: mailboxes (`mbox` messages and `mbox_kb` kilobytes, counted from the moment a message is sent), outstanding asks (`ask_slots`), and handles and framebuffers (`handles` , charged to the world that made each handle, and `bufs`). The boot fails unless the quotas fit the kernel's tables, so no world can exhaust a table for the others. A send or ask to a full mailbox fails at the sender, a channel post to a full subscriber is dropped and counted, and a timed-out ask that the receiver has not started is taken out of its mailbox. This section describes the system before that pass.

Mailboxes and nucleus memory. A message is copied into memory the nucleus allocates (`new_msg` duplicates the type and the JSON text) and sits in the receiver's mailbox until the receiver handles it. Mailboxes have no length limit, and the memory they hold is charged to no world's heap. The messaging benchmark shows the scale without any malice: its sender puts 1,000 messages into one receiver's mailbox in a single segment, and the receiver works through them for 64 ms. The nucleus tracks each mailbox's peak length (`mbox_peak` , visible in the system monitor), but nothing acts on it. A world that sends faster than its receiver handles messages grows the nucleus's memory without bound, which on the device would end with the whole system out of memory. Flow control between worlds is a matter for the protocols of Chapter 12.

Outstanding asks. An ask is recorded in a table of 1,024 entries shared by all worlds and indexed by the ask's number modulo 1,024:

▸ nucleus/kernel.c

```
static ask_t *find_ask(int id) { ask_t *a = &K.asks[id % MAX_ASKS]; return a->id == id &&
a->live ? a : NULL; }
```

A new ask overwrites its slot without checking whether the slot is still in use. If more than 1,024 asks are outstanding at once, the newest overwrite the oldest, and the overwritten asks are forgotten: their replies find no entry, their timeouts find no entry, and their promises never settle. An experiment in a copy of the system confirms it. A world issued 1,100 asks at once to a service that takes ten seconds per request, each with a 500 ms timeout. After three seconds, 1,024 of the promises had settled (by timeout) and 76 never did. A world awaiting all of them would have stayed in its turn forever, with its mailbox closed (Chapter 3). The limit is generous for a handheld, but its failure mode is silent, where a refusal ("too many outstanding asks") would have been a fault the asker could see.

A related property belongs to Chapter 12 but is worth noting here: a timed-out ask is not withdrawn. The request stays in the receiver's mailbox and is handled later, and its reply is dropped. In the experiment above, the slow service would have spent hours answering requests nobody was waiting for.

Bus time. The two buses are shared resources with their own queues: a transfer starts when the segment that requested it ends and the bus is free, so transfers run in the order they were requested, with no priority and no per-world share. On the I2C bus this already matters. The keyboard driver's status read takes 6.0 ms every 20 ms, so the bus is 30% busy with no key pressed (Chapter 13), and the power driver's battery read waits behind it when they coincide. What limits bus use is not a budget but authority: only worlds whose manifest entries claim `i2c0:0x1f` or `spi2` can obtain handles for those buses (Chapter 9), and in the shipped manifest those are the three drivers. Possession of the handle is the whole of the resource control.

Fixed tables. Handles (128), DMA framebuffers (16) and worlds (32) live in fixed-size tables. A world that exhausts one gets an error from the call that needed a new entry ("out of handles", "out of DMA buffers"), which it can handle. Only worlds with the `dma` claim can allocate framebuffers, so in the shipped system only the compositor can exhaust that table; any world holding a handle can narrow it repeatedly, and each narrowing takes an entry in the handle table until it is released.

Timers. A world may declare up to 16 periodic timers. Their ticks are coalesced: if a tick fires while the previous tick of the same timer is still waiting in the mailbox, no second one is queued (`tick_queued` in `handle_event`). A slow world therefore never accumulates a backlog of ticks, only a delay.

11.9 Choosing budgets

A budget that is too small turns normal work into crashes; one that is too large lets a broken world block the lane for longer before it is caught. The traces recorded by the benchmarks give the longest segment each world actually ran, which is the number to compare with its budget.

Table 11.4. Budgets from the manifest against the longest segment observed in the benchmark traces (virtual ms). The snake row is the median autopilot tick from the utilisation benchmark.

World	Budget	Longest segment seen	Handling
keyboard	20	0.08	its 20 ms poll
power	20	0.07	its start
display	30	0.50	a flush
compositor	80	1.86	a redraw
launcher	50	0.22	a timer
snake (autopilot)	50	11.1	a tick, median
sandbox	200	133.1	the 100,000-iteration loop typed at the REPL

Most budgets exceed the longest observed segment by a factor of 40 or more. That margin is deliberate for drivers, whose segments vary with the hardware's answers, but it has a price that Chapter 5 makes exact: in a non-preemptive system, the worst-case delay a high-priority world can suffer includes the longest segment any lower-priority world is allowed to run, and that is its budget, not its typical segment. The compositor's 80 ms budget, for instance, is a bound on how long the keyboard driver can be kept from polling by a compositor bug. The load experiment of Section 4.7 shows the effect directly: a background world that computes for 20 ms every 50 ms raises the launcher's median key-to-pixel latency from 30.9 to 39.9 ms, whatever its priority. Tighter budgets for apps, closer to what they actually need, would shorten that blocking term; Chapter 5 analyses the trade.

11.10 Budgets elsewhere

PicoCalc's mechanisms have counterparts in general-purpose systems, and comparing them clarifies which choices follow from its segment model.

Linux control groups (version 2) give a group of processes a CPU bandwidth limit and a memory limit. `cpu.max` holds a quota and a period: the group "may consume up to `$MAX` in each `$PERIOD` duration", and a group that exhausts its quota is throttled until the next period. `memory.max` is a "memory usage hard limit": if a group's usage reaches it and cannot be reduced, "the OOM killer is invoked in the cgroup" [cgroup v2 docs 2026]. Processor time is delayed and memory is killed, because a preempted thread can always be resumed later while an allocation that cannot be satisfied cannot. PicoCalc stops the world in both cases, because a segment that is stopped halfway has left nothing outside the world in an inconsistent state, so stopping is as safe as delaying and much simpler.

KaffeOS, a Java virtual machine built as an operating system, ran each process "as if it were run in its own virtual machine, including separate garbage collection of its own heap" so that processes could be isolated and their resource use limited inside one runtime [Back et al. 2000]. PicoCalc goes one step further in the same direction: each world has not only its own heap but its own runtime, so there is no shared engine state through which one world's garbage or allocation pattern could slow another.

WebAssembly runtimes meter instructions with fuel (Section 11.2), and browsers stop scripts that hold a page's main thread for too long; Chapter 16 returns to how the browser, the most widely deployed language-isolated system, controls the resources of the pages it runs.

KEY POINTS

- Isolation stops a world from touching others' state; resource control stops it from taking everything, and needs accounting, limits and enforcement.
- PicoCalc OS enforces both its time and memory limits by stopping the world, which atomic segments make safe: nothing a stopped segment did is visible outside it.
- A segment is charged c_0 + steps·τ + native work, and its budget covers all three; the budget is a fault detector, not a time slice.
- Checks at every 1,000 interpreter steps and after every host call bound a segment's overshoot to Nτ + h, at most 0.665 ms plus one host call, which the nucleus now bounds at 1.64 ms; the measured overshoots are 0.18 to 0.64 ms.
- Accounting must cover every way a principal consumes a resource: counting only steps would have let a fill loop run more than a hundred times past its budget, until native work was added to the check.
- Work one world causes another to do is charged to the one that does it, which let an app publishing large frames crash the compositor until it was stopped.

- Mailbox length, outstanding asks, bus time and the kernel's fixed tables had no per-world budget; the ask table overwrote its oldest entries silently past 1,024 outstanding asks. The first pass gave each world quotas for its mailbox, its asks, its handles and its framebuffers.
- In a non-preemptive system a budget is also the blocking term for every higher-priority world, so budgets should be close to what a world needs, not merely large enough.

Resource control is where isolation meets scheduling: a budget protects the lane from a world, and at the same time sets how long every other world may have to wait. The next chapter turns to how worlds cooperate rather than how they are kept apart: the messages they send, when those messages arrive, in what order, and at what cost.

EXERCISES

1. BASIC Using Table 11.2, compute the cost on the virtual clock of a segment that executes 2,000 steps, fills a 100 × 50 rectangle, draws 20 characters in the 8 × 16 font and sends a 200-byte message.
2. BASIC A world has a 30 ms budget. Its segment executes a loop with no host calls. What is the latest virtual time, measured from the segment's start, at which it can be stopped? What if each iteration also fills the full screen?
3. ANALYSIS The overshoots in Table 11.3 differ between budgets. Explain why, using the fact that the interrupt counter is not reset at the start of a segment. Which values of the overshoot are possible for a `for (;;) {}` loop, and why can none exceed 0.665 ms?
4. ANALYSIS Suppose the interrupt period were raised back to QuickJS's default of 10,000 steps. Recompute the overrun bound. What would it mean for the keyboard driver's worst-case delay if the sandbox (budget 200 ms) overran?
5. CODE Use the host build's `steps` command (`./build/picocalc-host -` reads commands from stdin) to count the steps of `for (let i = 0; i < 1000; i++) {}` and of a loop of 1,000 calls to an empty method. Explain the counts in terms of jumps, branches and calls.
6. CODE Add a world to a copy of `js/system.json` that allocates a large array inside a `try … catch` and keeps allocating after catching the out-of-memory error. Show from the UART log (the `#OCT1` frame) when and why it was stopped.

7. CODE Implement a frame size limit in `run_segment`: if a world's rendered frame exceeds 64 KB, crash the world with the reason "frame too large". Rerun the experiment of Section 11.6 and show that the compositor survives.

8. CODE Make `kernel_ask` refuse a new ask (reject its promise at once with "too many outstanding asks") when its slot in the table is still live, instead of overwriting it. Write a test world that demonstrates the difference.

9. ANALYSIS The mailbox has no limit. Propose a limit and a policy for what happens when it is reached (drop the new message, fail the sender's call, crash the sender, crash the receiver). For each policy, name a world in the shipped system that it would harm and one it would protect. (The first pass chose a limit and two of these policies; compare your answer with *From L4 to Small Worlds*, Chapter 7.)

10. DISCUSSION cgroups throttle processor time and kill on memory; PicoCalc kills on both. Under what design of the lane would throttling become possible and worthwhile? What would it cost in the rest of the system?

11. ANALYSIS Bus time is not charged to any segment. Design a charge: who should pay for the 6.0 ms of each keyboard status read, and how would a budget on bus time interact with the 2 ms gap the controller requires?

12. PROJECT Implement resource containers in the nucleus: a world can attach a "container" to an ask, and the receiver's segments handling it are charged to the asker's budget. Use it for the compositor's handling of frames and measure what changes in the frame-flood experiment.

FURTHER READING

[Banga et al. 1999] The clearest statement of why the principal that is charged must be separable from the principal that is protected, with experiments on a web server.

[Back et al. 2000] Processes inside one Java virtual machine, with separate heaps and resource limits: the closest earlier design to PicoCalc's worlds.

[Wasmtime docs 2026] A short, precise comparison of counted (fuel) and timed (epoch) interruption in a production WebAssembly runtime.

[cgroup v2 docs 2026] The Linux interface for limiting groups of processes; worth reading for its distinction between limits that throttle and limits that kill.

Messages and IPC

Worlds share no memory, so everything one world wants from another has to travel as a message. This chapter explains what a message is in PicoCalc OS, when it arrives, in what order relative to other messages, how a request and its reply are matched and how they fail, how a hardware handle can ride inside a message, and what all of this costs on the virtual clock. It places the design among the inter-process communication (IPC) mechanisms of other operating systems, from Mach and L4 to Erlang and the browser, and it ends with the protocol patterns that the compositor and the display driver use to keep talking to each other across crashes and restarts. By the end you should be able to predict the order in which a world will see a set of messages, list every way an `ask` can fail and what the asker sees in each case, and estimate a protocol's cost from the number of messages and segments it needs.

12.1 Why messages

Two programs that need to cooperate can do it in one of two ways. They can share memory: both map the same bytes, one writes and the other reads, and a lock or an atomic operation decides who goes first. Or they can exchange messages: one program hands a value to the system, and the system later hands a copy to the other. Lauer and Needham argued in 1978 that the two styles are duals, that a system built one way can be transformed into a system built the other way, and that a subsystem's performance is "identical to that of its dual system, given identical scheduling strategies" [Lauer and Needham 1979]. They offered the claim with little evidence, and later systems have treated it as a statement about structure rather than about speed, but it frames the choice correctly. The difference is not what can be computed but what each style makes easy to get wrong.

Shared memory is fast and dangerous. Chapter 3 showed what it costs to reason about: every interleaving of two threads' accesses must be considered, and the protection against the bad ones is locks the programmer must remember to take. It also requires trust. A program that shares memory with another can corrupt it, and a crash in one can leave the shared data half-updated for the other.

Messages are slower and safer. The receiver gets a copy, so the sender cannot change it afterwards; the receiver handles one message at a time, so its own state is never seen half-updated; and a crashed sender cannot leave a half-written message behind, because a message is either handed to the system or not.

For PicoCalc OS the choice was made by the isolation design before performance came into it. Each world is a separate QuickJS runtime with its own heap and its own garbage collector (Chapter 10). A JavaScript object belongs to one heap; there is no way to hand a reference to an object in one runtime to another runtime, because the second runtime's garbage collector knows nothing about it. Worlds can therefore exchange only values that can be copied out of one heap and rebuilt in another. PicoCalc copies them as JSON text, which the prelude produces with `JSON.stringify` and the receiver's prelude parses with `JSON.parse`. A payload is a tree of plain data, and a world that sends an object can mutate it the next moment without the receiver ever seeing the change.

12.2 Message passing in operating systems

Most of the design space was explored by earlier systems, and PicoCalc's position in it is easiest to see by comparison.

Mach made messages the kernel's central abstraction. In Mach, "a port is a communication channel – logically a queue for messages protected by the kernel", and "a message is a typed collection of data objects used in communication between threads" [Accetta et al. 1986]. Ports are named by capabilities: "access to a port is granted by receiving a message containing a port capability (to either send or receive)". Large messages are not copied: the sender's pages are mapped into the receiver and marked copy-on-write, so "a single message may transfer up to the entire address space of a task". Mach's messages were asynchronous and general, and they were slow enough that a generation of research was spent making microkernel IPC cheaper.

L4 took the opposite approach. Liedtke designed the kernel around one fast, synchronous operation in which the sender and the receiver meet (a rendezvous) and the kernel moves the message directly from one to the other without buffering it [Liedtke 1993]. The original L4 on a 50 MHz Intel 486 performed a one-way IPC in 250 cycles, 5 µs, which Elphinstone and Heiser describe as "a factor 10–20 faster than other

contemporary microkernels" [Elphinstone and Heiser 2013]. Synchronous IPC is fast because it needs no buffering, and it is hard to use safely between parties that do not trust each other, because a sender blocks until the receiver is ready.

Erlang builds whole systems from processes that share nothing. "Message passing is the only way for processes to interact", and Armstrong argues that "isolation implies that message passing is asynchronous": with synchronous messages, a faulty receiver could block its sender indefinitely [Armstrong 2003]. Messages between a pair of processes arrive in the order they were sent, and a `receive` statement matches incoming messages against patterns.

E organises programs into vats: each vat has its own heap and a single event loop, handles one message at a time to completion, and communicates with other vats by eventual sends that return promises [Miller 2006]. A vat never blocks on a promise. It registers what to do when the promise is resolved and goes on to its next message.

The browser gives each page and worker its own event loop and lets them communicate by `postMessage`. The value is copied by the structured clone algorithm, which exists "to support passing JavaScript objects, including platform objects, across realm boundaries", and some objects can be transferred instead of copied, which "is effectively recreating the object while sharing a reference to the underlying data" and leaves the sender's copy detached [WHATWG structured clone 2026]. Chapter 16 returns to the browser as an operating system.

Table 12.1. Message passing in six systems.

System	Blocking	Payload	Addressed to	What a sender needs
Mach	asynchronous (queued at the port)	typed data; large regions by copy-on-write	a port	a send right (a capability)
L4	synchronous rendezvous	copied directly between the parties	a thread	depends on the version
Erlang	asynchronous	copied terms	a process identifier	the identifier
E	asynchronous, with promises	copies, or references to objects	an object	a reference to it
Browser	asynchronous	structured clone, or transfer	a window, worker or port	a reference to it
PicoCalc OS	asynchronous, with promises for `ask`	JSON text, plus handles	a world's name or a channel	the name (send, ask); a self-declared channel (publish)

PicoCalc OS sits closest to Erlang and E. Its messages are asynchronous and copied, and its request-reply operation returns a promise, as E's eventual send does. Its unusual features are in the last two columns and in timing. Sending needs only a world's name, not a capability; and, as the next sections explain, a message is delivered at a precise virtual instant, the end of the sender's segment, which gives the system an ordering guarantee stronger than Erlang's.

12.3 Send, ask and publish

A world has three ways to talk, all defined in the prelude on top of the two host functions `__k` (for operations that finish at once) and `__ka` (for operations that finish later and return a promise):

▸ history/first-pass/js/prelude.js

```
  g.send = (to, type, payload) => { const [j, hs] = enc(payload); k("send", to, type, j,
hs); };
  g.publish = (ch, payload) => { const [j, hs] = enc(payload); k("publish", ch, j, hs);
};
  g.ask = (to, type, payload, opts) => {
    const [j, hs] = enc(payload);
    return ka("ask", to, type, j, hs, (opts && opts.timeout) || 2000).then((v) => (v &&
typeof v === "object" ? reviveTree(v) : v));
  };
```

- `send(to, type, payload)` delivers one message to the world named `to`. The receiver's handler for `type` runs with the decoded payload. The sender learns nothing more: not when the message was handled, not whether the handler succeeded.
- `ask(to, type, payload, {timeout})` sends a message and returns a promise of the handler's return value. The promise rejects if the receiver is stopped, crashes while handling the message, has no handler for it, or does not answer within the timeout, 2,000 ms unless the caller gives another.
- `publish(channel, payload)` delivers a message to every world that declared, with `.needs(channel)`, that it wants the channel. Only a world that declared `.provides(channel)` may publish on it.

On the receiving side there is a single mechanism. Every message, whatever operation sent it, arrives in the receiver's **mailbox**, a first-in, first-out queue in the nucleus, and is handled in a turn by the handler registered with `.on(type, fn)`. A handler's return value is the reply if the message was an ask, and is ignored otherwise. The prelude's dispatch function, shown in Chapter 3, makes the distinction.

In the nucleus, `send` is a short host call. It looks the receiver up by name, checks that the manifest lets the sender reach it (`may_talk`, added by the first pass; see the note at the end of this section), checks the payload's size and the handles named in it (Section 12.7), charges the JSON text to the sender's segment, reserves room in the receiver's mailbox and creates a deferred delivery:

▸ history/first-pass/nucleus/vm.c

```
static JSValue op_send(JSContext *ctx, world_t *W, int argc, JSValueConst *argv) {
  char *to = sarg(ctx, argv, argc, 0), *type = sarg(ctx, argv, argc, 1), *json =
sarg(ctx, argv, argc, 2);
  int hs[MAX_MSG_HANDLES], nh = argc > 3 ? int_list(ctx, argv[3], hs, MAX_MSG_HANDLES) :
0;
  world_t *T = to ? world_by_name(to) : NULL;
  JSValue r = JS_UNDEFINED;
  if (!T) r = fail(ctx, "there is no world called \"%s\"", to ? to : "");
  else if (!may_talk(ctx, W, T, type ? type : "", false)) r = JS_EXCEPTION;
  else if (too_big(ctx, json)) r = JS_EXCEPTION;
  else if (check_grants(ctx, W, hs, nh)) {
    msg_t *m = new_msg(M_ON, type, json, W->id);
    memcpy(m->hs, hs, sizeof(int) * nh); m->nh = nh;
    charge_json(json ? strlen(json) : 0);
    if (!mbox_reserve(T, m)) {
      r = fail(ctx, "%s's mailbox is full (%d messages, %zu KB queued)", T->name, T-
>mq_count, T->mq_bytes / 1024);
      free_msg(m);
    } else defer_deliver(T, m);
  } else r = JS_EXCEPTION;
  free(to); free(type); free(json);
  return r;
}
```

A channel post fans out into one delivery per subscriber, skipping the publisher itself and stopped worlds:

▸ nucleus/kernel.c

```
void kernel_publish(world_t *from, const char *ch, const char *json, const int *hs, int
nh) {
  K.published++;
  for (int i = 0; i < K.n_worlds; i++) {
    world_t *T = K.worlds[i];
    if (T == from || T->status == W_STOPPED || !world_needs(T, ch)) continue;
    msg_t *m = new_msg(M_ON, ch, json, from ? from->id : -1);
    for (int k = 0; k < nh && k < MAX_MSG_HANDLES; k++) m->hs[m->nh++] = hs[k];
    defer_deliver(T, m);
  }
}
```

The kernel itself publishes on two channels. `worlds.status` carries a notice whenever a world crashes, is restarted or is stopped (Chapter 14), and `frames` carries every app's rendered screen to the compositor after each segment in which it changed (Chapter 13). In the shipped system, channels carry key events (`keys`, published by the keyboard driver), the battery and screen state (`power`), frames and world status.

Who may talk to whom deserves a precise answer, because it is part of the system's protection. A world may `send` to or `ask` any world whose name it knows; the nucleus checks only that the name exists. A world may `publish` only on channels it declared in `.provides`, and may receive only from channels it declared in `.needs`. But both declarations are made by the world's own code, not by the manifest, so any world can declare that it provides `keys` and then publish fake key events to the compositor. Names and channel declarations are therefore conventions between cooperating worlds, not authority. The only authority in messages is the handles they carry. Chapter 8 classifies these two kinds of protection, and Exercise 10 asks how to move channel declarations into the manifest.

NOTE

The first pass, a later revision described in a separate book, *From L4 to Small Worlds* (Chapters 5 and 7), made the manifest the authority for messages. A world may send only to the worlds its entry lists under `sends` and ask only those under `asks` (`W may not send to X: its manifest entry does not list it under "sends"`); it may publish and hear only the channels its entry lists under `provides` and `needs`, and a world whose file declares more is stopped at start; and a message whose type is a channel the receiver hears is refused, so a forged `keys` or `frames` message cannot be sent at all. Each mailbox is bounded by the entry's `mbox` and `mbox_kb` quotas: a send or ask to a full mailbox fails at the sender, and a channel post to a full subscriber is dropped for that subscriber and counted. The rest of this chapter describes the system before that pass, except where it says otherwise.

12.4 From send to mailbox

A send does not deliver anything when it is called. Like every other effect, it is appended to the running segment's list of deferred effects and applied when the segment ends; if the segment crashes, the list is discarded (Section 3.7 explains the reasons). Applying a delivery schedules an event at the segment's end time, and when that event is handled the message joins the receiver's mailbox:

▸ history/first-pass/nucleus/kernel.c

```
    case E_DELIVER: {
      msg_t *m = e->msg;
      if (!W || W->status == W_STOPPED) {
        if (m->ask) { char t[96]; snprintf(t, sizeof t, "%s is stopped", W ? W->name :
"?"); settle_ask(m->ask, 0, t, NULL, 0); }
        free_msg(m);
        break;
      }
      for (int i = 0; i < m->nh; i++)
        for (int k = 0; k < MAX_HANDLES; k++) if (K.handles[k].live && K.handles[k].id ==
m->hs[i]) handle_grant(&K.handles[k], W);
      m->t = e->t; m->seq = ++K.seq;
      trace_msg(e->t, m->from, W, m);
      if (W->mbox_tail) W->mbox_tail->next = m; else W->mbox = m;
      W->mbox_tail = m;
      if (++W->mbox_len > W->mbox_peak) W->mbox_peak = W->mbox_len;
      K.delivered++;
      break;
    }
```

Four properties of the system follow from this code and from the rule that effects wait for the end of the segment.

1. **A message is never visible before its sender's segment has ended.** The receiver cannot run in virtual time before the sender has finished, which is what a single lane requires.
2. **A crash recalls everything its segment sent.** A handler that sends three messages and then throws has sent nothing.
3. **A message to a stopped world is answered by the kernel, not lost.** If it was an ask, the asker's promise is rejected with "X is stopped"; if it was a send or a post, it is dropped, since nobody is waiting for it.
4. **A message to a crashed world that is restarting waits for the new instance.** The mailbox belongs to the world, not to its runtime, and survives the restart (Chapter 14). This is usually what the sender wants, and occasionally a trap, as Section 12.10 shows.

Once in the mailbox, a message waits for the lane. The receiver takes its next message only when it is not in a turn: a world handling an asynchronous turn keeps its mailbox closed until the turn ends, even while it waits for a bus transfer or an ask of its own (Section 3.6). Messages that arrive meanwhile queue up behind the turn.

TRY 12.1

Watch a message wait behind an asynchronous turn

The widget steps through two messages sent to the same world. The first one's handler asks a third world and awaits the reply. Step through and watch the second message arrive in the mailbox while the first turn is still waiting, and stay there until the reply has come back and the first turn has ended. Then switch to the scenario in which the first handler throws in its second segment, and see which effects of the first segment were already out.

```
app("weather")
  .state({ temp: null })
  .on("refresh", async (S) => {
    send("log", "note", "refreshing");
    const t = await ask("sensor", "read");
    S.temp = t;
    publish("weather", { temp: t });
    return t;
  });
```

Only now does weather take the system monitor's message (refresh (ask from sysmon)): a new turn, starting at 0.206 ms. Two refreshes never interleave inside one world.

world	segment	start ms	end ms	what
weather	1	0.000	0.073	refresh
sensor	1	0.073	0.136	read
weather	2	0.136	0.206	refresh (resumed)
weather	3	0.206	running	refresh (second message)

applied at ms	effect
0.073	deliver note "refreshing" to log
0.073	deliver read (ask #1) to sensor
0.073	start the 2,000 ms timeout of ask #1
0.136	settle ask #1 for weather: 21.5

applied at ms	effect
0.206	deliver weather {temp: 21.5} to each subscriber
0.206	reply 21.5 to launcher's ask

12.5 Ordering

Programs depend on the order of messages far more often than their authors notice. A driver that is told "attach this framebuffer" and then "flush these rectangles" must see them in that order. PicoCalc OS's ordering guarantees follow from three facts in the code: deferred effects are applied in the order the segment created them, every event in the nucleus is ordered by its time and then by a sequence number assigned when it was created, and segments run one at a time, each ending strictly later than the one before, because every segment costs at least $c_0 = 40$ µs.

From these facts:

- **Messages from one segment arrive in the order they were sent.** They are applied in creation order and become delivery events with the same time and increasing sequence numbers.
- **Messages from one world to another arrive in the order they were sent**, across segments as well: a later segment ends later, so its deliveries happen later. This is the guarantee Erlang gives between a pair of processes.
- **All deliveries in the system are totally ordered** by the end time of the segment that sent them.
- **Delivery respects causality.** If a world sends message m_1 to world C, and something that happens because of a later message of the same segment leads, through any chain of handlers, to a message m_3 to C, then m_1 arrives first.

The last property is worth a moment, because it is the one distributed systems usually cannot provide. In Lamport's terms, the events of a system are partially ordered by "happened before": an event happens before another if they are in the same process in that order, or if the first is the sending of a message whose receipt leads to the second [Lamport 1978]. Messages can still arrive out of causal order in a distributed system, because they travel by different routes with different delays. Figure 12.1 shows why they cannot here. Every message is delivered at the end of the segment that sent it, and every segment that a message causes starts after that delivery. A chain of causes is therefore a chain of segments with increasing end times, and the message at the end of the chain is delivered after every message sent earlier in it.

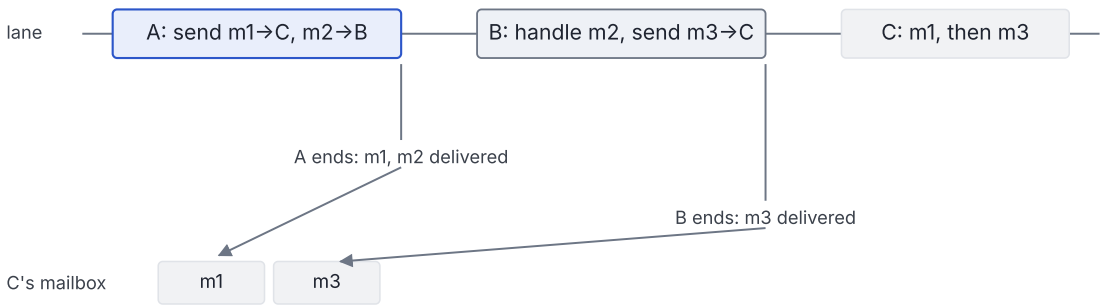


Figure 12.1. Causal order on one lane. World A's segment sends m1 to C and m2 to B; both are delivered when it ends. B's segment, which handles m2, can only start after that, so m3 reaches C after m1. With separate processors and separate network paths, m3 could overtake m1.

What the system does not guarantee is just as important. Promise settlements (a bus transfer finishing, an ask's reply arriving, a `sleep` ending) do not go through the mailbox; they wait in a separate queue and are taken even while the mailbox is closed, since they are what an asynchronous turn is waiting for. A world can therefore see a reply to its ask before a message that arrived earlier, because the message waits for the turn to end and the reply does not. Timer ticks are messages, but a tick is not queued if the previous tick of the same timer is still waiting, so a busy world sees fewer ticks rather than a backlog (Section 4.6.2). And the order in which different worlds *handle* their messages is decided by the lane's priorities (Chapter 4), not by the order of delivery: when the keyboard driver publishes a key event, the compositor (a service) and the Keys app (an app) receive it at the same instant, and the compositor handles it first.

12.6 Asks: replies, timeouts and failure

An ask is a send plus bookkeeping. The nucleus records the ask in a table, together with who asked, in which epoch (the asking world's restart count), and which promise to settle; it delivers the message, marked with the ask's number; and it starts a timer for the timeout:

▸ history/first-pass/nucleus/kernel.c

```
int kernel_ask(world_t *from, world_t *to, const char *type, const char *json, const int
*hs, int nh, int promise, vtime_t timeout) {
  if (to->status == W_STOPPED) {
    char t[160]; snprintf(t, sizeof t, "%s is stopped", to->name);
    char *j = json_string(t);
    defer_settle(from, promise, 0, j, 0);
    free(j);
    return 0;
  }
  // a slot in the table of waiting asks; one that is still waiting is never overwritten
  int id = 0;
  for (int tries = 0; tries < MAX_ASKS && !id; tries++) { int c = ++K.next_ask; if
(!K.asks[c % MAX_ASKS].live) id = c; }
  if (!id) {
    char *j = json_string("too many asks are waiting for replies");
    defer_settle(from, promise, 0, j, 0);
    free(j);
    return 0;
  }
  ask_t *a = &K.asks[id % MAX_ASKS];
  *a = (ask_t){ .id = id, .from = from->id, .from_epoch = from->epoch, .to = to->id,
.promise = promise, .live = true, .deadline = timeout };
  msg_t *m = new_msg(M_ON, type, json, from->id);
  m->ask = id;
  for (int k = 0; k < nh; k++) m->hs[m->nh++] = hs[k];
  defer_deliver(to, m);
  deferred_t *d = calloc(1, sizeof(*d));
  d->run = run_ask_timer; d->op = id; d->delay = timeout;
  defer(d);
  return id;
}
```

The ask is settled exactly once, by whichever comes first: the receiver's turn ending (with its return value, or a rejection), the receiver crashing, or the timer. Settling marks the entry dead, so the second arrival finds nothing to do:

▸ history/first-pass/nucleus/kernel.c

```
static void settle_ask(int id, int ok, const char *json, const int *hs, int nh) {
  ask_t *a = find_ask(id);
  if (!a) return;
  a->live = false;
  world_t *F = world_by_id(a->from);
  if (!F || F->epoch != a->from_epoch || !alive(F)) return;
  for (int i = 0; i < nh; i++)
    for (int k = 0; k < MAX_HANDLES; k++) if (K.handles[k].live && K.handles[k].id ==
hs[i]) handle_grant(&K.handles[k], F);
  deferred_t *d = calloc(1, sizeof(*d));
  d->run = run_settle; d->w = F->id; d->epoch = F->epoch; d->promise = a->promise; d->ok
= ok;
  d->json = ok ? xstrdup(json ? json : "null") : json_string(json ? json : "failed");
  defer(d);
}
```

The epoch check is the reason replies to dead worlds are harmless. If the asking world crashed and was restarted while its ask was outstanding, its epoch has changed; the reply belongs to a runtime that no longer exists, and the nucleus drops it rather than resolving a promise number that the new runtime may have reused.

Table 12.2 lists every way an ask can fail, with the message the asker's promise is rejected with. Each string is produced by the code shown in this chapter or in `vm.c`.

Table 12.2. How an ask fails, when the asker finds out, and what it is told.

Situation	When the asker finds out	The error
no world has that name	at once (the call throws)	there is no world called "X"
the world asks itself	at once (the call throws)	a world cannot ask itself (it would wait for its own turn to end)
a handle in the payload may not be passed	at once (the call throws)	handle N cannot be passed on by W
the receiver is stopped	when the asker's segment ends	X is stopped
the receiver has no handler for the type	when the receiver's turn runs; the receiver does not crash	X has no handler for "type"
the receiver crashes while handling it	when the receiver crashes	X crashed: (the reason)
no reply within the timeout	after the timeout (2,000 ms by default)	no reply from X within N ms
the asker crashed meanwhile	never (the reply is dropped)	(none)

The fault benchmark shows the failures in sequence. A world that loops forever on every message, with a 20 ms budget, was asked six times in a row, each ask waiting for the previous failure. The first four asks failed after 21, 221, 421 and 821 ms: 20.6 ms of looping each time, preceded by waits for restarts that grew from 0 to 200, 400 and 800 ms because the world had already crashed once before the experiment began. The fourth failure was its fifth crash within ten seconds, so the world was stopped, and the fifth and sixth asks failed with "spin20 is stopped" after 0.057 ms, the time for the asker's own segment to end and its continuation to run.

Two properties of timeouts surprise people who expect an ask to behave like a remote procedure call. First, a timeout does not withdraw the request. When the timer fires, the asker's promise is rejected, but the message stays in the receiver's mailbox and is handled when its turn comes; the reply then finds a dead entry and is dropped. A receiver that is slow because it is overloaded therefore stays overloaded after its callers have given up, working through requests nobody is waiting for. Second, a timeout cannot distinguish a slow receiver from a dead one, or a request that was never handled from one that was handled and whose reply

was late. An asker that retries after a timeout must assume that the first request may have taken effect. Requests that are safe to repeat (idempotent requests) are the cure, and Chapter 15 makes them one of the rules for restartable drivers.

The first pass changed the first property for an ask the receiver has not started: when the timer fires, the nucleus takes the message out of the receiver's mailbox, so it is never handled (*From L4 to Small Worlds*, Chapter 6). An ask the receiver has already started still runs to its end, and its reply is dropped. The pass also added three failures to Table 12.2: an ask to a world the asker's `asks` does not list throws at once, and an ask from a world whose `ask_slots` are all in use (`W already has 8 asks waiting for replies, its quota`) or to a full mailbox rejects when the asker's segment ends.

12.7 Handles in messages

A payload is JSON, but it can carry hardware authority. The prelude represents a handle in JSON as an object with a single key, `{"$h": 7}` (the `Handle` class's `toJSON` produces it), and the encoder collects the numbers of all such objects into a list that travels beside the text:

▸ js/prelude.js

```
  const revive = (key, v) => (v && typeof v === "object" && typeof v.$h === "number" &&
Object.keys(v).length === 1
    ? known.get(v.$h) || wrap(k("h.info", v.$h)) || new Handle({ id: v.$h, type: "gone",
rights: "" }) : v);
  function enc(v) {
    if (v === undefined) return [undefined, []];
    const hs = [];
    const json = JSON.stringify(v, (key, x) => {
      if (x && typeof x === "object" && typeof x.$h === "number" && Object.keys(x).length
=== 1) hs.push(x.$h);
      return x;
    });
    return [json, hs];
  }
  const decode = (json) => (json == null ? undefined : JSON.parse(json, revive));
```

The nucleus does not trust the list. At the send, `check_grants` verifies that the sender holds each handle and is allowed to pass it on (Chapter 9 gives the rule and its proof); at the delivery, the receiver is added to each handle's set of holders, as the `E_DELIVER` code above shows; and on the receiving side, the prelude turns each `{"$h": n}` back into a `Handle` object by asking the nucleus for the handle's description. A

number the receiver does not hold describes nothing, so the receiver gets a handle object of type "gone" that every operation will refuse. Forging a handle by writing `{"$h": 3}` into a payload therefore achieves nothing: the number is just a number unless the nucleus's table says the world holds it.

Three details of the mechanism matter in practice. A handle is granted at delivery, not at the send, so a sender whose segment crashes gives nothing away. The receiver gets the same handle, not a copy: both worlds now hold it, and a world that wants to give less than it has must narrow it first, as the compositor does when it gives the display driver a read-only view of its framebuffer:

▸ `js/system/compositor.js`

```
function attach(s) {
  // the view handed to a previous display instance died with it; let go of our hold on
it, or every
  // restart of the driver would keep one more handle alive
  if (s.view) s.view.release();
  s.view = s.fb.narrow("r");
  send("display", "attach", { fb: s.view });
  s.fb.invalidate();
  dirty(s);
}
```

And a message carries at most eight handles (`MAX_MSG_HANDLES`). The nucleus reads only the first eight numbers of the list, so a ninth handle in a payload is not granted, and the receiver sees it as "gone". A limit that silently drops authority is the kind of rule worth knowing before it bites.

12.8 What a message costs

The messaging benchmark measures each primitive on the virtual clock. A pair of worlds exchanges asks with payloads from 0 to 10,000 bytes, one world sends 10, 100 and 1,000 one-way messages to another, and a publisher posts to channels with 1, 4 and 8 subscribers.

Table 12.3. The cost of messages on the virtual clock (host build). An ask is measured from the call to the resumption of the asker; a send and a post until every receiver has handled the message.

Operation	Cost	What it consists of
a turn that does almost nothing	58 µs	one segment: 40 µs to enter and leave the world, the dispatch code
ask, empty payload	119 µs	the receiver's segment and the asker's resumption
ask, 100-byte payload	125 µs	plus 25 ns per byte when encoded and again when decoded
ask, 1,000 bytes	171 µs	
ask, 10,000 bytes	621 µs	
send (1,000 in a row)	64 µs per message	one receiving segment each, plus the sender's host call
publish, 1 subscriber	65 µs	one receiving segment
publish, 4 subscribers	222 µs	about 52 µs more per subscriber
publish, 8 subscribers	430 µs	

The cost model of Chapter 11 explains every row. An ask is two segments: the receiver handles the message and returns, and the asker's continuation runs when the reply settles its promise. Each segment pays the 40 µs entry cost c_0 plus the steps of the prelude's dispatch code, about 60 µs in all. The payload is charged 25 ns per byte when the sender encodes it and again when the receiver decodes it, which is why 10,000 bytes add half a millisecond. A channel post costs one segment per subscriber, because each subscriber handles its own copy. The lane can run about 17,000 trivial turns per second.

The numbers put PicoCalc's IPC in a different regime from a microkernel's. L4's one-way IPC took 250 cycles on a 1993 processor. A PicoCalc message takes about 60 µs, roughly 24,000 cycles of a 400 MHz core, and almost all of it is the cost of entering a JavaScript runtime and running the prelude's dispatch, not of moving bytes. Two cautions apply to the comparison: c_0 is an estimate rather than a measurement on the device (Chapter 18), and an L4 IPC transfers control between address spaces protected by hardware, while a PicoCalc message transfers control between runtimes protected by the language. The lesson that survives

both cautions is that a protocol's cost on this system is dominated by the number of segments it needs. A design that halves the number of messages halves the cost; a design that halves the size of each message changes almost nothing until the payloads reach kilobytes.

12.9 Following one key through the system's messages

The trace below was recorded by the host build while a short expression was evaluated at the REPL. It covers the 25 ms after Enter was pressed, and every message, segment and transfer in it can be read with the rules of this chapter. Section 3.8 reads a similar trace for the turn structure of a long evaluation; here the subject is the messages.

▸ bench/traces/repl.jsonl


```
{"k":"seg","w":"keyboard","t":5633087,"dt":75,"steps":49,"native":3,"kind":"settle","type
":"every 20 ms"}
{"k":"msg","t":5633162,"from":"keyboard","to":"compositor","type":"keys","ask":false,"byt
es":61}
{"k":"msg","t":5633162,"from":"keyboard","to":"keys","type":"keys","ask":false,"bytes":61
}
{"k":"seg","w":"compositor","t":5633162,"dt":80,"steps":56,"native":3,"kind":"msg","type"
:"keys"}
{"k":"msg","t":5633242,"from":"compositor","to":"repl","type":"key","ask":false,"bytes":6
1}
{"k":"seg","w":"keys","t":5633242,"dt":67,"steps":38,"native":2,"kind":"msg","type":"keys
"}
{"k":"seg","w":"repl","t":5633309,"dt":118,"steps":98,"native":13,"kind":"msg","type":"ke
y"}
{"k":"msg","t":5633427,"from":"repl","to":"sandbox","type":"eval","ask":true,"bytes":56}
{"k":"msg","t":5633427,"from":"repl","to":"compositor","type":"frames","ask":false,"bytes
":429}
{"k":"seg","w":"compositor","t":5633427,"dt":127,"steps":115,"native":11,"kind":"msg","ty
pe":"frames"}
{"k":"msg","t":5633554,"from":"compositor","to":"compositor","type":"draw","ask":false,"b
ytes":0}
{"k":"seg","w":"compositor","t":5633554,"dt":1064,"steps":859,"native":453,"kind":"msg","
type":"draw"}
{"k":"msg","t":5634618,"from":"compositor","to":"display","type":"flush","ask":true,"byte
s":58}
{"k":"seg","w":"display","t":5634618,"dt":212,"steps":204,"native":37,"kind":"msg","type"
:"flush"}
…
{"k":"seg","w":"sandbox","t":5634830,"dt":85,"steps":64,"native":3,"kind":"msg","type":"e
val"}
…
{"k":"seg","w":"repl","t":5635038,"dt":105,"steps":80,"native":12,"kind":"settle","type":
"key"}
{"k":"msg","t":5635143,"from":"repl","to":"compositor","type":"frames","ask":false,"bytes
":468}
…
{"k":"seg","w":"display","t":5638710,"dt":63,"steps":31,"native":3,"kind":"settle","type"
:"flush"}
{"k":"seg","w":"compositor","t":5638773,"dt":63,"steps":34,"native":1,"kind":"settle","ty
pe":"draw"}
{"k":"seg","w":"compositor","t":5638836,"dt":134,"steps":124,"native":12,"kind":"msg","ty
pe":"frames"}
```

Read in order (times in virtual ms):

1. **One post, two deliveries.** The keyboard driver's last segment publishes the key event on `keys`. Both subscribers, the compositor and the Keys app, receive it at 5,633.162, the instant the driver's segment ends. The compositor runs first because it is a service and the Keys app is an app.
2. **A send.** The compositor routes the key to the app in front, the REPL, with `send`; the REPL's copy arrives at 5,633.242.
3. **An ask and a frame, at one instant.** The REPL's handler asks the sandbox to evaluate the line, and its screen changes (it now shows that it is busy), so the kernel publishes its frame. Both effects are applied when the REPL's segment ends, at 5,633.427.
4. **A message to oneself.** The compositor stores the frame and marks the screen dirty, which sends itself a `draw` message. The draw is one 1.064 ms segment, and it ends by asking the display driver to flush the changed rectangles.
5. **Priority decides who runs, not when messages arrive.** The sandbox has had the REPL's ask in its mailbox since 5,633.427, but it runs at priority 0, so the compositor's draw and the display driver's flush go first. The eval segment itself takes 85 µs, and the REPL's continuation runs at 5,635.038: the ask took 1.6 ms from send to resumption, against 119 µs on an idle lane, and almost all of the difference is waiting behind higher-priority worlds.
6. **A closed mailbox.** The REPL's second frame, showing the result, reaches the compositor at 5,635.143. But the compositor is in the middle of an asynchronous turn: its `draw` handler is awaiting the display driver's answer to the flush. The frame waits in the mailbox until the flush completes and the draw turn ends at 5,638.836, 3.7 ms later, and only then does the compositor handle it.

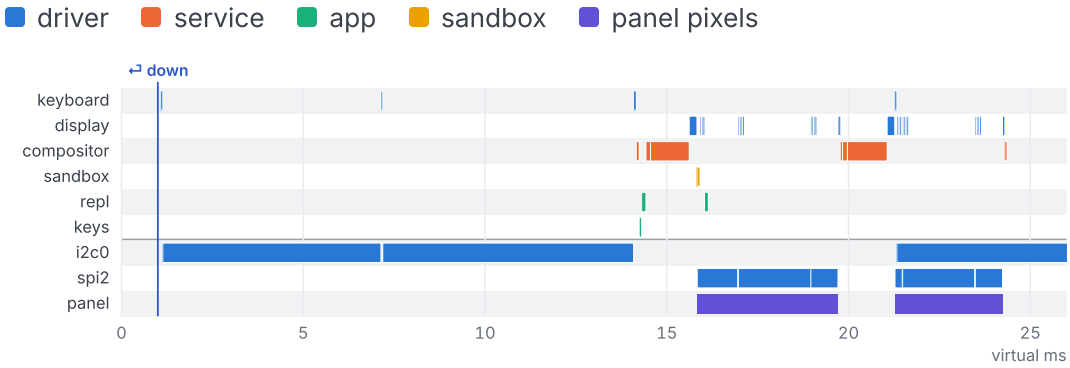
TRY 12.2

Find the messages in the timeline

The same 25 ms on a timeline: one row per world, then the buses and the panel. Hover the bars to read each segment and transfer, or switch to the event list to see the messages in order. Find the two deliveries of the key event, the REPL's ask and the sandbox's late segment, and the frame that waits in the compositor's mailbox while the SPI transfers of the previous flush run.

Enter at the REPL 26 virtual ms

Enter is pressed at 5,620 ms in the REPL on a short expression; the key travels as a post, a send, an ask and two frames.



12.10 Designing protocols between worlds

A protocol is an agreement between worlds about which messages to send, in what order, and what to do when something goes wrong. The compositor and the display driver are the most demanding pair in the system, because they must keep the panel correct across each other's crashes. Their protocol shows five patterns that apply to any pair of worlds.

Coalesce work that can be repeated. The compositor does not redraw for every frame it receives. The first change marks the screen dirty by sending the compositor a `draw` message, and later changes only update the stored trees until that message is handled:

▸ history/first-pass/js/system/compositor.js

```
function dirty(s) {
  if (s.queued) return;
  s.queued = true;
  send(self, "draw");
}
```

Because the `draw` message joins the back of the compositor's own mailbox, every frame that arrived before it has been stored by the time the draw runs. Ten frames in quick succession cost one redraw.

Ask when you need the outcome, and give the ask a timeout you can act on. The draw handler asks the display driver to flush the changed rectangles, with a 1.5 s timeout, and handles both an explicit refusal and a failure:

▸ js/system/compositor.js

```
    try {
      const r = await ask("display", "flush", { rects }, { timeout: 1500 });
      if (r.skipped) {   // the panel did not get these pixels: send everything again
        s.fb.invalidate();
        if (!s.resent) { s.resent = true; dirty(s); }
      } else s.resent = false;
      s.flushError = "";
    } catch (e) { s.flushError = e.message; s.fb.invalidate(); }
```

On either path the compositor marks its whole framebuffer as changed (`invalidate`), so the next flush sends every pixel. That makes the flush **idempotent at the level of the screen**: whatever happened to the last flush, the next successful one leaves the panel showing exactly what the compositor drew.

Expect messages addressed to a previous instance. A mailbox survives its world's restart, so the new instance can receive messages that only made sense to the old one. PicoCalc met this in the simulator: a flush sent just before the display driver was killed waited through the restart, and the new driver, which had not yet been given the framebuffer, threw while handling it and crashed a second time. The fix was in the driver, which now answers such a request instead of failing:

▸ js/drivers/display.js

```
    // after a restart, flushes queued for the old instance can arrive before the
compositor re-attaches
    if (!s.fb || !s.ready) return { px: 0, skipped: true };
```

Let the client rebuild the server's state. The display driver keeps no state that it cannot get again: it holds the SPI handle (which it reopens at start) and a read-only view of the framebuffer (which the compositor gives it). When the driver restarts, the compositor learns of it from `worlds.status` and repeats the attachment, so the new instance gets everything it needs from the one world that has it:

▸ js/system/compositor.js

```
    } else if (st.state === "running") {
      s.banner = { text: `${st.world} is running again (restart ${st.restarts})`, color:
C.ok, until: now() + 1500 };
      if (st.world === "display") attach(s);   // a fresh panel: give it the framebuffer
and send every pixel
    }
```

Keep turns short where others are waiting. Trace step 6 above showed the price of the compositor's asynchronous draw: while it awaits a flush, which can take 41 ms for a full screen at 40 MHz, it handles no keys and no frames. The protocol accepts this because the flush is what the user is waiting for anyway. A world whose callers need quick answers should not await slow work inside the turn that answers them; it can reply at once and send the result later, or hand the slow work to another world.

These patterns are the beginning of the design rules for self-healing drivers in Chapter 15, which treats them systematically: where state lives, how requests are made safe to repeat, and how authority is handed back to a restarted driver.

KEY POINTS

- Worlds cannot share objects, because each has its own heap and garbage collector; they exchange JSON copies, so a receiver never sees a sender's later changes.
- `send` is one-way, `ask` returns a promise of the handler's return value, and `publish` delivers a copy to every subscriber of a channel; all three arrive in the receiver's mailbox and are handled in turns.

- Every message is delivered at the end of the sender's segment, so a crash recalls all of a segment's messages, and nothing is visible before its sender has finished.
- On one lane, messages between two worlds arrive in send order, all deliveries are totally ordered, and delivery respects causality; promise settlements, which bypass the mailbox, are the exception.
- An ask can fail in seven ways, each with its own error message, and its reply is dropped if the asker has crashed meanwhile; a timeout rejects the asker's promise but does not withdraw the request from the receiver's mailbox (since the first pass, it does when the receiver has not yet started it).
- Handles ride in payloads as `{"$h": n}`, are checked at the send and granted at delivery, and a number the receiver does not hold is worthless.
- A message costs about 60 µs on the virtual clock, almost all of it entering a runtime; protocols should be judged by the number of segments they need.
- Protocols that survive restarts coalesce repeatable work, make requests idempotent, tolerate messages meant for a previous instance, and let the client rebuild the server's state.

Messages connect worlds that cannot otherwise touch. The most important messages in the system are the ones that reach hardware: the keyboard driver's polls, the display driver's flushes and the handles that let them happen. The next chapter turns to device drivers, the worlds that translate between these messages and the registers of real chips.

EXERCISES

1. BASIC A world's handler calls `send("b", "x")`, then `send("c", "y")`, then throws. Which messages are delivered? What if the throw happens after an `await sleep(1)` between the two sends?
2. BASIC Using Table 12.3, estimate the virtual time for one world to ask each of four others a question with a 200-byte payload, one after another, with the lane otherwise idle. How much would asking all four at once (with `Promise.all`) save, and why?
3. ANALYSIS Prove that messages from world A to world B arrive in the order A sent them, using only the three facts listed at the start of Section 12.5. Which of the facts would you have to give up to lose the guarantee?
4. ANALYSIS Construct a sequence of events in which a world receives the reply to its ask before a message that was delivered to its mailbox earlier. Is this a violation of causal order? Explain.

5. ANALYSIS In trace step 5, the REPL's ask took 1.6 ms instead of 119 µs. Which segments did the sandbox wait behind, and how long would the ask have taken if the sandbox had priority 2?

6. CODE Write two worlds for a copy of `js/system.json`: a client that asks a server with a 100 ms timeout, and a server whose handler awaits `sleep(500)`. Show from the UART log which requests the server still handles after the client has given up, and that their replies are dropped; explain why the requests that timed out before the server started them are not handled (the first pass's change, Section 12.6).

7. CODE Send a payload containing nine handles from the compositor to a test world and print what the receiver gets for each. Then change `MAX_MSG_HANDLES` handling so that a send with too many handles fails instead of dropping some.

8. CODE Measure an ask's round trip with a 100,000-byte payload. Does it follow the 25 ns per byte model? What limits the largest payload you can send?

9. DESIGN Design a request-reply protocol between the compositor and a new audio driver that survives either side's restart. State what each side keeps, what it sends after the other's restart, and which requests must be idempotent.

10. DESIGN Channel declarations (`.provides`, `.needs`) are made by the world's own code. Change the manifest format and the nucleus so that channels are declared there instead, and the nucleus refuses a publish on an undeclared channel. Which worlds in the shipped manifest need which channels? (The first pass made this change; compare your design with the one in *From L4 to Small Worlds*, Chapter 5.)

11. DISCUSSION L4 chose synchronous IPC for speed; Erlang chose asynchronous messages for isolation. Which arguments from each apply to PicoCalc OS, given that its cost per message is dominated by entering a runtime rather than by buffering?

12. PROJECT Add promise pipelining in the style of E: let a world send a message to the eventual result of an ask before the ask has been answered, so that a chain of dependent asks to the same world costs one round trip. Measure the saving for a chain of three asks.

FURTHER READING

[Accetta et al. 1986] The paper that made messages and ports the centre of a kernel; read the definitions of ports, messages and port rights.

[Liedtke 1993] How far careful design can push synchronous IPC; [Elphinstone and Heiser 2013] looks back twenty years later at which of those choices survived.

[Lauer and Needham 1979] The argument that message-oriented and procedure-oriented systems are duals.

[Armstrong 2003] Why isolated processes need asynchronous messages, from the designer of Erlang.

[Lamport 1978] Happened-before and logical clocks: the vocabulary for reasoning about message order.

Device drivers

A device driver is the part of an operating system that knows how one kind of hardware works. Everything above it deals in abstractions (key events, battery levels, rectangles of pixels); the driver turns requests in those terms into the register reads and writes, bus transactions and waits that the device understands, and turns the device's answers back into events. This chapter explains what a driver is responsible for, surveys how Linux, Windows, Fuchsia, Zephyr and macOS organise drivers, and then reads PicoCalc OS's three drivers and its display pipeline line by line. By the end you should be able to write a driver for a new peripheral as a PicoCalc world, explain why two drivers sharing one chip need atomic transactions, follow a frame from an app's state to the glass, and say what a register-level model of a device can and cannot tell you about a driver before the hardware exists.

13.1 What a driver is responsible for

A peripheral is a small machine with its own state. The display controller of the PicoCalc, the ST7365P, is asleep or awake, has its display output on or off, has a pixel format, a memory access order and a write window, and holds 320 × 480 pixels in its own memory; the STM32 that scans the keyboard keeps a FIFO of key events, two backlight levels and a battery reading (Chapter 2 reads both datasheets). A driver's first job is to know that state machine and to keep the device in states the rest of the system can use. Figure 13.1 shows the part of the panel's state machine its driver must navigate.

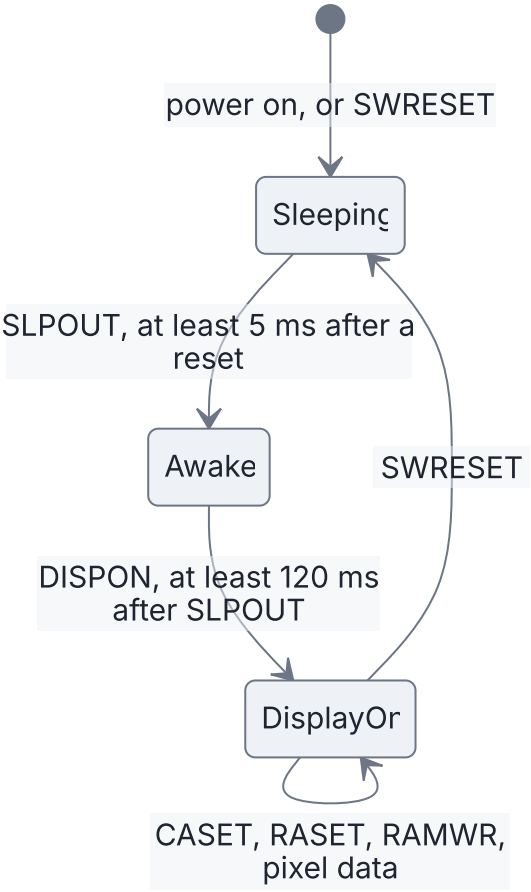


Figure 13.1. The display controller's power states as its driver sees them. Each transition is a command; two of them must be followed by a wait before the next command, which the driver, not the hardware, has to enforce.

The full list of a driver's responsibilities is longer, and it is the same whether the driver is in a kernel or in a JavaScript world:

- **Binding.** Decide which device instance the driver controls, and claim it so that no other driver does.
- **Initialisation.** Bring the device from whatever state it powered up in to a known working state, observing the delays its datasheet requires.

- **Translation.** Turn each request from a client into the device's commands, and the device's answers into results.
- **Completion.** Find out when the device has finished, by an interrupt or by polling, and tell the client.
- **Errors and recovery.** Notice a device that does not answer or answers nonsense, and bring it back to a known state.
- **Power.** Put the device into low-power states when it is idle and wake it when it is needed.
- **Sharing.** Arbitrate between several clients of one device, and between drivers that share a bus or a chip.
- **Teardown.** Leave the device in a safe state when the driver stops.

The proportions are not what the word "driver" suggests. Kadav and Swift studied all of Linux's drivers and found that initialisation and cleanup make up almost 36% of driver code on average, configuration 15% and power management 7.4%; the code that moves data, which is what most people picture, is the smaller part [Kadav and Swift 2012]. The same paper notes that device drivers make up 70% of the Linux code base. Drivers are most of an operating system's code, and most of a driver is about getting the device into the right state and keeping it there. That is also why drivers fail so often and why Chapter 15 is about recovering them.

13.2 How operating systems structure drivers

Every operating system answers the same questions about drivers: where a driver's code runs, how it is matched to a device, how clients reach it, and what is protected from it. The answers differ widely.

Linux runs most drivers in the kernel. Its driver model has three kinds of object: buses, devices and drivers. When a device appears on a bus, the bus driver compares it with the identifiers each driver supports, through a match callback the bus provides; on a match, the driver's probe callback runs, in which "the driver initializes the device and allocates and initializes per-device data structures". When the device goes away, "the remove callback of the driver is called" [Linux driver model docs 2026]. Clients reach the driver through the kernel's subsystems (input, framebuffer, network) and system calls. The driver shares the kernel's address space, so a bug in it is a bug in the kernel [Corbet et al. 2005].

Windows offers two frameworks. Kernel-mode drivers run in the kernel, as in Linux; drivers written with the User-Mode Driver Framework (UMDF) "run in the user-mode environment", in a driver host process that "provides an execution environment for user-mode drivers, and routes messages between drivers in a user-mode stack", and every host process is managed by a driver manager service [Microsoft UMDF docs 2026]. Not every device can have a user-mode driver: "file system drivers, display drivers (for full display devices, not display-only display devices), and print drivers cannot be UMDF drivers".

Fuchsia, built on the Zircon microkernel, runs every driver in user space. "Every driver lives in a driver host", and "each driver host is a process, meaning it has its own address space and manages its own set of threads". A driver manager "starts and stops drivers in a Fuchsia system and routes FIDL capabilities for the drivers"; drivers are matched to devices by bind rules, and "most of the communication occurs using FIDL calls over Zircon channels" [Fuchsia docs 2026]. A driver host isolates its drivers from the rest of the system, though drivers in the same host share an address space.

Zephyr, a real-time operating system for microcontrollers, links drivers into the single application image. Each device is an object with three parts: a read-only configuration ("the config member is for read-only configuration data set at build time"), mutable runtime data, and an API structure that "maps generic subsystem APIs to the device-specific implementations in the driver". Devices are defined at build time and initialised at boot in levels, from `PRE_KERNEL_1` for devices with no dependencies to `APPLICATION` [Zephyr docs 2021]. Drivers are compiled into the same image as the kernel and nothing isolates one from the rest of it; there is also no dynamic binding, because the board's device tree is known at build time.

User-space drivers on Unix. Linux's Userspace I/O (UIO) lets most of a driver live in a process: "only a very small kernel module is needed", the device's memory is mapped through `/dev/uioX`, and "interrupts are handled by reading from /dev/uioX" [Linux UIO docs 2026]. VFIO goes further, "an IOMMU/device agnostic framework for exposing direct device access to userspace" in "a secure, IOMMU protected environment", so that the device itself cannot use DMA to write outside the memory the process was given [Linux VFIO docs 2026]. Apple's DriverKit moves macOS drivers out of the kernel: "the drivers you build with DriverKit run in user space, rather than as kernel extensions, which improves system stability and security" [Apple DriverKit docs 2026].

Table 13.1. Where drivers run and how they are bound to devices in six systems.

System	Where a driver runs	How it is bound to a device	Isolated from the rest of the system by
Linux	in the kernel	the bus's match callback, then the driver's probe	nothing
Windows (UMDF)	a driver host process	per device stack; a device's drivers share one host	the process boundary
Fuchsia	a driver host process	bind rules evaluated by the driver index	the process boundary (per host)
Zephyr	the application image	statically, from the device tree, initialised by level	nothing
Linux (UIO, VFIO)	a user process, with a kernel stub (UIO)	the stub module, or VFIO's device groups	the process boundary, and the IOMMU (VFIO)
PicoCalc OS	a world: its own QuickJS runtime	statically, by claims in the manifest	the language, budgets and supervision

The trend is plain. Drivers have moved out of kernels and into processes, at the cost of a message for every request, because a driver in its own process can crash without taking the system with it and can be restarted. Chapter 15 follows that line of work in detail. PicoCalc OS sits at the end of the trend on a machine with no memory protection between components: its drivers are separate runtimes, isolated by the language rather than by the processor, and restarted by the same supervisor as every other world.

13.3 PicoCalc OS's driver model

In PicoCalc OS a driver is a world of kind `driver`. That gives it four properties that the nucleus enforces. It runs at the highest priority, 3, so that when the lane comes free a driver goes first. It has a tight budget, 20 or 30 ms against 50 ms for apps, because a driver's handler that runs long is almost certainly wrong. It has a small heap, 320 KB. And it can reach hardware only through handles obtained by the claims its manifest entry lists:

▸ history/first-pass/js/system.json

```
    { "name": "keyboard",   "file": "drivers/keyboard.js",  "claims": ["i2c0:0x1f"],
"budget_ms": 20,  "mem_kb": 320,
      "provides": ["keys"] },
    { "name": "power",      "file": "drivers/power.js",     "claims": ["i2c0:0x1f"],
"budget_ms": 20,  "mem_kb": 320,
      "provides": ["power"] },
    { "name": "display",    "file": "drivers/display.js",   "claims": ["spi2"],
"budget_ms": 30,  "mem_kb": 320 },
```

Binding is static. The PicoCalc's hardware does not change, so there is no discovery: the manifest says which world claims which device, and the world opens it by name (`hw.i2c("i2c0:0x1f", 10000)` , `hw.spi("spi2", 40e6)`). The nucleus checks the claim, creates a handle and returns it (Chapter 9).

A driver's life has three phases, each a kind of handler. Its `onStart` handler initialises the device, and runs again from the beginning whenever the world is restarted, so initialisation is also recovery. Its `every` handlers poll. Its `on` handlers serve requests from other worlds. There are no interrupt handlers: the host build models no interrupt lines, and the keyboard controller has none wired to the host (Chapter 2), so a driver learns about the device either by asking it periodically or by being asked to do something.

Every bus operation is asynchronous. `dev.read(reg, n, gap)` on an I2C handle or `spi.cmd(c, data)` on an SPI handle returns a promise. The nucleus records the transfer as an effect of the current segment; the transfer starts when the segment has ended and the bus is free, takes the time its bits take on the wire, and settles the promise when it is done. Figure 13.2 shows the layers a transfer passes through.

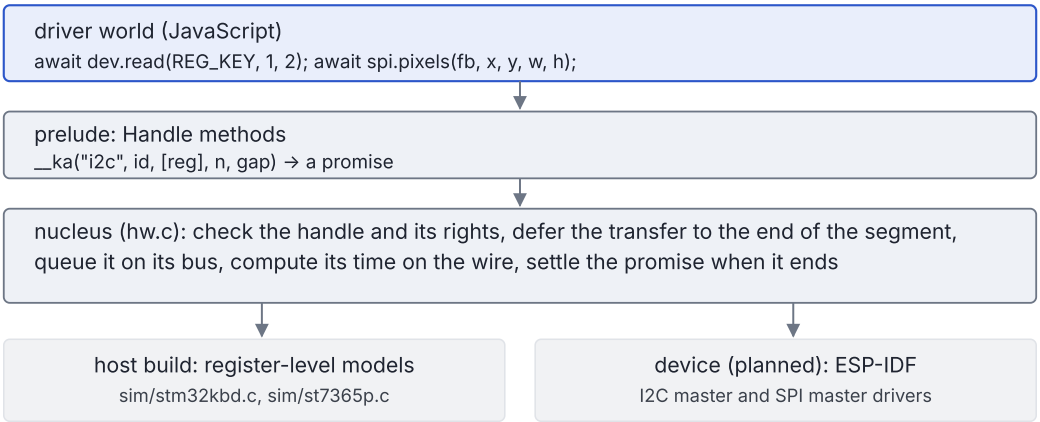


Figure 13.2. The layers under a PicoCalc driver. The driver's code calls methods on handle objects; the prelude turns them into host calls; the nucleus checks the handle, queues the transfer on its bus and computes its duration; the board carries it out. On the host build the board is a pair of register-level models; on the device it will be ESP-IDF's I2C and SPI drivers (planned).

The nucleus's part is small. An I2C transaction is a write phase and a read phase, each costing nine clocks per byte (eight data bits and an acknowledge [NXP 2021]) plus the address byte and the start and stop conditions, with an optional gap between them during which the bus stays held:

▸ history/first-pass/nucleus/hw.c

```
static void run_i2c(deferred_t *d, vtime_t at) {
  handle_t *h = find(d->handle);
  vtime_t start = bus_start(BUS_I2C0, at);
  // a write phase (address + bytes, 9 clocks each), an optional gap with the bus held,
then a read phase
  double bits = (d->nbytes ? (1 + d->nbytes) * 9 + 2 : 0) + (d->nread ? (1 + d->nread) *
9 + 2 : 0);
  vtime_t end = start + wire_us(bits, h ? h->hz : 10000) + d->delay;
```

`bus_start` returns the later of the segment's end and the time the bus becomes free, so transfers on one bus run one after another in the order they were requested. Nothing in this layer knows what the bytes mean. That knowledge lives in the driver, in JavaScript, where it can be read, changed and restarted like any other world's code.

13.4 The keyboard driver

The keyboard driver is 45 lines. Its device is the STM32 microcontroller that scans the keyboard matrix and answers on the I2C bus at address 0x1F; Chapter 2 describes its register protocol, in which a read is a write of the register number, a wait while the controller prepares its answer, and a read of two bytes. The driver's whole job is to empty the controller's FIFO of key events every 20 ms and publish each event:

▸ history/first-pass/js/drivers/keyboard.js

```
const readReg = (dev, reg, n) => dev.read(reg, n, 2);

driver("keyboard")
  .provides("keys")
  .state({ polls: 0, events: 0, errors: 0, lastError: "" })
  .onStart(async (s) => {
    s.dev = hw.i2c("i2c0:0x1f", 10000);  // the controller runs its I2C at 10 kHz
    await sleep(100);                     // and wants 100 ms after reset before the
first access
  })
  .every(20, async (s) => {
    if (!s.dev) return;
    s.polls++;
    try {
      const [status] = await readReg(s.dev, REG_KEY, 1);
      let count = status & 0x1f;
      while (count-- > 0) {
        const [state, code] = await readReg(s.dev, REG_FIF, 2);
        if (!state) break;
        const key = NAMES[code] || (code >= 0x20 && code < 0x7f ?
String.fromCharCode(code) : "0x" + code.toString(16));
        const ch = code >= 0x20 && code < 0x7f ? String.fromCharCode(code) : "";
        s.events++;
        publish("keys", { code, key, ch, state: STATES[state] || String(state), caps: !!
(status & 0x20) });
      }
    } catch (e) {
      s.errors++;
      if (e.message !== s.lastError) log("i2c:", e.message);
      s.lastError = e.message;
    }
  })
```

Three choices in this code are worth naming. The driver does not trust the count: if a FIFO read returns state 0 (an empty slot), it stops rather than publishing garbage. It does not crash on a bus error. A failed transfer rejects its promise, the `catch` counts it and logs it once per distinct message, and the next poll tries again, because a transient bus error is not a reason to restart the world. And it holds no state that matters: the event counters are for the system monitor, and the device handle is reopened by `onStart`. A restarted keyboard driver is exactly as good as the old one. In the fault experiment that kills it, a key pressed at the moment of the kill is not lost: it waits in the controller's FIFO and reaches the app 213 ms later, after 100 ms of backoff, the new instance's 100 ms start-up wait and one poll (Chapter 15).

The poll has a cost that the keyboard's users pay in latency and the bus pays in occupancy. At 10 kHz the status read holds the bus for 6.0 ms (one byte read, with the 2 ms gap) and each event read for 6.9 ms (two bytes), so one key event costs 12.9 ms of bus time, and polling every 20 ms keeps the bus 30% busy when no key is pressed. A key press waits on average half a poll period before the driver looks. The polling benchmark measured the trade on the host build: halving the period to 10 ms took the median key-to-pixel latency in the launcher from 30.9 to 24.8 ms and the idle bus from 30% to 60% busy; at 5 ms the reads no longer fit in the period, ticks are coalesced and the bus is 98% busy. A faster bus helps less than expected, because the controller's 2 ms gap dominates: at 100 kHz and 400 kHz the status read still takes 2.4 and 2.1 ms. The best configuration measured, a 5 ms poll at 400 kHz, had a median latency of 13.7 ms with the bus 42% busy; whether the STM32's firmware accepts that clock is a question for the hardware.

TRY 13.1

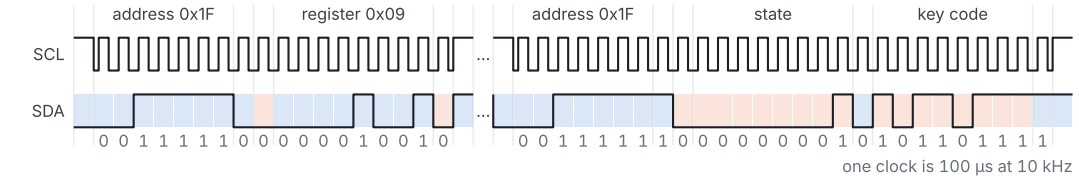
Price a keyboard poll

The calculator below uses the nucleus's timing rule for the bus. Its initial setting is the keyboard driver's FIFO read: one byte written, the controller's 2 ms gap with the bus held, two bytes read, at 10 kHz, which comes to 6.90 ms. Set one byte read to price the status poll (6.0 ms). Then raise the clock to 100 and 400 kHz and watch how little the gap lets you save.

6.90 ms

`(1 + 1)·9 + 2 + (1 + 2)·9 + 2 = 49 clocks at 10 kHz = 4.90 ms, plus 2 ms held`

driven by the processor driven by the keyboard controller



13.5 Two drivers, one chip

The STM32 also holds the battery level and the screen backlight, so the power driver uses the same chip at the same address. Two drivers sharing one device is common (a power-management chip, a sensor hub, a codec with control and data interfaces), and it is where the first bug the simulator found came from.

The first version of the keyboard driver read a register the way the reference C driver does, as two separate transfers: write the register number, wait 2 ms, read the answer. The power driver did the same. On the first boot of the full system in the simulator, the keyboard driver reported a key `R` in state 11, and the battery read 0%. Figure 13.3 shows what had happened. The two drivers' transfers had interleaved on the bus, and because the STM32 answers a read with whatever the last register write prepared, each driver got the other's answer.

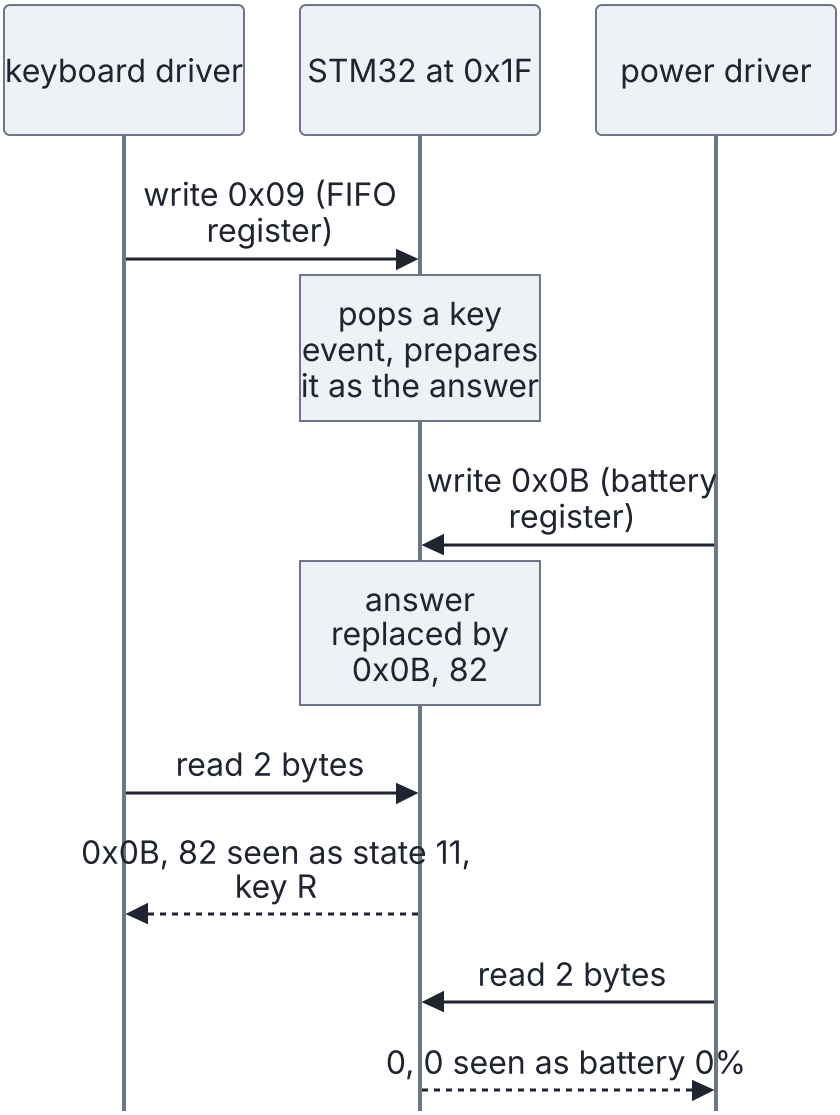


Figure 13.3. How two drivers' register reads interleaved. The keyboard driver's read collects the answer to the power driver's write: 0x0B (11) is the battery register's number and 82 is the battery level, which is also the ASCII code for R. The key event the keyboard driver's write had popped from the FIFO is lost.

Neither driver was wrong on its own. Each issued a correct sequence; the sequence was simply not atomic with respect to the other driver's. There are two general remedies for a device with more than one user. One is a single owner: one driver talks to the chip and the others ask it, which serialises access at the cost of an

extra world and a message per request. The other is an atomic transaction: make the write, the wait and the read one bus operation that nothing else can come between. I2C itself provides for this, since a controller can follow a write with a read without releasing the bus by using a repeated START condition [NXP 2021].

PicoCalc OS took the second remedy, in the nucleus's API rather than in either driver. `dev.read(reg, n, gapMs)` writes the register number, holds the bus for `gapMs` and then reads, all as one transfer; `run_i2c` above adds the gap (`d->delay`) to the transaction's time on the wire, and `bus_start` keeps any other transfer off the bus until it ends. Both drivers now read registers with a 2 ms gap inside the transaction. The fix is in the right layer: a third driver for the same chip gets it for free, and no driver has to know which others exist.

The bug also says something about where it was found. On the device, the same interleaving would have happened whenever the two drivers' polls coincided, and the symptom (an occasional phantom key, a battery that occasionally reads empty) would have looked like noise. In the simulator it appeared at the first boot of the full system and, because the virtual clock is deterministic, at every boot after that until it was fixed.

13.6 The power driver

The power driver is the keyboard driver's neighbour on the chip and the simplest of the three. It reads the backlight and battery registers at start, samples the battery every 5 s (one 6.9 ms transaction) and publishes a `power` event when the level or the charging state changes; it serves `backlight` and `screen` requests from other worlds by writing the backlight register and reading it back:

▸ history/first-pass/js/drivers/power.js

```
async function setBacklight(s, level) {
  await s.dev.transfer([REG_BKL | 0x80, level], 0);
  const [, v] = await readReg(s.dev, REG_BKL);
  s.backlight = v;
  publish("power", { pct: s.pct, charging: s.charging, backlight: s.backlight, screen:
s.screen });
}
```

Reading back the value it has just written is a small instance of a general rule: a driver should report the device's state, not its own belief about it. If the controller clamped or ignored the write, the published state says so.

13.7 The display driver

The display driver owns the SPI bus to the ST7365P. Its start-up handler is the reference C driver's initialisation sequence, transcribed into JavaScript:

▸ js/drivers/display.js

```
  .onStart(async (s) => {
    s.spi = hw.spi("spi2", 40e6);   // 40 MHz: 80 MHz worked once but ghosted on the
removable display
    await s.spi.cmd(SWRESET);
    await sleep(150);
    await s.spi.cmd(SLPOUT);
    await sleep(150);
    await s.spi.cmd(COLMOD, [0x55]);  // 16 bits per pixel, RGB565
    await sleep(10);
    await s.spi.cmd(MADCTL, [0x48]);  // column order mirrored (MX), BGR panel
    await s.spi.cmd(INVON);           // this panel shows true colours with inversion on
    await sleep(10);
    await s.spi.cmd(DISPON);
    await sleep(50);
    s.ready = true;
  })
```

The waits add up to 370 ms, against the 5 ms after reset and 120 ms between sleep-out and display-on that the controller requires (Chapter 2 tabulates each command and its timing). This is why the panel stays dark for most of the first half second after boot, and why the screen is whole again 515 ms after the display driver is killed, while a key reaches its app 213 ms after the keyboard driver is.

The driver does not own the pixels. The compositor allocates the framebuffer and gives the driver a read-only view of it (the `attach` message of Section 12.7); the driver owns only the bus and the knowledge of how to address the panel. When the compositor asks it to flush a list of changed rectangles, it queues, for each rectangle, the column range, the row range, the memory-write command and the pixels, and then awaits them all:

▸ js/drivers/display.js

```
  .on("flush", async (s, { rects }) => {
    // after a restart, flushes queued for the old instance can arrive before the
compositor re-attaches
    if (!s.fb || !s.ready) return { px: 0, skipped: true };
    const t0 = now();
    const jobs = [];
    let px = 0;
    for (const [x, y, w, h] of rects) {
      // queued together: the transfers run back to back on the bus once this turn ends
      jobs.push(s.spi.cmd(CASET, span(x, x + w - 1)), s.spi.cmd(RASET, span(y, y + h -
1)), s.spi.cmd(RAMWR), s.spi.pixels(s.fb, x, y, w, h));
      px += w * h;
    }
    await Promise.all(jobs);
```

Queueing everything before the first `await` matters. All the transfers become effects of one segment and start back to back when it ends, so the bus never waits for the lane. Had the driver awaited each command before issuing the next, every rectangle would have cost four extra segments and four gaps on the bus.

The pixels go out in chunks the size of a DMA transfer, each charged a fixed setup time on top of the time its bits take:

▸ nucleus/hw.c

```
  enum { CHUNK = 32768 };
  int chunks = 0;
  vtime_t t = start;
  for (size_t off = 0; off < n; off += CHUNK) {
    size_t m = n - off < CHUNK ? n - off : CHUNK;
    st7365p_data(px + off, (int)m, t);
    vtime_t t1 = t + wire_us(m * 8.0, hz) + PROFILE.spi_chunk_us;
    int changed = st7365p_take_changes();
    if (changed) photon(t, t1, changed);
    t = t1;
    chunks++;
  }
```

A full 320 × 320 frame is 204,800 bytes, seven chunks, 41.0 ms at 40 MHz. The comment in the driver records why the clock is not higher: 80 MHz halves that time and worked once, but ghosted on the PicoCalc's removable display. For the small updates that dominate interactive use the difference is small

anyway; the first pixels after a move in the launcher arrive 30.9 ms after the key press at 40 MHz and 28.3 ms at 80 MHz.

The design has one cost that shows in traces. Each of the four transfers per rectangle settles its own promise, and each settlement is a segment of the display world: 41 µs each, almost all of it the cost of entering the runtime. A flush of three rectangles is twelve transfers and therefore twelve short settle segments, about half a millisecond of lane time in which the driver does nothing but notice that a transfer finished. A host call that sent a whole list of rectangles and settled one promise would remove eleven of them; Exercise 6 asks you to build it.

The `skipped` answer at the top of the flush handler is the second bug the simulator found, and Section 12.10 tells it from the protocol's side. Killing the display driver from the Faults app made it crash a second time as it came back, because a flush sent to the old instance waited in the mailbox through the restart and reached the new instance before the compositor had given it the framebuffer. The driver now answers such a flush with `skipped`, and the compositor resends the whole screen.

13.8 The display pipeline

Apps in PicoCalc OS never touch the display. An app describes its screen as data: a tree of plain objects (`col`, `row`, `text`, `list`, `table`, `grid` and so on) returned by its `screen` function. After every segment of the app, the kernel calls that function, and if the JSON differs from the last frame it publishes it on the `frames` channel. The compositor, a service, does all the drawing. Figure 13.4 shows the path from an app's state to the glass.

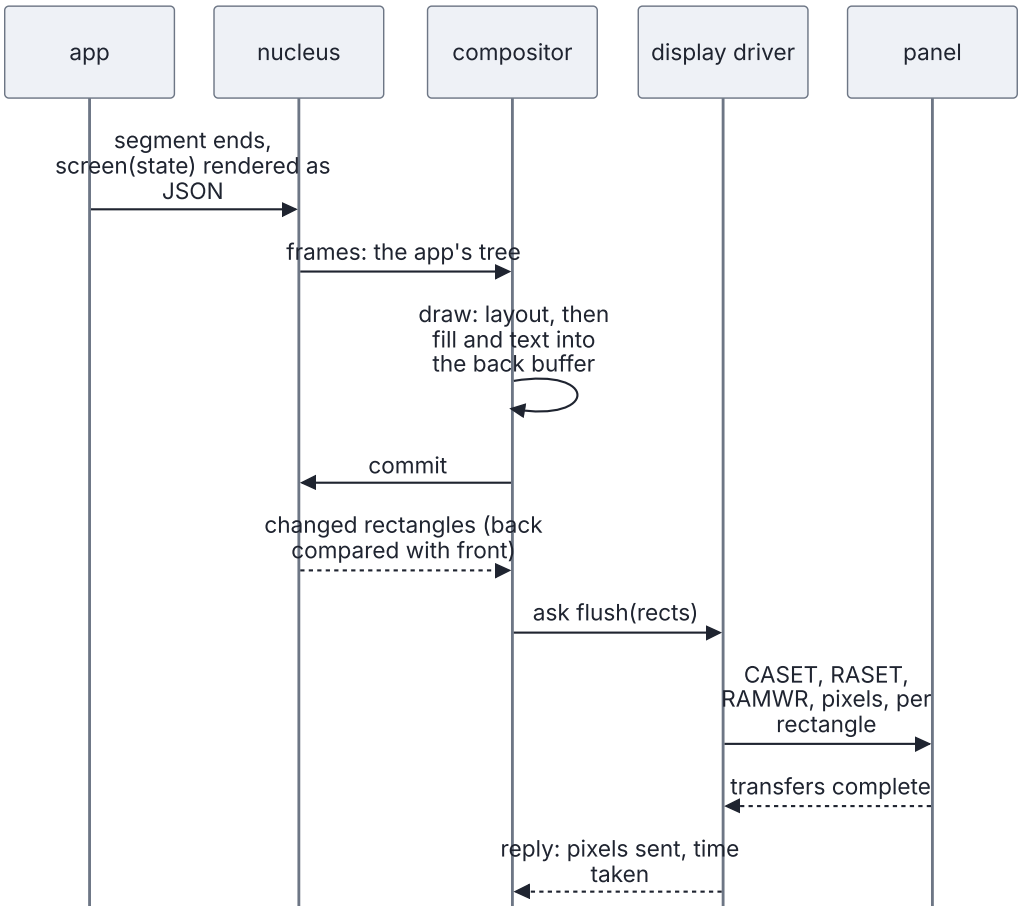


Figure 13.4. From an app's state to the glass. Only the compositor draws, and only the display driver touches the bus; the nucleus does the per-pixel work (fill, text, diff) in C and charges it to the compositor's segment.

The compositor's `draw` handler repaints the whole screen into the back copy of the framebuffer every time: the status bar, the focused app's tree laid out in two passes (measure, then draw), any crash banner, and the hint bar. It does not track which parts of the tree changed. Instead, `commit` asks the nucleus to compare the back copy with the front copy, which holds what the panel shows, and to return the changed areas:

▸ nucleus/gfx.c

```
    int rx = minx, ry = miny, rw = maxx - minx + 1, rh = maxy - miny + 1;
    if (n > 0) {   // merge with the band above when that costs little
      int *p = rects + (n - 1) * 4;
      int ux = p[0] < rx ? p[0] : rx, ur = (p[0] + p[2] > rx + rw ? p[0] + p[2] : rx +
rw);
      int uy = p[1], ub = ry + rh;
      long ua = (long)(ur - ux) * (ub - uy), sa = (long)p[2] * p[3] + (long)rw * rh;
      if (p[1] + p[3] == ry && ua * 10 <= sa * 13) { p[0] = ux; p[2] = ur - ux; p[3] = ub
- uy; continue; }
    }
```

The comparison works in bands of 16 rows. In each band it finds the leftmost and rightmost changed columns and the first and last changed rows, which gives one rectangle per band; a band's rectangle is merged into the one directly above it when the two touch and their union is at most 1.3 times the sum of their areas. The changed rectangles are copied from the back to the front copy, and the display driver later sends pixels from the front copy, so what the driver sends is always what the compositor committed.

Repainting everything and diffing is a deliberate trade. It costs a full-screen fill (0.26 ms) and a full-screen comparison (102,400 pixels at 1.2 ns, 0.12 ms) on every draw, whether one character changed or the whole screen did. In exchange the compositor has no invalidation logic to get wrong: whatever it draws, the panel ends up showing exactly that, and the damage rectangles are correct by construction. Table 13.2 shows how small the updates are in practice.

Table 13.2. The display pipeline in four scenarios (host build, medians per update). The share is of the 320 × 320 screen; draw time is the compositor's segment, split into JavaScript layout and native drawing and diffing.

Scenario	Pixels sent	Share of screen	Rectangles	SPI time	Compositor draw (JS + native)
moving the launcher's selection	21,407	20.9%	2	8.60 ms	1.47 ms (0.86 + 0.57)
system monitor, steady	3,289	3.2%	5	1.41 ms	2.09 ms (1.49 + 0.56)
snake on autopilot	1,122	1.1%	1	0.46 ms	0.96 ms (0.31 + 0.61)
typing a character at the REPL	180	0.18%	1	0.08 ms	0.87 ms (0.38 + 0.45)

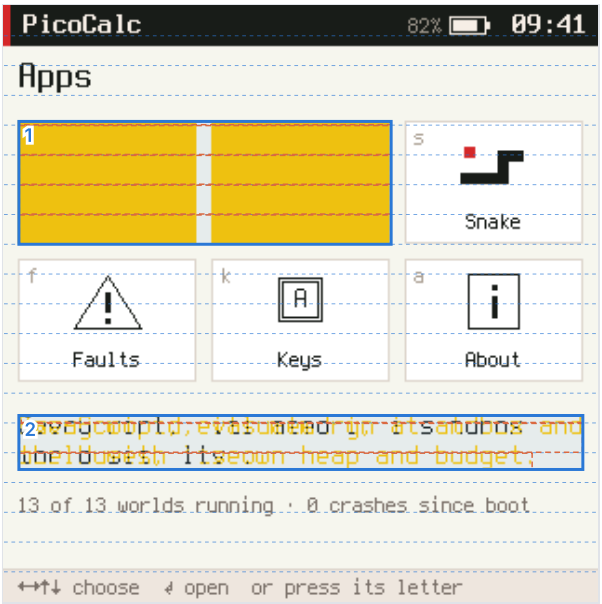
A typed character sends 180 pixels, a five-hundredth of what sending the whole screen would. The launcher's update is the largest because moving the selection repaints the old and the new tile and the description of the selected app below the grid. When the two tiles are in the same row of the grid, the bands they cover merge into one 200 × 66 rectangle, 5.3 ms on the bus; when the selection moves to the other row, each tile is its own 96 × 66 rectangle of 2.5 ms (Chapter 19 decomposes the latency).

TRY 13.2

Find the damage

The widget runs a port of the nucleus's damage algorithm on captures from the host build. Its initial scene is the launcher with the selection moving one step to the right: the changed pixels, the 16-row bands, the rectangles, and the time they take on the bus at 40 MHz. The two tiles are in one row, so the result is a single 200 × 66 rectangle of 5.30 ms. Switch the move to "down" and the tiles become two 96 × 66 rectangles of 2.55 ms each. Try the other scenes, or paint your own changes and watch when two bands' rectangles are merged and when the 1.3 rule keeps them apart.

The selection moves one tile right, within the same row.



8.84 ms

on SPI at 40 MHz; the whole screen's pixels take 41.0 ms

changed pixels	13,875
bands with changes	8
rectangles sent	2
pixels sent	21,987
share of the screen	21.5%
sent per changed pixel	1.6×

#	size	at	ms
1	200 × 66	8, 62	5.30
2	303 × 29	8, 220	3.54

Dashed orange: each band's rectangle. Solid blue: what is sent, after merging.

13.9 Testing drivers against models

The three drivers in this chapter were written and debugged before any of them ran on a PicoCalc. That was possible because the host build's board is not a stub that accepts any call: it is a pair of register-level models that behave like the chips.

The panel model holds the ST7365P's state and its 320 × 480 frame memory, decodes every command the driver sends, and draws captures from its memory exactly as the glass would show them, including the effects of three settings that a driver can get wrong without any error: column order (MADCTL's MX bit), colour order (the BGR bit) and inversion. A driver with the wrong MADCTL produces a mirrored capture; one without INVON produces inverted colours; one that writes pixels before sleep-out produces the noise of uninitialised memory. The model also warns about the two timing rules a driver can break silently: a command within 5 ms of a reset, and display-on within 120 ms of sleep-out. The keyboard controller's model follows the firmware in clockworkpi's repository [ClockworkPi n.d.], including the prepared-answer behaviour that made the interleaving bug of Section 13.5 reproducible.

Models catch what they encode and nothing else. These two cannot tell you that 80 MHz ghosts on the removable display, that the controller's 2 ms gap is enough on every unit, whether the STM32 firmware accepts a 400 kHz clock, or how the panel behaves when its supply sags. Those are properties of electrons, not of registers, and only the hardware answers them. A register-level model is valuable precisely for the class of bug that dominated this project: sequences, orderings and states, which are hard to see on hardware (a phantom key once a day) and trivial to see in a deterministic simulator (the same phantom key on every boot).

The tools that made the bugs visible are the ones the rest of this book uses: scripts that drive the system on the virtual clock, captures of the panel's memory, and traces of every segment and transfer. Figure 13.5 is one key press in the launcher as a trace: the keyboard driver's three segments and two I2C transfers, the compositor's draw, and the display driver's flush and its SPI transfers.

One key press 85 virtual ms

The launcher, the right-arrow key pressed at 25 ms. The keyboard driver finds it on its next poll, two I2C reads later the event is published, and the compositor draws and flushes the change.

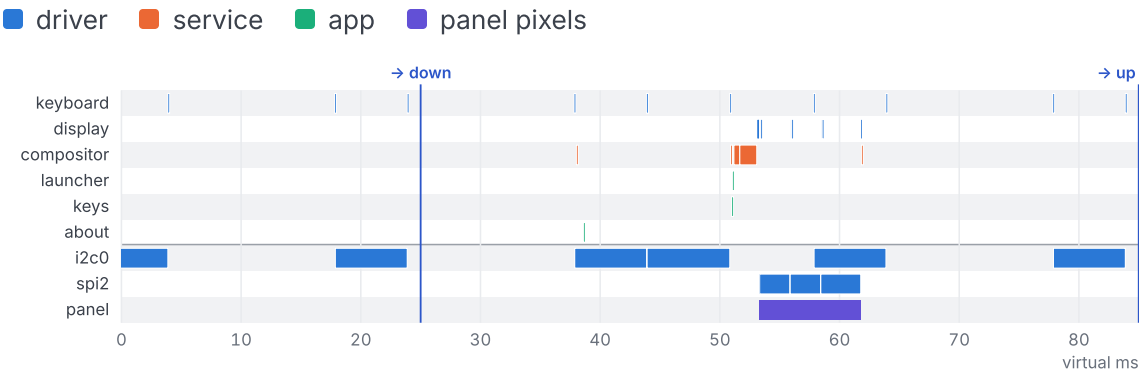


Figure 13.5. One key press in the launcher, recorded by the host build: the keyboard driver's poll and its two I2C reads, the routing and redraw, and the display driver's SPI transfers ending in changed pixels on the panel.

KEY POINTS

- A driver keeps a device's state machine in states the system can use; initialisation, configuration and power management are most of a driver's code, and moving data is the smaller part.
- Operating systems have moved drivers from the kernel into processes (UMDF, Fuchsia driver hosts, UIO and VFIO, DriverKit) so that a driver can fail and be restarted without taking the system down.
- A PicoCalc driver is a world of kind driver: highest priority, tight budget, small heap, hardware only through handles named by claims in the manifest; its `onStart` handler both initialises the device and recovers it after a restart.
- Bus operations are asynchronous effects: they start when the segment ends and the bus is free, run in request order, and settle a promise after the time their bits take on the wire.
- Two drivers sharing one chip need atomic transactions or a single owner; PicoCalc's write-gap-read transaction fixed the interleaving bug in the nucleus's API, where every driver benefits.

- The display driver owns the bus but not the pixels: it gets a read-only view of the compositor's framebuffer, queues every transfer of a flush in one segment, and answers flushes it cannot serve with `skipped`.
- The compositor repaints everything and lets the nucleus diff it in 16-row bands, which makes damage tracking correct by construction and keeps a typed character to 180 pixels on the bus.
- Register-level models expose ordering, sequence and state bugs deterministically; electrical and timing margins remain questions for the hardware.

Drivers are where an operating system meets hardware that has its own state, its own timing and its own ways of going wrong. Every driver in this chapter can crash, and two of them did, in ways the simulator made visible. The next chapter steps back from drivers to failure in general: what a fault is, why restarting a component so often works, and how a supervisor decides when to stop trying.

EXERCISES

1. BASIC List the responsibilities of a driver from Section 13.1 and, for each, the line or lines of `js/drivers/display.js` that carry it out. Which responsibilities does the display driver not have, and why?

2. BASIC Using the I2C timing rule, compute the time of the keyboard driver's status read and FIFO read at 10 kHz, 100 kHz and 400 kHz, with the 2 ms gap. What fraction of each is the gap?

3. ANALYSIS With the keyboard polled every *P* ms and each poll's status read taking 6.0 ms, what is the bus occupancy with no key pressed? At what *P* does a poll that finds one key event no longer fit in the period? Compare with the measured 98% at 5 ms.

4. ANALYSIS Explain why the interleaving of Figure 13.3 also loses a key event. Would a single-owner design (the power driver asking the keyboard driver for the battery level) have prevented the loss? What would it cost per battery reading?

5. CODE Change `js/drivers/keyboard.js` back to reading registers as two separate transfers (`dev.transfer([reg], 0)` then `dev.transfer([], n)` after `await sleep(2)`), boot the host build with `-v`, and find the symptoms in the log and in the system monitor. Then restore the fix.

6. CODE Add a host call `spi.flush(fb, rects)` that queues the commands and pixels for a whole list of rectangles and settles one promise when all have been sent. Use it in the display driver and measure, from a trace, how many display segments a launcher update needs before and after.

7. CODE Change MADCTL in the display driver to 0x08 and capture the home screen. Explain the capture using the panel model's comments in `sim/st7365p.c`. Then remove INVON and explain the second capture.

8. ANALYSIS The compositor repaints the whole screen and diffs it. Estimate the per-update cost of an alternative in which the compositor tracks which nodes of the tree changed and repaints only those. Under what workload would the alternative win, and by how much?

9. ANALYSIS In Table 13.2 the steady system monitor sends 3.2% of the screen in five rectangles. Use the merge rule to explain why there are several rectangles rather than one, and compute the bus time saved compared with sending their bounding box.

10. DESIGN The STM32 firmware has an interrupt-status register but does not drive an interrupt line. Suppose a revised board wires one to a GPIO of the ESP32-P4. Describe how the keyboard driver and the nucleus would change, and estimate the new key-to-pixel latency from the stages in Chapter 19.

11. DESIGN Write the driver model of Section 13.3 as a short specification: what a driver may assume about its handles, its restarts, its mailbox and its budget, and what it must guarantee to its clients. Compare it with Zephyr's three-part device object.

12. PROJECT Write a driver for a peripheral the PicoCalc does not have, an I2C temperature sensor of your choice, together with a register-level model of it in `sim/`. Make the model enforce the datasheet's conversion time, and show that your driver's first version breaks the rule and your fixed version does not.

FURTHER READING

[Kadav and Swift 2012] A study of every Linux driver: what drivers spend their code on, how they interact with devices and buses, and where they could share more.

[Corbet et al. 2005] The standard reference for writing Linux drivers; its chapters on the device model and on concurrency are the background to Table 13.1.

[Fuchsia docs 2026] A current design in which every driver is a user-space component and talks to its clients through typed channels.

[Linux VFIO docs 2026] How a process can drive a device directly while an IOMMU keeps the device's DMA inside the process's memory.

[NXP 2021] The I2C specification: start and stop conditions, acknowledges and repeated starts, in the original.

Failure and recovery

Every component of a real system eventually does the wrong thing. A handler dereferences a value that is not there, a loop never ends, an allocation pattern exhausts a heap, a device answers with bytes nobody expected. An operating system cannot prevent all of this, so its job is narrower and more useful: to keep the failure of one component from becoming the failure of the system, and to bring the component back. This chapter is about that job. It gives the vocabulary for talking about failures precisely, explains why restarting a component cures so many of them, describes the design discipline (crash-only software) that makes restarting safe, and then reads PicoCalc OS's supervisor line by line: what the nucleus does in the instant a world crashes, how it discards work addressed to the dead instance, and how it decides between restarting the world and giving up on it.

By the end you should be able to say exactly what survives a crash in PicoCalc OS and what does not, design a supervision policy for a component and defend its numbers, and argue where in a system the responsibility for recovery belongs. Chapter 15 then applies all of this to the hardest case, device drivers.

14.1 Faults, errors and failures

The words *fault*, *error* and *failure* are used loosely in conversation, and the looseness causes real confusion when a system is designed to tolerate one but not another. The dependability community settled on precise meanings [Avižienis et al. 2004], and they are worth adopting.

> **DEFINITION: FAULT, ERROR, FAILURE**
>
> A **failure** is an event: the service a component delivers deviates from correct service. An **error** is the part of the component's state that may lead to a failure. A **fault** is the adjudged or hypothesised cause of an error.

The three form a chain. A fault (a bug in a line of code, a cosmic-ray bit flip, a misconfigured register) is dormant until it is activated and produces an error (a wrong value in memory, a corrupted data structure, an exception unwinding the stack). The error propagates until it reaches the component's service interface, where it becomes a failure (a wrong reply, no reply, a crash). The chain is recursive: the failure of a component is a fault from the point of view of the system that contains it. When the keyboard driver fails, the system as a whole has a fault, and whether that fault becomes a system failure depends on what the rest of the system does next.

PicoCalc OS has a deliberately failing service, `flaky`, that makes the chain easy to see:

▸ `js/system/flaky.js`

```
service("flaky")
  .state({ pings: 0 })
  .on("boom", () => { throw new Error("boom, thrown on purpose"); })
  .on("slow", async (s, { ms }) => { await sleep(ms); return "finally done"; })
  .on("ping", (s) => { s.pings++; return "pong " + s.pings; });
```

The fault is the `throw` on line 4. It is activated when a `boom` message arrives. The error is the exception unwinding through the handler and out of the prelude's `__dispatch`. The failure is what other worlds observe: an ask to `flaky` that rejects with "flaky crashed: boom, thrown on purpose", and a status message on the `worlds.status` channel. Whether the *system* fails is decided by the nucleus, and the nucleus's answer is that it does not: the crash is contained to the world, the world is restarted, and every other world keeps running.

Fault tolerance, in the same vocabulary, is the avoidance of service failures in the presence of faults, carried out by error detection followed by recovery [Avižienis et al. 2004]. Recovery itself has two parts. *Error handling* removes the error from the state: rolling back to a previous good state, rolling forward to a new one, or masking the error with redundancy. *Fault handling* prevents the fault from being activated again: diagnosis, isolation, reconfiguration. Restarting a world is error handling of the bluntest kind, discarding the whole state of the component and rebuilding it from its initial definition. It does nothing about the fault. That is both why it is cheap and why it is not always enough.

Before a system can recover from an error it has to detect one. Table 14.1 lists every way a world can crash in PicoCalc OS and what in the nucleus notices it.

Table 14.1. How a world's failure is detected in PicoCalc OS. Every path ends in the same function, `crash_at` in `nucleus/kernel.c`.

What happens in the world	What notices it	Reason recorded
A handler throws	`JS_Call` returns an exception in `vm_dispatch` or `vm_settle`	the error message and its location
An async handler's promise rejects	the prelude calls `turn.fail`	the rejection's message
A segment runs past its budget	the interrupt handler or `after_host_call` sets `overrun`	"ran past its turn budget of N virtual ms"
The heap limit is reached	the world's allocator refuses and sets `oom`	"ran out of memory (its limit is N KB)"
Evaluating the world's file fails or takes over 600 ms	`boot_world` returns an error	the error, or the start budget
Another world kills it (`sys.fault`) or the host does (`crash`)	an explicit call	"killed by faults", "killed from the host"

The table is as interesting for what it leaves out. Nothing in it notices a world that returns *wrong* answers, which [Avižienis et al. 2004] would call a content failure and the distributed-systems literature a Byzantine one. Nothing notices a world that is alive but stuck: an `async` handler awaiting a promise that never settles keeps its turn open, its mailbox closed, and the nucleus satisfied, because no individual segment has overrun its budget. The world's clients see this as asks timing out, and their timeouts are the only detector. (The first pass added a second detector, a turn deadline: a turn still unfinished `turn_ms` after its first segment started, 8,000 ms unless the manifest says otherwise, is a crash; *From L4 to Small Worlds*, a separate book, Chapter 6.) Detection that relies on a component stopping cleanly is called *fail-stop* detection; Section 14.3 explains why a system should try to turn as many failures as it can into that kind, and Section 15.4.5 returns to the stuck-driver case.

14.2 Why restarting works

The most effective recovery technique in practice is also the least sophisticated: throw the component away and start a new one. Jim Gray's study of Tandem's fault-tolerant systems explained why [Gray 1985]. Tandem built its software from *fail-fast* modules, each of which either functions properly or stops; processes check their inputs, outputs and data structures and halt the moment they detect an error. A failed process is replaced by a backup running the same program (a *process pair*). If software faults were deterministic, this would be pointless: the backup would execute the same code on the same input and fail the same way.

Gray's observation was that most production software faults are not deterministic. He distinguished *Bohrbugs*, which are solid and reproducible and therefore usually found in testing, from *Heisenbugs*, which depend on timing, on the state left by earlier work, or on rare combinations of events, and which go away when the operation is retried in a fresh state. In an informal study of spooler error logs from several dozen systems, he found that one of 132 software faults was a Bohrbug; the rest were Heisenbugs [Gray 1985]. Restarting works because it does not need to understand the fault. It discards the error, which is the accumulated state the fault corrupted, and replays the work in a state where the fault is unlikely to be activated again.

PicoCalc OS restarts at the granularity of a world, and a world's restart is cheap. The trace of the `flaky` service crashing shows the cost directly:

▸ book/traces/14-flaky-loop.jsonl


```
{"k":"seg","w":"flaky","t":1993492,"dt":45,"steps":8,"native":0,"kind":"msg","type":"boom
","crash":"boom, thrown on purpose (system/flaky.js:4:38)"}
{"k":"crash","t":1993537,"w":"flaky","text":"boom, thrown on purpose
(system/flaky.js:4:38)"}
{"k":"status","t":1993537,"w":"flaky","text":"crashed"}
…
{"k":"status","t":2093599,"w":"flaky","text":"running"}
{"k":"seg","w":"flaky","t":2093537,"dt":62,"steps":31,"native":2,"kind":"boot","type":"st
art"}
```


Times in traces are virtual microseconds. The segment that threw took 45 µs. The world was then dead for 100 ms of backoff (from 1,993.537 to 2,093.537 ms), and the boot segment that created a fresh QuickJS runtime, evaluated the prelude and `flaky.js`, and read the world's definition took 62 µs of lane time on the virtual clock. A restart is therefore not a meaningful cost to the rest of the system; it is cheap enough to

use at the first sign of trouble, which is the central claim of the microreboot work [Candea et al. 2004]: a fine-grained restart of one component recovers from most of the same failures as a full reboot, an order of magnitude faster and with an order of magnitude less lost work.

Restarting has two limits that the rest of this chapter is about. A Bohrbug will be activated again: if the same input arrives, the new instance fails the same way, and a supervisor that restarts it forever converts one fault into a permanent load on the system. And restarting discards state, so anything the world knew that nobody else knew is lost. The first limit is handled by supervision policy (Section 14.6); the second by designing components so that losing their state is harmless (Section 14.3, and for drivers Section 15.4.2).

PicoCalc OS also makes one choice that limits the damage of a deterministic fault triggered by a single message. A segment that crashes was started by removing its message from the mailbox, and the crash frees that message rather than putting it back. The message that killed a world is therefore delivered at most once; the new instance does not receive it again. A *poison message*, a request that crashes every instance that handles it, cannot cause a restart loop on its own. The `flaky` world in Section 14.6 crashes five times because it was sent five `boom` messages, not because one was redelivered.

14.3 Fail-stop, fail-fast and crash-only software

A component is *fail-stop* if, when it fails, it stops: it produces no further output, and its failure is detectable by others. Fail-stop components are much easier to build systems from than components that may fail by producing wrong output, because a stop is unambiguous and a wrong answer is not. *Fail-fast* is the discipline that turns other failures into stops: check invariants aggressively and halt on the first violation, so that an error is converted into a crash before it propagates into a wrong answer.

Candea and Fox pushed this to its conclusion with *crash-only software* [Candea and Fox 2003]. A crash-only component has exactly one way to stop, which is to crash, and exactly one way to start, which is to recover. There is no separate clean shutdown path and no separate cold-start path, so the recovery code is the start-up code and is exercised every time the component starts. The argument is that rarely executed code is wrong code: a graceful shutdown path that runs once a month, or a recovery path that runs only after a crash, will contain bugs that the common path does not. If every start is a recovery, recovery is tested constantly.

Crash-only components need properties from the system around them. The paper lists them [Candea and Fox 2003]:

- All important non-volatile state is kept in dedicated state stores, so a component holds only program logic and soft state it can rebuild.

- Components have externally enforced boundaries, so a crash cannot corrupt a neighbour.
- All interactions between components have timeouts, so an unresponsive peer is treated as failed rather than waited for forever.
- All resources are leased rather than permanently allocated, so a dead component's resources are reclaimed.
- Requests are entirely self-describing, carrying what is needed to process them (and whether they are idempotent), so a request can be retried against a new instance.

PicoCalc OS is crash-only by construction, and it is worth checking each property against the code. There is no shutdown handler in the world API (Appendix A): a world stops only by crashing, whether it throws or is killed with `sys.fault`. A world's only start path is its file being evaluated and its `onStart` handler running, and that is the same path at boot and after every crash. Boundaries are enforced by the separate QuickJS runtimes and the handle checks of Chapter 9. Interactions have timeouts: every `ask` carries one, two seconds unless the caller sets another, enforced by the nucleus rather than by the callee. Resources are not leased in the strict sense, but they are owned in a way that has the same effect: every handle a world holds is released when it crashes (Section 14.4), and a DMA buffer is freed when the last handle to it goes. Requests are JSON messages naming their type, which makes them self-describing; whether they are idempotent is up to the protocol between two worlds, and Section 15.4.3 shows where that matters.

The one property PicoCalc OS satisfies only partly is the first. Worlds keep their state in their own heaps, and when a world crashes its heap is gone. For most worlds that is harmless, because their state is either rebuilt at start (the display driver re-initialises the panel) or owned by another world (the compositor owns what is on the screen). For a few it is not, and Section 15.7 examines the one in this system whose lost state is visible to the user.

14.4 Anatomy of a crash

Every detection path in Table 14.1 ends in `crash_at`. It is the most important function in the nucleus for reliability, because everything a crash does and does not destroy is decided there. Its first half records the crash and tears the world down with `teardown`, which the first pass made a function of its own so that reloading a world from a changed file (a later addition, described in a separate book, *From L4 to Small Worlds*) loses exactly what a crash loses:

▸ history/first-pass/nucleus/kernel.c

```
static void crash_at(world_t *W, const char *why) {
  K.crashes++;
…
  // crashes in the last ten seconds decide between a restart and a stop
  vtime_t now = kernel_now();
  int keep = 0;
  for (int i = 0; i < W->n_crash_times; i++) if (now - W->crash_times[i] < MS(10000)) W-
>crash_times[keep++] = W->crash_times[i];
  W->n_crash_times = keep;
  if (W->n_crash_times < 8) W->crash_times[W->n_crash_times++] = now;
  int recent = W->n_crash_times;
  klog("%s crashed%s%s: %s", W->name, W->in_turn ? " handling " : "", W->in_turn ? W-
>turn_type : "", first);
  uart_fault(W, first, strchr(why, '\n') ? strchr(why, '\n') + 1 : W->trace);
  trace_note("crash", now, W->name, first);
…
  teardown(W, t);
…
// Everything a world loses when it stops running, whether it crashed or is being
reloaded.
static void teardown(world_t *W, const char *for_asker) {
  if (K.cur == W) discard_deferred();
  if (W->in_turn && W->turn_ask) settle_ask(W->turn_ask, 0, for_asker, NULL, 0);
  vm_dispose(W);
  handles_drop_world(W);
  while (W->settles) { settle_t *s = W->settles; W->settles = s->next; free(s->json);
free(s); }
  W->settles_tail = NULL;
  clear_turn(W);
  free(W->frame); W->frame = NULL;
  for (int i = 0; i < MAX_LIST; i++) W->tick_queued[i] = false;
  W->epoch++;
```

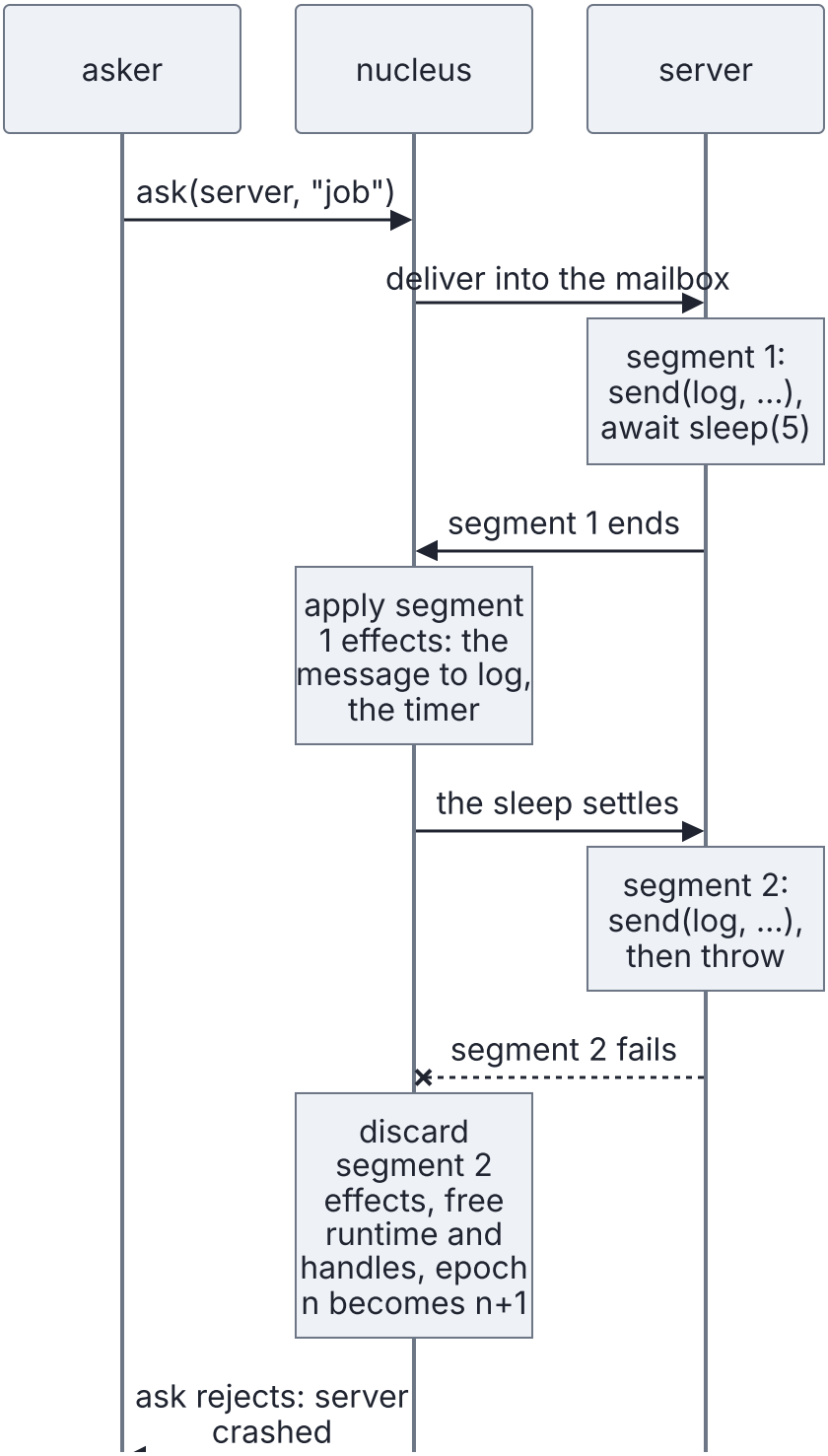
Together, the two functions do nine things; `teardown` does the first and the last six.

1. **It discards the crashing segment's effects.** If the world crashed while running (rather than being killed while idle), `discard_deferred` throws away every message, channel post, bus transfer, timer and revocation the segment asked for. This is the segment atomicity of Chapter 3 at work: a segment that crashes has no effects. Effects of *earlier* segments of the same turn have already been applied and stay applied.

2. **It records the crash time** in a short list of crashes in the last ten seconds, which the supervision policy reads below.
3. **It reports the crash** three ways: the kernel log, a `#OCT1` fault frame on the UART (Section 14.7), and the trace.
4. **It fails the ask being answered.** If the world was handling an ask, the asker's promise is rejected with "*world* crashed: *why*", immediately, rather than waiting for its timeout.
5. **It frees the runtime.** `vm_dispose` frees the QuickJS context and runtime, and with them every object, closure and pending promise of the world.
6. **It drops the world's handles.** `handles_drop_world` removes the world from the holders of every handle it held; a handle that nobody else holds is freed, and a DMA buffer is freed when no handle to it remains (Chapter 9).
7. **It drops pending completions** already queued for the world: bus transfers and sleeps that had settled but not yet run.
8. **It clears the turn and the cached frame**, so a restarted app publishes its first frame afresh.
9. **It increments the world's epoch** (Section 14.5).

Two things survive. The first is the world's identity: its name, its manifest entry, its claims, its counters. The second is its mailbox. Messages that were waiting when the world crashed, and messages that arrive while it is restarting, stay in the mailbox and are handled by the new instance. That is deliberate. Most messages remain meaningful to a new instance (a request to flush pixels, a key event, a ping), and failing all of them would turn one crash into failures in every world that had sent the dead one a message. Section 15.6 shows a message that did *not* remain meaningful, and what the display driver had to learn to cope with it.

Figure 14.1 shows the effect on an asker. A server world handles an ask in an `async` handler; its first segment sends a message and starts a timer; its second segment throws.


asker
nucleus
server
ask(server, "job")
deliver into the mailbox
segment 1: send(log, ...), await sleep(5)
segment 1 ends
apply segment 1 effects: the message to log, the timer
the sleep settles
segment 2: send(log, ...), then throw
segment 2 fails
discard segment 2 effects, free runtime and handles, epoch n becomes n+1
ask rejects: server crashed

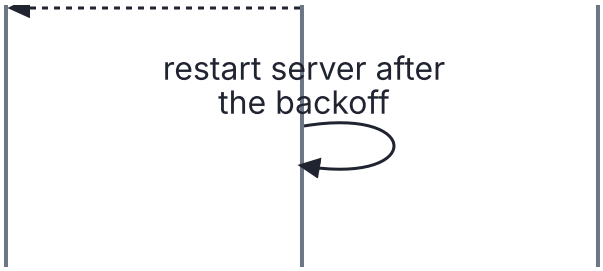


Figure 14.1. An asynchronous turn that crashes in its second segment (schematic). The first segment's effects were applied when it ended; the second segment's are discarded; the ask fails at once; the world's epoch advances so that anything still addressed to the old instance is ignored.

The nucleus never has to undo anything to achieve this. Because effects are collected during a segment and applied only at its end, the only effects that exist when a crash happens are the crashing segment's own, and those are simply dropped. Systems that let a component act on the world directly (write to shared memory, send on a socket, program a device register) at any instant cannot offer this guarantee, and recovery in those systems must reason about which partial actions were taken. Chapter 15 returns to the cost of that difference, because a device driver's interactions with hardware are exactly the effects that cannot be held back.

The second half of `crash_at` decides what happens next:

▸ history/first-pass/nucleus/kernel.c

```
  if (recent >= 5 || W->config_error) {
    W->status = W_STOPPED;
    char s[160]; snprintf(s, sizeof s, "%s is stopped", W->name);
    fail_mailbox(W, s);
    if (W->config_error) klog("%s cannot start until its file or its manifest entry is
fixed; it stays stopped", W->name);
    else klog("%s crashed %d times in ten seconds; it stays stopped", W->name, recent);
    publish_status(W, "stopped", first, 0);
    return;
  }
  W->status = W_RESTARTING;
  vtime_t delay = MS(100) << (recent - 1);
  if (delay > MS(2000)) delay = MS(2000);
  event_t *e = new_event(E_RESTART, now + delay);
  e->w = W->id; e->epoch = W->epoch;
  schedule(e);
  publish_status(W, "crashed", first, delay);
}
```

If this is the fifth crash within ten seconds, or the world failed to start because its file declares a channel its manifest entry does not give it (a check the first pass added), the world is stopped: every ask waiting in its mailbox fails with "*world* is stopped", and the stop is published. Otherwise a restart event is scheduled after a delay that doubles with each recent crash. When the event fires, the world is marked ready to boot, and the next time the lane is free it is the first world picked (booting has the highest priority of all, Chapter 4). The boot segment creates a fresh runtime, evaluates the prelude and the world's file, and if this is a restart, publishes that the world is running again:

▸ nucleus/kernel.c

```
if (booting) {
  W->boot_ready = false;
  budget_kind = "start";
  budget = MS(600);
  rc = boot_world(W);
  if (rc == 0 && W->restarts++ > 0) publish_status(W, "running", NULL, 0);
```

`boot_world` puts a `start` message at the *front* of the mailbox, ahead of anything that waited through the restart, so the world's `onStart` handler always runs before it handles old messages. That ordering is what lets a driver re-initialise its device before it answers requests addressed to its predecessor.

Figure 14.2 collects these transitions into the life cycle of a world. The state names are the ones the system monitor shows (`sys.worlds()` reports `idle`, `busy`, `waiting`, `restarting` and `stopped`); the three events on `worlds.status` are `crashed`, `running` and `stopped`.

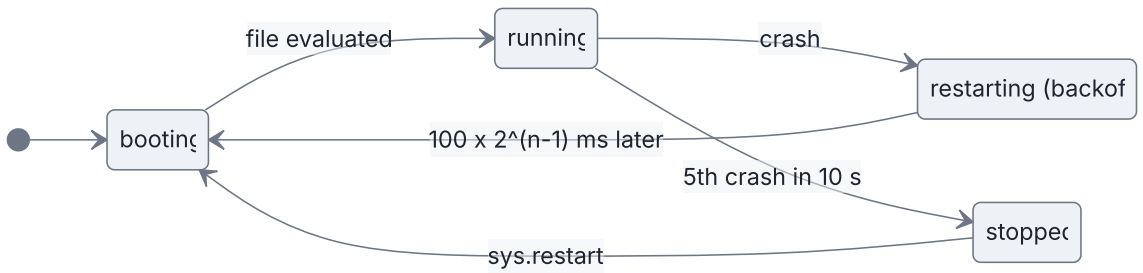


Figure 14.2. The life cycle of a world. Running covers the idle, busy and waiting states of the system monitor. Booting has its own 600 ms budget, and a world that fails while booting is treated as having crashed; the fifth crash within ten seconds, while running or while booting, stops it.

14.5 Epochs: discarding work addressed to the dead

A crash happens at an instant, but a world's activity is spread over time. When a world crashes, the system may still hold completions that belong to it: a bus transfer that is still on the wire and will settle a promise when its last bit has gone out, a `sleep` whose timer has not expired, a reply to an ask the world sent, the periodic timer that delivers its `every` ticks, even the restart event of an earlier crash. If these reached the new instance, some would merely waste work and some would be wrong. A settle for a promise of the old runtime would find no such promise in the new one (promise numbers are allocated by the nucleus and never reused), so it would only waste a segment on the lane. The periodic timer is worse: each tick re-arms the next, so without a check the old instance's chain of ticks would keep running beside the new instance's own, and every restart would add another chain, waking the world twice, then three times, as often as it asked. A stale restart event, left over from an earlier crash, could boot a world that is already running.

The nucleus solves this with an *epoch*: a counter per world, incremented by every crash. Everything addressed to a world later is stamped with the epoch it was addressed to. A bus transfer copies the issuing world's epoch when it is issued, a timer when it is started, an ask records the asker's epoch, and the restart event records the epoch of the crash that scheduled it. When the completion arrives, the event loop compares:

▸ nucleus/kernel.c

```
    case E_TICK:
      if (!W || W->epoch != e->epoch || W->status != W_IDLE) break;
…
    case E_SETTLE:
      if (W && W->epoch == e->epoch && W->status == W_IDLE) {
…
    case E_RESTART:
      if (W && W->epoch == e->epoch && W->status == W_RESTARTING) { W->boot_ready = true;
W->boot_t = e->t; W->boot_seq = ++K.seq; }
      break;
```

and a reply to an ask is dropped if the asker has crashed since asking:

▸ nucleus/kernel.c

```
  world_t *F = world_by_id(a->from);
  if (!F || F->epoch != a->from_epoch || !alive(F)) return;
```

A mismatched epoch means the completion belongs to an instance that no longer exists, and it is dropped. Note what is *not* undone: a bus transfer the old instance started still happens on the wire, because it was an effect of a segment that completed, and the bus serialises it ahead of whatever the new instance sends. Only its completion is discarded. Section 15.5 shows a case where that matters: a byte read from the keyboard controller by a driver that died before it could use it.

Epochs are an instance of a general technique: name each incarnation of a component with a number that never repeats, and attach the number to everything that might outlive the incarnation. The alternative, finding and cancelling every outstanding operation at the moment of the crash, requires the nucleus to know every place an operation can be queued, including the bus queues, the timer heap and other worlds' ask tables. The epoch check needs only one comparison at the point of delivery, and it remains correct when new kinds of operation are added later.

14.6 Supervision: backoff and restart intensity

Restarting cures Heisenbugs and does nothing for Bohrbugs. A supervisor therefore needs a policy for the second case: how soon to restart, and when to stop trying. Erlang's OTP libraries, built on the reliability philosophy of Joe Armstrong's thesis [Armstrong 2003], made this policy a first-class part of program structure. A *supervisor* is a process whose only job is to start, watch and restart child processes. Its *restart strategy* says which children to restart when one fails: only the failed child (`one_for_one`), all of them (`one_for_all`), or the failed child and those started after it (`rest_for_one`). Its *restart intensity* says how many restarts it will perform within a period: if more than `intensity` restarts occur within `period` seconds, the supervisor terminates all its children and then itself, passing the problem up to its own supervisor. The defaults are an intensity of 1 and a period of 5 seconds [Erlang/OTP n.d.].

The same two ideas, a delay before restarting and a limit on restarts per unit of time, appear in every supervision mechanism. Table 14.2 compares several.

Table 14.2. Restart policies in five systems. The numbers are defaults; all of them are configurable.

System	What is restarted	Delay before restarting	Limit	When the limit is exceeded
Erlang/OTP supervisor [Erlang/OTP n.d.]	a child process	immediate	more than 1 restart in 5 s	the supervisor terminates its children and itself, and its own supervisor decides
systemd services [systemd n.d.]	a service's processes	100 ms	5 starts in 10 s	the service is not started again
Windows display drivers (TDR) [Microsoft n.d.]	the graphics stack, when the GPU misses a 2 s preemption timeout	—	5 recoveries in 60 s	the computer crashes
MINIX 3 reincarnation server [Herder et al. 2007]	a driver or server process	set by a policy script, e.g. exponential backoff	set by the script	set by the script, e.g. alert the administrator
PicoCalc OS	a world	100 ms × 2^{n-1} for the nth crash in 10 s	5 crashes in 10 s	the world is stopped; asks to it fail at once

The table shows a consensus about magnitudes. A restart should be fast enough that a single transient failure costs a fraction of a second, which is why the first delay is 100 ms or less everywhere. And a component that fails repeatedly within a few seconds should be given up on, because it is almost certainly failing deterministically, and each restart costs the rest of the system something: lane time to boot it, messages announcing the crash and the restart, the work of its clients reconnecting, and log output that buries other information. PicoCalc OS's limit of five crashes in ten seconds happens to be the same as systemd's default start limit, and its first delay the same as systemd's default restart delay.

PicoCalc OS adds a doubling delay. The nth crash within ten seconds schedules a restart after

$$d(n) = 100\ \text{ms} \times 2^{n-1}, \quad n = 1, 2, 3, 4 \tag{14.1}$$

which is 100, 200, 400 and 800 ms; the fifth crash stops the world. (The code also caps the delay at 2 s, a cap the five-crash rule makes unreachable.) Exponential backoff serves the same purpose as it does in network protocols: if a failure is caused by something outside the component (a device that needs time to settle, a server it depends on that is itself restarting), spacing the attempts out gives that cause time to clear, while the first attempt still happens quickly.

The Faults app has an experiment that sends `flaky` five `boom` messages at once. All five wait in the mailbox; each instance handles one and crashes. Figure 14.3 shows the result in the trace, and the status lines tell the whole story:

▸ book/traces/14-flaky-loop.jsonl


```
{"k":"status","t":1993537,"w":"flaky","text":"crashed"}
{"k":"status","t":2093599,"w":"flaky","text":"running"}
{"k":"status","t":2093644,"w":"flaky","text":"crashed"}
{"k":"status","t":2293706,"w":"flaky","text":"running"}
{"k":"status","t":2293751,"w":"flaky","text":"crashed"}
{"k":"status","t":2693813,"w":"flaky","text":"running"}
{"k":"status","t":2693858,"w":"flaky","text":"crashed"}
{"k":"status","t":3493920,"w":"flaky","text":"running"}
{"k":"status","t":3493965,"w":"flaky","text":"stopped"}
```


The crashes are at 1,993.5, 2,093.6, 2,293.8, 2,693.9 and 3,494.0 ms of virtual time. The gaps between them are the backoff delays of Equation 14.1 plus 62 µs to boot and 45 µs to handle the next message. The fifth crash, exactly 1.5 seconds after the first, stops the world. The next item in the Faults app restarts it with `sys.restart`, which only a world with the `sys.admin` claim may call; the restart resets the crash count, and a `ping` to the new instance is answered "pong 1", its counter starting again from zero because its state is new.

Five crashes in 1.5 seconds 1580 virtual ms

flaky handles one boom per instance; the restarts wait 100, 200, 400 and 800 ms; the fifth crash stops it.

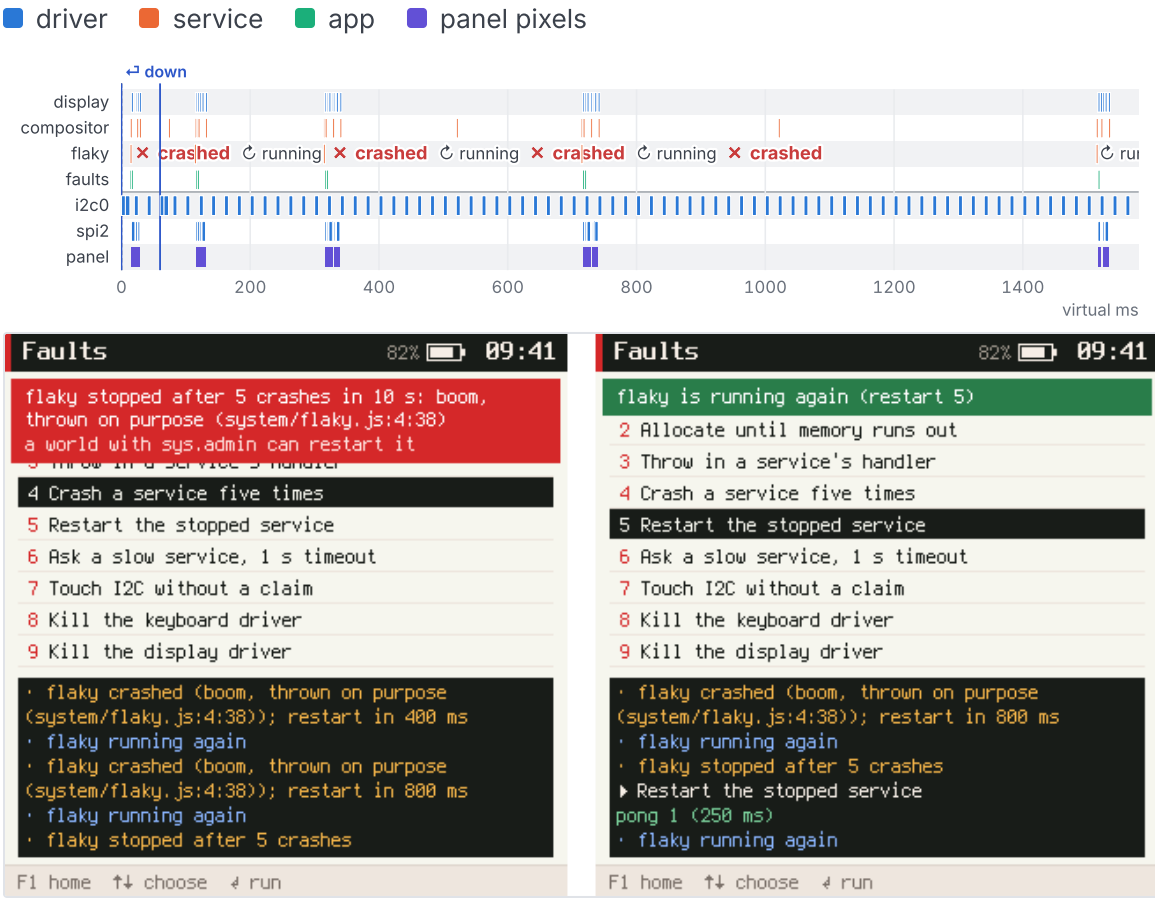


Figure 14.3. The flaky service crashing five times. Top: the trace, with the flaky and faults rows; each crash is followed by a longer wait. Bottom: the Faults app after the fifth crash (left) and after restarting the stopped service (right), as captured by the host build.

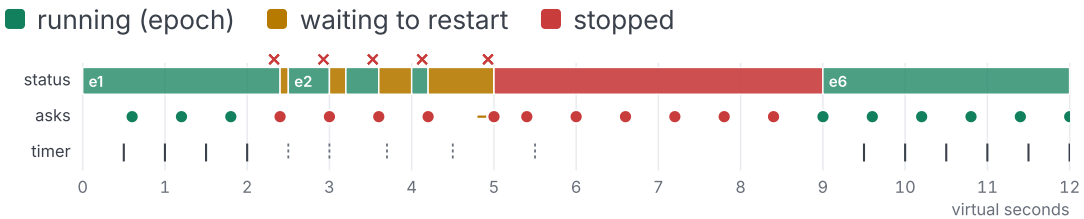
The benchmark suite measures the same policy from the asker's side. A world that overruns its budget on every request, having already crashed once in an earlier test, is asked six times in a row. The first four asks fail after 21, 221, 421 and 821 ms: each request waited for the restart that the previous crash had scheduled

(0, 200, 400 and 800 ms) and then took 20.7 ms to run into the 20 ms budget. The fourth failure is the world's fifth crash within ten seconds, so the fifth and sixth asks fail at once with "spin20 is stopped", in 57 µs.

TRY 14.1

Supervise a failing world

The simulation below applies PicoCalc OS's policy to a world that crashes on every message, with a client sending it a message every 600 ms. Follow the timeline: the restarts wait 100, 200, 400 and 800 ms, the fifth crash stops the world, and at 9 s an administrator's `sys.restart` brings it back with epoch 6. Look in the log for the timer ticks that were dropped because they carried an old epoch, and for the asks that waited in the mailbox through a restart and were answered by the new instance. Then switch to the free scenario and change the policy: with a limit of 10 crashes, how long does the world keep consuming restarts? With no backoff, how many restarts happen in the first second?



Asks row: a green dot is an answer, a red dot a failure, an amber line a wait in the mailbox. Timer row: solid ticks ran, dashed ticks came from an old epoch and were dropped.

now `12.0 s` weather is `running` epoch `6` crashes in the last 10 s `0`

asks answered `9 of 20`

`0.000 s`	↻	weather starts (epoch 1)
`2.000 s`	◆	weather now crashes on every message it handles
`2.400 s`	×	weather crashed: Error: crashes on every message. Crash 1 in the last 10 s: restart in 100 ms, as epoch 2.
`2.500 s`	·	a timer tick from epoch 1 arrives; weather is in epoch 2, so it is dropped
`2.500 s`	↻	weather starts again (epoch 2)
`3.000 s`	×	weather crashed: Error: crashes on every message. Crash 2 in the last 10 s: restart in 200 ms, as epoch 3.
`3.000 s`	·	a timer tick from epoch 2 arrives; weather is in epoch 3, so it is dropped

3.200 s	↻	weather starts again (epoch 3)
3.600 s	×	weather crashed: Error: crashes on every message. Crash 3 in the last 10 s: restart in 400 ms, as epoch 4.
3.700 s	·	a timer tick from epoch 3 arrives; weather is in epoch 4, so it is dropped
4.000 s	↻	weather starts again (epoch 4)
4.200 s	×	weather crashed: Error: crashes on every message. Crash 4 in the last 10 s: restart in 800 ms, as epoch 5.
4.500 s	·	a timer tick from epoch 4 arrives; weather is in epoch 5, so it is dropped
5.000 s	↻	weather starts again (epoch 5); 1 ask waited in its mailbox
5.001 s	■	weather crashed: Error: crashes on every message. That is 5 crashes in 10 s: it stays stopped.
5.500 s	·	a timer tick from epoch 5 arrives; weather is in epoch 6, so it is dropped
8.500 s	◆	weather's fault is fixed (its next start runs the good code)
9.000 s	↻	sys.restart: weather starts again (epoch 6); its crash history is forgotten

PicoCalc OS has only one restart strategy, the equivalent of `one_for_one`: the nucleus restarts exactly the world that crashed. Dependencies between worlds (the compositor depends on the display driver; every app depends on the compositor) are not expressed in the supervisor at all. Instead, worlds that care about another world's health subscribe to `worlds.status` and react themselves. The compositor, for example, resends the framebuffer and every pixel when the display driver is running again (Section 15.6). This is the opposite of OTP's choice, which encodes dependencies in the supervision tree so that `rest_for_one` can restart a dependent along with what it depends on. PicoCalc's choice keeps the nucleus small and puts the knowledge of what a restart means where that knowledge lives, in the dependent. Its cost is that every dependent must be written to handle its dependency's restart, and a dependent that is not, as Section 15.7 shows, leaves visible damage behind.

14.7 Telling others: worlds.status and the fault frame

A crash is reported in two places, one for the system and one for a person or program diagnosing it.

The system's report is the `worlds.status` channel, published by the nucleus itself. Each message names the world, its new state (`crashed`, `running` or `stopped`), the reason, its restart and crash counts, and for a crash the delay before the restart. Any world can subscribe by declaring `needs("worlds.status")`. Two do: the compositor, which shows a banner over the screen and reattaches the display when it returns, and the Faults app, which logs each event. The banner is what a user sees: "keyboard crashed: killed by faults, restarting in 100 ms", replaced 100 ms later by "keyboard is running again (restart 1)".

The diagnostic report is the *fault frame*, one line written to the console UART for every crash. Here is the frame from the first crash in Figure 14.3, as the host build wrote it to its UART log:

```
#OCT1
{"ev":"fault","t":1993.537,"world":"flaky","file":"system/flaky.js","handling":"boom","wh
y":"boom, thrown on purpose
(system/flaky.js:4:38)","crashes":1,"mem_kb":183,"limit_kb":256,"detail":"    at
<anonymous> (system/flaky.js:4:38)\n    at <anonymous> (prelude.js:139:17)\n"}
```

The frame is designed to be read by a program. It starts with a fixed tag (`#OCT1`, the protocol name and version) so that a reader on the other end of a serial line can pick frames out of ordinary log output, and the rest is one JSON object: the virtual time, the world, the file it was loaded from, the message type it was handling, the error with its position, the stack, how many times it has crashed recently, and its heap use against its limit. That is enough to begin a diagnosis without access to the device: the file and line identify the code, the handler identifies the input that triggered it, and the heap figures distinguish a world that ran out of memory from one that threw. Section 15.9 discusses what a program on the other end of the cable could do with it. On the host build the frames go to a log file today; on the device they will go out on the console UART (a planned part of the port, Chapter 7).

14.8 Where recovery belongs

The supervisor restarts a world. It does not, and cannot, make the world's clients whole. When `flaky` crashes while answering an ask, the asker gets a rejected promise; the nucleus has no way to know whether the request should be retried, whether retrying is safe, or whether the caller would rather report the failure to the user. That decision belongs to the caller.

This is an instance of the *end-to-end argument* [Saltzer et al. 1984]: a function can be completely and correctly implemented only with the knowledge and help of the application at the end points of the communication, so providing it in a lower layer is not possible, though a lower layer may usefully offer an incomplete version of it as a performance enhancement. In PicoCalc OS the restart is that performance enhancement. It makes the server available again quickly, which is what makes a retry worth attempting. Whether to retry, and how to make retrying safe, remains the job of the worlds at the ends.

The system's worlds show the range of reasonable answers. The Faults app reports every failure to the user verbatim and retries nothing, because its purpose is to show failures. The REPL asks the sandbox to evaluate each line with a five-second timeout and shows the error if the sandbox crashed or overran; the user decides whether to type the line again. The compositor does retry, in a specific way: when a flush to the display driver fails or is skipped, it marks the whole framebuffer as changed, so the next flush sends every pixel and the screen converges to the right content regardless of which earlier flushes were lost:

▸ `js/system/compositor.js`

```
    try {
      const r = await ask("display", "flush", { rects }, { timeout: 1500 });
      if (r.skipped) {   // the panel did not get these pixels: send everything again
        s.fb.invalidate();
        if (!s.resent) { s.resent = true; dirty(s); }
      } else s.resent = false;
      s.flushError = "";
    } catch (e) { s.flushError = e.message; s.fb.invalidate(); }
```

The pattern is worth naming. The compositor does not try to remember which rectangles were lost and resend those. It resends the *state* (the whole screen) rather than replaying the *operations*, which makes the recovery idempotent: sending the full screen twice is harmless, and sending it once after any combination of lost flushes produces the right picture. The `resent` flag limits it to one immediate retry, so a display driver that skips every flush does not cause an endless loop of full-screen redraws. Chapter 15 turns this observation into a design rule for drivers.

KEY POINTS

- A fault causes an error, an error that reaches a component's interface is a failure, and a component's failure is a fault for the system around it; fault tolerance is the art of stopping that chain at a boundary.

- Restarting a component discards the error, not the fault. It works because most production software faults are transient Heisenbugs, and it fails against deterministic Bohrbugs, which is why every supervisor needs a restart limit.
- Crash-only design makes the only start path a recovery path, so recovery code is exercised every time a component starts; it requires state stores, enforced boundaries, timeouts, reclaimable resources and self-describing requests.
- In PicoCalc OS a crash discards the crashing segment's effects, fails the ask being answered, frees the runtime and the world's handles, and advances its epoch; the world's mailbox survives and is handled by the new instance, with its `start` message first.
- Epochs stamp everything addressed to a world with the incarnation it was addressed to, so completions, timers and replies meant for a dead instance are dropped with one comparison at delivery.
- Supervision policies everywhere combine a short first delay with a limit on restarts per unit of time; PicoCalc OS restarts after 100, 200, 400 and 800 ms and stops a world at its fifth crash within ten seconds.
- The message that crashed a world is not redelivered, so a single poison message cannot cause a restart loop.
- Recovery that makes clients whole belongs at the ends: the supervisor makes a retry possible, and the client decides whether retrying is safe; resending state rather than replaying operations makes the retry idempotent.

A supervisor is only as good as the components it supervises. A world that can be thrown away and recreated at any instant, holds no state that matters only to itself, and answers requests that are safe to repeat is easy to supervise; a world that holds the only copy of something, or that has told a piece of hardware to do something halfway, is not. Device drivers are the components where these properties are hardest to achieve, because their purpose is to hold the state of a device that does not restart when the driver does. Chapter 15 takes up that problem.

EXERCISES

1. BASIC Classify each of the following as a fault, an error or a failure, and say which component's point of view you are taking: a missing `await` in the keyboard driver; the compositor's `s.apps` table holding a frame for an app that has crashed; the REPL showing "sandbox crashed: ran past its turn budget of 200 virtual ms".

2. BASIC List what survives a world's crash in PicoCalc OS and what does not. For each item that does not survive, name the line of `crash_at` responsible.

3. BASIC A world crashes at virtual times 0 s, 3 s, 6 s, 9 s and 12 s. Using `crash_at`'s logic, give the restart delay after each crash and say whether it is ever stopped.

4. ANALYSIS A world crashes deterministically on every message, and a client sends it one message every 10 ms. How long after the first crash is it stopped? How many messages are in its mailbox at that moment, and what happens to each of them?

5. ANALYSIS Suppose epochs were removed and the nucleus instead freed every pending settle at the moment of a crash. Name two completions that this would fail to catch, and describe the incorrect behaviour each would cause in the new instance.

6. ANALYSIS The display driver's `onStart` awaits seven bus commands and five sleeps. If it crashes after `SLPOUT` but before `COLMOD`, which commands has the panel received, which effects were discarded, and what state is the panel in when the new instance starts? Why does the new instance not need to know?

7. ANALYSIS Compare PicoCalc's policy with systemd's defaults in Table 14.2. For a service that fails deterministically 5 ms after every start, how many starts does each allow in the first ten seconds, and how much time does each spend dead between the first failure and giving up?

8. CODE Run the Faults app's "Crash a service five times" experiment in the host build with a trace (`wait 1200`, `key f`, `wait 400`, three `key down`, `trace flaky.jsonl`, `key enter`, `wait 2500`) and confirm the crash times in Section 14.6. Then change the restart limit in `crash_at` to 3 and predict, before running it again, what the Faults app's log shows.

9. CODE Write a world that answers an ask in two segments and throws in the second, as in Figure 14.1, with a `send` in each segment to a logging world. Use the trace to show that the first message is delivered and the second is not.

10. CODE Add a `stop` reason to `worlds.status` messages that distinguishes a world stopped by the restart limit from one killed by an administrator, and make the Faults app show it.

11. DESIGN PicoCalc OS uses only a `one_for_one` strategy. Describe a situation in this system where `rest_for_one` would be the better choice, and sketch how the manifest could express it. What would the nucleus have to know that it does not know today?

12. DISCUSSION Candea and Fox argue that a component should have no clean shutdown path. Give one thing a PicoCalc world might want to do on a clean shutdown (for example on power-off), and say how a crash-only design should achieve the same result instead.

FURTHER READING

[Gray 1985] The origin of fail-fast modules, process pairs and the Bohrbug/Heisenbug distinction, with failure statistics from Tandem's customers. Short and still the best argument for restart-based recovery.

[Avižienis et al. 2004] The reference taxonomy of dependability: faults, errors, failures, and the means to attain dependability. Worth reading once for the vocabulary alone.

[Candea and Fox 2003] Crash-only software in five pages: why stop should equal crash and start should equal recover, and what that requires of the surrounding system.

[Candea et al. 2004] Microreboots: the evidence that fine-grained restarts recover most failures an order of magnitude faster than whole-system reboots.

[Armstrong 2003] Joe Armstrong's thesis on building reliable systems in Erlang, including the "let it crash" philosophy and supervision trees.

[Saltzer et al. 1984] The end-to-end argument, which explains why a lower layer's recovery can make a system faster but cannot make it correct.

Self-healing device drivers

A self-healing device driver is one whose failure the system detects and repairs while it keeps running: no reboot, no human, and ideally no application that notices anything worse than a pause. This chapter explains why such drivers are worth the trouble, how operating systems over the past two decades have built them, and what a driver has to be like for the technique to work. The argument rests on three things. First, evidence: drivers are where operating systems fail. Second, mechanism: isolating a driver keeps its failure from spreading, and recovering it restores service, and the two are separate problems with separate solutions. Third, design: a driver is restartable only if it obeys a small set of rules about where state lives, what requests mean when repeated, and how authority is granted, and those rules are worth stating explicitly because most drivers break at least one of them.

PicoCalc OS is the laboratory. Its keyboard and display drivers are JavaScript worlds that the nucleus restarts when they crash, and its compositor, the display driver's only client, is restarted the same way. We will kill each of them in the host build, read the traces event by event, and find that two recover completely, that one recovers incompletely in ways a user can see, and that a key press can be lost in a window of about seven milliseconds. Each of those outcomes follows from one of the rules. By the end of the chapter you should be able to judge whether a driver is restartable by reading it, and to say what it would take to make it so.

15.1 The evidence: drivers fail more

The case for treating drivers specially is empirical, and it has been made repeatedly against different systems over two decades.

Chou and colleagues ran static checkers over 21 snapshots of Linux, from version 1.0 to 2.4.1, covering seven years of development [Chou et al. 2001]. They found that device drivers had error rates up to three to seven times higher than the rest of the kernel for the types of error their checkers detect: about three times higher for most checkers and almost seven times for the checker that finds misuse of locks. Drivers were also most of the code, about 70% of it at both ends of the period studied. A large body of code with a high error rate is where most of the errors are.

The equivalent evidence for Windows comes from crash reports. The Nooks paper reported that drivers accounted for 85% of recently reported Windows XP failures, a figure attributed to a private communication from Microsoft's vice president of Windows core technology [Swift et al. 2003]. An independent analysis of 2,528 kernel crashes reported by 617 volunteers over a year reached the same qualitative conclusion: crashes were predominantly caused by poorly written device drivers, and drivers from seven organisations accounted for 75% of them [Ganapathi et al. 2006]. The authors were careful to say that their volunteer population was self-selected and their sample too small to generalise from; the direction of the result is what matters here.

The picture has shifted somewhat since. A repetition of the Linux study over versions 2.6.0 to 2.6.33 found that drivers still contained the most faults, because they are still the largest part of the kernel, but that their fault *rate* had fallen below that of the architecture-specific code and the file systems [Palix et al. 2011]. Drivers improved; they did not stop being the largest source of faults.

Two further studies explain why drivers are hard to get right. Ryzhyk and colleagues classified real Linux driver defects and found that 39% of them came from two architectural causes: poorly defined protocols between the driver and the device and operating system, and the multithreaded computation model drivers are written in [Ryzhyk et al. 2009]. Kadav and Swift measured what driver code does and found that initialisation and clean-up are about 36% of it, configuration about 15%, and error handling about 5%; 44% of drivers contain code that their class definition does not describe [Kadav and Swift 2012]. A driver is mostly code that runs rarely (initialisation, configuration, error paths), written against a device whose behaviour is specified in a datasheet of uneven quality, with concurrency between interrupts, requests and timers. Rarely run code is wrong code, as Section 14.3 argued, and drivers are mostly rarely run code.

The practical conclusion is not that drivers should be written more carefully, though they should. It is that an operating system should be built on the assumption that its drivers will fail, and should survive it.

15.2 Isolation, then recovery

Surviving a driver failure takes two things. *Isolation* keeps the failure from spreading: a driver that writes through a bad pointer must not corrupt the scheduler, and a driver that loops must not stop the rest of the system. *Recovery* restores the service the driver provided: something must notice the failure, clean up after the dead driver, start a new one, and bring the new one into the state its clients expect. Isolation is a precondition for recovery, because a failure that has already corrupted the rest of the kernel cannot be recovered by restarting the driver. But isolation alone is not recovery: a perfectly isolated driver that has died still leaves its device unusable.

The research systems differ mostly in where they draw the isolation boundary.

- **Nooks** kept drivers in the kernel but ran each in a *lightweight kernel protection domain*, with write access limited to its own memory, and mediated every call between driver and kernel through a special procedure call (XPC) that tracked the kernel objects the driver used, so they could be reclaimed [Swift et al. 2003]. In fault-injection experiments it prevented 99% of the system crashes that the injected faults caused in unmodified Linux (4 against 317 in 2,000 trials), at a performance cost ranging from nothing to just over 60%, the worst case being an in-kernel web server.
- **Drivers in virtual machines** go further: LeVasseur and colleagues ran unmodified Linux drivers, each with its own copy of the Linux kernel, in separate virtual machines, isolating faults in defective or malicious drivers from the rest of the system, with network performance within 3–8% of native [LeVasseur et al. 2004].
- **Microdrivers** split a driver in two: the performance-critical data path stays in the kernel and the rest (initialisation, configuration, error handling, exactly the rarely run code) moves to a user-mode process. A tool divided existing drivers with a few annotations, and up to 65% of driver code could leave the kernel without hurting common-case performance [Ganapathy et al. 2008].
- **SafeDrive** used a language-based approach: annotations on the C code of a driver let a compiler insert run-time checks for type and memory safety, and wrappers around the kernel API tracked the invariants the driver was maintaining so they could be restored when a check failed [Zhou et al. 2006]. It recovered all 44 crashes caused by faults injected into a network driver, with less than 4% of the driver source changed.
- **MINIX 3** made every driver an ordinary user-mode process with restricted privileges, communicating with the rest of the system only by messages [Herder et al. 2007]. That is the most complete isolation of the five, and the one closest to PicoCalc OS.

PicoCalc OS draws the boundary with the language. Each driver is a separate QuickJS runtime with its own heap; it can affect the rest of the system only through the nucleus's host calls, which check every handle on every use (Chapter 9); and its CPU and memory are bounded by its budget and heap limit (Chapter 11). The boundary is as strong as the interpreter and the nucleus are correct, which is the trade-off Chapter 10 examined. For recovery purposes it has the property that matters: when a driver world crashes, nothing outside it has been corrupted, so restarting it is enough.

15.3 Recovery architectures

Restarting a driver is the easy part of recovery. The hard part is the state: the configuration the kernel gave the old driver, the requests it had accepted but not completed, the connections applications had open, and the state of the device itself. The systems below differ in who keeps that state and who replays it.

15.3.1 Shadow drivers

Swift and colleagues built *shadow drivers* on top of Nooks to make recovery transparent to applications [Swift et al. 2004]. A shadow driver is written once per *class* of driver (one for network cards, one for sound cards, one for disks) against the kernel's interface for that class rather than against any particular driver. Every call between the kernel and a driver passes through a *tap*, which in normal operation calls the real driver and then the shadow with the same arguments and results. In this *passive mode* the shadow records what it will need later: the configuration calls the kernel made (ioctl commands and their arguments, driver parameters), the requests that are outstanding until each completes, per-connection state for connection-oriented drivers, and the kernel objects the driver registered (timers, interrupt lines, memory regions), so they are not leaked. It does not copy the device's state.

When the driver fails, the tap switches the shadow to *active mode*. The shadow disables the driver, masks the device's interrupt line, removes its memory mappings to block stray DMA, and reclaims its resources. It then restarts the driver from a clean copy of its data section and re-runs the kernel's initialisation sequence, impersonating the kernel to the restarting driver: it answers the driver's registration calls itself, reusing the registrations the old driver made. Finally it restores state by replaying the logged configuration and resubmitting outstanding requests. While all this happens, the shadow also impersonates the driver to the kernel, and for each request that arrives it chooses one of five responses: return recorded information, drop the request, queue it, block the caller, or report that the device is busy. The sound shadow blocked reads and writes and answered configuration calls itself; the network shadow reported "busy" on sends, which the network stack already knows how to handle.

The results are the strongest evidence that driver recovery works. In 2,100 fault-injection trials on three drivers (network, sound and disk) and six applications, 390 failures occurred; the 65% the system detected automatically were all masked from applications, and of the remaining 135, detected only when triggered by hand, 127 were recovered [Swift et al. 2004]. Recovery took about five seconds for the network driver, about four for the disk and about a tenth of a second for the sound card, heard as a click. The cost in normal operation was small: across nine applications performance averaged 99% of native and was never below 97%. The authors were explicit about the limits, and they are the limits of every system in this chapter: deterministic failures recur during recovery; a request cannot in general be guaranteed to execute exactly once; persistent side effects such as bad disk writes are not undone; and transient device state that the shadow does not copy must be recreated by clients, their example being a frame buffer that must be redrawn.

15.3.2 MINIX 3's reincarnation server

MINIX 3 recovers drivers with a user-mode server, the *reincarnation server*, that is the only process privileged to create new servers and drivers [Herder et al. 2007]. It detects failure four ways: a driver that exits or is killed by a CPU or MMU exception is reported to it by the process manager; it periodically asks drivers for a heartbeat, and a driver that fails to respond a configured number of consecutive times is restarted; authorised servers can complain about a driver, as the file server does when a disk driver sends unexpected messages or stops responding; and a user can ask for a driver to be restarted or replaced explicitly. A policy script, given the component, the reason and the failure count, decides when to restart; the example script applies exponential backoff and can send an alert. A *data store* maps stable names to the current process of each component, so that clients can find a restarted driver, and offers publish–subscribe so they can be told when it changes.

What happens next depends on the class of driver, and the classification is the most useful idea in the paper.

- For a **network driver**, the network server (INET) is told through the data store that the driver has been restarted. It reinitialises the driver, including modes such as promiscuous reception, and resumes I/O; requests made during the gap are postponed. Data lost in the gap is recovered by TCP's retransmission, end to end, and UDP applications tolerate loss as they always must. Recovery is transparent.
- For a **block driver**, the file server marks the interrupted request as pending, and when the driver is back it reissues the failed operations. This is safe because disk block I/O is idempotent: writing the same block twice leaves it in the same state as writing it once. Recovery is transparent.
- For a **character driver** (a printer, an audio device, a CD writer), neither the driver nor the server can know how much input was lost or how much output was produced, so the failure is reported to the

application, which must decide. A printer daemon might resubmit a job and risk a duplicate page; a failed CD burn must be reported to the user. Recovery is not transparent.

The experimental results are again strong. Over 12,500 trials injecting faults into an Ethernet driver, 347 crashes triggered recovery (226 internal panics, 109 exceptions, 12 missing heartbeats), and the system continued correctly after more than 99.9% of them [Herder et al. 2007]. In the few failures, the network card could not be reinitialised without a BIOS reset because it lacked a master reset command, which is a hardware limit no software design can remove. Recovery-specific code was about 1,072 of 39,011 lines, most of it in the reincarnation server and the file system; drivers needed only a few lines, such as the reply to a heartbeat. A later report described a campaign of 2.4 million injected faults that produced thousands of driver crashes without the system crashing once [Tanenbaum et al. 2010].

15.3.3 Recovery in production systems

The same ideas appear, in narrower forms, in systems people use daily.

Windows recovers display drivers with *Timeout Detection and Recovery* [Microsoft n.d.]. If the GPU delays a preemption request from the GPU scheduler beyond a timeout (two seconds by default), the system treats the graphics stack as hung and, at the default setting, recovers it instead of hanging with it. A second limit guards against storms: by default, more than five such recoveries within sixty seconds crash the computer, on the reasoning that a driver that cannot stay up should not be restarted indefinitely.

Fuchsia runs drivers in *driver host* processes. A driver's component manifest can set `host_restart_on_crash`, which tells the driver framework to restart the driver host when it crashes; without it, a crashed driver's node is removed from the device topology [Fuchsia driver runner n.d.]. QNX, a microkernel system whose drivers and services are user-mode resource managers, provides a High Availability Manager that attaches to a process, watches for conditions such as its death, and performs actions such as restarting it [QNX n.d.].

Table 15.1 compares the systems of this section with PicoCalc OS.

Table 15.1. Driver isolation and recovery in six systems. 'Transparent' means applications see at most a delay, not an error.

System	Isolation	Detection	Recovery	Driver state after restart	Transparent to applications	Cost reported
Nooks [Swift et al. 2003]	kernel protection domains, tracked kernel objects	faults in the domain	release resources, unload, reload	lost	not by itself	0 to just over 60%
Shadow drivers [Swift et al. 2004]	Nooks	Nooks, plus manual triggering	shadow restarts the driver, replays configuration, resubmits requests	rebuilt from the shadow's log	yes, for all automatically detected failures	99% of native on average
SafeDrive [Zhou et al. 2006]	run-time type and bounds checks in the driver	a failed check	restore the kernel invariants its wrappers tracked	—	44 of 44 injected crashes recovered	4–23% kernel CPU
MINIX 3 [Herder et al. 2007]	user-mode processes, message passing	exit or exception, heartbeats, complaints, explicit	reincarnation server, policy script	rebuilt by clients (INET, file server)	network and block yes, character no	recovery code 1,072 of 39,011 lines
Windows TDR [Microsoft n.d.]	none beyond the kernel	GPU preemption timeout, 2 s	recover on timeout (the default level)	lost	—	5 recoveries in 60 s, then crash
PicoCalc OS	separate QuickJS runtime, handles	exception, budget, heap limit, kill	nucleus restarts with backoff; clients react to `worlds.status`	rebuilt at start; framebuffer re-granted by the compositor	keyboard and display yes (after a delay); compositor partly	about 0.1 ms of lane time per restart

15.4 Seven rules for restartable drivers

The systems above succeed and fail for the same reasons, and the reasons can be stated as rules for the driver writer. Each rule below is argued from the literature and then checked against PicoCalc OS's drivers, which obey most of them and break two in instructive ways.

15.4.1 Rule 1: reset the device to a known state on every start

When a driver crashes, the device is in whatever state the dead driver left it: halfway through an initialisation sequence, in the middle of a transfer, configured with a mode the new driver does not know about. A restarted driver that assumes the device is freshly powered on will misbehave. The only safe assumption is that the device's state is unknown, and the only safe response is to put it into a known state before using it: a reset command if the device has one, then the full initialisation. This is crash-only design applied to hardware (Section 14.3): the driver's start path is its recovery path, so it must work from any device state, not only from power-on.

PicoCalc's display driver does exactly this. Its `onStart` is the same at boot and after a crash, and its first command is a software reset:

▸ js/drivers/display.js

```
  .onStart(async (s) => {
    s.spi = hw.spi("spi2", 40e6);   // 40 MHz: 80 MHz worked once but ghosted on the
removable display
    await s.spi.cmd(SWRESET);
    await sleep(150);
    await s.spi.cmd(SLPOUT);
    await sleep(150);
    await s.spi.cmd(COLMOD, [0x55]);  // 16 bits per pixel, RGB565
    await sleep(10);
    await s.spi.cmd(MADCTL, [0x48]);  // column order mirrored (MX), BGR panel
    await s.spi.cmd(INVON);           // this panel shows true colours with inversion on
    await sleep(10);
    await s.spi.cmd(DISPON);
    await sleep(50);
    s.ready = true;
  })
```

`SWRESET` returns the ST7365P's registers to their defaults whatever the old driver did, and the rest of the sequence configures it from there. The cost is the sequence's waits: 370 ms, the largest part of the display driver's recovery time (Section 15.6). That cost is the price of the rule, and it is worth paying; MINIX 3's one class of unrecoverable network failures came from a card that had no master reset command, so its driver could not bring it to a known state [Herder et al. 2007].

The keyboard driver does not reset its device, and that is a deliberate exception. The STM32 keyboard controller keeps a 31-entry FIFO of key events, and a register write could reset it (the register map the simulator models after clockworkpi's firmware includes a reset register), but resetting it would discard keys the user pressed while the driver was down. The driver's start sequence only claims the bus and waits the 100 ms the controller needs after its own reset. This works because the controller is a separate microcontroller with its own firmware, whose state does not depend on what the host driver did: the driver's crash cannot leave it half-configured, because the driver never configures it. The rule is really "know the device's state before using it", and for the keyboard controller its state is always known.

15.4.2 Rule 2: hold only state that can be rebuilt; let clients own intent

A restarted driver has lost everything in its memory. If that memory held something only the driver knew, recovery cannot restore it; something else must hold it. The systems differ in what that something is. The shadow driver keeps a log of the configuration calls and outstanding requests [Swift et al. 2004]. MINIX 3 makes the client responsible: INET reconfigures a restarted network driver, and the file server reissues block requests [Herder et al. 2007]. Crash-only design asks for dedicated state stores [Candea and Fox 2003]. All three say the same thing: the driver should hold the *device*, and someone else should hold the *intent*, meaning what the device is supposed to be doing.

PicoCalc's display driver holds almost nothing: a flag saying the panel is initialised, its SPI handle, counters, and a read-only handle to the framebuffer. The framebuffer, which is the only thing that determines what the screen should show, belongs to the compositor. When the display driver restarts, the compositor gives it the framebuffer again and invalidates its own record of what the panel shows, so the next flush sends every pixel:

▸ js/system/compositor.js

```
function attach(s) {
  // the view handed to a previous display instance died with it; let go of our hold on
it, or every
  // restart of the driver would keep one more handle alive
  if (s.view) s.view.release();
  s.view = s.fb.narrow("r");
  send("display", "attach", { fb: s.view });
  s.fb.invalidate();
  dirty(s);
}
```

▸ js/system/compositor.js

```
    } else if (st.state === "running") {
      s.banner = { text: `${st.world} is running again (restart ${st.restarts})`, color:
C.ok, until: now() + 1500 };
      if (st.world === "display") attach(s);   // a fresh panel: give it the framebuffer
and send every pixel
    }
```

This is the arrangement the shadow-driver paper recommends for device state it cannot copy: the client recreates it, by redrawing the frame buffer [Swift et al. 2004]. It is also the reason the display driver's recovery is complete.

The compositor is the counter-example. It is a service rather than a driver, but it is the display driver's client and it holds state that nothing else holds: which app is in front, the last frame each app published, and the last battery reading. When the compositor restarts, all of that is gone, and Section 15.7 shows what the user sees as a result.

15.4.3 Rule 3: make requests idempotent, or know which ones are not

When a driver dies with a request in progress, the request's fate is unknown to its sender. It may have reached the device and completed, reached the device and half-completed, or never reached it. A client that retries risks doing the work twice; a client that does not risks never doing it. The way out is to make requests *idempotent*, so that doing them twice is the same as doing them once; then a retry is always safe. MINIX 3 recovers block drivers transparently because block I/O is idempotent, and does not recover character drivers transparently because their I/O is not [Herder et al. 2007]. Crash-only design asks for requests

that say whether they are idempotent so that the system knows when it may retry [Candea and Fox 2003]. The shadow-driver paper names the limit: in general, a request cannot be guaranteed to execute exactly once [Swift et al. 2004].

The display driver's requests are idempotent. `attach` replaces the framebuffer handle, releasing the old one only if it is different, so a repeated attach changes nothing. A `flush` sends the current contents of a rectangle of the framebuffer, and sending the same pixels twice leaves the panel the same; better, the compositor answers any doubt about which flushes arrived by resending the whole screen (Section 14.8), which is idempotent by construction.

The keyboard driver's reads are not idempotent, because reading the controller's FIFO register removes the event it returns:

▸ sim/stm32kbd.c

```
      case REG_FIF:
        if (S.count) { answer[0] = S.fifo[S.head].state; answer[1] = S.fifo[S.head].key;
S.head = (S.head + 1) % FIFO_SIZE; S.count--; }
        break;
```

The read is a destructive operation on the device, and the driver publishes the event only after the read's result has arrived, in a later segment. If the driver dies between the two, the event has been taken out of the FIFO and never published. Section 15.5 measures the window. Keys are therefore delivered *at most once*: never twice, but occasionally not at all. For a keyboard that is the right side of the trade-off (a lost key is an annoyance; a doubled one types the wrong command), but it should be a decision, not an accident.

15.4.4 Rule 4: re-grant authority; never inherit it

A driver's authority over its device must not outlive the driver instance. If a dead instance's permissions, mappings or pending operations remain live, then a stale piece of the old instance (an in-flight DMA, a timer callback, a thread that had not yet noticed the crash) can act on the device while the new instance is initialising it. Two instances driving one device is the driver version of *split brain*, and it produces exactly the corruption recovery was supposed to prevent. The shadow driver masks the device's interrupt and removes its memory mappings before restarting the driver for this reason [Swift et al. 2004].

In PicoCalc OS this rule is enforced by the nucleus. When a world crashes, `crash_at` frees its runtime (so no code of the old instance can run again), discards its pending effects (so nothing it asked for in its last segment happens), drops all its handles, and advances its epoch so that every completion addressed to the

old instance is ignored (Section 14.4 and Section 14.5). Dropping a handle uses the same reference counting as releasing it:

▸ nucleus/hw.c

```
void handle_release(handle_t *h, world_t *W) {
  h->holders &= ~bit(W);
  while (h && !h->holders && !has_children(h->id)) {
    int parent = h->parent;
    buf_unref(h->buf);
    h->live = false;
    h = parent ? find(parent) : NULL;
  }
}

void handles_drop_world(world_t *W) {
  for (int i = 0; i < MAX_HANDLES; i++) if (K.handles[i].live && (K.handles[i].holders &
bit(W))) handle_release(&K.handles[i], W);
}
```

The new instance starts with no handles at all. It regains authority in two ways, and the difference between them is the point of the rule. Authority that comes from the manifest is regained by claiming it again: the display driver calls `hw.spi("spi2", 40e6)` and gets a new handle, because its manifest entry still lists the claim. Authority that was delegated by another world must be delegated again: the display driver can read the framebuffer only after the compositor sends it a newly narrowed handle in an `attach`. The compositor decides whether the new instance should have it, which is the right place for the decision, because the compositor owns the framebuffer.

One thing does outlive the old instance: a bus transfer that one of its *completed* segments started. Such a transfer is already on the wire and finishes, but the bus serialises it ahead of anything the new instance sends, and its completion is dropped by the epoch check. The new instance's first command, `SWRESET`, therefore reaches the panel after the old instance's last transfer, which is the order Rule 1 needs.

15.4.5 Rule 5: detect failure in more than one way

Recovery begins with detection, and a failure that is not detected is not recovered. Crashes are the easy case. Drivers also fail by *hanging*: waiting for an interrupt that never comes, looping on a status register, deadlocking. The production systems detect hangs with timeouts (TDR's two seconds [Microsoft n.d.]) and the research systems with heartbeats (MINIX 3's reincarnation server, which caught 12 of its 347 recoveries

that way [Herder et al. 2007]). Drivers also fail by doing the wrong thing without stopping, which no generic detector can see; MINIX 3 relies on complaints from the driver's clients, which know what correct behaviour looks like [Herder et al. 2007].

PicoCalc OS detects exceptions, rejected promises, budget overruns, heap exhaustion and explicit kills (Table 14.1). Its budget catches a driver that loops inside a segment, because the segment overruns. It does not catch a driver that hangs *between* segments: an `async` handler awaiting a promise that never settles keeps its turn open and its mailbox closed indefinitely, and no segment ever overruns. In this system every bus transfer and every sleep does settle, so a driver cannot hang that way by accident, but a bug that awaited the wrong thing could. Its clients would see the hang as asks timing out (the compositor's flushes time out after 1.5 s, and the compositor then invalidates the screen and tries again), but nothing would restart the driver. A nucleus-enforced deadline on a whole turn, or a heartbeat ask from a supervising world with `sys.admin`, would close the gap; Exercise 9 asks you to add one. The first pass, a later revision described in a separate book, *From L4 to Small Worlds* (Chapter 6), added the first: a turn still unfinished `turn_ms` after its first segment started (8,000 ms unless the manifest says otherwise) is a crash, and the supervisor restarts the world. It catches a driver that awaits the wrong thing, though only after eight seconds by default; it does not catch a silent device, because the keyboard driver catches every bus error and its turns end.

15.4.6 Rule 6: contain restart storms

A driver that fails deterministically, on every start or on every request, will be restarted forever by a naive supervisor, and each restart of a driver costs more than a restart of an ordinary component: the device's initialisation sequence, its clients' reconfiguration, and in PicoCalc's case a full-screen transfer for every display restart. Section 14.6 covered backoff and restart limits in general; for drivers the production systems' numbers are telling. Windows gives a display driver five recoveries in sixty seconds before deciding the system cannot continue [Microsoft n.d.]. PicoCalc OS gives any world five crashes in ten seconds and then stops it, leaving the rest of the system running. For a display driver, being stopped means a frozen screen on a working system: worse for the user than a working screen, better than a reboot loop, and recoverable by a world with `sys.admin` once the cause is fixed.

15.4.7 Rule 7: test recovery by injecting faults

Recovery code runs only when something fails, so it will be wrong unless failures are made to happen deliberately and often. Every system in Section 15.3 was evaluated by fault injection: 2,000 trials for Nooks [Swift et al. 2003], 2,100 for shadow drivers [Swift et al. 2004], 12,500 for MINIX 3's network driver and later millions [Herder et al. 2007; Tanenbaum et al. 2010]. Fault injection is also how their authors discovered the limits they report.

PicoCalc OS offers three ways to inject faults. The Faults app kills the keyboard and display drivers with `sys.fault`. The host build's `crash WORLD` command kills any world at an exact instant of virtual time. And the benchmark suite's recovery experiment kills the keyboard, display and compositor on every run and measures how long each takes to recover. Because the host build is deterministic (Chapter 18), a kill at a given virtual instant produces the same trace on every run, so a recovery bug found once can be reproduced as often as needed. That determinism is how the lost-key window of Section 15.5 was found: by running the same key press with the kill moved a few milliseconds at a time.

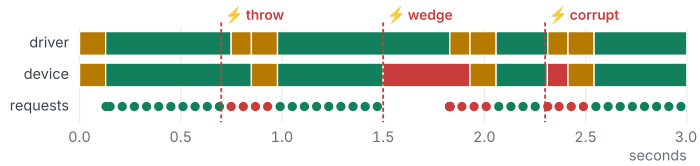
TRY 15.1

Break a driver and choose how it heals

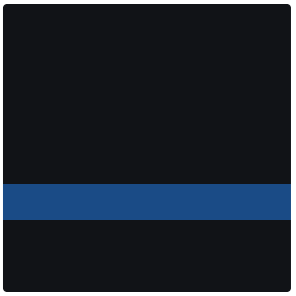
This simulation runs a device, a driver in a Web Worker and a client that issues requests, and injects three faults on a schedule: the driver throws at 0.7 s, the device wedges at 1.5 s, and the driver corrupts its own state at 2.3 s. A supervisor on the page restarts the driver after each, and a watchdog kills a driver that hangs. Compare the three recovery strategies. With *restart*, the new driver resets the device, so requests in flight when the old one died fail and the configuration the client had set is lost. With *replay*, the client replays its unacknowledged requests to the new driver, which recovers them but not the configuration. With *shadow*, a shadow records both the configuration and the outstanding requests and replays them after the restart, as Swift's shadow drivers do. Which strategy leaves the device configured as the client left it? Then switch the device to the keyboard with *replay*: what happens to keys that were waiting in the FIFO when the new driver resets the controller?

display, strategy “restart”: a recorded run on a virtual clock with faults at 700 ms (throw), 1500 ms (wedge), 2300 ms (corrupt).

ready / completed starting or restarting
crashed, hung, wedged / failed
ring: completed after a replay



THE DISPLAY



state ready
brightness 50 (client set 85)

issued 49 completed 32 failed 17 replayed by the client 0
replayed by the shadow 0 last recovery 242 ms

0.000	↻	driver starts (epoch 1) and resets the device
0.129	✓	driver ready
0.700	⚡	a bug: the driver will throw on its next request

0.749	×	driver crashed: injected bug: index 64 out of range. The kernel terminates it; a new one starts in 100 ms.
0.849	↻	driver starts (epoch 2) and resets the device
0.978	✓	driver ready 278 ms after the fault
1.500	⚡	the device wedges: it stops answering
1.829	×	driver is hung: no reply within 300 ms: the watchdog declares it hung. The kernel terminates it; a new one starts in 100 ms.
1.929	↻	driver starts (epoch 3) and resets the device
2.058	✓	driver ready 558 ms after the fault
2.300	⚡	a bug: the driver will send the device a stray command
2.311	×	the device accepted a stray command and is now in an error state
2.313	×	driver crashed: device error: status 0x80. The kernel terminates it; a new one starts in 100 ms.
2.413	↻	driver starts (epoch 4) and resets the device
2.542	✓	driver ready 242 ms after the fault

15.5 Case study: the keyboard driver

The keyboard driver polls the STM32 every 20 ms over I2C at 10 kHz. Each poll reads a status register to learn how many events are in the FIFO, then reads the FIFO register once per event and publishes each event on the `keys` channel. The benchmark experiment kills the driver and presses the right-arrow key in the same instant, then measures when the key reaches the launcher. The result is 213 ms, and the trace shows where every millisecond went.

▸ book/traces/15-keyboard-restart.jsonl


```
{"k":"crash","t":1500000,"w":"keyboard","text":"killed from the host"}
{"k":"status","t":1500000,"w":"keyboard","text":"crashed"}
{"k":"key","t":1500000,"w":"","text":"0xb7 down"}
…
{"k":"status","t":1600078,"w":"keyboard","text":"running"}
{"k":"seg","w":"keyboard","t":1600000,"dt":78,"steps":55,"native":2,"kind":"boot","type":
"start"}
{"k":"seg","w":"keyboard","t":1600078,"dt":62,"steps":30,"native":3,"kind":"msg","type":"
start"}
…
{"k":"seg","w":"keyboard","t":1700140,"dt":49,"steps":13,"native":1,"kind":"settle","type
":"start"}
{"k":"seg","w":"keyboard","t":1700189,"dt":59,"steps":26,"native":2,"kind":"msg","type":"
every 20 ms"}
{"k":"bus","bus":"i2c0","w":"keyboard","t":1700248,"t1":1706248,"bytes":4,"what":"write
0x04, wait, read 1"}
{"k":"seg","w":"keyboard","t":1706248,"dt":50,"steps":14,"native":1,"kind":"settle","type
":"every 20 ms"}
{"k":"bus","bus":"i2c0","w":"keyboard","t":1706298,"t1":1713198,"bytes":5,"what":"write
0x09, wait, read 2"}
{"k":"seg","w":"keyboard","t":1713198,"dt":73,"steps":46,"native":3,"kind":"settle","type
":"every 20 ms"}
{"k":"msg","t":1713271,"from":"keyboard","to":"compositor","type":"keys","ask":false,"byt
es":62}
…
{"k":"msg","t":1713351,"from":"compositor","to":"launcher","type":"key","ask":false,"byte
s":62}
```


Read it as a sequence of waits.

1. **Backoff, 100 ms.** The driver was killed at 1,500.000 ms; its first crash in ten seconds schedules a restart 100 ms later. The key was pressed at the same instant and went into the STM32's FIFO, where it waits.
2. **Boot, 0.08 ms.** At 1,600.000 the boot segment creates a fresh runtime and evaluates the prelude and `keyboard.js` in 78 µs, and the nucleus publishes that the keyboard is running.
3. **Start-up wait, 100 ms.** The `start` turn claims the I2C device and sleeps 100 ms, as the controller requires after its own reset. The driver's periodic timer was started at boot and became due at 1,620 ms, but its ticks wait in the mailbox, which is closed for the whole asynchronous start turn; ticks of the same timer coalesce, so only one is waiting when the turn ends at 1,700.189.
4. **The poll, 13 ms.** The first poll reads the status register (6.0 ms on the bus: the register number written, a 2 ms gap with the bus held, one byte read, all at 10 kHz), learns there are two events, and reads the first

from the FIFO (6.9 ms).

5. **Delivery, 0.15 ms.** The settle segment at 1,713.198 publishes the key-down event; the compositor forwards it to the launcher at 1,713.351.

The total is 213.351 ms. Nothing was lost, because the device buffered the event across the driver's death; Rule 1's exception (do not reset the keyboard controller) and Rule 2's principle (the driver holds nothing the device does not) together made this recovery complete. The 213 ms is two deliberate waits and one poll, and Table 15.2 shows which could be shortened.

Table 15.2. Where the keyboard driver's 213 ms of recovery goes, from the trace.

Stage	Virtual time	Set by
Backoff before the first restart	100.000 ms	supervision policy (`crash_at`)
Boot segment: new runtime, prelude, `keyboard.js`	0.078 ms	interpreter work on the virtual clock
Start turn: claim the bus, wait for the controller	100.111 ms	the driver (`sleep(100)` in `onStart`)
First poll: status read and one FIFO read	13.009 ms	the 10 kHz I2C bus
Publish and forward to the launcher	0.153 ms	the compositor's routing
Total	**213.351 ms**	

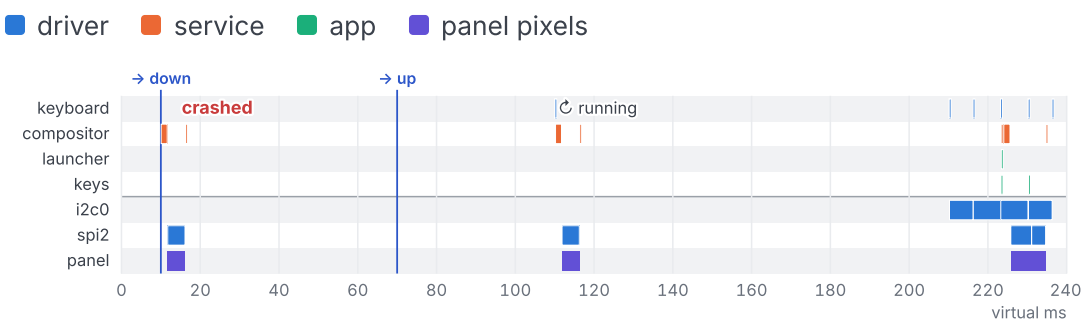
TRY 15.2

Follow a key through the keyboard driver's restart

The trace below starts 10 ms before the kill. Find the 100 ms gap before the boot segment, the second 100 ms gap inside the start turn, and the two I2C transfers of the first poll. Switch to the event list and find the moment the launcher receives the key.

Keyboard driver killed, key pressed at once 240 virtual ms

The key waits in the STM32's FIFO through the restart and reaches the launcher 213 ms later.



15.5.1 The lost-key window

The same experiment with the kill moved shows the cost of Rule 3's non-idempotent read. Press the key while the driver is alive, and kill the driver 10 ms later:

▸ book/traces/15-keyboard-lost-key.jsonl


```
{"k":"key","t":1500000,"w":"","text":"0xb7 down"}
{"k":"key","t":1500000,"w":"","text":"0xb7 up"}
…
{"k":"bus","bus":"i2c0","w":"keyboard","t":1506187,"t1":1513087,"bytes":5,"what":"write 0x09, wait, read 2"}
{"k":"crash","t":1510000,"w":"keyboard","text":"killed from the host"}
…
{"k":"msg","t":1723274,"from":"keyboard","to":"compositor","type":"keys","ask":false,"bytes":60}
```

The driver's poll had already read the status register and started the FIFO read at 1,506.187 ms. The bus transfer had popped the key-down event off the controller's FIFO when the driver was killed at 1,510.000, 3 ms before the transfer completed. The transfer finished on the wire, as Rule 4 says it must, but its completion carried the old epoch and was dropped. After the restart, the new driver found one event in the FIFO, the key-up, and published it (60 bytes, two fewer than a key-down event because its state is "up" rather than "down"). The compositor ignores key-up events, so the launcher received nothing: the key press was lost.

Figure 15.1 shows the window. It opens when a FIFO read is applied to the device and closes when the segment that publishes its result ends: about 7 ms per event at 10 kHz. Moving the kill to 15 ms after the key press, after the publish, delivers the key normally. A driver that wanted at-least-once delivery would have to read without removing (a peek register, if the controller had one) and remove only after publishing; the STM32's firmware offers no such register, so with this device at-most-once is the best available, and the window is the price.

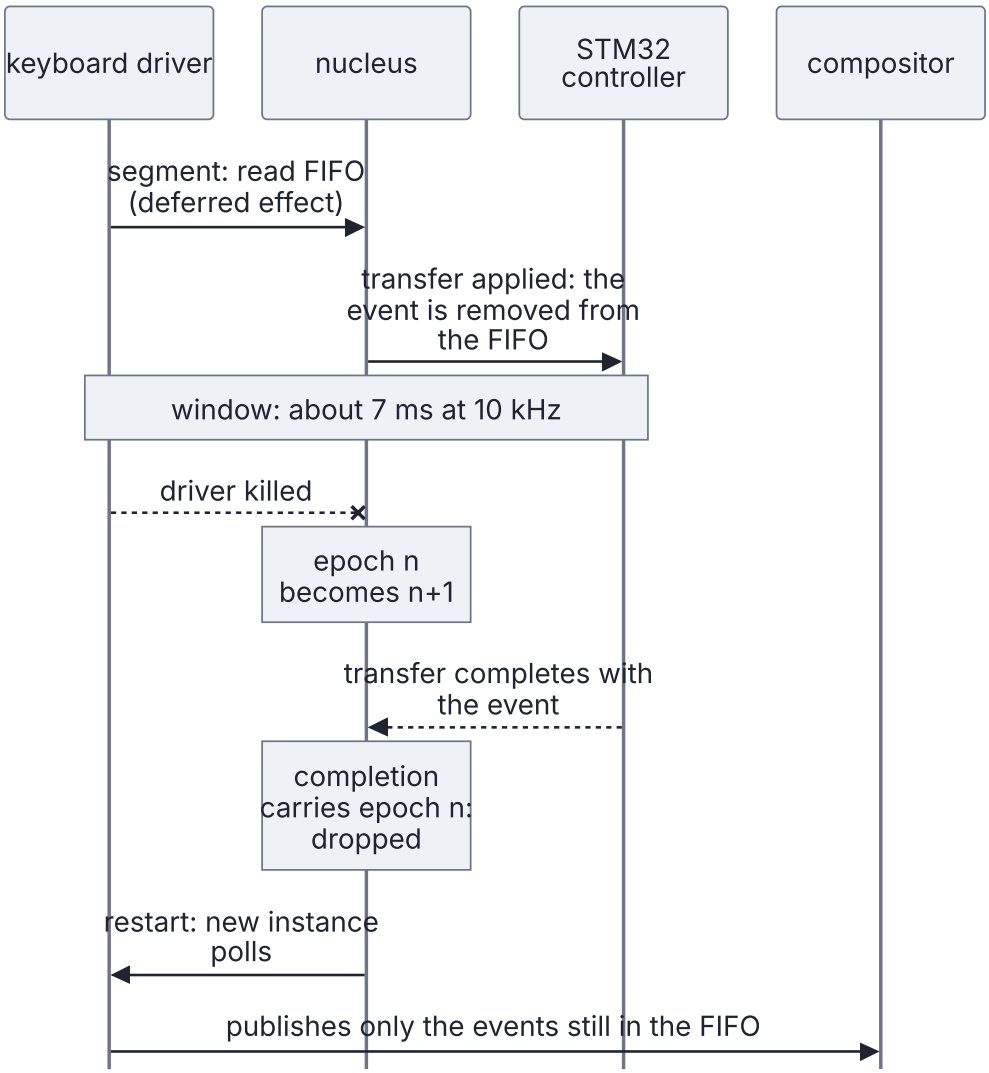


Figure 15.1. The window in which a key can be lost. The FIFO read removes the event from the controller when it is applied; the event is published only in the segment that receives the read's result. A crash between the two drops the result by its epoch, and the event is gone.

15.6 Case study: the display driver

The display driver's recovery is longer and more interesting, because it involves a device that must be re-initialised, a client that must re-grant authority, and a message that outlived the instance it was sent to. The benchmark kills the driver while the launcher is on screen and measures two times: when the first changed

pixels reach the panel (481 ms) and when the whole screen is correct again (515 ms). The key lines of the trace are these.

▸ bench/traces/display-restart.jsonl


```
{"k":"crash","t":1870000,"w":"display","text":"killed from the host"}
{"k":"status","t":1870000,"w":"display","text":"crashed"}
{"k":"msg","t":1871654,"from":"compositor","to":"display","type":"flush","ask":true,"byte
s":25}
{"k":"status","t":1970062,"w":"display","text":"running"}
{"k":"seg","w":"display","t":1970000,"dt":62,"steps":31,"native":2,"kind":"boot","type":"
start"}
{"k":"bus","bus":"spi2","w":"display","t":1970125,"t1":1970127,"bytes":1,"what":"cmd
0x01"}
{"k":"bus","bus":"spi2","w":"display","t":2120337,"t1":2120339,"bytes":1,"what":"cmd
0x11"}
{"k":"bus","bus":"spi2","w":"display","t":2290657,"t1":2290659,"bytes":1,"what":"cmd
0x29"}
{"k":"seg","w":"display","t":2340703,"dt":49,"steps":13,"native":1,"kind":"settle","type"
:"start"}
{"k":"seg","w":"display","t":2340752,"dt":83,"steps":63,"native":2,"kind":"msg","type":"f
lush"}
{"k":"seg","w":"compositor","t":2340835,"dt":74,"steps":47,"native":3,"kind":"settle","ty
pe":"draw"}
{"k":"seg","w":"compositor","t":2341103,"dt":102,"steps":84,"native":7,"kind":"msg","type
":"worlds.status"}
{"k":"msg","t":2341205,"from":"compositor","to":"display","type":"attach","ask":false,"by
tes":15}
{"k":"msg","t":2343579,"from":"compositor","to":"display","type":"flush","ask":true,"byte
s":25}
{"k":"px","t":2343948,"t1":2350514,"changed":16383}
{"k":"bus","bus":"spi2","w":"display","t":2343948,"t1":2384994,"bytes":204800,"what":"pix
els 320x320 at 0,0"}
```


The sequence, which Figure 15.2 draws, has five phases.

The crash and a message for the dead. At 1,870.000 ms the driver is killed. The nucleus publishes the crash, the compositor draws a banner announcing it, and at 1,871.654 asks the display driver to flush the banner. The display world is restarting, but its mailbox survives (Section 14.4), so the flush is delivered there and waits. The compositor's draw turn is now awaiting the reply, and because the mailbox of a world in an asynchronous turn stays closed, the compositor will handle nothing else until the reply comes.

The restart. After the 100 ms backoff the display world boots at 1,970.000, and the nucleus publishes that it is running. The `start` message is at the front of its mailbox, ahead of the old flush, so `onStart` runs first.

Re-initialising the panel. The start turn sends the ST7365P its initialisation sequence: `SWRESET` (0x01) at 1,970.125, `SLPOUT` (0x11) 150 ms later, then `COLMOD`, `MADCTL`, `INVON` and `DISPON` (0x29) with their waits, finishing at 2,340.703. This is Rule 1, and it costs 370 ms.

The stale flush. The next message in the mailbox is the flush the compositor sent to the dead instance. The new instance has no framebuffer yet, because the compositor has not re-granted it. Here the simulator found a real bug while the system was being built: the first version of the driver used the missing framebuffer and threw, so the display driver crashed a second time as it came back. The fix is the first line of the flush handler:

▸ `js/drivers/display.js`

```
  .on("flush", async (s, { rects }) => {
    // after a restart, flushes queued for the old instance can arrive before the
compositor re-attaches
    if (!s.fb || !s.ready) return { px: 0, skipped: true };
```

The driver answers `skipped` at 2,340.752, and the compositor, whose draw turn has been waiting for this reply for 469 ms, invalidates its framebuffer so that the next flush will send everything.

Re-granting and redrawing. Now that its draw turn has ended, the compositor handles the messages that queued up behind it, including the `worlds.status` message saying the display driver is running. It lets go of the view it gave the dead instance, sends a fresh read-only framebuffer handle in an `attach` at 2,341.205 (Rule 4), draws the screen, and asks for a flush of the whole screen at 2,343.579. The 204,800 bytes take 41 ms on the 40 MHz SPI bus; the first 32 KiB chunk lands at 2,350.514 (480.5 ms after the kill) and the last at 2,384.994 (515.0 ms).

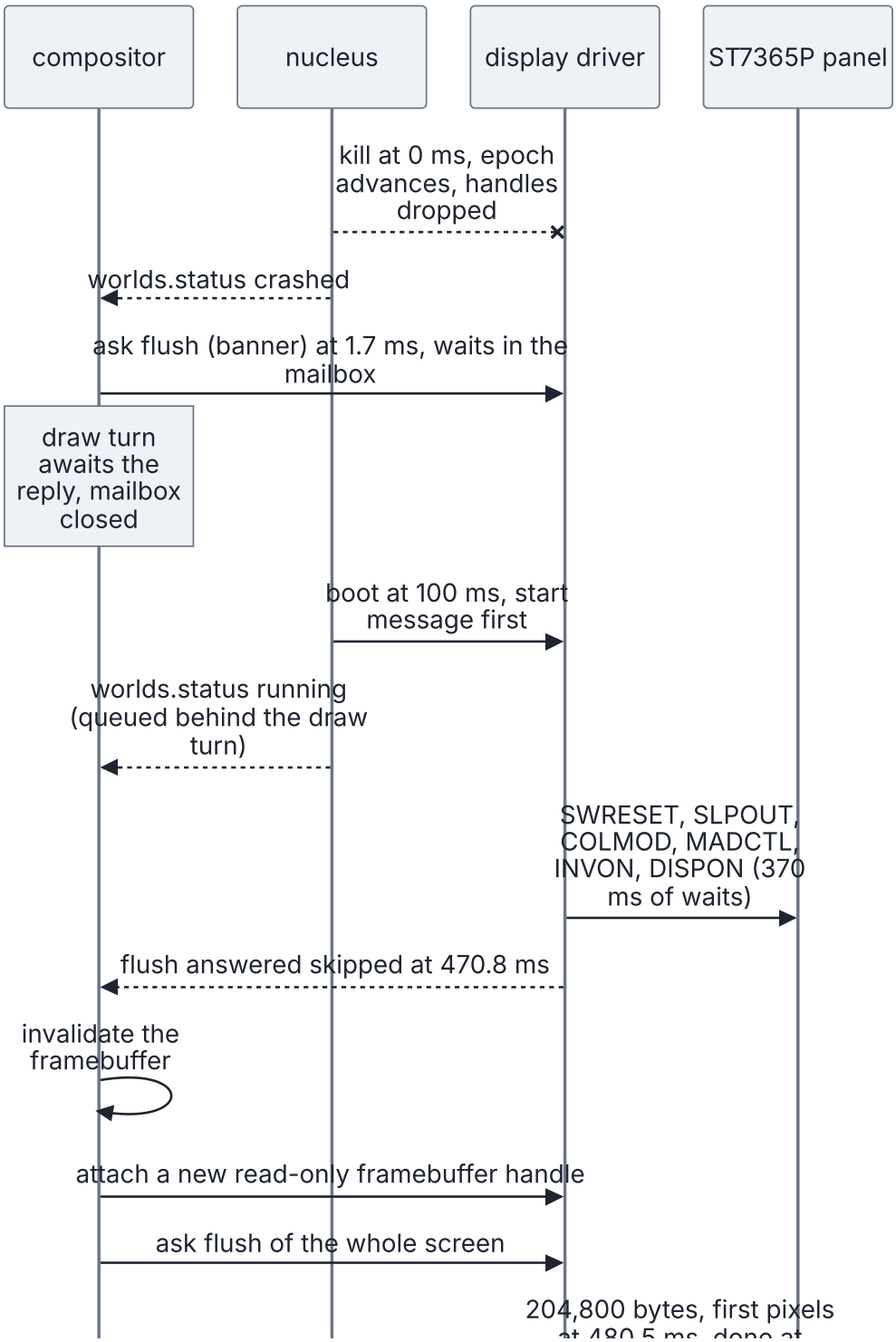

compositor
nucleus
display driver
ST7365P panel
kill at 0 ms, epoch advances, handles dropped
worlds.status crashed
ask flush (banner) at 1.7 ms, waits in the mailbox
draw turn awaits the reply, mailbox closed
boot at 100 ms, start message first
worlds.status running (queued behind the draw turn)
SWRESET, SLPOUT, COLMOD, MADCTL, INVON, DISPON (370 ms of waits)
flush answered skipped at 470.8 ms
invalidate the framebuffer
attach a new read-only framebuffer handle
ask flush of the whole screen
204,800 bytes, first pixels



Figure 15.2. The display driver's recovery. The flush sent to the dead instance waits in the mailbox through the restart and is answered 'skipped'; the compositor then re-grants the framebuffer and resends every pixel. Times are virtual milliseconds after the kill.

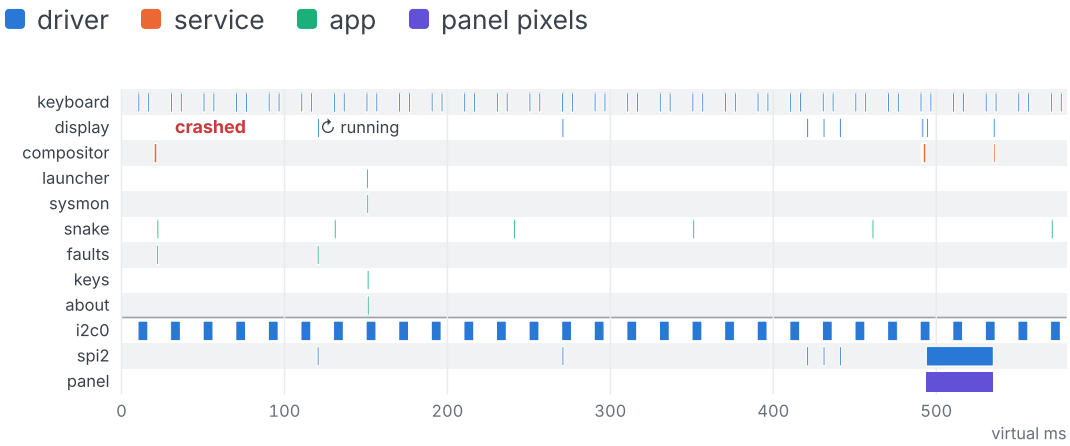
TRY 15.3

Watch the display driver come back

This is the full trace of the display driver's recovery. Zoom into the first two milliseconds after the kill to see the compositor's banner flush go to the dead driver, then into the start-up sequence to see the six commands spaced by the panel's required waits, and finally into the full-screen transfer at the end.

The display driver killed 580 virtual ms

The display driver is killed. It restarts after 100 ms and spends 370 ms re-initialising the panel; the compositor then hands it the framebuffer again and resends every pixel.



The trace also exposes a cost the benchmark numbers hide. For 469 ms the compositor was inside one turn, waiting for a reply from a world that was busy re-initialising a panel. Every app frame, every status message and every timer tick addressed to the compositor in that time waited too. The screen could not have been updated anyway, so in this case nothing was lost, but the same structure would block the compositor

for the full 1.5 s ask timeout if the display driver hung rather than crashed. A compositor that sent flushes without awaiting them in the same turn, and handled their replies as separate messages, would keep processing during a display outage; that redesign is Exercise 11.

15.7 Case study: the compositor

The compositor is not a driver, but its recovery completes the picture, because it is the display driver's client and the owner of the framebuffer, and because it breaks Rule 2. Its state is the focused app, the last frame each app published, the battery level from the power driver, and the framebuffer itself.

Killing it while the launcher is on screen gives first pixels after 108 ms and a complete screen after 167 ms. The trace shows the framebuffer changing hands:

▸ book/traces/15-compositor-restart.jsonl


```
{"k":"crash","t":1870000,"w":"compositor","text":"killed from the host"}
{"k":"seg","w":"compositor","t":1970000,"dt":69,"steps":41,"native":2,"kind":"boot","type
":"start"}
{"k":"seg","w":"compositor","t":1970069,"dt":93,"steps":71,"native":6,"kind":"msg","type"
:"start"}
{"k":"msg","t":1970162,"from":"compositor","to":"display","type":"attach","ask":false,"by
tes":15}
{"k":"seg","w":"display","t":1970162,"dt":69,"steps":41,"native":2,"kind":"msg","type":"a
ttach"}
{"k":"seg","w":"compositor","t":1970564,"dt":650,"steps":254,"native":442,"kind":"msg","t
ype":"draw"}
{"k":"px","t":1971583,"t1":1978149,"changed":6872}
{"k":"bus","bus":"spi2","w":"display","t":1971583,"t1":2012629,"bytes":204800,"what":"pix
els 320x320 at 0,0"}
{"k":"msg","t":2000556,"from":"launcher","to":"compositor","type":"frames","ask":false,"b
ytes":1146}
…
{"k":"px","t":2036348,"t1":2037412,"changed":419}
```


When the compositor crashes, its handle to the framebuffer is dropped. The display driver still holds the read-only handle the compositor narrowed for it, and because a DMA buffer is freed only when no handle to it remains, the old buffer stays allocated, and the panel goes on showing the last image. When the compositor restarts at 1,970.000 its `onStart` allocates a *new* framebuffer and attaches it; the display driver's `attach` handler releases the old handle, which was the last reference, and the old buffer is freed. Figure 15.3 shows the hand-over.

▸ js/drivers/display.js

```
  .on("attach", (s, { fb }) => {
    if (s.fb && s.fb !== fb) s.fb.release();
    s.fb = fb;
  })
```

The first draw, at 1,970.563, is a draw with almost no state: the compositor knows which app should be in front only because its initial state says the launcher, it has no frame from any app, and it has no battery reading. It draws the status bar, a placeholder where the launcher's grid belongs, the hint bar, and a banner saying that the compositor is running again (it learned of its own crash and restart from two `worlds.status` messages that waited in its mailbox through the restart), and sends the whole screen; the first chunk lands at 1,978.148 (108 ms after the kill). Only 6,872 pixels of that chunk actually changed, because the panel still showed the old picture. Then, at 2,000.556, the launcher publishes a frame, the compositor draws the grid, and the screen is complete at 2,037.411 (167 ms).

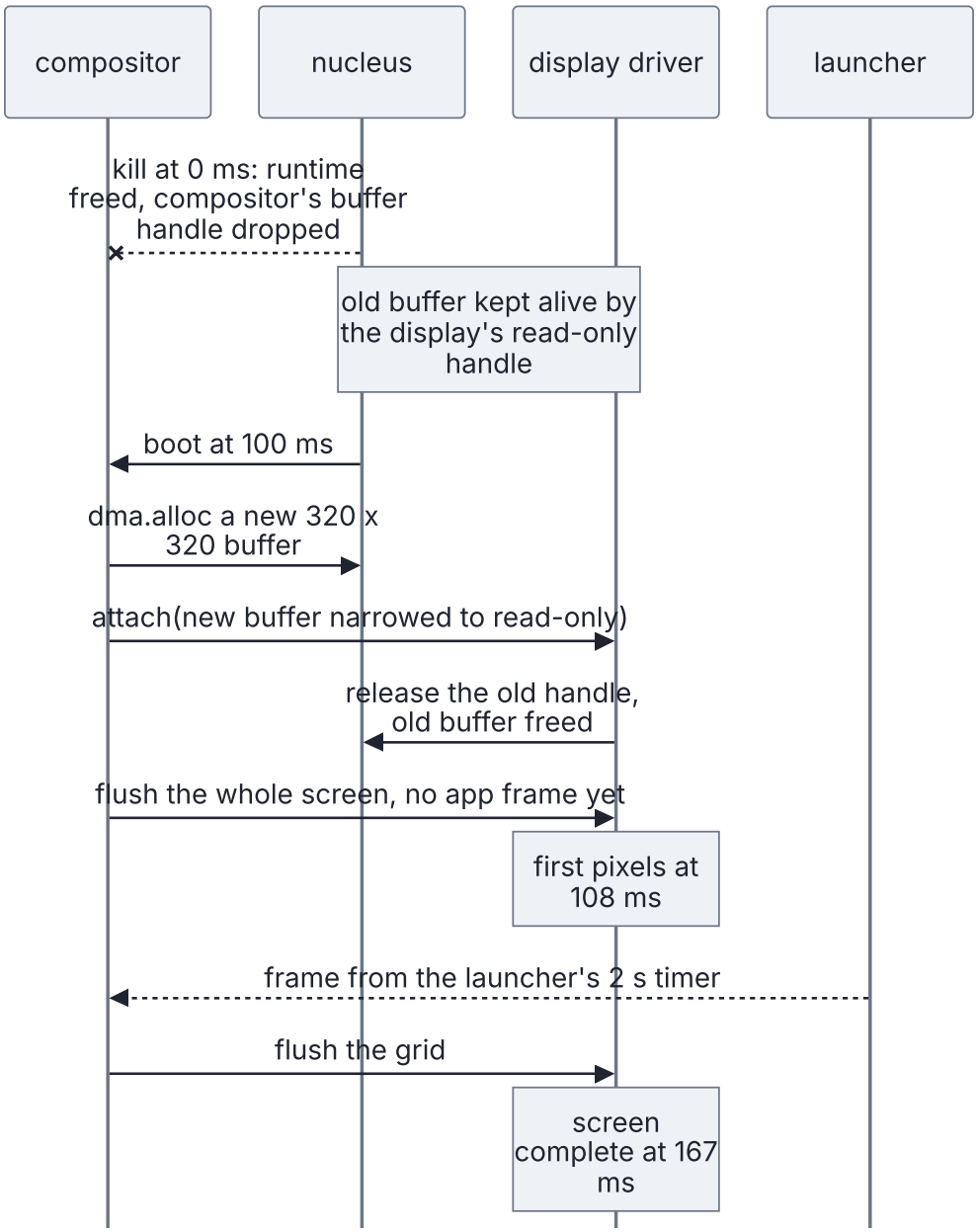


Figure 15.3. The compositor's restart and the framebuffer hand-over. The display driver's read-only handle keeps the old buffer alive until the new compositor attaches a new one; then the old buffer's last reference goes and it is freed.

The launcher's frame at 2,000.556 is the weak point of this recovery. Nothing asked the launcher to publish it. The launcher refreshes its count of running worlds every two seconds, and its frame happened to change because the count of crashes since boot had gone from 0 to 1; the nucleus publishes a frame only when it differs from the app's previous one, and that cache is the only place a copy of each app's last frame survives. Had the compositor crashed just after the launcher's tick, the grid would have been missing for up to two seconds.

With a different app in front, the losses are worse and visible. Figure 15.4 shows the REPL before and after the compositor is killed. After the restart the user is on the home screen, not in the REPL, because the compositor's initial state says the launcher is in front and nothing told it otherwise; the REPL world itself is untouched, still running and still holding its history, but no longer on the screen. The battery indicator is empty, and stays empty: the power driver publishes the battery level only when it changes, so a subscriber that restarts has no way to learn the current value until the battery level next moves. Twelve seconds after the restart, in the host build, it was still empty.

▸ history/first-pass/js/drivers/power.js

```
async function sample(s) {
  const [, v] = await readReg(s.dev, REG_BAT);
  const pct = v & 0x7f, charging = !!(v & 0x80);
  if (pct !== s.pct || charging !== s.charging) {
    s.pct = pct; s.charging = charging;
    publish("power", { pct, charging, backlight: s.backlight, screen: s.screen });
  }
}
```



Figure 15.4. The compositor's restart as the user sees it, captured from the host build's panel. Left: the REPL before. Middle: three seconds after the compositor was killed; the home screen is in front and the battery level is gone. Right: after pressing up, the key goes to the launcher, not the REPL.

Every one of these losses is a Rule 2 violation of the same shape: the compositor held state that only it held, and the worlds that could have rebuilt it (the apps, the power driver) were not designed to be asked. There are three ways to repair it, and they correspond to the three answers of Section 15.4.2.

- **A state store**, after crash-only design and MINIX 3's data store: the compositor writes the focused app to a store outside itself and reads it back in `onStart`.
- **Retained messages**: the nucleus already keeps each app's last frame to decide whether a new frame differs. It could deliver the retained last message of each channel to a world that subscribes after a restart, as publish–subscribe systems often do, so a restarted compositor would receive every app's current frame and the current battery level at once.
- **Client-driven rebuild**, after MINIX 3's INET and file server: the compositor, on start, asks each app and the power driver to publish again. This needs a new message in every world's protocol, and a world that does not implement it remains a gap.

Exercise 12 asks you to implement the second, which fixes every case with one change in the nucleus.

The compositor killed 185 virtual ms

A new framebuffer is attached at 100 ms; the screen is complete when the launcher next publishes a frame.

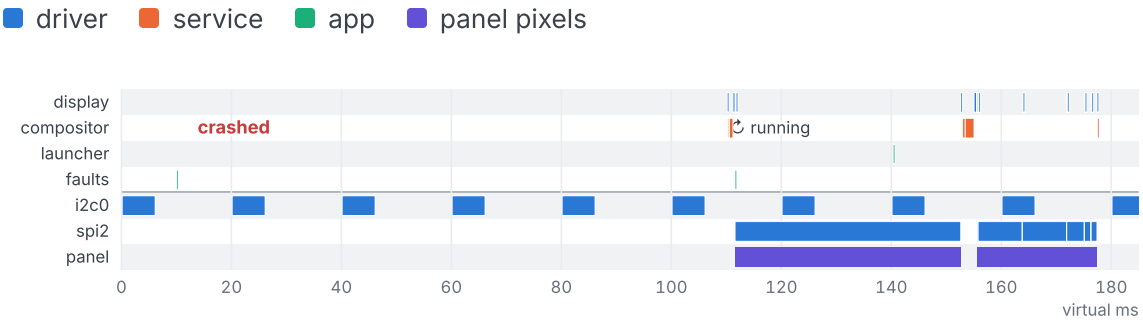


Figure 15.5. The compositor's restart in the trace: the kill, the boot 100 ms later, the attach to the display driver, the first full-screen flush, and the launcher's frame that completes the screen.

15.8 What cannot be healed

Restarting a driver cures a large class of failures, and the case studies show it working. It is worth being as precise about the classes it does not cure.

Deterministic faults. A driver that fails on every start fails again after every restart. A display driver whose initialisation sequence threw would crash five times in about a second and a half (100, 200, 400 and 800 ms of backoff, plus its start-up time) and then be stopped, leaving the rest of the system running behind a frozen screen. Supervision converts an infinite loop of restarts into a stopped component and a report; it does not fix the bug. The `#0CT1` frames of those five crashes are the material for the fix (Section 15.9).

Wedged hardware. If the device itself is in a state its interface cannot get it out of, no driver can recover it. MINIX 3's failures came from a network card with no master reset command [Herder et al. 2007]. On the PicoCalc, if the keyboard controller stopped answering on the I2C bus, every instance of the keyboard driver would see its transfers fail, and the supervisor would eventually stop it. Recovery would need something below the driver, such as a reset line or a power switch for the chip, if the board has one; that is a property of the hardware design, and a reason driver writers should ask for it.

State that only the device held. A device that holds data the system needs, and loses it on reset, loses it for good. After the display driver has reset and re-initialised the panel, nothing in the system knows what the panel's frame memory holds, which is why the compositor treats it as unknown and resends every pixel. For

a display that costs 41 ms; for a device whose state cannot be resent (a sensor's calibration, a radio's association with a network), recovery must include whatever protocol re-establishes it.

Work in flight. A request that a driver had started but not completed has an unknown outcome. Idempotent requests can be retried; others cannot (Rule 3). The lost-key window is the PicoCalc instance, and every system in Section 15.3 reports the same limit.

Wrong answers. A driver that returns incorrect data without crashing is not detected by any mechanism in this chapter. A display driver that sent the right number of pixels to the wrong place would pass every check the nucleus makes. Detection would need knowledge of what correct output is, which only the client or an end-to-end check has [Saltzer et al. 1984].

Time. Recovery takes time, and during it the device is unavailable: 213 ms for the keyboard, 515 ms for the display on the virtual clock. For an interactive handheld that is a pause. For a device with real-time obligations (Chapter 5) it is a missed deadline, and a system that must meet deadlines through a driver failure needs redundancy, not restart.

15.9 Diagnosis: from healing to repair

Self-healing as described so far stops at recovery: the error is discarded and service resumes. The fault stays. A system that could also repair the fault would close the loop, and PicoCalc OS was designed with the first half of that loop in place.

Every crash writes a `#OCT1` fault frame to the console (Section 14.7): the world, the file, the message it was handling, the error with its position and stack, its crash count and its heap use. The frame is meant for a program on the other end of the USB cable, for example an agent that reads the frame, opens the named file, and proposes a fix to the handler that threw. On the host build the frames go to a log, and the host build can be driven from standard input, so an agent can already reproduce a crash with a script, read the frame, and look at the screen.

The missing half is applying a fix to a running device: replacing a world's source at run time, starting the new version through the same supervisor, and rolling back if the new version crashes too. Supervision already provides the restart and the crash detection; what does not exist yet is a way to load new source into a world and a policy for rolling back. MINIX 3's work on live update shows one route, in which a component saves its state to the data store for its replacement to read [Tanenbaum et al. 2010]. For PicoCalc OS this is planned work, described in the paper's future-work section, not something the system does today.

KEY POINTS

- Device drivers have been the largest source of operating-system failures in every study of Linux and Windows, because they are most of the code, mostly rarely run code, and written against devices with poorly specified protocols.
- Surviving a driver failure takes isolation, so the failure does not spread, and recovery, so the service returns; isolation is necessary for recovery but does not provide it.
- Recovery systems differ in who keeps the state a restarted driver needs: shadow drivers log it per driver class, MINIX 3 makes clients rebuild it, and PicoCalc OS rebuilds the device at start and has the client re-grant what it owns.
- A restartable driver resets its device to a known state on every start, holds only state that can be rebuilt, uses idempotent requests where it can, regains authority by claim or re-grant rather than inheriting it, is watched by more than one detector, is protected from restart storms, and has its recovery tested by fault injection.
- PicoCalc's keyboard driver recovers in 213 ms without losing the key, because the controller buffers events; but a crash between a FIFO read and the publish loses the event, a 7 ms window that follows from the read being destructive.
- The display driver recovers in 515 ms, 370 ms of it the panel's initialisation; a flush addressed to the dead instance survives in the mailbox and must be answered `skipped`, after which the compositor re-grants the framebuffer and resends every pixel.
- The compositor recovers its screen in 167 ms in the measured case but loses the focused app and the battery level, because it held state nobody else held and the power driver publishes only on change; retained channel messages in the nucleus would fix all of these at once.
- Restart cannot cure deterministic faults, wedged hardware, state only the device held, work in flight, wrong answers or the time recovery takes; the fault frame is the starting point for repairing what restart cannot.

A self-healing driver is not a special kind of code. It is ordinary driver code that obeys a few rules about state and authority, running under a supervisor that restarts it and a nucleus that guarantees a dead instance cannot act. PicoCalc OS shows both the strength of the approach (two drivers that recover completely with no recovery-specific code in them beyond a three-line guard) and its characteristic failure (a client that held

state it should have been able to rebuild). The same two lessons apply at much larger scale in the web browser, which restarts crashed renderer processes and has had to decide what a page should lose when it does. Chapter 16 turns to the browser as an operating system.

EXERCISES

1. BASIC For each of the seven rules, name the line or lines of PicoCalc OS code that implement it, or say that the system does not implement it.
2. BASIC From Table 15.2, which stages of the keyboard driver's recovery are set by policy, which by the device, and which by the bus? Which could be shortened without changing the hardware, and by how much?
3. ANALYSIS The keyboard controller's FIFO holds 31 events. While the keyboard driver is restarting, how fast would a user have to type to overflow it, given the recovery time in Table 15.2? What does the controller do with events that arrive when the FIFO is full?
4. ANALYSIS In the lost-key experiment the kill came 10 ms after the key press. Using the trace, give the exact interval of kill times (relative to the key press) for which the key-down event is lost, and explain each end of the interval.
5. ANALYSIS Classify each of PicoCalc's driver messages (`flush` , `attach` , the `keys` events, the power driver's `backlight` and `screen`) as idempotent or not, and for each non-idempotent one say what a client should do when it does not know whether the message was handled.
6. ANALYSIS Using MINIX 3's classification into network, block and character drivers, which class does each of PicoCalc's three drivers belong to? Does the classification predict which of them recover transparently?
7. CODE Kill the display driver in the host build at several moments during its own start sequence (for example 50, 160 and 300 ms after a first kill) with `crash display` , and record when the screen is whole again each time. Explain the results from the backoff policy and Rule 1.
8. CODE Remove the `if (!s.fb || !s.ready)` guard from the display driver's flush handler, kill the driver from the Faults app, and read the trace and the UART log. Reproduce the second crash described in Section 15.6 and explain why it happens.
9. CODE Add liveness detection to PicoCalc OS: either a per-world turn deadline in the manifest, enforced by the nucleus, or a supervising world with `sys.admin` that asks each driver for a heartbeat every second and kills one that fails to answer three times. Test it with a driver whose flush handler awaits a promise that never settles.

10. CODE Make the keyboard driver reset the STM32 on start by writing its reset register, and measure what is lost when the driver is killed and a key pressed at once. Argue whether the change is an improvement.

11. PROJECT Restructure the compositor so that it never awaits a flush inside its draw turn: send the flush, record that one is outstanding, handle the reply as a separate message, and invalidate the screen if no reply arrives within a deadline. Measure how long the compositor stays responsive to app frames during a display-driver restart before and after the change.

12. PROJECT Implement retained channel messages in the nucleus: keep the last message published on each channel (per publisher for `frames`), and deliver the retained messages to a world when it boots after a restart. Show that a compositor restart with the REPL in front then restores the battery level, and decide whether it should also restore the focused app.

13. PROJECT Make the display driver restartable without a full-screen resend. What would the driver or the compositor need to know about the panel's frame memory after a crash, and when, if ever, is it safe to assume the memory survived?

14. DISCUSSION Windows crashes the whole computer after five display-driver recoveries in sixty seconds; PicoCalc OS stops the display world after five crashes in ten seconds and keeps running. Argue for each choice from the point of view of the user of each system.

FURTHER READING

[Swift et al. 2004] Shadow drivers: the most complete account of what a transparent driver recovery must track and replay, with an honest list of what it cannot do.

[Herder et al. 2007] MINIX 3's driver recovery, including the division of drivers into network, block and character classes by whether their recovery can be transparent.

[Swift et al. 2003] Nooks: isolating drivers inside an unmodified kernel, and the experiments that show how many crashes isolation alone prevents.

[Chou et al. 2001] The study that established drivers as the buggiest part of an operating system, with a careful account of its methods.

[Ryzhyk et al. 2009] Why drivers are buggy: protocols and concurrency, and a driver model that removes much of the second.

[Candea and Fox 2003] Crash-only software, whose requirements read as a checklist for restartable drivers.

CHAPTER 16

PART V · THE BROWSER AS AN OPERATING-SYSTEM PLATFORM

The browser as an operating system

A web browser runs programs written by strangers, many at once, on a machine that belongs to someone else, and it must stop those programs from reading each other's data, from taking the machine over, and from freezing the screen. Those are the problems an operating system exists to solve, and over the last two decades browsers have solved them with the same mechanisms: protection domains, a privileged kernel that mediates every sensitive operation, message passing between isolated components, a scheduler, resource limits, and access to devices granted one at a time. This chapter maps each of those mechanisms onto the web platform precisely, follows how the mapping came about, and looks at where the browser's isolation stands after the speculative-execution attacks of 2018. By the end you should be able to take any operating-system concept from the first fifteen chapters and say what plays its role in a browser, what plays it in PicoCalc OS, and where the two differ.

The comparison is not incidental. PicoCalc OS borrowed its central design from the browser: isolated JavaScript event loops that share nothing and talk only by messages. A world in PicoCalc OS is closer to a browser tab than to a Unix process. Seeing the same design at the scale of billions of users shows which of its properties are fundamental and which are accidents of a small machine.

16.1 A program that runs other people's programs

Start with what a browser is asked to do. A user opens a mail client in one tab, a bank in another and a news site in a third. Each tab runs JavaScript that its site sent a moment ago; the news site's page also runs scripts from a dozen advertising and analytics companies. None of these programs was installed, reviewed or cho-

sen by the user in any meaningful sense, and some are hostile. The browser must let each one compute, draw, store data and talk to its own server, while guaranteeing that the news site's advertisement cannot read the bank's page, that a hung script in one tab does not freeze the others, and that no page can read the user's files or turn on the camera without the user's say-so.

In operating-system terms, the browser is a **multi-principal** system. A **principal** is the entity on whose behalf code runs and to which permissions are granted. In a time-sharing Unix system the principal is a user account. In the browser it is the **origin**: the triple of scheme, host and port from which a document or script came, so that `https://mail.example.com` and `https://news.example.org` are different principals even when both run in the same window. The **same-origin policy** is the browser's basic access rule: a script may read data belonging to its own origin and may not, by default, read another origin's data, even though it can display that data (an image, a frame) and send requests to it.

The research projects that first described the browser this way stated the analogy directly. The OP browser [Grier et al. 2008] split the browser into subsystems for the web page, the network, storage and the user interface, each in its own operating-system process, around "a small browser kernel" that creates and deletes those processes, interposes on all communication between them and enforces the access policy on every message. Gazelle [Wang et al. 2009] went further and called its design a "multi-principal OS construction": a browser kernel that owns resource allocation and cross-principal protection, with each origin's content running in its own process and reaching other origins or the system only through calls to that kernel. Both designs put the browser's own logic in the position an operating-system kernel occupies: the only component trusted to enforce the policy, and the only path between principals.

PicoCalc OS has the same shape at a smaller scale. Its principals are worlds, named in the manifest. Its kernel is the nucleus, which sits on every path between worlds and between a world and the hardware. Its policy is written in the manifest (claims) and enforced on every use (Chapter 9). The difference is in what separates the principals. The browser puts them in different operating-system processes, so the processor's memory protection keeps them apart. PicoCalc OS puts them in different JavaScript runtimes within one address space, so only the safety of the language and the correctness of the nucleus keep them apart (Chapter 10). Most of this chapter is about the consequences of that one difference.

16.2 From one process to many

The first graphical browsers were ordinary single-process applications. Every page, every frame and every script ran in the same address space as the browser's own code, which had two consequences. A bug that crashed the browser while rendering one page closed every page, and a script that ran a long loop froze the

whole application, since the one thread that ran scripts was also the thread that redrew the window. A memory-safety bug in the renderer, of which there were many, gave an attacker the browser's full authority: everything the user could do.

Google Chrome was released on 2 September 2008 with a different structure, which the press described at the time as tabs that "run as separate processes" so that "a crash only kills that one process" [The Register 2008]. Reis and Gribble analysed the design soon after [Reis and Gribble 2009]. Its central idea is to identify the **web programs** a browser runs (roughly, the pages of one site that can script each other) and give each its own **rendering engine process**, while a single **browser kernel process** keeps the privileged parts: the network stack, storage, the user interface and access to the file system. Rendering processes run in a sandbox that denies them most operating-system calls. When a page needs something only the kernel can do, such as fetching a URL, reading a cookie or showing a file chooser, its rendering process sends a message to the browser process, which checks the request against the policy for that page's origin and performs it.

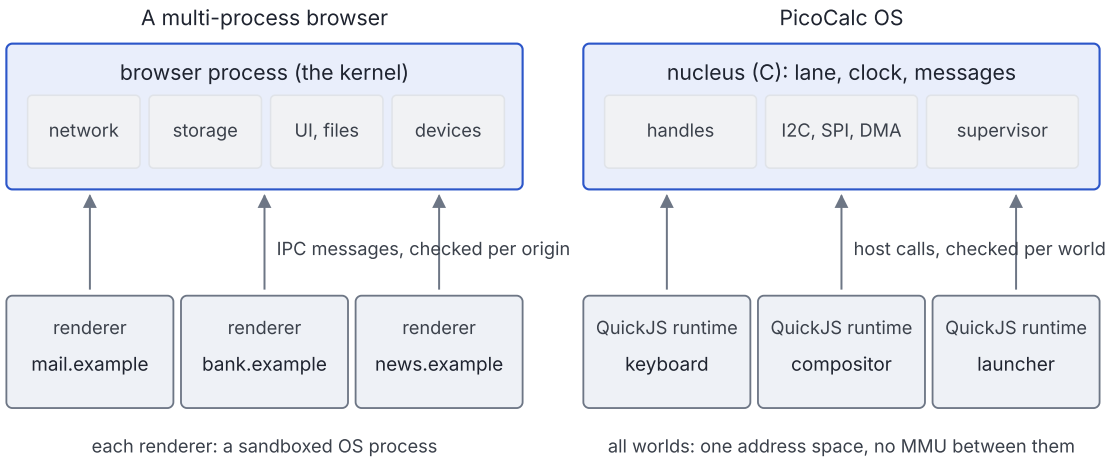


Figure 16.1. Two kernels and their protection domains. Left: a multi-process browser, where the browser process holds the privileged resources and each site's pages run in a sandboxed renderer process, so the processor's memory protection separates principals. Right: PicoCalc OS, where the nucleus holds the hardware and each world is a QuickJS runtime in the same address space, so the language runtime separates them.

Figure 16.1 puts the two structures side by side. The resemblance is close enough that the vocabulary transfers almost word for word. The browser process is a reference monitor: it is always invoked (every privileged operation is a message to it), it is the only place the policy is enforced, and it is meant to be small

enough to get right (Chapter 8). A renderer process is a protection domain whose authority is exactly the set of messages the browser process will honour for its origin. The messages themselves are the browser's system calls.

Two things in the browser have no counterpart in PicoCalc OS. The first is the second layer of protection: a renderer is not only a separate address space but also a sandboxed one, so a compromised renderer that tries to open a file directly is stopped by the operating system beneath the browser, not by the browser itself. PicoCalc OS has a single layer, and a bug in QuickJS or in the nucleus's host calls is a bug in the only barrier there is. The second is cost. A process carries an address space, page tables, a stack and the browser's own per-process state, which is why Site Isolation's main price was memory (Section 16.3). PicoCalc OS spends about 190 KB per world (Chapter 6) and has no processes to pay for.

16.3 Principals, sites and the end of same-process isolation

Chrome's first architecture did not give every origin its own process. It grouped pages into processes by tab and by how they were opened, and it let a page embed frames from other origins in the same process. Within one process, the same-origin policy was enforced by the rendering engine's own checks: the JavaScript engine refused to let a script from one origin touch objects belonging to another. That is language-based isolation inside an address space, the same mechanism PicoCalc OS relies on, and for a decade it was the browser's main defence against one site reading another's data.

Two developments ended that arrangement. The first was the steady supply of memory-safety bugs in rendering engines, each of which let an attacker who controlled one page's script read anything in the process, including other origins' data. The second was Spectre [Kocher et al. 2019]. Spectre showed that a program can read memory it is not allowed to access, without any bug in the software, by training the processor's branch predictor to execute a bounds-checked access speculatively and then measuring, through cache timing, which cache line the speculative access touched. A JavaScript program can do this to the rest of its own process. Once that was demonstrated, any isolation that depended on code in the same address space respecting a check could no longer be trusted: the check holds architecturally and leaks speculatively.

The browsers' answer was to make the operating-system process the unit of isolation between sites, not just between tabs. Chrome's Site Isolation [Reis et al. 2019] puts documents from each **site** in their own renderer process, including cross-site frames inside a page, and filters responses so that cross-site data a page is not allowed to read never enters its process. A site is coarser than an origin: it is the scheme plus the registrable domain, so `https://bar.foo.example.com:8000` belongs to the site `https://example.com`. The coarser unit is needed because pages from different origins on one site can still make themselves same-origin by setting `document.domain`, and so must share a process. Site Isolation was deployed to all desk-

top Chrome users in mid-2018. Its authors report a total memory overhead of 9 to 13 percent on real workloads and an increase in page-load latency of less than 2.25 percent. They are explicit about its scope: it mitigates attacks within one address space, and it must be combined with hardware and operating-system mitigations for leaks between processes.

The same reasoning removed two powerful tools from ordinary pages. A precise clock is what turns a speculative cache access into a readable bit, and `SharedArrayBuffer`, shared memory between a page and its workers, lets a script build a precise clock from a counting thread. Browsers turned `SharedArrayBuffer` off in January 2018. Chrome brought it back on desktop in July 2018, relying on Site Isolation, and from version 92 it, like the finer timers, is available only to pages that opt into **cross-origin isolation** [Archibald and Kitamura 2021]. A page becomes cross-origin isolated by serving two headers: `Cross-Origin-Opener-Policy: same-origin`, which puts it in its own group of browsing contexts so no cross-origin window shares its process, and `Cross-Origin-Embedder-Policy: require-corp`, which refuses to load any cross-origin resource that has not opted in to being loaded. A script can check `self.crossOriginIsolated` [Kitamura and Denicola 2020]. The coarse timer is visible directly: in the measurements for Chapter 17, `performance.now()` in an ordinary page in Chromium 141 advanced in steps of exactly 0.1 ms, so a single message round trip of a few tens of microseconds measured as zero and had to be timed in batches of thousands.

This history matters for PicoCalc OS in a specific way. Its worlds are isolated by the language, inside one address space, which is the arrangement the browsers abandoned. The browsers abandoned it because their principals are adversaries who choose which code to run and can run it for as long as they like with a clock. On the handheld, every world is installed by the owner, the virtual clock is the only clock a world can read, and the sandbox world that runs typed code has a 200 ms budget. That does not rule out timing side channels on the handheld, but it changes the threat from "any web page" to "code the owner chose to run". Chapter 10 takes up what that means for the design.

16.4 The event loop is the scheduler

Within a renderer, JavaScript runs on an **event loop**, and the browser's specification of that loop is a scheduler in all but name. The HTML Standard [WHATWG HTML 2026] defines it this way. An event loop has one or more **task queues**, each holding tasks from a family of sources (user input, timers, network responses, messages from workers). Despite the name a task queue is a set, because the loop takes the first *runnable* task, not necessarily the oldest. Each turn of the loop chooses a task queue in an implementation-defined manner, which is where a browser applies priorities (input before timers, for example), takes the first runnable task from it and runs it to completion. Then it performs a **microtask checkpoint**: it runs every job in the **microtask queue**, which holds promise reactions and a few other callbacks, including any

microtasks those jobs add, until the queue is empty. Then, if this is a window's loop and a rendering opportunity has arrived (typically once per display frame), it updates the rendering: it runs animation-frame callbacks, recomputes style and layout, and paints.

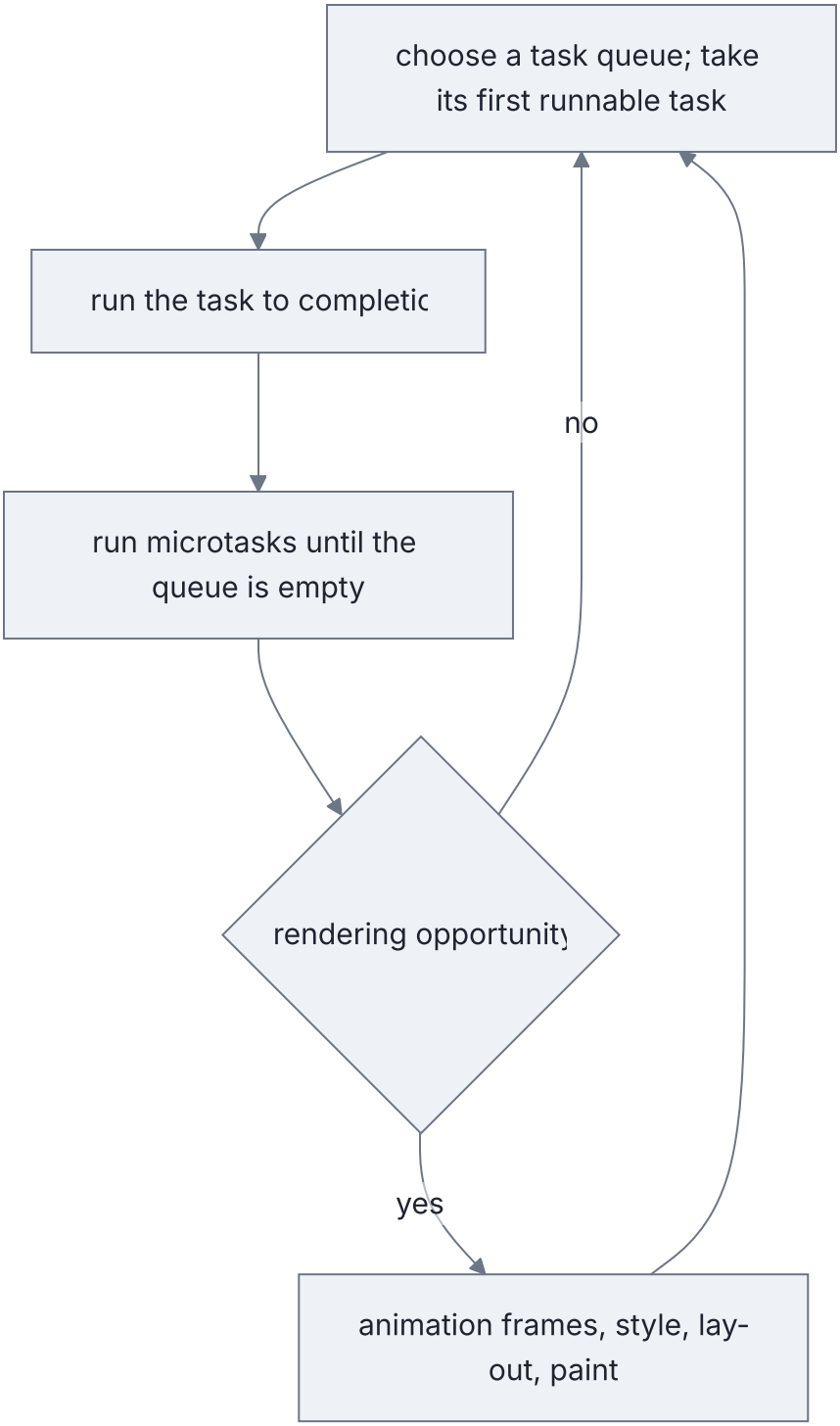

choose a task queue; take its first runnable task
run the task to completic
run microtasks until the queue is empty
rendering opportunity
no
yes
animation frames, style, lay-out, paint

Figure 16.2. One turn of the HTML event loop. A task runs to completion, then every microtask runs, then the page may be rendered. Nothing interrupts a task: a task that runs for 133 ms delays input, timers and painting by 133 ms.

Three properties of this loop are the properties of PicoCalc OS's lane. First, a task is never preempted. A task that loops for 133 ms holds the thread for 133 ms, and every input event, timer and frame waits behind it. Second, ordering is total within one loop: two tasks never interleave, and a handler can update its state without locks. Third, promises split work into pieces that run as separate scheduling decisions only at defined points: an `await` returns control at the end of the current microtask, and the continuation runs later as another microtask. PicoCalc OS's segments are the same idea with the boundaries placed differently. A world's segment ends when its handler awaits something that the nucleus must do later (an ask, a bus transfer, a sleep), and the continuation is a later segment chosen by the lane like any other (Chapter 3).

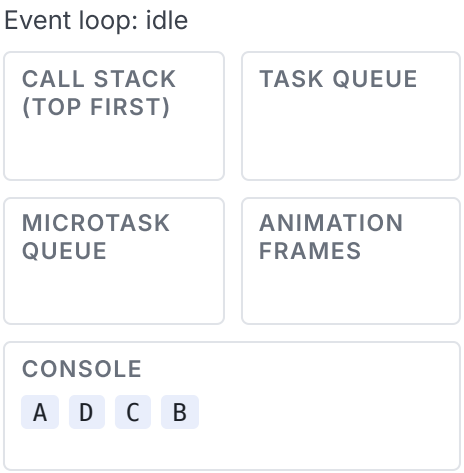
TRY 16.1

Predict the order

The script logs `A`, schedules a zero-delay timer that logs `B`, queues a promise reaction that logs `C`, and logs `D`. Step through it and watch which queue each callback enters. The output is A, D, C, B: the synchronous lines first, then the promise reaction, which runs before the timer even though the timer was scheduled first, and then the timer, in a later turn of the loop. Choose **await and then** to see that each `await` gives every microtask already queued a turn before the function resumes, and **A timer and an animation frame** to see where rendering fits.

All queues are empty. The output is A, D, C, B: synchronous code first, then microtasks, then the next task.

```
1  console.log("A");
2  setTimeout(() => console.log("B"),
3  Promise.resolve().then(() => consol
4  console.log("D");
```



1. The script is a task. The event loop takes it and starts running it on an empty call stack.
2. console.log runs at once and prints A.
3. setTimeout hands the callback to the timer system. After 0 ms it becomes a task in the timer task source; it cannot run before the current task ends.
4. The promise is already resolved, so then() queues its reaction as a microtask straight away.
5. console.log prints D.
6. The script returns and the stack is empty. Before any other task, the loop performs a microtask checkpoint.
7. The promise reaction runs and prints C.

8. The microtask queue is empty. The loop may render now; nothing has changed on the page, so there is nothing to paint.
9. The loop takes the next task, the timer callback, which prints B.
10. All queues are empty. The output is A, D, C, B: synchronous code first, then microtasks, then the next task.

The browser does not stop a task that runs too long. It cannot: a task has no budget, and JavaScript gives a running script no point at which the engine could safely abandon it without leaving the page's objects half-updated. When a page's script runs for many seconds, the browser eventually offers the user a dialog to stop the page, which kills the whole page. PicoCalc OS takes the opposite position. Every segment has a budget (20 ms for a driver, 200 ms for the sandbox), the interpreter checks it every 1,000 steps, and a segment that exceeds it is abandoned together with the world's whole runtime, which removes the problem of half-updated objects by discarding every object (Chapter 11). A browser could make the same trade for workers, whose state is private, and the operating system in Chapter 17 does exactly that.

The cost of the browser's choice is visible in the trace of PicoCalc OS when a world holds the lane. In the trace below, the REPL has asked the sandbox to run the 100,000-iteration loop of Chapter 18's calibration. The sandbox holds the lane for 133 virtual ms; the keyboard driver's polls, which normally run every 20 ms, wait until it ends, exactly as input waits behind a long task in a browser.

A 133 ms loop in the sandbox 430 virtual ms

Enter is pressed in the REPL on a 100,000-iteration loop. The sandbox holds the lane for 133 ms: the keyboard's polls, and a key pressed meanwhile, wait until it ends.

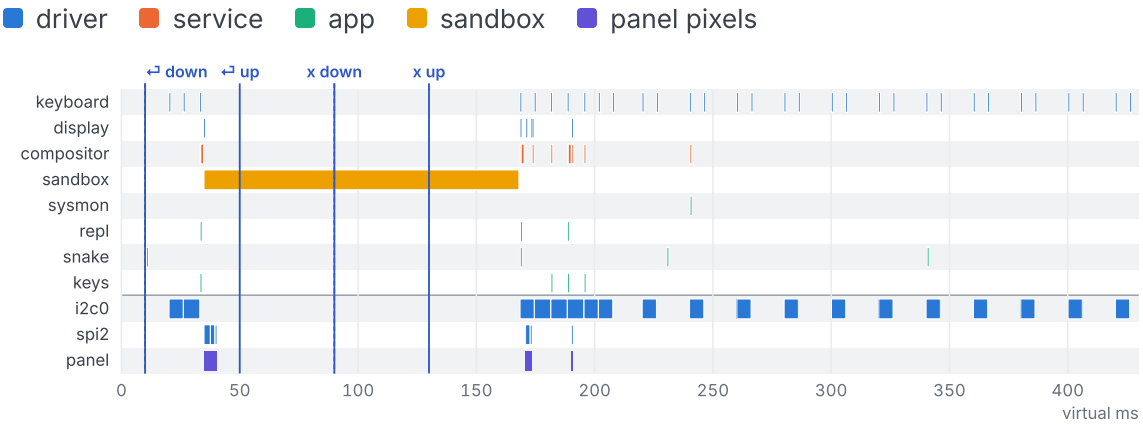


Figure 16.3. A long task on the lane: the sandbox's 133 ms segment delays the keyboard driver's polls. A browser tab running a 133 ms script delays input events in the same way.

16.5 Workers are processes, postMessage is IPC

Inside a page, the unit of isolation the platform offers JavaScript is the **worker**. A dedicated worker is created from a script URL, runs on its own thread with its own global object and its own event loop, and has no access to the page's document. It shares no JavaScript objects with the page or with other workers. The only way to communicate with it is `postMessage`, which **structured-clones** its argument: it walks the object graph and builds a copy on the receiving side, preserving cycles and most built-in types and refusing functions and DOM nodes. A worker is therefore much closer to a process than to a thread: separate state, separate scheduler, communication by copying.

Two refinements give messages the character of capabilities. A **transferable** object, such as an `ArrayBuffer` or a `MessagePort`, can be listed in the second argument of `postMessage`, and then it is moved rather than copied: after the call the sender's copy is detached and unusable, and only the receiver holds it. A `MessageChannel` creates a pair of entangled ports; whoever holds a port can send to whoever holds the other, and passing a port in a message hands that ability to someone else. Holding the port is the authority to communicate, which is exactly the object-capability discipline (Chapter 9): no ambient naming, only references that were given. PicoCalc OS's handles travel in messages in the same way, and the nucleus grants the receiver use of a handle when the message is delivered.

Workers have measurable costs, and they are useful to compare with PicoCalc OS's. In Chromium 141 on the 2.1 GHz Xeon used for this book's host measurements, starting a dedicated worker from a blob URL and getting its first reply took a median of 7 ms; a round trip of a small message between a page and a worker took between 21 and 50 µs across runs; and a round trip of a 70 KB object graph (a thousand small objects) took 1.1 to 1.25 ms, almost all of it structured cloning in both directions. A PicoCalc OS world's start, by comparison, is charged about 66 µs of virtual time on the lane (compiling its file is work the model does not see, Chapter 18), and an ask round trip with an empty payload costs 119 µs of virtual time on the device profile (Chapter 12). The numbers are not directly comparable, since one is wall-clock time on a desktop and the other is virtual time on a modelled microcontroller, but they show the same structure: crossing an isolation boundary costs a copy, and the copy dominates for large messages.

The one thing workers can share is memory, and only under cross-origin isolation. A `SharedArrayBuffer` posted to a worker is not copied; both sides see the same bytes, and `Atomics` provides atomic operations and a wait/notify primitive on them. This reintroduces threads with shared memory into a platform otherwise built on message passing, with all the hazards of Chapter 3, and it is the reason the feature was the first to be withdrawn after Spectre.

16.6 Resource control in the browser

An operating system that isolates principals but lets any one of them consume all the processor or memory has protected their data and abandoned their availability. The browser controls resources at a coarse grain. Its main lever is the operating-system process: a runaway renderer can be lowered in priority or killed by the browser process without affecting other sites, and an out-of-memory failure kills one renderer, which then shows an error page in place of the site's content until the user reloads. The browser also manages pages the user is not looking at. Chrome's Page Lifecycle model [Walton 2023] gives a page two states beyond the visible ones. In the **frozen** state the browser stops running the page's freezable tasks until it is unfrozen, which it does to save processor time, battery and data, and in the **discarded** state the browser has unloaded the page entirely to reclaim its memory, so that returning to the tab reloads it; a page can detect this afterwards through `document.wasDiscarded`.

Within a page there is little control. A worker can be stopped with `terminate()`, which ends its thread at once, running no further code in it, whatever it is doing. That is the only preemption of JavaScript the platform offers to page code, and it is total: the worker's state is gone. There is no way for a page to cap the memory a worker allocates or to give a task a time budget. PicoCalc OS's resource control is finer and stricter: a heap limit per world, enforced by the allocator the nucleus gives each runtime, and a budget per

segment, enforced by the interpreter's interrupt counter. The browser's model trusts each site to manage its own scripts and protects only sites from each other; PicoCalc OS protects each component from every other, including components of the same application.

16.7 Devices from JavaScript

For most of its history the web had no access to devices beyond those the browser itself drove: the screen, the keyboard, the network, later the camera and microphone through dedicated permission prompts. That changed with a family of interfaces that expose general-purpose buses to page script. WebUSB, available in Chrome since version 61 in 2017, lets a page talk to a USB device's endpoints directly [Beaufort 2016]; Web Serial and WebHID, which graduated from trials in Chrome 89 in 2021, expose serial ports and human-interface devices [LePage 2021]; and WebGPU, which reached all three major browser engines in 2025, exposes the graphics processor's command queues [Beaufort 2025].

These interfaces move device drivers into web pages. A page that programs a microcontroller through Web Serial, or reads a sensor through WebHID, contains the protocol logic that would otherwise be in a driver: it knows the device's commands, its timing and its error behaviour. That is a user-space driver in the sense of Chapter 13, running at the far end of a long chain of mediation. The security model is a capability model with the user as the source of grants. `navigator.usb.requestDevice()` can be called only from a user gesture on a secure (HTTPS) page; it opens a chooser in which the user picks one device; and what the page receives is a `USBDevice` object for that device alone [Beaufort 2016]. The page cannot enumerate other devices or open one by name. Holding the object is the authority to use the device, just as holding a handle is in PicoCalc OS, and the grant is made by the one party the browser trusts to make it.

PicoCalc OS's drivers are the same idea at the bottom of the stack. The keyboard driver is JavaScript that writes a register number to the STM32 and reads two bytes back; the display driver is JavaScript that sends the ST7365P its initialisation sequence and then pixel data. Their authority is a handle obtained through a claim in the manifest, checked by the nucleus on every transfer (Chapter 13). The difference is in who grants: on the web the user, at run time, one device at a time; on the handheld the manifest's author, at installation, one bus address at a time.

16.8 Operating systems made of the web

Three products took the analogy literally and built a whole operating system whose applications were web pages. Palm's webOS, shown at CES in January 2009, ran a Linux kernel underneath, but its entire user interface and every application were built with HTML, CSS and JavaScript using a framework called Mojo [InformationWeek 2009]. Google announced Chrome OS in July 2009 as an open-source, lightweight oper-

ating system initially aimed at netbooks, built around the Chrome browser [Pichai and Upson 2009]. Mozilla's Firefox OS, developed as Boot to Gecko, reached phones in July 2013 when the ZTE Open went on sale in Spain through Telefónica, presented as a phone that "uses the Internet as the platform for all its functions and applications" [Malay Mail 2013]; the project was discontinued in 2016 [Computer Hope 2025].

All three kept a conventional kernel underneath, so they were not operating systems in the sense of replacing process management, memory management and drivers. What they replaced was the application model: applications became origins, installation became visiting or bookmarking, and the browser's permission prompts became the system's permission model. Their histories show where that model strains. Applications on a phone want to run in the background, to talk to hardware the browser does not expose, and to keep working without a network, and each of those wants a new web interface with its own permission story. The service worker, a script the browser runs on behalf of an origin in the background, starts and stops at the browser's discretion and intercepts the origin's network requests, came out of that pressure; it is the web's daemon.

16.9 WebAssembly: an executable format for the web

A process runs a program, and an operating system needs an executable format. For most of the web's history that format was JavaScript source. WebAssembly [Haas et al. 2017] added a second one: a compact binary encoding of a typed stack machine, designed to be compiled ahead of execution to machine code that runs at close to native speed, and to be safe to run on untrusted input. A WebAssembly module's code cannot address anything outside its own **linear memory**, a bounds-checked array of bytes; it cannot jump to arbitrary addresses, since control flow is structured and indirect calls go through a typed table; and it can affect the world only through the functions it **imports**, which the host supplies when it instantiates the module. A module's import list is therefore a complete statement of its authority: it can do what the imported functions do and nothing else.

WASI, the WebAssembly System Interface, standardises one such import list for programs that expect an operating system. Its first stable version is a single module, `wasi_snapshot_preview1`, with 45 functions, and it "uses a global file descriptor table with preopened directories for capability-based sandboxing" [WASI 2026]: a program can open files only beneath directories the host chose to give it at start-up.

PicoCalc OS compiles to WebAssembly unchanged except for its entry point (Chapter 17), and its import list illustrates the point about authority. The C file that replaces the native `main` declares the only two functions the module imports from its host that are not WASI:

▸ host/main_wasm.c

```
#define EXPORT(name) __attribute__((export_name(#name)))
#define IMPORT(name) __attribute__((import_module("env"), import_name(#name)))

IMPORT(pc_emit) void js_emit(int channel, const char *bytes, int n);
IMPORT(pc_capture) void js_capture(const char *name, const uint8_t *rgb, int w, int h);
```

The WASI functions it imports are served by a small shim in the loader, and most of them refuse. There is no filesystem (the shim reports no preopened directories, so `path_open` has nothing to open), standard input is empty, and the clock returns a fixed instant:

▸ wasm/picocalc.js

```
fd_prestat_get() { return EBADF; },            // no preopened directories: there is no
filesystem
…
// the host's clock is never read by the system's behaviour (worlds see the virtual
clock), so a
// fixed instant keeps runs identical
clock_time_get(id, precision, outPtr) { self._dv().setBigUint64(outPtr,
1767225600000000000n, true); return ESUCCESS; },
```

The whole operating system, thirteen worlds and two device models included, holds exactly the authority to print lines and hand over screenshots. That is the object-capability idea applied to an entire program, and it is the reason WebAssembly is an attractive executable format for running untrusted code in places other than the browser.

16.10 Drivers in the page, JavaScript in the device

The movement has gone both ways. While browsers acquired device access, JavaScript went down onto microcontrollers. Espruino, created by Gordon Williams in 2012 and open-sourced in 2013, interprets JavaScript directly from source, without building a syntax tree or bytecode, and runs in as little as 8 KiB of RAM [Espruino 2026]. Moddable's XS engine brings a complete modern JavaScript to microcontrollers such as the ESP32 family, with an SDK for networking, displays and sensors [Moddable 2026]. ECMA-419, first published in 2021, standardises an API for ECMAScript programs on embedded systems [Ecma

International 2021], and Moddable's SDK has migrated to it. These systems run one program in one heap. PicoCalc OS's contribution is to run many isolated programs, each in its own QuickJS runtime, with a kernel between them, which is the browser's structure rather than the scripting-language one.

Table 16.1. Operating-system concepts, their web-platform counterparts, and the corresponding mechanism in PicoCalc OS.

Concept	On the web platform	In PicoCalc OS
Principal	origin; site for process allocation	world, named in the manifest
Protection domain	renderer process per site; dedicated worker	QuickJS runtime per world
Kernel, reference monitor	browser process	nucleus
System call	IPC message to the browser process	host call `__k` / `__ka`
IPC	`postMessage`, `MessageChannel`	`send`, `ask`, `publish`
Capability	`MessagePort`, transferables, granted device objects	handle
Scheduler	event loop: task queues, microtasks, rendering	the lane: priorities, arrival order
Preemption	none within a task; `terminate()` for a worker	budget checked every 1,000 steps
Clock	`performance.now()`, coarsened to 0.1 ms unless cross-origin isolated	virtual clock only
Memory limit	per process, enforced by the OS; none per worker	heap limit per world
Background service	service worker	service world
Device driver	page code using WebUSB, Web Serial, WebHID	driver world with a handle
Executable format	JavaScript source, WebAssembly module	world file (JavaScript)
Failure handling	error page, reload by the user	supervisor restarts with backoff

Table 16.1 collects the mapping. Reading it row by row shows what PicoCalc OS took from the browser: principals that are units of code rather than users, a privileged kernel that mediates through messages, run-to-completion event loops, and capabilities that travel inside messages. It also shows three things the browser could take back. One is a time budget per task, enforced by discarding the task's state, which would let a browser stop a runaway worker in a defined way instead of offering a dialog. Another is supervision: a crashed component restarted automatically with backoff, so that a crashed tab or worker does not depend on the user to reload it. The third is a virtual clock for testing, so that a page's behaviour under a script of inputs is reproducible run to run, which Chapter 18 develops.

KEY POINTS

- A browser is a multi-principal operating system: its principals are origins (and, for process allocation, sites), and its browser process is a kernel that mediates every privileged operation through checked messages.
- Multi-process browsers isolate principals with the processor's memory protection plus an operating-system sandbox; PicoCalc OS isolates worlds with the language runtime alone, inside one address space.
- Spectre showed that checks enforced by code in the same address space leak speculatively, which is why browsers moved to one process per site and withdrew shared memory and precise timers from pages that are not cross-origin isolated.
- The HTML event loop is a non-preemptive scheduler: tasks run to completion, microtasks run after each task, and rendering happens between tasks, so a long task delays everything behind it, exactly as a long segment holds PicoCalc OS's lane.
- Workers are the platform's processes: separate state and event loop, communication only by structured clone, with transferable objects and message ports acting as capabilities.
- The browser controls resources coarsely (whole processes, frozen and discarded pages, `terminate()` for workers) and has no per-task budget or per-worker memory limit.
- Device interfaces such as WebUSB make drivers out of page code, with the user granting one device at a time by picking it in a chooser; the granted object is a capability.
- A WebAssembly module's imports are its entire authority; PicoCalc OS compiled to WebAssembly imports only two functions of its own and a handful of WASI calls, most of which refuse.

The browser shows that a system of isolated JavaScript event loops, mediated by a kernel and communicating by messages, can carry the weight of hostile code at enormous scale, provided the isolation boundary is one the hardware enforces. The next chapter builds such a system inside a single page, with Web Workers as its worlds and the page's main thread as its kernel, and runs PicoCalc OS itself next to it, so that the two designs can be compared on the same machine.

EXERCISES

1. BASIC For each row of Table 16.1, say whether the web-platform mechanism or the PicoCalc OS mechanism gives the stronger guarantee, and why.

2. BASIC Two pages are loaded from `https://a.example.com` and `https://b.example.com:8443`. Are they the same origin? The same site? Which of the two answers decides whether Site Isolation may put them in one process, and why does the other not suffice?

3. ANALYSIS Explain why `document.domain` forces process allocation by site rather than by origin. What would break if a browser allocated processes by origin while still honouring `document.domain`?

4. ANALYSIS A script schedules `setTimeout(f, 0)`, then resolves a promise whose reaction calls `g`, then loops for 50 ms. In what order do `f`, `g` and the next rendering update run, and how long after the script started does each begin? Check your answer with Try 16.1.

5. ANALYSIS The Site Isolation paper reports a 9 to 13 percent memory overhead. Using the per-world cost of PicoCalc OS (about 190 KB), estimate what the equivalent overhead of giving each of the 13 worlds its own operating-system process would be on a system with an MMU, assuming a minimum of 64 KB of page tables and kernel structures per process. Is the comparison fair? What does it leave out?

6. ANALYSIS The coarsened `performance.now()` advances in steps of 0.1 ms. Describe how to measure a 20 µs message round trip with it, and estimate how many round trips you need to bound the error at 1 percent.

7. CODE Write a page that posts a 1 MB `ArrayBuffer` to a worker 1,000 times, once copied and once transferred (and transferred back by the worker). Measure both. Explain the difference, and what the sender can no longer do after a transfer.

8. CODE List the imports of `wasm/picocalc.wasm` (for example with `wasm-objdump -x` or by reading the import section with a short script). For each, state what authority it gives the module and how `wasm/picocalc.js` limits it.

9. CODE Add a world to PicoCalc OS that loops for 100 ms in each message handler, and send it a message every 50 ms from another world. Use the host build's `trace` command to show its effect on the keyboard driver's polls. Then describe the same experiment in a browser tab and what the browser would show the user.

10. DISCUSSION WebUSB asks the user to pick a device; PicoCalc OS asks the manifest's author. Argue for and against moving PicoCalc OS's grants to run time, with the owner choosing from a list on the handheld's screen.

11. DISCUSSION webOS, Chrome OS and Firefox OS all kept a conventional kernel beneath the browser. Could a browser-like system replace that kernel on a microcontroller, as PicoCalc OS's nucleus does? What does the browser get from the kernel beneath it that PicoCalc OS must provide itself?

12. PROJECT Implement a per-task time budget for workers in a page: a wrapper that posts work to a worker and terminates and recreates it when a task exceeds its budget, preserving pending messages. Compare its behaviour with PicoCalc OS's budget on the same workload, and describe what state each loses.

FURTHER READING

[Reis and Gribble 2009] The analysis of web programs and process allocation that underlies the multi-process browser; short and concrete.

[Wang et al. 2009] Gazelle's multi-principal design states the browser-as-operating-system argument in its strongest form, including display and resource sharing.

[Reis et al. 2019] How Site Isolation was made practical after Spectre, with the costs measured on real users.

[Kocher et al. 2019] The original Spectre paper; section 4's JavaScript attack is the one that changed browser architecture.

[Haas et al. 2017] The design of WebAssembly, with the reasoning behind structured control flow and linear memory.

[WHATWG HTML 2026] The event loop processing model, read directly: the definitions of task queues, microtasks and rendering opportunities.

CHAPTER 17

PART V · THE BROWSER AS AN OPERATING-SYSTEM PLATFORM

An operating system inside the page

The previous chapter mapped operating-system concepts onto the web platform. This chapter uses the mapping to build something: a small operating system that runs inside this page, whose worlds are Web Workers and whose kernel is a few hundred lines of JavaScript on the page's main thread. It routes messages, hands each world one message at a time, answers asks, enforces a time budget on every segment, guards a simulated device, and restarts crashed worlds with backoff, following PicoCalc OS's rules wherever the browser allows. Then the chapter runs PicoCalc OS itself next to it, compiled to WebAssembly, so that the two designs can be compared on the same machine with the same clock. By the end you should be able to explain each mechanism of the in-page system from its code, say precisely where it matches PicoCalc OS and where the browser forces it to differ, and read the measurements that compare the two.

The in-page system is called **minios**. Its source is `book/widgets/minios.js`, the same file that runs the interactive figure in Try 17.1, so every excerpt below is the code you can exercise.

17.1 The design

The starting point is the correspondence of Chapter 16, narrowed to what a page can do. A world needs its own state, its own scheduler and no shared objects: a dedicated worker has all three. The nucleus needs to sit on every path between worlds: if worlds can talk only by posting to the page, and the page decides where each message goes, the page's main thread is that path. Effects must be applied when a segment ends:

a world's prelude can collect them and post them as one message when its code yields. A budget needs a way to stop a running world: `terminate()` is the only one the platform provides, and it is enough, because PicoCalc OS also discards a world's whole runtime when it overruns.

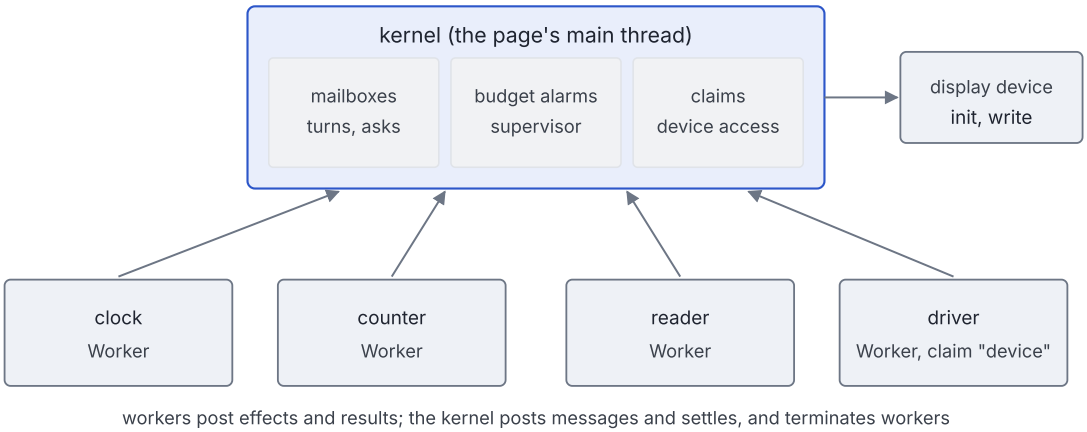


Figure 17.1. minios. The kernel runs on the page's main thread; each world is a Web Worker made of the prelude followed by the world's code. Worlds post effects to the kernel, which delivers messages, answers asks, performs device operations for the world that holds the claim, and terminates and restarts workers.

Table 17.1 lists the correspondence. Most rows are direct. Three are not, and they shape the rest of the chapter. Time in minios is the page's real time, read with `performance.now()`, so budgets, backoff and the clock a world reads are all in real milliseconds; there is no virtual clock and no model of the work a world does. Worlds run in parallel, because each worker has its own thread, so there is no single lane and no single order of events. And there is no memory limit per world, because the platform gives a page no way to cap what a worker allocates.

Table 17.1. PicoCalc OS's mechanisms and their counterparts in minios.

PicoCalc OS	minios
world: a QuickJS runtime	world: a dedicated Web Worker
nucleus	kernel on the page's main thread
prelude, evaluated first in every runtime	prelude function, its source prepended to the world's code
host calls `__k`, `__ka`	`postMessage` to the kernel, captured before the seal
effects applied at the end of a segment	effects collected in the worker, posted when the segment ends
one segment at a time on one lane	one segment at a time per world; worlds run in parallel
virtual clock, c_o + steps·τ + native	real time, `performance.now()` on the main thread
budget checked every 1,000 steps	alarm with `setTimeout`, then `terminate()`
heap limit per world	none
claims and handles	claims in the manifest, checked per device operation
supervisor: backoff 100, 200, 400, 800 ms; stop at the fifth crash in 10 s	the same policy
epochs discard stale settles	the same

17.2 A world is a Worker

A world is defined by a short piece of code written against an API like PicoCalc OS's. The counter world counts the clock world's ticks, shows the count on the device through the driver, and answers asks for the count:

▸ book/widgets/minios.js

```
world("counter")
  .needs("time")
  .state({ n: 0 })
  .on("time", (S) => {
    S.n++;
    send("driver", "show", { row: 0, text: "ticks " + S.n });
  })
  .on("get", (S) => S.n);
```

To start a world, the kernel builds the worker's whole script as one string: the prelude function's source text, called immediately with the global object, the world's name and a seed; then the world's code; then a call that tells the kernel the world is ready. It creates a worker from that string through a blob URL, and it installs two handlers that accept events only from the current epoch, so that a message still in flight from a worker the kernel has already terminated is ignored.

▸ book/widgets/minios.js

```
// A world starts as a fresh Worker: the prelude, then its code, then __start().
start(w) {
  w.epoch++;
  w.status = "starting";
  const epoch = w.epoch;
  const seed = [...w.name].reduce((h, c) => Math.imul(h ^ c.charCodeAt(0), 16777619) >>>
0, 2166136261);
  const src = `(${prelude})(self, ${JSON.stringify(w.name)},
${seed});\n${w.src}\n;__start();`;
  w.worker = Book.worker(src);
  w.worker.onmessage = (ev) => { if (w.epoch === epoch) this.fromWorld(w, ev.data); };
  w.worker.onerror = (ev) => { ev.preventDefault(); if (w.epoch === epoch) this.crash(w,
ev.message || "an error at load time"); };
  this.arm(w, START_BUDGET_MS, "start");
}
```

Using the prelude's source text, rather than a separate file, keeps the prelude an ordinary function in the kernel's file, where it can be read, linted and tested; its text is what the worker evaluates. A syntax error in the world's code makes the whole script fail to load, which arrives as an `error` event and is treated as a crash at start. A world that never calls `__start`, because its code loops at the top level, is caught by the start budget of one second.

Starting a world costs more than starting a PicoCalc OS world. In Chromium 141 on the book's measurement machine, a bare worker from a blob URL answered its first message after a median of 7 ms. A minios world, which also evaluates the prelude and its own code, reported ready in 6 to 28 ms (usually under 12) when restarted alone and in 25 to 56 ms when four started together as the page loaded, since they compete with each other and with the page for the processor. PicoCalc OS charges a world's start about 66 µs of virtual time on the lane; the comparison is not exact, because the virtual clock does not see the work of compiling a world's file (Chapter 18), but it shows that a worker is a heavyweight unit: an operating-system thread, a JavaScript engine instance and an event loop.

17.3 Sealing the global scope

A worker's global object is the world's entire authority, and by default it is a lot of authority. It can `fetch` any URL the page could, open WebSockets, store data in IndexedDB under the page's origin, create further workers, set timers that run code whenever they fire, and read a clock precise to a tenth of a millisecond. A world in PicoCalc OS has none of these: its prelude deletes the two host functions from the global object after capturing them, and the runtime was created without any I/O. The minios prelude does the equivalent by removing names from the global object before the world's code runs:

▸ book/widgets/minios.js

```
const post = G.postMessage.bind(G), listen = G.addEventListener.bind(G);
// Seal: remove what would let a world reach the network, storage, other workers, real
time or true
// randomness, or run code outside a turn. Each name is deleted wherever it lives on the
prototype
// chain (fetch and setTimeout are inherited from WorkerGlobalScope.prototype), then
pinned to
// undefined on the global object.
const SEALED = ["fetch", "XMLHttpRequest", "WebSocket", "EventSource", "importScripts",
"indexedDB", "caches",
  "BroadcastChannel", "Worker", "SharedWorker", "setTimeout", "setInterval",
"requestAnimationFrame",
  "MessageChannel", "performance", "crypto", "postMessage", "close"];
for (const k of SEALED) {
  for (let o = G; o; o = Object.getPrototypeOf(o)) {
    if (Object.prototype.hasOwnProperty.call(o, k)) { try { delete o[k]; } catch (e) { /*
not configurable */ } }
  }
  try { Object.defineProperty(G, k, { value: undefined, writable: false, configurable:
false }); } catch (e) { /* already pinned */ }
}
```

The walk up the prototype chain is necessary, and the reason is a detail of how browsers lay out a worker's global object. Some names are own properties of the global object and others are inherited. In Chromium 141, probing a fresh dedicated worker finds `postMessage`, `close`, `XMLHttpRequest`, `WebSocket`, `Worker`, `MessageChannel` and `requestAnimationFrame` as own properties, while `fetch`, `setTimeout`, `setInterval`, `importScripts`, `indexedDB`, `performance` and `crypto` live on `WorkerGlobalScope.prototype`. A prelude that wrote `delete self.fetch` would delete nothing, and `self.fetch` would still work through inheritance. The loop deletes each name from every object on the chain that owns it, and then pins the name on the global object itself as a non-writable, non-configurable `undefined`, so that a later assignment in the world's code fails and no inherited value shows through.

The list is chosen by authority, not by name. Network and storage interfaces go because a world should reach the outside only through the kernel. `Worker` goes because a world should not create unsupervised children. `setTimeout`, `setInterval`, `requestAnimationFrame` and `MessageChannel` go for a less obvious reason: each lets a world schedule its own code to run later, outside any turn, where no budget alarm is armed and where any effect it records would be posted with some later turn's effects. A world that wants to wait asks the kernel with `sleep(ms)`, which ends the segment and resumes the turn when the kernel says so. `performance` and `crypto` go because they are sources of nondeterminism: a precise clock and true randomness. Time comes instead from the kernel with each message, through a replacement `Date` built exactly as PicoCalc OS's prelude builds it, and `Math.random` is a xorshift generator seeded from the world's name (Chapter 18 discusses why).

When the reader of Try 17.1 replaces the clock world's code with a probe that inspects its own global object, the world logs:

```
fetch=undefined; XMLHttpRequest=undefined; WebSocket=undefined; importScripts=undefined;
indexedDB=undefined;
setTimeout=undefined; requestAnimationFrame=undefined; MessageChannel=undefined;
performance=undefined;
crypto=undefined; postMessage=undefined; close=undefined; navigator=object;
WebAssembly=object;
on prototypes: none; real Date reachable: false
```

The seal is a convenience, not a security boundary, and it is worth being exact about why. What a world can still do is visible in the last entries: it can read `navigator` and `location`, which leak information about the browser but grant no new ability; it can compile WebAssembly, which can compute but has no I/O except through imports the world supplies from what it already has. More importantly, it can allocate memory without limit, since no browser interface caps a worker's heap, and it can run until its budget alarm fires.

The isolation that matters, between this world and the page and between this world and others, is provided by the browser: separate heaps, no shared objects, communication only through `postMessage`. The seal reduces what each world can do with its own share of the page's authority, which is the principle of least authority applied inside a protection domain that someone else built.

17.4 The kernel: mailboxes, turns and effects

The kernel's job is to decide, for each world, what it should run next, and to apply what each segment asked for when it ends. A world in minios is in one of a few states, shown in Figure 17.2. The kernel runs a segment in a world only when the world is idle, meaning it is not running one now. If the world is in the middle of an asynchronous turn and a result it was waiting for has arrived, the next segment is that result. Otherwise, if the world is between turns, the next segment is the first message in its mailbox. A world in the middle of a turn receives no new message: its mailbox is closed until the turn ends, as in PicoCalc OS.

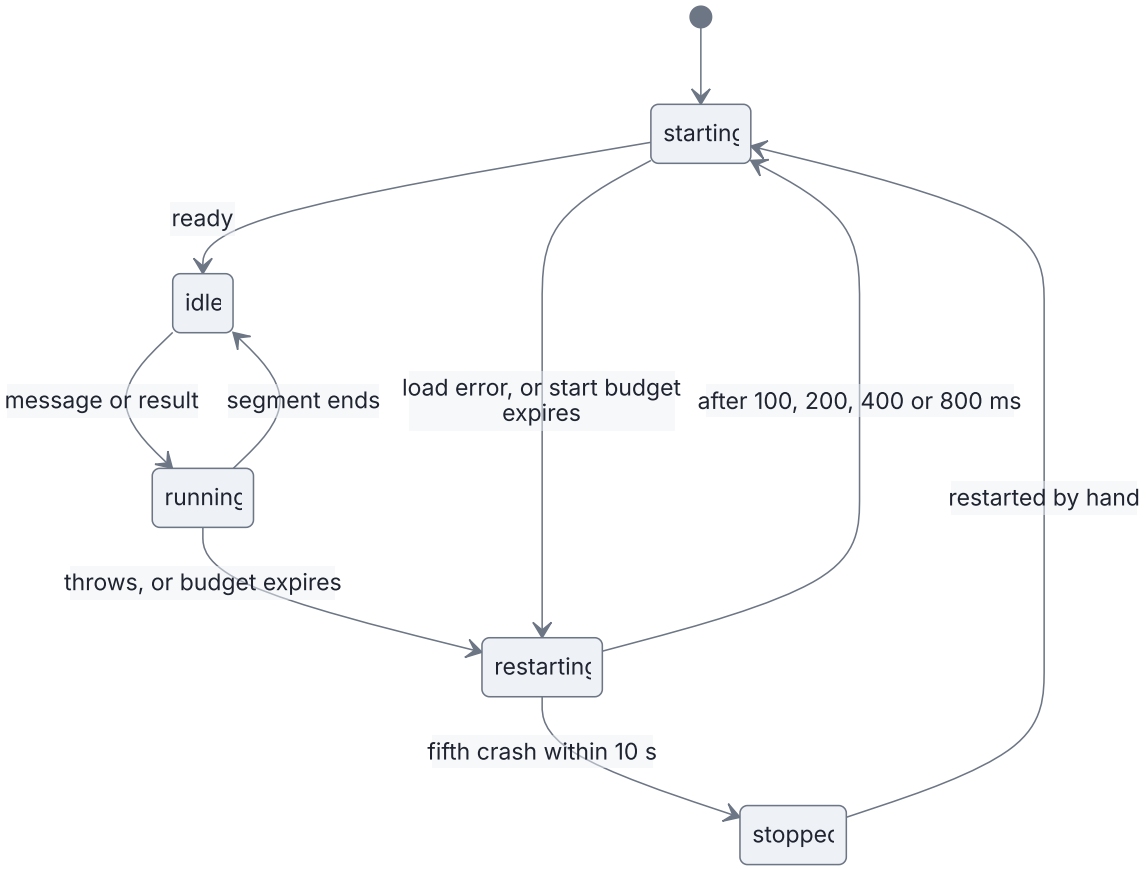


Figure 17.2. The life of a world in minios. A segment runs between idle states; a crash, a failed turn or an expired budget terminates the worker and either schedules a restart or, at the fifth crash within ten seconds, stops the world.

The decision fits in a dozen lines. Results take precedence over messages, and both start a budget alarm:

▸ book/widgets/minios.js

```
pump(w) {
  if (w.status !== "idle" || this.paused) return;
  if (w.settles.length) {
    const s = w.settles.shift();
    w.status = "running";
    this.arm(w, w.budget, "turn");
    w.worker.postMessage({ op: "settle", id: s.id, ok: s.ok, value: s.value, now:
this.now() });
  } else if (!w.inTurn && w.mbox.length) {
    const m = w.mbox.shift();
    w.inTurn = true; w.turn = m; w.status = "running";
    this.arm(w, w.budget, "turn");
    w.worker.postMessage(Object.assign({ op: "msg", now: this.now() }, m));
  }
}
```

On the worker's side, the prelude turns the world's calls into **effects** and holds them until the segment ends. A segment ends in one of three ways: the handler finishes (the turn is done), the handler throws (the turn failed), or the handler calls something whose result comes later (an ask, a device operation, a sleep), which ends the segment and leaves the turn open. The third case is the mechanism by which minios has segments at all, and it is the analogue of PicoCalc OS's rule that a segment ends when a world awaits something the nucleus must do later:

▸ book/widgets/minios.js

```
let fx = [];
const waiting = new Map();
let nextId = 1;
function later(effect) {          // an effect whose result comes later ends the segment
  const id = nextId++;
  fx.push(Object.assign(effect, { id }));
  post({ op: "segment", fx });
  fx = [];
  return new Promise((resolve, reject) => waiting.set(id, { resolve, reject }));
}
G.send = (to, type, payload) => { fx.push({ k: "send", to, type, payload }); };
G.publish = (ch, payload) => { fx.push({ k: "publish", ch, payload }); };
G.log = (...a) => { fx.push({ k: "log", text: a.map(String).join(" ") }); };
G.ask = (to, type, payload) => later({ k: "ask", to, type, payload });
G.device = (op, args) => later({ k: "device", op, args });
G.sleep = (ms) => later({ k: "sleep", ms });
```

Because effects are only recorded until the segment ends, a segment that throws has no effects: the prelude discards the list and posts a failure, and the kernel never sees the sends the handler made before throwing. This is Property 1 of PicoCalc OS, segment atomicity, implemented by buffering in the world instead of in the nucleus. The buffering is trustworthy only because the prelude is: a world that captured `postMessage` before the seal could post effects directly. In minios the prelude runs first and captures it privately, which is the same ordering argument that makes PicoCalc OS's prelude safe (Chapter 10).

The kernel applies a segment's effects in order. Sends and publishes go into mailboxes; an ask is delivered with an identifier and a timeout of two seconds; a device operation is performed only for a world whose manifest entry claims the device; a sleep becomes a timer. Every result is delivered as a settle, and a settle for a world that has restarted since it asked is dropped, because the promise it would resolve died with the old worker:

▸ book/widgets/minios.js

```
// A result for a world: dropped if the world has restarted since it asked (its epoch
moved on).
settle(w, epoch, id, ok, value) {
  if (w.epoch !== epoch || w.status === "stopped") { this.log("stale", w.name, `dropped a
result from epoch ${epoch}`); return; }
  w.settles.push({ id, ok, value });
  this.pump(w);
}
```

Figure 17.3 follows one ask through the system: the reader world asks the counter for its count every three seconds and writes the answer to the device's second line through the driver.

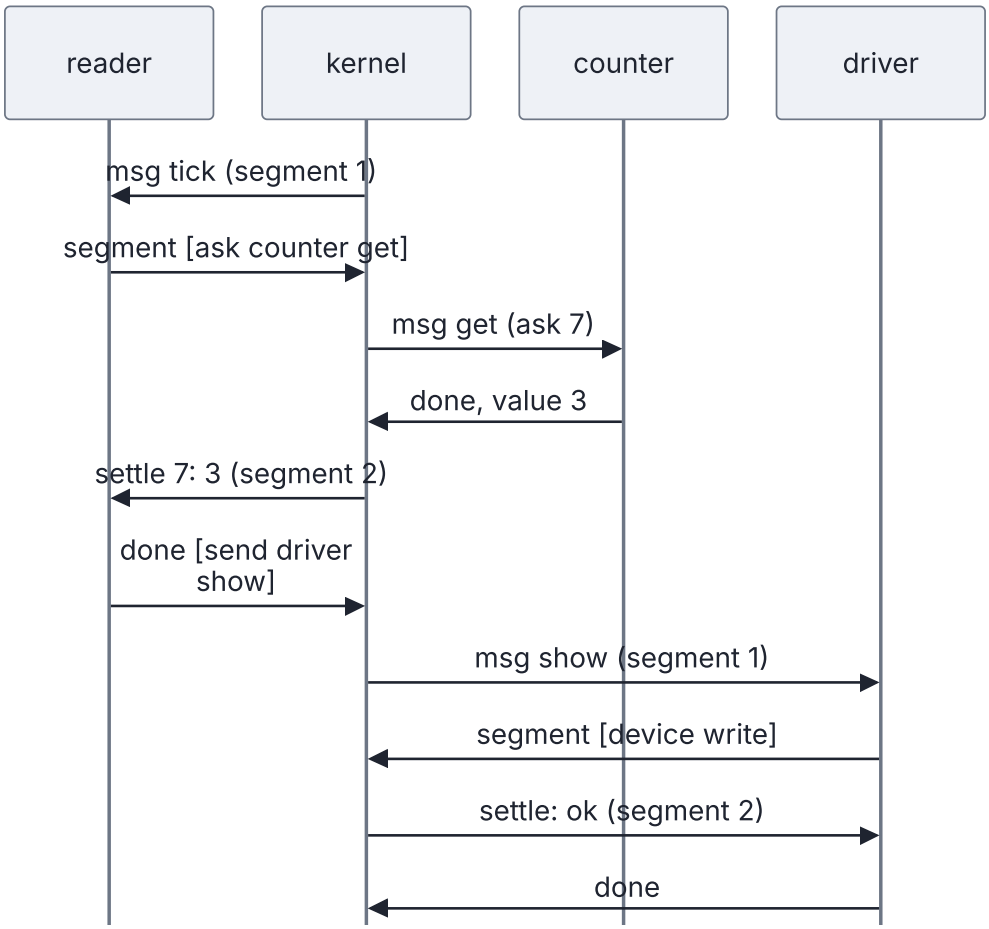


Figure 17.3. An ask in minios. The reader's first segment ends at the ask; the kernel delivers the ask to the counter, whose turn returns the count; the result resumes the reader in a second segment, whose send reaches the driver, whose turn ends at its device operation.

17.5 Budgets by termination

Every segment runs against an alarm. When the kernel starts a segment it records the time and sets a timer for the world's budget; when the segment's end arrives it cancels the timer and records how long the segment took. If the timer fires first, the world is crashed:

▸ book/widgets/minios.js

```
arm(w, ms, what) {
  clearTimeout(w.alarm);
  w.segStart = performance.now();
  w.segKind = what;
  w.alarm = setTimeout(() => this.crash(w, `ran past its ${what} budget of ${ms} ms`),
ms);
}
disarm(w) {
  clearTimeout(w.alarm);
  const ms = performance.now() - w.segStart;
  if (w.segKind === "turn") w.longest = Math.max(w.longest, ms);
  else w.startMs = ms;
}
```

The crash calls `terminate()`, which stops the worker's thread wherever it is, including in the middle of an endless loop that no other mechanism could interrupt. This is possible only because the kernel and the world run on different threads: a loop in a worker does not stop the main thread's timers. The budget therefore protects the page from a world, but not the world's peers from a page whose main thread is busy: if the page's own code holds the main thread, the alarm fires late, and so does every kernel decision.

The budget is measured differently from PicoCalc OS's in three ways. It is real time, not virtual, so it includes the latency of the two `postMessage` calls that start and end the segment and any delay in the main thread's handling of them. It is enforced at the resolution of `setTimeout`, typically a millisecond or worse, not at a fixed count of interpreter steps. And it cannot be checked inside native work, because it is not checked inside the world at all: the kernel only observes that the segment has not ended. In practice the difference matters little for the size of budget minios uses. Normal segments in the recorded run took at most 0.8 ms from the kernel's point of view, a hang injected into the driver was terminated about 100 ms after its turn began, and the driver was running again 100 ms after that.

What termination costs is the same in both systems: the world's entire state. PicoCalc OS reaches the same outcome by a different route. An overrun in a QuickJS runtime raises an uncatchable error at the next interrupt, the nucleus discards the segment's effects and then disposes of the whole runtime, because a runtime interrupted at an arbitrary point cannot be trusted to hold consistent objects. minios cannot interrupt a worker at all, only end it, and so it never faces the question. In both cases a restarted world begins from its initial state, and anything it knew must be rebuilt from its clients, its device or its own messages (Chapter 15).

17.6 Supervision and a driver

minios's supervisor follows PicoCalc OS's policy exactly, so that the effect of a policy can be observed in a system with real time and real threads:

▸ book/widgets/minios.js

```
crash(w, why) {
  if (w.status === "stopped" || w.status === "restarting") return;
  clearTimeout(w.alarm);
  w.worker.terminate();
  w.worker = null;
  w.timers.forEach(clearInterval);
  w.timers = [];
  if (w.turn && w.turn.ask) this.settleAsk(w.turn.ask, false, `${w.name} crashed:
${why}`);
  w.inTurn = false; w.turn = null; w.settles = [];
  w.mbox = w.mbox.filter((m) => m.type !== "__start" && m.type !== "__tick");
  const t = this.now();
  w.crashes = w.crashes.filter((c) => t - c < WINDOW_MS).concat(t);
  if (w.crashes.length >= STOP_AFTER) {
    w.status = "stopped";
    for (const m of w.mbox) if (m.ask) this.settleAsk(m.ask, false, `${w.name} is
stopped`);
    w.mbox = [];
    this.log("stop", w.name, `${why}; ${STOP_AFTER} crashes in ${WINDOW_MS / 1000} s, so
it stays stopped`);
    return;
  }
  const delay = BACKOFF_MS[w.crashes.length - 1];
  w.status = "restarting";
  this.log("crash", w.name, `${why}; restarting in ${delay} ms`);
  const epoch = w.epoch;
  setTimeout(() => { if (w.epoch === epoch && w.status === "restarting") { w.restarts++;
this.start(w); } }, delay);
}
```

Each line corresponds to a rule from Chapter 14. The ask the world was handling fails at once, so its asker does not wait for the timeout. Results the world was waiting for are discarded, and the epoch check discards any that arrive later. Messages in the mailbox survive, because most of them are still meaningful to the next instance; timer ticks and the start message are removed, because the new instance will schedule its own. The crash times within the last ten seconds decide between a restart after 100, 200, 400 or 800 ms and a stop.

The device is a two-line character display behind a 5 ms bus. It accepts `write` only after `init`, and `init` clears it. The driver world initialises it when it starts and writes whatever it is sent:

▸ book/widgets/minios.js

```
world("driver")
  .onStart(async () => { await device("init"); })
  .on("show", async (S, p) => { await device("write", p); });
```

This driver holds no state of its own, and that is what makes it restartable without help. When it crashes and restarts, it reinitialises the display, which goes blank; the next tick from the counter repaints the first line and the next answer from the reader repaints the second. The display's content is rebuilt by its clients, not remembered by its driver. The counter is the opposite case: its state is the count, and when it crashes the count starts again from zero. In the recorded run behind the print edition of Try 17.1, the counter was crashed by hand at 3.2 s; one second later the display read `ticks 1` on its first line, from the new counter, and still showed `asked: 3` on its second, from the reader's last ask of the old one. Whether losing the count is acceptable is a design decision about where state should live, which Chapter 15 makes in general.

TRY 17.1

An operating system of Web Workers

Four worlds run in this page: a clock that publishes the time every second, a counter that counts the ticks, a reader that asks the counter for its count every three seconds, and a driver for the display at the top. Press **Crash** on the counter and watch the first line of the display start again from one while the second line keeps the old count. Press **Hang** on the driver: its budget of 100 ms expires, the kernel terminates its worker and restarts it, and the display goes blank until the clients repaint it. Crash the reader five times within ten seconds and it stays stopped, and **Restart** brings it back. Open the editor to change a world's code and restart it with the new version; try a world that calls `fetch`, or one that loops at the top level.

```
ticks 1
asked: 3
```

initialised · 6 operations

World	Status	Epoch	Restarts	Turns	Mailbox	Start	Longest segment
clock	idle	1	0	4	0	37.8 ms	0.7 ms
counter	idle	2	1	5	0	11.3 ms	0.8 ms
reader	idle	1	0	1	0	56.1 ms	0.7 ms
driver	idle	1	0	6	0	55.9 ms	0.6 ms

Kernel events from a recorded run, the counter crashed by hand at 3.2 s:

3308 ms	↑	**counter** started in 11.3 ms (epoch 2)
3195 ms	×	**counter** a fault injected by the kernel; restarting in 100 ms
64 ms	↑	**driver** started in 55.9 ms (epoch 1)
63 ms	↑	**reader** started in 56.1 ms (epoch 1)
43 ms	↑	**clock** started in 37.8 ms (epoch 1)
43 ms	↑	**counter** started in 36.4 ms (epoch 1)

17.7 PicoCalc OS itself, in the page

minios imitates PicoCalc OS with the browser's own mechanisms. The other way to run PicoCalc OS in a page is to run PicoCalc OS: compile the nucleus, the simulated board and QuickJS to WebAssembly and drive the result from JavaScript. The two are different kinds of system. minios is an operating system built from the browser's processes. PicoCalc OS in WebAssembly is a complete operating system, with its own scheduler and its own isolation, running inside one of the browser's processes as a single program.

The port changed almost nothing. The same C sources are compiled with clang for the `wasm32-wasip1` target against the WASI system library, with one file replacing the native host's `main`. That file has no main loop: the page calls its exported functions. There is no filesystem either, so the page gives the build the files it would have read from `js/` through an export that stores them in a table in memory, and the kernel's `read_file` looks them up there. The virtual clock advances only when the page asks it to:

▸ host/main_wasm.c

```
// Run the system for ms virtual milliseconds.
EXPORT(pc_run) void pc_run(double ms) {
  if (!booted) return;
  kernel_run_until(K.vnow + (vtime_t)(ms * 1000));
  fflush(stdout); fflush(stderr);
}
```

The page decides how virtual time relates to real time. The demonstration page (and the interactive figure below) runs the system, on every animation frame, for the real time that has passed since the last frame, capped at 50 ms so that a page returning from the background does not run minutes of simulation in one frame:

▸ wasm/demo.html

```
// pacing: each animation frame runs the system for the real time that passed (at most 50
ms)
let last = performance.now(), busy = false, lastStats = 0, realSpent = 0, virtSpent = 0;
async function frame(now) {
  requestAnimationFrame(frame);
  if (busy) return;
  busy = true;
  const dt = Math.min(50, now - last);
  last = now;
…
  else { pc.run(dt); screen = pc.screen(); vnow = pc.now(); }
```

The determinism of Chapter 18 survives the port, and checking it took care. The simulated panel fills its memory with noise at power-on from the C library's `rand()`, seeded with 1, and WASI's C library computes a different sequence from glibc's. The WebAssembly build therefore carries its own copy of glibc's generator, checked equal to glibc's over two million outputs for four seeds, so the noise is identical. The host's clock is never read by anything the system does, and the loader's clock returns a fixed instant in case anything does. With those two details, the screens the WebAssembly build produced in Chromium matched the native 32-bit build's captures pixel for pixel: 11 of 11 captures from three scripts when run on the page's main thread, and 8 of 8 when run in a worker. Run under Node, 21 of 22 captures matched; the one that differed is the About screen, which prints heap sizes, and WASI's allocator (dlmalloc) lays out the heap differently from glibc's. That is the same exception the native builds have among themselves.

The build is 1,167,140 bytes (384 KB compressed with gzip) and imports two functions of its own and eleven WASI functions, served by the loader's shim. Measured in headless Chromium 141 on the book's measurement machine, taking the best of three runs, compiling it took about 5 ms and booting the thirteen worlds 7 to 9 ms. A minute of the idle home screen then ran in about 92 ms, roughly 650 times faster than real time, and a minute of snake on autopilot in 0.63 to 0.75 s, 80 to 96 times faster. The native 32-bit build, measured the same way after its start-up on the same machine, ran the idle minute about 1,000 times faster than real time and the snake minute about 113 times faster, so the WebAssembly build runs at roughly two-thirds to three-quarters of native speed here. When paced to real time, running the idle system for one 16.7 ms frame took about 0.03 ms, and copying its screen out of WebAssembly memory and drawing it on a canvas took about 0.8 ms. Most of the cost of a live PicoCalc OS in a page is drawing its screen, not running it.

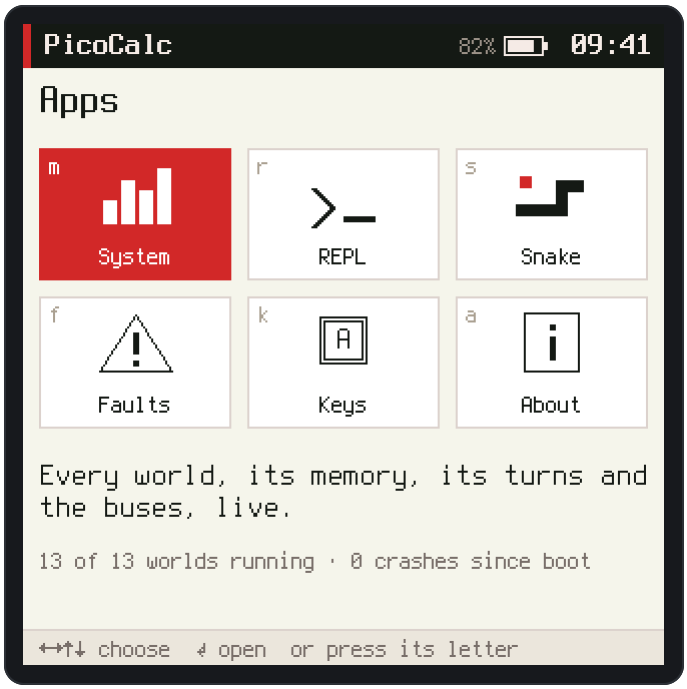
One failure mode is specific to pages. Compiling WebAssembly is something a page's Content Security Policy can forbid, and some hosting environments do. The loader detects it and rejects with an error whose code is `wasm_blocked`, and the interactive figure shows a screenshot in place of the live system when that

happens.

TRY 17.2

PicoCalc OS in this page

This is PicoCalc OS running in WebAssembly: thirteen worlds, the compositor, and the simulated keyboard controller and panel, paced to real time. Click the screen and use the arrow keys and Enter to move around the launcher, or the on-screen keys on a phone. Press **Kill display** and watch the screen come back after the driver's 370 ms start-up sequence; press **Kill keyboard** and then a key during its restart. The worlds table shows each restart, and the console UART panel shows the `#OCT1` fault frame each kill produces. If this viewer does not allow WebAssembly, a screenshot of the home screen appears instead, with the reason.



17.8 Two designs compared

The two systems in this chapter make opposite choices on almost every axis, and Table 17.2 sets them out. minios gets its isolation from the browser: each world has its own heap, garbage collector and thread, protected by the browser's process and memory model, so a bug in one world cannot corrupt another's objects and an endless loop in one cannot stop another. It pays for that with start-up cost, message cost and nondeterminism: workers take milliseconds to start, every message is structured-cloned across threads, and the order of events depends on the operating system's thread scheduling and the page's own load.

PicoCalc OS in WebAssembly gets its isolation from QuickJS and the nucleus, inside one WebAssembly instance that is itself inside one browser process. It pays for that by trusting more code: a bug in QuickJS or in a host call can break isolation between worlds, and the browser cannot help. In exchange it is deterministic, it accounts for work exactly on a virtual clock, it enforces memory limits per world, and its worlds are cheap.

Table 17.2. minios and PicoCalc OS compiled to WebAssembly, run in the same page.

	minios	PicoCalc OS in WebAssembly
Unit of isolation	Web Worker: own thread, heap, event loop	QuickJS runtime inside one WebAssembly instance
Isolation enforced by	the browser	QuickJS and the nucleus
Parallelism	worlds run in parallel	one lane
Time	real time	virtual time, paced to real time by the page
Determinism	none: depends on thread scheduling	screens identical to the native build
Budget	alarm and `terminate()`, real milliseconds	interrupt counter every 1,000 steps, virtual milliseconds
Memory limit per world	none	yes
Starting a world	6–56 ms measured	about 66 µs charged
A message	`postMessage` with structured clone	copied JSON inside one address space
What a crash costs	the worker	the runtime

Neither is the better design in general. A browser application that wants to run plug-ins written by strangers should use workers, because only the browser's isolation is strong enough for code that is actively hostile. A system that must be predictable, testable and small, and whose components are chosen by its owner, gains more from the virtual clock and the per-world accounting than it loses by trusting its interpreter. PicoCalc OS made the second choice for a handheld; this chapter shows the first choice working in a page, under the same supervisor, so that what each costs can be measured rather than assumed.

KEY POINTS

- A dedicated Web Worker provides what a world needs from a protection domain: its own heap, its own event loop and no shared objects; a kernel on the page's main thread can sit on every path between worlds because workers can only post messages.
- A worker's authority is its global object, and some of its most powerful members are inherited from `WorkerGlobalScope.prototype`, so sealing must delete names along the whole prototype chain and pin them on the global object.
- Removing timers, `MessageChannel` and `requestAnimationFrame` keeps a world's code inside turns, where budgets apply; removing `performance` and `crypto` removes a precise clock and true randomness.
- Effects buffered by the prelude and posted when a segment ends give segment atomicity without kernel support, provided the prelude runs first and holds `postMessage` privately.
- `terminate()` is the only preemption a page has, so budgets are alarms followed by termination; like PicoCalc OS's overrun handling, it costs the world all of its state.
- A driver that holds no state can be restarted without help, because its clients repaint the device; a world whose state is its purpose, such as a counter, loses it on restart unless it is stored elsewhere.
- PicoCalc OS compiled to WebAssembly produces screens identical to the native 32-bit build, runs about 650 times faster than real time when idle in Chromium, and spends most of its in-page cost on drawing its screen.
- Workers buy hardware-backed isolation with start-up cost, message cost and nondeterminism; PicoCalc OS buys determinism, exact accounting and cheap worlds by trusting its interpreter.

Building the same operating system twice, once from the browser's parts and once by compiling it, separated what the design requires from what the platform provides. The virtual clock was the one feature minios could not imitate: real time made its budgets approximate and its runs unrepeatable. The next chapter is about that clock: how a deterministic simulation of a whole system with devices is built, how it is calibrated against hardware, and what it cannot tell you.

EXERCISES

1. BASIC List every way a minios world can affect anything outside its own worker. For each, name the code in the kernel that decides whether it happens.
2. BASIC Why does `start()` capture `epoch` in a local variable and compare it in the `onmessage` handler? Describe a sequence of events that would go wrong without the check.

3. ANALYSIS The reader world asks the counter, and the counter crashes while handling the ask. Trace what the reader sees and when, using `crash()` and `settleAsk()`. What would the reader see if the counter were stopped instead?

4. ANALYSIS A world awaits two asks at once with `Promise.all`. How many `segment` messages does it post, and in what order can the two settles arrive? Show that the kernel never runs two segments of the same world at once.

5. ANALYSIS The budget alarm is set with `setTimeout` on the main thread. If the page's own code runs a 300 ms task, what happens to a world that hangs during that task? To a world whose 20 ms segment completes during it? Is the second world ever wrongly terminated?

6. CODE The seal leaves `queueMicrotask`. Show that a world cannot use it to run code outside a turn, by explaining when microtasks run relative to the task that queued them.

7. CODE Add a world that calls itself with `ask(self, ...)` from inside a handler. Predict what happens, run it in Try 17.1, and explain the result in terms of closed mailboxes. How does PicoCalc OS behave in the same case?

8. CODE Add a memory guard to minios: have the prelude estimate its heap with a periodic `performance.measureUserAgentSpecificMemory()` call, or explain why that is not possible in this page (consider cross-origin isolation, Section 16.3). Propose an alternative.

9. CODE Give the counter world persistent state: before each turn ends, it sends its count to a new `store` world, and when it starts it asks `store` for the last value. Crash the counter and show that the count survives. What happens if `store` crashes?

10. CODE Using the PicoCalc OS loader, run the same script twice in two separate instances in one page, and compare their screens after each `wait`. Then run one of them in a worker. Are they identical? Explain why or why not.

11. ANALYSIS From the measurements in Section 17.7, estimate the processor share of a live PicoCalc OS page at 60 frames per second if the screen were drawn only when a frame changed. How would you find out from inside the page whether the screen changed?

12. PROJECT Give minios a single lane: only one world may run a segment at a time, chosen by priority and arrival order as in PicoCalc OS's `pick`. Measure what it costs in throughput with four busy worlds, and what it buys in the order of events in the log.

13. DISCUSSION For a plug-in system in a web application (third-party extensions in an editor, say), which properties of minios would you keep, which of PicoCalc OS would you want, and which would you give up?

FURTHER READING

[WHATWG HTML 2026] The worker and event-loop sections of the HTML Standard define exactly what a worker's global object contains and when its tasks and microtasks run.

[Haas et al. 2017] Why WebAssembly's linear memory and structured control flow make it safe to run a whole operating system as one untrusted module.

[Reis et al. 2019] The browser's own answer to the trade-off in Section 17.8, measured at scale.

Deterministic simulation

PicoCalc OS was written, debugged and measured before it ever ran on the handheld it was written for. Every screen in this book, every latency and every recovery time came from a simulator: the same nucleus and the same JavaScript worlds, linked with models of the two peripheral controllers instead of the ESP32-P4's drivers, running on a clock that advances by the work the system does rather than by the passing of real time. This chapter explains how that simulator is built and why it can be trusted for some questions and not others. By the end you should be able to build a deterministic simulator for a system with devices: a discrete-event loop with a virtual clock, a cost model calibrated against hardware, register-level models of the peripherals, and a list of every source of nondeterminism with the way each one is closed. You should also be able to say, for a given claim about the system, whether the simulator is evidence for it.

18.1 Why simulate a system you could run

A system that touches hardware is hard to test for four reasons, and a simulator answers each. The first is **repeatability**. When a bug depends on the order in which a key press, a timer and a bus transfer happen to arrive, running the system again rarely reproduces it, and a bug that cannot be reproduced cannot be studied. A simulator that makes the same choices every time it is given the same inputs turns every observed failure into a test case. The second is **observability**. On the device, watching the system changes its timing (a log line over a serial port takes milliseconds), and some quantities, such as the exact moment pixels changed on the panel, are not observable at all without an oscilloscope. In a simulator every event can be recorded at no cost to the system's own timeline. The third is **speed**. A simulated system that is idle can skip straight to its next event, so an hour of a mostly idle handheld runs in seconds. The fourth is **control**: a simulator can kill a world at an exact moment, press a key at the worst possible phase of a poll, or make a bus fail, and it can do so in a script that runs the same way every time.

The idea is old in hardware design and was made famous in software by FoundationDB, a distributed database whose engineers built it so that the whole cluster could run inside one deterministic process. In their account, "all sources of nondeterminism and communication are abstracted, including network, disk, time, and pseudorandom number generator", multithreading is avoided, and the simulation "injects machine, rack, and data-center failures and reboots" along with network partitions and disk corruption [Zhou et al. 2021]. Determinism means "every such bug can be reproduced and investigated", and because the clock jumps to the next event, the simulation "can run arbitrarily faster than real-time if CPU utilization within the simulation is low". FoundationDB applies this to a cluster of servers; PicoCalc OS applies it to one handheld and its two peripheral chips, and the questions are the same: what must be abstracted, and how faithfully.

The design constraint that makes it possible is the one Chapter 8 and Chapter 9 argued for on other grounds: a world can affect nothing except through the nucleus. Every message, every bus transfer, every timer and every clock reading passes through a small number of C functions. Replace what is underneath those functions and the worlds cannot tell the difference, because they never had any other way to observe the machine.

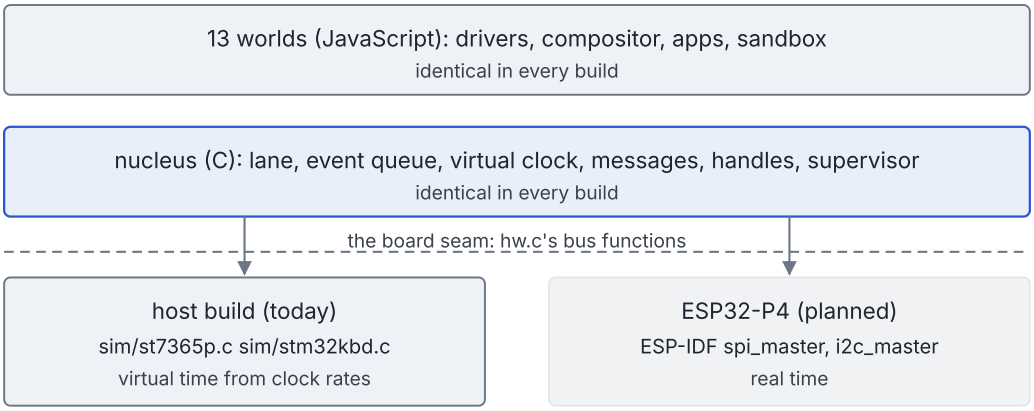


Figure 18.1. The board seam. The worlds and the nucleus are the same in every build. On the host, the nucleus's hardware layer calls register-level models of the panel and keyboard controllers and charges time from clock rates; on the device, the planned port replaces those calls with ESP-IDF's SPI and I2C drivers and real time.

18.2 Discrete-event simulation

A **discrete-event simulation** represents a system's state as data and its evolution as a sequence of **events**, each of which happens at an instant of **simulated time** and changes the state. Between events nothing happens, so the simulator does not need to model the passage of time at all: it keeps the pending events in a queue ordered by time, repeatedly removes the earliest, sets the clock to its time, and handles it, which may add further events to the queue. The loop is short:

```
clock = 0
queue = priority queue of (time, sequence, event)
while queue is not empty and queue.earliest().time <= until:
    (time, _, event) = queue.pop_earliest()
    clock = time
    handle(event)          # changes the state; may schedule events at clock + delay
clock = until
```

Two details decide whether such a loop is deterministic. The first is how ties are broken. Two events scheduled for the same instant must be handled in some order, and if that order depends on anything other than the inputs, such as the memory addresses of the event records or the iteration order of a hash table, two runs can diverge. The standard remedy is a **sequence number**: a counter incremented each time an event is created, used as the second key of the ordering, so that events at the same time are handled in the order they were scheduled. The second detail is that handling an event must itself be deterministic, which is the subject of Section 18.5.

PicoCalc OS's event queue is a binary heap of event records ordered by exactly that pair:

▸ nucleus/kernel.c

```
// ---- the event queue: a binary heap ordered by (time, sequence) ----------------------
---------------------
static event_t **heap; static int heap_n, heap_cap;
static bool before(event_t *a, event_t *b) { return a->t < b->t || (a->t == b->t && a-
>seq < b->seq); }
```

Events in this queue are the things that happen to the system from outside the lane: a message arriving in a mailbox, a timer firing, a bus transfer finishing and settling a promise, a world's restart coming due, a key going down on the simulated keyboard. The lane itself is not driven by events. Whenever the lane is free

and some world has something to run, the kernel runs a segment, which takes the virtual time its cost model says it takes. So the kernel's main loop interleaves two kinds of progress, choosing at each step whichever comes first in virtual time:

▸ nucleus/kernel.c

```
void kernel_run_until(vtime_t until) {
  for (;;) {
    bool is_settle;
    world_t *R = pick(&is_settle);
    vtime_t t_lane = K.vnow > K.lane_free ? K.vnow : K.lane_free;
    vtime_t t_ev = heap_n ? heap[0]->t : VT_INF;
    if (R && t_lane <= until && t_lane < t_ev) { K.vnow = t_lane; run_segment(R,
is_settle); continue; }
    if (heap_n && t_ev <= until) { event_t *e = pop(); if (e->t > K.vnow) K.vnow = e->t;
handle_event(e); continue; }
    break;
  }
  if (K.vnow < until) K.vnow = until;
}
```

Figure 18.2 draws the same decision. The comparison `t_lane < t_ev` is strict, so when a segment could start at the same instant an event is due, the event is handled first: a message that arrives at the moment the lane frees up is in the mailbox before the next pick. That is a choice, and a different one would also be deterministic; what matters is that it is a fixed rule.

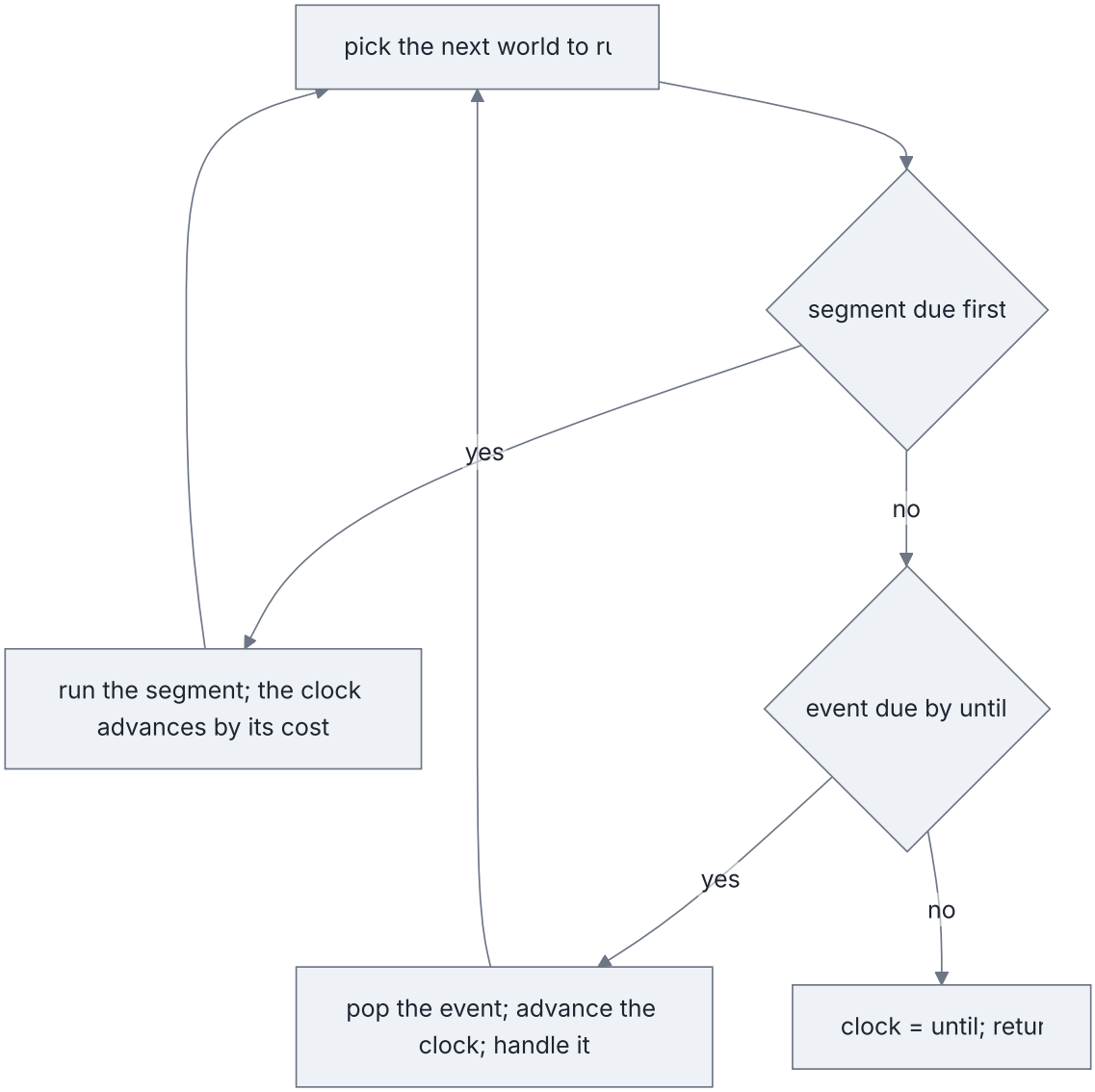


Figure 18.2. kernel_run_until. A segment is due first when some world is runnable and the lane frees up before the next event and before the end of the run; otherwise the earliest event is handled if it is due. Both advance the virtual clock; nothing else does.

The speed of the simulator follows from this loop. When the handheld sits on its home screen, the only activity is the keyboard driver's poll every 20 ms, the power driver's battery read every 5 s and a few redraws; between them the queue's earliest event is milliseconds away and the loop jumps straight to it. A minute of

idle home screen is about 3,000 keyboard polls, each a handful of segments and events, and the native 64-bit host build runs it in about 41 ms of host time after boot, some 1,470 times faster than real time (Section 18.7 has the full measurements). Snake on autopilot keeps the lane busier, about 11 ms of work every 110 ms tick, and runs about 165 times faster than real time.

18.3 A clock driven by work

Events give the simulator its skeleton; the cost model gives its segments their length. A segment of a world that executes s interpreter steps and asks for native work costing n takes

$$\text{cost} = c_0 + s \cdot \tau + n \tag{18.1}$$

of virtual time, where $c_0 = 40$ µs is the fixed cost of entering and leaving a world and τ is the cost of one step (Chapter 4 introduced the lane, Chapter 11 the budget that bounds a segment). A **step** is one decrement of QuickJS's interrupt counter, which the interpreter performs at every jump, every conditional branch and every function call. Steps are a measure of control flow, chosen because QuickJS already counts them: the interrupt counter exists so that an embedding can stop a runaway script, and PicoCalc OS reads it to learn how much a segment did. Native work, the pixels a world fills, the glyphs it draws, the framebuffer bytes the compositor compares and the bytes of message payload encoded or decoded, is charged per unit by the host calls that do it. While a segment runs, the kernel can say what time it is in the segment's own frame, which is what `Date.now()` returns to the world:

▸ nucleus/kernel.c

```
vtime_t kernel_now(void) {
  if (frozen_now >= 0) return frozen_now;
  if (!K.cur) return apply_at >= 0 ? apply_at : K.vnow;
  return K.vnow + PROFILE.turn_base_us + (vtime_t)(vm_steps(K.cur) * PROFILE.ns_per_step
/ 1000.0) + K.seg_cost;
}
```

The model is checkable line by line in a trace. These are the first lines of the recorded key press in `bench/traces/keypress.jsonl`, where times are in virtual microseconds:

▸ bench/traces/keypress.jsonl


```
{"k":"key","t":1200000,"w":"","text":"0xb7 down"}
{"k":"seg","w":"keyboard","t":1200078,"dt":59,"steps":26,"native":2,"kind":"msg","type":"
every 20 ms"}
{"k":"bus","bus":"i2c0","w":"keyboard","t":1200137,"t1":1206137,"bytes":4,"what":"write
0x04, wait, read 1"}
{"k":"seg","w":"keyboard","t":1206137,"dt":50,"steps":14,"native":1,"kind":"settle","type
":"every 20 ms"}
{"k":"bus","bus":"i2c0","w":"keyboard","t":1206187,"t1":1213087,"bytes":5,"what":"write
0x09, wait, read 2"}
```


The keyboard driver's poll segment executed 26 steps and 2 µs of native work, so it cost 40 + 26 × 0.665 + 2 = 59.3 µs, recorded as 59. Its continuation after the first bus read executed 14 steps and 1 µs of native work: 40 + 9.3 + 1 = 50.3, recorded as 50. The bus transfers in between take the time their bits take on the wire, which Section 18.4 derives.

That leaves τ, the only constant in the model that rests on a measurement of the device. A measurement made earlier on the ESP32-P4, with QuickJS running natively under ESP-IDF, timed three programs: a 10,000-iteration integer loop at 11 ms, the same loop with 100,000 iterations at 133 ms, and a recursive `fib(20)` at 31 ms. The kernel counts the steps of the 100,000-iteration loop when it starts and sets τ so that the loop costs exactly 133 ms:

▸ nucleus/vm.c

```
void vm_calibrate(void) {
  int64_t loop = count_steps("(()=>{let s=0; for(let i=0;i<100000;i++) s+=i; return s;})
()");
  int64_t fib = count_steps("(()=>{function fib(n){return n<2?n:fib(n-1)+fib(n-2)} return
fib(20);})()");
  if (loop > 0) PROFILE.ns_per_step = 133e6 / (double)loop;
  vm_fib20_ms = fib * PROFILE.ns_per_step / 1e6;
```

The loop takes 200,002 steps (two per iteration, the loop's back-edge and its condition), so τ = 133 ms / 200,002 = 665 ns. The host build prints the calibration when run with `-v`, and its `steps` command prices any expression:

```
$ ./build32/picocalc-host -v -
calibration: the 100k loop is 200002 steps, so a step is 665 virtual ns; fib(20) is 43783
steps = 29.1 virtual ms (31 ms on the device)
[     0.000] booting 13 worlds from js/system.json
steps (()=>{function fib(n){return n<2?n:fib(n-1)+fib(n-2)} return fib(20);})()
RESULT {"bench":"steps","code":"(()=>{function fib(n){return n<2?n:fib(n-1)+fib(n-2)}
return fib(20);})()","steps":43783,"ms":29.115}
```

A model with one parameter fitted to one measurement matches that measurement by construction, so only the other measurements test it. `fib(20)` is dominated by calls rather than loops, and the model prices it at 29.1 ms against 31 ms measured, 6 percent low. The 10,000-iteration loop is priced at 13.3 ms against 11 ms, 21 percent high, a difference that more likely reflects fixed overheads included or excluded by the device's timer at that short scale than a difference in the cost of a step. Table 18.1 lists these and other programs. The last rows show the model's blind spot: allocating an object and an array in every iteration costs exactly as many steps as the integer loop, and serialising a 50-row table to JSON a hundred times costs 354 steps, about 0.24 ms, because the work happens inside built-in functions that never touch the interrupt counter.

Table 18.1. Interpreter steps and modelled cost for small programs (counted by the host build's steps command). Device times are from the earlier ESP32-P4 measurement where one exists.

Program	Steps	Model	Device
100,000-iteration integer loop	200,002	133.0 ms	133 ms
10,000-iteration integer loop	20,002	13.3 ms	11 ms
`fib(20)`, recursive	43,783	29.1 ms	31 ms
10,000 iterations allocating an object and an array	20,002	13.3 ms	
`sort` of 1,000 numbers with a comparator	10,748	7.1 ms	
`JSON.stringify` of a 50-row table, 100 times	354	0.24 ms	

18.4 Buses and devices as models

The simulated board has two halves: the buses, which turn bytes into time, and the devices, which turn bytes into state. The buses are modelled from their clock rates. An I2C transaction sends nine clocks per byte (eight data bits and an acknowledge), plus the address byte, plus two clocks for the start and stop conditions per phase, and a transaction may hold the bus for a gap between its write and its read phases:

▸ history/first-pass/nucleus/hw.c

```
static vtime_t wire_us(double bits, int hz) { return (vtime_t)(bits * 1e6 / (hz > 0 ? hz
: 1) + 0.5); }
…
  // a write phase (address + bytes, 9 clocks each), an optional gap with the bus held,
then a read phase
  double bits = (d->nbytes ? (1 + d->nbytes) * 9 + 2 : 0) + (d->nread ? (1 + d->nread) *
9 + 2 : 0);
  vtime_t end = start + wire_us(bits, h ? h->hz : 10000) + d->delay;
```

The keyboard driver's first read in the trace above writes one byte and reads one: (1 + 1) × 9 + 2 = 20 clocks to write, 20 to read, 40 clocks at 10 kHz is 4.0 ms, and the 2 ms gap makes 6.0 ms, from 1,200,137 µs to 1,206,137 µs. The second reads two bytes: 20 + 29 = 49 clocks, 4.9 ms, plus the gap is 6.9 ms. Those two numbers, 12.9 ms together, are the largest part of the key-to-screen latency of Chapter 19, and they come straight from the PicoCalc's 10 kHz I2C clock. SPI is modelled the same way at 8 clocks per byte, with 2 µs per command for the chip-select and data/command toggles and 12 µs of DMA setup per 32 KiB chunk.

Each bus is serialised: a transfer starts when the bus is free and when the segment that asked for it ends, whichever is later. Transfers run concurrently with the lane, so a world can ask for a 6 ms read, end its segment, and let other worlds run while the bits go out. When the transfer finishes, its completion is an event that settles the world's promise.

The devices are modelled at the level of their registers, which is the level the drivers see. The STM32 keyboard controller model follows the firmware in clockworkpi's repository: a write of a register number (with bit 7 set for a write) prepares a two-byte answer, a read returns it and clears it, and reading the FIFO register pops one key event. The ST7365P panel model keeps the controller's state: whether it sleeps, whether the display is on, the pixel format, the memory access control bits, the column and row window, the write pointer, and a 320 × 480 frame memory. A capture reads that memory through the same transformations the glass applies (mirrored columns unless the MX bit is set, swapped colours unless the BGR bit is set, inversion unless it is turned on), so a capture shows what a person would see, including the effect of a driver getting a bit wrong.

The question for any device model is how faithful it must be, and the answer this project used is: faithful enough that every mistake a driver can make shows up somewhere. The panel controller's datasheet requires a driver to wait 5 ms after a software reset before sending another command and 120 ms after leaving sleep before turning the display on. A driver that skips these waits works on many units and fails on others, which makes it the kind of bug a simulator should catch, so the model checks them:

▸ sim/st7365p.c

```
void st7365p_cmd(uint8_t cmd, vtime_t t) {
  if (t - P.t_reset < MS(5)) warn(t, "command 0x%s arrived less than 5 ms after SWRESET",
(char[]){ "0123456789abcdef"[cmd >> 4], "0123456789abcdef"[cmd & 15], 0 });
…
    case 0x29:
      if (t - P.t_slpout < MS(120)) warn(t, "DISPON %s less than 120 ms after SLPOUT",
"sent");
```

The panel also starts with its frame memory full of noise, as real panels do, so a driver that turns the display on before writing a full frame shows garbage in the capture rather than a clean black screen.

Two real bugs were found this way before the code ever ran on hardware, and both are bugs of interaction that a unit test of either driver alone would not have found. In the first, the keyboard driver and the power driver both talked to the STM32 with a write followed, as a separate transfer, by a read. On the first boot of the full system in the simulator, the keyboard driver reported a key `R` in state 11 and the battery read 0 percent: the power driver had written the battery register's number, the keyboard driver's read had collected the battery's answer (register 0x0B is 11, and 82 percent is the character `R`), and the power driver's read had found the answer already cleared. The fix was a bus transaction that holds the bus between its write and its read (Chapter 13). In the second, killing the display driver made it crash again as it came back: a flush request sent just before the kill waited in the mailbox through the restart and reached the new instance before the compositor had handed it the framebuffer again (Chapter 15). Neither depended on timing precision; both depended on the simulator running the real drivers against a device whose protocol was modelled exactly.

18.5 Sources of nondeterminism

A simulator is deterministic only if every input to every decision is either part of the script or derived from it. The way to make that true is to list every source of nondeterminism a program can reach and close each one. For PicoCalc OS the list has six entries.

Time. A world that could read the host's clock would behave differently on every run. The prelude replaces `Date` with a function that reads the virtual clock, and makes the engine's own `Date` unreachable even through a date object's `constructor` property (that path was open in an early version and was found by checking the paper's claims against the code, Chapter 10):

▸ js/prelude.js

```
const RealDate = Date;
const vnowMs = () => BOOT + k("now");
function VDate(...a) {
  if (!new.target) return new RealDate(vnowMs()).toString();
  return a.length ? new RealDate(...a) : new RealDate(vnowMs());
}
VDate.prototype = RealDate.prototype;
VDate.now = vnowMs;
```

The WebAssembly build closes the same source one level down: the loader's WASI clock returns a fixed instant, so even the C library cannot read real time (Chapter 17).

Randomness. `Math.random` in each world is a xorshift generator seeded from a hash of the world's name, so a world's random choices (the snake's autopilot breaks ties randomly) repeat exactly, and two worlds do not share a stream:

▸ js/prelude.js

```
let seed = (k("seed") >>> 0) || 1;
Math.random = function () {
  seed ^= seed << 13; seed >>>= 0;
  seed ^= seed >>> 17;
  seed ^= seed << 5; seed >>>= 0;
  return seed / 4294967296;
};
```

The C side has one random source, the panel's power-on noise, drawn from the C library's `rand()` after `srand(1)`. Because WASI's C library has a different generator, the WebAssembly build carries a copy of glibc's so that its noise, and therefore its first screens, match.

Order. Every choice between equally ready alternatives is broken by a sequence number, not by addresses or hash order: the event queue by (time, sequence), the lane's pick by priority, then arrival time, then sequence.

Threads. The simulator has exactly one thread. Bus transfers that would run concurrently with the processor on the device are events in the same queue.

Input. Keys, battery levels and kills come from the script at virtual times written in it. The `latency` benchmark presses keys at varied phases of the keyboard poll, but the variation is a deterministic function of the press number, not a random draw:

▸ host/script.c

```
kernel_run_until(K.vnow + MS(gap) + (vtime_t)((i * 7919) % 160000));   // + 0-160 ms, so presses land at every phase of a poll
```

Memory. This is the source that is not closed, and it shows how subtle the question is. A world can ask the nucleus how much heap each world uses (the system monitor shows it), and heap sizes depend on pointer width and on the allocator. A world that prints those numbers prints different numbers on a 64-bit build than on a 32-bit one. Worse, the different text has a different length, the message that carries it has a different payload, and payload is charged per byte (Section 18.3): the leak feeds back into the clock. The effect is visible in a direct experiment. The tour script, run twice on the 32-bit build and once on the 64-bit build:

```
capture                 32-bit run 1   32-bit run 2   64-bit
01-home.png             45f3b44b       45f3b44b       45f3b44b
02-home-repl.png        629b4c15       629b4c15       629b4c15
03-sysmon.png           8c68f916       8c68f916       d85ae6e5
04-sysmon-sel.png       764b8eda       764b8eda       c7976b7e
05-sysmon-handles.png   231c28c6       231c28c6       231c28c6
06-sysmon-buses.png     61434ab5       61434ab5       3ace99bd
07-screen-off.png       144288ff       144288ff       144288ff
08-screen-on.png        45f3b44b       45f3b44b       45f3b44b
pixels written to panel  574,231       574,231        573,447
```

The two runs of one build are identical in every capture (the columns are the first eight hexadecimal digits of each file's MD5). The 64-bit build differs on exactly the three system-monitor screens that show heap sizes, and the panel model's count of pixels written differs too: the different numbers made different damage rectangles, so the compositor sent a different amount to the panel. Across all six scripts, 18 of 22 captures are identical between builds; the four that differ are system-monitor and About screens. The PicoCalc OS paper calls this the one leak in practice. It could be closed by reporting heap sizes in a unit independent of the build, which would make them less useful as measurements of the 32-bit device.

TRY 18.1

Let nondeterminism in

A small simulation of three components (a sensor, a filter and a logger) runs twice, side by side, from the same seed, and the widget compares the two event logs and their hashes. With nothing leaking in, they agree on every event. Choose **Host clock** and the logger stamps each record with `performance.now()`: the logs differ from the first record. Choose **Math.random** and the sensor's noise comes from the unseeded generator: the first reading differs, and every later decision of the filter follows from it. Choose **Arrival order** and two sensors run in Web Workers while the filter takes their readings in the order they arrive; run it a few times, and the logs usually differ because the browser scheduled the two threads differently.

Both runs read only the virtual clock and the seeded generator, so they agree on every event.

identical Both runs logged 66 events with hash 1c72c7f5.

Run A

```
 1     0.00  sensor reads  18.51
 2     1.82  sensor → filter  18.51  ema 18.51
 3     8.44  sensor reads  21.95
 4    10.69  sensor → filter  21.95  ema 19.54  ALERT
 5    10.69  filter → logger  alert 19.54
 6    17.79  sensor reads  22.94
 7    19.42  sensor → filter  22.94  ema 20.56  ALERT
 8    19.42  filter → logger  alert 20.56
 9    28.69  sensor reads  22.05
10    30.58  sensor → filter  22.05  ema 21.01
11    30.58  filter → logger  summary 21.01
12    39.84  sensor reads  24.16
```

Run B

```
 1     0.00  sensor reads  18.51
 2     1.82  sensor → filter  18.51  ema 18.51
 3     8.44  sensor reads  21.95
 4    10.69  sensor → filter  21.95  ema 19.54  ALERT
 5    10.69  filter → logger  alert 19.54
 6    17.79  sensor reads  22.94
 7    19.42  sensor → filter  22.94  ema 20.56  ALERT
 8    19.42  filter → logger  alert 20.56
 9    28.69  sensor reads  22.05
```

```
  30.58  sensor → filter  22.05  ema 21.01
  30.58  filter → logger  summary 21.01
  39.84  sensor reads  24.16
```

18.6 Scripts, traces and fault injection

A simulator is used through its inputs and its records. The host build's input is a script of commands, one per line: `wait` runs the system for a number of virtual milliseconds, `key` presses and releases a key, `type` types text, `capture` writes what the panel shows as a PNG, `crash` kills a world from outside as a hardware fault would, `battery` sets what the STM32 reports, and `stats` prints the worlds, the lane and the buses. A script can come from a file or from standard input, which makes the simulator easy to drive from another program. This one boots the system, moves the launcher's selection, captures the screen, kills the display driver and checks on it 600 ms later:

```
$ cat demo.txt
wait 1200
key right
capture home-right.png
crash display
wait 600
print display restarted
stats
$ ./build32/picocalc-host -o out demo.txt
[   1270.0 ms] captured out/home-right.png
[   1870.0 ms] display restarted
t = 1870.0 ms, lane busy 30.0 ms (1.6%), 152 turns, 13 published, 31 delivered, 1 crashes
  world       kind     state        turns    cpu ms   mem KB  limit  rst  last error
  keyboard    driver   idle            90     10.22    187.9    320    0
  power       driver   idle             1      0.30    185.7    320    0
  display     driver   idle             9      4.76    183.7    320    1  killed from the
host
  compositor  service  idle            23     10.87    265.2    768    0
…
  bus i2c0  busy   561.60 ms, 93 transfers, 376 bytes, 0 errors
  bus spi2  busy   113.92 ms, 56 transfers, 567657 bytes, 0 errors
$ cat out/uart.log
[     0.000] booting 13 worlds from js/system.json
[  1270.000] display crashed: killed from the host
#OCT1
{"ev":"fault","t":1270.000,"world":"display","file":"drivers/display.js","handling":"","w
hy":"killed from the host","crashes":1,"mem_kb":183,"limit_kb":320}
```

The output records what the device would show on its console UART, including the `#OCT1` fault frame for the kill (Chapter 14), and the statistics show the display driver restarted once. Running the script again produces the same output, line for line.

The richer record is the **trace**, enabled by the `trace` command: one JSON object per line for every segment (its world, start, duration, steps, native work, kind and the message it handled), every message delivered, every bus transfer, every key and every change of pixels on the panel. The traces in this book's figures, including the timelines of Chapter 16 and Chapter 15, are such files, and Chapter 19 builds its latency analysis from them. Because they are produced by a deterministic run, a trace is not a sample of the system's behaviour but the behaviour: rerunning the script reproduces it exactly, so a question that a trace raises can be answered by adding instrumentation and rerunning, without wondering whether the instrumentation changed the outcome.

Fault injection in this simulator is deliberately simple: `crash WORLD` kills a world at an exact virtual time, the Faults app runs nine experiments from inside the system (an endless loop, a runaway allocation, five crashes in a row, an ask that times out, a device opened without a claim), and a world's code can be edited to throw wherever a question needs it. FoundationDB goes further, with what its authors call **buggification**: code paths, enabled only in simulation, that make components return unnecessary errors, add delays or choose unusual parameters while still keeping their contracts [Zhou et al. 2021]. A PicoCalc OS equivalent would make the simulated STM32 occasionally stretch its clock or return an empty FIFO, and make the panel occasionally ignore a command; Chapter 15's exercises ask for some of these.

18.7 One model, four builds

The same sources build four ways: a native 64-bit host build, a 32-bit build whose pointers are the device's size so that its heap figures are the device's, a 64-bit build with AddressSanitizer and the undefined-behaviour sanitizer, and the WebAssembly build of Chapter 17. They agree with each other as closely as the memory leak of Section 18.5 allows: the 32-bit and 64-bit builds produce identical captures except for the four heap-dependent screens; the sanitizer build matches the 64-bit build the same way and found no memory errors or leaks in the nucleus; and the WebAssembly build's screens match the 32-bit build's pixel for pixel, which is the strongest check that nothing in the system depends on the platform it runs on.

They differ in speed, which Table 18.2 collects. The table separates the cost of starting (process creation, calibration and booting thirteen worlds, about 36 ms for the native builds) from the cost of running, because for short scripts the start-up dominates; the benchmark suite's 79 runs take 21 s in all.

Table 18.2. How much faster than real time the simulator runs, measured on a 2.1 GHz Xeon (best of several runs). Idle: a minute of the home screen. Snake: a minute of the snake game on autopilot. Running only: after boot. The WebAssembly build ran in headless Chromium 141.

Build	Idle, including start-up	Idle, running only	Snake, running only
native 64-bit	778×	1,470×	165×
native 32-bit	633×	1,010×	113×
WebAssembly (Chromium)	595×	650×	80–96×

18.8 What the model leaves out

A simulator is evidence only for the questions its model can answer, and the honest way to use one is to state what it leaves out. The PicoCalc OS model is exact about three things: the system's own control flow (which world runs, in what order, handling which message), the bytes on its buses and the time they take at the configured clock rates, and the state of the two peripheral controllers as the drivers see it. It is approximate or blind about the rest.

- **Work inside built-in functions.** JSON encoding, string building, array methods and regular expressions run without touching the interrupt counter, so they cost nothing beyond the per-byte payload charge, which covers only message encoding. Code that leans on them is undercharged.
- **Allocation and garbage collection.** Allocating an object costs nothing in the model, and QuickJS's reference counting and cycle collection are invisible. The runaway-allocation experiment hits its memory limit in fractions of a millisecond of virtual time, which says nothing about how long it would take on the device.
- **Caches and memory latency.** The ESP32-P4 runs code from flash through a cache and keeps world heaps in PSRAM, whose latency is far higher than on-chip SRAM. All of that is folded into the single constant τ, which was measured on a loop whose working set fits in cache.
- **Interrupts and the platform's own overhead.** On the device, FreeRTOS's tick, other interrupts and the ESP-IDF drivers' own processing take time that the model does not charge, and bus transfers start when a driver has set them up, not the instant a segment ends.
- **Electrical behaviour.** The I2C model has no clock stretching, no noise, no bus errors and no device that stops answering; the panel model never misses a byte.
- **The device's constants.** c_0, the per-pixel costs and the per-byte payload cost are estimates, not measurements (the device profile at the top of `nucleus/kernel.c` lists them).

These gaps do not make the simulator wrong; they define what it is for. It answers questions about the design (which world waits for which, how much a change to the poll period or the SPI clock moves a latency, whether a recovery path works, whether two drivers can interleave on a bus) and it reproduces behaviour exactly. It does not predict absolute performance on the device to better than the tens of percent its calibration supports. The most valuable next measurement is a second calibration point of the kind the model is worst at, an allocation-heavy loop or a JSON-heavy one, timed on the ESP32-P4; after that, the same traces recorded on the device and compared event by event with the simulator's, which the port in Chapter 7 is planned to make possible.

KEY POINTS

- A deterministic simulator answers four needs that a device cannot: repeatable runs, complete observation, speed, and exact control of faults and inputs.
- PicoCalc OS can be simulated without changing it because every effect a world has passes through a few nucleus functions; replacing what is under them changes the machine without the worlds noticing.
- A discrete-event loop keeps pending events in a queue ordered by time and a sequence number; the sequence number makes ties deterministic, and jumping to the next event makes idle time free.
- The lane adds work-driven time to the event skeleton: each segment costs c_0 + steps · τ + native work, and τ = 665 ns is fitted to one device measurement and checked against two others (fib(20) 29.1 against 31 ms).
- Buses are modelled from their clock rates and devices at register level, faithful enough that driver mistakes, including datasheet waits skipped and two drivers interleaving on one bus, show up in the simulator.
- Every source of nondeterminism must be listed and closed: time, randomness, order, threads and input are closed in PicoCalc OS; memory sizes are not, and they change four of 22 captures between 32-bit and 64-bit builds.
- Scripts and traces make every run a reproducible experiment; the WebAssembly build's pixel-identical screens are the strongest evidence that nothing depends on the host platform.
- The model is exact about control flow, bus traffic and device state, and blind to built-in work, allocation, caches, interrupts and electrical faults; it is a tool for comparing designs, not a substitute for measuring the port.

A simulator turns a system into something that can be measured as precisely as one likes, which raises the question of how to measure it well. The next chapter is about measurement itself: choosing what to measure, collecting distributions rather than averages, avoiding the traps this project fell into, and decomposing a latency into stages that explain it.

EXERCISES

1. BASIC Using Equation 18.1, compute the cost of a segment that executes 1,289 steps and 572 µs of native work. Find that segment in `bench/traces/keypress.jsonl` and check your answer against its `dt`.

2. BASIC Compute the time of an I2C transaction that writes three bytes and reads four at 100 kHz with no gap, and the time of an SPI transfer of a 96 × 66 rectangle of 16-bit pixels at 40 MHz.

3. ANALYSIS `kernel_run_until` handles an event before running a segment when both are due at the same instant. Construct a scenario in which the opposite rule would change what a world observes, and say which rule better matches the device.

4. ANALYSIS The 10,000-iteration loop is priced 21 percent above the device measurement while the 100,000-iteration loop matches exactly. Fit a two-parameter model (a fixed cost per program plus a cost per step) to the two loop measurements and predict `fib(20)` with it. Is the fit better or worse than the one-parameter model, and what does that suggest about the fixed cost?

5. ANALYSIS List every place in `nucleus/` and `sim/` where a value from the host could influence the system's behaviour (time, randomness, addresses, environment). For each, say whether it is closed and how.

6. CODE Close the memory leak of Section 18.5: make the system monitor's heap figures identical on 32-bit and 64-bit builds without changing what the 32-bit build shows. Run the tour on both builds and check that all captures match.

7. CODE Add a `jitter` command to the host build that delays every I2C transfer by a pseudo-random 0 to 200 µs drawn from a seeded generator. Show that runs with the same seed are identical and runs with different seeds differ, and measure the effect on the launcher's key-to-pixel latency distribution.

8. CODE Implement simple buggification for the STM32 model: with a probability set from the script, a FIFO read returns empty although a key is queued. Does any driver behaviour change? Should it?

9. CODE Write a script that reproduces the first driver bug of Section 18.4 on the current system by sending the keyboard driver's old two-transfer read from a test world while the power driver reads the battery. Show the corrupted key in a trace.

10. ANALYSIS From Table 18.2, estimate how long the host build takes to simulate a day of the handheld left on its home screen, and a day of snake. What dominates in each case?

11. PROJECT Record a trace on the device once the port of Chapter 7 exists (or, until then, design the experiment): which events can be timestamped on the ESP32-P4 without disturbing them, and how would you align the device's trace with the simulator's to find where the model is wrong?

12. DISCUSSION FoundationDB's engineers report that simulation finds most of their bugs. Which classes of bug in PicoCalc OS would its simulator find, and which would it never find?

FURTHER READING

[Zhou et al. 2021] Section 4 of the FoundationDB paper describes deterministic simulation testing in production use: what is simulated, how faults are injected, and why it works.

[Haas et al. 2017] The properties of WebAssembly that let the whole simulator run, unchanged and deterministic, in a browser.

Measuring systems

A measurement is a claim about a system, and like any claim it can be wrong in ways that are hard to see from the number alone. This chapter is about how to make latency and throughput measurements that mean what they appear to mean. It does so by retelling one study in detail: how long PicoCalc OS takes to put pixels on the panel after a key is pressed. By the end of the chapter you should be able to define the endpoints of a latency precisely, read a distribution rather than a single number, recognise four errors that this study actually made (and corrected), decompose a latency into stages and check the decomposition against the measurement, and report the result so that a reader can tell what was measured and what was not.

All the numbers in this chapter come from the host build of PicoCalc OS running on its virtual clock (Chapter 18 explains the clock), produced by `bench/run.py` and recorded in `bench/results.json`. They are measurements of a calibrated model of the device, not of the device. That makes them repeatable to the microsecond, which turns out to be both a gift and a trap: it removes noise, and with it one of the signals that usually tells an experimenter something is wrong.

19.1 What to measure

Three quantities describe most systems. **Latency** is the time from a cause to its effect: from a key press to the pixels that show it, from a request to its reply. **Throughput** is the rate at which work completes: messages handled per second, bytes per second on a bus. **Utilisation** is the fraction of time a resource is busy: the share of the lane spent running segments, the share of the I2C bus spent transferring bits. The three are related. A resource that is busy most of the time makes requests wait, so latency rises with utilisation; a system can often raise throughput by batching, at the price of latency.

For an interactive handheld, latency is the quantity a person feels. Nobody notices that the lane is 0.72% busy on the home screen. Everybody notices when a key press takes a tenth of a second to show. So the central measurement of PicoCalc OS's evaluation is **key-to-pixels latency**, and the other quantities are measured mostly to explain it.

The study measured the quantities in Table 19.1. Each has a unit, a clock it is read from and an instrument that reads it. Being explicit about the clock matters in this project because there are two: virtual time, which models the device, and the host's wall-clock time, which says only how fast the simulator runs.

Table 19.1. What the PicoCalc OS evaluation measures, on which clock, and with which instrument.

Quantity	Unit	Clock	Instrument
Key to first changed pixels, key to complete update	ms	virtual	`latency` script command (`host/script.c`)
Time of each stage of a key press	ms	virtual	JSON-lines trace (`nucleus/trace.c`), analysed in `paper/data.py`
Lane, I2C and SPI utilisation	% of time	virtual	`snapshot` counters before and after a window
Ask, send and publish cost	µs	virtual	the bench worlds `pinger`, `ponger`, `sub`
Time to stop an overrunning world	ms	virtual	the bench worlds `spin`, `painter`, `hog`
Recovery after a driver is killed	ms	virtual	`crash` command plus trace
Heap per world	KB	(none)	`memprobe`, `snapshot` on the 32-bit build
Simulator speed	× real time	host wall clock	best of three runs of a 60 s script

A single number is rarely enough for any of these. A latency is a distribution: the same press takes a different time depending on what the system was doing when it arrived. A report that gives only the mean hides the slowest presses, which are the ones people remember. The rest of this chapter treats every latency as a set of samples, and the first decision about any set of samples is what exactly each sample measures.

19.2 Endpoints: when an interaction starts and ends

A latency is the difference between two timestamps, and most errors in latency measurement come from choosing them carelessly. The start must be the cause as the user experiences it, not some later point where the system first notices it. The end must be the effect as the user experiences it, not some earlier point where the system believes it has finished.

For a key press, the natural start is the moment the key goes down. In the host build, the script runner schedules a `key_event` at a virtual time, and that event calls the STM32 keyboard controller model, which puts the press into its FIFO. That is the start: the moment the keyboard controller knows about the press. The keyboard driver will not know about it until its next poll, up to 20 ms later, and that wait is part of what the user experiences, so it must be inside the measurement. A harness that started the clock at the driver's next poll would report latencies 10 ms shorter on average and would hide one of the two largest stages of the delay.

The natural end is the moment the new pixels appear on the glass. The host build cannot see the glass, but it can see the panel controller's memory. The ST7365P model in `sim/st7365p.c` counts, as pixel data arrives over SPI, how many pixels change value in the 320 rows the glass shows. The SPI transfer code in `nucleus/hw.c` sends pixel data in chunks of up to 32 KiB, and after each chunk it asks the model how many pixels that chunk changed. If any did, it calls `photon`, which records the time the chunk finished.

▸ nucleus/trace.c

```
void photon(vtime_t t0, vtime_t t1, int changed) {
  if (K.photon_watch >= 0 && t1 >= K.photon_watch) {
    if (K.photon_seen < 0) K.photon_seen = t1;
    K.photon_last = t1;
  }
…
}
```

Two endpoints come out of this. **First pixels** is the end of the first chunk, after the press, that changed a visible pixel. **Complete** is the end of the last such chunk in the same update. The first is what a person sees begin to happen; the second is when the screen has finished changing. Both matter: a screen that starts changing quickly but takes 80 ms to finish feels different from one that changes all at once.

"In the same update" needs a definition, because an update is not an object the panel knows about. The panel receives a stream of chunks; the harness must decide where one update ends. The script runner's `key_to_photon` decides that an update is over when the panel has received no changed pixels for 15 ms.

▸ host/script.c

```
static double key_to_photon(int code, int hold_ms, int timeout_ms) {
  vtime_t t = K.vnow;
  K.photon_watch = t; K.photon_seen = K.photon_last = -1;
  schedule_fn(t, key_event, NULL, code | 1 << 8);
  schedule_fn(t + MS(hold_ms), key_event, NULL, code | 3 << 8);
  while (K.photon_seen < 0 && K.vnow < t + MS(timeout_ms)) kernel_run_until(K.vnow +
250);   // 0.25 ms steps
  double lat = K.photon_seen >= 0 ? (K.photon_seen - t) / 1000.0 : -1;
  if (K.photon_seen >= 0) while (K.vnow < K.photon_last + MS(15)) kernel_run_until(K.vnow
+ 250);
  done_ms = K.photon_seen >= 0 ? (K.photon_last - t) / 1000.0 : -1;
  K.photon_watch = -1;
  if (K.vnow < t + MS(hold_ms) + MS(10)) kernel_run_until(t + MS(hold_ms) + MS(10));
  return lat;
}
```

Why 15 ms? The threshold has to be longer than any pause inside one logical update and shorter than the gap between independent updates. The second bound is easy: the harness waits at least 100 ms between presses, and the slowest periodic redraw in the system is the system monitor's, every 500 ms. The first bound came from a surprise. Opening an app does not produce one update; it produces two. In twenty openings of the system monitor recorded for this chapter, every one produced two bursts of changed pixels, 3.1 to 3.3 ms apart, and the first burst changed the same 40,208 pixels each time.

The reason is in the scheduler. When the launcher asks the compositor to open the system monitor, the compositor (a service, priority 2) changes its focus, sends the monitor a `focus` message, and redraws at once with whatever frame it last received from the monitor. The monitor is an app, priority 1, so it runs after the compositor; and while hidden it renders an empty tree, so the compositor's first draw is a mostly blank body under the monitor's title. Only when the monitor has handled `focus` and published a real frame does the compositor draw the screen the user wanted. Figure 19.1 shows the sequence.

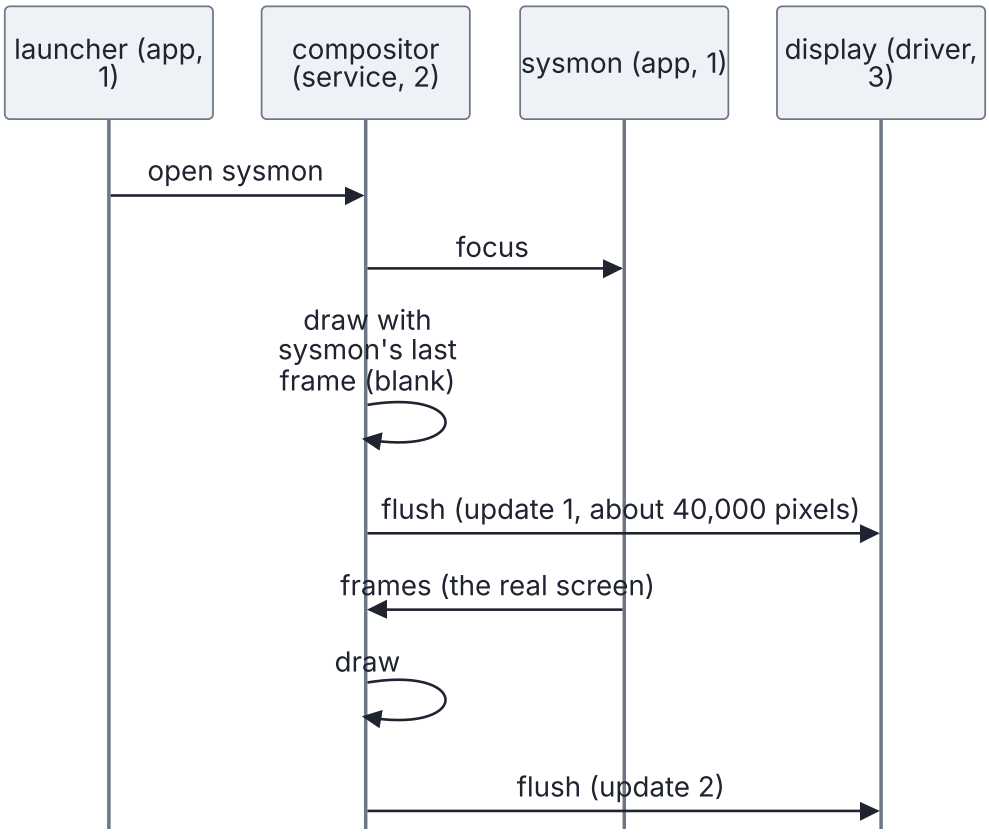


Figure 19.1. Opening the system monitor produces two updates. The compositor outranks apps, so it draws the monitor's stale (blank) frame before the monitor has rendered its real one, about 3 ms later.

An earlier version of the harness used an idle threshold shorter than that gap, and it reported "complete" at the end of the first, blank update, so the complete-update latency for opening an app measured the wrong screen. Raising the threshold above the measured 3.3 ms gap fixed the endpoint; looking at why there were two updates found a real inefficiency in the compositor, which the paper reports as a finding: a compositor that waited briefly for the focused app's first frame would save one full transfer.

Two further limits of the endpoint deserve to be stated rather than discovered by the reader. First, "first pixels" is when data reaches the controller's memory. The panel then shows that memory on its next refresh, and the liquid crystal takes time to change state; neither is modelled. On the device, the time to light at the glass will be longer than the time to memory by an amount that has to be measured there. Second, a press can change nothing: pressing the same character twice in the REPL changes no pixels if the line already

shows it. Early runs that typed one character repeatedly timed out. The REPL scenario now cycles through `a`, `b` and `c`, and the timeout is recorded as −1 rather than as a large number, so a sample that never completed cannot quietly inflate the tail.

> **DEFINITION: ENDPOINT**
>
> The **endpoints** of a latency measurement are the two events whose times are subtracted. A well-chosen start is the cause as the user experiences it; a well-chosen end is the effect as the user experiences it. Every latency number should be reported with its endpoints.

19.3 Distributions, percentiles and CDFs

With endpoints fixed, the harness presses a key many times and collects one number per press. The launcher scenario presses the right-arrow key 100 times; Table 19.2 summarises all five scenarios of the paper. The summary is computed by one small function in `bench/run.py`:

▸ bench/run.py

```
def summary(xs):
    xs = sorted(x for x in xs if x >= 0)
    if not xs:
        return {}
    q = lambda f: xs[min(len(xs) - 1, int(round(f * (len(xs) - 1))))]
    return {"n": len(xs), "min": xs[0], "p50": q(0.5), "p90": q(0.9), "p95": q(0.95),
"max": xs[-1], "mean": statistics.fmean(xs)}
```

A **percentile** p*k* is a value below which *k*% of the samples fall. This function uses the nearest-rank form: sort the samples and take the one at position $k/100 \cdot (n - 1)$, rounded. With $n = 100$, p95 is the 95th smallest sample and the maximum is a single sample. That has a consequence worth stating plainly: with 100 samples, p95 rests on the five slowest presses and p99 would rest on two. A tail percentile estimated from few samples is itself a noisy number; to say something about p99 you need thousands of samples, and the harness reports *n* next to every percentile so a reader can judge. The function also drops samples marked −1 (timeouts). That is reasonable only because *n* is reported: a scenario that timed out would show *n* below the number of presses.

Table 19.2. Key-to-pixels latency in virtual ms for the paper's five scenarios: first changed pixels (min, median, p95, max) and the complete update (median, max).

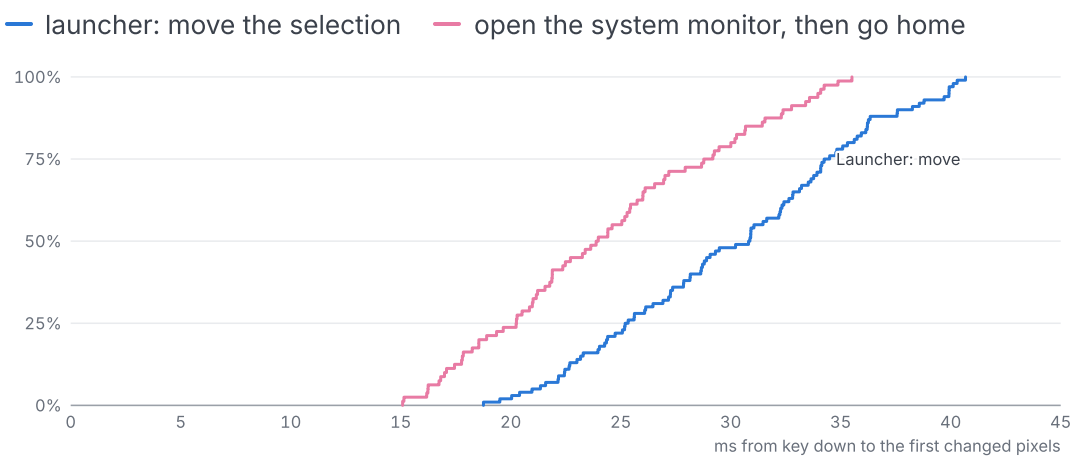
Interaction	n	min	median	p95	max	complete median	complete max
Launcher: move the selection	100	18.8	30.9	39.9	40.7	34.6	44.0
System monitor: next tab	80	16.6	27.5	38.7	41.7	52.4	64.3
REPL: type a character	100	14.6	24.6	33.5	35.4	24.6	35.5
Keys app: show the event	80	16.9	28.4	35.6	36.7	38.8	47.6
Open the system monitor, then go home	80	15.1	24.0	34.0	35.5	67.1	86.3

The table is a compression. The full information is the **cumulative distribution function** (CDF): for every latency x, the fraction of presses that took at most x. A CDF is the right picture for latency because every percentile can be read from it directly (p95 is where the curve crosses 0.95), because two scenarios can be compared at every percentile at once, and because its shape tells a story that summary numbers do not.

TRY 19.1

Read a latency distribution

The chart shows the CDF of key-to-first-pixels latency for moving the launcher's selection and for the scenario that alternately opens the system monitor and goes home. Hover to read percentiles. Notice that the launcher's curve is close to a straight ramp between about 19 and 41 ms: that is the signature of a uniformly distributed delay, and the next sections show which one. Switch the metric to the complete update and see how far the second curve moves to the right: switching apps changes most of the screen, so the last pixels arrive long after the first.



Scenario	n	min	median	p95	max
launcher: move the selection	100	18.8	30.9	39.9	40.7
open the system monitor, then go home	80	15.1	24.0	34.0	35.5

The launcher's median is 30.9 ms and its mean is 30.1 ms. For a distribution this symmetric the two agree; for a skewed one (a long tail of slow presses) the mean is pulled toward the tail and describes no actual press. Medians and percentiles are robust in a way means are not, which is why the paper reports them. The mean is still useful for one thing: multiplied by a rate it gives a total, such as the lane time spent per second on key handling.

19.4 Phase-locking: when the harness measures itself

The first latency runs of this study produced distributions that were nearly constant. Press after press took the same time to a tenth of a millisecond. A real system rarely behaves like that, and this one did not; the harness did.

The keyboard driver polls the STM32 every 20 ms. A press waits for the next poll, so its latency depends on its **phase**: where in the 20 ms poll period it lands. If presses land at every phase, the wait is spread uniformly between 0 and 20 ms. The early harness pressed a key, waited a fixed interval, and pressed again, and the interval (including the key's hold time) was a multiple of 20 ms. Every press therefore landed at the same phase, waited the same time, and produced the same latency. On a virtual clock with no noise, nothing broke the lock.

The effect is easy to reproduce. Table 19.3 shows two runs of 60 presses in the launcher, each pressing the right-arrow key for 60 ms and then waiting. In the first, the period from one press to the next is 200 ms, ten poll periods; in the second it is 207 ms.

Table 19.3. Sixty presses in the launcher at a fixed period. With a period that is a multiple of the 20 ms poll, almost every press lands at the same phase and the distribution collapses.

Period between presses	min	median	p95	max	distinct values (0.1 ms)
200 ms (10 polls)	18.1	20.9	20.9	27.8	6
207 ms	18.1	29.9	38.9	39.9	41

With the 200 ms period, the median and the 95th percentile are the same number, and 60 presses produced only 6 distinct values. What variation remains has two causes: the selection sometimes wraps to the other row of the launcher's grid, which changes the size of the first rectangle sent (Section 19.6), and two presses had to wait for the power driver's battery read, which shares the I2C bus. A reader shown only the first row would conclude that the launcher responds in 21 ms almost every time. The truth, in the second row and in Table 19.2, is that it responds in anything from 18 to 41 ms depending on luck.

The fix in the harness is to add a varying offset to each gap, so presses land at every phase. The `latency` command in `host/script.c` adds up to 160 ms, using a multiplicative step that is coprime with the poll period:

▸ host/script.c

```
kernel_run_until(K.vnow + MS(gap) + (vtime_t)((i * 7919) % 160000));   // + 0-160 ms, so
presses land at every phase of a poll
```

The general lesson is that any periodic process in the system can alias with a periodic measurement. PicoCalc OS has many: the 20 ms keyboard poll, the 500 ms compositor clock check, the 5 s battery read, every app's `every` timer. A measurement schedule with a fixed period samples each of them at a fixed phase. Real users press keys at times uncorrelated with the system's timers, and a measurement that wants to represent them must do the same, either with randomised gaps or with gaps whose offsets sweep the period systematically. A collapsed distribution, or one with far fewer distinct values than samples, is the symptom to look for.

19.5 Contamination: measuring someone else's pixels

The harness attributes the first changed pixels after a press to that press. That attribution is only correct if nothing else changes pixels in the meantime. Two apps in PicoCalc OS redraw on timers: the system monitor samples the kernel's tables every 500 ms and redraws its charts, and the keys app asks the keyboard driver for its counters every second. A press that happens to arrive a few milliseconds before one of those redraws gets credit for the redraw's pixels.

Table 19.4 shows the effect on the system monitor's "next tab" scenario, run once with the monitor's timer as shipped and once with the timer slowed down.

Table 19.4. The system monitor's next-tab scenario (80 presses) with its 500 ms redraw running, and with the redraw slowed to once every 10 minutes.

Monitor's timer	min	median	p95	max	complete median	complete max
every 500 ms (as shipped)	8.4	27.9	37.1	40.1	53.2	72.0
every 600 s (quiet)	16.6	27.5	38.7	41.7	52.4	64.3

The medians barely move, so a summary that reported only the median would never notice. The minimum is the giveaway. The two I2C reads that fetch a key from the STM32 take 12.9 ms on their own (Section 19.6), so no press can show pixels sooner than about 13 ms after it reaches the controller, and in practice the processing after the reads adds a few more. A sample of 8.4 ms is physically impossible for a

key press. It is a redraw that happened to begin just after the press. The complete-update maximum is wrong in the same way: 72.0 ms includes a redraw's pixels that arrived within 15 ms of the key's own update and so extended it.

The benchmark suite fixes this by running the latency scenarios on a copy of the `js/` tree in which those two timers are slowed to once every ten minutes. The patch is applied by text substitution, so the shipped sources stay unchanged:

▸ bench/run.py

```
QUIET = [("apps/sysmon.js", ".every(500, sample)", ".every(600000, sample)"),
         ("apps/keys.js", ".every(1000, async (s) =>", ".every(600000, async (s) =>")]
```

This is a controlled experiment in the ordinary sense: remove the sources of change you are not studying. It also changes the system being measured, so the paper says so in its method section. The alternative, attributing pixels to their cause by following the trace from the press to the flush, is more faithful and more work; Section 19.8 uses it where the simple attribution fails.

Two habits catch contamination early. Know the physical lower bound of the quantity you measure, and treat any sample below it as a bug in the measurement. And look at the minimum and maximum, not only the median: contamination lives at the edges of a distribution.

19.6 Decomposing a latency

A distribution says how long presses take. It does not say why. To explain a latency you decompose it into stages and measure each stage, and then you check that the stages add up to the whole. The instrument for this in PicoCalc OS is the trace: `trace FILE` makes the nucleus write one JSON line for every segment that runs on the lane, every message delivered, every bus transfer and every chunk of changed pixels (Appendix B lists the record types). Here is the trace of the press nearest the launcher's median, abridged.

▸ bench/traces/keypress.jsonl


```
{"k":"bus","bus":"i2c0","w":"keyboard","t":16980137,"t1":16986137,"bytes":4,"what":"write 0x04, wait, read 1"}
{"k":"key","t":16987260,"w":"","text":"0xb7 down"}
{"k":"seg","w":"keyboard","t":17000078,"dt":59,"steps":26,"native":2,"kind":"msg","type":"every 20 ms"}
{"k":"bus","bus":"i2c0","w":"keyboard","t":17000137,"t1":17006137,"bytes":4,"what":"write 0x04, wait, read 1"}
{"k":"bus","bus":"i2c0","w":"keyboard","t":17006187,"t1":17013087,"bytes":5,"what":"write 0x09, wait, read 2"}
{"k":"msg","t":17013163,"from":"keyboard","to":"compositor","type":"keys","ask":false,"bytes":62}
{"k":"seg","w":"launcher","t":17013310,"dt":127,"steps":89,"native":28,"kind":"msg","type":"key"}
{"k":"seg","w":"compositor","t":17013929,"dt":1412,"steps":1204,"native":572,"kind":"msg","type":"draw"}
{"k":"msg","t":17015341,"from":"compositor","to":"display","type":"flush","ask":true,"bytes":55}
{"k":"px","t":17015589,"t1":17018135,"changed":6336}
{"k":"bus","bus":"spi2","w":"display","t":17015589,"t1":17018135,"bytes":12672,"what":"pixels 96x66 at 216,62"}
…
{"k":"px","t":17020697,"t1":17024027,"changed":1025}
```


Times are virtual microseconds since boot. Reading it top to bottom: a poll's status read ended at 16,986,137 µs with nothing in the FIFO; the key went down 1.1 ms later, at 16,987,260; the next poll began at 17,000,078 and its status read (write register 0x04, wait 2 ms, read one byte) took 6.0 ms; the FIFO read (write 0x09, wait, read two bytes) took 6.9 ms; the keyboard driver published the event; the launcher handled it in 127 µs and published a 1,135-byte frame; the compositor drew and diffed in 1.4 ms; and the first changed rectangle, 96 × 66 pixels, finished arriving at 17,018,135. That is 30.9 ms after the press. The last changed chunk ended at 17,024,027, 36.8 ms after the press.

`paper/data.py` turns any press in the trace into named stages by finding these records in order:

▸ paper/data.py

```
S = [
    ("wait for the next poll", t0, status["t"]),
    ("read the key count (I2C 0x04)", status["t"], status["t1"]),
    ("read the key event (I2C 0x09)", fifo["t"], fifo["t1"]),
    ("route: keyboard → compositor → launcher; launcher renders", fifo["t1"], app["t"] +
app["dt"]),
    ("compositor: take the frame, lay out, draw, diff", frames["t"], draw["t"] +
draw["dt"]),
    ("display driver queues the transfers", flush["t"], pixb[0]["t"]),
    ("SPI: first changed rectangle (first pixels)", pixb[0]["t"], first),
    ("SPI: the rest of the update", first, last),
]
```

For the median press the stages are: 12.9 ms waiting for the poll, 12.9 ms on the I2C bus, 2.5 ms of processing on the lane, and 2.5 ms of SPI for the first rectangle. The poll wait and the I2C reads are 83% of the time to first pixels; everything the processor does, from the keyboard driver's JavaScript to the compositor's layout and the nucleus's diff, is 8%. That single observation redirects any effort to make the system feel faster. Optimising the compositor could save a millisecond; changing how the keyboard is read could save twenty.

A decomposition of one press is an anecdote. The useful step is to decompose all of them and build a model that predicts the whole distribution from its parts. Across the 100 launcher presses the stages behave as follows.

- **The poll wait** ranges from 0.14 to 19.9 ms. It is uniform over the 20 ms period, because the harness makes presses land at every phase (Section 19.4).
- **The two I2C reads and the processing** add a near-constant 15.5 ms. The processing part varies only between 2.5 and 2.6 ms.
- **The first rectangle on SPI** takes one of two values. When the old and new selections are in the same row of the launcher's grid, the changed band is 200 × 66 pixels and its first chunk takes 5.3 ms (67 presses). When the selection wraps to the other row, each tile is its own 96 × 66 rectangle and the first takes 2.5 ms (33 presses).

The I2C times themselves follow from the bus clock. An I2C transaction that writes *w* bytes and reads *r* bytes clocks the address and each byte as 9 bits (8 data bits and an acknowledge), plus start and stop conditions, and the STM32 needs 2 ms between the register write and the read:

▸ paper/data.py

```
def i2c_ms(nw, nr, hz, gap=2.0):
    return ((1 + nw) * 9 + 2 + (1 + nr) * 9 + 2) / hz * 1000 + gap
```

At 10 kHz the status read (one byte each way) is 40 bit times, 4.0 ms, plus the 2 ms wait: 6.0 ms. The FIFO read (one byte written, two read) is 49 bit times plus 2 ms: 6.9 ms. Together 12.9 ms, exactly what the trace shows.

Putting the parts together, the model predicts that the fastest press takes 15.5 + 2.5 = 18.1 ms (no wait, small rectangle) and the slowest 20 + 15.5 + 5.3 = 40.8 ms (a full poll period of waiting, large rectangle). The measured range is 18.8 to 40.7 ms. A model built from the parts that predicts the extremes of the whole to within a millisecond is strong evidence that the decomposition is right and that nothing important is missing from it. When a model and a measurement disagree, one of them is wrong, and finding out which is usually the most informative thing an evaluation does.

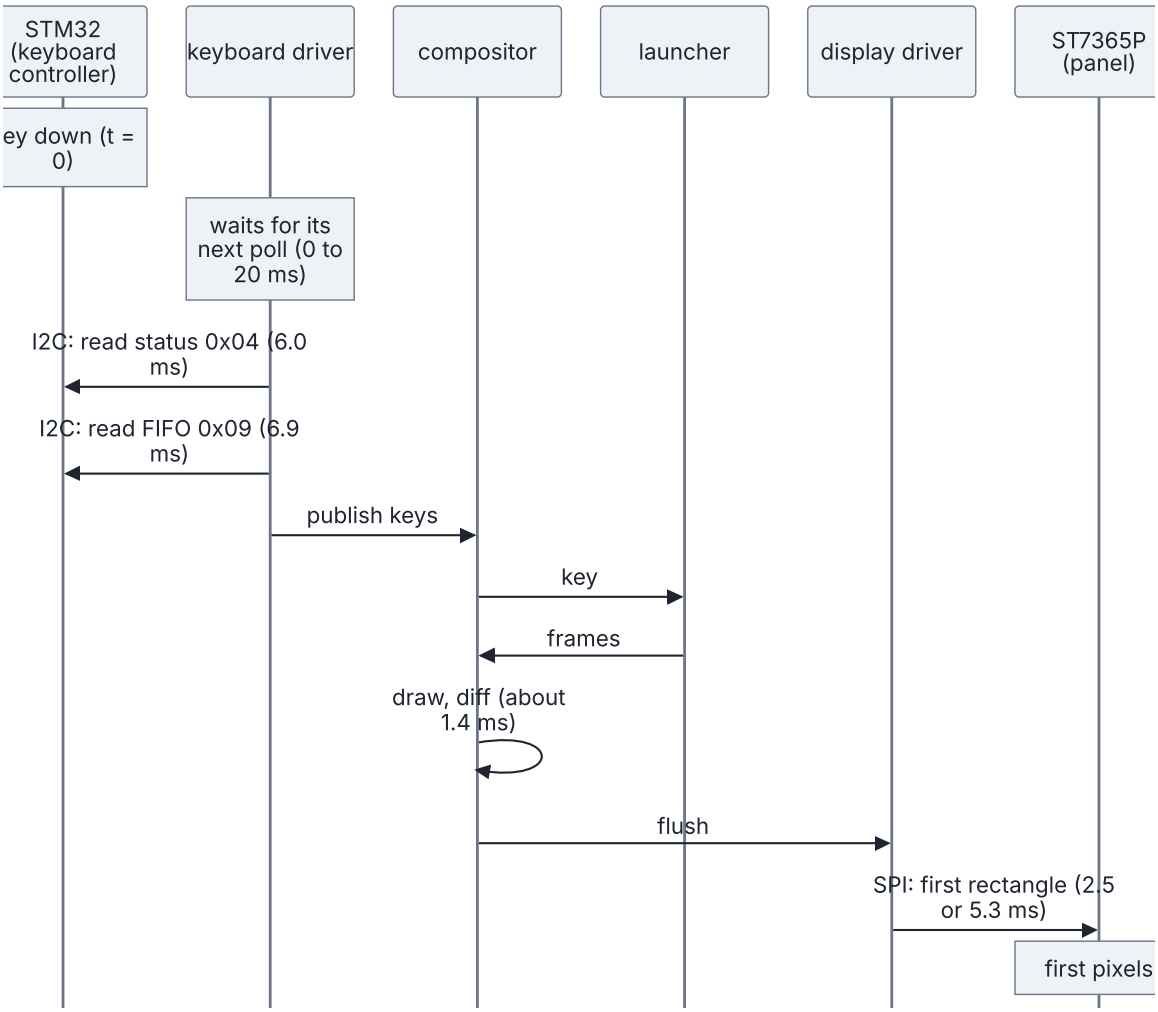
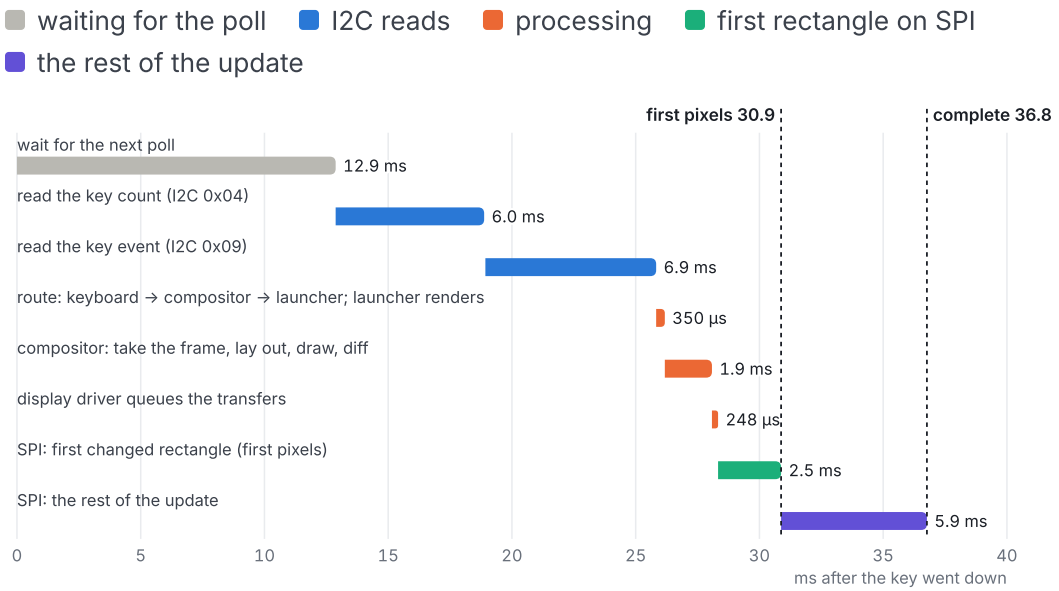


Figure 19.2. The stages of one key press in the launcher, from the press reaching the keyboard controller to the first changed pixels reaching the panel controller's memory.

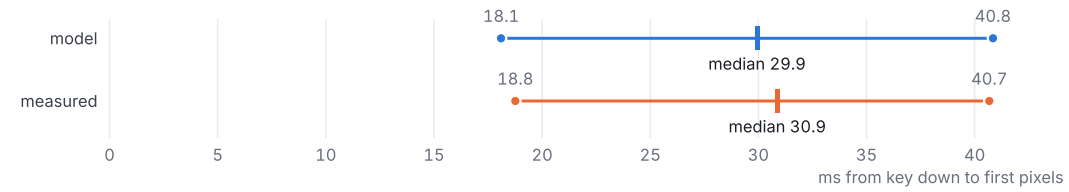
TRY 19.2

Change the hardware, predict the latency

The bars show the stages of the median press. The controls below them change the parameters the model depends on: the I2C clock, the keyboard poll period and the SPI clock. Watch which ones move the predicted range. Raising the I2C clock from 10 kHz to 400 kHz removes most of the 12.9 ms of bus time but not the two 2 ms waits the controller needs; halving the poll period halves the worst-case wait; doubling the SPI clock saves 1.3 or 2.7 ms on the first rectangle, depending on its size. The measured range at the default settings is shown for comparison. Try a 5 ms poll with the 10 kHz bus: the two reads take longer than the period, the bus is 98% busy, and the model's assumption of a uniform wait no longer holds.



The model at other settings. Measured values appear where the benchmark ran those settings.



The model: wait 0–20 ms for the poll, 12.9 ms of I2C reads at 10 kHz, 2.7 ms of processing, then the first rectangle: 2.5 ms (33% of presses) or 5.3 ms at 40 MHz. On average the wait for the poll (half the period) and the I2C reads are about 76% of the model's median; the traced press above, which waited 12.9 ms for its poll, spent 83% of its time on that path. Measured: 18.8–40.7 ms, median 30.9 ms (n = 100).

19.7 Sweeps: change one thing at a time

A decomposition says which stages dominate. A **sweep** tests the claim by changing one parameter over a range while holding everything else fixed, and measuring the effect. The study ran three: the keyboard poll period and I2C clock together, the SPI clock, and a background load.

The poll sweep varies the period from 5 to 80 ms at three bus clocks, and measures both the I2C bus's utilisation with no key pressed and the launcher's latency. Table 19.5 shows part of it.

Table 19.5. Keyboard poll period and I2C clock against the I2C bus's idle load and the launcher's key-to-first-pixels latency (100 presses each).

Bus clock	Poll period	I2C busy	median	p95
10 kHz	40 ms	15%	38.9	57.8
10 kHz	20 ms (as shipped)	30%	30.9	39.9
10 kHz	10 ms	60%	24.8	30.2
10 kHz	5 ms	98%	22.9	26.6
100 kHz	20 ms	12%	22.8	31.9
400 kHz	20 ms	11%	22.1	31.2
400 kHz	5 ms	42%	13.7	16.7

Two things stand out. Polling faster lowers latency but costs bus time, and at 10 kHz the cost is steep: a 5 ms poll keeps the bus 98% busy, because each status read takes 6 ms and no longer fits in the period (the kernel coalesces timer ticks, so the driver simply polls as fast as the bus allows). And a faster bus helps less

than its clock ratio suggests. At 100 kHz the status read takes 2.4 ms and at 400 kHz 2.1 ms, not 0.6 and 0.15, because the 2 ms the controller needs to prepare its answer is a fixed cost the clock cannot reduce. The sweep confirms the decomposition's verdict that the keyboard path is where latency lives, and it adds a refinement no single measurement could: the controller's preparation time, not the bus clock, becomes the limit once the clock is raised.

The SPI sweep runs the same two scenarios as Table 19.2 at 10, 20, 40 and 80 MHz. A whole frame takes 163.9, 82.0, 41.0 and 20.6 ms, inversely proportional to the clock as it should be. Opening the system monitor completes at the median in 224.4, 125.7, 75.0 and 51.9 ms, because nearly the whole screen changes. But the first pixels after moving the launcher's selection arrive at 43.3, 34.7, 30.9 and 28.3 ms: doubling the clock from 40 to 80 MHz saves 2.6 ms, because the update is small and the keyboard path dominates. The 40 MHz column of the sweep reproduces Table 19.2 exactly, because the sweep runs the same scripts with the same jitter. That is deliberate. An earlier version of the sweep ran different scripts from the latency section, and its 40 MHz numbers did not match the latency table, for no reason a reader could see. Reusing the runs makes the two tables checks on each other.

The load sweep adds a world that computes for L ms every 50 ms and measures the launcher. Up to $L = 5$ ms (12% of the lane) the median and p95 do not move from 30.9 and 39.9 ms. At 10 ms (22% of the lane) the p95 rises to 43.2; at 20 ms (42%) the median is 39.9 and the p95 54.2; at 40 ms (82%) the median is 77.6, the p95 91.4 and the maximum 103.8. Running the load world at priority 0 instead of 1 gave identical results to a tenth of a millisecond. That last result is the most instructive in the sweep, because it contradicts an intuition: lowering a background task's priority should protect the foreground. It does not here, because PicoCalc OS does not preempt (Chapter 4). Priority decides which waiting world runs next, but a key's path through the system waits behind whatever segment is already running when it becomes ready, and the load world's segments are 40 ms long. A sweep that varied only priority would have found nothing; one that varied priority and load together found the mechanism.

19.8 Open and closed loops

The latency harness is a **closed loop**. It presses a key, waits until the update it caused has completed and the panel has been idle for 15 ms, waits a further gap, and only then presses again. The rate at which it issues presses therefore depends on how fast the system responds. When the system is slow, the harness slows down with it.

A person does not. Someone typing keeps typing while the screen lags. This difference has a name. Gil Tene called it **coordinated omission**: a load generator that waits for each response before sending the next request omits exactly the requests that would have arrived during a stall, so the samples it collects under-

represent the stall [Tene 2013]. Schroeder, Wierman and Harchol-Balter showed more generally that closed and open load models can give very different response times for the same system at the same load, and that the choice of model should follow the workload rather than the convenience of the generator [Schroeder et al. 2006].

The effect is easy to produce in PicoCalc OS. Add an app that computes for 400 ms every 2 s, a stand-in for an expensive background job. (Its source is one line, `app(self).every(2000, () => { const end = now() + 400; while (now() < end) {} });`, with a 500 ms budget, at priority 1.) Then measure the launcher twice: once with the closed-loop `latency` command, and once open-loop, pressing the right-arrow key every 113 ms whatever the screen is doing. In the open-loop run the first pixels after a press might belong to the previous press's update, so each press is attributed by following the trace: from the press, to the launcher's segment that handled it, to the compositor's next draw, to the first pixels of that draw's flush.

Table 19.6. Launcher latency (100 presses) with a background app that computes for 400 ms every 2 s, measured closed-loop and open-loop. The stall occupies 20% of the time.

Harness	median	p80	p85	p90	p95	max	presses over 60 ms
closed loop (`latency right 100 60`)	31.7	39.2	39.9	285.4	335.0	405.9	12%
open loop (a press every 113 ms)	32.9	99.7	179.9	268.0	336.1	424.8	23%

The medians agree and so do the extremes. The difference is in the middle of the tail. The stall covers 20% of the time, and an open-loop stream of presses lands in it about 20% of the time: 23% of the open-loop presses took more than 60 ms. The closed-loop harness, which stops pressing while it waits for a stalled update, sampled only one press per stall and saw 12%. Its 80th and 85th percentiles describe a system without a stall at all. Anyone who reported "p85: 40 ms" from the closed-loop run would be wrong by a factor of four about what a typist experiences.

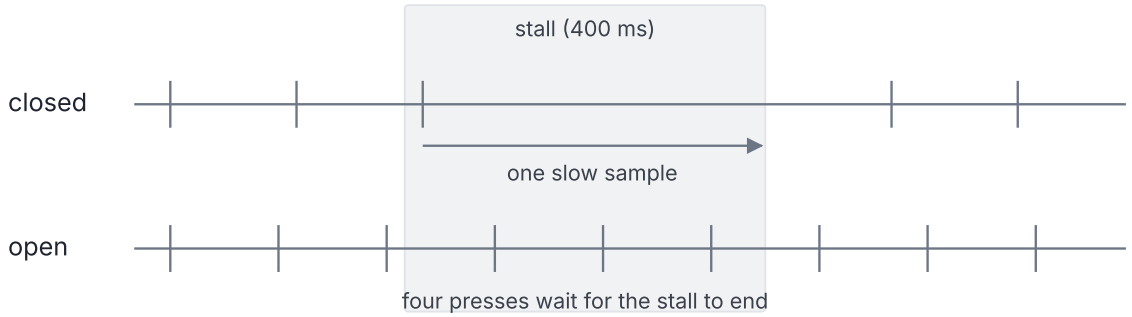


Figure 19.3. A stall (shaded) seen by a closed-loop harness, which waits for each response before the next press, and by an open-loop stream of presses at a fixed rate. The closed loop records one slow sample per stall; the open loop records one for every press that arrives during it.

How much does this matter for the paper's own numbers? Less than it might, for a measurable reason. The paper's latency scenarios have no stalling world, and the gaps between presses (at least 100 ms plus jitter) are longer than the slowest press, so presses never queue behind one another and the closed loop omits nothing. The load sweep is the exception. Under a steady 40 ms load the closed-loop run gives a p95 of 91.4 ms and the open-loop method gives 100.9 ms (median 77.6 against 78.1). With steady load the gap is modest; with intermittent stalls, as in Table 19.6, it is large. The rule that follows is simple. If the users of a system issue requests independently of its responses, measure it with an open-loop generator, or at least check the closed-loop numbers against one.

19.9 Determinism and what the numbers are about

Most of the literature on measurement is about noise: timers with coarse resolution, interrupts, caches warmed by previous runs, processes on the same machine. Mytkowicz and his colleagues showed that even the size of the Unix environment and the order in which object files are linked can change a program's measured performance enough to reverse a conclusion, so that an experimenter can produce "wrong data without doing anything obviously wrong" [Mytkowicz et al. 2009]. Their remedy is to randomise the setup and report the variation across setups.

Virtual time removes that kind of noise. Every run of a script on one build produces the same trace, byte for byte, and the same 22 screen captures. Across three builds (64-bit, 32-bit, and 64-bit with AddressSanitizer) 18 of the 22 captures are identical; the 4 that differ are screens of the system monitor and the About app, which display heap sizes or are timed by them, and heap sizes depend on pointer width and on the allocator.

That is a strong property for an experimenter. A change in a number is caused by a change in the code or the script, never by chance, and an experiment can be rerun months later with the same result. The whole suite of 79 runs takes about 21 seconds of host time, so rerunning everything after every change is cheap.

The price is a different kind of error, which repetition cannot reveal: **model error**. A virtual-time measurement is exactly right about the model and only as right about the device as the model is. The paper's calibration section measures this directly against three programs timed on the ESP32-P4. The 100,000-iteration loop sets the step cost, so it matches by construction (133 ms, 200,002 steps, 665 ns per step). `fib(20)` takes 43,783 steps, which the model prices at 29.1 ms against 31 ms measured, 6% low. The 10,000-iteration loop is priced at 13.3 ms against 11 ms measured, 21% high. And some work the model does not see at all: a loop that allocates an object per iteration takes the same 20,002 steps for 10,000 iterations as the integer loop, because allocation happens inside the engine without touching the step counter. Two consequences follow for anyone reading this book's numbers:

- **Comparisons are more reliable than absolutes.** A change that halves the bus time on the virtual clock will roughly halve it on the device, because bus time comes from clock rates and byte counts. The absolute latency of 30.9 ms carries the calibration's uncertainty and the unmodelled panel refresh.
- **The host's own clock measures only the simulator.** The host build runs 823 times faster than real time on an idle system and 149 times faster with the snake game on autopilot, best of three runs each on one machine, boot included (Chapter 18 measures after boot and gets higher figures). Those numbers do have noise, which is why they are reported as the best of three and labelled with the host's processor. They say nothing about the device.

19.10 Reporting

A measurement that cannot be checked is an assertion. The PicoCalc OS paper follows a discipline that is worth copying. No number in its text is typed by hand. `bench/run.py` writes every measurement to `bench/results.json`; `paper/data.py` computes every number the text uses from that file and puts it in a dictionary; the paper's template refers to the numbers by name, and the build refuses to produce a document if a name has no value. Here is how the launcher's median gets into the prose:

▸ paper/data.py

```
D[f"lat_{key}_min"] = f1(s["min"]); D[f"lat_{key}_p50"] = f1(s["p50"]);
D[f"lat_{key}_p95"] = f1(s["p95"]); D[f"lat_{key}_max"] = f1(s["max"])
```

When the benchmark is rerun, the text changes with it. When this book's chapters quote a number from the paper, they take it from the same dictionary. The cost is a little indirection; the benefit is that the text and the data cannot drift apart, which in a long project they otherwise always do. The paper's own history shows why it matters: its fact-check (Chapter 20) found prose that described an earlier version of the system, but no number that disagreed with the data.

Beyond that, a latency report should let a reader answer the following questions without asking the author.

Table 19.7. What a latency report should state.

Question	In the PicoCalc OS evaluation
What are the endpoints?	key reaches the keyboard controller; first changed visible pixel in the panel controller's memory; update complete when the panel has been idle for 15 ms
On which clock, and how is it related to the real system?	virtual time, calibrated by one device measurement and checked against two more
How many samples, and how were they spread?	80 or 100 presses per scenario, with gaps jittered over 160 ms
What else was running, and what was changed to control it?	two periodic redraws slowed (the quiet roots)
Open or closed loop?	closed loop; gaps longer than the slowest press except under load
Which statistics?	min, median, p95, max, and the CDF; nearest-rank percentiles
What does the model leave out?	panel refresh, built-in and allocation cost, caches, interrupts
Where are the raw data and the code?	`bench/results.json`, `bench/traces/`, `bench/run.py`

Raj Jain's textbook lists the classic ways performance studies go wrong, from unstated goals and biased workloads to presenting ratios that flatter the author's system [Jain 1991]. Dean and Barroso explain why the tail, not the median, governs the experience of a system built from many parts: when one user action

waits on many components, the slowest of them sets the pace, and rare slow responses become common ones [Dean and Barroso 2013]. PicoCalc OS is small enough that one key press touches six components; the same reasoning applies at that scale.

KEY POINTS

- A latency is defined by its two endpoints, and the start must be the cause as the user experiences it: the key reaching the keyboard controller, not the driver reading it.
- An endpoint like "the update is complete" needs a rule; the rule's threshold must exceed the pauses inside one update (3.3 ms when opening an app) and stay below the gaps between independent updates.
- Latencies are distributions; report the number of samples, the median, tail percentiles and the extremes, and show the CDF.
- A periodic measurement aliases with periodic processes in the system; presses every 200 ms gave a median equal to the p95, while presses every 207 ms showed the true 18 to 40 ms spread.
- Contamination by other activity shows up at the edges of a distribution; a sample below the physical lower bound (8.4 ms against a 13 ms I2C floor) is a measurement bug.
- Decomposing a latency into stages and predicting the whole from the parts (18.1 to 40.8 ms predicted, 18.8 to 40.7 ms measured) both explains the latency and validates the measurement.
- A sweep tests a decomposition; varying priority alone found nothing, varying it with load revealed the cost of non-preemption.
- A closed-loop harness omits the requests that would arrive during a stall; under 400 ms stalls every 2 s it reported a p85 of 40 ms where an open-loop stream saw 180 ms.
- Virtual time removes noise but not model error; report how the model was calibrated and what it leaves out.

Measurement is where a system's design meets its behaviour, and every claim in the earlier chapters of this book about what PicoCalc OS costs rests on the methods described here. The next chapter steps back from measurement to the practice of building systems software as a whole: how to structure it, how to keep its claims true, and how the lessons of this study (generate numbers from data, check claims against code) apply to the rest of the work. Chapter 20 closes the book.

EXERCISES

1. BASIC The launcher's latency samples have $n = 100$. Using the `summary` function in `bench/run.py`, which sorted sample (counting from 1) is reported as the median, and which as p95? Which would be reported as p99, and how many samples is that estimate resting on?

2. BASIC Compute, from the I2C formula in Section 19.6, the time of the keyboard driver's status read and FIFO read at 100 kHz and at 400 kHz, and check your results against the 2.4 and 2.1 ms the paper reports for the status read.

3. ANALYSIS The poll wait is uniform on [0, 20) ms. Assuming the rest of the latency is a constant 15.5 + 2.5 ms, what are the mean, median and p95 of the latency? Compare with the measured launcher values and explain the difference using the two rectangle sizes.

4. ANALYSIS In the phase-locked run (Table 19.3, 200 ms period), 38 of the 60 presses took 20.9 ms and 19 took 18.1 or 18.2 ms. Using the decomposition in Section 19.6, work out how long these presses waited for the poll and explain the difference between the two groups. Two presses took 25.0 and 27.8 ms; say what delayed them and how you would confirm it from a trace.

5. CODE Reproduce Table 19.3. Write a script for the host build that starts a trace, then presses `right` 60 times with `key right 60` followed by `wait 130`, and compute from the trace the time from each `key` record with `down` to the next `px` record's `t1`. Then change the wait to 137 and repeat.

6. CODE Reproduce the contamination in Table 19.4 by running `wait 1200`, `key m`, `wait 800`, `latency tab 80 237` with the shipped `js/` tree. Find the press that produced the 8.4 ms sample in a trace and identify the segment whose pixels it measured.

7. CODE The open-loop measurement in Section 19.8 attributes each press by following the trace. Write the attribution as a function over the trace records and run it on an open-loop launcher run with no background load. Does it agree with the closed-loop `latency` command's distribution? It should; explain why.

8. ANALYSIS For the stall experiment in Table 19.6, estimate from first principles the fraction of open-loop presses that should take more than 60 ms, and the fraction of closed-loop presses, given that a normal closed-loop cycle (latency, 15 ms idle, 100 ms gap and on average 30 ms of jitter) lasts about 175 ms.

9. DESIGN The 15 ms idle threshold for "complete" was chosen by hand. Propose an endpoint for "complete" that does not depend on a threshold, using information the compositor has (for example, which flush answered which key). What would you have to add to the trace?

10. ANALYSIS The first-pixels endpoint is the panel controller's memory, not the glass. If the panel refreshes at 60 Hz and a refresh scans the rows from top to bottom, what is the additional delay before a changed pixel in row *y* is visible, as a distribution? How would it change the launcher's CDF?

11. CODE Using `snapshot` before and after a 10-second `wait`, measure the lane and I2C utilisation of the idle home screen and compare with the paper's 0.72% and 30%. Then open the system monitor and measure again.

12. PROJECT Build an open-loop latency command into `host/script.c`: given a key list, a mean inter-press interval and a count, schedule presses at exponentially distributed intervals (seeded), and report per-press latency by attributing each press through the compositor's `keys` and `draw` segments. Use it to redo the paper's load sweep and compare the tails.

13. DISCUSSION The paper slows two apps' timers for its latency runs. Argue for and against this choice. What would a reader of the paper need to know to judge whether the slowed timers make its numbers optimistic?

FURTHER READING

[Jain 1991] The standard textbook on performance evaluation; its early chapters on common mistakes and on workload selection are worth reading before any benchmark.

[Tene 2013] The post that named coordinated omission, with the argument for why closed-loop load generators under-report tail latency.

[Schroeder et al. 2006] A careful comparison of open and closed load models, with rules of thumb for choosing between them.

[Mytkowicz et al. 2009] Shows how innocuous details of an experimental setup bias wall-clock measurements, and how to detect it.

[Dean and Barroso 2013] Why tail latency dominates in systems built from many components, and techniques for tolerating it.

Systems engineering practice

The earlier chapters of this book were about mechanisms: how a scheduler chooses, how a handle confines authority, how a supervisor restarts a driver. This last chapter is about the practice of building such mechanisms so that they work and so that what is said about them is true. It draws its lessons from the history of PicoCalc OS itself, including the part of that history that was embarrassing: a careful review of the system's own paper found three real defects in code that had been described as correct. By the end of the chapter you should be able to choose interfaces that let a system change, keep a trusted core small and know its size, make a system observable and recoverable by design rather than by retrofit, check a document's claims against code systematically, and plan the step from a simulated machine to a real one.

None of the practices here is new. Most were written down in the 1970s and 1980s, and the chapter cites the people who wrote them. What a small system adds is the chance to see each practice pay off, or fail, on code you can read in an afternoon.

20.1 Interfaces decide what can change

David Parnas argued in 1972 that a system should be divided into modules not by the steps of its processing but by the design decisions that are likely to change: each module hides one such decision behind an interface, so that when the decision changes, only that module changes [Parnas 1972]. The test of a decomposition is therefore a question about the future. When this changes, what else must change?

PicoCalc OS has five interfaces that matter, and each was drawn to answer that question for a specific change the project expected. Table 20.1 lists them.

Table 20.1. The five main interfaces of PicoCalc OS, the decision each hides, and the change it allows.

Interface	What sits on each side	What it hides	The change it allows
The world API (`js/prelude.js`)	world code / the prelude	how messages, handles and timers are represented	changing the host calls without touching any world
The host calls `__k` and `__ka`	the prelude / the nucleus	everything a world can do, as 34 named operations	auditing a world's whole authority in one table
The board (`nucleus/hw.c` and `sim/`)	the nucleus / the hardware	whether the buses and controllers are real or modelled	the port to the ESP32-P4 (planned)
The manifest (`js/system.json`)	the system's configuration / the nucleus	which world may claim which hardware	granting or removing authority without editing code
Frames as data	apps / the compositor	how a screen is drawn and sent	changing the drawing, damage tracking or panel without touching apps

The second row is the one that makes the others enforceable. A world reaches the nucleus through exactly two functions, and the prelude takes them out of the global object before any world code runs, so world code can only use the API the prelude builds on top of them:

▸ `js/prelude.js`

```
const k = g.__k, ka = g.__ka;
delete g.__k;
delete g.__ka;
```

Behind those two functions is a table of named operations. The synchronous ones are listed in one array in `nucleus/vm.c`:

▸ history/first-pass/nucleus/vm.c

```
static const struct { const char *name; op_fn fn; } SYNC_OPS[] = {
  { "gfx.text", op_gfx_text }, { "gfx.fill", op_gfx_fill }, { "gfx.rect", op_gfx_rect },
{ "gfx.line", op_gfx_line },
  { "gfx.commit", op_gfx_commit }, { "gfx.fonts", op_gfx_fonts }, { "gfx.invalidate",
op_gfx_invalidate },
  { "now", op_now }, { "send", op_send }, { "publish", op_publish }, { "log", op_log }, {
"self", op_self }, { "seed", op_seed },
  { "turn.async", op_turn_async }, { "turn.done", op_turn_done }, { "turn.fail",
op_turn_fail }, { "turn.reject", op_turn_reject },
  { "hw.i2c", op_hw_i2c }, { "hw.spi", op_hw_spi }, { "dma.alloc", op_dma_alloc },
  { "h.info", op_h_info }, { "h.attenuate", op_h_attenuate }, { "h.revoke", op_h_revoke
}, { "h.release", op_h_release },
  { "sys.worlds", op_sys_worlds }, { "sys.handles", op_sys_handles }, { "sys.stats",
op_sys_stats },
  { "sys.fault", op_sys_fault }, { "sys.restart", op_sys_restart },
};
```

That is 29 operations, and `__ka` adds five more that finish later (`sleep`, `ask`, `i2c`, `spi.cmd` and `spi.pixels`). Thirty-four operations are the entire authority any world has. This is the operating system's equivalent of a system-call table, and keeping it in one place has a practical benefit beyond tidiness: a reviewer who wants to know what a world can do reads one array, not the whole nucleus. When Chapter 9 analyses what handles do and do not control, this table is where the analysis starts.

John Ousterhout calls a module **deep** when its interface is small compared with what it does [Ousterhout 2018]. The prelude is a deep module in that sense. It turns two functions that take an operation name and loose arguments into the builder API of Appendix A (`driver("keyboard").every(20, …).on("stats", …)`), with handles as objects, payloads encoded and decoded, async handlers tracked, and frames rendered after each turn. A world author never sees an operation name. The nucleus author can rename, split or merge operations, provided the prelude follows.

The last row of the table is the decision that surprised its authors most by how much it bought. Apps in PicoCalc OS never draw. An app's `screen` function returns a tree of plain objects; after each turn the nucleus publishes it on the `frames` channel if it changed; the compositor lays out and draws the focused app's tree. Because a frame is data, an app cannot draw outside its area, cannot corrupt the status bar, and cannot leave the screen half-drawn when it crashes. The compositor could switch from full redraws to damage tracking (Chapter 13) without any app changing. And a frame is something the system can store: when the compositor restarts after a crash, the apps' next frames restore the screen, and when the display driver restarts, the compositor still holds everything it needs to resend it.

20.2 Hints that held up

Butler Lampson's "Hints for computer system design" collects slogans from building the Alto, the Dorado and their software, each illustrated with cases from real systems [Lampson 1983]. Several of them describe decisions in PicoCalc OS so exactly that they are worth naming, because a slogan with a concrete instance is easier to apply next time than an abstract principle.

Table 20.2. Some of Lampson's hints and where PicoCalc OS follows them.

Hint	Where it appears in PicoCalc OS
Keep secrets	Worlds see handles as objects; the nucleus keeps the rights, the parent chain and the holders. A world cannot inspect or forge them.
Make actions atomic	A segment's effects (messages, transfers, timers, revocations) are applied at its end and discarded if it crashes (Chapter 3).
Separate normal and worst case	The normal path delivers messages and runs handlers; the worst case (a budget overrun, an out-of-memory) is detected at interrupts and after host calls and handled by tearing the world down.
Shed load	Timer ticks coalesce: a world that is busy gets one tick, not a backlog. An ask that nobody answers fails after its timeout instead of waiting forever.
End-to-end	After a display driver restart, the compositor does not try to work out which pixels the new driver is missing; it invalidates its copy and resends every pixel.
Use a good idea again	The virtual clock prices bus transfers and interpreter steps the same way: by counting what was done and multiplying by a calibrated cost.
Do one thing well	The nucleus does mechanism: it runs segments, moves messages and checks handles. Policy (what the screen shows, which app has focus, what a key means) lives in JavaScript worlds.

Two of these deserve a comment because they cost something. "Make actions atomic" made the nucleus more complicated: every operation with an outside effect has to be wrapped in a deferred record, applied in order at the segment's end, and freed if the segment dies. The `revoke` operation was the last to be converted, and only after a review noticed that it acted immediately (Section 20.6). "End-to-end" costs bandwidth: resending a whole frame after a display restart takes 41 ms of SPI at 40 MHz when a few kilobytes might have done. Both costs are worth paying, but a design document should say they were paid.

20.3 Keep the trusted core small, and count it

The **trusted computing base** of a system is the code whose correctness its guarantees depend on (Chapter 8 develops the idea). For PicoCalc OS's isolation guarantees, that is the nucleus, the QuickJS engine that every world runs in, and the prelude that every world starts with. A bug in any of them can let one world affect another or escape its budget. A bug in a driver, the compositor or an app cannot, because each is confined to its own runtime and to the handles it holds.

It is easy to say a core is small. It is more useful to count it, and to keep counting it, because cores grow. Table 20.3 counts PicoCalc OS's trusted base as it is today.

Table 20.3. Lines of code in PicoCalc OS's trusted computing base for isolation (counted with wc -l), and in the parts outside it.

Part	Files	Lines	Trusted for isolation?
Nucleus: lane, clock, messages, supervision	`nucleus/kernel.c`	634	yes
Nucleus: runtimes, host calls, accounting	`nucleus/vm.c`	835	yes
Nucleus: handles and buses	`nucleus/hw.c`	282	yes
Nucleus: fill, text, diff	`nucleus/gfx.c`	179	yes
Nucleus: types, utilities, trace, PNG	`nucleus/nucleus.h`, `util.c`, `trace.c`, `png.c`	444	yes
The prelude	`js/prelude.js`	206	yes, for each world's own sealing (the clock, randomness)
QuickJS (engine, regular expressions, Unicode, number formatting)	`third_party/quickjs/*.c`, `*.h`	80,864	yes
Drivers	`js/drivers/*.js`	129	no
Compositor	`js/system/compositor.js`	429	no (trusted only for what the screen shows)

The nucleus is 2,374 lines; the engine it depends on is 80,864, about 34 times larger. That ratio is the honest summary of language-based isolation: the operating system's own code is small, but it rests on a language runtime that is large and was written for speed and completeness rather than for isolation between mutually hostile programs. Chapter 10 discusses what that means for security. For engineering, it means the nucleus should stay small precisely because it is the part this project can actually review line by line, and that review should be where effort goes.

The review described in the next sections is evidence for both halves of that sentence. All three real defects it found were in trusted code: two in `nucleus/vm.c` and one in the prelude. The most carefully written part of the system still had bugs, and they were found because the trusted part was small enough to check against every claim made about it.

20.4 Observability is a design decision

A system you cannot see into is a system whose behaviour you can only guess. PicoCalc OS was built with four ways to see into it, each added because a question could not be answered without it.

The **trace** writes one JSON line for every segment, message, bus transfer and changed-pixel chunk (Appendix B lists the fields). It is 67 lines of C, and its first comment states its contract: "It observes; it does not change what the system does." On a virtual clock that contract is easy to keep, because writing the trace takes host time but no virtual time, so a traced run and an untraced run are identical. On real hardware every probe has some cost, and the instrument perturbs what it measures; DTrace was designed around making that cost zero when a probe is off and small when it is on, so that instrumentation could be left in production systems [Cantrill et al. 2004]. Every latency, recovery time and stage duration in Chapter 19 came from this trace.

Counters answer "how much?" without a trace. The `stats` script command prints them; `snapshot` emits them as JSON so a benchmark can subtract two snapshots and get the utilisation of a window. Here is `stats` after 1.5 s of an idle system on the 64-bit host build:

```
t = 1500.0 ms, lane busy 18.8 ms (1.3%), 108 turns, 6 published, 12 delivered, 0 crashes
  world       kind     state        turns    cpu ms   mem KB  limit  rst  last error
  keyboard    driver   idle            71      7.96    189.2    320    0
  power       driver   idle             1      0.30    186.8    320    0
  display     driver   idle             4      2.90    191.8    320    0
  compositor  service  idle            12      4.79    272.0    768    0
  launcher    app      idle             1      0.28    200.8    384    0
  …
  bus i2c0  busy   433.80 ms, 72 transfers, 290 bytes, 0 errors
  bus spi2  busy    64.02 ms, 38 transfers, 318842 bytes, 0 errors
```

One line of it led to a finding the paper reports: the I2C bus had been busy for 434 of the first 1,500 ms with almost no keys pressed. The keyboard driver's polls take about a third of the bus at 10 kHz (Chapter 13).

The **system monitor** is an app that shows the same counters on the device's own screen: worlds with their CPU time, heap and restarts; the handles the kernel has given out, with their rights and holders; the buses' load. It holds the `sys.read` claim, so the kernel lets it read those tables and other apps cannot. Building it early meant that every experiment in this book could be watched as it happened, on the screen of the thing being studied.

The **fault frame** is one line written to the console UART when a world crashes. It is designed to be read by a program, a decision discussed in Section 20.8.

The lesson is not that every system needs these four instruments. It is that the instruments are part of the design. They determine which questions can be answered cheaply, and a question that is expensive to answer tends not to be asked.

20.5 Recovery designed in

Chapter 14 and Chapter 15 describe how PicoCalc OS recovers from failures. The practical point for this chapter is that recovery could not have been added at the end. It depends on properties that had to be true from the start:

- **A crash leaves nothing half-done.** Because a segment's effects are applied only when it completes, a world that crashes in the middle of a turn has sent no partial set of messages and started no partial set of transfers. Recovery then only has to restart the world, not undo what it did.
- **State that matters can be rebuilt.** A driver holds as little state as possible and rebuilds it on start (the display driver repeats the panel's power-up sequence); the state that must survive lives in a world that

has not crashed (the compositor holds every app's latest frame).

- **Authority is given again, not inherited.** When a world restarts it holds no handles. The compositor listens on `worlds.status`, and when the display driver is running again it hands over a fresh read-only view of the framebuffer and marks every pixel for resending.
- **Old work cannot reach a new instance.** Each restart increments the world's epoch, and settles and timers from an older epoch are discarded.

Recovery code has a peculiar problem: it runs rarely, so it is rarely tested, so it is where bugs survive. One survived in PicoCalc OS until the review. After a restart, flushes that had been queued for the old display driver can reach the new one before the compositor has re-attached the framebuffer, and the driver answers them with `skipped`. The compositor used to accept that answer and move on, so the pixels in those flushes were never sent and parts of the screen stayed stale until something else changed. It now treats a skipped flush as lost pixels:

▸ `js/system/compositor.js`

```
      const r = await ask("display", "flush", { rects }, { timeout: 1500 });
      if (r.skipped) {   // the panel did not get these pixels: send everything again
        s.fb.invalidate();
        if (!s.resent) { s.resent = true; dirty(s); }
      } else s.resent = false;
```

The `resent` flag limits the resend to one, so a driver that keeps skipping cannot make the compositor loop. The way to keep recovery code honest is to run it on purpose and often. The host build's `crash WORLD` command, the Faults app's experiments and the `sys.fault` operation exist for that, and the benchmark suite kills the keyboard driver, the compositor and the display driver on every run and measures how long each takes to come back (213, 167 and 515 ms).

20.6 Checking claims against code

After the first complete draft of the PicoCalc OS paper, every claim in it was checked against the code and the benchmark data by a reviewer who had not written either. The method was plain and slow: for each sentence that said what the system does, find the code that does it, read it, and decide whether the sentence is true. Michael Fagan's inspections at IBM in the 1970s worked the same way, with a defined checklist and readers who are not the author, and Fagan reported that they improved both the quality of the programs and the productivity of the programmers who used them [Fagan 1976]. Here the checklist was the paper itself.

Most of what the review found was prose that had drifted from the code. The paper described the ESP32-P4 port as if it existed; it said a step happens at every backward jump and call, where QuickJS actually decrements its counter at every jump, every conditional branch and every call; it stated an overrun bound with a factor of two that the code no longer needed; it described a backoff cap of 2 s that the stop at the fifth crash makes unreachable. Each was fixed in the text. Three findings were different: the text described the system correctly as designed, and the code did not do what the design said. Those were bugs.

Confinement. The paper said that a handle can be passed on only if it carries the grant right or by the world that narrowed it, "so the compositor can give the display driver a read-only framebuffer that the driver cannot pass to anyone else." The rule as stated has a hole. The driver cannot pass its read-only view, but it can narrow that view again, to the same rights, and it is the world that narrowed the new copy. Under the rule as written, the copy was passable. The fix makes narrowing preserve the restriction: a world may pass a handle it narrowed only if it could pass the handle it narrowed it from, checked all the way up the chain.

▸ nucleus/vm.c

```
// A world can pass a handle on if the handle carries the grant right, or if the world
narrowed it from
// a handle it could pass on itself. (Narrowing a handle you may not pass does not make
it passable.)
static bool can_pass(world_t *W, handle_t *h) {
  for (int depth = 0; h && depth < 32; depth++) {
    if (h->rights & R_GRANT) return true;
    if (h->minter != W->id || !h->parent) return false;
    handle_t *p = NULL;
    for (int k = 0; k < MAX_HANDLES; k++) if (K.handles[k].live && K.handles[k].id == h-
>parent) { p = &K.handles[k]; break; }
    h = p;
  }
  return false;
}
```

The regression test is a pair of worlds in the benchmark manifest. `owner` allocates a framebuffer and gives `relay` a read-only view; `relay` tries every way out:

▸ bench/worlds/relay.js

```
service(self).on("try", (s, { view }) => {
  const out = {};
  try { send("ponger", "count", { h: view }); out.direct = "passed"; } catch (e) {
out.direct = e.message; }
  try { send("ponger", "count", { h: view.narrow("r") }); out.renarrowed = "passed"; }
catch (e) { out.renarrowed = e.message; }
  try { view.fill(0, 0, 1, 1, "#fff"); out.write = "wrote"; } catch (e) { out.write =
e.message; }
  return out;
});
```

Every benchmark run records the result, and the current one is three refusals: "handle 2 cannot be passed on by relay", "handle 3 cannot be passed on by relay", "handle 2 is read-only". Chapter 9 develops the rule and its proof.

TRY 20.1

Reproduce the confinement bug

The playground implements PicoCalc OS's handle rules for three worlds. Step through the scenario: the compositor allocates a framebuffer and gives the display driver a read-only view of it. Then, acting as the display driver, try to pass the view to the third world directly, and then narrow it and pass the narrowed copy. Switch to the old pass rule and repeat: the narrowed copy now goes through, which is the leak the review found. Read the kernel's reason each time a pass is refused.

```
can_pass(W, h): walk up from h
  if h has g: yes
  if W did not narrow h (or h has no parent): no
  h = h's parent, and repeat
```

compositor `claims dma`

#1 `rwg` by compositor

#2 `r` from #1, by compositor

display `claims spi2`

#2 `r` from #1, by compositor

#3 `r` from #2, by display

snake `no claims`

holds no handles

✓ **1. compositor:** `dma.alloc(320, 320)` → handle 1: a new framebuffer, rights rwg
compositor claims dma in the manifest; framebuffer 0 has 1 handle

✓ **2. compositor:** `h1.narrow("r")` → handle 2: rights r, a child of handle 1
the child gets the parent's rights (rwg) intersected with "r"; compositor is recorded as the world that narrowed it

✓ **3. compositor:** `send("display", "fb", {fb: h2})` → display holds handle 2 once the message is delivered (at the end of compositor's segment)
handle 2 has no g; compositor narrowed it from handle 1, so look there
handle 1 has g: yes

✓ **4. display:** `spi.pixels(…, h2, …)` → display may send handle 2's pixels to the panel

× **5. display:** `h2.fill(0, 0, 8, 8, "#fff")` → handle 2 is read-only

✗ **6. display:** `send("snake", "fb", {fb: h2})` → handle 2 cannot be passed on by display
handle 2 has no g and display did not narrow it: no

✓ **7. display:** `h2.narrow("r")` → handle 3: rights r, a child of handle 2
the child gets the parent's rights (r) intersected with "r"; display is recorded as the world that narrowed it

✗ **8. display:** `send("snake", "fb", {fb: h3})` → handle 3 cannot be passed on by display
handle 3 has no g; display narrowed it from handle 2, so look there
handle 2 has no g and display did not narrow it: no

The clock. The paper said that `Date` reads the virtual clock and "no world can read the host's clock". The prelude replaced the global `Date`, but the engine's own `Date` constructor was still reachable from objects a world could make: a date's `constructor` property led back to it, so `new (new Date().constructor)()` read the host's real time. That broke two guarantees at once, the determinism of replays and the confidentiality of the host's clock (a timer is the raw material of most side channels, Chapter 10). The fix makes the replacement a plain function that shares the engine's prototype and points the prototype's `constructor` back at the replacement:

▸ js/prelude.js

```
  const RealDate = Date;
  const vnowMs = () => BOOT + k("now");
  function VDate(...a) {
    if (!new.target) return new RealDate(vnowMs()).toString();
    return a.length ? new RealDate(...a) : new RealDate(vnowMs());
  }
  VDate.prototype = RealDate.prototype;
  VDate.now = vnowMs;
  VDate.UTC = RealDate.UTC;
  VDate.parse = RealDate.parse;
  Object.defineProperty(RealDate.prototype, "constructor", { value: VDate, writable:
true, configurable: true });
  g.Date = VDate;
```

A world that probes for the escape now finds none. This is the console log of a small world added to the host build for this chapter, which tries both routes at start-up:

```
[     1.303] probe: Object.getPrototypeOf(Date) === Function.prototype: true
[     1.323] probe: new Date().constructor === Date: true
[     1.351] probe: new (new Date().constructor)().getTime() - Date.now(): -0.3369140625
[     1.379] probe: Date.now(): 1792662060001.353 now(): 1.356
```

The constructed date reads the virtual clock (the difference is the fraction of a millisecond that a `Date` object discards), and `Date.now()` is the boot time the prelude fixes plus the virtual milliseconds since boot. The class of bug is worth remembering. Replacing a global in JavaScript does not remove the original; it removes one name for it, and every other path to it (prototypes, `constructor` properties, functions that return instances) has to be closed too. Chapter 10 calls this auditing **ambient authority**.

TRY 20.2

Probe a sealed clock

This sandbox runs JavaScript in a Web Worker with a prelude that seals `Date` and `Math.random` the way PicoCalc OS's prelude does, and stops the code when it runs past its budget. Run the probe, then edit it to try other routes to real time or real randomness (a `Date` method, `performance.now()`, the timing of a long loop) and see which ones the seal closes. In a browser the budget is enforced in real time by terminating the worker, not on a virtual clock.

```
console.log(Object.getPrototypeOf(Date) === Function.prototype);
console.log(new Date().constructor === Date);
console.log(new (new Date().constructor)().getTime() -
Date.now());
console.log(typeof performance);
```

run 1 virtual clock at +1 s world epoch 1 ok

```
true
true
0
undefined
→ undefined
```

The budget. The paper's Property 3 said that a segment with budget *B* is stopped soon after *B*. The kernel checked the budget only at interrupts, which QuickJS raises every 1,000 interpreter steps, and it counted only those steps. Native work that a world asks the nucleus to do (filling rectangles, drawing glyphs, queuing DMA) was charged to the segment's cost but never compared with its budget. A loop of full-screen fills is almost all native work. The benchmark's `painter` world runs exactly that loop, and its trace shows the proportions: in its 20.26 ms segment it executed 254 interpreter steps and 20.05 ms of native fills, about 3.3 steps for each 0.26 ms fill. Counting steps alone, a 20 ms budget would have allowed about 9,000 fills, roughly 2.4 s of virtual time, before the check fired. The fix compares the whole cost with the budget and checks it after every host call as well as at interrupts:

▸ nucleus/vm.c

```
static bool over_budget(world_t *W) {
  vtime_t used = PROFILE.turn_base_us + (vtime_t)(vm_steps(W) * PROFILE.ns_per_step /
1000.0) + K.seg_cost;
  return used > W->seg_budget_us;
}
…
static JSValue after_host_call(JSContext *ctx, world_t *W, JSValue r) {
  if (!W->overrun && !over_budget(W)) return r;
  W->overrun = true;
  JS_FreeValue(ctx, r);
  JS_ThrowInternalError(ctx, "ran past its budget");
  JS_SetUncatchableException(ctx, 1);
  return JS_EXCEPTION;
}
```

The exception is uncatchable, so a world cannot wrap its loop in `try` and keep going. The regression test is the `painter` world itself, now stopped at 20.25 ms of its 20 ms budget, the overshoot being part of one fill. Chapter 11 derives the bound.

Two smaller defects came out of the same review. `revoke` took effect at the moment it was called rather than at the end of the segment, so a segment that revoked a handle and then crashed had an effect despite crashing; it is now deferred like every other effect, which restores the atomicity property. And the compositor's handling of skipped flushes, described in Section 20.5, was fixed.

What made these findings possible? Each defect sat where a claim was stated generally ("cannot pass to anyone else", "no world", "a segment with budget *B*") and the code handled the cases its author had thought of. A general claim invites an adversarial reading: what is the most devious thing a world could do here?

The benchmark worlds `relay` and `painter` are those adversarial readings turned into code, and they now run on every benchmark run, so the claims are checked continuously rather than once. For a claim that matters, the best test is a world written by someone trying to break it.

20.7 Documentation that stays true

The review found prose that had drifted from the code, and no number that had drifted from the data. That asymmetry has a cause. Every number in the paper is computed from the benchmark results by `paper/data.py` and inserted into the text by name, and the paper's build fails if a name has no value (Chapter 19 shows the mechanism). Prose has no such link to the code it describes, so prose drifts.

While this book was being written, the repository showed the drift in miniature. The README and the QuickJS patch notes described an interpreter step as "one step per loop back-edge or call"; QuickJS's interpreter actually decrements the counter at every `goto`, at every conditional branch whether it is taken or not, and at every function call (and in a few engine loops that walk prototype chains). The patch notes said the budget was checked "about every 1.3 virtual ms", a figure from before the step cost was calibrated; with $\tau = 665$ ns, 1,000 steps is 0.67 ms. The README and the kernel's header comment said restarts back off "to at most 2 s", which matched a cap in the code, but a world is stopped at its fifth crash in ten seconds, so the cap never takes effect and the longest wait that ever happens is 800 ms. None of these was a bug in the system. Each was a sentence that had been true, or nearly true, when it was written. All three have since been corrected, and the corrections came from the same habit as the bug fixes: reading the sentence next to the code it describes.

Three practices keep documentation closer to the truth:

- **Generate what can be generated.** Numbers come from data; tables of operations, script commands and manifest fields can come from the source. This book's own excerpts of PicoCalc OS code are checked against the source files when the book is built, so a renamed function or a changed line breaks the build rather than misleading a reader.
- **State claims so they can be checked.** "The compositor cannot pass the framebuffer on" is checkable; "handles provide strong isolation" is not. A checkable claim can become a test.
- **Read the documentation against the code at fixed points.** Before a release, before a paper is submitted, after a large change. A review is cheapest when the document is short and the code is small, which is one more argument for both.

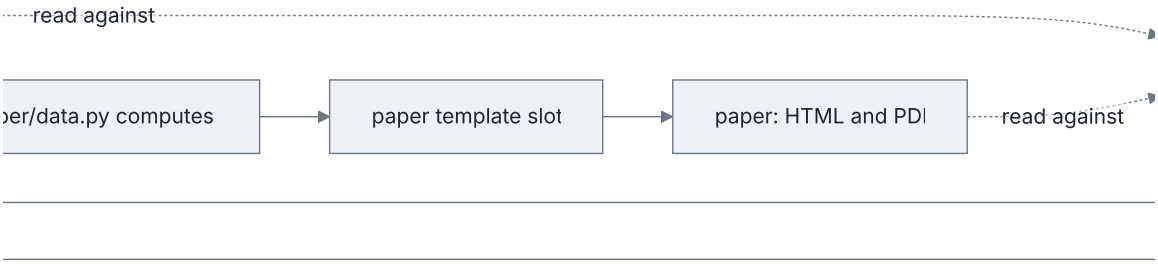


Figure 20.1. How the paper's numbers stay tied to the system: the benchmark writes results, a script computes every number the text uses, and the build refuses a document with a missing value. The adversarial bench worlds turn claims into tests on every run.

20.8 Working with an agent on the console

The fault frame was designed for a reader that is a program. When a world crashes, the nucleus writes one line to the console UART: the prefix `#OCT1`, then a JSON object. Here is the one the `flaky` service produces when the Faults app makes it throw:

```
[  2033.506] flaky crashed handling boom: boom, thrown on purpose (system/flaky.js:4:38)
#OCT1
{"ev":"fault","t":2033.506,"world":"flaky","file":"system/flaky.js","handling":"boom","wh
y":"boom, thrown on purpose
(system/flaky.js:4:38)","crashes":1,"mem_kb":183,"limit_kb":256,"detail":"    at
<anonymous> (system/flaky.js:4:38)\n    at <anonymous> (prelude.js:139:17)\n"}
```

The first line is for a person reading the log. The second is for a program, and its design follows from what a program on the other end of a serial cable needs. A fixed prefix lets it find frames in a stream that also carries ordinary log lines. The version in the prefix (`OCT1`) lets the format change without breaking old readers. One frame per line means a reader never has to reassemble a record. And the fields are chosen so that the reader can act without asking the device anything: which world, which source file, which message it was handling, why it failed, how many times it has crashed in the last ten seconds (so whether the supervisor is about to stop it), how much heap it was using against its limit, and the stack.

The program on the other end can be a diagnostic agent, including one built on a language model, and the PicoCalc OS paper names the next step: a handler that reads a frame, proposes a corrected version of the world's file, and sends it back on the same line for the nucleus to load into the restarted world. Two pieces of the design make that idea safer than it sounds, and one is missing.

- **Authority stays in the manifest.** A world's claims come from `js/system.json`, not from its source. A replacement file can change what a world does with its handles, but it cannot give the world a claim it did not have. An agent can fix the keyboard driver; it cannot turn the keyboard driver into something that holds the display.
- **Supervision bounds a bad fix.** If the replacement crashes, the supervisor restarts it with backoff, and after five crashes in ten seconds it stops it. A fix that makes things worse is contained by the same mechanism that contained the original bug.
- **What is missing** is a way to replace a world's source while the system runs and to roll back to the previous version if the new one fails. The nucleus can restart a world; it cannot yet change what it restarts. That is planned work, not a feature.

The host build adds a property that matters for any program working on the system: determinism. `./build/picocalc-host -` reads script commands from standard input, so a program can drive the system interactively, and because every run of a script is identical, the script that reproduced a fault can be rerun against a proposed fix to see whether the fault is gone before anything reaches the device. A fault that can be replayed exactly is one that can be fixed with evidence.

20.9 The road to the device

Everything measured in this book was measured on a model of the device. The port to the ESP32-P4 is planned, and its plan follows from the interfaces in Section 20.1. The board seam is where it happens: the functions in `nucleus/hw.c` that now drive the simulated controllers (`run_i2c`, `run_spi_cmd`, `run_spi_pixels`) will call ESP-IDF's I2C and SPI master drivers, with DMA from framebuffers in PSRAM. The nucleus's loop will run in one FreeRTOS task, with the real clock in place of the virtual one; budgets will still be enforced by counting interpreter steps and native work, because that mechanism does not depend on what the clock is. World heaps will come from PSRAM. The fault frames will go out on the board's USB console. The JavaScript, drivers included, is not expected to change. Chapter 7 describes the platform in detail.

The order of the first measurements on the device follows from where the model is least certain (Chapter 18):

1. **The step cost τ, from more than one program.** The model has one calibrated constant, set by a single loop. The first job is to time a set of programs on the device, including allocation-heavy code and code that leans on built-in functions such as `JSON.stringify`, which the step counter does not see. If τ fits all of them, the model's latency numbers carry over; if not, the second calibration point says by how much and in which direction.

2. **The keyboard path.** It is 83% of the median press's latency in the model, and every millisecond of it rests on two assumptions: that the STM32 needs 2 ms between the register write and the read, and that its I2C runs at 10 kHz. A logic analyser on the I2C lines measures both directly, and can test whether the controller accepts a faster clock and whether it can raise an interrupt instead of being polled.
3. **SPI and the panel.** The model sends a full frame in 41.0 ms at 40 MHz. The analyser confirms that; the panel's own refresh and the liquid crystal's response, which the model leaves out, need a measurement at the glass.
4. **Memory per world.** The 32-bit host build predicts about 190 KB per world at the median and 2.5 MB for all thirteen. The device's allocator and PSRAM placement will differ, and the limits in the manifest may need adjusting.

Each of these is a comparison between a prediction already written down and a measurement not yet made. That is the most useful position a simulation can leave a project in.

KEY POINTS

- Divide a system by the decisions likely to change, and hide each behind an interface; PicoCalc OS's board seam exists so that the device port changes one layer.
- Put a system's entire authority in one place: the 34 host operations behind `__k` and `__ka` are everything a world can do.
- Count the trusted base and keep counting: the nucleus is 2,374 lines resting on an 80,864-line engine, and every real defect found by review was in trusted code.
- Instruments are part of the design; the trace, the counters, the system monitor and the fault frame decided which questions could be answered cheaply.
- Recovery depends on properties that must hold from the start: atomic segments, rebuildable state, re-granted authority and epochs.
- Recovery code runs rarely, so run it on purpose; the skipped-flush bug lived in a path that only a restart exercises.
- Read every claim against the code; general claims ("no world can…") deserve an adversarial test, and the `relay` and `painter` worlds now check two of them on every run.
- Generate documentation from data and source where you can; the paper's numbers never drifted, its prose did.

- Design console output for programs as well as people: a versioned prefix, one record per line, enough context to act.

This book began with a single key press and the four jobs an operating system does when one program's key becomes another program's pixels: multiplexing the processor and the buses, isolating thirteen programs that share one address space, mediating every use of the hardware, and recovering when a part fails. It has followed three threads through PicoCalc OS. Isolation by language showed what a type-safe runtime can enforce and where it depends on a large engine and on closing every path to ambient authority. Time showed how a calibrated virtual clock, budgets and a little real-time theory make a non-preemptive system's behaviour predictable and measurable. Recovery showed how drivers that crash can come back without the rest of the system noticing more than a pause. The practices in this chapter are what keep all three honest: interfaces that let the design change, a core small enough to check, instruments built in, and every claim read against the code. The next step for PicoCalc OS is the device. The next step for the reader is to change something in it, predict what will happen, and measure whether you were right.

EXERCISES

1. BASIC List the five asynchronous operations behind `__ka` and, for each, the world-API function in `js/prelude.js` that calls it.
2. BASIC Using Table 20.3, compute what fraction of the trusted base for isolation is PicoCalc OS's own code (nucleus and prelude) and what fraction is QuickJS. Which of the three defects in Section 20.6 was in which part?
3. ANALYSIS In the painter's overrun segment the trace records 254 steps and 20,047 µs of native work. Verify the claim that step-only accounting would have allowed about 9,000 fills (about 2.4 s). State your assumptions about the cost of a step and of a fill.
4. ANALYSIS Give a sequence of operations by which, under the old pass rule ("grant right, or the world that narrowed it"), the display driver could have handed a writable view of the framebuffer to an app. If no such sequence exists, explain why the leak was limited to read-only views.
5. CODE Write a benchmark world in the style of `relay.js` that tests the claim "a segment that crashes has no effects": it should revoke a handle, send a message and start a transfer, then throw. Add it to `bench/system-bench.json` and report what another world observes.

6. CODE Write a world that probes for other paths from inside a world to the host's clock or to real randomness: `performance`, `Date.prototype` methods, error stack timing, `Math.random` after `delete Math.random`, and anything else you can think of. Run it on the host build and report which paths exist.

7. CODE Before they were corrected, the README and `third_party/quickjs/PATCHES` described an interpreter step as a loop back-edge or a call. Find, in `third_party/quickjs/quickjs.c`, every place where the interrupt counter is decremented, and write a correct one-sentence definition with a reference to each opcode family.

8. DESIGN Design a version 2 of the fault frame (`#OCT2`) for an agent that proposes fixes. What would you add (for example, the world's source hash, the last messages it received, the handles it held)? What would you deliberately leave out, and why?

9. DESIGN Specify the missing piece of Section 20.8: replacing a world's source at run time with rollback. Define the operation, who may call it (which claim), what happens to the world's state and handles, and when the nucleus rolls back.

10. ANALYSIS For each of Lampson's hints in Table 20.2, find one place in PicoCalc OS where the hint is not followed, or where following it would have been worse. Justify each.

11. PROJECT Extend the paper's approach to the README: generate the script-command table and the manifest-field table from `host/script.c` and `nucleus/kernel.c`, so that a renamed command or a new field updates the README automatically. Decide what to do when the source and the README disagree.

12. PROJECT Plan the first day with a PicoCalc on the bench: write the scripts, the measurements and the expected values (from this book) for the four measurements of Section 20.9, and say what result would make you distrust each of the book's latency numbers.

13. DISCUSSION The review that found the three defects was done by a reader who had not written the code. Argue whether an automated checker (static analysis, a model checker, or a language model reading claims against code) could replace that reader for this kind of review, and what it would need to be given.

FURTHER READING

[Parnas 1972] The original argument for dividing systems by the decisions they hide; short, and still the best statement of why interfaces matter.

[Lampson 1983] A catalogue of design hints from systems that were built and used, each with examples; worth rereading after every project.

[Ousterhout 2018] A modern treatment of module depth, information hiding and complexity, with examples from teaching students to build systems.

[Fagan 1976] The paper that made code inspection a measured engineering practice.

[Cantrill et al. 2004] How to build instrumentation that can stay in a production system, and what it found when it did.

APPENDIX A

The world API

This appendix is the reference for writing a world. Everything a world can do is defined in two places: `js/prelude.js`, which every world evaluates before its own file, and the table of host operations in `nucleus/vm.c`, which the prelude calls through the two functions `__k` (operations that finish during the call) and `__ka` (operations that finish later and return a promise). The prelude deletes both functions from the global object before the world's own code runs, so the API below is the whole of what a world sees.

Two words are used throughout. An operation is **immediate** if it acts during the call. It is an **effect** if the nucleus records it and applies it when the current segment ends, in the order it was recorded; if the segment crashes, its effects are discarded (Chapter 3 explains why). Errors are thrown as JavaScript `Error` objects whose messages are the kernel's own; the messages quoted here are taken from the source, with `%s` and `%d` standing for a name and a number.

A.1 Defining a world

A world's file calls one of three functions and chains methods on the builder it returns.

```
app("counter")                              // or driver(name), service(name)
  .title("Counter")                         // shown in the status bar
  .state({ n: 0 })                          // the initial state, copied on every start
  .needs("keys")                            // channels this world receives
  .on("key", (s, k) => { if (k.key === "up") s.n++; })
  .on("value", (s) => s.n)                  // answers an ask
  .every(1000, (s) => { s.n++; })           // a timer
  .screen((s) => ui.text(`n = ${s.n}`, { font: "xl", align: "center" }))
  .hints("↑ adds one");
```

(This example is not in the repository; the apps in `js/apps/` are complete worlds of the same shape.)

Table A.1. The builder returned by driver(), service() and app().

Method	Meaning
`driver(name)`, `service(name)`, `app(name)`	Declare the world's kind and name. The kind sets its default priority: driver 3, service 2, app 1. The name should match the manifest's; if it does not, the prelude logs `declares itself "x" but the manifest calls it "y"`. Many worlds write `service(self)`.
`.title(t)`	The title the compositor shows in the status bar (default: the world's name).
`.state(obj)`	The initial state. It is copied through JSON at every start, so it may hold only JSON values; the copy is the `s` passed to every handler.
`.needs(...channels)`	Channels this world receives. A message posted on one arrives as a message whose type is the channel name. Since the first pass the manifest entry's `needs` must list each one.
`.provides(...channels)`	Channels this world may post on with `publish`. Since the first pass the manifest entry's `provides` must list each one (Section A.7).
`.on(type, fn)`	The handler for messages of `type`: `fn(s, payload, msg)`, where `msg` is `{ from, type, ask }`.
`.every(ms, fn)`	A timer: `fn(s)` runs every `ms` virtual milliseconds. A world that is busy when a tick is due gets one tick, not a backlog.
`.onStart(fn)`	`fn(s)` runs as the world's first turn after every start.
`.screen(fn)`	`fn(s)` returns a tree of `ui` nodes (Section A.8).
`.hints(h)`	A string, or `fn(s)` returning one, shown in the hint bar at the bottom of the screen.

A handler's turn ends when it returns. If it returns a promise (an `async` function does), the turn lasts until the promise settles, and the world's mailbox stays closed until then: no other message is handled in between. If the message was an `ask`, the handler's return value (or the value its promise resolves to) is the reply; it must be JSON-encodable and may contain handles. A handler that throws, or whose promise rejects, crashes the world (Section A.6). An `ask` for a type the world has no handler for is refused without a crash; the asker's promise rejects with `<world> has no handler for "<type>"`.

After every segment except the one that starts the world, if the world has a `screen` function, the nucleus calls it as part of the same segment. If the resulting frame differs from the last one, the nucleus publishes it on the `frames` channel for the compositor. Apps never draw directly.

A.2 Messages

Table A.2. Sending, asking and publishing.

Function	Kind	Meaning and errors
`send(to, type, payload)`	effect	Deliver a message to the world named `to`. Throws `there is no world called "%s"`. If `to` is stopped wh arrives, it is dropped.
`ask(to, type, payload, { timeout })`	effect, returns a promise	Deliver a message and wait for the reply. The default t virtual ms. Throws at once `there is no world called` `a world cannot ask itself (it would wait for its` The promise rejects with `no reply from %s within %d` `%s is stopped`, `%s crashed: <why>` or `%s has no han`
`publish(channel, payload)`	effect	Deliver the payload to every world that `needs` the cha sender and stopped worlds. Throws `%s does not provide the channel "%s"` if the world d the channel (since the first pass, if its manifest entry c under `provides`).
`sleep(ms)`	effect, returns a promise	Resolves `ms` virtual milliseconds after the segment en

Payloads travel as JSON text. A handle inside a payload is encoded as `{ "$h": id }` and the receiver is given the handle when the message is delivered; a message carries at most 8 handles. Every handle named in a message must be held by the sender and passable by it (Section A.4), or the call throws. The nucleus charges 25 virtual ns per byte for encoding and again for decoding.

The nucleus itself posts on two channels. On `frames` it posts `{ app, title, frame }` whenever an app's frame changes (the compositor needs `frames`). On `worlds.status` it posts `{ world, state, why, restarts, crashes, in_ms, t }` whenever a world crashes (`state` is `"crashed"` and `in_ms` the delay before its restart), is stopped (`"stopped"`), or is running again (`"running"`).

A.3 Time, randomness, identity and logging

Table A.3. The rest of the global environment.

Name	Kind	Meaning
`now()`	immediate	Virtual milliseconds since boot, including the cost of the current segment so far. A busy loop on `now()` therefore ends.
`Date`	immediate	Reads the virtual clock. The clock reads 2026-10-22 09:41:00 UTC at boot. `Date()` called as a function returns a string; `Date.now()` returns virtual milliseconds since 1970 (with a fractional part); `Date.UTC` and `Date.parse` are the engine's. The engine's own constructor is not reachable (Chapter 20).
`Math.random()`	immediate	A xorshift generator seeded from a hash of the world's name, so each world gets the same sequence on every run and after every restart.
`self`	value	The world's name in the manifest.
`log(...values)`, `print(...)`	immediate	Write one line to the kernel log (the console UART), with the virtual time. Strings are written as they are, other values through `show`. A log line is written even if the segment later crashes.
`show(value)`	immediate	A short readable rendering of a value: nested objects to depth 2, the first 20 array elements and 12 keys, handles as `[handle id type rights]`.
`errText(e)`	immediate	An error as `Name: message (file:line)`, using the first stack frame outside the prelude.

A.4 Hardware: handles

A world obtains hardware through a **claim** that the manifest gives it, and receives a **handle**: an object whose `id` the nucleus checks on every use. A handle has `id`, `type` (`"i2c"`, `"spi"` or `"dma"`), `rights` (a string of `r`, `w` and `g`, for read, write and grant), `claim`, and for a framebuffer `w` and `h`. A newly obtained handle has all three rights. Chapter 9 explains the rules; this section lists the calls.

Table A.4. Obtaining handles. All three are immediate.

Call	Needs	Errors
`hw.i2c(claim, hz = 10000)`	a claim like `"i2c0:0x1f"`	`"%s" is not an i2c claim (like "i2c0:0x1f")`; `%s has no claim on %s`; `out of handles`
`hw.spi(claim, hz = 40e6)`	the claim `"spi2"` (the clock is limited to 80 MHz)	`"%s" is not an spi claim (the panel is on "spi2")`; `%s has no claim on %s`; `out of handles`
`dma.alloc(w = 320, h = 320)`	the claim `"dma"`	`%s has no claim on dma`; `a framebuffer is at most 640 x 640`; `out of DMA buffers`; `out of handles`

Table A.5. Methods of a handle. Bus transfers are effects: they start when the segment ends and the bus is free, take the time their bits take on the wire, and settle their promise when done.

Method	Handle	Kind	Meaning
`transfer(bytes, nread, gapMs)`	i2c, `w`	effect, promise	Write `bytes`, optionally hold the bus for `gapMs` `nread` bytes, as one transaction. Resolves to th bytes read. Rejects with `no device answered at 0x%02x on i2c0` for an address.
`read(reg, n = 2, gapMs = 0)`	i2c, `w`	effect, promise	`transfer([reg], n, gapMs)`. The STM32 need `gapMs = 2`.
`write(reg, value)`	i2c, `w`	effect, promise	`transfer([reg | 0x80, value], 0)`: the regist with its top bit set, then the value.
`cmd(c, data)`	spi, `w`	effect, promise	Send command byte `c` and its data bytes to th
`pixels(fb, x, y, w, h)`	spi, `w`; `fb` needs `r`	effect, promise	Send a rectangle of the framebuffer's front buf data, in chunks of up to 32 KiB.
`fill(x, y, w, h, c)`, `rect(…)`, `line(x0, y0, x1, y1, c)`	dma, `w`	immediate	Draw into the back buffer. Charged 2.5 virtual drawn (10 ns per pixel for lines).
`text(x, y, s, font, c, maxw)`	dma, `w`	immediate	Draw text in one of the fonts of `gfx.fonts()`; r width drawn. Charged 6.0 ns per pixel of each
`commit()`	dma, `w`	immediate	Compare the back buffer with the front buffer i 16 rows, copy the changes to the front buffer, the changed rectangles as `[x, y, w, h]` array 64). Charged 1.2 ns per pixel compared.
`invalidate()`	dma, `w`	immediate	Make the front buffer differ everywhere, so the `commit` returns the whole screen.
`narrow(rights)`	any	immediate	Make a child handle with the intersection of the rights and `rights` (for example `"r"`). The calle
`revoke()`	a handle the caller narrowed	effect	Revoke the handle and all its descendants. Thr `only the world that narrowed handle %d can`

Method	Handle	Kind	Meaning
`release()`	any	immediate	Stop holding the handle; returns `true` if it was framebuffer's memory is freed when no handle remains.

Colours are strings (`"#rgb"` or `"#rrggbb"`) or 16-bit RGB565 numbers. `gfx.fonts()` returns the five fonts' cell sizes: `s` 6 × 12, `m` 8 × 16, `mb` 8 × 16 bold, `l` 10 × 20 and `xl` 16 × 32.

Every use of a handle is checked, and a failed check throws one of: `there is no handle %d`, `%s does not hold handle %d`, `handle %d was revoked`, `handle %d is a %s handle, not %s`, `handle %d is read-only`, `handle %d is not grantable`, `handle %d is write-only`, or `the framebuffer is gone`. A handle can be passed in a message if it carries the grant right, or if the sender narrowed it from a handle the sender could pass; otherwise the sending call throws `handle %d cannot be passed on by %s`.

A.5 System tables and administration

Table A.6. System calls guarded by the sys.read and sys.admin claims.

Call	Needs	Kind	Returns or does
`sys.worlds()`	`sys.read`	immediate	An array with, per world: `name`, `kind`, `title`, `state` (`idle`, `busy`, `waiting`, `restarting`, `stopped`), `turns`, `msgs`, `cpu_ms`, `mem_kb`, `limit_kb`, `mbox`, `mbox_peak`, `restarts`, `budget_ms`, `last_error`, `claims`.
`sys.handles()`	`sys.read`	immediate	Every live handle: `id`, `type`, `claim`, `rights`, `parent`, `revoked`, `holders`, and its buffer size, address or clock.
`sys.stats()`	`sys.read`	immediate	The kernel's counters: `t_ms`, `lane_busy_ms`, `turns`, `segments`, `published`, `delivered`, `crashes`, `handles`, `step_ns`, and per bus `busy_ms`, `bytes`, `transfers`, `errors`.
`sys.fault(name)`	`sys.admin`	effect	Crash the named world with the reason `killed by <caller>`.
`sys.restart(name)`	`sys.admin`	effect	Restart a stopped world, or restart a running one (as a crash with the reason `restarted by an administrator`); in both cases its crash count is reset.

Without the claim each call throws `%s has no claim on sys.read` (or `sys.admin`); an unknown name throws `there is no such world`.

A.6 Budgets, memory and crashes

Each segment of a world's turns has the budget given in the manifest; starting a world (evaluating its file and running its first jobs) has a fixed budget of 600 virtual ms. The budget covers the 40 µs cost of entering the world, its interpreter steps at 665 ns each and every native operation it asks for. The nucleus checks it every 1,000 interpreter steps and after every host call, and stops a world that exceeds it with an exception the world cannot catch. Each world's heap has the limit given in the manifest, enforced at every allocation.

A world crashes when any of the following happens, and the reason recorded is:

- an uncaught exception in a handler, or a rejected promise returned by one: the exception's message and location, for example `boom, thrown on purpose (system/flaky.js:4:38)`;
- a segment runs past its budget: `ran past its turn budget of %d virtual ms` (or `start budget` while starting);
- an allocation would exceed the heap limit: `ran out of memory (its limit is %d KB)`, or `ran out of memory while starting (its limit is %d KB)`;
- another world kills it with `sys.fault`: `killed by %s`; the host build's `crash` command: `killed from the host`.

A rejected promise that nothing handles, outside a handler's own promise, does not crash the world; the kernel logs `a promise was rejected and nothing handled it`.

When a world crashes, the effects of the running segment are discarded; an `ask` it was handling fails with `<world> crashed: <why>`; it stops holding all its handles; its pending settles and timers are discarded (its epoch increases); messages already in its mailbox stay there for the next instance. The crash is published on `worlds.status` and written to the console as an `#OCT1` frame (Appendix B). The world is restarted after 100 ms if it is its first crash in the last ten seconds, after 200, 400 and 800 ms for the second, third and fourth; the fifth crash within ten seconds stops it, and asks to a stopped world fail with `<world> is stopped` until a world holding `sys.admin` restarts it.

A.7 The manifest

The manifest, `js/system.json` by default, lists the worlds the nucleus starts, in order. This is the one the system ships with:

▸ history/first-pass/js/system.json

```
{
  "groups": {
    "apps": ["launcher", "sysmon", "repl", "snake", "faults", "keys", "about"]
  },
  "defaults": { "handles": 4, "ask_slots": 8, "mbox": 64, "mbox_kb": 128, "bufs": 1,
"turn_ms": 8000 },
  "worlds": [
    { "name": "keyboard",   "file": "drivers/keyboard.js",  "claims": ["i2c0:0x1f"],
"budget_ms": 20,  "mem_kb": 320,
      "provides": ["keys"] },
    { "name": "power",      "file": "drivers/power.js",     "claims": ["i2c0:0x1f"],
"budget_ms": 20,  "mem_kb": 320,
      "provides": ["power"] },
    { "name": "display",    "file": "drivers/display.js",   "claims": ["spi2"],
"budget_ms": 30,  "mem_kb": 320 },
    { "name": "compositor", "file": "system/compositor.js", "claims": ["dma"],
"budget_ms": 80,  "mem_kb": 768,
      "needs": ["frames", "keys", "power", "worlds.status"], "sends": ["display",
"power", "apps"], "asks": ["display"],
      "handles": 6, "bufs": 2, "mbox": 128 },
    { "name": "launcher",   "file": "apps/launcher.js",     "claims": ["sys.read"],
"mem_kb": 384,
      "sends": ["compositor"] },
    { "name": "sysmon",     "file": "apps/sysmon.js",       "claims": ["sys.read"],
"mem_kb": 512 },
    { "name": "repl",       "file": "apps/repl.js",
"mem_kb": 384,
      "asks": ["sandbox"] },
    { "name": "sandbox",    "file": "system/sandbox.js",
"budget_ms": 200, "mem_kb": 512, "priority": 0 },
    { "name": "snake",      "file": "apps/snake.js",
"mem_kb": 320 },
    { "name": "faults",     "file": "apps/faults.js",       "claims": ["sys.admin",
"sys.read"],                   "mem_kb": 384,
      "needs": ["worlds.status"], "sends": ["flaky"], "asks": ["flaky", "sandbox"] },
    { "name": "flaky",      "file": "system/flaky.js",
"mem_kb": 256 },
    { "name": "keys",       "file": "apps/keys.js",
"mem_kb": 320,
      "needs": ["keys"], "asks": ["keyboard"] },
    { "name": "about",      "file": "apps/about.js",        "claims": ["sys.read"],
"mem_kb": 320 }
  ]
}
```

Table A.7. Fields of a manifest entry, with the defaults the kernel applies (nucleus/kernel.c, kernel_boot).

Field	Default	Meaning
`name`	(required)	The world's name: how other worlds address it, and the seed of its random numbers. At most 31 characters.
`file`	(required)	Its source, relative to the JavaScript root (`js/`).
`claims`	none	Hardware and system access it may obtain: `"i2c0:0xNN"` (an I2C address), `"spi2"` , `"dma"` , `"sys.read"` , `"sys.admin"` . At most 16.
`budget_ms`	50	The budget of each segment of its turns, in virtual ms.
`mem_kb`	1024	Its heap limit, in KB.
`priority`	from its kind: driver 3, service 2, app 1	Its priority on the lane; higher runs first. The sandbox sets 0.

At most 32 worlds can be listed. A world cannot add to its own claims: they come only from the manifest, which is how a system's authority can be read in one file.

The manifest above also has keys this table does not list. The first pass, a later revision described in a separate book, *From L4 to Small Worlds* (whose Appendix A gives the complete schema and every message), added `groups` and `defaults` at the top level and, per world, `provides` , `needs` , `sends` and `asks` (the channels it may publish and hear, and the worlds it may send to and ask) and the quotas `handles` , `ask_slots` , `mbox` , `mbox_kb` , `bufs` and `turn_ms` . Since then a world's `.needs()` and `.provides()` must agree with its entry, and `send` , `ask` , `publish` , `dma.alloc` and `narrow` can fail for the further reasons that book lists.

A.8 The UI vocabulary

An app's `screen` function returns a tree built with `ui.*` . The compositor lays the tree out in the body of the screen, a 304 × 270-pixel area below the 22-pixel status bar and above the 18-pixel hint bar, and draws it. Every function returns a plain object, so a frame is data and can be compared, stored and sent. Containers take an optional options object as their first argument, then children; `null` , `false` and nested arrays among the children are allowed and flattened.

Table A.8. Containers.

Node	Options (default)	Layout
`ui.col(opts, ...kids)`	`pad` (0), `gap` (0), `bg`, `border`, `grow`	Children stacked vertically; children that grow share the leftover height.
`ui.row(opts, ...kids)`	`gap` (8)	Children side by side; a child with a `w` option gets that width, the others share the rest equally.
`ui.box(opts, ...kids)`	`pad` (6), `gap` (4), `bg`, `border`, `grow`	A padded vertical stack, usually with a border or background.

Table A.9. Content nodes.

Node	Options (default)	Draws
`ui.text(s, opts)`	`font` (`m`), `color`, `align` (`left`, `center`, `right`), `wrap`, `pad`, `lead`, `bg`	A line of text, or wrapped lines with `wrap`.
`ui.title(s, opts)`	as text, font `l`	A heading.
`ui.gap(h = 8)`		Empty vertical space.
`ui.sep(opts)`	`color` (`line`), `h` (9)	A horizontal rule.
`ui.bar(v, opts)`	`label`, `right`, `h` (8), `color`	A horizontal bar filled to `v` between 0 and 1.
`ui.spark(values, opts)`	`label`, `right`, `h` (36), `max`, `points`, `style` (line or `"bars"`), `color`, `bg`	A small line or bar chart.
`ui.list(items, sel, opts)`	`font` (`m`), `lead`	A selectable list; items are strings or `{ label, icon, iconColor, color, detail, right, rightColo` Scrolls to keep `sel` visible.

Node	Options (default)	Draws
	(7), `lines`, `visible`, `grow`	
`ui.lines(items, opts)`	`font` (`s`), `lead` (1), `bg`, `bottom`, `scroll`	Wrapped log lines; items are strings or `{ s, color, indent }`; `bottom` keeps the newest visible.
`ui.table(cols, rows, opts)`	`font` (`s`), `head`, `sel`, `visible`	Columns are `{ label, w, align }`; rows are arrays of cells (strings or `{ s, color }`) or `{ cells, color }`.
`ui.grid(items, sel, opts)`	`cols` (3), `gap` (8), `cell` (64)	Tiles with `{ label, icon, key, color, ops }`; the launcher us it.
`ui.kv(pairs, opts)`	`font` (`s`), `lead` (4), `keyW`	Key–value rows from `[key, value, color]` triples.
`ui.canvas(w, h, ops, opts)`	`bg`, `border`	A drawing: `ops` are `["fill", x, y, w, h, c]`, `["rect", …]`, `["line", x0, y0, x1, y1, c]` and `["text", x, y, s, font, c, maxw]`.
`ui.input(value, opts)`	`font` (`m`), `prompt`, `cursor`, `focus`, `busy`, `bg`	A one-line text field with a cursor.

`list`, `lines` and `table` without a `visible` option take the height left over in their container. Colours in the tree may be the compositor's names (`paper`, `ink`, `muted`, `faint`, `line`, `soft`, `accent`, `ok`, `warn`, `blue`) or `"#rrggbb"`.

APPENDIX B

The host build

The host build is PicoCalc OS compiled for Linux against a simulated board: the same C nucleus and the same JavaScript worlds as the planned device build, with models of the ST7365P panel controller and the STM32 keyboard controller in place of the real chips. It runs on a virtual clock, so it is deterministic and much faster than real time, and it is the instrument behind every measurement in this book (Chapter 18 explains how it works). This appendix is its manual.

B.1 Building

The native build needs a C compiler (gcc) and zlib, nothing else; QuickJS is in `third_party/quickjs`.

Table B.1. Make targets.

Command	Builds	Use
`make`	`build/picocalc-host`, 64-bit	everyday work; memory figures are scaled to the device's 32-bit sizes
`make M32=1`	`build32/picocalc-host`, 32-bit	memory measurements (worlds use the memory they would on the ESP32-P4)
`make SAN=1`	`build-san/picocalc-host`, with AddressSanitizer and UBSan	checking the nucleus for memory errors
`make shots`	runs every script in `scripts/`	writes the screen captures to `out/`
`make wasm WASI_SYSROOT=… WASI_RT=…`	`wasm/picocalc.wasm`, `wasm/files.json`	the browser build (Section B.6)
`make clean`		removes all build directories

B.2 Running

```
$ ./build/picocalc-host [-j js_dir] [-m manifest] [-o out_dir] [-u uart_log] [-t ms] [-v
| -vv] [script | -]
```

Table B.2. Command-line options.

Option	Default	Meaning
`-j DIR`	`js`	the JavaScript root: the prelude and every world's file
`-m FILE`	`DIR/system.json`	the manifest
`-o DIR`	`out`	where captures and traces are written
`-u FILE`	`OUT/uart.log`	where the console UART's output (the kernel log and fault frames) goes
`-t MS`	2000	with no script, how long to run
`-v`		copy the kernel log to standard error
`-vv`		and log every message delivered
`script` or `-`		a script file, or `-` to read commands from standard input

Reading commands from standard input makes the host build a program another program can drive: it boots, then waits for commands, and runs the virtual clock only as far as each command says. Every run of the same script on the same build produces the same output.

B.3 Script commands

A script is a text file with one command per line; `#` starts a comment. The scripts in `scripts/` produce the screen captures shown in this book. This one tours the home screen and the system monitor:

▸ scripts/tour.txt

```
# A tour of the home screen and the system monitor.
wait 1200
capture 01-home.png
key right
wait 150
capture 02-home-repl.png
key m
wait 1200
capture 03-sysmon.png
…
```

Table B.3. Every script command (host/script.c). Times are virtual milliseconds.

Command	Effect
`wait MS` (or `run MS`)	Run the system for `MS` virtual milliseconds.
`key NAME [HOLD_MS]`	Press a key, hold it (60 ms by default) and release it; the STM32 model also reports a hold if the key is held 300 ms or more. Runs until 10 ms after the release.
`type TEXT`	Type each character: a 40 ms press, then 40 ms of waiting, so one character every 90 ms.
`capture [FILE [SCALE]]`	Write what the glass shows to `OUT/FILE` as PNG (default `shot-NNN.png`, scale 2).
`battery PCT [charging]`	Set what the STM32 model reports as the battery level.
`crash WORLD`	Kill a world from outside, with the reason `killed from the host`; its supervisor restarts it.
`trace FILE`	Start writing the JSON-lines trace to `OUT/FILE` (Section B.4), replacing any trace already open.
`mark TEXT`	Put a labelled mark in the trace.
`latency KEYS N GAP`	Press the keys (one name, or several separated by commas, used in turn) `N` times (default 10), `GAP` ms apart (default 150) plus a varying 0 to 160 ms; for each press, record the time to the first changed pixels and to the end of the update (the panel idle for 15 ms). Prints a `RESULT` line.
`photonwait LABEL MS`	Run until pixels on the panel change, for at most `MS` (default 2000); print how long it took as a `RESULT` line.
`steps JS`	Count the interpreter steps the expression `JS` takes in a fresh runtime (without the prelude) and print them with their virtual cost.
`memprobe`	Print the bytes taken by an empty runtime and by one with the prelude evaluated.
`snapshot LABEL`	Print the kernel's counters, every world's and every handle as one `RESULT` line.
`stats`	Print the worlds, the lane and the buses as a table.

Command	Effect
`panel`	Print the panel controller model's state.
`print TEXT`	Print `TEXT` with the virtual time.
`quit`	Stop.

Key names are the letters and digits themselves, plus `enter` (or `return`), `bs` (or `backspace`), `tab`, `esc`, `left`, `up`, `down`, `right`, `space`, `f1` to `f10`, `power`, `break`, `insert`, `home`, `del`, `end`, `pgup`, `pgdn`, and any code written as `0x81`. The codes are the STM32 firmware's.

Measurement commands print one line beginning `RESULT` followed by a JSON object, so a benchmark can pick its results out of the output. This is a `latency` run of ten presses in the launcher on the 64-bit build:

```
$ printf 'wait 1500\nlatency right 10 137\nquit\n' | ./build/picocalc-host -
RESULT {"bench":"latency","keys":1,"t":1500.000,"samples":
[20.901,33.921,36.216,36.143,25.339,23.967,40.116,25.622,20.403,32.816],"complete":
[24.436,37.270,42.108,39.446,28.479,29.813,43.651,28.971,26.295,36.119]}
```

A sample of −1 means nothing changed on the panel within one second of the press.

B.4 Traces

`trace FILE` makes the nucleus write one JSON object per line for everything that happens on the lane, the buses and the panel. Times are virtual microseconds since boot. The trace observes and does not change what the system does: traced and untraced runs are identical.

Table B.4. Trace records (nucleus/trace.c and host/script.c). The field k gives the record's kind.

k	Fields	Written when
`seg`	`w` world, `t` start, `dt` duration, `steps` interpreter steps, `native` µs of native work, `kind` (`boot`, `msg` or `settle`), `type` (the message type, `every N ms`, or `start`), `crash` (the reason, if it crashed)	a segment ends
`msg`	`t`, `from` (a world or `kernel`), `to`, `type`, `ask` (true or false), `bytes` (payload length)	a message is put in a mailbox
`bus`	`bus` (`i2c0` or `spi2`), `w` (the world that issued it), `t` start, `t1` end, `bytes`, `what` (for example `write 0x04, wait, read 1`, `cmd 0x2a`, `pixels 96x66 at 216,62`)	a transfer finishes
`px`	`t` start, `t1` end, `changed` (pixels whose visible value changed)	a chunk of pixel data changed the panel
`key`	`t`, `text` (code and state, for example `0xb7 down`)	the script presses or releases a key
`mark`	`t`, `text`	a `mark` command
`crash`	`t`, `w`, `text` (the reason)	a world crashes
`status`	`t`, `w`, `text` (`crashed`, `stopped` or `running`)	the kernel publishes on `worlds.status`

These lines are from the trace of one key press in the launcher, the one analysed in Chapter 19:

▸ bench/traces/keypress.jsonl

```
{"k":"key","t":16987260,"w":"","text":"0xb7 down"}
{"k":"bus","bus":"i2c0","w":"keyboard","t":17000137,"t1":17006137,"bytes":4,"what":"write
0x04, wait, read 1"}
{"k":"seg","w":"launcher","t":17013310,"dt":127,"steps":89,"native":28,"kind":"msg","type
":"key"}
{"k":"msg","t":17013437,"from":"launcher","to":"compositor","type":"frames","ask":false,"
bytes":1135}
{"k":"px","t":17015589,"t1":17018135,"changed":6336}
```

B.5 The console and fault frames

The UART log (`out/uart.log` by default) is what the device would write on its console. Each kernel log line starts with the virtual time in milliseconds. When a world crashes, the kernel writes a second line for programs: the prefix `#OCT1` and one JSON object.

```
[  2033.506] flaky crashed handling boom: boom, thrown on purpose (system/flaky.js:4:38)
#OCT1
{"ev":"fault","t":2033.506,"world":"flaky","file":"system/flaky.js","handling":"boom","wh
y":"boom, thrown on purpose
(system/flaky.js:4:38)","crashes":1,"mem_kb":183,"limit_kb":256,"detail":"    at
<anonymous> (system/flaky.js:4:38)\n    at <anonymous> (prelude.js:139:17)\n"}
```

Table B.5. Fields of an #OCT1 fault frame (nucleus/kernel.c, uart_fault).

Field	Meaning
`ev`	`"fault"`
`t`	virtual time of the crash, in ms
`world`, `file`	the world's name and its source file
`handling`	the message type it was handling, `every N ms` for a timer, or empty
`why`	the reason, first line only (an exception's message and location, a budget, a memory limit, a kill)
`crashes`	crashes of this world in the last ten seconds, including this one; at 5 the world is stopped
`mem_kb`, `limit_kb`	its heap at the crash and its limit
`detail`	the stack, when there is one

Chapter 20 discusses why the frame is designed this way.

B.6 The WebAssembly build

`make wasm` builds the same nucleus, simulated board, script interpreter and worlds as a WebAssembly module for browsers, with clang's `wasm32-wasip1` target and the WASI sysroot and compiler builtins from wasi-sdk. There is no PNG writer and no file system: `host/main_wasm.c` replaces `host/main_host.c` and exports functions that the page calls (`pc_add_file`, `pc_boot`, `pc_run`, `pc_key`, `pc_crash`, `pc_command`, `pc_trace`, `pc_screen`, `pc_stats` and a few more). The module imports two functions from the page, one that receives the console UART and the trace, and one that receives captures, and a handful of WASI calls that `wasm/picocalc.js` implements itself. The page's shim returns a fixed instant for the WASI clock, because nothing in the system's behaviour reads the host's time.

`wasm/bundle.py` writes `wasm/files.json`, the prelude, the manifest and every world's file keyed by the paths the kernel asks for. The loader puts them into the module before it boots:

```
const pc = await PicoCalc.load({ wasmUrl: "picocalc.wasm", filesUrl: "files.json" });
pc.onLog((line, stream) => console.log(stream, line));   // "uart" carries the kernel log
and #OCT1 frames
pc.boot();
pc.run(16);                                              // 16 virtual ms
pc.key("enter", true); pc.key("enter", false);
pc.draw(canvas.getContext("2d"));                        // what the glass shows, 320 x
320
pc.crash("display");                                     // its supervisor restarts it
```

(That sketch follows the comment at the top of `wasm/picocalc.js`.) `PicoCalc.spawn` runs the same module in a Web Worker with the same methods returning promises. `load` rejects with an error whose `code` is `wasm_blocked` when the page may not compile WebAssembly, and `fetch_failed` when the files cannot be fetched. `wasm/demo.html` is a test page: serve `wasm/` over HTTP and open it, adding `?worker` to run the system in a worker. Chapter 17 builds on this.

The module is about 1.2 MB. Because `wasm32` is a 32-bit target, scripts produce the same pixels as the native 32-bit build (`make M32=1`), except on the screens that show heap sizes, which differ by a few KB because WASI's allocator is not glibc's. To keep even the simulated panel's power-on noise identical, `host/main_wasm.c` carries a copy of glibc's `rand()` algorithm.

B.7 The benchmark suite

```
$ make && make M32=1 && make SAN=1
$ python3 bench/run.py                      # everything: about 80 runs
$ python3 bench/run.py latency spi_sweep    # only these experiments, merged into the
existing results
```

`bench/run.py` runs the host build on scripted scenarios, some on patched copies of `js/` (with the system monitor's and the keys app's timers slowed, Chapter 19) or with extra worlds from `bench/worlds/` and the manifest `bench/system-bench.json`. It writes `bench/results.json` and the traces in `bench/traces/`. Everything is in virtual time except the `host` experiment.

Table B.6. The experiments in bench/run.py, the key each writes in bench/results.json, and the chapter whose subject each measures.

Experiment	Key in results.json	Measures	Chapter
`calibration`	`calibration`	interpreter steps and virtual cost of seven small programs; the step cost	Chapter 18
`memory`	`memory`	heap of an empty runtime, the prelude and every world, 32- and 64-bit	Chapter 6
`boot`	`boot`	when worlds start, the panel turns on and the first frame arrives (trace `boot.jsonl`)	Chapter 13
`latency`	`latency`	key-to-pixels latency in five scenarios (trace `keypress.jsonl`)	Chapter 19
`pipeline`	`pipeline`	pixels, rectangles and SPI time per update; the compositor's draw cost (trace `repl.jsonl`)	Chapter 13
`spi_sweep`	`spi_sweep`	latency and full-frame time at 10 to 80 MHz	Chapter 19
`poll_sweep`	`poll_sweep`	I2C load and latency against poll period and bus clock	Chapter 19
`load_sweep`	`load_sweep`	latency under a background load, at two priorities	Chapter 4
`messaging_and_faults`	`messaging`, `faults_bench`, `confine`, `overrun_segments`, `turn_floor_us`	ask, send and publish costs; overruns, memory exhaustion, crash loops; the confinement test	Chapter 12, Chapter 11
`recovery`	`recovery`	time to recover after the keyboard driver, display driver and compositor are killed (trace `display-restart.jsonl`)	Chapter 15
`utilisation`	`utilisation`, `snake_tick_ms`	share of the lane per world in four scenarios	Chapter 4

Experiment	Key in results.json	Measures	Chapter
`host`	`host`	the simulator's speed and the determinism of captures across builds	Chapter 18
`code_size`	`code`	lines of code by part; object sizes on the 32-bit build	Chapter 8
`extra_traces`		the traces `repl-loop.jsonl` and `sandbox-overrun.jsonl`	Chapter 5

B.8 Reproducing a number

Every measured number in the PicoCalc OS paper, and every number this book takes from it, is computed from `bench/results.json` by `paper/data.py` and stored under a name. To see where a number comes from, print it and read the line of `data.py` that computes it:

```
$ python3 -c "import sys; sys.path.insert(0, 'paper'); import data;
print(data.D['lat_launch_p50'], data.D['rec_disp_done'])"
30.9 515
```

After rerunning the benchmark, rebuild the paper with `python3 paper/build.py --fonts DIR` (the directory of the font packages for the PDF; `--no-pdf` skips the PDF) and this book with `python3 book/build.py`. The book's build also checks that every excerpt of PicoCalc OS code it quotes still matches the source.

APPENDIX C

Glossary

The first section defines the vocabulary of PicoCalc OS, which the book uses consistently. The second defines the general terms of operating systems, real-time systems, security and measurement that the chapters introduce. Each entry names the chapter where the idea is developed.

C.1 Terms of PicoCalc OS

ask. A request to another world that returns a promise of the reply. The request is delivered when the asking segment ends; the promise rejects if the other world does not reply within the timeout (2,000 ms by default), is stopped, or crashes while handling it. Chapter 12.

band. A strip of 16 rows of the framebuffer. When the compositor commits a frame, the nucleus compares the back and front buffers band by band and returns one changed rectangle per band, merged with the rectangle above when that costs little. Chapter 13.

board. The layer under the nucleus that talks to hardware. In the host build it is a set of register-level models of the ST7365P and the STM32; on the device it will be ESP-IDF's I2C and SPI drivers (planned). Chapter 18.

budget. The most virtual time one segment of a world may take, counting the entry cost, interpreter steps and native work. Set per world in the manifest (50 ms by default); starting a world has its own budget of 600 ms. A segment that exceeds it is stopped and the world crashes. Chapter 11.

c_0. The fixed virtual cost of running one segment, entering a world and leaving it: 40 µs. Chapter 18.

channel. A named stream of messages. A world's manifest entry lists the channels it posts on (`provides`) and those it receives (`needs`), and its own file may declare no others (before the first pass, the file's declarations alone decided); `publish` delivers a payload to every world that needs the channel. The kernel itself posts on `frames` and `worlds.status` . Chapter 12.

claim. An entry in the manifest naming hardware or a system privilege a world may obtain: an I2C address (`i2c0:0x1f`), the SPI bus (`spi2`), DMA memory (`dma`), or the system tables (`sys.read` , `sys.admin`). A world cannot give itself claims. Chapter 9.

complete update. In the latency measurements, the end of the last chunk of changed pixels in the update a key press caused, where an update is taken to end when the panel has received no changed pixels for 15 ms. Chapter 19.

compositor. The service that owns the framebuffer. It lays out the focused app's frame with the status and hint bars, draws it into the back buffer, has the nucleus compute the changed rectangles, and asks the display driver to send them. It also routes keys to the focused app. Chapter 13.

damage. The parts of the screen that changed since the last update, expressed as rectangles. Sending only the damage keeps SPI transfers small. Chapter 13.

driver, service, app, sandbox. The four kinds of world, with lane priorities 3, 2, 1 and 0. A driver talks to hardware; a service provides something to other worlds (the compositor); an app has a screen; the sandbox evaluates code typed at the REPL. Chapter 4.

effect. An action a segment asks of the nucleus that is visible outside the world: a message, a bus transfer, a timer, a revocation. Effects are applied in order when the segment ends and discarded if it crashes. Chapter 3.

epoch. A world's restart counter. Settles, timers and restarts are tagged with the epoch they were issued in, and those from an older epoch are discarded, so nothing meant for a crashed instance reaches its successor. Chapter 14.

first pass. The revision of PicoCalc OS, made after this book was written, that took authority and quotas from the manifest: channels, whom a world may send to and ask, an acyclic ask graph, per-world shares of the kernel's tables and mailboxes, and turn deadlines. A separate book, *From L4 to Small Worlds*, describes it; Appendix D lists what it fixed.

first pixels. In the latency measurements, the end of the first chunk of pixel data, after a key press, that changed a visible pixel in the panel controller's memory. Chapter 19.

flush. The compositor's request to the display driver to send a list of rectangles from the framebuffer to the panel. Chapter 13.

frame. The tree of `ui` nodes an app's `screen` function returns. The nucleus publishes it on the `frames` channel when it changes; apps never draw. Chapter 13.

framebuffer. A DMA buffer of 320 × 320 16-bit pixels with a back buffer that is drawn into and a front buffer that holds what the panel was last sent. Obtained with `dma.alloc` and held as a handle. Chapter 13.

handle. A kernel object that grants a world use of a piece of hardware (an I2C device, the SPI bus or a framebuffer), named by a number the world cannot forge. It carries rights, and the nucleus checks the holder and the rights on every use. Chapter 9.

host build. PicoCalc OS compiled for Linux (or for the browser as WebAssembly) against the simulated board. It is deterministic, scriptable and faster than real time. Chapter 18, Appendix B.

lane. The single sequence in which segments run, one at a time. The next segment is chosen by priority, then by arrival order; a running segment is not preempted. Chapter 4.

mailbox. The queue of messages waiting for a world. It is closed for the length of an asynchronous turn, so a world handles one message at a time. Chapter 12.

manifest. The file (`js/system.json`) that lists the worlds the nucleus starts, with each world's source file, claims, budget, heap limit and priority, and, since the first pass, the channels it provides and needs, the worlds it may send to and ask, and its quotas. Appendix A.

narrow. To make a child handle with fewer rights than its parent. The world that narrows a handle can revoke the child; it can pass the child on only if it could pass the parent. Chapter 9.

nucleus. The C core of PicoCalc OS: the lane, the virtual clock, messages and channels, handles, supervision, and the host calls through which worlds reach it. About 2,300 lines. Chapter 1.

#OCT1 frame. The one-line fault report the nucleus writes to the console UART when a world crashes: the prefix `#OCT1` and a JSON object with the world, its file, the message it was handling, the reason, its recent crash count, its heap and the stack. Designed to be read by a program. Chapter 20.

prelude. The JavaScript file every world evaluates before its own. It turns the two host calls into the world API, seals the clock and randomness, and holds the world's definition and state. Chapter 10, Appendix A.

publish. To post a payload on a channel. See **channel**.

quiet root. In the benchmarks, a copy of the JavaScript tree in which apps that redraw on timers have those timers slowed, so that every pixel change measured is caused by the key press under study. Chapter 19.

release. To stop holding a handle. A framebuffer's memory is freed when no handle to it remains.

restart intensity. The rule that limits restarts: a world that crashes for the fifth time within ten seconds is stopped instead of restarted. Before that, restarts wait 100, 200, 400 and 800 ms. Chapter 14.

revoke. To invalidate a handle one has narrowed, and all handles narrowed from it. In PicoCalc OS a revocation is an effect, applied at the end of the segment. Chapter 9.

rights. The operations a handle permits: `r` (read), `w` (write) and `g` (grant, meaning it may be passed to another world). Chapter 9.

segment. One uninterrupted run of a world's code on the lane: the start of a turn, or the continuation of an asynchronous turn after a promise settles. Chapter 3.

send. To deliver a one-way message to a named world. The message is delivered when the sending segment ends.

settle. The resolution or rejection of a promise a world is waiting on (a reply, a finished transfer, an elapsed sleep), delivered by the nucleus as the start of a new segment. Chapter 3.

step. One decrement of QuickJS's interrupt counter, which the interpreter performs at every jump, every conditional branch and every function call. The virtual clock charges τ per step. Chapter 18.

supervisor. The part of the nucleus that handles a crashed world: it discards the crashing segment's effects, fails the ask it was handling, drops its handles, increments its epoch, and restarts it with backoff or stops it. Chapter 14.

τ (tau). The virtual time charged per step: 665 ns, calibrated so that a 100,000-iteration loop costs the 133 ms it took on the ESP32-P4. Chapter 18.

turn. The handling of one message, timer tick or start to completion. A synchronous turn is one segment; an asynchronous turn lasts until its promise settles and may span several segments. Chapter 3.

virtual clock. The clock of the host build: time advances by what the system does (c_0 per segment, τ per step, the cost of native work, and the time bits take on a bus) rather than by the host's real time. Chapter 18.

world. One isolated component of PicoCalc OS: a separate QuickJS runtime with its own heap and heap limit, a budget per segment, a mailbox and an entry in the manifest. Chapter 1.

C.2 General terms

access control list (ACL). A list attached to an object naming which principals may perform which operations on it. The column view of the access matrix. Chapter 9.

access matrix. A table with a row for each principal, a column for each object, and in each cell the rights that principal has on that object. Chapter 9.

address space. The set of memory addresses a program can name. With a memory-management unit each process has its own; without one, as on PicoCalc OS's microcontroller, all code shares one. Chapter 6.

ambient authority. Authority a program has without being given it explicitly, such as a global clock or a file system reachable by name. It makes confinement hard because it cannot be withheld from one component without changing the environment of all. Chapter 9, Chapter 10.

atomicity. The property that an operation takes effect entirely or not at all. In PicoCalc OS, segments are atomic with respect to their effects. Chapter 3.

backoff. Waiting longer before each successive retry or restart, so that a component that keeps failing does not consume the system. Chapter 14.

bus (I2C, SPI). A shared set of wires over which a processor talks to peripheral chips. I2C uses two open-drain lines and addresses each device; SPI uses a clock, data lines and a chip-select line per device and is much faster. Chapter 2.

capability. An unforgeable reference that both names an object and grants the right to use it; holding it is the permission. Chapter 9.

closed loop, open loop. A closed-loop load generator waits for each response before issuing the next request; an open-loop one issues requests on its own schedule. Closed loops under-sample periods when the system is slow. Chapter 19.

confused deputy. A program with authority of its own that is tricked into using it on behalf of a less privileged caller, because the system cannot tell which authority a request is exercising. Chapter 9.

coordinated omission. The bias of a load generator that, while waiting for a slow response, omits the requests that would have arrived in the meantime, so its samples under-represent slow periods. Chapter 19.

crash-only software. Software designed so that crashing and restarting is its only way to stop and its normal way to recover, which keeps the recovery path well exercised. Chapter 14.

cumulative distribution function (CDF). For a set of measurements, the fraction that are at most each value. Every percentile can be read from it. Chapter 19.

deadline. The time by which a real-time task must finish. A hard deadline must never be missed; a soft deadline may be missed occasionally at a cost. Chapter 5.

deterministic simulation. Running a system so that every run with the same inputs produces exactly the same behaviour, by controlling time, randomness and every other source of nondeterminism. Chapter 18.

device driver. The code that translates requests from the rest of a system into the register-level protocol of a particular device, and handles its completions and errors. Chapter 13.

direct memory access (DMA). Hardware that moves data between memory and a peripheral without the processor copying each byte. Chapter 2.

discrete-event simulation. A simulation that keeps a queue of future events ordered by time and jumps from one to the next, rather than advancing time in fixed steps. Chapter 18.

earliest deadline first (EDF). A scheduling policy that always runs the ready task with the nearest deadline; optimal for preemptive scheduling on one processor. Chapter 5.

endpoint. One of the two events whose times a latency measurement subtracts. Chapter 19.

event loop. A single thread that takes events from a queue and runs a handler for each to completion before taking the next. Chapter 3, Chapter 16.

fail-stop. A failure mode in which a component stops and its failure is detectable, rather than continuing with wrong results. Chapter 14.

fault, error, failure. A fault is the cause of a problem (a bug, a bit flip); an error is the incorrect state it produces; a failure is when the incorrect state becomes visible as wrong service. Chapter 14.

garbage collection. Automatic reclamation of memory that a program can no longer reach. QuickJS combines reference counting with a collector for cycles. Chapter 6.

idempotence. The property of an operation that performing it twice has the same effect as performing it once, which makes it safe to replay after a failure. Chapter 15.

interrupt. A hardware signal that makes the processor stop what it is doing and run a handler. Chapter 2.

isolation. Preventing one component from affecting another's state except through agreed channels. It can be enforced by hardware (address spaces, privilege modes) or by a language (type and memory safety). Chapter 8, Chapter 10.

latency, throughput, utilisation. The time from a cause to its effect; the rate at which work completes; the fraction of time a resource is busy. Chapter 19.

mediation. Passing every use of a shared resource through a component that checks it is allowed. Chapter 1.

memory-management unit (MMU). Hardware that translates virtual addresses to physical ones and enforces per-page permissions. A memory protection unit (MPU) or RISC-V's physical memory protection (PMP) enforces permissions on regions without translating. Chapter 2, Chapter 8.

microkernel. A kernel that provides only the minimal mechanisms (address spaces, threads, communication) and runs drivers, file systems and other services as separate processes. Chapter 8.

monolithic kernel. A kernel in which drivers, file systems and other services run in the same privileged address space as the core. Chapter 8.

multiplexing. Sharing one resource among many users, in time (the processor) or in space (memory). Chapter 1.

object capability. A capability that is a reference to an object in a memory-safe language, so that the language's rules about references are the access-control rules. Chapter 9.

percentile. The value below which a given percentage of measurements fall; the median is the 50th. Chapter 19.

phase-locking. A measurement whose schedule shares a period with a periodic process in the system, so that every sample sees that process at the same phase. Chapter 19.

polling. Asking a device repeatedly whether it has something, instead of waiting for it to interrupt. Chapter 2.

preemption. Stopping a running task before it finishes, to run another. PicoCalc OS does not preempt segments; it stops only those that exceed their budget. Chapter 4.

priority inversion. A high-priority task waiting for a low-priority one, typically because the low-priority task holds a resource, while medium-priority tasks run. Chapter 5.

priority inheritance. A protocol that temporarily raises a task holding a resource to the priority of the highest task waiting for it, bounding priority inversion. Chapter 5.

process, thread. A process is a running program with its own address space and resources; a thread is one sequence of execution within it, sharing its memory with the process's other threads. Chapter 3.

rate-monotonic scheduling. Fixed-priority scheduling of periodic tasks in which a shorter period means a higher priority. Chapter 5.

real-time system. A system whose correctness depends on when results are produced as well as on what they are. Chapter 5.

reference monitor. A component that mediates every access by a subject to an object and is tamper-proof and small enough to verify. Chapter 8.

reincarnation server. The MINIX 3 component that monitors drivers and servers running as separate processes and restarts them when they fail. Chapter 15.

response-time analysis. Computing the worst-case time from a task's release to its completion under a scheduling policy, by summing its own cost and the interference from others. Chapter 5.

run to completion. An execution model in which a handler, once started, runs until it returns, without being interleaved with other handlers of the same component. Chapter 3.

scheduler. The part of an operating system that decides which task runs next and for how long. Chapter 4.

shadow driver. A recovery component that observes a driver's interactions with the rest of the kernel and, after the driver is restarted, replays them so that the new instance reaches the old one's state. Chapter 15.

side channel. A path by which information leaks through an effect that is not part of an interface, such as timing or cache state. Chapter 10.

software fault isolation (SFI). Rewriting or checking machine code so that it cannot access memory outside its region, without hardware protection. Chapter 10.

Spectre. A family of attacks that use speculative execution to leak data across a boundary the program's semantics says cannot be crossed, which limits what language-based isolation can protect within one address space. Chapter 10.

supervision tree. In Erlang/OTP, an arrangement of processes in which supervisors start, monitor and restart workers according to a strategy and a limit on restart intensity. Chapter 14.

trusted computing base (TCB). The code whose correctness a system's security or isolation guarantees depend on. Chapter 8.

virtual time. A notion of time that advances according to a model of the work done rather than according to a physical clock. Chapter 18.

watchdog. A timer that resets or signals the system unless software restarts it periodically, catching software that has hung. Chapter 7.

worst-case execution time (WCET). An upper bound on how long a piece of code can take to run on given hardware. Chapter 5.

APPENDIX D

Known issues

This appendix lists what is known to be wrong, missing or approximate in PicoCalc OS as this book describes it. Every item has been reproduced on the host build or read from the code, and the chapter that discusses it is named. Some are deliberate limits of a small design; others are bugs that a later version should fix. Defects that were found while writing and reviewing the book and have been fixed (four in the handle system, five ways a segment could overrun its budget past the stated bound, and an ask table that could lose waiting asks) are not listed here; Chapter 9, Chapter 11, Chapter 12 and Chapter 20 tell those stories.

D.1 Open defects and limits in the system

Input

- **Key loss under load.** The STM32 keeps a 31-entry FIFO, and the keyboard driver (since the second pass, the STM32 driver) reads one event per segment, each separated by I2C transfers. While lower-priority worlds run long segments (the sandbox may run 200 ms segments; since the second pass, on the REPL's budget), the driver drains the FIFO more slowly than a fast typist fills it, and events are lost when it overflows. One review run typed 40 keys at ten per second during a long computation and lost 30 of 80 events. (Chapter 5)
- **At-most-once delivery across a driver crash.** If the keyboard driver (now the STM32 driver) dies between reading an event from the FIFO and publishing it, that event is lost; the window is about 7 ms per event. (Chapter 15)

Recovery

- **A silent device is not a crash.** The keyboard driver catches every bus error, so a keyboard controller that stops answering produces log lines but never a crash, and the supervisor never acts. (Chapter 15)
- **The compositor loses state when it restarts.** Focus returns to the launcher, and the battery indicator stays empty until the battery level next changes, because the power driver (now the power service) publishes only on change. (Chapter 15)
- **The compositor waits for a dead display driver.** During a display-driver restart the compositor sits for about 469 ms in a draw turn waiting for a flush the dead instance will never answer, with its mailbox closed. (Chapter 15)

Messages and memory

- **Framebuffer bytes are not counted against the heap.** Since the first pass a world's framebuffers are bounded by count (its `bufs` quota, checked at boot against the kernel's 16), and its mailbox by messages and bytes (`mbox`, `mbox_kb`); but a framebuffer's pixels (up to 640 × 640, two copies) are still not charged to the world's `mem_kb`. (Chapter 6, Chapter 11)
- **An endpoint's types are names, not contents.** Since the second pass a world sends and asks through typed endpoints; the launcher's endpoint to the compositor carries only `open`, but `open` for any app. (`SECOND-PASS.md`)
- **The second pass's costs.** The passive sandbox serves one ask at a time; the REPL draws a typed line only when its answer comes, so a long evaluation leaves it in the input box for up to 200 ms; keys and battery share one driver's restarts; and the supervisor world is trusted for availability, with the nucleus's own policy after a second and a ceiling of 50 crashes in ten seconds. (`SECOND-PASS.md`)

D.2 Fixed in the second pass

The second pass took the designs of *From L4 to Small Worlds*, Chapter 12. It is recorded in `SECOND-PASS.md`; `bench/secondpass.py` checks each item (17 checks) and `bench/passes.py` measures the numbers on both passes.

- **Authority per world, not per message type.** A world sends or asks only through an endpoint handle that carries the message's type and the right to send or ask; endpoints can be narrowed, passed on and revoked, and a transfer that would let asks form a cycle is refused. Each world numbers its handles from 1 in its own table. (Chapter 9, Chapter 12)
- **Ask starvation and budget laundering.** The sandbox is passive: it answers each ask at the asker's priority and on the asker's budget, and its lane time is billed to the asker. Beside an always-busy app the REPL's answer takes 40.3 ms instead of timing out at 5 s; the Faults app's endless loop stops at its own

50 ms. With the REPL asking before it redraws, its ask takes 0.085 ms instead of the 1.6 ms of Chapter 12. (Chapter 12, Chapter 4)

- **Two drivers on one chip.** One STM32 driver owns the chip at 0x1F and serves keys, battery and backlight; a power service asks it. The nucleus no longer holds the bus through a 2 ms gap, and two claims on one device address do not boot. (Chapter 13)
- **One restart policy for every world.** A supervisor world restarts or stops each crashed world by the policy in its manifest entry; five loops typed at the REPL no longer stop the sandbox. Recovery times moved by less than 0.3 ms. (Chapter 14, Chapter 15)
- **Orphaned events.** A crashed instance's pending sleeps and timers leave the event queue at teardown. (Chapter 14)

D.3 Fixed in the first pass

The first pass made the manifest the authority for channels, messages and the kernel's shared tables, after the seL4 Microkit's static system description. It is recorded in `FIRST-PASS.md` in the repository and described in a separate book, *From L4 to Small Worlds* (Chapters 5 to 7, and its Appendix A for the manifest's schema and messages). `bench/rogue.py` runs the attacks behind these items; each succeeds against the nucleus from before the first pass (`bench/rogue-before.txt`) and is refused now (`bench/rogue-after.txt`). `bench/firstpass.py` checks the rest.

- **Handle-table exhaustion.** Every handle a world makes, narrowed copies included, counts against its `handles` quota until it is freed, and every framebuffer against its `bufs` quota; the boot fails unless the quotas fit the kernel's tables (128 handles, 1,024 asks, 16 framebuffers), so no world can exhaust a table for the others. (Chapter 9)
- **Self-declared channels.** The manifest's `provides` and `needs` decide who may publish and who hears a channel. A world whose own file declares more is stopped at start with the reason, a channel has at most one provider, and `frames` and `worlds.status` belong to the kernel. (Chapter 9)
- **Unauthenticated messages.** A world may send only to the worlds (or groups) its manifest entry lists under `sends` , and ask only those under `asks` ; and a message whose type is a channel the receiver hears is refused, so a forged `frames` message is refused even from a world allowed to talk to the compositor. Only the compositor may ask the display driver to flush. (Chapter 9, Chapter 12)
- **Ask cycles.** The boot fails if the manifest's `asks` form a cycle, printing it (the deadlock Chapter 3 describes can no longer be configured). Asks to a world of lower priority are allowed and noted in the log. (Chapter 3)
- **No hang detection between segments.** A turn still unfinished `turn_ms` after it started (8,000 ms unless the manifest says otherwise) is a crash, and the supervisor restarts the world. (Chapter 15)

- **Timed-out asks are still delivered.** When an ask times out before the callee has started it, the message is taken out of the callee's mailbox. (Chapter 12)
- **Unbounded mailboxes.** A world's mailbox holds at most `mbox` messages and `mbox_kb` of payload, counted from the moment a message is sent. A send or ask to a full mailbox fails at the sender; a channel post to a full subscriber is dropped for that subscriber and counted (`dropped` in the host's view of the worlds). (Chapter 11, Chapter 12)

D.4 Where the timing model is approximate

The virtual clock is calibrated with one parameter against one measurement on the ESP32-P4, and Chapter 18 explains what it leaves out. The approximations a reader is most likely to meet are these.

- **Small native work is rounded down.** Each fill, line or glyph is charged in whole microseconds, rounded down, so glyphs of the small fonts and fills under 400 pixels cost only the 1 µs charged for every host call. With exact accounting the compositor's launcher redraw would cost about 687 µs rather than the 572 µs the traces show. (Appendix A)
- **Allocation and garbage collection are free.** Neither is charged to the virtual clock. (Chapter 6)
- **Start-up is cheap.** Creating a runtime and evaluating a world's file is charged about 66 µs; the device measurement this book's calibration comes from suggests milliseconds. (Chapter 6, Chapter 7)
- **Bus overheads are estimates.** An SPI command is charged 2 µs and a DMA chunk 12 µs; ESP-IDF's SPI master documents per-transaction overheads of tens of microseconds. (Chapter 2)
- **The display bus runs faster than its datasheet allows.** The ST7365P datasheet's write cycle corresponds to about 15 MHz; the driver runs the bus at 40 MHz, which worked on the panel used but is outside the specification. (Chapter 2)
- **Caches, PSRAM latency and interrupts are not modelled.** (Chapter 18)

D.5 Planned, not done

- **The ESP32-P4 port.** PicoCalc OS runs today on the host build and in the browser (WebAssembly), against simulated peripherals. The board layer for ESP-IDF, the nucleus as a FreeRTOS task, the console UART on the device and a measurement of τ on the device are planned. (Chapter 7, Chapter 20)
- **Hardware protection of the nucleus.** The worlds and the nucleus share one address space; isolation rests on QuickJS and the nucleus being correct. Using the ESP32-P4's physical memory protection to fence the nucleus is planned. (Chapter 8, Chapter 10)

D.6 Errata

Before publication, independent reviewers checked every chapter against the code, the benchmark data and the cited sources. Their findings for chapter 1, the overrun bound (chapters 5 and 11), the key-loss limit (chapter 5) and the line counts were applied. The findings below were reported but not yet worked into the text; read each passage with its correction. Minor findings (wording, page counts, small stale numbers) are kept with the repository's review notes in `book/review/`.

- **Chapter 2.** The ST7365P datasheet does not require 120 ms between sleep-out and display-on; it requires 120 ms after sleep-out before a sleep-in command, and only the simulator enforces the wait before display-on (the same claim appears in chapters 13 and 18). The display driver's start-up sends six commands, not eight. ClockworkPi's example keyboard drivers end the write with a STOP and wait 16 ms, not 2 ms with the bus held. The "battery read 0%" half of the interleaving bug appears only in the simulator, whose model clears its answer after a read. In the anatomy Try, raising I2C to 100 kHz cuts the status read from 6.0 to 2.4 ms, and only the model's range moves, not the stage bars. "A key press takes about 31 ms" is a simulated figure.
- **Chapter 4.** The explanation of why shortest-job-first minimises waiting has the roles reversed. In the first Try, moving C's arrival to 0 changes neither mean; make A arrive at 3 instead (FCFS 7.33 → 4.67, round robin 6.67 → 5.33).
- **Chapter 5.** In the rate-monotonic Try, task 3 with $C = 4$ still meets its deadline ($R = 11$); the first miss needs $C = 6$; the harmonic example (1, 2, 6) over (4, 8, 16) has $U = 0.875$, and (1, 2, 8) gives $U = 1$. In exercise 4, $f = 4$ is the only valid frame length. The blocking analysis allows one display and one power segment ahead of the keyboard, but several same-priority segments can run first (five were observed), and it mixes the keyboard's observed 0.08 ms into a budget-only sum. The calibration sentence omits the 10,000-iteration loop, which the model misses by 21%, and two examples given exceed the 83 ms figure they are said to sit below.
- **Chapter 6.** The compositor's heap is now 273 KB (91 KB of its own), not 269 KB (87 KB).
- **Chapter 7.** Creating the runtimes on the device would not hide inside the display's 370 ms of waits: every world boots before any start handler runs, so that cost delays the display's first command and the first frame.
- **Chapter 8.** `InternalError` is a QuickJS extension and three of the listed globals belong to a Stage 3 proposal, not the standard. Run as exercise 4 describes, the exhaustion test stops after 121 narrowings, not 126. Chapter 20 does not list PMP among the port's open problems.
- **Chapter 9.** Levy describes the Plessey System 250 and the Intel iAPX 432 as keeping capabilities in separate segments; the IBM System/38 is his tagged example. Before the fixes, the display driver's confused-deputy harm was larger than stated: six flush requests from a world with no claims stopped the

driver for good. Releasing a handle in the same segment that sends it frees it before delivery, because release acts at once and the send waits for the segment's end.

- **Chapter 10.** In Try 10.1 a second run prints the next random number, because the widget reuses its worker. The widget's clock stands still during a run; PicoCalc's advances with counted steps.
- **Chapter 11.** The variation in measured overshoot comes from where each budget falls between checks, not from the counter not being reset (this affects exercise 3). The allocation preset of Try 11.1 stops at the worker's own 64 MB cap, not at the budget. Lines are charged 10 ns per pixel.
- **Chapter 12.** Since the fix to the ask table, an ask also fails at once with "too many asks are waiting for replies" when all 1,024 slots hold waiting asks; the failure table and "settled exactly once" should say so. Payloads, replies and frames are capped at 64 KiB of JSON. Liedtke's 1993 paper describes L3, the predecessor of L4.
- **Chapter 13.** The panel's state diagram leaves out the datasheet's 120 ms between reset and sleep-out and gives a sleep-out-to-display-on minimum that only the simulator enforces. In the damage Try, the launcher move produces two rectangles, 8.84 ms of SPI in all, because the description text under the grid changes too.
- **Chapter 14.** Exercise 6 should count six commands. Exercise 8's script needs `wait 100` before `trace` to reproduce the crash times. In the supervisor Try, the crash-limit control stops at 8 and the shortest backoff is 25 ms, and in the starting run no waiting ask is answered by the new instance.
- **Chapter 15.** A keyboard controller that stops answering would not get the driver stopped: the driver catches every bus error (see Recovery above). Ryzhyk et al.'s 39% covers driver–OS protocol and concurrency defects, not device protocol. The screen is not complete at 167 ms after a compositor restart: the battery level is lost, and the first draw used the Faults app's frame. The third fault in the self-healing Try puts the device in an error state rather than corrupting the driver. Browsers show an error page for a crashed renderer rather than restarting it (chapter 16). A restart costs 0.2 to 0.6 ms of lane time once the start turn is counted.
- **Chapter 17.** The in-page system (`minios`) differs from the text in four ways: a result that arrives during a world's restart delay can reach the new instance, because its epoch changes at restart rather than at the crash; a segment's effects are posted when `ask`, `device` or `sleep` is called, so a throw does not undo earlier ones; the seal leaves `scheduler.postTask`, `navigator.storage`, `navigator.locks`, `WebTransport` and `WebSocketStream` reachable; and exercise 4's `Promise.all` usually ends with the world stopped at its budget. In the WebAssembly figures, `pc.boot()` starts no worlds (they start in the first `run()`), and its screens match the native 32-bit build on 21 of 22 captures, while the native builds differ among themselves on four.
- **Chapters 18 and 19.** The benchmark suite is now 83 runs. Chapter 18's 1,470× (64-bit, after boot) and chapter 19's 823× (boot included) measure different things. In chapter 19's exercise 8, the closed-loop

run used a 60 ms gap with about 78 ms of jitter on average, so a cycle lasts about 210 ms.

- **Chapter 20.** The account of defects found by checking claims against code predates the ten found while writing and reviewing this book (chapters 9, 11 and 12, and the introduction to this appendix); "one survived until the review" is no longer the whole story.
- **Appendix A.** Each fill, line or glyph charge is rounded down to whole microseconds (see the timing model above). The message "ran out of memory while starting" is never reported; the kernel reports the plain form. The new errors are "a message is at most 65536 bytes of JSON; this one is N", "a reply is at most …", "its screen is N bytes of JSON; a frame is at most 65536" and "too many asks are waiting for replies".

BACK MATTER

References

1. **Accetta et al. 1986.** Mike Accetta, Robert Baron, William Bolosky, David Golub, Richard Rashid, Avadis Tevanian et al.. Mach: a new kernel foundation for UNIX development. *Proceedings of the Summer 1986 USENIX Conference*.

2. **Adya et al. 2002.** Atul Adya, Jon Howell, Marvin Theimer, William J. Bolosky and John R. Douceur. Cooperative task management without manual stack management. *Proceedings of the USENIX Annual Technical Conference*, 289–302. https://www.usenix.org/conference/2002-usenix-annual-technical-conference/cooperative-task-management-without-manual-stack

3. **Anderson 1972.** James P. Anderson. Computer Security Technology Planning Study, Volume I. *Technical report ESD-TR-73-51, Electronic Systems Division, US Air Force*. https://csrc.nist.gov/files/pubs/conference/1998/10/08/proceedings-of-the-21st-nissc-1998/final/docs/early-cs-papers/ande72a.pdf

4. **Apple DriverKit docs 2026.** Apple. DriverKit: develop device drivers that run in user space. *Apple Developer documentation, accessed October 2026*. https://developer.apple.com/documentation/driverkit

5. **Archibald and Kitamura 2021.** Jake Archibald and Eiji Kitamura. SharedArrayBuffer updates in Android Chrome 88 and Desktop Chrome 92. *Chrome for Developers blog*. https://developer.chrome.com/blog/enabling-shared-array-buffer

6. **Armstrong 2003.** Joe Armstrong. Making reliable distributed systems in the presence of software errors. *PhD thesis, Royal Institute of Technology (KTH), Stockholm*. https://erlang.org/download/armstrong_thesis_2003.pdf

7. **Arpaci-Dusseau and Arpaci-Dusseau 2018.** Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau. Operating Systems: Three Easy Pieces (version 1.00). *Arpaci-Dusseau Books*. https://research.cs.wisc.edu/wind/OSTEP/

8. **Audsley et al. 1993.** N. Audsley, A. Burns, M. Richardson, K. Tindell and A. J. Wellings. Applying new scheduling theory to static priority pre-emptive scheduling. *Software Engineering Journal 8(5)*, 284–292. https://doi.org/10.1049/sej.1993.0034

9. **Aviram et al. 2010.** Amittai Aviram, Sen Hu, Bryan Ford and Ramakrishna Gummadi. Determinating timing channels in compute clouds. *Proceedings of the 2010 ACM Cloud Computing Security Workshop (CCSW '10)*, 103–108. https://doi.org/10.1145/1866835.1866854

10. **Avižienis et al. 2004.** Algirdas Avižienis, Jean-Claude Laprie, Brian Randell and Carl Landwehr. Basic concepts and taxonomy of dependable and secure computing. *IEEE Transactions on Dependable and Secure Computing 1(1)*, 11–33. https://doi.org/10.1109/TDSC.2004.2

11. **Back et al. 2000.** Godmar Back, Wilson C. Hsieh and Jay Lepreau. Processes in KaffeOS: isolation, resource management, and sharing in Java. *Proceedings of the 4th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2000)*. https://www.usenix.org/conference/osdi-2000/processes-kaffeos-isolation-resource-management-and-sharing-java

12. **Bacon and Rajan 2001.** David F. Bacon and V. T. Rajan. Concurrent cycle collection in reference counted systems. *ECOOP 2001: Object-Oriented Programming, Lecture Notes in Computer Science 2072, Springer*, 207–235. https://doi.org/10.1007/3-540-45337-7_12

13. **Bacon et al. 2003.** David F. Bacon, Perry Cheng and V. T. Rajan. A real-time garbage collector with low overhead and consistent utilization. *Proceedings of the 30th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '03)*, 285–298. https://doi.org/10.1145/604131.604155

14. **Baker 1991.** T. P. Baker. Stack-based scheduling of realtime processes. *Real-Time Systems 3(1)*, 67–99. https://doi.org/10.1007/BF00365393

15. **Baker and Shaw 1989.** T. P. Baker and Alan Shaw. The cyclic executive model and Ada. *Real-Time Systems 1(1)*, 7–25. https://doi.org/10.1007/BF02341919

16. **Banga et al. 1999.** Gaurav Banga, Peter Druschel and Jeffrey C. Mogul. Resource containers: a new facility for resource management in server systems. *Proceedings of the 3rd USENIX Symposium on Operating Systems Design and Implementation (OSDI '99)*. https://www.usenix.org/legacy/events/osdi99/banga.html

17. **Barry 2016.** Richard Barry. Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide (maintained as the FreeRTOS Kernel Book). *Real Time Engineers Ltd.; current edition in the GitHub repository FreeRTOS/FreeRTOS-Kernel-Book*. https://github.com/FreeRTOS/FreeRTOS-Kernel-Book

18. **Beaufort 2016.** François Beaufort. Access USB devices on the web. *Chrome for Developers (first published 30 March 2016)*. https://developer.chrome.com/docs/capabilities/usb

19. **Beaufort 2025.** François Beaufort. WebGPU is now supported in major browsers. *web.dev blog*. https://web.dev/blog/webgpu-supported-major-browsers

20. **Bellard 2025.** Fabrice Bellard. MicroQuickJS: a JavaScript engine for embedded systems (first public version 22 December 2025). *GitHub repository bellard/mquickjs*. https://github.com/bellard/mquickjs

21. **Bellard 2026.** Fabrice Bellard. QuickJS Javascript Engine documentation, version 2026-06-04. *bellard.org*. https://bellard.org/quickjs/quickjs.pdf

22. **Bershad et al. 1995.** Brian N. Bershad, Stefan Savage, Przemysław Pardyak, Emin Gün Sirer, Marc E. Fiuczynski, David Becker et al.. Extensibility, safety and performance in the SPIN operating system. *Proceedings of the 15th ACM Symposium on Operating Systems Principles (SOSP '95)*, 267–283. https://doi.org/10.1145/224056.224077

23. **Boos et al. 2020.** Kevin Boos, Namitha Liyanage, Ramla Ijaz and Lin Zhong. Theseus: an experiment in operating system structure and state management. *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20)*, 1–19. https://www.usenix.org/conference/osdi20/presentation/boos

24. **Candea and Fox 2003.** George Candea and Armando Fox. Crash-only software. *Proceedings of the 9th Workshop on Hot Topics in Operating Systems (HotOS IX)*, 67–72. https://www.usenix.org/legacy/events/hotos03/tech/full_papers/candea/candea_html/

25. **Candea et al. 2004.** George Candea, Shinichi Kawamoto, Yuichi Fujiki, Greg Friedman and Armando Fox. Microreboot: a technique for cheap recovery. *Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI '04)*, 31–44. https://www.usenix.org/legacy/events/osdi04/tech/candea.html

26. **Cantrill et al. 2004.** Bryan M. Cantrill, Michael W. Shapiro and Adam H. Leventhal. Dynamic instrumentation of production systems. *Proceedings of the 2004 USENIX Annual Technical Conference*, 15–28. https://www.usenix.org/conference/2004-usenix-annual-technical-conference/dynamic-instrumentation-production-systems

27. **cgroup v2 docs 2026.** The Linux kernel developers. Control Group v2. *Linux kernel documentation (Documentation/admin-guide/cgroup-v2.rst), accessed October 2026*. https://www.kernel.org/doc/Documentation/admin-guide/cgroup-v2.rst

28. **Chou et al. 2001.** Andy Chou, Junfeng Yang, Benjamin Chelf, Seth Hallem and Dawson Engler. An empirical study of operating systems errors. *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP '01)*, 73–88. https://doi.org/10.1145/502034.502042

29. **ClockworkPi n.d..** ClockworkPi. PicoCalc: hardware, keyboard firmware (Code/picocalc_keyboard) and examples. *GitHub repository clockworkpi/PicoCalc*. https://github.com/clockworkpi/PicoCalc

30. **CNX Software 2025.** Jean-Luc Aufranc. PicoCalc Kit is a Raspberry Pi Pico handheld terminal with a backlit STM32 QWERTY keyboard. *CNX Software, 14 March 2025*. https://www.cnx-software.com/2025/03/14/picocalc-kit-raspberry-pi-pico-handheld-terminal-backlit-stm32-qwerty-keyboard/

31. **Coffman et al. 1971.** E. G. Coffman, M. Elphick and A. Shoshani. System deadlocks. *ACM Computing Surveys 3(2)*, 67–78. https://doi.org/10.1145/356586.356588

32. **Computer Hope 2025.** Computer Hope. What is Firefox OS?. *Computer Hope dictionary (last updated 7 September 2025)*. https://www.computerhope.com/jargon/f/firefox-os.htm

33. **Corbató et al. 1962.** Fernando J. Corbató, Marjorie Merwin-Daggett and Robert C. Daley. An experimental time-sharing system. *Proceedings of the AFIPS Spring Joint Computer Conference (May 1962)*, 335–344. https://doi.org/10.1145/1460833.1460871

34. **Corbet et al. 2005.** Jonathan Corbet, Alessandro Rubini and Greg Kroah-Hartman. Linux Device Drivers, third edition. *O'Reilly Media*. https://lwn.net/Kernel/LDD3/

35. **Davis et al. 2007.** Robert I. Davis, Alan Burns, Reinder J. Bril and Johan J. Lukkien. Controller Area Network (CAN) schedulability analysis: refuted, revisited and revised. *Real-Time Systems 35(3)*, 239–272. https://doi.org/10.1007/s11241-007-9012-7

36. **Dean and Barroso 2013.** Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM 56(2)*, 74–80. https://doi.org/10.1145/2408776.2408794

37. **Dennis and Van Horn 1966.** Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM 9(3)*, 143–155. https://doi.org/10.1145/365230.365252

38. **Dorward et al. 1997.** Sean M. Dorward, Rob Pike, David Leo Presotto, Dennis M. Ritchie, Howard W. Trickey and Philip Winterbottom. The Inferno operating system. *Bell Labs Technical Journal 2(1)*, 5–18. https://doi.org/10.1002/bltj.2028

39. **Ecma International 2021.** Ecma International. ECMA-419: ECMAScript embedded systems API specification (1st edition June 2021). *Ecma International*. https://ecma-international.org/publications-and-standards/standards/ecma-419/

40. **Elphinstone and Heiser 2013.** Kevin Elphinstone and Gernot Heiser. From L3 to seL4: what have we learnt in 20 years of L4 microkernels?. *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP '13), Farmington, Pennsylvania*, 133–150. https://doi.org/10.1145/2517349.2522720

41. **Endo n.d..** The Endo and Agoric authors. SES: Hardened JavaScript (README of the ses package). *Endo project documentation*. https://github.com/endojs/endo/blob/HEAD/packages/ses/README.md

42. **Engler et al. 1995.** Dawson R. Engler, M. Frans Kaashoek and James O'Toole Jr.. Exokernel: an operating system architecture for application-level resource management. *Proceedings of the 15th ACM Symposium on Operating Systems Principles (SOSP '95)*, 251–266. https://doi.org/10.1145/224056.224076

43. **Erlang/OTP n.d..** Ericsson. supervisor: generic supervisor behavior (STDLIB reference manual). *Erlang/OTP documentation*. https://www.erlang.org/doc/apps/stdlib/supervisor.md

44. **ESP-IDF fatal errors 2026.** Espressif Systems. ESP-IDF Programming Guide v5.4.1, ESP32-P4: Fatal Errors. *Espressif Systems, online documentation, accessed October 2026*. https://docs.espressif.com/projects/esp-idf/en/v5.4.1/esp32p4/api-guides/fatal-errors.html

45. **ESP-IDF FreeRTOS 2026.** Espressif Systems. ESP-IDF Programming Guide v5.3.4, ESP32-P4: FreeRTOS (IDF). *Espressif Systems, online documentation, accessed October 2026*. https://docs.espressif.com/projects/esp-idf/en/v5.3.4/esp32p4/api-reference/system/freertos_idf.html

46. **ESP-IDF heap 2026.** Espressif Systems. ESP-IDF Programming Guide v5.1.2, ESP32: Heap Memory Allocation. *Espressif Systems, online documentation, accessed October 2026*. https://docs.espressif.com/projects/esp-idf/en/v5.1.2/esp32/api-reference/system/mem_alloc.html

47. **ESP-IDF I2C 2026.** Espressif Systems. ESP-IDF Programming Guide v5.2.5, ESP32-P4: Inter-Integrated Circuit (I2C). *Espressif Systems, online documentation, accessed October 2026*. https://docs.espressif.com/projects/esp-idf/en/v5.2.5/esp32p4/api-reference/peripherals/i2c.html

48. **ESP-IDF memory protection n.d..** Espressif Systems. ESP-IDF Programming Guide (ESP32-P4): Migration from 6.1 to 6.2, Security. *Espressif documentation*. https://docs.espressif.com/projects/esp-idf/en/latest/esp32p4/migration-guides/release-6.x/6.2/security.html

49. **ESP-IDF multi-heap 2026.** Espressif Systems. ESP-IDF source: components/heap/multi_heap.c (heaps built on the TLSF allocator in components/heap/tlsf). *GitHub repository espressif/esp-idf, master branch, accessed October 2026*. https://github.com/espressif/esp-idf/blob/master/components/heap/multi_heap.c

50. **ESP-IDF SPI master 2026.** Espressif Systems. ESP-IDF Programming Guide v5.5, ESP32-P4: SPI Master Driver. *Espressif Systems, online documentation, accessed October 2026*. https://docs.espressif.com/projects/esp-idf/en/v5.5/esp32p4/api-reference/peripherals/spi_master.html

51. **ESP-IDF startup 2026.** Espressif Systems. ESP-IDF Programming Guide v5.5.1, ESP32-P4: Application Startup Flow. *Espressif Systems, online documentation, accessed October 2026*. https://docs.espressif.com/projects/esp-idf/en/v5.5.1/esp32p4/api-guides/startup.html

52. **ESP-IDF watchdogs 2026.** Espressif Systems. ESP-IDF Programming Guide, ESP32-P4: Watchdogs. *Espressif Systems, online documentation (stable), accessed October 2026*. https://docs.espressif.com/projects/esp-idf/en/stable/esp32p4/api-reference/system/wdts.html

53. **Espressif 2026a.** Espressif Systems. ESP32-P4 Chip Revision v1.3 Datasheet (ESP32-P4 Series Datasheet v1.2). *Espressif Systems, online documentation, accessed October 2026*. https://documentation.espressif.com/esp32-p4-chip-revision-v1.3_datasheet_en.html

54. **Espressif 2026b.** Espressif Systems. ESP32-P4 Series Datasheet, version 0.7 (preliminary; chip revision v3.x). *Espressif Systems, online documentation, accessed October 2026*. https://documentation.espressif.com/esp32-p4_datasheet_en.html

55. **Espressif ESP32-P4 n.d..** Espressif Systems. ESP32-P4: High-Performing SoC (product page). *espressif.com*. https://www.espressif.com/en/products/socs/esp32-p4

56. **Espruino 2026.** Wikipedia contributors. Espruino. *Wikipedia (revision of 6 July 2026)*. https://en.wikipedia.org/wiki/Espruino

57. **Fagan 1976.** Michael E. Fagan. Design and code inspections to reduce errors in program development. *IBM Systems Journal 15(3)*, 182–211. https://doi.org/10.1147/sj.153.0182

58. **Fuchsia docs 2026.** The Fuchsia authors. Driver framework (DFv2). *fuchsia.dev, accessed October 2026*. https://fuchsia.dev/fuchsia-src/concepts/drivers/driver_framework

59. **Fuchsia driver runner n.d..** The Fuchsia project. Driver runner (component manifest fields colocate and host_restart_on_crash). *fuchsia.dev documentation*. https://fuchsia.dev/fuchsia-src/concepts/components/v2/driver_runner

60. **Ganapathi et al. 2006.** Archana Ganapathi, Viji Ganapathi and David Patterson. Windows XP kernel crash analysis. *Proceedings of the 20th Large Installation System Administration Conference (LISA '06)*, 149–159. https://static.usenix.org/events/lisa06/tech/full_papers/ganapathi/ganapathi_html/index.html

61. **Ganapathy et al. 2008.** Vinod Ganapathy, Matthew J. Renzelmann, Arini Balakrishnan, Michael M. Swift and Somesh Jha. The design and implementation of microdrivers. *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XIII)*, 168–178. https://doi.org/10.1145/1346281.1346303

62. **Goldberg and Robson 1983.** Adele Goldberg and David Robson. Smalltalk-80: The Language and Its Implementation. *Addison-Wesley*.

63. **Golm et al. 2002.** Michael Golm, Meik Felser, Christian Wawersich and Jürgen Kleinöder. The JX operating system. *Proceedings of the 2002 USENIX Annual Technical Conference*. https://www.usenix.org/legacy/event/usenix02/full_papers/golm/golm.pdf

64. **Gray 1985.** Jim Gray. Why do computers stop and what can be done about it?. *Tandem Computers Technical Report 85.7*. https://lasr.cs.ucla.edu/classes/239_1.fall00/papers/gray85.txt

65. **Grier et al. 2008.** Chris Grier, Shuo Tang and Samuel T. King. Secure web browsing with the OP web browser. *2008 IEEE Symposium on Security and Privacy*, 402–416. https://doi.org/10.1109/SP.2008.19

66. **Haas et al. 2017.** Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman et al.. Bringing the web up to speed with WebAssembly. *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017)*, 185–200. https://doi.org/10.1145/3062341.3062363

67. **Hansen 1970.** Per Brinch Hansen. The nucleus of a multiprogramming system. *Communications of the ACM 13(4)*, 238–241, 250. https://doi.org/10.1145/362258.362278

68. **Hardy 1985.** Norman Hardy. KeyKOS architecture. *ACM SIGOPS Operating Systems Review 19(4)*, 8–25. https://doi.org/10.1145/858336.858337

69. **Hardy 1988.** Norm Hardy. The confused deputy (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review 22(4)*, 36–38. https://doi.org/10.1145/54289.871709

70. **Henzinger et al. 2003.** Thomas A. Henzinger, Benjamin Horowitz and Christoph M. Kirsch. Giotto: a time-triggered language for embedded programming. *Proceedings of the IEEE 91(1)*, 84–99. https://doi.org/10.1109/JPROC.2002.805825

71. **Herder et al. 2006.** Jorrit N. Herder, Herbert Bos, Ben Gras, Philip Homburg and Andrew S. Tanenbaum. MINIX 3: a highly reliable, self-repairing operating system. *ACM SIGOPS Operating Systems Review 40(3)*, 80–89. https://doi.org/10.1145/1151374.1151391

72. **Herder et al. 2007.** Jorrit N. Herder, Herbert Bos, Ben Gras, Philip Homburg and Andrew S. Tanenbaum. Failure resilience for device drivers. *Proceedings of the 37th IEEE/IFIP International Conference on Dependable Systems and Networks (DSN '07)*, 41–50. https://doi.org/10.1109/DSN.2007.46

73. **Hewitt et al. 1973.** Carl Hewitt, Peter Bishop and Richard Steiger. A universal modular ACTOR formalism for artificial intelligence. *Proceedings of the Third International Joint Conference on Artificial Intelligence (IJCAI)*, 235–244. https://www.ijcai.org/proceedings/1973/31

74. **Hildebrand 1992.** Dan Hildebrand. An architectural overview of QNX. *Proceedings of the USENIX Workshop on Micro-kernels and Other Kernel Architectures*.

75. **Hoare 1974.** C. A. R. Hoare. Monitors: an operating system structuring concept. *Communications of the ACM 17(10)*, 549–557. https://doi.org/10.1145/355620.361161

76. **Hunt et al. 2007.** Galen Hunt, Mark Aiken, Manuel Fähndrich, Chris Hawblitzel, Orion Hodson, James Larus et al.. Sealing OS processes to improve dependability and safety. *Proceedings of the 2nd ACM SIGOPS/EuroSys European Conference on Computer Systems (EuroSys '07)*, 341–354. https://doi.org/10.1145/1272996.1273032

77. **InformationWeek 2009.** Eric Ogren. CES 2009: Palm shakes its Mojo at developers. *InformationWeek, 10 January 2009*. https://www.informationweek.com/it-leadership/ces-2009-palm-shakes-its-mojo-at-developers

78. **Jain 1991.** Raj Jain. The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling. *Wiley, New York (ISBN 0-471-50336-3)*. https://classes.engineering.wustl.edu/~jain/books/perf_toc.htm

79. **Jones 1997.** Mike Jones. What really happened on Mars Rover Pathfinder (a summary of David Wilner's keynote at the IEEE Real-Time Systems Symposium, December 1997). *Electronic mail, 7 December 1997*. https://www.cs.cornell.edu/courses/cs614/1999sp/papers/pathfinder.html

80. **Joseph and Pandya 1986.** M. Joseph and P. Pandya. Finding response times in a real-time system. *The Computer Journal 29(5)*, 390–395. https://doi.org/10.1093/comjnl/29.5.390

81. **Kadav and Swift 2012.** Asim Kadav and Michael M. Swift. Understanding modern device drivers. *Proceedings of the 17th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XVII)*, 87–98. https://doi.org/10.1145/2150976.2150987

82. **Kitamura and Denicola 2020.** Eiji Kitamura and Domenic Denicola. Why you need "cross-origin isolated" for powerful features. *web.dev*. https://web.dev/articles/why-coop-coep

83. **Klein et al. 2009.** Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin et al.. seL4: formal verification of an OS kernel. *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP '09)*, 207–220. https://doi.org/10.1145/1629575.1629596

84. **Kocher et al. 2019.** Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas et al.. Spectre attacks: exploiting speculative execution. *2019 IEEE Symposium on Security and Privacy*, 1–19. https://doi.org/10.1109/SP.2019.00002

85. **Kopetz 2011.** Hermann Kopetz. Real-Time Systems: Design Principles for Distributed Embedded Applications (second edition). *Springer*. https://doi.org/10.1007/978-1-4419-8237-7

86. **Lamport 1978.** Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM 21(7)*, 558–565. https://doi.org/10.1145/359545.359563

87. **Lampson 1971.** Butler W. Lampson. Protection. *Proceedings of the 5th Princeton Conference on Information Sciences and Systems; reprinted in ACM SIGOPS Operating Systems Review 8(1), 1974*, 18–24 (reprint). https://doi.org/10.1145/775265.775268

88. **Lampson 1973.** Butler W. Lampson. A note on the confinement problem. *Communications of the ACM 16(10)*, 613–615. https://doi.org/10.1145/362375.362389

89. **Lampson 1983.** Butler W. Lampson. Hints for computer system design. *Proceedings of the Ninth ACM Symposium on Operating Systems Principles (SOSP '83)*, 33–48. https://doi.org/10.1145/800217.806614

90. **Lampson and Redell 1980.** Butler W. Lampson and David D. Redell. Experience with processes and monitors in Mesa. *Communications of the ACM 23(2)*, 105–117. https://doi.org/10.1145/358818.358824

91. **Lauer and Needham 1979.** Hugh C. Lauer and Roger M. Needham. On the duality of operating system structures. *ACM SIGOPS Operating Systems Review 13(2) (first presented at the Second International Symposium on Operating Systems, IRIA, 1978)*, 3–19. https://doi.org/10.1145/850657.850658

92. **Lee 2006.** Edward A. Lee. The problem with threads. *IEEE Computer 39(5)*, 33–42. https://doi.org/10.1109/MC.2006.180

93. **LePage 2021.** Pete LePage. New in Chrome 89. *Chrome for Developers blog*. https://developer.chrome.com/blog/new-in-chrome-89/

94. **Leung and Whitehead 1982.** Joseph Y.-T. Leung and Jennifer Whitehead. On the complexity of fixed-priority scheduling of periodic, real-time tasks. *Performance Evaluation 2(4)*, 237–250. https://doi.org/10.1016/0166-5316(82)90024-4

95. **LeVasseur et al. 2004.** Joshua LeVasseur, Volkmar Uhlig, Jan Stoess and Stefan Götz. Unmodified device driver reuse and improved system dependability via virtual machines. *Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI '04)*, 17–30. https://static.usenix.org/event/osdi04/tech/levasseur.html

96. **Levy 1984.** Henry M. Levy. Capability-Based Computer Systems. *Digital Press*. https://homes.cs.washington.edu/~levy/capabook/

97. **Levy et al. 2017.** Amit Levy, Bradford Campbell, Branden Ghena, Daniel B. Giffin, Pat Pannuto, Prabal Dutta et al.. Multiprogramming a 64kB computer safely and efficiently. *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP '17)*, 234–251. https://doi.org/10.1145/3132747.3132786

98. **Liedtke 1993.** Jochen Liedtke. Improving IPC by kernel design. *Proceedings of the 14th ACM Symposium on Operating Systems Principles (SOSP '93)*, 175–188. https://doi.org/10.1145/168619.168633

99. **Liedtke 1995.** Jochen Liedtke. On µ-kernel construction. *Proceedings of the 15th ACM Symposium on Operating Systems Principles (SOSP '95)*, 237–250. https://doi.org/10.1145/224056.224075

100. **Linux driver model docs 2026.** The Linux kernel developers. Driver binding. *Linux kernel documentation (driver-api/driver-model), accessed October 2026*. https://docs.kernel.org/driver-api/driver-model/binding.html

101. **Linux kernel documentation 2023.** The Linux kernel documentation. CFS Scheduler (as of Linux 6.6). *Documentation/scheduler/sched-design-CFS.rst*. https://kernel.org/doc/html/v6.6/scheduler/sched-design-CFS.html

102. **Linux kernel documentation 2025.** The Linux kernel documentation. EEVDF Scheduler. *Documentation/scheduler/sched-eevdf.rst (Linux 6.15)*. https://kernel.org/doc/html/v6.15/_sources/scheduler/sched-eevdf.rst.txt

103. **Linux UIO docs 2026.** The Linux kernel developers. The Userspace I/O HOWTO. *Linux kernel documentation (driver-api), accessed October 2026*. https://docs.kernel.org/driver-api/uio-howto.html

104. **Linux VFIO docs 2026.** The Linux kernel developers. VFIO: Virtual Function I/O. *Linux kernel documentation (driver-api), accessed October 2026*. https://docs.kernel.org/driver-api/vfio.html

105. **Liu and Layland 1973.** C. L. Liu and James W. Layland. Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM 20(1)*, 46–61. https://doi.org/10.1145/321738.321743

106. **Madhavapeddy et al. 2013.** Anil Madhavapeddy, Richard Mortier, Charalampos Rotsos, David Scott, Balraj Singh, Thomas Gazagnaire et al.. Unikernels: library operating systems for the cloud. *Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '13)*, 461–472. https://doi.org/10.1145/2451116.2451167

107. **Malay Mail 2013.** AFP-Relaxnews. First Firefox smartphone launches in Spain. *Malay Mail, 4 July 2013 (AFP-Relaxnews report)*. https://www.malaymail.com/news/tech/gadgets/2013/07/04/first-firefox-smartphone-launches-in-spain/490417

108. **Masmano et al. 2004.** Miguel Masmano, Ismael Ripoll, Alfons Crespo and Jorge Real. TLSF: a new dynamic memory allocator for real-time systems. *Proceedings of the 16th Euromicro Conference on Real-Time Systems (ECRTS 2004)*, 79–88. https://doi.org/10.1109/EMRTS.2004.1311009

109. **Mcilroy et al. 2019.** Ross Mcilroy, Jaroslav Sevcik, Tobias Tebbi, Ben L. Titzer and Toon Verwaest. Spectre is here to stay: an analysis of side-channels and speculative execution. *arXiv:1902.05178*. https://arxiv.org/abs/1902.05178

110. **Microsoft n.d..** Microsoft. Testing and debugging TDR during driver development (TDR registry keys). *Windows driver documentation, Microsoft Learn*. https://learn.microsoft.com/en-us/windows-hardware/drivers/display/tdr-registry-keys

111. **Microsoft UMDF docs 2026.** Microsoft. Overview of UMDF. *Windows driver documentation, Microsoft Learn, accessed October 2026*. https://learn.microsoft.com/windows-hardware/drivers/wdf/overview-of-the-umdf

112. **Miller 2006.** Mark S. Miller. Robust composition: towards a unified approach to access control and concurrency control. *PhD thesis, Johns Hopkins University*. http://www.erights.org/talks/thesis/

113. **Miller et al. 2003.** Mark S. Miller, Ka-Ping Yee and Jonathan Shapiro. Capability myths demolished. *Technical report SRL2003-02, Systems Research Laboratory, Johns Hopkins University*. https://papers.agoric.com/papers/capability-myths-demolished/full-text

114. **Moddable 2026.** Moddable Tech. Moddable: JavaScript for embedded systems (XS engine and Moddable SDK). *moddable.com*. https://www.moddable.com/

115. **Moddable n.d..** Moddable Tech. XS Compartment. *Moddable SDK documentation*. https://www.moddable.com/documentation/xs/XS%20Compartment

116. **Mytkowicz et al. 2009.** Todd Mytkowicz, Amer Diwan, Matthias Hauswirth and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong!. *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '09)*, 265–276. https://doi.org/10.1145/1508244.1508275

117. **Narayanan et al. 2020.** Vikram Narayanan, Tianjiao Huang, David Detweiler, Dan Appel, Zhaofeng Li, Gerd Zellweger et al.. RedLeaf: isolation and communication in a safe operating system. *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20)*, 21–39. https://www.usenix.org/conference/osdi20/presentation/narayanan-vikram

118. **Node.js n.d..** OpenJS Foundation. VM (executing JavaScript), Node.js API documentation. *Node.js documentation*. https://nodejs.org/api/vm.html

119. **NXP 2021.** NXP Semiconductors. UM10204: I2C-bus specification and user manual, Rev. 7.0. *NXP Semiconductors, 1 October 2021*. https://www.nxp.com/docs/en/user-guide/UM10204.pdf

120. **Ousterhout 1996.** John Ousterhout. Why threads are a bad idea (for most purposes). *Invited talk, USENIX Annual Technical Conference*. https://web.stanford.edu/~ouster/cgi-bin/misc.php

121. **Ousterhout 2018.** John Ousterhout. A Philosophy of Software Design. *Yaknyam Press, Palo Alto (ISBN 978-1-7321022-0-0)*.

122. **Palix et al. 2011.** Nicolas Palix, Gaël Thomas, Suman Saha, Christophe Calvès, Julia Lawall and Gilles Muller. Faults in Linux: ten years later. *Proceedings of the 16th International Conference on Architectural Support for*

Programming Languages and Operating Systems (ASPLOS XVI), 305–318. https://doi.org/10.1145/1950365.1950401

123. **Parnas 1972.** David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM 15(12)*, 1053–1058. https://doi.org/10.1145/361598.361623

124. **Pichai and Upson 2009.** Sundar Pichai and Linus Upson. Introducing the Google Chrome OS. *Official Google Blog, 7 July 2009*. https://googleblog.blogspot.com/2009/07/introducing-google-chrome-os.html

125. **QNX n.d..** BlackBerry QNX. Simple restart (High Availability Framework Developer's Guide, QNX SDP 8.0). *QNX documentation*. https://qnx.com/developers/docs/8.0/com.qnx.doc.ham/topic/examples_SIMPLE.html

126. **Redell 1974.** David D. Redell. Naming and protection in extendable operating systems. *Technical report MIT-LCS-TR-140 (MAC-TR-140), Massachusetts Institute of Technology*. https://dspace.mit.edu/handle/1721.1/149438

127. **Redell et al. 1980.** David D. Redell, Yogen K. Dalal, Thomas R. Horsley, Hugh C. Lauer, William C. Lynch, Paul R. McJones et al.. Pilot: an operating system for a personal computer. *Communications of the ACM 23(2)*, 81–92. https://doi.org/10.1145/358818.358822

128. **Reeves 1997.** Glenn E. Reeves. Re: [Fwd: FW: What really happened on Mars?] (account of the Mars Pathfinder resets by the leader of the flight software team). *Electronic mail circulated with Mike Jones's summary, 15 December 1997*. https://www.cs.cornell.edu/courses/cs614/1999sp/papers/pathfinder2.html

129. **Reis and Gribble 2009.** Charles Reis and Steven D. Gribble. Isolating web programs in modern browser architectures. *Proceedings of the 4th ACM European Conference on Computer Systems (EuroSys '09)*, 219–232. https://doi.org/10.1145/1519065.1519090

130. **Reis et al. 2019.** Charles Reis, Alexander Moshchuk and Nasko Oskov. Site Isolation: process separation for web sites within the browser. *28th USENIX Security Symposium*, 1661–1678. https://www.usenix.org/conference/usenixsecurity19/presentation/reis

131. **Ritchie and Thompson 1974.** Dennis M. Ritchie and Ken Thompson. The UNIX time-sharing system. *Communications of the ACM 17(7)*, 365–375. https://doi.org/10.1145/361011.361061

132. **Ryzhyk et al. 2009.** Leonid Ryzhyk, Peter Chubb, Ihor Kuz and Gernot Heiser. Dingo: taming device drivers. *Proceedings of the 4th ACM European Conference on Computer Systems (EuroSys '09)*, 275–288. https://doi.org/10.1145/1519065.1519095

133. **Saltzer and Schroeder 1975.** Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE 63(9)*, 1278–1308. https://doi.org/10.1109/PROC.1975.9939

134. **Saltzer et al. 1984.** Jerome H. Saltzer, David P. Reed and David D. Clark. End-to-end arguments in system design. *ACM Transactions on Computer Systems 2(4)*, 277–288. https://doi.org/10.1145/357401.357402

135. **Schrage 1968.** Linus Schrage. A proof of the optimality of the shortest remaining processing time discipline. *Operations Research 16(3)*, 687–690. https://doi.org/10.1287/opre.16.3.687

136. **Schroeder et al. 2006.** Bianca Schroeder, Adam Wierman and Mor Harchol-Balter. Open versus closed: a cautionary tale. *3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '06)*, 239–252. https://www.usenix.org/conference/nsdi-06/open-versus-closed-cautionary-tale

137. **Sha et al. 1990.** Lui Sha, Ragunathan Rajkumar and John P. Lehoczky. Priority inheritance protocols: an approach to real-time synchronization. *IEEE Transactions on Computers 39(9)*, 1175–1185. https://doi.org/10.1109/12.57058

138. **Shapiro et al. 1999.** Jonathan S. Shapiro, Jonathan M. Smith and David J. Farber. EROS: a fast capability system. *Proceedings of the 17th ACM Symposium on Operating Systems Principles (SOSP '99)*, 170–185. https://doi.org/10.1145/319151.319163

139. **Sitronix 2023.** Sitronix Technology. ST7365P: 320RGB x 480 dot 16.7M color with frame memory, single-chip TFT controller/driver, datasheet V1.0. *Sitronix, February 2023; distributed as ST7365P_SPEC_V1.0.pdf in the PicoCalc repository*. https://github.com/clockworkpi/PicoCalc

140. **Stoica and Abdel-Wahab 1995.** Ion Stoica and Hussein Abdel-Wahab. Earliest eligible virtual deadline first: a flexible and accurate mechanism for proportional share resource allocation. *Technical report TR-95-22, Department of Computer Science, Old Dominion University*. https://people.eecs.berkeley.edu/~istoica/papers/eevdf-tr-95.pdf

141. **Swift et al. 2003.** Michael M. Swift, Brian N. Bershad and Henry M. Levy. Improving the reliability of commodity operating systems. *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP '03)*, 207–222. https://doi.org/10.1145/945445.945466

142. **Swift et al. 2004.** Michael M. Swift, Muthukaruppan Annamalai, Brian N. Bershad and Henry M. Levy. Recovering device drivers. *Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI '04)*, 1–16. https://www.usenix.org/legacy/events/osdi04/tech/swift/swift_html/

143. **systemd n.d..** The systemd project. system.conf: manager configuration defaults (DefaultRestartSec, DefaultStartLimitIntervalSec, DefaultStartLimitBurst). *systemd source tree, src/core/system.conf.in*. https://third-party-mirror.googlesource.com/systemd/+/refs/heads/master/src/core/system.conf.in

144. **Tanenbaum et al. 2010.** Andrew S. Tanenbaum, Raja Appuswamy, Herbert Bos, Lorenzo Cavallaro, Cristiano Giuffrida, Tomáš Hrubý et al.. MINIX 3: status report and current research. *;login: The USENIX Magazine 35(3)*, 7–13. https://download.vusec.net/papers/login-2010.pdf

145. **Tene 2013.** Gil Tene. Coordinated Omission. *Post to the mechanical-sympathy Google group, 3 August 2013*. https://groups.google.com/g/mechanical-sympathy/c/icNZJejUHfE/m/BfDekfBEs_sJ

146. **The Register 2008.** John Oates. Google releases open source browser. *The Register, 2 September 2008*. https://www.theregister.com/2008/09/02/google_browser/

147. **von Behren et al. 2003.** Rob von Behren, Jeremy Condit and Eric Brewer. Why events are a bad idea (for high-concurrency servers). *Proceedings of the 9th Workshop on Hot Topics in Operating Systems (HotOS IX)*, 19–24. https://www.usenix.org/conference/hotos-ix/why-events-are-bad-idea-high-concurrency-servers

148. **Wahbe et al. 1993.** Robert Wahbe, Steven Lucco, Thomas E. Anderson and Susan L. Graham. Efficient software-based fault isolation. *Proceedings of the 14th ACM Symposium on Operating Systems Principles (SOSP '93)*, 203–216. https://doi.org/10.1145/168619.168635

149. **Waldspurger and Weihl 1994.** Carl A. Waldspurger and William E. Weihl. Lottery scheduling: flexible proportional-share resource management. *Proceedings of the First USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 1–11. https://www.usenix.org/conference/osdi-94/lottery-scheduling-flexible-proportional-share-resource-management

150. **Walton 2023.** Philip Walton. Page Lifecycle API. *Chrome for Developers documentation (published 1 December 2023)*. https://developer.chrome.com/docs/web-platform/page-lifecycle-api

151. **Wang et al. 2009.** Helen J. Wang, Chris Grier, Alexander Moshchuk, Samuel T. King, Piali Choudhury and Herman Venter. The multi-principal OS construction of the Gazelle web browser. *18th USENIX Security Symposium*, 417–432. https://www.usenix.org/conference/usenixsecurity09/technical-sessions/presentation/multi-principal-os-construction-gazelle

152. **WASI 2026.** WASI Subgroup, W3C WebAssembly Community Group. WASI 0.1 (wasi_snapshot_preview1). *wasi.dev, accessed October 2026*. https://wasi.dev/releases/wasi-p1

153. **Wasmtime docs 2026.** Bytecode Alliance. Interrupting Wasm execution. *Wasmtime documentation, accessed October 2026*. https://docs.wasmtime.dev/examples-interrupting-wasm.html

154. **Waterman et al. 2021.** Andrew Waterman, Krste Asanović and John Hauser. The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Document Version 20211203. *RISC-V International*. https://github.com/riscv/riscv-isa-manual/releases/download/Priv-v1.12/riscv-privileged-20211203.pdf

155. **Watson et al. 2010.** Robert N. M. Watson, Jonathan Anderson, Ben Laurie and Kris Kennaway. Capsicum: practical capabilities for UNIX. *Proceedings of the 19th USENIX Security Symposium*. https://www.usenix.org/conference/usenixsecurity10/capsicum-practical-capabilities-unix

156. **WHATWG HTML 2026.** WHATWG. HTML Living Standard, section 8.1.7: Event loops. *WHATWG*. https://html.spec.whatwg.org/multipage/webappapis.html#event-loops

157. **WHATWG structured clone 2026.** WHATWG. HTML Living Standard, section 2.7: Safe passing of structured data. *WHATWG, accessed October 2026*. https://html.spec.whatwg.org/multipage/structured-data.html

158. **Wilhelm et al. 2008.** Reinhard Wilhelm, Jakob Engblom, Andreas Ermedahl, Niklas Holsti, Stephan Thesing, David Whalley et al.. The worst-case execution-time problem: overview of methods and survey of tools. *ACM Transactions on Embedded Computing Systems 7(3), article 36*, 36:1–36:53. https://doi.org/10.1145/1347375.1347389

159. **Wilson et al. 1995.** Paul R. Wilson, Mark S. Johnstone, Michael Neely and David Boles. Dynamic storage allocation: a survey and critical review. *Memory Management: International Workshop IWMM 95, Lecture Notes in Computer Science 986, Springer*, 1–116. https://doi.org/10.1007/3-540-60368-9_19

160. **Wirth and Gutknecht 1992.** Niklaus Wirth and Jürg Gutknecht. Project Oberon: The Design of an Operating System and Compiler. *ACM Press and Addison-Wesley*.

161. **Yee et al. 2009.** Bennet Yee, David Sehr, Gregory Dardyk, J. Bradley Chen, Robert Muth, Tavis Ormandy et al.. Native Client: a sandbox for portable, untrusted x86 native code. *2009 IEEE Symposium on Security and Privacy*, 79–93. https://doi.org/10.1109/SP.2009.25

162. **Zephyr docs 2021.** The Zephyr Project. Device driver model. *Zephyr Project documentation, version 2.5.0*. https://docs.zephyrproject.org/2.5.0/reference/drivers/index.html

163. **Zhou et al. 2006.** Feng Zhou, Jeremy Condit, Zachary Anderson, Ilya Bagrak, Rob Ennals, Matthew Harren et al.. SafeDrive: safe and recoverable extensions using language-based techniques. *Proceedings of the 7th USENIX*

Symposium on Operating Systems Design and Implementation (OSDI '06), 45–60. https://static.usenix.org/event/osdi06/tech/zhou.html

164. **Zhou et al. 2021.** Jingyu Zhou, Meng Xu, Alexander Shraer, Bala Namasivayam, Alex Miller, Evan Tschannen et al.. FoundationDB: a distributed unbundled transactional key value store. *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)*, 2653–2666. https://doi.org/10.1145/3448016.3457559

165. **Zircon rights n.d..** The Fuchsia Authors. Rights (Zircon kernel concepts). *Fuchsia documentation*. https://fuchsia.googlesource.com/fuchsia/+/HEAD/docs/concepts/kernel/rights.md